

Communication Protocols

for SoC & ASIC Design

The Complete Guide - From Beginner to Expert

Signalling & SerDes · Line Coding · CRC & ECC · APB · AHB · AXI4 ·
AXI-Stream · ACE & CHI · TileLink · NoC · UART · SPI / QSPI / xSPI · I2C ·
I3C · CAN · LIN · I2S · USB · PCIe & CXL · Ethernet · DDR / LPDDR / HBM ·
MIPI CSI-2 & DSI · HDMI · DisplayPort · JESD204 · UCIe · JTAG · SWD ·
CoreSight

Contents	27 chapters in 5 parts, plus 4 appendices
Scope	Every protocol an SoC talks through - on-chip buses, peripheral buses, high-speed serial links, memory, display and camera, chip-to-chip and die-to-die, debug and trace
Depth	Signals, framing, timing diagrams, state machines, flow control, error handling, performance numbers, controller and PHY architecture, verification and integration
Every example runs	Protocol building blocks were written in Verilog/SystemVerilog and simulated with Icarus Verilog 12 or Verilator 5.020 ; CRCs and encodings were cross-checked against Python. Green cards show the real output
Specifications	Explained precisely but not reproduced; many are members-only. Where a number depends on a spec version it is labelled, and Appendix D lists the source documents
Companions	<i>RTL Design for SoC & ASIC</i> and <i>Verilog & SystemVerilog for SoC & ASIC</i> in this repository

How To Use This Book

A modern SoC is as much a communication system as a computer. Inside it, dozens of blocks exchange data over **on-chip buses** and networks; at its edges, it talks to sensors, flash, memory, displays, cameras, other chips and the debugger over **off-chip protocols**, each with its own wires, timing, framing and rules. A large share of an SoC team's work - and a large share of its bugs - lives at these interfaces: a misread handshake rule, a CRC with the wrong bit order, a clock-domain crossing inside a controller, a burst that crosses a 4 KB boundary, a link that trains in the lab and fails in the field.

This book explains every protocol on that map at the depth an SoC or ASIC engineer needs. For each one it shows the signals and the physical layer, the frame or transaction format, timing diagrams, the state machines, flow control and error handling, the numbers that matter for performance, how the controller and PHY are built in RTL, how they are verified, and which team owns which piece.

The five parts

Part	Chapters	What you get out of it
I - Foundations	1-4	The protocol map of an SoC; signalling, clocking schemes, line coding and SerDes; handshakes, flow control, ordering, CRC and ECC; how protocols are implemented and verified in RTL.
II - On-Chip Protocols	5-10	APB, AHB, AXI4, AXI4-Lite and AXI4-Stream in depth; cache coherency with MESI/MOESI, ACE and CHI; Wishbone, Avalon, TileLink, OCP and networks-on-chip.
III - Peripheral Protocols	11-16	UART and RS-232/485; SPI, Quad/Octal SPI and xSPI; I2C, SMBus and PMBus; MIPI I3C; CAN, CAN FD, LIN, FlexRay and automotive Ethernet; I2S, PDM, S/PDIF, 1-Wire and MDIO.
IV - High-Speed & Memory	17-23	USB; PCI Express and CXL; Ethernet MACs, PHYs and the xMII family; DDR, LPDDR, HBM, DFI and flash interfaces; MIPI camera and display, HDMI and DisplayPort; JESD204, Interlaken and UCIe; JTAG, IEEE 1500/1687, CoreSight and trace.
V - Designing With Protocols	24-27	Controller architecture and PHY integration; verification IP, assertions and compliance; reliability, safety and link security; choosing and budgeting protocols for an SoC, and the learning roadmap.

Reading paths

- **Beginner:** 1-4, then 11, 12, 13 and 5 - the protocols you can build and test on an FPGA board in a weekend each - then 7.
- **RTL designer joining an SoC team:** 3, 5-8, then the peripheral or high-speed chapters your block touches, then 24.
- **Verification engineer:** 3, 4, 7, 25, then the chapter for the protocol you are verifying.
- **Architect or lead:** 1, 2, 9, 10, 17-22, 26 and 27.
- **Interview preparation:** 3, 5, 7, 11-13, 18, 20, then Appendix C.

Conventions

Style	Meaning
A grey card	RTL, a testbench or a script, exactly as it was run
A green card	The output that code produced when run with Icarus Verilog 12, Verilator 5.020 or Python while this book was written
An ASCII figure	A timing diagram, frame or packet layout, state machine or block diagram

Style	Meaning
Coloured boxes	KEY IDEA = the one thing to remember. MATH = a formula or bandwidth calculation. INTUITION = the mental picture. PRACTICAL TIP = what to do. PITFALL = a bug that reaches silicon or the field. EXPERT CORNER = depth you can skip on a first read.
Speeds	Mb/s and Gb/s are bits per second; GT/s is transfers (symbols) per second per lane, before line-coding overhead; MB/s and GB/s are bytes per second

BUILD THE SMALL ONES

UART, SPI, I2C and APB can each be written, simulated and run on a cheap FPGA board with a logic analyser in a few evenings. Nothing teaches protocol rules faster than watching your own controller fail against a real device - and every chapter in Part III gives you a working starting point.

THREE QUESTIONS TO ASK OF ANY PROTOCOL

Who may talk, and when? (arbitration, handshake and flow control). **How does the receiver know where data starts and whether it is correct?** (clocking, framing, coding and error detection). **What happens when something goes wrong?** (errors, retry, timeouts and recovery). Every protocol in this book is a different set of answers to these three questions.

Table of Contents

PART I - Foundations	14
1. Communication in an SoC: a Map of Protocols, Layers and the Roadmap	15
1.1 Why protocols dominate SoC engineering	15
1.2 The protocol map of a modern SoC	16
1.3 Taxonomy: six ways to classify a link	17
1.4 The layered model: PHY, PCS, link, transaction, application	18
1.5 Controller, PHY and IP: who builds what	19
1.6 Trade-offs: bandwidth, latency, pins and power	20
1.7 Anatomy of one transaction: a CPU reads a sensor	22
1.8 Roadmap: from chapter to job role	22
1.9 Summary	23
1.10 Exercises	23
2. Signalling Fundamentals: Single-Ended vs Differential, Clocking, Line Coding, SerDes, Jitter and Eyes	24
2.1 Single-ended signalling and CMOS logic levels	24
2.2 Open-drain outputs and pull-ups	25
2.3 Differential signalling: LVDS, CML and SLVS	26
2.4 Termination and reflections	26
2.5 Clocking schemes: how the receiver knows when to sample	27
2.6 Line coding	28
2.7 Scrambling	34
2.8 SerDes architecture	36
2.9 Jitter, eye diagrams and bit error ratio	37
2.10 Summary	38
2.11 Exercises	38
3. Handshakes, Flow Control, Ordering and Error Detection	39
3.1 The valid/ready handshake	39
3.2 Request/acknowledge: 2-phase and 4-phase handshakes	40
3.3 Flow control	40
3.4 Retry: acknowledgements, windows and go-back-N	42
3.5 Ordering models	42
3.6 Deadlock basics	43
3.7 Error detection: parity and checksums	44
3.8 CRC: the mathematics	44
3.9 ECC: Hamming codes and SECDED	49
3.10 Summary	51
3.11 Exercises	51
4. Implementing and Verifying Protocols in RTL	53
4.1 Generic controller architecture	53

4.2 Bit timing generation: prescalers and fractional baud	54
4.3 Shift registers, serializers and oversampling receivers	54
4.4 Synchronizers and clock-domain crossing	55
4.5 Tri-state and open-drain modelling in RTL	55
4.6 Timeouts and error recovery	56
4.7 Run example: a framed serializer/deserializer with a checker	57
4.8 Protocol checkers with SystemVerilog assertions	60
4.9 Bus functional models, VIP and compliance	62
4.10 Summary	63
4.11 Exercises	63

PART II - On-Chip Protocols **64**

5. AMBA APB: The Advanced Peripheral Bus	65
5.1 Where APB sits and why it exists	65
5.2 History: APB2, APB3, APB4 and APB5	65
5.3 Signal reference	66
5.4 The transfer state machine	67
5.5 Timing diagrams	67
5.6 PSTRB and PPROT in practice	69
5.7 Bridges: how APB is fed	69
5.8 Decoding and multiplexing multiple completers	69
5.9 Verification: the protocol rules as assertions	70
5.10 Hands-on: APB4 slave, decoder and a testbench master	71
5.11 Who owns what, and interview insights	74
5.12 Summary	75
5.13 Exercises	75
6. AMBA AHB and AHB-Lite	76
6.1 Three generations: AHB, AHB-Lite and AHB5	76
6.2 Signal reference (AHB-Lite / AHB5)	76
6.3 The pipeline: address phase and data phase	77
6.4 Transfer types: HTRANS	77
6.5 Bursts, wrapping and the 1KB boundary	78
6.6 Wait states and the two-cycle error response	78
6.7 Multiple masters: arbitration, multi-layer AHB and HMASTLOCK	79
6.8 The AHB-to-APB bridge	80
6.9 Hands-on: AHB-Lite SRAM slave with wait states and ERROR	80
6.10 Verification and integration notes	83
6.11 Summary	84
6.12 Exercises	84
7. AMBA AXI4 in Depth	85
7.1 Five channels	85

7.2 Signal reference	85
7.3 The handshake and its dependency rules	86
7.4 Burst types and address calculation	87
7.5 The 4KB boundary rule	90
7.6 Narrow, unaligned and sparse transfers	90
7.7 IDs, ordering and outstanding transactions	91
7.8 Exclusive access	91
7.9 Memory attributes: AxCACHE and AxPROT	92
7.10 Responses	92
7.11 AXI3, AXI4 and AXI5	92
7.12 Interconnect building blocks	93
7.13 Performance: bandwidth, latency and outstanding transactions	93
7.14 Hands-on: an AXI4 memory slave with FIXED, INCR and WRAP bursts	94
7.15 Verification and ownership	98
7.16 Summary	98
7.17 Exercises	99
8. AXI4-Lite and AXI4-Stream	100
8.1 AXI4-Lite: what is removed	100
8.2 Designing an AXI4-Lite register slave	100
8.3 AXI4-Stream: signals	101
8.4 Packets, framing and backpressure	102
8.5 Data width conversion	102
8.6 Switching and routing streams	103
8.7 Video over AXI4-Stream	103
8.8 DMA: memory-mapped to stream and back	103
8.9 Hands-on: a 32-to-8 bit stream downsizer with TKEEP and TLAST	104
8.10 Summary	106
8.11 Exercises	107
9. Cache Coherency Protocols: MESI/MOESI, ACE and CHI	108
9.1 Why coherence, and what exactly it guarantees	108
9.2 MSI, MESI and MOESI	108
9.3 Snooping versus directories	109
9.4 AMBA ACE: AXI Coherency Extensions	109
9.5 AMBA CHI: the Coherent Hub Interface	111
9.6 Memory consistency models and barriers	112
9.7 Preview: CXL.cache and CXL.mem	113
9.8 Hands-on: a MESI model with invariant checking	113
9.9 Ownership and integration	116
9.10 Summary	116
9.11 Exercises	117
10. Other On-Chip Protocols: Wishbone, Avalon, TileLink, OCP and NoCs	118

10.1 Wishbone B4	118
10.2 Intel Avalon: Avalon-MM and Avalon-ST	120
10.3 TileLink	121
10.4 OCP: the Open Core Protocol	121
10.5 IBM CoreConnect	121
10.6 Networks-on-chip: packets, flits, routing and flow control	122
10.7 Virtual channels and deadlock	123
10.8 Commercial NoCs and AMBA over a NoC	123
10.9 Protocol comparison	123
10.10 Hands-on: a 3x3 mesh of XY wormhole routers	124
10.11 Summary	128
10.12 Exercises	128

PART III - Low-Speed Peripheral Protocols

129

11. UART, RS-232 and RS-485	130
11.1 The asynchronous frame	130
11.2 Baud-rate generation and divisor error	131
11.3 Receiver: 16x oversampling and mid-bit sampling	131
11.4 The clock tolerance budget	132
11.5 Line conditions: errors and break	134
11.6 Hardware flow control: RTS/CTS	135
11.7 The 16550 register model	135
11.8 Electrical layers: TTL, RS-232, RS-422 and RS-485	136
11.9 Modbus RTU: a protocol on top of RS-485	137
11.10 Auto-baud detection	137
11.11 Building and verifying a UART in an SoC	138
11.12 Summary	138
11.13 Exercises	139
12. SPI, Quad/Octal SPI and xSPI	140
12.1 Four wires, two shift registers	140
12.2 CPOL, CPHA and the four modes	141
12.3 Multiple peripherals: chip selects and daisy chains	143
12.4 How fast can SPI go? The round-trip limit	143
12.5 SPI NOR flash: the command set	144
12.6 Dual, Quad and QPI modes	147
12.7 Octal SPI, xSPI (JESD251) and HyperBus	147
12.8 Execute in place (XIP) controllers	148
12.9 Verification and integration notes	148
12.10 Summary	149
12.11 Exercises	149
13. I2C, SMBus and PMBus	150

13.1 Open drain and the wired-AND bus	150
13.2 START, STOP and the bit rules	150
13.3 Addressing, ACK/NACK and transactions	151
13.4 Clock stretching and arbitration	151
13.5 An I2C master in RTL, against an open-drain slave	152
13.6 Speed modes, rise time and pull-up sizing	155
13.7 Timing parameters	156
13.8 Glitch filters and bus recovery	157
13.9 SMBus: I2C with rules	157
13.10 PMBus: power management on SMBus	158
13.11 I2C in an SoC: controllers, slaves and verification	158
13.12 Summary	158
13.13 Exercises	159
14. MIPI I3C	160
14.1 Bus roles and backward compatibility	160
14.2 Open drain where needed, push-pull everywhere else	160
14.3 SDR mode, the T-bit and the 9th bit	161
14.4 Dynamic address assignment (ENTDAA)	161
14.5 Common Command Codes (CCCs)	163
14.6 In-band interrupts and hot-join	164
14.7 HDR modes	164
14.8 Timing and electrical overview	164
14.9 I3C Basic versus full I3C	165
14.10 Building an I3C controller	165
14.11 Summary	165
14.12 Exercises	165
15. Automotive and Industrial Buses: CAN, CAN FD, CAN XL, LIN, FlexRay and Automotive Ethernet	167
15.1 CAN physical layer: dominant and recessive	167
15.2 Frame formats	167
15.3 Bitwise arbitration	168
15.4 Bit stuffing and the CRC-15	169
15.5 Bit timing	172
15.6 Error detection and fault confinement	173
15.7 CAN FD	173
15.8 CAN XL	174
15.9 LIN	174
15.10 FlexRay	175
15.11 Automotive Ethernet	175
15.12 Industrial buses in brief	176
15.13 A CAN controller in an SoC	176
15.14 Summary	177

15.15 Exercises	177
16. Audio and Miscellaneous Links: I2S, TDM, PDM, S/PDIF, 1-Wire, MDIO and SWD	178
16.1 I2S: the inter-IC sound bus	178
16.2 TDM: many channels on one data line	180
16.3 PDM microphones and decimation	181
16.4 S/PDIF and AES3: self-clocked audio between boxes	181
16.5 1-Wire	182
16.6 MDIO / MDC: managing Ethernet PHYs	183
16.7 SWD as a two-wire serial link	184
16.8 GPIO bit-banging	184
16.9 Where these blocks sit in an SoC	184
16.10 Summary	185
16.11 Exercises	185

PART IV - High-Speed and Memory Interfaces 186

17. USB 2.0, USB 3.x and USB4	187
17.1 Versions, speeds and names	187
17.2 Topology: host, hubs, devices and tiers	187
17.3 USB 2.0 electrical layer: D+, D-, J, K and SE0	188
17.4 Packets: PIDs, tokens, data, handshakes and CRCs	191
17.5 Transfer types and bandwidth	193
17.6 Control transfers, enumeration and descriptors	194
17.7 USB 3.x SuperSpeed	195
17.8 USB Type-C and USB Power Delivery	195
17.9 USB4 and Thunderbolt tunnelling	196
17.10 Controller architectures	196
17.11 PHY interfaces: UTMI, UTMI+, ULPI and PIPE	197
17.12 Verification and bring-up	197
17.13 Summary	198
17.14 Exercises	198
18. PCI Express and CXL	199
18.1 Generations, encoding and bandwidth	199
18.2 Topology: root complex, switches, endpoints and BDF	199
18.3 The layered stack	200
18.4 Transaction layer	200
18.5 Data link layer: sequence numbers, LCRC and replay	205
18.6 Physical layer: LTSSM, training sets and equalization	206
18.7 Configuration space	206
18.8 Power management	207
18.9 Errors, AER and advanced features	208
18.10 PIPE: between controller and PHY	208

18.11 Controller design considerations	208
18.12 CXL: Compute Express Link	209
18.13 Summary	210
18.14 Exercises	210
19. Ethernet: MAC, PHY and the xMII Family	211
19.1 IEEE 802.3 and the layer model	211
19.2 The frame	211
19.3 Speeds and PHY types	212
19.4 MAC functions	212
19.5 The xMII family	212
19.6 Auto-negotiation and MDIO management	213
19.7 Flow control: PAUSE and PFC	214
19.8 Precision time: IEEE 1588 PTP	214
19.9 Time-Sensitive Networking (TSN)	214
19.10 NIC offloads	215
19.11 A MAC in RTL: TX framer and RX FCS checker	215
19.12 Where Ethernet sits in an SoC	218
19.13 Summary	218
19.14 Exercises	218
20. Memory Interfaces: SDRAM, DDR4/DDR5, LPDDR, HBM, DFI and Flash	220
20.1 SDRAM basics: banks, rows, columns	220
20.2 The DDR family	224
20.3 Source-synchronous capture: DQS and training	224
20.4 Bank groups and refresh modes	225
20.5 Memory controller architecture	226
20.6 DFI: the controller-PHY interface	226
20.7 Bandwidth and efficiency	227
20.8 HBM: stacks, TSVs and interposers	227
20.9 Flash interfaces	227
20.10 Who owns what	229
20.11 Summary	229
20.12 Exercises	229
21. Display and Camera: MIPI D-PHY/C-PHY, CSI-2, DSI, HDMI and DisplayPort	230
21.1 Where display and camera links sit in an SoC	230
21.2 Video timing: the raster every link carries	230
21.3 MIPI D-PHY: the physical layer	232
21.4 MIPI C-PHY: three wires, three levels, embedded clock	234
21.5 CSI-2: the camera protocol	234
21.6 Bandwidth math: a 4K60 camera over CSI-2	237
21.7 DSI and DSI-2: driving a panel	238
21.8 HDMI: TMDS, FRL and HDCP	239

21.9 DisplayPort and eDP	241
21.10 Integration and verification checklist	242
21.11 Summary	243
21.12 Exercises	243
22. Chip-to-Chip and Die-to-Die: SerDes Links, JESD204B/C, Aurora, Interlaken, UCIe and BoW	244
22.1 The SerDes link, reviewed	244
22.2 A protocol-agnostic PCS	244
22.3 64b/66b in depth: blocks, scrambler and block lock	245
22.4 JESD204B and JESD204C: data converters	249
22.5 Aurora 8B/10B and 64B/66B	250
22.6 Interlaken	251
22.7 Chiplets and die-to-die interfaces	251
22.8 Advanced packaging: 2.5D and 3D	253
22.9 Summary	253
22.10 Exercises	254
23. Debug, Test and Trace: JTAG, IEEE 1149.1, IJTAG, CoreSight, RISC-V Debug and Trace	255
23.1 JTAG: IEEE 1149.1 boundary scan	255
23.2 IEEE 1149.6: AC-coupled and differential nets	259
23.3 IEEE 1500: embedded core test wrappers	260
23.4 IEEE 1687 IJTAG: instrument access networks	260
23.5 Arm CoreSight: the Debug Access Port	260
23.6 Serial Wire Debug (SWD)	261
23.7 The RISC-V debug specification	263
23.8 Trace: recording what happened	264
23.9 Cross-triggering: CTI and CTM	265
23.10 Post-silicon debug and debug security	265
23.11 Summary	265
23.12 Exercises	266

PART V - Designing With Protocols 267

24. Architecture of a Protocol Controller and PHY Integration	268
24.1 The layered controller	268
24.2 The host interface and the register block	269
24.3 DMA engines and descriptors	269
24.4 Buffers, FIFOs and how big they must be	270
24.5 The protocol engine and the PCS	270
24.6 Standard PHY interfaces	271
24.7 Clock domains and CDC points	271
24.8 Reset, power-up and PHY bring-up sequencing	272
24.9 Interrupts	272
24.10 Firmware and driver interaction	273

24.11 IP selection: build or buy, hard or soft	273
24.12 Pad ring and IO cells	273
24.13 Worked example: a descriptor-based DMA-to-stream engine	274
24.14 Integration checklist	278
24.15 Summary	278
24.16 Exercises	278
25. Verifying Protocols: VIP, Assertions, Compliance and Interoperability	280
25.1 Why protocols are hard to verify	280
25.2 Architecture of protocol verification IP	280
25.3 Protocol assertion libraries	281
25.4 Scoreboards for packet protocols	283
25.5 Error injection	283
25.6 Coverage models for protocols	286
25.7 Compliance test suites and certification	287
25.8 Interoperability plugfests	288
25.9 Emulation and FPGA prototyping	288
25.10 Post-silicon validation	288
25.11 Protocol verification sign-off checklist	289
25.12 Summary	289
25.13 Exercises	290
26. Reliability and Security on Links: CRC/ECC, Retry, Safety and Link Encryption	291
26.1 Error models: how links fail	291
26.2 What a CRC can detect	291
26.3 Forward error correction	294
26.4 ECC on memories and links	295
26.5 Link-level retry versus end-to-end protection	297
26.6 Timeouts and error reporting	297
26.7 Functional safety: ISO 26262 on links and buses	298
26.8 Security on links	299
26.9 On-chip access control: TrustZone, AxPROT and firewalls	299
26.10 Secure debug and lifecycle	300
26.11 Fault-injection and physical attacks	300
26.12 Summary	301
26.13 Exercises	301
27. Choosing and Budgeting Protocols for an SoC, and the Learning Roadmap	302
27.1 From requirements to a protocol	302
27.2 The big comparison table	302
27.3 Bandwidth budgeting: a worked example	303
27.4 Latency budgets	305
27.5 Pin and IO budgets	306
27.6 Power, area and cost	307

27.7 A reference edge-AI SoC and its protocol map	307
27.8 The learning roadmap	308
27.9 Roles: who does what with protocols	308
27.10 Interview preparation	309
27.11 Final checklist	309
27.12 Summary	310
27.13 Exercises	310

APPENDICES 311

Appendix A. Protocol Comparison Quick Reference 312

On-chip buses (Chapters 5-10)	312
Low-speed peripheral and field buses (Chapters 11-16)	313
High-speed serial links (Chapters 17-22)	314
Memory interfaces (Chapter 20)	315
CRC polynomials per protocol (verified)	315
Line coding (Chapter 2)	318

Appendix B. Glossary 319

Appendix C. 100 Communication Protocol Interview Questions with Answers 324

Fundamentals and signalling	324
Handshakes, flow control and CRC	325
APB and AHB	325
AXI	326
Cache coherency	327
UART, SPI, I2C and I3C	327
CAN	328
USB	328
PCI Express	329
Ethernet	330
DDR and memory	330
MIPI, display and video	331
Debug and test	331
Design puzzles	331

Appendix D. Specifications, Standards Bodies and Resources 333

Standards bodies and key documents	333
Open-source IP to study	334
Tools	334
Books and courses	335

PART I

Foundations

Every protocol in this book, from a 100 kHz I2C sensor bus to a 64 GT/s PCIe link, is built from the same small set of ideas: how bits become voltages and back, how the receiver knows when to sample, how two sides agree that a transfer happened, how a sender avoids overrunning a receiver, and how corrupted data is detected and repaired. This part builds that toolkit once - with a map of the SoC, the physics of signalling, line codes, scramblers, CRC and ECC, and the RTL and verification patterns every controller reuses - so the protocol chapters that follow can concentrate on what is unique to each standard.

1. Communication in an SoC: a Map of Protocols, Layers and the Roadmap

A system-on-chip is a city of intellectual-property (IP) blocks: CPU clusters, GPUs and NPUs, memory controllers, DMA engines, security islands, dozens of peripherals and a handful of very fast I/O ports. None of those blocks is useful alone. What turns them into a product is communication - on-chip buses carrying loads and stores, low-speed serial buses talking to sensors and power ICs, multi-gigabit links talking to other chips, memory interfaces feeding the processors, and debug ports that let engineers look inside. Each of those conversations follows a **protocol**: a precise contract for signals, timing, framing, ordering and error handling that two independently designed blocks can both implement and still interoperate.

This chapter draws the map. It places every protocol in this book on one SoC block diagram, classifies protocols along the axes that actually drive design decisions, introduces the layered model that lets us reason about a PCIe or USB stack one layer at a time, explains who builds which part of an interface (and who buys it), and closes with typical numbers and a roadmap from chapter to job role.

1.1 Why protocols dominate SoC engineering

On a typical large SoC the majority of the gates in the design are licensed or reused IP, and most of the integration effort is spent on the **interfaces** between blocks: bus fabrics, bridges, clock and reset crossings, PHY integration and the verification of all of it. A protocol is what makes that reuse possible: a USB controller from one vendor connects to a USB PHY from another because both implement UTMI+ or PIPE; an Arm core connects to your accelerator because both speak AXI. When a protocol is misunderstood, the result is not a local bug but a system failure - a hang, silent data corruption, or a device that fails certification.

What a protocol specifies	Example questions it answers
Signals and electrical levels	Which wires exist, their direction, voltage swing, termination, drive strength
Timing	When is data valid? Which clock edge samples it? What are setup/hold, turnaround and minimum pulse widths?
Framing / encoding	How does the receiver find the start of a byte, packet or burst? How are bits encoded on the wire?
Transactions	What is a read, a write, a burst, a message, an interrupt? What response does each produce?
Flow control	How does a slow receiver stop a fast sender without losing data?
Ordering and coherency	Which operations may overtake which? When does a write become visible to other agents?
Errors and recovery	How are corruption, timeouts and protocol violations detected, reported and recovered?
Power and link management	How does the link enter and leave low-power states? Who may wake whom?
Discovery and configuration	How does software learn what is attached and set it up (addresses, enumeration, capability registers)?

Table 1.1 - The dimensions every protocol specification covers.

THE ONE-SENTENCE DEFINITION

A protocol is a contract between two independently designed state machines. Everything in this book - wires, codes, handshakes, packets, credits, CRCs - exists so that both machines always agree on what the other one has done.

1.2 The protocol map of a modern SoC

Figure 1.1 shows a representative application-processor or edge-AI SoC. The exact mix changes by market (a phone SoC has MIPI camera and display, a server chip has many PCIe/CXL lanes and DDR5 channels, a microcontroller may stop at AHB and APB), but the **structure** is universal: a high-performance coherent core, a hierarchy of on-chip buses, peripheral buses behind bridges, high-speed PHYs at the die edge, memory interfaces, and debug.

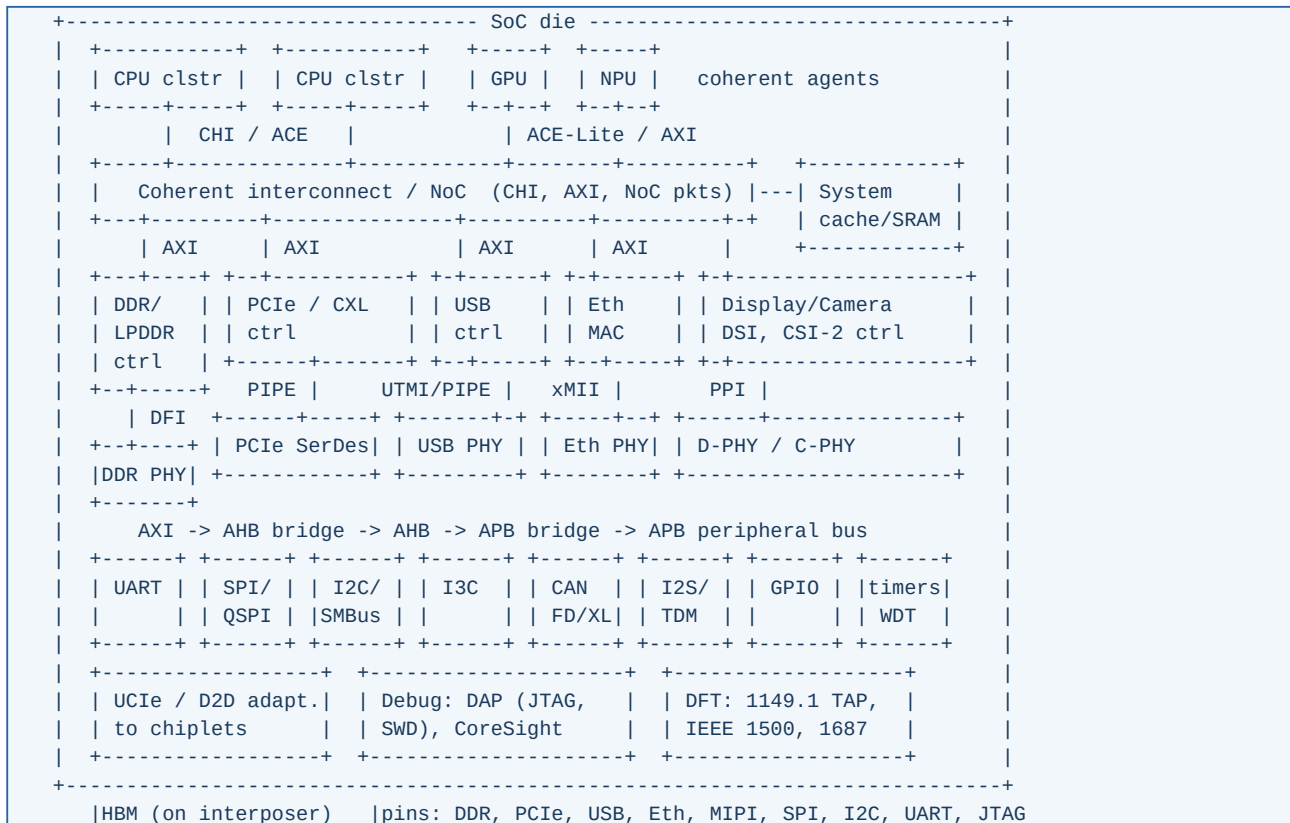


Figure 1.1 - Where every protocol in this book lives on an SoC. CHI/ACE/AXI/AHB/APB inside; SPI, I2C, UART, I3C, CAN and I2S to peripherals; USB, PCIe, Ethernet, MIPI, DDR, HBM and UCIe at high speed; JTAG/SWD for debug and test.

Reading the figure from the processors outward:

- **Coherent core.** CPU clusters and coherent accelerators exchange cache lines with a coherent interconnect using **AMBA CHI** (packet-based, used in large meshes) or **ACE** (an AXI extension). Chapter 9 covers MESI/MOESI, ACE and CHI.
- **System interconnect.** Bulk data moves on **AXI4** (Chapter 7), control registers on **AXI4-Lite** and streams on **AXI4-Stream** (Chapter 8). Large SoCs packetize these into a **network-on-chip** (Chapter 10).
- **Peripheral buses.** Bridges step down to **AHB** (Chapter 6) and **APB** (Chapter 5), where the register blocks of slow peripherals live.
- **Low-speed peripheral protocols** leave the chip on a few pins: **UART** (11), **SPI/QSPI/xSPI** (12), **I2C/SMBus/PMBus** (13), **I3C** (14), **CAN/LIN/FlexRay** (15) and **I2S/TDM/PDM** audio (16).
- **High-speed interfaces** pair a digital **controller** with an analog-heavy **PHY**: **USB** (17), **PCIe/CXL** (18), **Ethernet** (19), **DDR/LPDDR/HBM** and flash (20), **MIPI D-PHY/C-PHY**, **CSI-2**, **DSI**, **HDMI** and **DisplayPort** (21), and **SerDes**, **JESD204** and **UCIe** for chip-to-chip and die-to-die (22).
- **Debug and test:** the **JTAG** TAP (IEEE 1149.1), **IEEE 1500** core wrappers, **IJTAG** (IEEE 1687) and Arm **CoreSight** with **SWD** (23).

Group	Protocols	Typical owner in the SoC team	Ch.
On-chip coherent	CHI, ACE, ACE-Lite	Interconnect/fabric team, CPU subsystem	9

Group	Protocols	Typical owner in the SoC team	Ch.
On-chip non-coherent	AXI4, AXI4-Lite, AXI4-Stream, AHB, APB, TileLink, Wishbone, Avalon, OCP, NoC	Fabric team, every IP designer	5-10
Low-speed peripheral	UART, SPI, QSPI, xSPI, I2C, SMBus, I3C, CAN, LIN, I2S, PDM, 1-Wire, MDIO	Peripheral subsystem RTL, firmware	11-16
High-speed serial	USB 2/3/USB4, PCIe, CXL, Ethernet, MIPI, HDMI, DP	Controller RTL + PHY (analog/mixed-signal) + SW	17-19, 21
Memory	DDR4/5, LPDDR4/5, HBM, DFI, ONFI, eMMC, SD, UFS	Memory subsystem team, PHY vendor, PD/SI	20
Chip-to-chip / D2D	SerDes links, JESD204B/C, Aurora, Interlaken, UCIe, BoW	Architecture, PHY, packaging	22
Debug / test	JTAG, IEEE 1500, IEEE 1687, SWD, CoreSight, trace	DFT team, debug architects	23

Table 1.2 - Protocol families, the team that usually owns them, and the chapter that covers them.

1.3 Taxonomy: six ways to classify a link

Hundreds of protocols exist, but a handful of binary choices explain most of their differences. When you meet a new interface, place it on each axis below and you already know most of its architecture.

On-chip versus off-chip

On-chip wires are short, cheap, numerous and share one silicon process, so on-chip protocols are **wide, parallel and synchronous** (an AXI data bus of 128-512 bits is normal) and rarely need error correction on the wires. Off-chip wires cost package balls, board area and power, see reflections, crosstalk and ground noise, and connect parts from different vendors - so off-chip protocols are **narrow**, carefully specified electrically, and usually protected by CRC or ECC.

Parallel versus serial

A **parallel** bus sends many bits per clock on many wires (AXI, DDR's 64-bit data bus). Its limit is **skew**: all bits must arrive inside the same sampling window. A **serial** link sends bits one after another on one wire or differential pair; high-speed serial links embed the clock in the data and scale by adding **lanes** that are deskewed independently (PCIe x16, USB4 two lanes). Above a few hundred Mb/s per wire off-chip, serial wins because each lane recovers its own timing.

Synchronous versus asynchronous (and how the clock travels)

A **synchronous** interface shares or forwards a clock (SPI's SCLK, I2C's SCL, DDR's CK and DQS). An **asynchronous** interface has no clock wire: UART agrees on a nominal baud rate and resynchronizes on each start bit; PCIe and USB 3 recover the clock from data transitions with a CDR. On chip, 'asynchronous' usually means the two sides run on unrelated clocks and the protocol must tolerate a clock-domain crossing. Chapter 2 treats clocking schemes in depth.

Master/slave versus peer-to-peer

In a **master/slave** protocol (the current AMBA specifications say **Manager/Subordinate**; I2C and I3C now say **controller/target**) only the initiator starts transactions and the responder only answers - APB, AHB, AXI, SPI, I2C. **Peer** protocols let either side initiate: CAN nodes arbitrate for the bus, Ethernet stations send whenever they like, and PCIe endpoints issue their own DMA reads and writes (the root/endpoint distinction is about topology and configuration, not about who may start a transfer). Multi-master buses need **arbitration** - centralized (an AHB arbiter) or distributed (CAN's bitwise arbitration, I2C's wired-AND arbitration).

Memory-mapped versus streaming versus packet

Style	What moves	Addressing	Examples
Memory-mapped	Reads and writes of addressed locations; every request gets a response	Address on every transaction	APB, AHB, AXI4, AXI4-Lite, CHI, register access over I2C/SPI
Streaming	An ordered flow of data words with framing markers (last, keep, user bits), usually no response	None, or a destination ID (TDEST)	AXI4-Stream, I2S, CSI-2 pixel streams inside the chip, JESD204 sample streams
Packet / message	Self-describing packets with a header (type, length, addresses or IDs), payload and CRC; may be routed and retried	Header fields; routing by address or ID	PCIe TLPs, USB packets, Ethernet frames, CAN frames, CHI flits, NoC packets

Table 1.3 - The three transfer styles. Many systems stack them: PCIe carries memory-mapped reads and writes inside packets.

Putting the axes together

Protocol	On/off	Width	Clocking	Initiators	Style
APB	on	parallel 8-32 b	sync	1 manager	mem-mapped
AXI4	on	parallel 32-1024 b	sync	many (via fabric)	mem-mapped
AXI4-Stream	on	parallel	sync	producer	streaming
CHI	on	parallel flits	sync	many	packet
UART	off	serial 1 wire/dir	async (start bit)	peer	byte stream
SPI	off	serial, 1/2/4/8 data	sync (SCLK)	1 host	mem-mapped/cmd
I2C	off	serial, 2 wires	sync (SCL), open-drain	multi-controller	mem-mapped/cmd
CAN	off	serial, 1 diff pair	async + bit sync	peer	message
USB 2.0	off	serial, 1 diff pair	embedded (NRZI)	host only	packet
PCIe	off	serial lanes x1-x16	embedded (CDR)	peer (RC/EP)	packet
DDR5	off	parallel 2 x 32 b (+ECC)	source-sync (DQS)	1 controller	command/burst
UCle	off (in-package)	parallel lanes	forwarded clock	peer	flit/packet
JTAG	off	serial (TDI/TDO)	sync (TCK)	1 debugger	scan shift

Table 1.4 - A dozen protocols on the six axes.

1.4 The layered model: PHY, PCS, link, transaction, application

Complex protocols are specified - and built - in **layers**. Each layer offers a service to the one above and hides the details of the one below. This is the idea behind the seven-layer OSI reference model, but SoC protocols use a flatter, more hardware-oriented stack:

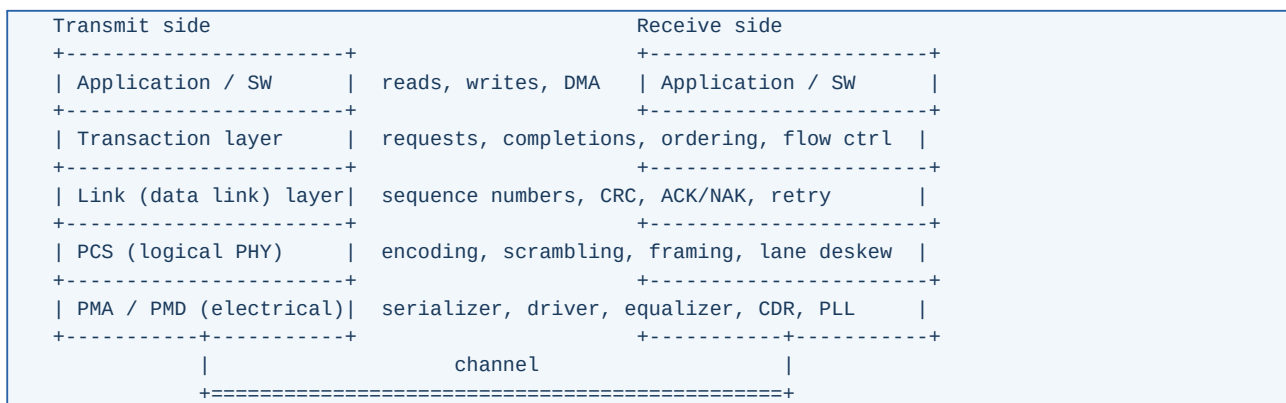


Figure 1.2 - The generic layered stack of a high-speed protocol. Each layer talks logically to its peer; physically, data goes down one side and up the other.

Layer	OSI analogue	Job	PCIe name	Ethernet name	Usually built as
PMD / PMA	1 Physical	Drive and receive the analog signal, serialize, recover clock	Electrical sub-block	PMD, PMA	Analog/mixed-signal hard macro
PCS	1 Physical (upper)	Line coding, scrambling, block/symbol alignment, lane deskew, clock compensation	Logical sub-block	PCS (+ FEC)	Synthesizable RTL, often in the PHY IP
Link	2 Data link	Framing, CRC, sequence numbers, ACK/NAK retry, link-level flow control	Data Link Layer (DLLP)	MAC	Controller RTL
Transaction	3-4 Network/Transport (loosely)	Requests and completions, tags, ordering, credits, routing	Transaction Layer (TLP)	(above MAC: IP/TCP in software)	Controller RTL
Application	5-7	The user's reads, writes, DMA, messages	Software, drivers	Software stack	SoC fabric bridge, firmware, driver

Table 1.5 - Layers mapped to OSI and to two real stacks. OSI mapping is approximate; hardware standards define their own layers.

WHY LAYERING MATTERS TO AN RTL ENGINEER

Layers are also **team and IP boundaries**. The PHY vendor owns PMA/PMD (and often the PCS); your controller starts at a standard interface such as PIPE. The link layer can be verified with a transaction-level PHY model, the PHY with bit-level stimulus. When a bug appears, the first question is always: which layer dropped the ball?

Simple protocols collapse the stack. UART has a physical layer (voltage levels, or RS-232/RS-485 transceivers) and a link layer (start bit, data bits, optional parity, stop bit) and nothing else - any message structure is left to software. I2C adds addressing and an ACK bit per byte. SPI does not even define a word length. The higher the speed and the longer the distance, the more layers you need.

1.5 Controller, PHY and IP: who builds what

Almost every off-chip interface on an SoC is split into a **controller** (digital logic that implements the protocol layers) and a **PHY** (the circuits that touch the pins). Between them sits a standardized, or at least documented, **PHY interface**. Knowing these interfaces is what lets you mix vendors and reason about integration.

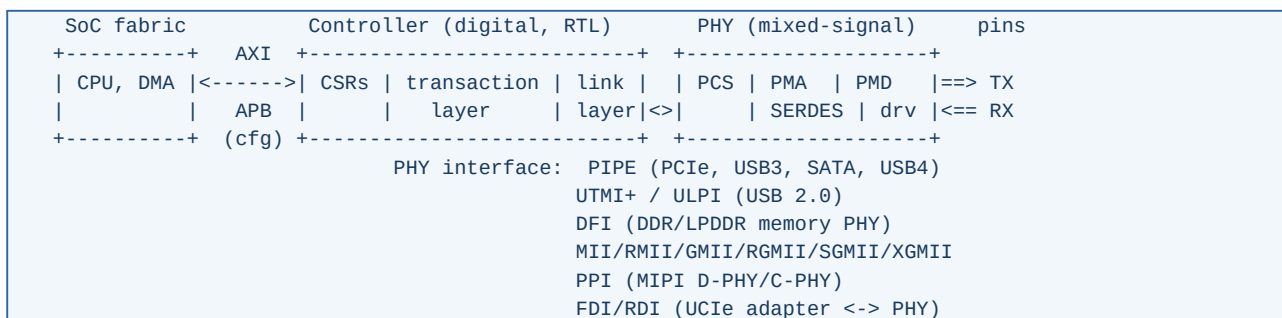


Figure 1.3 - The controller/PHY split and the standard interface at the boundary for each protocol family.

Deliverable	Controller IP	PHY IP	SoC integrator
Design	Synthesizable RTL (Verilog/SV), sometimes encrypted	Hard macro: GDS, LEF, timing libs (.lib), IBIS/AMI models; plus RTL for the PCS/digital wrapper	Glue logic, bridges, clock/reset, pad ring, interrupts
Verification	Testbench, VIP hooks, coverage, compliance test results	SPICE/AMS verification, silicon test chips, characterization reports	SoC-level tests with VIP and real firmware; emulation
Physical	Constraints (SDC), floorplan guidance	Placement rules, bump map, power-grid and ESD guidance, package/board guidelines	Floorplan, PD, SI/PI analysis of package and board
Software	Driver reference code, register descriptions	PHY firmware (training, calibration, adaptation) and settings	Boot firmware, OS drivers, bring-up scripts
Certification	Compliance-tested configurations	Electrical compliance reports	Product certification (USB-IF, PCI-SIG workshops, MIPI conformance, HDMI ATC)

Table 1.6 - Who delivers what for a typical high-speed interface.

- **Build vs buy.** Low-speed peripherals (UART, SPI, I2C, timers) are commonly built in-house or taken from a free/open library; APB/AHB/AXI fabric is generated by tools or licensed; high-speed controllers and almost all multi-gigabit PHYs are licensed, because PHY design needs analog expertise, silicon proof in the target process and a compliance history.
- **Analog vs digital ownership.** PLLs, drivers, receivers, equalizers and CDRs are designed by analog/mixed-signal engineers in schematic; RTL engineers own the controller and PCS; firmware engineers own link training sequences that run on a microcontroller inside many modern PHYs (DDR training is the classic example).
- **Configuration explosion.** Licensed IP is highly parameterized (lane count, data-path width, buffer sizes, optional features). Every parameter choice is a verification and compliance risk - choose the configuration early and freeze it.

PITFALL - THE 'IT IS JUST INTEGRATION' TRAP

Teams often schedule a licensed PCIe or DDR subsystem as a quick integration task. In practice the PHY's clocking, reset sequencing, power domains, test modes (scan, BIST, loopback), firmware loading and package/board signal integrity dominate the schedule. Read the PHY integration guide before the floorplan is frozen, not after.

1.6 Trade-offs: bandwidth, latency, pins and power

Protocol choice is an engineering trade. Table 1.7 gives typical orders of magnitude for the interfaces in this book; later chapters give the exact rates of each generation. Numbers are per link or per lane as stated and are typical, not limits.

Interface	Raw rate (typical)	Pins / wires	Latency scale	Energy scale
APB / AHB / AXI (on-chip)	32-1024 b x 0.2-2 GHz; e.g. 128 b at 1 GHz = 16 GB/s per direction	hundreds of on-chip wires	1-10s of cycles	lowest (short wires)
UART	9600 b/s to a few Mb/s (115200 common)	2 (TX, RX) + optional RTS/CTS	~10 bit times per byte	negligible
SPI / QSPI / Octal xSPI	tens of MHz to ~200 MHz SCLK; octal DDR at 200 MHz = 400 MB/s	4 to 11	few clocks	low
I2C / I3C	I2C 100 k/400 k/1 M/3.4 Mb/s; I3C SDR 12.5 MHz	2	tens of us per register access	low; pull-ups burn static power when low
CAN / CAN FD / CAN XL	CAN up to 1 Mb/s; FD data phase typically 2-8 Mb/s; XL around 10-20 Mb/s	2 (CANH/CANL)	arbitration + frame time	low
USB 2.0 / 3.x / USB4	480 Mb/s; 5/10/20 Gb/s; USB4 20/40 Gb/s, v2.0 up to 80 Gb/s	2 (USB 2) to 4 pairs	us-scale	moderate
PCIe Gen3/4/5/6	8/16/32/64 GT/s per lane, x1-x16	4 wires per lane	~100s of ns per round trip	few pJ/bit (typical SerDes)
Ethernet	10 Mb/s to 800 Gb/s	1-8 lanes	us-scale incl. MAC/PCS/FEC	few pJ/bit
DDR5 / LPDDR5	DDR5 4800-6400+ MT/s x 64 b = 38-51+ GB/s per DIMM channel	~100+ signals per channel	tens of ns	tens of pJ per bit incl. DRAM core (typical)
HBM3	6.4 Gb/s/pin x 1024 = ~819 GB/s per stack	~1024 data bumps + CA	tens of ns	low pJ/bit I/O (short interposer wires)
UCIe	4-32 GT/s per lane (v1.x), 16-64 lanes per module	bumps, not package balls	few ns	well below 1 pJ/bit (advanced package, typical)
JTAG / SWD	TCK/SWCLK up to tens of MHz	4-5 / 2	slow	negligible

Table 1.7 - Orders of magnitude. Always check the exact generation and configuration in the relevant chapter.

Two formulas cover most first-order bandwidth estimates:

$$\text{effective BW} = \text{lanes} \times \text{rate_per_lane} \times \text{coding_efficiency} \times \text{protocol_efficiency}$$

PCIe Gen3 x4 : $4 \times 8 \text{ GT/s} \times 128/130 = 31.5 \text{ Gb/s} = 3.94 \text{ GB/s per direction (before TLP overhead)}$

DDR5-6400 : $6400 \text{ MT/s} \times 64 \text{ bit} / 8 = 51.2 \text{ GB/s peak per DIMM (2 x 32-bit subchannels)}$

HBM3 stack : $6.4 \text{ Gb/s} \times 1024 \text{ bit} / 8 = 819.2 \text{ GB/s peak}$

Eq. 1.1 - Peak and effective bandwidth.

Protocol efficiency is where the surprises live: headers, CRCs, idles, flow-control packets, DRAM refresh, bank conflicts and read/write turnarounds routinely cost 10-40% of the raw rate. A PCIe memory write with a 256-byte payload carries roughly 20-30 bytes of framing, header, sequence number and CRC, so small payloads are dramatically less efficient than large ones (Chapter 18 does the arithmetic).

INTERVIEW INSIGHT: LATENCY VERSUS BANDWIDTH

Bandwidth can be bought with width or lanes; latency cannot. A serial link adds serializer, encoder, CDR, elastic buffer and deskew latency on top of the flight time - tens of nanoseconds even for a very short channel. That is why CPUs still use wide parallel on-chip fabrics and why die-to-die standards such as UCIe use a forwarded clock and simple coding: in-package they can afford many slower wires and want the latency of a wire, not of a network.

1.7 Anatomy of one transaction: a CPU reads a sensor

To see how the layers and protocols cooperate, follow one register read of an I2C temperature sensor from software:

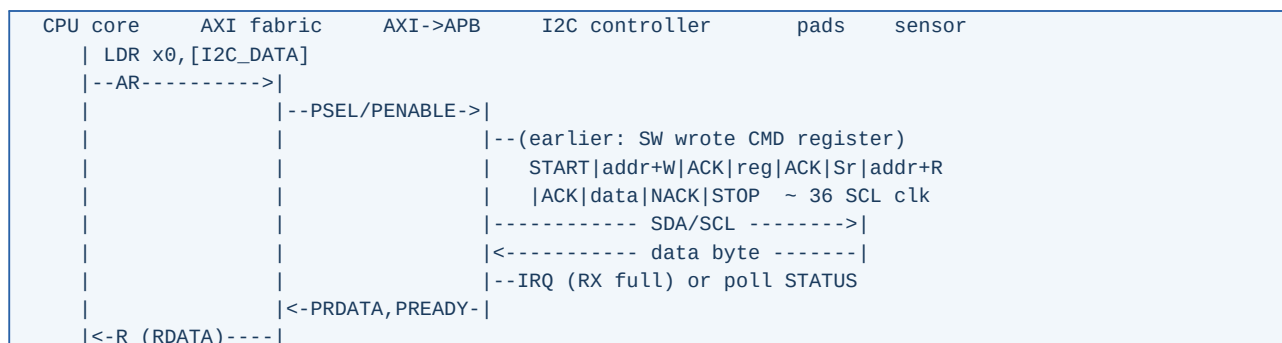


Figure 1.4 - One register read of an I2C sensor crosses a memory-mapped on-chip bus, a bridge, a controller and an off-chip serial protocol.

- Software sees a **memory-mapped** register. The AXI read (Chapter 7) takes a few fabric cycles; the AXI-to-APB bridge (Chapter 5) adds at least two APB cycles.
- The I2C controller turns register writes into a **framed serial transaction** (Chapter 13): START, 7-bit address plus W, register index, repeated START, address plus R, one data byte, NACK, STOP - four 9-bit slots, about 36 SCL cycles plus START/STOP overhead, roughly 90 us at 400 kHz.
- Because 90 us is an eternity for a CPU (about 180 000 cycles at 2 GHz), the controller buffers data in a FIFO and signals completion by interrupt or DMA; software never stalls the bus waiting for the sensor. This **decoupling** of a fast memory-mapped side from a slow serial side is the central architecture pattern of every peripheral controller (Chapter 4 and Chapter 24).
- Errors at each layer are reported differently: a NACK from the sensor becomes a status bit and interrupt; a decode error in the fabric becomes an AXI SLVERR/DECERR response that may raise a CPU exception.

1.8 Roadmap: from chapter to job role

The book is ordered from foundations to integration. Readers with a specific role can follow a shorter path:

Role	Core chapters	Then	Focus
RTL / IP designer	1-8, 11-13	9, 10, 24	FSMs, handshakes, CDC, bus slaves and masters, controller micro-architecture
SoC integration / fabric	1, 3, 5-10	18, 20, 22, 24, 27	Interconnect, bridges, ordering, deadlock, QoS, address maps
Design verification (DV)	1-4, 5-8	25, and each protocol's verification section	Assertions, VIP, scoreboards, coverage, compliance
PHY / analog / SI	2, 17-22	26	Signalling, equalization, jitter, eye and BER, PHY interfaces
Firmware / drivers / bring-up	1, 11-16	17-21, 23	Register models, link training, error handling, debug access
DFT / debug	1, 4, 23	25	JTAG, IEEE 1500/1687, CoreSight, SWD
Architect	1, 3, 9, 18, 20, 22	26, 27	Protocol selection, bandwidth/latency budgets, reliability, security

Table 1.8 - Suggested paths through the book.

Before you move on, you should be able to:

- ☐ Place any protocol on the six axes of Section 1.3.
- ☐ Sketch the SoC map of Figure 1.1 from memory and name the PHY interface of PCIe, USB 2.0, DDR and Ethernet.

- ☐ Explain what the controller, the PHY and the integrator each deliver.
- ☐ Estimate peak bandwidth with Eq. 1.1 and name three sources of protocol inefficiency.

1.9 Summary

- A protocol is a contract covering signals, timing, framing, transactions, flow control, ordering, errors, power and discovery.
- Every SoC has the same structure: coherent core (CHI/ACE), system fabric (AXI), peripheral buses (AHB/APB), low-speed serial peripherals, high-speed controller+PHY pairs, memory interfaces and debug/test.
- Six axes classify links: on/off chip, parallel/serial, clocking, initiator model, and memory-mapped/streaming/packet style.
- High-speed stacks are layered (PMD/PMA, PCS, link, transaction, application) and the layers double as IP and team boundaries.
- Controllers are RTL, PHYs are mixed-signal hard macros joined by standard interfaces (PIPE, UTMI+, DFI, xMII, PPI, FDI/RDI).
- Bandwidth scales with width and lanes; latency and energy per bit decide many architectural choices.

1.10 Exercises

- Classify MIPI I3C, AXI4-Stream, CAN FD and HBM3 on all six axes of Section 1.3. Which classification was hardest, and why?
- Compute the peak per-direction bandwidth of PCIe Gen4 x8 (16 GT/s, 128b/130b) and of a 256-bit AXI bus at 1.2 GHz. How many such AXI ports does it take to keep the PCIe link busy in both directions?
- For the I2C read of Figure 1.4, count SCL cycles precisely (including ACK/NACK bits) and compute the transaction time at 100 kHz, 400 kHz and 1 MHz. How many 2 GHz CPU cycles is each?
- List the deliverables you would request from a PHY vendor before the floorplan freeze, and explain which SoC team consumes each.
- Choose an interface for (a) a 12-bit ADC at 1 MS/s, (b) a 4K60 camera, (c) a board-management power controller, (d) a second die in the same package. Justify each choice using Table 1.7.

2. Signalling Fundamentals: Single-Ended vs Differential, Clocking, Line Coding, SerDes, Jitter and Eyes

Before a protocol can define packets it must get individual bits across a wire. This chapter covers the physical and logical techniques every interface is built from: how a '1' and a '0' are represented electrically, how the receiver knows when to sample, how bit streams are coded so that clocks can be recovered and DC balance kept, how multi-gigabit SerDes transceivers are organized, and how link quality is measured with jitter, eye diagrams and bit error ratio. Three pieces of RTL are built and simulated: an 8b/10b encoder checked exhaustively against a Python reference, a Manchester encoder/decoder pair with clock recovery, and additive and self-synchronous scramblers.

2.1 Single-ended signalling and CMOS logic levels

A **single-ended** signal is one wire referenced to ground. The driver pulls the line towards VDD for '1' and towards ground for '0'; the receiver compares the voltage against input thresholds. Four numbers define a logic family:

Parameter	Meaning
V_{OH} (min)	Lowest voltage a driver guarantees for a '1' at its rated current
V_{OL} (max)	Highest voltage a driver guarantees for a '0'
V_{IH} (min)	Lowest voltage a receiver guarantees to read as '1'
V_{IL} (max)	Highest voltage a receiver guarantees to read as '0'

$$NM_H = V_{OH} - V_{IH}$$

$$NM_L = V_{IL} - V_{OL}$$

Eq. 2.1 - Noise margins: how much noise the wire may add before a receiver misreads a bit.

Family (JEDEC-style)	VDDIO	V_{IH} min	V_{IL} max	Typical use
LVTTL / LVCMOS 3.3 V	3.3 V	2.0 V	0.8 V	Legacy GPIO, UART/SPI/I2C on boards
LVCMOS 2.5 V	2.5 V	1.7 V	0.7 V	Older FPGA banks, RGMII
LVCMOS 1.8 V	1.8 V	0.65 x VDD	0.35 x VDD	Mobile GPIO, SPI flash, SD, eMMC
LVCMOS 1.2 V	1.2 V	0.65 x VDD	0.35 x VDD	Low-voltage sensors, I3C at 1.2 V

Table 2.1 - Common single-ended I/O standards (typical thresholds; always confirm in the device datasheet).

Single-ended signalling is cheap - one pin per bit - but it has three weaknesses that limit speed and distance: it shares a noisy ground reference between chips; many outputs switching together draw large currents through package inductance, creating **simultaneous switching output (SSO)** noise or ground bounce; and the full-swing signal radiates and couples into neighbours. Practical ceilings are in the low hundreds of Mb/s per pin on a board, which is why parallel single-ended buses (DDR, the old PCI) needed ever lower swings, on-die termination and training as speeds rose.

WHAT AN I/O CELL GIVES THE RTL

A general-purpose I/O pad has an output-data input, an output-enable, and an input-data output, plus static controls: drive strength, slew rate, pull-up/pull-down enable, Schmitt-trigger enable and input enable. The RTL controls only the digital triplet (Section 4.5); the analog behaviour is set by the pad library and the pin-mux configuration.

2.2 Open-drain outputs and pull-ups

An **open-drain** (open-collector in bipolar days) output can only pull the line **low**; to send a '1' it releases the line and an external **pull-up resistor** R_p charges the bus capacitance C_b towards VDD. Any device pulling low wins, which gives a **wired-AND**: the line is high only if every device releases it. I2C, SMBus, the I3C open-drain phase, 1-Wire, and interrupt or reset lines shared by several chips all rely on this property - for multi-controller arbitration, clock stretching and ACK bits.

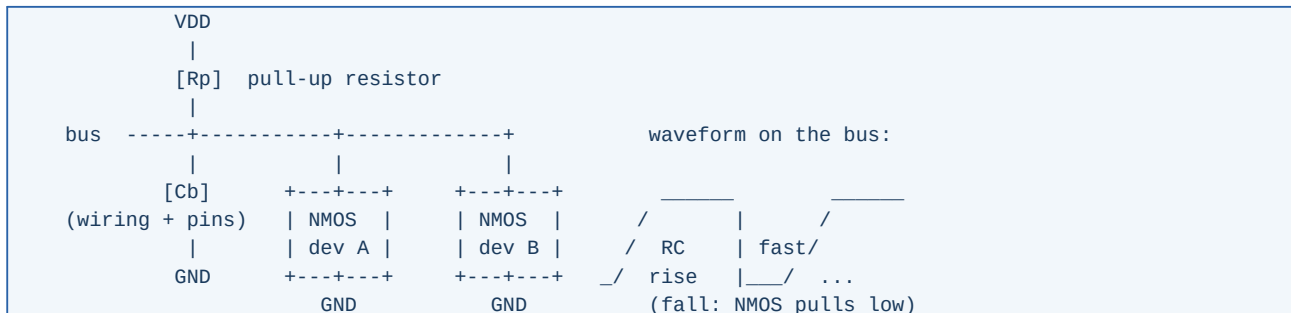


Figure 2.1 - Open-drain bus. Any device can pull low; the resistor and total capacitance set the slow rising edge.

The rising edge is a simple RC charge, so its 30%-70% rise time (the definition used by the I2C specification) is:

$$V(t) = VDD \times (1 - \exp(-t / (R_p \times C_b)))$$

$$t_r(30\% \rightarrow 70\%) = R_p \times C_b \times \ln(0.7/0.3) = 0.8473 \times R_p \times C_b$$

$$R_{p_max} = t_r(max) / (0.8473 \times C_b) \quad R_{p_min} = (VDD - V_{OL(max)}) / I_{OL}$$

Eq. 2.2 - Pull-up sizing: too large and edges are too slow; too small and a driver cannot pull the line below V_{OL} .

```
# Open-drain bus: rise time (30% -> 70% VDD) of an RC pull-up and the pull-up range
import math
k = math.log(0.7 / 0.3)          # t_r = RC * ln(0.7/0.3) = 0.8473 RC
print("t_r(30%->70%) = %.4f x Rp x Cb" % k)
vdd, vol = 3.3, 0.4
for mode, tr, iol in ((("Sm 100 kHz", 1000e-9, 3e-3), ("Fm 400 kHz", 300e-9, 3e-3),
                      ("Fm+ 1 MHz", 120e-9, 20e-3))):
    rmin = (vdd - vol) / iol
    rmax = ["%dpF:%5.0f" % (c * 1e12, tr / (k * c)) for c in (50e-12, 100e-12, 400e-12)]
    print("%s t_r<=%4.0f ns Rp_min=%4.0f Rp_max: %s" % (mode, tr * 1e9, rmin,
                                                         ", ".join(rmax)))
```

Python used to size I2C pull-ups for Standard-mode, Fast-mode and Fast-mode Plus (rise-time limits 1000/300/120 ns, sink current 3 mA or 20 mA at $V_{OL} = 0.4$ V).

```
t_r(30%->70%) = 0.8473 x Rp x Cb
Sm 100 kHz t_r<=1000 ns Rp_min= 967 Rp_max: 50pF:23604, 100pF:11802, 400pF: 2951
Fm 400 kHz t_r<= 300 ns Rp_min= 967 Rp_max: 50pF: 7081, 100pF: 3541, 400pF: 885
Fm+ 1 MHz t_r<= 120 ns Rp_min= 145 Rp_max: 50pF: 2833, 100pF: 1416, 400pF: 354
```

Real output. With 400 pF (the Sm/Fm bus limit), Fast-mode needs a pull-up of at most ~885 ohm yet at least ~967 ohm at 3 mA: no solution - which is why heavily loaded buses use buffers, current-source pull-ups or Fm+ drivers.

PITFALL - OPEN-DRAIN DESIGN AND INTEGRATION BUGS

- Driving '1' actively on an open-drain line (a push-pull GPIO configured by mistake) creates contention with any device pulling low and defeats arbitration and clock stretching.
- Forgetting that the rise time, not the clock frequency, limits speed: a 400 kHz bus with long cables may need 100 kHz.
- Pull-ups to the wrong rail when devices sit in different voltage domains - use a level shifter designed for bidirectional open-drain lines.
- Static power: every time the line is held low, VDD/R_p flows - about 1 mA for 3.3 V and 3.3 kohm.

2.3 Differential signalling: LVDS, CML and SLVS

A **differential** link sends the signal on two wires, P and N, driven in opposite directions; the receiver looks only at the difference $V_P - V_N$. Noise that couples equally onto both wires (**common-mode noise**) cancels, return current flows mostly in the pair itself, and the swing can be small - a few hundred millivolts - which reduces power and emissions and makes gigabit rates practical. Every high-speed serial protocol in this book is differential.

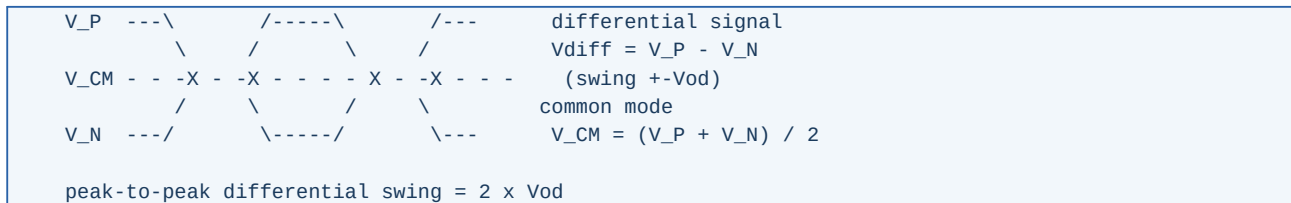


Figure 2.2 - A differential pair and its common-mode level.

Family	Swing (typical)	Common mode	Termination	Where used
LVDS (TIA/EIA-644)	$ V_{OD} \sim 350$ mV (247-454 mV); ~ 3.5 mA current loop	~ 1.2 V	100 ohm differential at RX	Display (FPD-Link), older camera/ADC links, FPGA I/O, clock distribution
CML (current-mode logic)	~ 400 - 1000 mV p-p differential	near VDD (resistive loads to VDD)	50 ohm per leg to VDD (on-die)	Multi-gigabit SerDes: PCIe, USB 3, Ethernet, SATA (inside PHYs)
SLVS (e.g. SLVS-400)	~ 200 mV	~ 200 mV	100 ohm differential	MIPI D-PHY high-speed mode; low-power camera and display links
Sub-LVDS / scalable LVDS	~ 150 mV	~ 0.9 V	100 ohm	Image sensors, low-power FPGA links

Table 2.2 - Differential signalling families (values typical; standards define min/max ranges).

High-speed SerDes transmitters are almost always **CML-style current steering** drivers or, increasingly at low supply voltages, voltage-mode (SST) drivers; both present a matched 50 ohm source per leg. AC coupling capacitors in series with each wire (required by PCIe and USB 3) block DC, so the two chips can use different common-mode levels - and so the data stream must itself be **DC-balanced**, which is the job of line coding and scrambling.

2.4 Termination and reflections

A board trace is a **transmission line** with characteristic impedance Z_0 (typically 50 ohm single-ended, 85-100 ohm differential). When a signal edge meets an impedance change, part of it reflects:

$$\Gamma = (Z_L - Z_0) / (Z_L + Z_0)$$

open end ($Z_L = \infty$): $\Gamma = +1$ short: $\Gamma = -1$ matched: $\Gamma = 0$

treat as a transmission line when edge time $< \sim 2 \times$ round-trip flight time

Eq. 2.3 - Reflection coefficient. Propagation on FR-4 is roughly 6-7 ps/mm (about 15-17 cm per ns).

Scheme	How	Pros / cons	Examples
Series (source)	R at the driver so $R_{drv} + R_s = Z_0$	No DC power; only for point-to-point, receiver at the end	LVC MOS clocks, SPI SCLK on long traces
Parallel (end)	$R = Z_0$ to ground or VTT at the receiver	Clean edges; burns DC power	Old DDR (SSTL to VTT)
On-die termination (ODT)	Switchable R inside the receiver, enabled only when receiving	No board parts, dynamic; needs calibration (ZQ)	DDR3/4/5, LPDDR, GDDR

Scheme	How	Pros / cons	Examples
Differential 100 ohm	R across P/N at RX (on-die in SerDes)	Matched pair; low power for current-mode drivers	LVDS, PCIe, USB, MIPI HS

Table 2.3 - Termination schemes.

WHAT THE DIGITAL DESIGNER MUST KNOW

You rarely compute reflections yourself - SI engineers do that with IBIS/IBIS-AMI models and channel simulations - but you configure ODT values, drive strengths and slew settings through PHY registers, and you own the firmware that calibrates them (ZQ calibration, impedance trim). Wrong settings produce 'software' bugs that are really signal integrity.

2.5 Clocking schemes: how the receiver knows when to sample

Every receiver must sample each bit inside its **valid window**. The clocking scheme decides where the sampling clock comes from, and it is the single most important property of an interface's timing.

Common-clock (system-synchronous)

Both chips receive the same clock from a board oscillator; data launched on one edge is captured on the next edge at the other chip. Classic PCI and early SDRAM worked this way. The whole round of delays must fit into one period, so frequency tops out quickly:

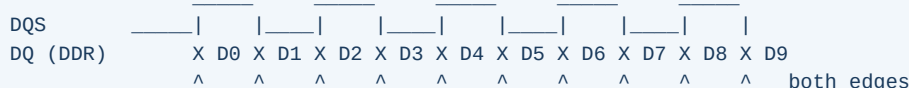
$$T_{\text{clk}} \geq t_{\text{co}} + t_{\text{flight}} + t_{\text{su}} + t_{\text{skew}}(\text{clock}) + t_{\text{jitter}}$$

$$t_{\text{hold}} \leq t_{\text{co}}(\text{min}) + t_{\text{flight}}(\text{min}) - t_{\text{skew}}(\text{clock})$$

Eq. 2.4 - Common-clock timing budget. Flight time and clock skew count against every cycle.

Source-synchronous and double data rate

The transmitter sends a **strobe or clock alongside the data**, routed with matched length, so both experience the same flight delay and it cancels out of the budget. DDR SDRAM's DQS strobe, RGMII's TXC/RXC and the MIPI D-PHY clock lane are examples. Most such interfaces use **DDR** (double data rate): data changes on both edges, doubling throughput for the same clock frequency. The strobe is edge-aligned with data on DDR reads (the controller delays it by ~90 degrees internally) and centre-aligned on DDR writes.



write (centre-aligned): strobe edge in the middle of each data eye
 read (edge-aligned) : RX delays DQS by ~1/4 period to centre it

Figure 2.3 - Source-synchronous DDR: a strobe travels with the data.

Forwarded clock

A forwarded-clock link sends a **clock lane** with a group of data lanes (HDMI TMDS up to HDMI 2.0, MIPI D-PHY, UCIe, HBM). The receiver still needs per-lane deskew (training) at high rates, but it avoids a full CDR per lane - lower power and latency, ideal for short, matched channels such as in-package die-to-die links.

Embedded clock and clock-data recovery (CDR)

Long-reach and high-rate serial links send **no clock at all**. The receiver's CDR locks a local oscillator to the **transitions in the data**, and places the sampling point in the centre of each bit. This removes skew between lanes from the timing budget entirely - each lane tracks its own phase - at the price of requiring enough transitions (coding or scrambling), a lock time, and elastic buffers to absorb the small frequency difference between the two ends' reference clocks (for example up to +-300 ppm in PCIe, which inserts and deletes SKP

ordered sets to compensate).

Scheme	Clock source at RX	Rate limit set by	Examples
Common clock	Shared board clock	Flight time + skew in one period	PCI, early SDRAM, simple SPI at low speed
Source-synchronous	Strobe/clock sent with data	Skew within the group, duty-cycle distortion	DDR DQS, RGMII, SPI flash (DQS in Octal DDR), ONFI
Forwarded clock	Dedicated clock lane	Lane-to-lane skew (trained), jitter tracking	D-PHY, HDMI (TMDS), UCIe, HBM
Embedded clock + CDR	Recovered from data transitions	Channel loss and jitter; CDR bandwidth	PCIe, USB 3, SATA, Ethernet SerDes, DisplayPort, C-PHY
Asynchronous oversampled	Local clock, resync per frame	Clock tolerance x frame length	UART, CAN (with resynchronization)

Table 2.4 - Clocking schemes compared.

2.6 Line coding

A **line code** maps data bits to the symbols actually sent. It exists to satisfy the channel and the receiver: enough transitions for clock recovery (**transition density**), a bounded run of equal symbols (**run length**), zero average (**DC balance**) for AC-coupled links, special symbols for **framing and alignment**, and sometimes error detection. Every property costs bandwidth, measured as **coding efficiency**.

data bits	1	0	1	1	0	0	0	1	
NRZ									level = bit
NRZI (USB2)									0 = toggle
Manchester									1 = low->high
Manchester has a transition in the middle of every bit: the clock is always there.									

Figure 2.4 - NRZ, NRZI (USB 2.0 convention: a '0' toggles the line) and Manchester for the same bits.

Code	Efficiency	DC balance / run length	Used by
NRZ	100%	None; unbounded runs	On-chip, UART, SPI; high-speed links only together with scrambling
NRZI + bit stuffing	~ 100% minus stuffing (worst case 1 bit in 7)	Stuff a 0 after six 1s	USB 2.0 (bit stuffing), HDLC-style framing
Manchester	50% (2 baud per bit)	Perfect DC balance, a transition every bit	10BASE-T Ethernet, many RFID/IR links, DALI
8b/10b	80%	Running disparity bounds DC; max run 5	PCIe Gen1/2, USB 3.x Gen1, SATA, 1000BASE-X, DisplayPort 1.x, JESD204B
64b/66b	96.97%	Scrambled payload + 2-bit sync header	10G/25G/100G Ethernet PCS, JESD204C, Interlaken
128b/130b	98.46%	Scrambled + 2-bit sync header	PCIe Gen3/4/5
128b/132b	96.97%	Scrambled + 4-bit header	USB 3.2 Gen 2, DisplayPort 2.x
PAM4 (+ FEC)	2 bits per symbol	Gray-coded 4 levels; precoding options	PCIe Gen6 (FLIT mode), 200G/400G/800G Ethernet, 112G SerDes

Table 2.5 - Line codes. Efficiency is payload bits / line bits.

8b/10b in detail

IBM's 8b/10b code (Widmer and Franaszek, 1983) splits each byte **HGF EDCBA** into a 5-bit part **EDCBA** (value x) and a 3-bit part **HGF** (value y), written **D.x.y** for data and **K.x.y** for control characters. The 5-bit part becomes six bits **abcdei** and the 3-bit part four bits **fghj**; bit **a** is transmitted first. Each sub-block has either equal numbers of ones and zeros (**disparity 0**) or a disparity of +2. The encoder tracks the **running disparity (RD)**: when RD is negative it chooses the variant with more ones, when positive the one with more zeros, so the line never drifts more than a few bits away from DC balance.

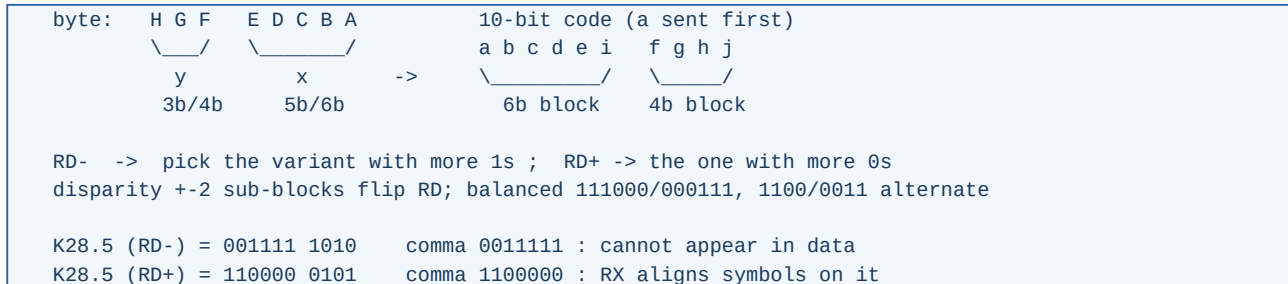


Figure 2.5 - 8b/10b structure, running disparity and the K28.5 comma.

Three special cases trip up every first implementation: the alternate encoding **D.x.A7** (**0111/1000** instead of **1110/0001**) for x = 17, 18, 20 with RD- and x = 11, 13, 14 with RD+, which prevents a run of five equal bits across the **i/f** boundary; the balanced sub-blocks **111000/000111** and **1100/0011**, which alternate with RD even though they do not change it; and the K-character 4-bit codes, whose RD+ forms are complements even when balanced. The encoder below stores only the RD- table and derives RD+ codes by complementing.

```
// 8b/10b encoder (combinational). code[9:0] = {a,b,c,d,e,i,f,g,h,j}; 'a' is sent first.
// Stores only the RD- column; the RD+ code is the bitwise complement whenever the
// sub-block is unbalanced or is 111000 / 1100 (balanced codes that alternate), and
// for every K-character 4b sub-block.
module enc8b10b (
    input logic [7:0] d,          // HGF EDCBA
    input logic k,               // 1 = control (K) character
    input logic rd_in,           // running disparity: 0 = RD-, 1 = RD+
    output logic [9:0] code,
    output logic rd_out
);
    logic [4:0] x; logic [2:0] y;
    logic [5:0] s6m, s6; logic [3:0] s4m, s4; logic rd_mid;
    assign x = d[4:0]; assign y = d[7:5];

    function automatic logic flips6(input logic [5:0] s); // unbalanced or 111000
        return ($countones(s) != 3) || (s == 6'b111000);
    endfunction
    function automatic logic flips4(input logic [3:0] s);
        return ($countones(s) != 2) || (s == 4'b1100);
    endfunction

    always_comb begin
        case (x) // 5b/6b, RD- column, abcdei
            5'd0: s6m = 6'b100111; 5'd1: s6m = 6'b011101; 5'd2: s6m = 6'b101101;
            5'd3: s6m = 6'b110001; 5'd4: s6m = 6'b110101; 5'd5: s6m = 6'b101001;
            5'd6: s6m = 6'b011001; 5'd7: s6m = 6'b111000; 5'd8: s6m = 6'b111001;
            5'd9: s6m = 6'b100101; 5'd10: s6m = 6'b010101; 5'd11: s6m = 6'b110100;
            5'd12: s6m = 6'b001101; 5'd13: s6m = 6'b101100; 5'd14: s6m = 6'b011100;
            5'd15: s6m = 6'b010111; 5'd16: s6m = 6'b011011; 5'd17: s6m = 6'b100011;
            5'd18: s6m = 6'b010011; 5'd19: s6m = 6'b110010; 5'd20: s6m = 6'b001011;
            5'd21: s6m = 6'b101010; 5'd22: s6m = 6'b011010; 5'd23: s6m = 6'b111010;
            5'd24: s6m = 6'b110011; 5'd25: s6m = 6'b100110; 5'd26: s6m = 6'b010110;
            5'd27: s6m = 6'b110110; 5'd28: s6m = k ? 6'b001111 : 6'b001110;
            5'd29: s6m = 6'b101110; 5'd30: s6m = 6'b011110; default: s6m = 6'b101011;
```

```

endcase
s6      = (rd_in && flips6(s6m)) ? ~s6m : s6m;
rd_mid = ($countones(s6) > 3) ? 1'b1 : ($countones(s6) < 3) ? 1'b0 :
        (s6 == 6'b000111) ? 1'b1 : (s6 == 6'b111000) ? 1'b0 : rd_in;

if (k) case (y) // 3b/4b for K28.y (x.7 also for K23/27/29/30), fghj
    3'd0: s4m = 4'b1011; 3'd1: s4m = 4'b0110; 3'd2: s4m = 4'b1010; 3'd3: s4m = 4'b1100;
    3'd4: s4m = 4'b1101; 3'd5: s4m = 4'b0101; 3'd6: s4m = 4'b1001; default: s4m = 4'b0111;
endcase
else case (y)
    3'd0: s4m = 4'b1011; 3'd1: s4m = 4'b1001; 3'd2: s4m = 4'b0101; 3'd3: s4m = 4'b1100;
    3'd4: s4m = 4'b1101; 3'd5: s4m = 4'b1010; 3'd6: s4m = 4'b0110;
    // D.x.A7 avoids a run of five equal bits across the i/f boundary
    default: s4m = ((!rd_mid && (x == 17 || x == 18 || x == 20)) ||
        (rd_mid && (x == 11 || x == 13 || x == 14))) ? 4'b0111 : 4'b1110;
endcase
// every K 4b pair is a complement pair (even balanced K.x.1/2/5/6)
s4      = (rd_mid && (k || flips4(s4m))) ? ~s4m : s4m;
rd_out = ($countones(s4) > 2) ? 1'b1 : ($countones(s4) < 2) ? 1'b0 :
        (s4 == 4'b0011) ? 1'b1 : (s4 == 4'b1100) ? 1'b0 : rd_mid;
code    = {s6, s4};
end
endmodule

```

8b/10b encoder: 5b/6b and 3b/4b tables, running disparity, D.x.A7 and K characters (K28.0-K28.7, K23.7, K27.7, K29.7, K30.7).

The reference model is written differently on purpose - as the two published tables, RD- and RD+ columns typed separately - so that a misunderstanding cannot be copied into both. It produces a hex file of 536 golden vectors ({code, RD_out} for every D and K character from both starting disparities).

```

# Independent Python reference: dictionary tables straight from the 8b/10b code tables
D6 = {0: ("0100111", "011000"), 1: ("011101", "100010"), 2: ("101101", "010010"), 3: ("110001", ) * 2,
      4: ("110101", "001010"), 5: ("101001", ) * 2, 6: ("011001", ) * 2, 7: ("111000", "000111"),
      8: ("111001", "000110"), 9: ("100101", ) * 2, 10: ("010101", ) * 2, 11: ("110100", ) * 2, 12: ("001101", ) * 2,
      13: ("101100", ) * 2, 14: ("011100", ) * 2, 15: ("010111", "101000"), 16: ("011011", "100100"),
      17: ("100011", ) * 2, 18: ("010011", ) * 2, 19: ("110010", ) * 2, 20: ("001011", ) * 2, 21: ("101010", ) * 2,
      22: ("011010", ) * 2, 23: ("111010", "000101"), 24: ("110011", "001100"), 25: ("100110", ) * 2,
      26: ("010110", ) * 2, 27: ("110110", "001001"), 28: ("001110", ) * 2, 29: ("101110", "010001"),
      30: ("011110", "100001"), 31: ("101011", "010100")}
K28_6 = ("001111", "110000")
D4 = {0: ("1011", "0100"), 1: ("1001", ) * 2, 2: ("0101", ) * 2, 3: ("1100", "0011"), 4: ("1101", "0010"),
      5: ("1010", ) * 2, 6: ("0110", ) * 2, 7: ("1110", "0001")}
A7 = ("0111", "1000")
K4 = {0: ("1011", "0100"), 1: ("0110", "1001"), 2: ("1010", "0101"), 3: ("1100", "0011"),
      4: ("1101", "0010"), 5: ("0101", "1010"), 6: ("1001", "0110"), 7: ("0111", "1000")}
KCODES = [(28, y) for y in range(8)] + [(23, 7), (27, 7), (29, 7), (30, 7)]

def rd_after(sub, rd): # rd: 0 = RD-, 1 = RD+
    ones = sub.count("1"); zeros = len(sub) - ones
    if ones > zeros: return 1
    if ones < zeros: return 0
    if sub in ("000111", "0011"): return 1
    if sub in ("111000", "1100"): return 0
    return rd

def enc(byte, k, rd):
    x, y = byte & 31, byte >> 5
    s6 = (K28_6 if (k and x == 28) else D6[x])[rd]
    rd = rd_after(s6, rd)
    if k:
        s4 = K4[y][rd]
    elif y == 7 and ((rd == 0 and x in (17, 18, 20)) or (rd == 1 and x in (11, 13, 14))):

```

```

        s4 = A7[rd]
    else:
        s4 = D4[y][rd]
    rd = rd_after(s4, rd)
    return s6 + s4, rd

if __name__ == "__main__":
    lines = []
    for k in (0, 1):
        vals = range(256) if not k else [x | (y << 5) for x, y in KCODES]
        for b in vals:
            for rd in (0, 1):
                c, r = enc(b, k, rd)
                lines.append("%03x" % ((int(c, 2) << 1) | r))
    open("golden_8b10b.hex", "w").write("\n".join(lines) + "\n")
    print(len(lines), "vectors")
    for name, b, k in [("D0.0", 0, 0), ("D21.5", 0xB5, 0), ("K28.5", 0xBC, 1)]:
        print(name, enc(b,k,0)[0], enc(b,k,1)[0])

```

Independent Python reference (ref8b10b.py) that writes golden_8b10b.hex.

```

536 vectors
D0.0 1001110100 0110001011
D21.5 1010101010 1010101010
K28.5 0011111010 1100000101

```

Real Python output (code shown as abcdei fghj; RD- and RD+ variants).

```

module tb;
    logic [7:0] d; logic k, rd_in, rd_out; logic [9:0] code;
    logic [10:0] gold [536]; // {code, rd_out} from ref8b10b.py
    // K28.0..K28.7, K23.7, K27.7, K29.7, K30.7
    function automatic logic [7:0] kbyte(input int i);
        return (i < 8) ? {i[2:0], 5'd28} : (i == 8) ? 8'hF7 : (i == 9) ? 8'hFB
            : (i == 10) ? 8'hFD : 8'hFE;

    endfunction
    enc8b10b dut (.*)
    int n = 0, bad = 0, run = 0, maxrun = 0, rds = 0, minrds = 0, maxrds = 0;
    logic last = 1'b0;
    initial begin
        $readmemh("golden_8b10b.hex", gold);
        // 1) exhaustive check against the Python reference: 256 D + 12 K codes, both RDs
        for (int kk = 0; kk < 2; kk++)
            for (int i = 0; i < (kk ? 12 : 256); i++)
                for (int r = 0; r < 2; r++) begin
                    k = kk; rd_in = r; d = kk ? kbyte(i) : i[7:0]; #1;
                    if ({code, rd_out} != gold[n]) bad++;
                    n++;
                end
        $display("table check : %0d codes vs Python reference, mismatches=%0d", n, bad);
        // 2) a few famous characters starting from RD-
        k = 0; rd_in = 0;
        d = 8'h00; #1 $display("D0.0 RD- -> %b %b", code[9:4], code[3:0]);
        d = 8'hB5; #1 $display("D21.5 RD- -> %b %b", code[9:4], code[3:0]);
        k = 1; d = 8'hBC; #1 $display("K28.5 RD- -> %b %b (comma 0011111)", code[9:4], code[3:0]);
        rd_in = 1; #1 $display("K28.5 RD+ -> %b %b (comma 1100000)", code[9:4], code[3:0]);
        // 3) 100k random characters as a serial stream: run length and running digital sum
        rd_in = 0;
        for (int c = 0; c < 100000; c++) begin
            k = ($urandom % 16 == 0); d = k ? kbyte($urandom % 12) : $urandom; #1;
            for (int b = 9; b >= 0; b--) begin
                run = (code[b] == last) ? run + 1 : 1; last = code[b];
                if (run > maxrun) maxrun = run;
                rds += code[b] ? 1 : -1;
            end
        end
    end

```



```

        if (rds < minrds) minrds = rds; if (rds > maxrds) maxrds = rds;
    end
    rd_in = rd_out;
end
$display("stream      : 1000000 bits, max run length=%0d, digital sum in [%0d,%0d]",
        maxrun, minrds, maxrds);
$finish;
end
endmodule

```

Testbench: exhaustive table comparison, famous characters, and a one-million-bit random stream with a 1-in-16 chance of a K character.

```

table check : 536 codes vs Python reference, mismatches=0
D0.0 RD- -> 100111 0100
D21.5 RD- -> 101010 1010
K28.5 RD- -> 001111 1010 (comma 0011111)
K28.5 RD+ -> 110000 0101 (comma 1100000)
stream      : 1000000 bits, max run length=5, digital sum in [-2,4]

```

Real Icarus Verilog output: all 536 codes match Python; the stream never exceeds a run of 5 and the running digital sum stays within a band of 6.

WHY THE COMMA MATTERS

A receiver that has just locked its CDR sees an endless bit stream with no idea where 10-bit symbols start. The **comma** sequences 0011111 and 1100000 occur only inside K28.1, K28.5 and K28.7 and never across data character boundaries, so the PCS slides its symbol boundary until a comma lands in the right place (comma alignment). PCIe Gen1/2 uses K28.5 as COM at the start of every ordered set for exactly this reason.

64b/66b and 128b/130b: block codes with sync headers

8b/10b's 25% overhead is too costly above ~5 Gb/s. Newer codes scramble the payload for transitions and DC balance and add only a short **sync header** per block. 64b/66b (10GBASE-R) prefixes each 64-bit block with **01** (data) or **10** (control); **00** and **11** are invalid, so the header always contains a transition and the receiver finds block boundaries by hunting for a position where headers are consistently valid. PCIe Gen3-5's 128b/130b uses the same idea with a 2-bit header per 128-bit block (**10** = data, **01** = ordered set). DC balance is only statistical, so the scrambler is mandatory.

64b/66b block:	2-bit sync	64 bits scrambled payload ($x^{58}+x^{39}+1$)	
	01 = data	10 = control	00/11 = invalid
128b/130b block:	2-bit sync	16 bytes scrambled payload	
	10 = data	01 = ordered set	(PCIe Gen3-Gen5)

Figure 2.6 - Block codes with sync headers.

PAM4

NRZ sends one bit per symbol (two levels). **PAM4** sends two bits per symbol using four levels, halving the symbol rate for the same bit rate and therefore the channel loss at Nyquist. The price: three stacked eyes, each about a third of the NRZ eye height (roughly a 9.5 dB SNR penalty), more sensitivity to non-linearity, and a raw bit error ratio that requires **forward error correction**. PCIe 6.0 (64 GT/s) pairs PAM4 with fixed-size FLITs and a lightweight FEC plus CRC; 400G/800G Ethernet uses RS(544,514) FEC. Gray coding (levels 00, 01, 11, 10) makes a one-level slip cost only one bit.

Manchester coding with clock recovery - run example

Manchester coding guarantees a transition in the middle of every bit, so the receiver can recover the clock with simple logic. The IEEE 802.3 convention is '1' = low-to-high and '0' = high-to-low at mid-bit. The decoder below oversamples with its own, unrelated clock at 8 samples per bit. It locks on the first edge (a preamble of 1010... contains only mid-bit edges), then ignores any edge during the first three quarters of a bit - those are bit-boundary edges - and re-times itself on each mid-bit edge. That re-timing is a tiny digital CDR.


```

// Manchester, IEEE 802.3 convention: '1' = low->high at mid-bit, '0' = high->low.
module manch_tx (
    input  logic clk,           // runs at 2x the bit rate: one half-bit per cycle
    input  logic rst_n,
    input  logic bit_in,       // sampled at the start of each bit
    output logic bit_req,      // pulses when bit_in is consumed
    output logic line
);
    logic half, cur;
    assign bit_req = !half;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin half <= 1'b0; cur <= 1'b0; line <= 1'b0; end
        else begin
            if (!half) begin cur <= bit_in; line <= ~bit_in; end // first half: complement
            else line <= cur; // second half: the bit
            half <= ~half;
        end
endmodule

// Decoder: samples the line at OSR samples per bit with its own, unrelated clock.
// It locks on the first edge (preamble of 1010... guarantees that is a mid-bit edge),
// then ignores edges for 3/4 of a bit so bit-boundary edges are skipped, and re-times
// itself on every mid-bit edge - this is the clock recovery.
module manch_rx #(parameter int OSR = 8) (
    input  logic clk, rst_n,
    input  logic line_in, // asynchronous to clk
    output logic bit_valid,
    output logic bit_out,
    output logic locked
);
    logic [2:0] sync; // 2-flop synchronizer + previous sample
    logic [$clog2(2*OSR)-1:0] cnt;
    wire s = sync[1], edge_det = sync[1] ^ sync[2];
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            sync <= '0; cnt <= '0; locked <= 1'b0; bit_valid <= 1'b0; bit_out <= 1'b0;
        end else begin
            sync <= {sync[1:0], line_in};
            bit_valid <= 1'b0;
            if (!locked) begin
                if (edge_det) begin locked <= 1'b1; cnt <= '0; bit_valid <= 1'b1; bit_out <= s; end
            end else if (edge_det && cnt >= (3*OSR)/4 - 1) begin // a mid-bit edge
                cnt <= '0; bit_valid <= 1'b1; bit_out <= s; // rising -> 1
            end else if (cnt == (3*OSR)/2) begin // no mid-bit edge: lost
                locked <= 1'b0;
            end else cnt <= cnt + 1'b1;
        end
endmodule

```

Manchester encoder (one half-bit per clock) and oversampling decoder with edge-based clock recovery.

```

rx clock error +0.0%: sent 2012 bits, decoded 2012, bit errors=0, locked=1
rx clock error +10.0%: sent 2012 bits, decoded 2012, bit errors=0, locked=1
rx clock error -10.0%: sent 2012 bits, decoded 2012, bit errors=0, locked=1
rx clock error +20.0%: sent 2012 bits, decoded 2012, bit errors=0, locked=1
rx clock error +30.0%: sent 2012 bits, decoded 2059, bit errors=3, locked=1
rx clock error -30.0%: sent 2012 bits, decoded 1428, bit errors=740, locked=1

```

Real Icarus output: 2 000 random bits after an 8-bit preamble at 12.5 Mb/s, for different receiver clock errors (TB tb_manch.sv, +OFS plusarg).

Because the decoder re-times on every bit, it tolerates enormous clock errors - +20% is error-free - compared with UART, which must survive a whole frame without resynchronization (Section 4.3). At +30% the ignore

window becomes too short relative to the slower incoming bits and boundary edges start to be accepted as data; at -30% the decoder loses bits. The cost of this robustness is the 2x bandwidth of Manchester coding.

2.7 Scrambling

A **scrambler** XORs the data with a pseudo-random sequence from a linear-feedback shift register (LFSR). It does not add bits, yet it makes long runs and repetitive patterns statistically rare, spreads the spectrum (lower EMI peaks) and gives the CDR transitions. Two architectures exist, with opposite trade-offs.

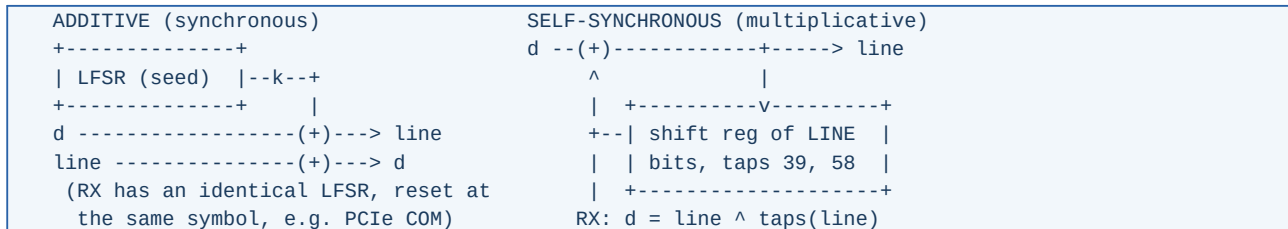


Figure 2.7 - Additive scrambling needs synchronized LFSRs; self-synchronous scrambling feeds back the line bits themselves.

	Additive (synchronous)	Self-synchronous
Keystream	Independent LFSR	Function of previous line bits
Synchronization	Both LFSRs must be reset together (explicit sync symbol or ordered set)	Automatic after N bits (N = LFSR length)
Error propagation	None: 1 line error -> 1 data error	1 line error -> one error per polynomial term (3 for $x^{58}+x^{39}+1$)
Examples	PCIe Gen1/2 and USB 3 Gen1 ($X^{16}+X^5+X^4+X^3+1$), PCIe Gen3+ (23-bit LFSR per lane), DisplayPort	64b/66b Ethernet ($x^{58}+x^{39}+1$), SONET/SDH payloads, JESD204C

Table 2.6 - Scrambler architectures.

The RTL below implements both. The additive scrambler is written byte-parallel: a loop of eight serial LFSR steps inside one combinational block, which synthesis flattens into an XOR network - the same unrolling technique used for parallel CRCs in Chapter 3. Its keystream is checked against a Python model of the same polynomial, seed FFFF, LSB first.

```

// Additive (synchronous) scrambler, G(X) = X^16+X^5+X^4+X^3+1 (the PCIe Gen1/2 and
// USB 3.x polynomial), 8 bits per clock, LSB first. Same module scrambles and
// descrambles: data XOR keystream. Both ends must reset the LFSR at the same symbol.
module scr_add (
    input logic      clk, rst_n,
    input logic      init,          // re-seed (PCIe: on every COM symbol)
    input logic      en,
    input logic [7:0] din,
    output logic [7:0] dout
);
    logic [15:0] lfsr, nxt;
    logic [7:0] o;
    always @* begin                                // (Icarus: always_comb + loop selects)
        nxt = lfsr; o = '0;
        for (int i = 0; i < 8; i++) begin          // unrolled: 8 serial steps per clock
            o = {din[i] ^ nxt[15], o[7:1]};        // bit i ends up in o[i] after 8 shifts
            nxt = {nxt[14:0], 1'b0} ^ (nxt[15] ? 16'h0039 : 16'h0000);
        end
    end
    assign dout = o;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) lfsr <= 16'hFFFF;
        else if (init) lfsr <= 16'hFFFF;
        else if (en) lfsr <= nxt;
end

```

```

endmodule

// Self-synchronous scrambler  $x^{58} + x^{39} + 1$  (the 64b/66b polynomial), 1 bit/clock.
module scr_ss58 #(parameter bit DESCRAMBLE = 0) (
    input  logic clk, rst_n, en,
    input  logic din,
    output logic dout
);
    logic [57:0] s; // the last 58 LINE bits
    wire  fb = s[38] ^ s[57];
    assign dout = din ^ fb;
    wire  line_bit = DESCRAMBLE ? din : dout; // the history always holds line bits
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) s <= DESCRAMBLE ? '0 : {58{1'b1}}; // deliberately different states
        else if (en) s <= {s[56:0], line_bit};
endmodule

```

Byte-parallel additive scrambler (PCIe Gen1/2 / USB 3 Gen1 polynomial) and a bit-serial self-synchronous $x^{58}+x^{39}+1$ scrambler/descrambler.

```

def pcie_scramble(data, seed=0xFFFF):
    """Additive scrambler,  $G(X)=X^{16}+X^5+X^4+X^3+1$ , Galois form, LSB of each byte first."""
    l, out = seed, []
    for b in data:
        o = 0
        for i in range(8):
            fb = (l >> 15) & 1
            o |= (((b >> i) & 1) ^ fb) << i
            l = ((l << 1) & 0xFFFF) ^ (0x0039 if fb else 0)
        out.append(o)
    return out
if __name__ == "__main__":
    ks = pcie_scramble([0]*16)
    print(" ".join("%02X" % x for x in ks))
    msg = b"SoC links!"
    print(" ".join("%02X" % x for x in pcie_scramble(msg)))

```

Python reference for the additive scrambler.

```

FF 17 C0 14 B2 E7 02 82 72 6E 28 A6 BE 6D BF 8D
AC 78 83 34 DE 8E 6C E9 01 4F

```

Real Python output: keystream (scrambling sixteen 0x00 bytes) and a scrambled ASCII string.

```

keystream (data=00): ff 17 c0 14 b2 e7 02 82
additive: 64 keystream bytes vs Python reference, mismatches=0
additive: 1 line-bit error -> 1 bit error(s) after descrambling
self-sync: wrong bits before lock=19 (all within the first 58)
self-sync: 1 line-bit error -> 3 bit errors (3-term polynomial)

```

Real Icarus output from tb_scram.sv: RTL keystream equals Python for 64 bytes; error propagation of both architectures.

The keystream **FF 17 C0 14 B2 E7 02 82 ...** is also the well-known start of the Gen1 scrambling sequence that appears in PCIe and USB 3 debug traces - useful to recognize on a protocol analyzer. The self-synchronous pair demonstrates both of its properties: the descrambler started in the wrong state and still produced correct data after at most 58 bits (19 wrong bits here), and one flipped line bit produced exactly three data errors.

PITFALL - SCRAMBLER BUGS THAT PASS SIMPLE TESTS

- Scrambling and descrambling with the same wrong polynomial or bit order still round-trips perfectly in a loopback test. Always compare against the specification's published sequence or an independent model, as done here.
- Additive scramblers must skip or reset on exactly the symbols the spec says (PCIe Gen1/2 does not advance on SKP symbols and resets on COM). An off-by-one here corrupts every byte after the first SKP ordered set.
- Bit order: most serial specs send LSB first; parallel data paths in your RTL may be MSB-first. Document the convention in the module header.

2.8 SerDes architecture

A **SerDes** (serializer/deserializer) transceiver is the PHY of every multi-gigabit link. Figure 2.8 shows its blocks; the PCS portion is RTL, the rest is analog/mixed-signal.

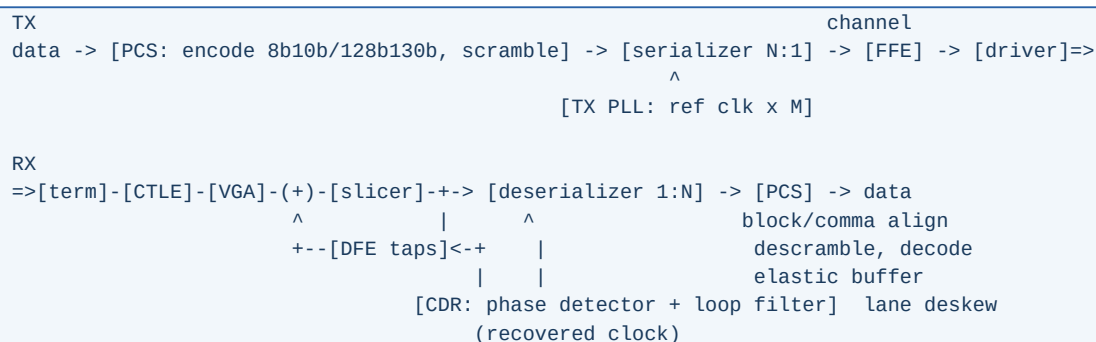


Figure 2.8 - SerDes transceiver. FFE = feed-forward equalizer (TX de-emphasis/pre-shoot), CTLE = continuous-time linear equalizer, DFE = decision-feedback equalizer, CDR = clock-data recovery.

Block	Function	Owner
PLL	Multiplies a clean reference (e.g. 100 MHz for PCIe) to the bit-rate clock (often half or quarter rate with multi-phase outputs)	Analog
Serializer	Parallel-to-serial conversion (e.g. 32:1) in a tree of multiplexers	Analog/custom digital
TX FFE	A short FIR filter (pre-cursor, main, post-cursor taps) that pre-distorts the signal to cancel channel low-pass loss - PCIe presets and de-emphasis	Analog, coefficients set by link training
RX CTLE	Analog high-pass peaking filter that boosts high frequencies attenuated by the channel	Analog, adaptive
RX DFE	Subtracts the ISI of previous decided bits (post-cursor taps); does not amplify noise, unlike a linear equalizer	Mixed-signal, adaptive
CDR	Phase detector (e.g. bang-bang/Alexander) plus loop filter steering a phase interpolator or VCO to the eye centre	Mixed-signal
Deserializer	Serial-to-parallel conversion, producing the PCS-side word clock	Custom digital
PCS	Encoding/decoding, scrambling, block/comma alignment, elastic buffer (clock compensation), lane-to-lane deskew, PRBS generators/checkers, loopback	RTL

Table 2.7 - SerDes building blocks.

Modern links **train** their equalizers at start-up: in PCIe Gen3+ the two ends negotiate TX FFE presets and coefficients during the Recovery.Equalization phase of the LTSSM, while each receiver adapts its CTLE and DFE. The controller RTL drives this process through the PIPE interface; the adaptation algorithms live inside the PHY, often in firmware.

2.9 Jitter, eye diagrams and bit error ratio

Jitter is the deviation of signal edges from their ideal positions in time. It is decomposed because different components add differently:

Component	Cause	Distribution
Random jitter (RJ)	Thermal and device noise in PLLs, drivers, receivers	Gaussian, unbounded; specified as RMS sigma
Data-dependent jitter (DDJ/ISI)	Channel loss and reflections: an edge's timing depends on previous bits	Bounded; reduced by equalization
Duty-cycle distortion (DCD)	Unequal rise/fall or clock duty cycle	Bounded, two values
Periodic jitter (PJ)	Coupling from supplies, spread-spectrum clocking, crosstalk at fixed frequencies	Bounded, sinusoidal
Bounded uncorrelated (BUJ)	Crosstalk from other lanes	Bounded

Table 2.8 - Jitter components. DJ = all bounded parts.

Overlaying many bit periods of the received waveform on an oscilloscope gives the **eye diagram**. The open area in the middle is where the receiver can sample safely; its height is the voltage margin and its width the timing margin.



Figure 2.9 - Eye diagram with a compliance mask. One unit interval (UI) = one bit time.

Because random jitter is Gaussian, the eye keeps closing slowly as you look for rarer events. Total jitter is therefore quoted **at a bit error ratio**, typically 10^{-12} :

$$TJ(BER) = DJ(\text{dual-Dirac}) + 2 \times Q(BER) \times RJ_{rms}$$

$$Q(1e-12) = 7.03 \rightarrow TJ(1e-12) = DJ + 14.07 \times RJ_{rms}$$

bits needed to claim BER < 1e-12 with 95% confidence and 0 errors ~ 3 / 1e-12 = 3e12

Eq. 2.5 - The dual-Dirac jitter model and BER test length. At 10 Gb/s, 3e12 bits take 5 minutes.

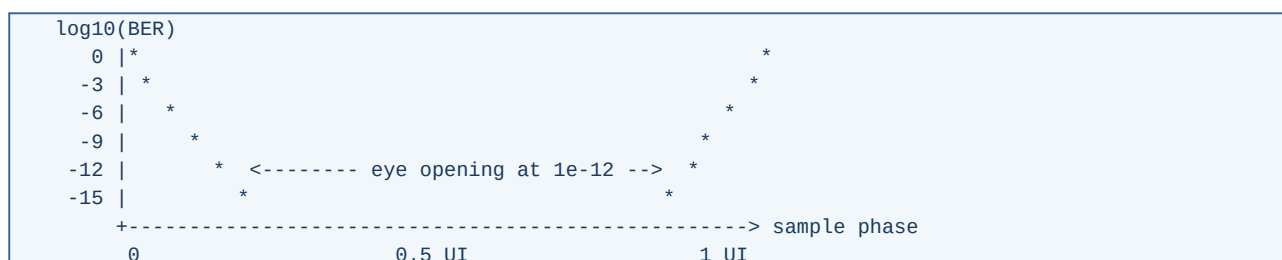


Figure 2.10 - Bathtub curve: BER versus sampling phase. The walls are the tails of the jitter distributions; the flat floor is where the CDR should sample.

INTERVIEW INSIGHT: BER IS A SYSTEM NUMBER

A 'BER of 1e-12' at 32 GT/s means an error every ~31 seconds per lane - about one every two seconds on an x16 link. That is why every high-speed protocol pairs its PHY with link-level CRC and retry (Chapter 3) and, at PAM4 rates, FEC: the PHY makes errors rare, the link layer makes them invisible.

2.10 Summary

- Single-ended CMOS signalling is simple but limited by shared ground, SSO noise and full swing; noise margins come from V_{OH}/V_{IH} and V_{IL}/V_{OL} .
- Open-drain buses give wired-AND behaviour; the RC pull-up sets the rise time ($t_r = 0.8473 R_p C_b$ for 30-70%) and bounds the bus capacitance and speed.
- Differential signalling (LVDS, CML, SLVS) rejects common-mode noise and enables small swings and gigabit rates; termination matches the line to suppress reflections.
- Clocking scheme - common, source-synchronous, forwarded, or embedded with CDR - determines an interface's timing budget.
- Line codes (NRZI, Manchester, 8b/10b, 64b/66b, 128b/130b, PAM4) trade efficiency for transitions, DC balance and framing; the 8b/10b RTL matched an independent Python model on all 536 codes.
- Scramblers are additive (need sync, no error spread) or self-synchronous (auto-sync, error multiplication).
- SerDes = PLL, serializer, FFE, driver, CTLE, DFE, CDR, deserializer and PCS; equalizers are trained at link start-up.
- Jitter ($RJ + DJ$) closes the eye; TJ and BER must be quoted together, and link-layer CRC/retry handles the residual errors.

2.11 Exercises

- A 1.8 V I2C bus has 250 pF of capacitance and must meet the Fast-mode 300 ns rise time. Compute R_{p_max} and R_{p_min} for a 3 mA sink at $V_{OL} = 0.4$ V. Is there a valid resistor?
- Encode the bytes 0x00, 0xBC (as K28.5) and 0xF1 starting from RD-, by hand, using Figure 2.5 and the tables in the encoder. Check your answer with ref8b10b.py.
- Write the matching 8b/10b decoder with code-violation and disparity-error outputs. Close the loop with enc8b10b and inject single-bit errors: what fraction are detected?
- Modify manch_rx to use 4 samples per bit. Measure the new clock-error tolerance with the +OFS plusarg and explain the result.
- Show that a single line-bit error through the self-synchronous $x^{58} + x^{39} + 1$ descrambler produces errors at offsets 0, 39 and 58. Why does 64b/66b not need to reset its scrambler, while PCIe Gen1 does?
- Using Eq. 2.5, a link has $DJ = 0.15$ UI and $RJ = 0.02$ UI RMS. What is the eye opening at BER $1e-12$ and at $1e-15$?

3. Handshakes, Flow Control, Ordering and Error Detection

Chapter 2 moved individual bits. This chapter is about **agreement**: how two sides agree that a transfer happened (handshakes), how a sender avoids overrunning a receiver (flow control), what order operations take effect in (ordering), how a system avoids waiting forever (deadlock), and how corrupted data is detected and repaired (parity, checksums, CRC, ECC and retry). These mechanisms recur in every protocol from APB to PCIe; learn them once here. The run examples are a credit-based link, bit-serial and byte-parallel CRC generators checked against Python's `zlib.crc32`, and a (72,64) SECDED encoder/decoder with error injection.

3.1 The valid/ready handshake

The most common on-chip handshake is **valid/ready** (AXI's VALID/READY, AXI4-Stream's TVALID/TREADY, and countless internal pipelines). The source drives **valid** with the payload; the destination drives **ready**. A transfer happens on **every clock edge where both are high** - no more, no less.

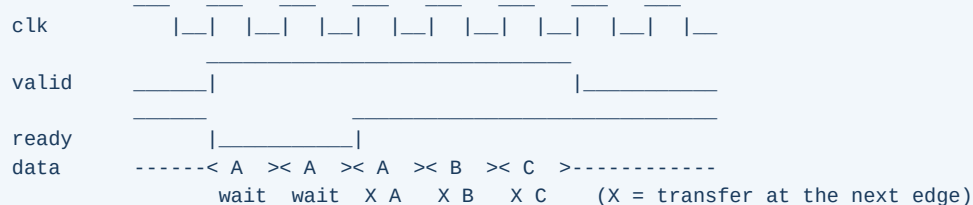


Figure 3.1 - Valid/ready: the source holds A through two stalled cycles; B and C then transfer back to back.

valid	ready	Meaning
0	0	Idle
0	1	Destination can accept; nothing offered (allowed)
1	0	Offer pending, destination stalls - source must hold valid and data
1	1	Transfer on this edge; source may present the next item in the next cycle

PITFALL - THE FOUR CLASSIC VALID/READY BUGS

- **Waiting for ready before raising valid.** If both sides wait for the other, the system deadlocks. AXI explicitly forbids the source from waiting; the destination may wait.
- **Retracting valid** (or changing data) before the transfer - e.g. an arbiter that switches to another requester while the current one is stalled. Chapter 4's checker (rule R5) and the SVA `a_hold` catch exactly this.
- **Combinational loop:** ready computed from valid on one side and valid from ready on the other.
- **Counting a transfer on valid alone** in a monitor or scoreboard, which double-counts stalled items.

Pipelining ready: skid buffers

In a long pipeline, **ready** becomes a long combinational path running backwards through every stage. Registering it breaks the path but introduces a one-cycle lag: when the downstream stage drops ready, the upstream stage does not see it until the next cycle and sends one more item. A **skid buffer** is a two-entry buffer that absorbs that extra item so no data is lost and full throughput is kept. The companion RTL Design guide builds one; here the key protocol insight is the general rule: **a stage whose ready is registered needs as many spare entries as the number of cycles between the stall and the sender seeing it**. Credit-based flow control, below, is the same rule scaled up to a long link.

3.2 Request/acknowledge: 2-phase and 4-phase handshakes

When there is no common clock - between clock domains, or in asynchronous logic - handshakes use **req/ack** signal events instead of sampled levels.

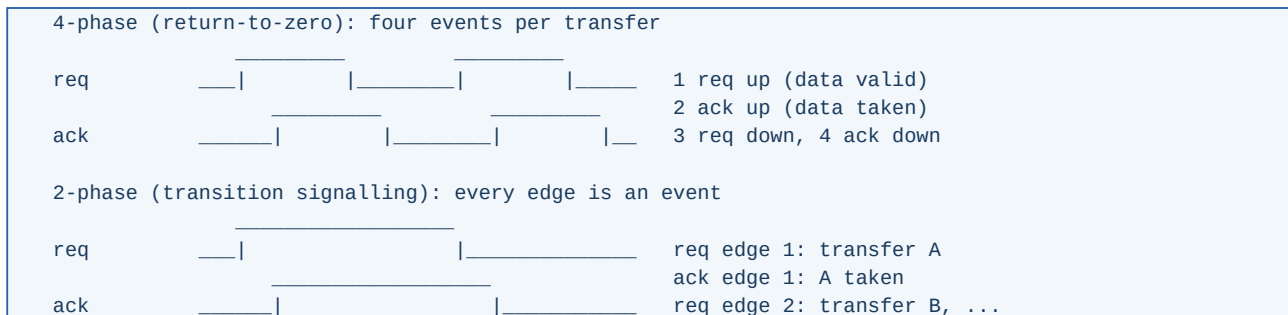


Figure 3.2 - 4-phase and 2-phase req/ack handshakes.

	4-phase (level)	2-phase (toggle)
Events per transfer	4	2
Logic	Simple level logic, easy to reason about	Needs XOR edge detectors / toggle flops
Throughput across a CDC	Lower: each transfer waits 2 round trips through synchronizers	About twice the 4-phase rate
Typical use	CDC of multi-bit control words, async bus bridges	Pulse synchronizers, toggle-based CDC handshakes

Both are the standard way to move a multi-bit value between clock domains: hold the data stable, send the request through a synchronizer, sample the data when the request arrives, and acknowledge. Chapter 4 revisits this for controllers whose interface clock differs from the system clock.

3.3 Flow control

Flow control prevents a fast sender from overflowing a slow receiver. The design question is always the same: when the receiver says 'stop', how much data is already in flight, and where does it go? Three families answer it differently.

Mechanism	Signal	Where used	Needs buffer headroom?
Back-pressure (ready/wait)	A per-cycle stall signal	AXI, AHB HREADY, APB PREADY, on-chip pipelines	Only the skid entries for registered ready
Start/stop (XON/XOFF, pause)	'Stop sending' / 'resume' messages or pins	UART RTS/CTS and XON/XOFF, Ethernet PAUSE and PFC	Yes: everything sent during the reaction time
Credit-based	The receiver grants a count of free buffers; the sender spends one per packet	PCIe, CXL, CHI, NoCs, USB4, InfiniBand	Buffers = credits; loss-free by construction

Table 3.1 - Flow-control families.

Credit-based flow control

With **credits**, the receiver advertises how many buffer slots it has. The sender keeps a counter, decrements it for every packet sent, and stops at zero; the receiver returns a credit each time it frees a slot. Overflow is impossible by construction - no stop signal can arrive too late. The cost is buffering: to keep the link busy, the credits must cover the whole **credit loop** - the time from sending a packet until its credit returns:

```
credits_needed >= ceil( RTT_credit_loop x rate / packet_size )  
throughput = min(1, credits / RTT)    (in packets per cycle, 1 packet per cycle max)
```

Eq. 3.1 - Credit sizing. RTT includes wire latency both ways plus processing and credit-return delays.


```

// Credit-based flow control: TX may send only while it holds a credit; RX returns one
// credit each time it frees a buffer slot. Both directions have LAT cycles of latency.
module credit_link #(parameter int CREDITS = 4, parameter int LAT = 3) (
    input  logic clk, rst_n,
    input  logic rx_drain,           // RX consumer takes one entry this cycle
    output logic sent, overflow,
    output int  occupancy
);
    logic [LAT-1:0] fwd, ret;        // data and credit-return pipelines
    int credits;
    assign sent = (credits > 0);      // TX always has data: send whenever allowed
    wire pop = rx_drain && (occupancy > 0);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            fwd <= '0; ret <= '0; credits <= CREDITS; occupancy <= 0; overflow <= 1'b0;
        end else begin
            fwd <= {fwd[LAT-2:0], sent}; // data takes LAT cycles to arrive
            ret <= {ret[LAT-2:0], pop};   // a credit takes LAT cycles to come back
            credits <= credits - sent + ret[LAT-1];
            occupancy <= occupancy + fwd[LAT-1] - pop;
            if (occupancy + fwd[LAT-1] - pop > CREDITS) overflow <= 1'b1; // must never happen
        end
    endmodule

```

A credit-based link with 3-cycle latency in each direction: data pipeline, credit-return pipeline, occupancy and an overflow detector.

credits= 2	throughput=0.250 flits/cycle	max RX occ=1	overflow=0
credits= 4	throughput=0.500 flits/cycle	max RX occ=1	overflow=0
credits= 6	throughput=0.750 flits/cycle	max RX occ=1	overflow=0
credits= 8	throughput=1.000 flits/cycle	max RX occ=1	overflow=0
credits=12	throughput=1.000 flits/cycle	max RX occ=1	overflow=0
credits= 8 slow RX	throughput=0.504 flits/cycle	max RX occ=5	overflow=0

Real Icarus output: the credit loop is 8 cycles, so throughput is credits/8 until 8 credits; a slow receiver throttles the link without ever overflowing.

The measured throughput follows Eq. 3.1 exactly: 2 credits give 25%, 6 give 75%, and 8 - the round-trip time in cycles - saturate the link. More credits than the RTT buy nothing when the receiver keeps up; when it does not, credits simply cap the in-flight data and the sender waits.

How PCIe does it

PCIe keeps **six** credit types per virtual channel: header and data credits for each of Posted requests (memory writes, messages), Non-Posted requests (reads, configuration and I/O writes) and Completions - PH, PD, NPH, NPD, CplH, CplD. One header credit covers one TLP header; one data credit covers 16 bytes (4 DW) of payload. Credits are initialized by InitFC DLLPs after link-up and returned with UpdateFC DLLPs; an advertisement of 0 means 'infinite'. Counters are modular (8-bit header, 12-bit data in the classic scheme), and a TLP may be sent only if:

$$(\text{CREDIT_LIMIT} - (\text{CREDITS_CONSUMED} + \text{credits_for_this_TLP})) \bmod 2^N \leq 2^N / 2$$

Eq. 3.2 - PCIe credit gating with modulo- 2^N counters (N = counter width). Half the counter space distinguishes 'enough' from 'wrapped'.

INTERVIEW INSIGHT: WHY POSTED/NON-POSTED/COMPLETION ARE SEPARATE

If reads, writes and completions shared one buffer pool, a flood of reads could consume all space while the completions that would free it are stuck behind them - a protocol deadlock. Separate credit pools per class (and ordering rules that let posted writes pass blocked reads) guarantee that completions can always make progress.

XON/XOFF and pause frames

Start/stop flow control is simpler but needs **headroom**: the receiver must say 'stop' early enough to absorb everything already in flight. UART software flow control sends the ASCII XOFF (DC3, 0x13) and XON (DC1, 0x11) characters; hardware flow control uses the RTS/CTS pins (Chapter 11). Ethernet's IEEE 802.3x **PAUSE** frame is a MAC Control frame (EtherType 0x8808, opcode 0x0001, destination 01-80-C2-00-00-01) carrying a pause time in quanta of 512 bit times; **Priority Flow Control** (IEEE 802.1Qbb, opcode 0x0101) pauses each of eight priorities separately, which lossless data-centre fabrics rely on.

$$\text{headroom} \geq \text{rate} \times (\text{t_detect} + \text{t_send_stop} + \text{t_flight} \times 2 + \text{t_sender_react}) + \text{max_frame}$$

Eq. 3.3 - Buffer headroom above the XOFF threshold.

3.4 Retry: acknowledgements, windows and go-back-N

Detecting an error is only half the job; a reliable link must also **recover** by resending. The link keeps a copy of every unacknowledged packet in a **replay buffer** and numbers packets so that the receiver can say which ones arrived.

Scheme	How it works	Buffering	Examples
Stop-and-wait	Send one packet, wait for ACK, then the next	1 packet; throughput limited by RTT	Simple sensor links, USB 2.0 control transfers (per transaction)
Go-back-N	Send up to N unacknowledged packets; on NAK or timeout, resend from the first unacknowledged one	N packets at TX; RX needs no reorder buffer	PCIe data link layer, CXL.io, many SerDes link layers
Selective repeat	Resend only the missing packets; RX reorders	N packets at both ends	TCP with SACK, some die-to-die link layers
FEC (no retry)	Add redundancy to correct errors in place	None, but latency and bandwidth for parity	400G Ethernet RS-FEC, PCIe 6 FEC (combined with CRC + retry)

Table 3.2 - Retry and correction strategies.

```

TX seq:  10  11  12  13  14  15      replay buffer holds 10..15
RX:      ok   ok   BAD (13,14,15 discarded: out of sequence)
RX -> TX:           NAK(11)  'last good = 11'
TX:      purge 10,11 ; replay 12 13 14 15 ; continue with 16
RX -> TX:           ACK(15) -> purge 12..15

```

Figure 3.3 - Go-back-N as in the PCIe data link layer: the NAK carries the last good sequence number.

PCIe's data link layer is the canonical hardware example. Every TLP carries a 12-bit **sequence number** and a 32-bit **LCRC**. The receiver returns ACK or NAK DLLPs naming the last good sequence number; ACKs may be coalesced to save bandwidth. A **REPLAY_TIMER** triggers a replay when no acknowledgement arrives, and a 2-bit **REPLAY_NUM** counter escalates to link retraining after repeated replays. The window cannot exceed half the sequence space (2048 TLPs) so that old and new numbers are never confused.

$$\text{replay buffer bytes} \geq \text{link rate} \times (\text{ACK round-trip time} + \text{ACK coalescing delay}) + \text{max TLP size}$$

Eq. 3.4 - Replay-buffer sizing - the retry analogue of Eq. 3.1.

3.5 Ordering models

When several transactions are in flight, the protocol must say which may overtake which. Too strict and performance suffers; too loose and software sees stale data.

Model	Rule	Examples
Strict in-order	Everything completes in issue order	APB, AHB (one outstanding), simple FIFOs, UART
ID-based	Same ID (or same tag/stream) in order; different IDs may complete out of order	AXI (per ID and per direction), CHI TxnID, NoCs
Producer-consumer (PCIe)	Posted writes stay in order; reads and completions may not pass earlier posted writes; posted writes must be able to pass blocked reads	PCIe, CXL.io
Relaxed	Opt-in attributes allow more reordering (PCIe Relaxed Ordering, ID-Based Ordering)	PCIe RO/IDO for bulk DMA
Weak memory model	The CPU architecture defines ordering; the fabric only guarantees completion and barriers	Arm, RISC-V RVWMO with fences

Table 3.3 - Ordering models.

The **producer-consumer** pattern explains most rules. A device writes a data buffer and then writes a 'done' flag (both posted). The CPU reads the flag, then the data. If the flag could overtake the data, or the CPU's data read could overtake the device's earlier data writes, the CPU would read stale data. That is why PCIe forbids a read or completion from passing a posted write in the same direction (unless relaxed attributes are set), and why AXI masters that need ordering across IDs must wait for a write response before issuing the dependent transaction.

PITFALL - ORDERING BUGS IN BRIDGES

Protocol bridges (AXI to PCIe, AXI to AHB, NoC to AXI) are where ordering bugs live: a bridge that accepts a write, returns BRESP early ('early response') and then lets a later read to the same device overtake the buffered write breaks producer-consumer semantics. Document the ordering guarantee of every bridge and test same-address write-then-read through it.

3.6 Deadlock basics

A **deadlock** is a cycle of agents each waiting for a resource held by the next. Four conditions (Coffman) must all hold: mutual exclusion, hold-and-wait, no pre-emption, and circular wait. Protocol designers break the **circular wait**:

- **Separate message classes.** Requests and responses use different channels or virtual channels, so a response is never stuck behind a request that needs that response to make progress (AXI's separate AR/R/AW/W/B channels, CHI's REQ/RSP/SNP/DAT channels, PCIe's P/NP/Cpl credits).
- **Guaranteed sinking.** Responses must always be accepted (consumed without waiting on anything else) - an AXI master should only issue reads it can accept the data for.
- **Routing restrictions in NoCs** (dimension-order routing on a mesh) prevent cyclic channel dependencies.
- **No hidden dependencies** in slaves: an AXI slave must not wait for write data of a later transaction before completing an earlier one, and must not make the AW channel wait for W in a way the specification does not allow.

```
Master A --- req ---> [ shared queue ] ---> Slave B
      ^                               |
      +---- rsp <--- [ same queue, full ] <-----+
A waits for space to send a request; the queue is full of requests
that B cannot complete because B's responses are stuck behind them.
Fix: give responses their own queue (message class) that always drains.
```

Figure 3.4 - A request/response deadlock through a shared buffer.

3.7 Error detection: parity and checksums

Parity adds one bit so that the number of ones is even (even parity) or odd (odd parity). It detects every odd number of flipped bits and no even number. Per-byte parity is cheap and common on internal buses, SRAMs and register files, and UART frames can carry a parity bit.

Checksums add words arithmetically. The Internet checksum (RFC 1071) is the 16-bit ones'-complement sum of 16-bit words. It is cheap in software but weak: it cannot detect reordered words and misses many bursts. Hardware links therefore use CRCs.

3.8 CRC: the mathematics

A **cyclic redundancy check** treats the message as a polynomial over $GF(2)$ - bits are coefficients, addition is XOR, there are no carries. With a generator polynomial $G(x)$ of degree n , the sender computes

$$\text{CRC} = (M(x) \times x^n) \bmod G(x) \quad \text{transmitted } T(x) = M(x) \times x^n + \text{CRC}$$

$$T(x) \bmod G(x) = 0 \quad \rightarrow \quad \text{the receiver divides again and checks for a zero (or fixed) remainder}$$

Eq. 3.5 - CRC as polynomial division modulo 2.

The division is ordinary long division in which subtraction is XOR. The classic textbook example divides 11010011101100 by x^3+x+1 (1011):

```
# Long division of 11010011101100 by 1011 (x^3+x+1) over GF(2), printing each step
msg, gen = "11010011101100", "1011"
r = list(msg + "000") # append 3 zero bits (multiply by x^3)
print(msg + " 000  <- message x^3")
for i in range(len(msg)):
    if r[i] == "1":
        for j in range(4):
            r[i + j] = "0" if r[i + j] == gen[j] else "1"
        pad = " " * (len(msg) - i - 3)
        print(" " * i + gen + pad + "    XOR at bit %2d -> %s" % (i, "".join(r)))
print("remainder (CRC) = " + "".join(r[-3:]))
import zlib
m = b"123456789"; fcs = zlib.crc32(m)
print("residue: crc32(msg+FCS) = 0x%08X" % zlib.crc32(m + fcs.to_bytes(4, "little")))
```

Python that prints each XOR step of the long division (and, at the end, the CRC-32 residue used by receivers).

```
11010011101100 000  <- message x^3
1011                  XOR at bit  0 -> 01100011101100000
 1011                  XOR at bit  1 -> 00111011101100000
   1011                XOR at bit  2 -> 00010111101100000
    1011               XOR at bit  3 -> 00000001101100000
     1011              XOR at bit  7 -> 00000000110100000
      1011             XOR at bit  8 -> 00000000011000000
       1011            XOR at bit  9 -> 00000000001110000
        1011           XOR at bit 10 -> 00000000000101000
         1011          XOR at bit 11 -> 00000000000000100
remainder (CRC) = 100
residue: crc32(msg+FCS) = 0x2144DF1C
```

Real Python output: the 3-bit CRC is 100.

An error pattern $E(x)$ goes undetected only if $G(x)$ divides $E(x)$. Choosing G well gives strong guarantees:

- all **single-bit** errors (G has at least two terms);
- all **odd numbers** of bit errors if $(x+1)$ divides G ;
- all **burst** errors of length $\leq n$, and all but a fraction $2^{-(n-1)}$ of bursts of length $n+1$ and 2^{-n} of longer bursts;
- all **double-bit** errors within the polynomial's period;

- a guaranteed **Hamming distance** that depends on message length - for CRC-32 (0x04C11DB7) Koopman's published tables give HD = 4 for messages up to about 91 kbit and HD = 5 up to about 3 kbit. Polynomial choice for a new protocol should come from such tables, not from intuition.

The parameter model

Two implementations of 'CRC-16' routinely disagree because a CRC is defined by more than its polynomial. The Rocksoft/Williams parameter model (used by the RevEng catalogue) pins it down with: **width**, **poly** (normal MSB-first form, top bit implicit), **init** (register preset), **refin** (process each byte LSB first), **refout** (bit-reverse the final register), **xorout** (value XORED at the end) and **check** (the CRC of the ASCII string "123456789").

Name	Width	Poly	Init	Refin/Out	XorOut	Check	Used by
CRC-5/USB	5	0x05	0x1F	yes/yes	0x1F	0x19	USB token packets
CRC-8/SMBUS	8	0x07	0x00	no/no	0x00	0xF4	SMBus PEC, many sensors
CRC-8/AUTOSAR	8	0x2F	0xFF	no/no	0xFF	0xDF	Automotive E2E protection
CRC-15/CAN	15	0x4599	0x0000	no/no	0x0000	0x059E	Classical CAN frames
CRC-16/IBM-3740 (CCITT-FALSE)	16	0x1021	0xFFFF	no/no	0x0000	0x29B1	Many embedded links
CRC-16/KERMIT	16	0x1021	0x0000	yes/yes	0x0000	0x2189	Kermit, Bluetooth-style variants
CRC-16/USB	16	0x8005	0xFFFF	yes/yes	0xFFFF	0xB4C8	USB data packets
CRC-32/ISO-HDLC	32	0x04C11DB7	0xFFFFFFFF	yes/yes	0xFFFFFFFF	0xCBF43926	Ethernet FCS, PCIe LCRC/ECRC, zlib, SATA
CRC-32C (Castagnoli)	32	0x1EDC6F41	0xFFFFFFFF	yes/yes	0xFFFFFFFF	0xE3069283	iSCSI, SCTP, storage file systems

Table 3.4 - Common CRCs. Check values computed by the Python reference below; they match the RevEng catalogue.

```
import zlib
def crc(data, w, poly, init, refin, refout, xorout):
    """Bit-serial Rocksoft-model CRC: the same algorithm as the serial RTL."""
    c, top, mask = init, 1 << (w - 1), (1 << w) - 1
    for b in data:
        for i in (range(8) if refin else range(7, -1, -1)):
            fb = ((c & top) != 0) ^ ((b >> i) & 1)
            c = ((c << 1) & mask) ^ (poly if fb else 0)
    if refout:
        c = int(format(c, "0%db" % w)[::-1], 2)
    return c ^ xorout
CAT = [ # name, width, poly, init, refin, refout, xorout
    ("CRC-5/USB", 5, 0x05, 0x1F, 1, 1, 0x1F),
    ("CRC-8/SMBUS", 8, 0x07, 0x00, 0, 0, 0x00),
    ("CRC-8/AUTOSAR", 8, 0x2F, 0xFF, 0, 0, 0xFF),
    ("CRC-15/CAN", 15, 0x4599, 0x0000, 0, 0, 0x0000),
    ("CRC-16/IBM-3740", 16, 0x1021, 0xFFFF, 0, 0, 0x0000),
    ("CRC-16/KERMIT", 16, 0x1021, 0x0000, 1, 1, 0x0000),
    ("CRC-16/USB", 16, 0x8005, 0xFFFF, 1, 1, 0xFFFF),
    ("CRC-32/ISO-HDLC", 32, 0x04C11DB7, 0xFFFFFFFF, 1, 1, 0xFFFFFFFF),
    ("CRC-32C", 32, 0x1EDC6F41, 0xFFFFFFFF, 1, 1, 0xFFFFFFFF),
]
if __name__ == "__main__":
    for n, *a in CAT:
```

```

v = crc(b"123456789", *a)
print("%-16s check('123456789') = 0x%0*X" % (n, (a[0] + 3) // 4, v))
for s in (b"123456789", b"The quick brown fox jumps over the lazy dog"):
    print("zlib.crc32(%r) = 0x%08X" % (s.decode(), zlib.crc32(s)))

```

Generic bit-serial CRC in Python (the same algorithm as the RTL), producing Table 3.4's check values, plus zlib.crc32 on two strings.

```

CRC-5/USB      check('123456789') = 0x19
CRC-8/SMBUS    check('123456789') = 0xF4
CRC-8/AUTOSAR  check('123456789') = 0xDF
CRC-15/CAN     check('123456789') = 0x059E
CRC-16/IBM-3740 check('123456789') = 0x29B1
CRC-16/KERMIT  check('123456789') = 0x2189
CRC-16/USB     check('123456789') = 0xB4C8
CRC-32/ISO-HDLC check('123456789') = 0xCBF43926
CRC-32C        check('123456789') = 0xE3069283
zlib.crc32('123456789') = 0xCBF43926
zlib.crc32('The quick brown fox jumps over the lazy dog') = 0x414FA339

```

Real Python output.

CRC-16-CCITT IS A FAMILY, NOT A CRC

The polynomial $x^{16}+x^{12}+x^5+1$ (0x1021) appears in HDLC/X.25, Bluetooth, SD cards and MIPI CSI-2 packet footers - with different init values, reflection and final XOR. Always implement from the full parameter set in the specification, and test with its published example packet.

Serial LFSR implementation

In hardware the division is a **Galois LFSR**: shift the register left one bit per message bit; if the bit shifted out XOR the incoming message bit is 1, XOR the polynomial into the register.

CRC-8 example, $G = x^8 + x^2 + x + 1$ (0x07), MSB-first:

```

+-----+-----+
|               v               v
+--> [c0] --> (XOR) --> [c1] --> (XOR) --> [c2] --> [c3] -> ... -> [c7] --+
^
+-----+-----+ fb ----- (XOR) <-----+
                        ^
                        din
next: c0 = fb, c1 = c0 ^ fb, c2 = c1 ^ fb, c3..c6 = c2..c6, fb = c7 ^ din

```

Figure 3.5 - Galois LFSR for CRC-8: an XOR gate before every register whose power appears in $G(x)$.

```

// Generic bit-serial CRC: a Galois LFSR, one message bit per clock (MSB-first form).
// Reflection (LSB-first bytes) is handled by the order the caller feeds bits in.
module crc_serial #(parameter int W = 32,
                    parameter logic [W-1:0] POLY = 32'h04C1_1DB7,
                    parameter logic [W-1:0] INIT = '1) (
    input  logic      clk, rst_n,
    input  logic      clear,      // load INIT at the start of a message
    input  logic      en,         // one message bit this clock
    input  logic      din,
    output logic [W-1:0] crc
);
    wire fb = crc[W-1] ^ din;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n)    crc <= INIT;
        else if (clear) crc <= INIT;
        else if (en)    crc <= {crc[W-2:0], 1'b0} ^ (fb ? POLY : '0);
endmodule

// CRC-32 (IEEE 802.3 / zlib), 8 bits per clock. The loop is unrolled by synthesis

```

```
// into one XOR tree (up to 14 inputs per bit, see Table 3.x). Bytes go LSB first.
module crc32_d8 (
    input logic      clk, rst_n, clear, en,
    input logic [7:0] data,
    output logic [31:0] crc_reg,      // raw LFSR state
    output logic [31:0] fcs           // final value: bit-reversed and inverted
);
function automatic logic [31:0] step8(input logic [31:0] c, input logic [7:0] d);
    for (int i = 0; i < 8; i++)
        c = {c[30:0], 1'b0} ^ ((c[31] ^ d[i]) ? 32'h04C1_1DB7 : 32'h0);
    return c;
endfunction
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n)    crc_reg <= '1;
    else if (clear) crc_reg <= '1;
    else if (en)    crc_reg <= step8(crc_reg, data);
always_comb
    for (int i = 0; i < 32; i++) fcs[i] = ~crc_reg[31 - i];
endmodule
```

A parameterized bit-serial CRC and a byte-parallel CRC-32 (Ethernet/zlib). Reflection is handled by feeding each byte LSB first; the final value is bit-reversed and inverted.

```
module tb;
    logic clk = 0, rst_n = 0, clear = 0, en = 0, en8 = 0; logic [7:0] byte_in;
    always #5 clk = ~clk;
    logic [31:0] c32, par_reg, par_fcs; logic [15:0] c16; logic [14:0] c15;
    logic [4:0] c5, c5_out;
    // one serial LFSR per standard, all fed the same bits (in each standard's bit order)
    logic b_lsb, b_msb;
    crc_serial #(32, 32'h04C1_1DB7, 32'hFFFFFFFF)
        u32 (.clk, .rst_n, .clear, .en, .din(b_lsb), .crc(c32)); // reflected
    crc_serial #(16, 16'h1021, 16'hFFFF)
        u16 (.clk, .rst_n, .clear, .en, .din(b_msb), .crc(c16)); // MSB first
    crc_serial #(15, 15'h4599, 15'h0000)
        u15 (.clk, .rst_n, .clear, .en, .din(b_msb), .crc(c15));
    crc_serial #(5, 5'h05, 5'h1F)
        u05 (.clk, .rst_n, .clear, .en, .din(b_lsb), .crc(c5));
    crc32_d8 upar (.clk, .rst_n, .clear, .en(en8), .data(byte_in), .crc_reg(par_reg),
        .fcs(par_fcs));

    function automatic logic [31:0] rev(input logic [31:0] v, input int w);
        rev = '0; for (int i = 0; i < w; i++) rev[i] = v[w-1-i];
    endfunction

    task automatic run(input string s, input logic [31:0] zlib_ref);
        @(negedge clk) clear = 1; @(negedge clk) clear = 0;
        for (int n = 0; n < s.len(); n++) begin
            byte_in = s[n]; en8 = 1; // parallel: 1 clock per byte
            for (int i = 0; i < 8; i++) begin // serial: 8 clocks per byte
                b_lsb = byte_in[i]; b_msb = byte_in[7-i]; en = 1;
                @(negedge clk) en8 = 0;
            end
            en = 0;
            $display("\n%s\\", s);
            $display("  CRC-32 serial   = %h   byte-parallel = %h   zlib.crc32 = %h %s",
                ~rev(c32, 32), par_fcs, zlib_ref,
                (~rev(c32, 32) == zlib_ref && par_fcs == zlib_ref) ? "MATCH" : "MISMATCH");
            c5_out = rev(c5, 5) ^ 5'h1F; // CRC-5/USB: refout, xorout=1F
            if (s == "123456789")
                $display("  CRC-16/IBM-3740 = %h   CRC-15/CAN = %h   CRC-5/USB = %h",
```



```

        c16, c15, c5_out);
    endtask
    initial begin
        #12 rst_n = 1;
        run("123456789", 32'hCBF43926);
        run("The quick brown fox jumps over the lazy dog", 32'h414FA339);
        $finish;
    end
endmodule

```

Testbench: four serial CRCs (CRC-32, CRC-16/IBM-3740, CRC-15/CAN, CRC-5/USB) and the byte-parallel CRC-32 run over the same strings; expected values come from Python's `zlib.crc32`.

```

"123456789"
CRC-32 serial    = cbf43926    byte-parallel = cbf43926    zlib.crc32 = cbf43926    MATCH
CRC-16/IBM-3740 = 29b1        CRC-15/CAN   = 059e        CRC-5/USB   = 19
"The quick brown fox jumps over the lazy dog"
CRC-32 serial    = 414fa339    byte-parallel = 414fa339    zlib.crc32 = 414fa339    MATCH

```

Real Icarus Verilog output: serial and byte-parallel RTL both equal `zlib.crc32` on '123456789' and on the pangram; the other CRCs match Table 3.4.

A useful receiver trick follows from Eq. 3.5: running the CRC over the message **and its transmitted FCS** gives a constant, the **residue**, whatever the message. For the Ethernet CRC-32 the final (inverted) value is 0x2144DF1C, as the last line of the long-division script shows; the raw register holds 0xC704DD7B (0xDEBB20E3 in reflected form). A MAC can therefore check the FCS on the fly without knowing where the frame ends until it ends.

Parallel CRC derivation

At 10 Gb/s and above a bit-serial CRC cannot keep up: the data path is 32 to 512 bits wide per clock. Because the CRC is **linear** over GF(2), the next state after D input bits is a fixed XOR function of the current state and the D data bits: $\text{crc}' = A \times \text{crc} \text{ XOR } B \times d$. The matrices can be derived by unrolling the serial loop (as `step8` in `crc32_d8` does - synthesis flattens it) or explicitly by superposition: feed a single 1 in each state or data position and record which output bits toggle.

```

# Derive the byte-parallel XOR equations of CRC-8 (poly 0x07, MSB first) by superposition:
# next_crc is linear in (crc, data), so feed each single '1' input bit and record which
# output bits it toggles.
def step(c, d, w=8, poly=0x07):
    for i in range(7, -1, -1):
        fb = ((c >> (w - 1)) & 1) ^ ((d >> i) & 1)
        c = ((c << 1) & ((1 << w) - 1)) ^ (poly if fb else 0)
    return c
terms = {o: [] for o in range(8)}
for j in range(8):
    for name, c, d in (("c", 1 << j, 0), ("d", 0, 1 << j)):
        r = step(c, d)
        for o in range(8):
            if (r >> o) & 1:
                terms[o].append((name, j))
for o in range(7, -1, -1):
    # c[j] and d[j] always appear together (x = c ^ d)
    xs = sorted({j for n, j in terms[o]}, reverse=True)
    assert all(("c", j) in terms[o] and ("d", j) in terms[o] for j in xs)
    print("crc'[%d] = %s" % (o, " ^ ".join("x[%d]" % j for j in xs)))

```

Deriving the byte-parallel CRC-8 (0x07) equations by superposition.


```

crc'[7] = x[7] ^ x[6] ^ x[5]
crc'[6] = x[6] ^ x[5] ^ x[4]
crc'[5] = x[5] ^ x[4] ^ x[3]
crc'[4] = x[4] ^ x[3] ^ x[2]
crc'[3] = x[7] ^ x[3] ^ x[2] ^ x[1]
crc'[2] = x[6] ^ x[2] ^ x[1] ^ x[0]
crc'[1] = x[6] ^ x[1] ^ x[0]
crc'[0] = x[7] ^ x[6] ^ x[0]

```

Real Python output ($x = \text{crc XOR data}$, bitwise; this simplification holds when the data width equals the CRC width).

The cost grows with the data width. Running the same superposition on CRC-32 gives the XOR fan-in per output bit:

```

D= 1 data bits/clock: XOR inputs per CRC bit  max= 3  avg= 1.8  total= 59
D= 8 data bits/clock: XOR inputs per CRC bit  max= 14  avg= 7.9  total= 252
D= 32 data bits/clock: XOR inputs per CRC bit  max= 34  avg= 28.2  total= 904
D= 64 data bits/clock: XOR inputs per CRC bit  max= 52  avg= 44.4  total=1422
D=128 data bits/clock: XOR inputs per CRC bit  max= 89  avg= 79.7  total=2550

```

Real Python output (par_cost.py): XOR inputs per CRC-32 bit for D data bits per clock.

At 128 bits per clock each CRC bit is an XOR of up to 89 inputs - about seven levels of 2-input XOR, which is why wide MACs pipeline the CRC or split it into per-lane partial CRCs combined later. Real links also must handle a **last word that is only partly valid** (the byte-enable problem): typical solutions compute the CRC for each possible valid-byte count in parallel and select, or process the tail in a narrower final stage.

3.9 ECC: Hamming codes and SECDED

Detection plus retry works for links; for **memories**, where there is nothing to retry from, errors must be **corrected**. The Hamming distance d of a code determines its power: it can detect $d-1$ errors or correct $\text{floor}((d-1)/2)$. A **Hamming code** ($d = 3$) places parity bits at positions 1, 2, 4, 8, ... of the codeword; parity bit 2^p covers every position whose index has bit p set. On reception, the recomputed parities - the **syndrome** - spell out the index of a single flipped bit directly. Adding one overall parity bit raises the distance to 4: **SECDED** (single-error correct, double-error detect).

Overall parity	Syndrome	Diagnosis	Action
0 (even)	0	No error	Pass data
1 (odd)	$\neq 0$, points inside the word	Single-bit error at position = syndrome	Flip that bit, report corrected error (CE)
1 (odd)	0	Error in the overall parity bit itself	Data is fine, report CE
0 (even)	$\neq 0$	Double-bit error	Report uncorrectable error (UE); do not use data
1 (odd)	points outside the word	≥ 3 errors	Report UE

Table 3.5 - SECDED decode table.

For 64 data bits, 7 Hamming check bits cover positions 1-71 and one overall parity bit makes the classic **(72,64)** code used on ECC DIMMs and in SRAM and cache ECC. Production designs usually use **Hsiao** codes - the same (72,64) size, but with odd-weight check-matrix columns chosen to minimize XOR depth and give faster double-error detection. DDR5 adds an internal **on-die ECC** (a SEC code over 128 data bits) in addition to any system-level ECC. The plain Hamming construction below is the easiest to understand.

```

// Hamming SECDED (72,64). Codeword bit k (k = 1..71) is Hamming position k; positions
// 1,2,4,...,64 hold the 7 check bits, the other 64 hold data. Bit 0 is overall parity.
module secDED72_enc (
    input  logic [63:0] d,
    output logic [71:0] cw
);
always @* begin

```

```

    int j; j = 0; cw = '0;
    for (int k = 1; k < 72; k++) // scatter data bits
        if ((k & (k - 1)) != 0) begin cw[k] = d[j]; j++; end
    for (int p = 0; p < 7; p++) // check bit at 2^p covers
        for (int k = 1; k < 72; k++) // every position with bit p set
            if (((k >> p) & 1) && (k != (1 << p))) cw[1 << p] ^= cw[k];
    cw[0] = ^cw[71:1]; // overall (DED) parity
end
endmodule

module secDED72_dec (
    input logic [71:0] cw_in,
    output logic [63:0] d,
    output logic [6:0] syndrome,
    output logic ce, // corrected single-bit error
    output logic ue // uncorrectable (double) error detected
);
    logic [71:0] cw; logic ov;
    always @* begin
        int j;
        syndrome = '0;
        for (int k = 1; k < 72; k++) if (cw_in[k]) syndrome ^= k[6:0]; // XOR of set positions
        ov = ^cw_in; // 1 = odd # of flips
        cw = cw_in; ce = 1'b0; ue = 1'b0;
        if (ov) begin // odd: assume one error, correct it
            if (syndrome < 72) begin cw[syndrome] = ~cw[syndrome]; ce = 1'b1; end
            else ue = 1'b1; // points outside the word: >= 3 flips
        end else if (syndrome != 0) ue = 1'b1; // even and nonzero: double error
        j = 0; d = '0;
        for (int k = 1; k < 72; k++)
            if ((k & (k - 1)) != 0) begin d[j] = cw[k]; j++; end
    end
endmodule

```

SECDED (72,64) encoder and decoder: Hamming positions 1-71, check bits at powers of two, overall parity in bit 0.

```

module tb;
    logic [63:0] d, dq; logic [71:0] cw, rx; logic [6:0] syn; logic ce, ue;
    secDED72_enc enc (.d, .cw);
    secDED72_dec dec (.cw_in(rx), .d(dq), .syndrome(syn), .ce, .ue);
    int n0 = 0, bad0 = 0, n1 = 0, fix1 = 0, n2 = 0, det2 = 0, miss2 = 0;
    initial begin
        d = 64'h0123_4567_89AB_CDEF; rx = cw; #1; // show one example
        $display("data=%h check bits (pos 64,32,16,8,4,2,1,0) = %b%b%b%b%b%b%b", d,
            cw[64], cw[32], cw[16], cw[8], cw[4], cw[2], cw[1], cw[0]);
        rx = cw ^ (72'd1 << 37); #1;
        $display("flip bit 37 -> syndrome=%0d ce=%b ue=%b data %s", syn, ce, ue,
            dq == d ? "restored" : "WRONG");
        rx = cw ^ (72'd1 << 37) ^ (72'd1 << 5); #1;
        $display("flip bits 37,5 -> syndrome=%0d ce=%b ue=%b (detected only)", syn, ce, ue);
        for (int t = 0; t < 2000; t++) begin
            d = {$urandom, $urandom}; #1;
            rx = cw; #1; n0++; if (ce || ue || dq != d) bad0++;
            for (int b = 0; b < 72; b++) begin // every single-bit error
                rx = cw ^ (72'd1 << b); #1; n1++;
                if (ce && !ue && dq == d) fix1++;
            end
            for (int e = 0; e < 20; e++) begin // random double-bit errors
                int a, b; a = $urandom % 72; do b = $urandom % 72; while (b == a);
                rx = cw ^ (72'd1 << a) ^ (72'd1 << b); #1; n2++;
                if (ue && !ce) det2++; else miss2++;
            end
        end
    end
end

```

```

end
$display("no error      : %0d words,  false flags/corruption=%0d", n0, bad0);
$display("single error: %0d cases,  corrected=%0d", n1, fix1);
$display("double error: %0d cases,  detected (ue=1)=%0d, missed=%0d", n2, det2, miss2);
$finish;
end
endmodule

```

Testbench: a worked example, then 2 000 random words with every single-bit error and 20 random double-bit errors each.

```

data=0123456789abcdef  check bits (pos 64,32,16,8,4,2,1,0) = 00111001
flip bit 37 -> syndrome=37 ce=1 ue=0 data restored
flip bits 37,5 -> syndrome=32 ce=0 ue=1 (detected only)
no error      : 2000 words,  false flags/corruption=0
single error: 144000 cases,  corrected=144000
double error: 40000 cases,  detected (ue=1)=40000, missed=0

```

Real Icarus Verilog output: all 144 000 single-bit errors corrected, all 40 000 double-bit errors detected, no false flags.

```
Python: check bits (pos 64,32,16,8,4,2,1,0) = 00111001
```

Real output of ref_secdec.py, an independent Python encoder: the same check bits for 0x0123456789ABCDEF.

PITFALL - ECC INTEGRATION BUGS

- **Partial writes.** Writing one byte of an ECC-protected 64-bit word requires read-modify-write; forgetting it corrupts check bits and turns every later read into a false error.
- **Uninitialized memory** has random check bits; reading it before initialization floods the system with UEs. Initialize (scrub) at boot.
- **Silent correction.** Log and count corrected errors - a rising CE rate is the only warning before a UE - and write the corrected value back (scrubbing) so single errors do not accumulate into doubles.
- **Address errors.** Plain ECC over data does not detect a read from the wrong address; fold address bits into the code if that matters (common in safety designs, Chapter 26).

3.10 Summary

- Valid/ready transfers on every edge where both are high; the source must not wait for ready and must hold valid and data until the transfer.
- 4-phase and 2-phase req/ack handshakes move data between unrelated clocks; 2-phase needs half the events.
- Back-pressure, start/stop and credits are the three flow-control families; credits make overflow impossible but must cover the credit round trip (throughput = credits/RTT, as simulated).
- Go-back-N retry with sequence numbers, ACK/NAK and a replay buffer (PCIe DLL) turns rare bit errors into invisible retries.
- Ordering rules (in-order, ID-based, producer-consumer, relaxed) and separate message classes keep software correct and systems deadlock-free.
- A CRC is polynomial division over GF(2), fully specified only by width, poly, init, refin, refout and xorout; serial and byte-parallel RTL matched zlib.crc32.
- Parallel CRCs follow from linearity; XOR depth grows with data width.
- Hamming SECDED (72,64) corrects all single and detects all double errors, as the exhaustive single-error and random double-error simulation showed.

3.11 Exercises

- A PCIe-like link has 400 ns of credit round-trip time, 16 GB/s of payload bandwidth and 256-byte payloads. How many data credits (16 bytes each) and header credits are needed for full throughput?
- Extend credit_link so that credits are returned in batches of four (like coalesced UpdateFC DLLPs). Predict and then measure the throughput for 8 and 12 credits.
- Compute by hand the CRC-8/SMBUS of the two bytes 0x01 0x02 using the parallel equations of the output card, and check with ref_crc.py.

- Modify `crc32_d8` to accept 32 bits per clock with a 2-bit count of valid bytes in the last word. Verify against `zlib.crc32` for message lengths 1 to 64.
- Show that the (72,64) code in `secded72_dec` miscorrects some triple errors. Write a testbench that measures the fraction of triple errors reported as CE, UE and silently wrong.
- Draw the go-back-N sequence of Figure 3.3 for the case where the NAK itself is lost. Which timer recovers the link, and how many TLPs are replayed?

4. Implementing and Verifying Protocols in RTL

Chapters 2 and 3 described what happens on the wire. This chapter is about building it: the architecture shared by almost every protocol controller, and the recurring RTL building blocks - baud and bit timing generators, shift registers, oversampling receivers with majority voting, synchronizers, tri-state and open-drain pad interfaces, timeouts - followed by the verification side: protocol checkers, SystemVerilog assertions, bus functional models, VIP and compliance. The run example is a generic, parameterized framed serializer/deserializer pair (start bit, data, optional parity, stop bit) running on two unrelated clocks, with an independent protocol checker that catches injected errors, and SVA properties run in Verilator.

4.1 Generic controller architecture

Open the block diagram of a UART, SPI, I2C, CAN or even USB controller and you find the same five layers. Learning the pattern once makes every new controller familiar:

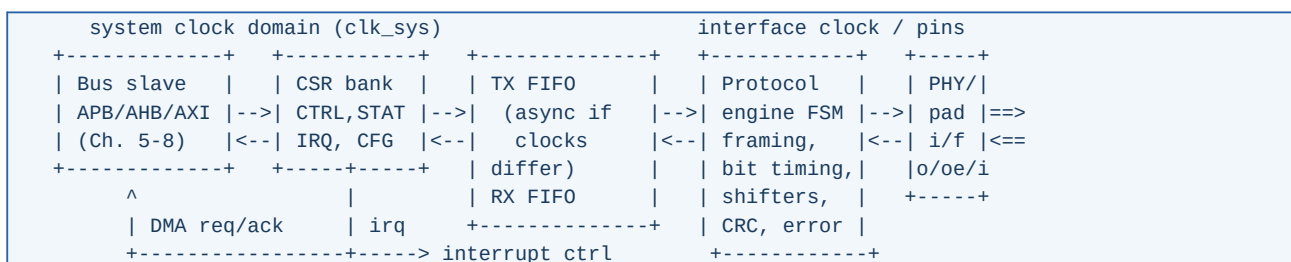


Figure 4.1 - The universal peripheral controller: register interface, CSRs, FIFOs, protocol engine, pad interface.

Block	Responsibility	Design notes
Bus slave / register interface	Decode APB/AHB/AXI accesses into register reads and writes	Usually generated from a register description (IP-XACT, SystemRDL); must respond to every access, including unmapped addresses
CSR bank	Control, status, interrupt enable/status, configuration (baud divisor, mode, polarity)	Clear rules for W1C status, sticky errors, read side effects (reading a data register pops the FIFO)
FIFOs	Decouple bursty software/DMA from the fixed-rate wire; cross clock domains	Depth from latency tolerance (interrupt latency x line rate); thresholds for interrupts/DMA
Protocol engine	The FSM that sequences frames: start, address, data, acknowledge, stop, turnaround, retries	Where the specification's state diagrams live; keep it separate from bit timing
Bit-level datapath	Shift registers, bit counters, baud/prescaler, oversampling, CRC, parity, stuffing	Parameterize widths; reuse across protocols
Pad interface	o/oe/i triplets, synchronizers on inputs, glitch filters	Never tri-state inside the core; keep inputs asynchronous until synchronized

Table 4.1 - Controller blocks and their responsibilities.

SPLIT CONTROL FROM TIMING

A robust controller separates **what** to send (the protocol FSM, which advances once per bit or per symbol) from **when** (a timing generator that emits single-cycle ticks). The FSM then runs on the system clock with a clock-enable, which keeps the whole block in one clock domain, simplifies STA, and lets one design serve many baud rates.

4.2 Bit timing generation: prescalers and fractional baud

Low-speed serial protocols run at rates far below the system clock, so the controller derives a **tick** - a one-cycle enable - at the bit rate or at a multiple of it (16x oversampling is traditional for UARTs). Generating a divided **clock** instead would create a new clock domain, a clock-tree to balance and CDC paths; a tick keeps everything synchronous.

Method	Rate	Error	Notes
Integer prescaler	f_{clk} / N	Up to half a count: for $f_{\text{clk}} = 50 \text{ MHz}$ and $16 \times 115200 \text{ Hz}$, $N = 27$ gives 1.852 MHz , $+0.47\%$	Simple; error grows as N gets small
Integer + fractional (UART DLL/DLM + DLF)	$f_{\text{clk}} / (N + F/16)$	Much smaller	Common in 16550-compatible UARTs
Phase accumulator	$f_{\text{clk}} \times \text{INC} / 2^K$	At most $f_{\text{clk}} / 2^K$ resolution; tick-to-tick jitter of one clk	Any rate; used below with $K = 16$

Table 4.2 - Bit-timing generators.

$$\begin{aligned} \text{INC} &= \text{round}(f_{\text{tick}} \times 2^{16} / f_{\text{clk}}) = \text{round}(1.8432 \text{ MHz} \times 65536 / 50 \text{ MHz}) = 2416 \\ f_{\text{tick}} &= 50 \text{ MHz} \times 2416 / 65536 = 1.843262 \text{ MHz} \rightarrow \text{error} = +34 \text{ ppm} \\ \text{RX at } 48 \text{ MHz: INC} &= 2517 \rightarrow 1.843506 \text{ MHz} \rightarrow +166 \text{ ppm} \end{aligned}$$

Eq. 4.1 - Phase-accumulator baud generation for 115200 baud x 16.

The accumulator's tick spacing alternates between 27 and 28 clocks so its average is exact to a few ppm; the resulting cycle-to-cycle jitter of one system clock (20 ns, under 0.25% of a bit at 115200 baud) is harmless.

4.3 Shift registers, serializers and oversampling receivers

The transmit datapath is a **parallel-in, serial-out** shift register: load a whole frame (start bit, data, parity, stop), then shift one bit per bit-time. LSB-first (UART, most SerDes PCS lanes) versus MSB-first (SPI usually, I2C, CAN) is just the shift direction, but mixing conventions between the RTL and the software driver is a classic bring-up bug.

The receiver is harder because it must find the bits in time. An asynchronous receiver samples the line at OSR times the bit rate (OSR = 16 here):

- **Start detection.** In idle, a low sample may be a start bit. Wait half a bit and check again: if the line is high again it was a glitch and the receiver returns to idle.
- **Centre sampling.** From the start-bit centre, sample every OSR ticks - each sample lands in the centre of a data bit.
- **Majority vote.** Take three samples around the centre (ticks 7, 8, 9 of 16) and vote, which suppresses short noise spikes.
- **Framing check.** The stop bit must be 1; otherwise flag a **framing error** (and a line held low for a whole frame is a **break** condition).
- **Resynchronize every frame.** Return to idle in the middle of the stop bit so the next start edge re-aligns the timing.

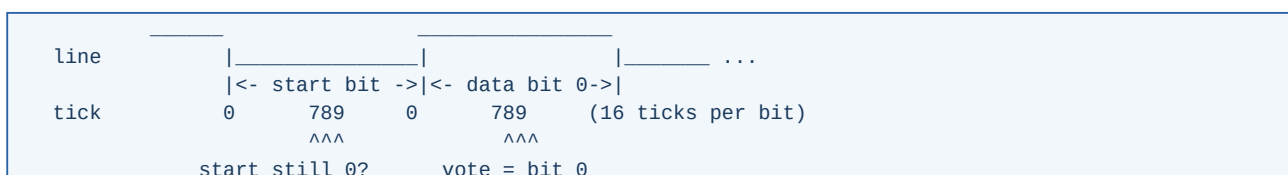


Figure 4.2 - 16x oversampling: start-bit validation and three-sample majority vote at the bit centre.

How much clock mismatch can such a receiver take? The last sample (stop-bit centre) is 9.5 bit times after the start edge for 8N1 (1 start + 8 data + half the stop bit), and the start edge itself is only known to within one tick

(1/16 bit). The accumulated timing error must stay within half a bit:

$$9.5 \times |\Delta f / f| + 1/16 < 0.5 \quad \rightarrow \quad |\Delta f / f| < 4.6 \% \text{ (total, TX + RX)}$$

with parity (8E1: 10.5 bits): < 4.2 %

in practice keep each side under ~1-2 % to leave margin for edge distortion

Eq. 4.2 - Clock tolerance of an oversampling asynchronous receiver.

4.4 Synchronizers and clock-domain crossing

A protocol controller almost always has at least two timing domains: the **system clock** of the bus interface and FIFOs, and the **interface timing** - either a derived tick (UART, I2C master), an external clock (SPI slave's SCLK, I2S BCLK, JTAG TCK), or a PHY's recovered clock (USB, PCIe, Ethernet). The standard CDC toolkit (covered in depth in the companion RTL Design guide) applies:

What crosses	Technique	Protocol example
A level-sensitive asynchronous input (RX line, SDA, CTS)	2-flop (or 3-flop) synchronizer before any logic uses it	UART RX (sync in ser_rx below), I2C SDA/SCL, GPIO
A single-cycle event	Toggle + synchronizer + edge detect (pulse synchronizer)	'frame done' from an SPI SCLK domain to clk_sys
A multi-bit value that changes rarely	Req/ack handshake (Section 3.2), data held stable	Configuration registers into a PHY clock domain
A stream of data	Asynchronous FIFO with Gray-coded pointers	USB/Ethernet PHY clock to system clock; SPI slave data
An externally supplied clock with its data	Capture in the external clock domain, then an async FIFO or handshake	SPI slave, I2S, JTAG TAP

Table 4.3 - CDC techniques in protocol controllers.

PITFALL - SAMPLING A SLOW BUS WITH A FAST CLOCK

- Oversampling an external clock such as SCL or SCLK with clk_sys (instead of using it as a clock) works only when clk_sys is several times faster than the external clock and the input is synchronized and glitch-filtered first. Without the filter, ringing on a slow edge produces double clock pulses - a classic I2C bug.
- The 2-flop synchronizer adds 2-3 cycles of latency. Protocols with tight turnaround (an I2C ACK, an SPI slave's first MISO bit) must budget for it.
- Never synchronize the bits of a multi-bit bus independently; they can resolve on different cycles.

4.5 Tri-state and open-drain modelling in RTL

Inside a synthesizable core there are **no tri-state buses**: every bidirectional pin is represented by three separate signals, and the actual tri-state buffer lives in the I/O pad cell instantiated at the chip top level:

Signal	Direction (core view)	Meaning
x_o	output	Value to drive when enabled
x_oe (or x_en)	output	1 = drive the pin, 0 = release (high impedance)
x_i	input	Value seen at the pin (asynchronous - synchronize before use)

An **open-drain** output is the special case [x_o](#) = 0 and [x_oe](#) = [pull_low](#): the core never drives a '1'. The pad model below and the [tri1](#) net (a wire with a pull-up) reproduce the wired-AND behaviour of an I2C line in simulation.

```
// Open-drain pad as seen from RTL: the core never drives '1'. It drives oe=1 to pull
// the pin low and oe=0 to release it; the board pull-up makes the released level '1'.
module od_pad (
```

```

    input  logic pull_low,          // from the controller: 1 = drive the pin low
    output logic in,                // back to the controller (through a synchronizer)
    inout  wire pin
);
    assign pin = pull_low ? 1'b0 : 1'bz;    // oe = pull_low, o = 0
    assign in  = pin;
endmodule

module tb;
    tri1 bus;                      // tri1 = net with a pull-up (resistor model)
    logic a = 0, b = 0; logic ia, ib;
    od_pad pa (.pull_low(a), .in(ia), .pin(bus));
    od_pad pb (.pull_low(b), .in(ib), .pin(bus));
    initial begin
        $display("pull_low a b | bus (wired-AND)");
        for (int i = 0; i < 4; i++) begin
            {a, b} = i[1:0]; #1 $display("      %0d %0d |  %b", a, b, bus);
        end
    end
endmodule

```

Open-drain pad model and a two-device wired-AND bus.

```

pull_low a b | bus (wired-AND)
    0 0 | 1
    0 1 | 0
    1 0 | 0
    1 1 | 0

```

Real Icarus output: the bus is high only when nobody pulls low.

WHERE THE TRI-STATE REALLY GOES

In the SoC top level the o/oe/i triplet connects to a pad cell (or to a pin-mux that routes it to one of several pads). Keeping tri-states out of the core lets synthesis, scan insertion, equivalence checking and emulation work normally; many FPGA and emulation flows cannot handle internal tri-states at all.

4.6 Timeouts and error recovery

Real links misbehave: devices are unplugged mid-transfer, a slave holds a line low after a brown-out, noise corrupts a frame. A controller that waits forever for an event that never comes is a hang that no software can fix. Every protocol engine therefore needs explicit recovery paths:

Mechanism	Example
Watchdog / timeout counters on every wait state	SMBus: a device holding SCL low longer than 25-35 ms must be treated as timed out; PCIe completion timeouts; AXI slave response timeouts in safety SoCs
Bus recovery sequences	I2C bus clear: if SDA is stuck low, the controller clocks SCL up to nine times until the slave releases SDA, then issues STOP
Error counters and states	CAN's transmit/receive error counters with error-active, error-passive and bus-off states (Chapter 15)
Retry with limits	PCIe REPLAY_NUM rollover triggers link retraining (Section 3.4)
Sticky status and interrupts	Framing, parity, overrun, NACK, arbitration-lost bits that stay set until software clears them (write-1-to-clear)
Soft reset / flush	A register bit that returns the protocol engine to idle and empties FIFOs without resetting the SoC

Table 4.4 - Recovery mechanisms every controller should consider.

PITFALL - ERROR PATHS ARE THE LEAST-TESTED PATHS

Directed tests exercise the happy path; the error recovery FSM transitions are often never simulated until silicon. Make every error condition injectable from the testbench (as the channel model below does), cover each recovery transition, and check that the controller returns to a clean idle afterwards.

4.7 Run example: a framed serializer/deserializer with a checker

The example ties the chapter together. `ser_tx` and `ser_rx` are parameterized by data width `DW`, oversampling ratio `OSR` and parity `PAR`; the frame is start (0), `DW` data bits LSB first, optional even parity, stop (1) - the UART frame of Chapter 11, but the same structure appears in LIN, DMX and many proprietary links. The transmitter runs at 50 MHz and the receiver at 48 MHz, each with its own phase-accumulator tick generator (Eq. 4.1), so the line is truly asynchronous to the receiver.

```
`timescale 1ns/1ps
// Fractional baud-tick generator: tick rate = f_clk * INC / 2^16 (phase accumulator).
module tick_gen #(parameter int INC = 2416) (
    input  logic clk, rst_n,
    output logic tick                // one-cycle pulse at OSR x baud
);
    logic [15:0] acc;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin acc <= '0; tick <= 1'b0; end
        else {tick, acc} <= {1'b0, acc} + INC;    // carry out = tick
endmodule

// Framed serializer: start(0) | DW data bits LSB first | optional even parity | stop(1)
module ser_tx #(parameter int DW = 8, OSR = 16, PAR = 1) (
    input  logic      clk, rst_n, tick,
    input  logic      in_valid,
    output logic      in_ready,
    input  logic [DW-1:0] in_data,
    output logic      line        // registered: no glitches on the pin
);
    localparam int NB = DW + PAR + 2;    // bits per frame
    logic [NB-1:0] sh, frame;
    logic [$clog2(NB+1)-1:0] nleft;    // bits still to send
    logic [$clog2(OSR)-1:0] ph;    // tick count inside the bit
    always_comb begin
        frame = '1; frame[0] = 1'b0; frame[DW:1] = in_data;
        if (PAR) frame[DW+1] = ^in_data;    // even parity
    end
    assign in_ready = (nleft == 0);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin sh <= '1; nleft <= '0; ph <= '0; line <= 1'b1; end
        else begin
            if (in_valid && in_ready) begin sh <= frame; nleft <= NB; ph <= '0; end
            else if (tick && nleft != 0) begin
                ph <= ph + 1'b1;
                if (ph == OSR - 1) begin sh <= {1'b1, sh[NB-1:1]}; nleft <= nleft - 1'b1; end
            end
            line <= (nleft != 0) ? sh[0] : 1'b1;    // idle high
        end
    end
endmodule

// Deserializer: 2-flop synchronizer, start-bit validation at mid-bit, 3-sample
// majority vote around each bit centre, framing and parity error flags.
module ser_rx #(parameter int DW = 8, OSR = 16, PAR = 1) (
    input  logic      clk, rst_n, tick,
    input  logic      line_async,
```

```

output logic      out_valid,
output logic [DW-1:0] out_data,
output logic      frame_err, par_err
);
localparam int NB = DW + PAR + 2;
logic [1:0] sync; wire ln = sync[1];
logic      busy;
logic [$clog2(OSR)-1:0] ph;
logic [$clog2(NB)-1:0] bitn;
logic [1:0] smp;           // first two of the three centre samples
logic [NB-1:0] fr;        // received frame, bit 0 = start
wire maj = (smp[1] & smp[0]) | (smp[1] & ln) | (smp[0] & ln);
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) begin
    sync <= 2'b11; busy <= 1'b0; ph <= '0; bitn <= '0; smp <= '0; fr <= '0;
    out_valid <= 1'b0; out_data <= '0; frame_err <= 1'b0; par_err <= 1'b0;
  end else begin
    sync <= {sync[0], line_async};
    out_valid <= 1'b0;
    if (tick) begin
      if (!busy) begin
        if (!ln) begin busy <= 1'b1; ph <= '0; bitn <= '0; end // possible start
      end else begin
        ph <= ph + 1'b1;
        if (ph == OSR/2 - 1 || ph == OSR/2) smp <= {smp[0], ln};
        if (ph == OSR/2 + 1) begin // third sample: decide
          fr[bitn] <= maj;
          if (bitn == 0 && maj) busy <= 1'b0; // glitch, not a start bit
          if (bitn == NB - 1) begin // stop bit centre: done
            busy <= 1'b0; // re-arm for next start
            out_valid <= 1'b1;
            out_data <= fr[DW:1];
            frame_err <= !maj;
            par_err <= PAR ? (^fr[DW+PAR:1]) : 1'b0; // even parity over data+P
          end
        end
        if (ph == OSR - 1) bitn <= bitn + 1'b1;
      end
    end
  end
endmodule

```

tick_gen, ser_tx and ser_rx: fractional baud ticks, parallel-load shift register with registered output, and a receiver with 2-flop synchronizer, start validation, majority vote, framing and parity checks.

The checker is written independently of the receiver, as a verification engineer would: it knows the timing (the same tick as the transmitter) and encodes the specification as five rules. It watches the line after the channel, so it sees exactly what a compliance analyzer would see, and it watches the valid/ready interface on the producer side.

```

`timescale 1ns/1ps
// Protocol checker, TX domain. It knows the bit timing (the same tick) and checks:
// R1 edges only at a bit boundary (+-1 tick)      R2 start bit is 0
// R3 stop bit is 1                                R4 parity is even
// R5 in_data is stable while in_valid && !in_ready (valid/ready rule)
module frame_chk #(parameter int DW = 8, OSR = 16, PAR = 1) (
  input logic clk, rst_n, tick, line,
  input logic in_valid, in_ready, input logic [DW-1:0] in_data
);
localparam int NB = DW + PAR + 2;
int ph, bitn, frames = 0, errors = 0; logic busy = 0, prev = 1, pv = 0;
logic [NB-1:0] fr; logic [DW-1:0] pd;

```

```

task automatic fail(input string rule);
    errors++; $display("  CHECK FAIL t=%0d us: %s (frame %0d)", $time / 1000, rule, frames);
endtask

always @(posedge clk) if (rst_n) begin
    if (pv && !in_ready && in_data != pd) fail("R5 in_data changed while valid && !ready");
    pv <= in_valid && !in_ready; pd <= in_data;
    prev <= line;
    if (!busy && prev && !line) begin busy <= 1; ph <= 0; bitn <= 0; end
    else if (busy) begin
        if (line != prev && ph > 1 && ph < OSR - 1) fail("R1 edge inside a bit");
        if (tick) begin
            if (ph == OSR/2) begin
                fr[bitn] = line;
                if (bitn == 0 && line) fail("R2 start bit not 0");
                if (bitn == NB-1) begin
                    if (!line) fail("R3 stop bit not 1");
                    if (PAR && ^fr[DW+PAR:1]) fail("R4 parity error");
                    frames++; busy <= 0;
                end
            end
            ph <= (ph == OSR - 1) ? 0 : ph + 1;
            if (ph == OSR - 1) bitn <= bitn + 1;
        end
    end
end
endmodule

```

frame_chk: an independent protocol checker with five rules, including the valid/ready stability rule from Chapter 3.

```

`timescale 1ns/1ps
module tb;
    // TX at 50 MHz, RX at 48 MHz: unrelated clocks, 115200 baud, 16x oversampling
    logic tclk = 0, rclk = 0, rst_n = 0;
    always #10.000 tclk = ~tclk;
    always #10.417 rclk = ~rclk;
    logic ttick, rtick, v = 0, rdy, line, ch, rv, fe, pe, kill = 0, flip = 0;
    logic [7:0] d = 0, rd;
    tick_gen #(2416) tg (.clk(tclk), .rst_n, .tick(ttick)); // 50e6*2416/65536 = 1.84326 MHz
    tick_gen #(2517) rg (.clk(rclk), .rst_n, .tick(rtick)); // 48e6*2517/65536 = 1.84351 MHz
    ser_tx tx (.clk(tclk), .rst_n, .tick(ttick), .in_valid(v), .in_ready(rdy), .in_data(d),
        .line);
    assign ch = (line & ~kill) ^ flip; // the "channel": error injection
    ser_rx rx (.clk(rclk), .rst_n, .tick(rtick), .line_async(ch), .out_valid(rv),
        .out_data(rd), .frame_err(fe), .par_err(pe));
    frame_chk chk (.clk(tclk), .rst_n, .tick(ttick), .line(ch), .in_valid(v), .in_ready(rdy),
        .in_data(d));
    logic [7:0] q [$]; int got = 0, bad = 0, nfe = 0, npe = 0;
    always @(posedge rclk) if (rv) begin
        got++; nfe += fe; npe += pe;
        if (!fe && !pe && rd != q[0]) bad++;
        void'(q.pop_front());
    end
    task automatic send(input logic [7:0] b);
        @(negedge tclk) v = 1; d = b; q.push_back(b);
        @(posedge tclk); while (!rdy) @(posedge tclk); // handshake completes here
        @(negedge tclk) v = 0;
    endtask
    initial begin
        #100 rst_n = 1;
        for (int i = 0; i < 200; i++) send($urandom);
        wait (tx.nleft == 0); #20000;
        $display("clean : sent 200, received %0d, data errors=%0d, frame_err=%0d, par_err=%0d,"

```

```

        , got, bad, nfe, npe);
$display("          checker frames=%0d violations=%0d", chk.frames, chk.errors);
// (1) channel forces the stop bit low; (2) channel flips data bit 0
fork send(8'h55); begin wait (tx.nleft == 1); kill = 1; wait (tx.nleft == 0); kill = 0; end
join
fork send(8'hA7);
    begin wait (tx.nleft == 10); flip = 1; wait (tx.nleft == 9); flip = 0; end
join
wait (tx.nleft == 0); #20000;
// (3) the producer breaks valid/ready: 0x22 waits for ready, then data changes to 0x33
@(negedge tclk) v = 1; d = 8'h11; q.push_back(8'h11); // accepted at once (TX idle)
@(negedge tclk)      d = 8'h22; q.push_back(8'h22); // TX busy: valid held high
repeat (3) @(negedge tclk); d = 8'h33; // VIOLATION of the rule
@(posedge tclk); while (!rdy) @(posedge tclk);
@(negedge tclk) v = 0;
#1000;
wait (tx.nleft == 0); #20000;
$display("injected: received %0d, frame_err=%0d, par_err=%0d, data errors=%0d,",
        got, nfe, npe, bad);
$display("          checker violations=%0d", chk.errors);
$finish;
end
endmodule

```

Testbench: 200 clean frames, then three injected faults - a stop bit forced low, a flipped data bit, and a producer that changes data while waiting for ready.

```

clean    : sent 200, received 200, data errors=0, frame_err=0, par_err=0,
          checker frames=200 violations=0
CHECK FAIL t=19208 us: R3 stop bit not 1 (frame 200)
CHECK FAIL t=19303 us: R4 parity error (frame 201)
CHECK FAIL t=19327 us: R5 in_data changed while valid && !ready (frame 202)
injected: received 204, frame_err=1, par_err=1, data errors=1,
          checker violations=3

```

Real Icarus Verilog output.

Reading the result: in the clean run all 200 bytes arrive intact across the two unrelated clocks and the checker sees 200 legal frames. Each injected fault is caught where it should be: the forced stop bit is a framing error in the receiver **and** an R3 violation in the checker; the flipped bit is a parity error in both; the valid/ready violation is invisible to the receiver - the frame is perfectly legal on the wire, it just carries the wrong byte (the 'data errors=1') - and only the checker's R5 identifies the real culprit. That asymmetry is the whole argument for protocol checkers at **interfaces**, not just end-to-end scoreboards.

4.8 Protocol checkers with SystemVerilog assertions

Hand-written checkers like `frame_chk` are portable, but **SystemVerilog Assertions** (SVA) express temporal rules far more compactly, work in simulation and formal verification alike, and report exactly which rule failed and when. Below is a full-SVA version of three rules for `ser_tx`, attached with `bind` so the RTL is untouched. It uses `default clocking`, `default disable iff` and sequence delays, which commercial simulators and formal tools support; Verilator 5.020 rejected those constructs, so this version is shown **without a run**.

```

// Concurrent assertions for ser_tx, attached with bind (no change to the RTL).
module ser_tx_sva #(parameter int DW = 8) (
    input logic clk, rst_n, in_valid, in_ready, line, input logic [DW-1:0] in_data
);
    default clocking cb @(posedge clk); endclocking
    default disable iff (!rst_n);
    // valid/ready: once valid is up without ready, valid and data must hold
    a_hold: assert property (in_valid && !in_ready | => in_valid && $stable(in_data))
        else $error("a_hold: valid dropped or data changed before ready");
    // an accepted word produces a start bit (0) two clocks later (line is registered)

```

```

a_start: assert property (in_valid && in_ready | => ##1 !line)
           else $error("a_start: no start bit after handshake");
// idle line is high
a_idle:  assert property (in_ready && $past(in_ready) |-> line)
           else $error("a_idle: line low while idle");
c_back2back: cover property (in_valid && in_ready ##[1:$] in_valid && in_ready);
endmodule

bind ser_tx ser_tx_sva #(.DW(DW)) u_sva (. *);

```

Full-SVA checker for ser_tx (not run: needs a simulator or formal tool with complete SVA support).

Rewriting the same properties in the subset Verilator 5.020 supports - explicit clocking and `disable iff` per property, `$past` instead of `##1` - gives a version that does run. The testbench (tb_sva.sv) reproduces the valid/ready violation of the previous section.

```

// Concurrent assertions for ser_tx, attached with bind (no change to the RTL).
module ser_tx_sva #(parameter int DW = 8) (
    input logic clk, rst_n, in_valid, in_ready, line, input logic [DW-1:0] in_data
);
// valid/ready: once valid is up without ready, valid and data must hold
a_hold: assert property (@(posedge clk) disable iff (!rst_n)
    in_valid && !in_ready | => in_valid && $stable(in_data))
    else $error("hold rule");
// an accepted word gives a start bit (0) two clocks later (line is registered)
a_start: assert property (@(posedge clk) disable iff (!rst_n)
    $past(in_valid && in_ready) | => !line)
    else $error("no start bit");
// idle line is high
a_idle:  assert property (@(posedge clk) disable iff (!rst_n)
    in_ready && $past(in_ready) |-> line)
    else $error("idle line low");
c_accept: cover property (@(posedge clk) disable iff (!rst_n) in_valid && in_ready);
endmodule

bind ser_tx ser_tx_sva #(.DW(DW)) sva (. *);

```

The Verilator-compatible version of the properties.

```

[1900000] %Error: ser_tx_sva.sv:8: Assertion failed in TOP.tb.dut.sva.a_hold: hold rule
-Info: ser_tx_sva.sv:8: Verilog $stop, ignored due to +verilator+error+limit
done at 175570 ns

```

Real Verilator 5.020 output (--binary --timing --assert; run with +verilator+error+limit+10 so the simulation continues). Times are in ps: the violation is flagged at 190 ns, the first clock edge after in_data changed.

Property style	Example	Checks
Stability	<code>valid && !ready => valid && \$stable(data)</code>	Handshake hold rules (AXI, AXI-Stream, any valid/ready)
Response	<code>req -> ##[1:16] ack</code>	Bounded latency; no hang
Sequence	<code>start => addr ##8 ack_bit</code>	Framing order of a serial protocol
Mutual exclusion	<code>\$onehot0({gnt0, gnt1, gnt2})</code>	Arbiter grants, one driver on a bus
Reset / idle	<code>!rst_n => line == 1'b1</code>	Idle-state requirements
Cover	<code>cover property (valid && ready ##1 valid && ready)</code>	Back-to-back transfers were actually exercised

Table 4.5 - Assertion patterns for protocol rules.

INTERVIEW INSIGHT: ASSERTIONS AT THE BOUNDARY

Put interface assertions in a separate checker module bound to the port list, not inside the RTL. The same file then checks the design in simulation, is proven by formal tools, travels with the IP to the SoC team, and - for AMBA protocols - can be replaced by the vendor's protocol-checker IP. Assume-guarantee reasoning makes the rules on one side of an interface the constraints of the other side in formal verification.

4.9 Bus functional models, VIP and compliance

A **bus functional model (BFM)** is testbench code that drives and samples an interface at the pin level from high-level calls - the `send()` task in `tb_serdes` is a minimal one. A **verification IP (VIP)** is a complete, reusable package for a protocol: typically a UVM agent with a driver (BFM), a monitor, a sequencer with sequence libraries, protocol assertions, functional coverage and, for complex protocols, a full reference model of the link partner.

Component	Role	In this chapter's example
Driver / BFM	Converts transactions into pin wiggles; can inject errors	<code>send()</code> task
Monitor	Reconstructs transactions from pins (passive)	The RX side of <code>frame_chk</code>
Checker / assertions	Encodes protocol rules	<code>frame_chk</code> rules R1-R5, SVA properties
Scoreboard	Compares observed with expected data end-to-end	The queue <code>q</code> and <code>data errors</code> count
Coverage	Measures which scenarios were exercised	(add: parity values, back-to-back frames, each error type)
Sequences / tests	Stimulus scenarios: random, directed, error injection	200 random frames + three fault injections

Table 4.6 - Anatomy of a VIP, mapped onto the run example.

VIP can be **active** (drives the interface, standing in for a missing link partner) or **passive** (only monitors and checks, for use at SoC level where both sides are real RTL). For standard protocols teams buy commercial VIP rather than write it, because the VIP's value lies in the thousands of spec rules and corner cases it already encodes; writing your own is justified for proprietary or very simple interfaces.

Compliance and interoperability

Passing your own tests proves only that your reading of the specification is self-consistent. Standard bodies therefore run **compliance programs**: electrical tests of the PHY (eye masks, jitter tolerance, return loss), protocol tests of the link and transaction layers (often scripted test suites run with an exerciser), and **interoperability** events (plugfests, PCI-SIG compliance workshops, USB-IF certification, MIPI conformance testing, HDMI Authorized Test Centers). For RTL teams the practical implications are: use a compliance-aware VIP in pre-silicon verification, keep PHY test modes (loopback, PRBS generators and checkers, compliance patterns) accessible from software, and plan for certification in the schedule.

Protocol-controller verification checklist

- ☐ Every rule in the specification's timing diagrams has an assertion or checker rule, bound at the interface.
- ☐ Random stimulus with back-pressure on every valid/ready interface, including long stalls and back-to-back transfers.
- ☐ Clock ratios swept (fast/slow system clock versus interface rate) and clock tolerance at the specification limits.
- ☐ Every error type injectable and every recovery path covered, including return to idle.
- ☐ Reset in the middle of a transfer; soft reset and FIFO flush.
- ☐ CDC and reset-domain crossing sign-off on the controller.
- ☐ Register model tests: reset values, access types, W1C, side effects.
- ☐ Compliance or VIP-based protocol tests for standard interfaces.

4.10 Summary

- Protocol controllers share one architecture: bus slave and CSRs, FIFOs, a protocol FSM, a bit-level datapath and a pad interface.
- Generate bit timing as clock-enable ticks - integer, fractional or phase-accumulator (Eq. 4.1) - rather than divided clocks.
- Oversampling receivers validate the start bit, sample at the bit centre with a majority vote and resynchronize every frame; 8N1 tolerates about 4.6% total clock error (Eq. 4.2).
- Synchronize every asynchronous input, use handshakes or async FIFOs for multi-bit data, and budget synchronizer latency.
- Model bidirectional pins as o/oe/i triplets; open-drain means o = 0 and oe = pull-low; real tri-states belong in pad cells.
- Every wait needs a timeout and every error a recovery path, all injectable in the testbench.
- The framed SerDes ran error-free across 50/48 MHz clocks; an independent checker and SVA caught injected framing, parity and valid/ready violations, one of which the receiver could not see.
- VIP = driver + monitor + checks + scoreboard + coverage + sequences; compliance programs close the gap between self-consistency and interoperability.

4.11 Exercises

- Compute the integer prescaler and the phase-accumulator INC for 921 600 baud x 16 from a 100 MHz clock. What is each method's rate error?
- Change ser_rx to OSR = 8 and sample at ticks 3, 4, 5. Re-derive Eq. 4.2 for this case and test the tolerance by changing the RX INC value in tb_serdes.
- Add a break detector to ser_rx (line low for a full frame) and an overrun flag (a new frame completes while out_valid data has not been read). Add checker rules for both.
- Write SVA properties for ser_rx: a framing error must be reported iff the stop-bit majority vote was 0; out_valid is a single-cycle pulse. Run them with Verilator's supported subset.
- Sketch the o/oe/i connections for an I2C controller (SCL and SDA) including clock stretching. Which inputs need synchronizers and glitch filters?
- Turn tb_serdes into a minimal UVM-style structure: separate the driver, monitor, scoreboard and a coverage group counting data values, parity values and error types.

PART II

On-Chip Protocols

Inside the die, every processor, accelerator, memory controller and peripheral talks through on-chip buses. This part covers them from the simplest to the most sophisticated: the AMBA family (APB, AHB, AXI4, AXI4-Lite and AXI4-Stream), the cache-coherency protocols that keep many caches consistent (MESI/MOESI, ACE and CHI), and the alternatives - Wishbone, Avalon, TileLink, OCP - together with the networks-on-chip that carry all of them across large SoCs.

5. AMBA APB: The Advanced Peripheral Bus

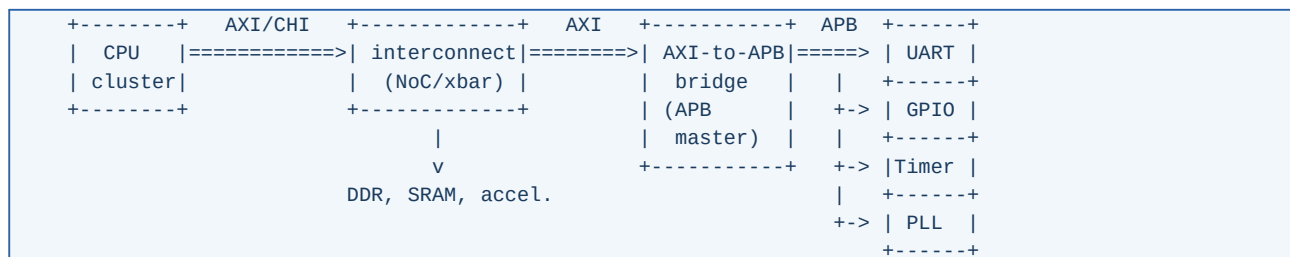
Every SoC has hundreds of small registers: UART baud dividers, GPIO direction bits, timer compare values, PLL settings, interrupt masks. None of them needs bandwidth; all of them need a bus that is cheap in gates, easy to decode, trivially timed and simple to verify. That bus is the **AMBA Advanced Peripheral Bus (APB)**. It is the slowest and simplest member of the AMBA family, and for exactly that reason it is the protocol that most engineers write first and that shows up in every tapeout: a typical smartphone SoC has dozens of APB subsystems hanging off AXI or AHB bridges.

This chapter covers APB in full: its history from APB2 to APB5, every signal, the three-state transfer machine, cycle-accurate timing with and without wait states, error signalling, protection and byte strobes, the bridge that feeds it and the decoder that fans it out. We finish with a verified APB4 slave, an address decoder with a default slave, and a testbench master whose trace shows wait states and PSLVERR cycle by cycle. The companion RTL Design guide built a minimal APB register block; here the focus is the protocol itself and the corner cases that interviewers and verification engineers probe.

5.1 Where APB sits and why it exists

APB is a **non-pipelined**, single-requester, single-outstanding protocol. One transfer occupies at least two clock cycles and nothing else happens on the bus until it finishes. Compare that with AHB (pipelined address/data) and AXI (independent channels, many outstanding transactions). APB gives up all performance to gain:

- **Tiny area:** no burst logic, no pipelining, no IDs. A slave is a decoder, a few flops and a read mux.
- **Easy timing:** all signals are registered by the requester and sampled on the next rising edge; APB clocks (PCLK) are usually a divided, slower copy of the system clock, which relaxes timing on long wires across the die.
- **Low power:** signals toggle only during an access; APB5 even adds a wake-up hint so the peripheral clock can be gated.
- **Simple verification:** the whole protocol is a handful of assertions (Section 5.9).



APB is the leaf of the SoC bus tree: one bridge (the only APB requester) fans out to many low-bandwidth peripherals.

TERMINOLOGY: MANAGER/SUBORDINATE

Older AMBA specifications say master/slave; issues published since about 2020 use **Requester/Completer** for APB and **Manager/Subordinate** for AHB and AXI. Signal names did not change. This book uses master/slave and manager/subordinate interchangeably, because both appear in RTL, documentation and interviews.

5.2 History: APB2, APB3, APB4 and APB5

Version	AMBA generation	What it added	Typical use today
APB2 (often just 'APB')	AMBA 2 (1999)	Base protocol: PSEL, PENABLE, PWRITE, PADDR, PWDATA, PRDATA. Every transfer exactly 2 cycles, no wait, no error	Legacy IP; still seen in old peripherals

Version	AMBA generation	What it added	Typical use today
APB3	AMBA 3 (2003-2004)	PREADY (wait states) and PSLVERR (error response)	Very common baseline for IP
APB4	AMBA 4 (2010)	PPROT[2:0] (privilege, security, instruction) and PSTRB[n-1:0] (write byte strobes)	Default choice for new designs
APB5	AMBA 5 (spec issues from about 2020 on)	PWAKEUP; optional user signals (PAUSER, PWUSER, PRUSER, PBUSER); optional parity check signals for functional safety; later issues add Realm Management Extension support (PNSE)	Safety (ISO 26262) and power-managed SoCs

The versions are backward compatible in a useful way: an APB3 slave works on an APB4 bus if you leave PPROT/PSTRB unconnected (it simply writes all bytes and ignores protection), and an APB2 slave works on an APB3 bus if you tie its PREADY high and PSLVERR low in the decoder. The reverse is not true: an APB4 slave that relies on PSTRB must see real strobes, so a bridge that drives PSTRB = 0 for writes (a common integration bug) turns every write into a no-op.

5.3 Signal reference

All APB signals are prefixed with **P**. The requester drives everything except PRDATA, PREADY and PSLVERR, which each completer drives back through a multiplexer in the decoder.

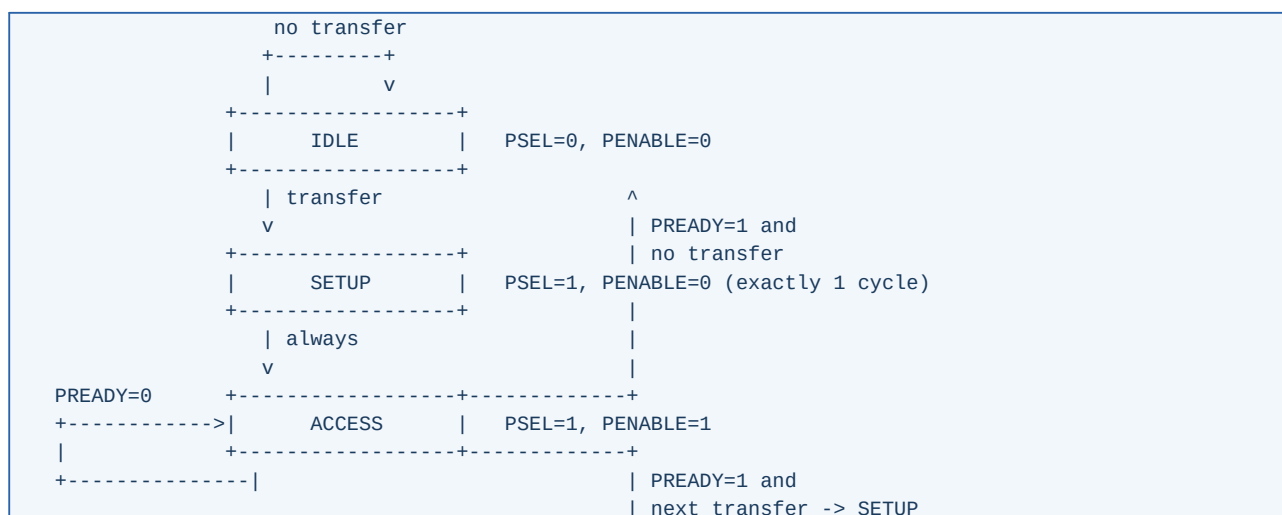
Signal	Width	Driver	Meaning
PCLK	1	clock	All signals are timed to its rising edge
PRESETn	1	reset	Active-low reset, usually the system reset synchronized to PCLK
PADDR	up to 32	requester	Byte address; may be unaligned in principle but real peripherals decode word addresses
PSELx	1 per slave	requester/decoder	Selects completer x; high for the whole transfer
PENABLE	1	requester	Low in the SETUP cycle, high in the ACCESS cycle(s)
PWRITE	1	requester	1 = write, 0 = read
PWDATA	8/16/32	requester	Write data, valid while PWRITE is high
PSTRB	PWDATA/8	requester	APB4: one bit per byte lane of PWDATA; must be all zero for reads
PPROT[2:0]	3	requester	APB4: [0] 1 = privileged, [1] 1 = non-secure , [2] 1 = instruction access
PRDATA	8/16/32	completer	Read data, sampled in the last ACCESS cycle
PREADY	1	completer	APB3+: low extends the ACCESS phase (wait states)
PSLVERR	1	completer	APB3+: error for this transfer, only meaningful when PSEL, PENABLE and PREADY are all high
PWAKEUP	1	requester	APB5: activity hint, may rise before PSEL so the completer can ungate clocks
PAUSER, PWUSER, PRUSER, PBUSER	user	both	APB5: user-defined sideband (for example a master ID or QoS hint)
xxxCHK	varies	both	APB5 optional odd-parity check bits per signal group, for safety mechanisms

PITFALL: PPROT[1] IS NON-SECURE

PPROT[1] = 1 means a **non-secure** access (the same polarity as AxPROT[1] in AXI). Designers who read it as 'secure = 1' build TrustZone firewalls that block the secure world and let the normal world in. Write the polarity into the register specification and cover both values in the testbench.

5.4 The transfer state machine

The APB specification describes the requester with three states. Every transfer visits SETUP exactly once and ACCESS one or more times:



APB requester operating states. Address, control and write data must stay stable from SETUP until the ACCESS cycle in which PREADY is high.

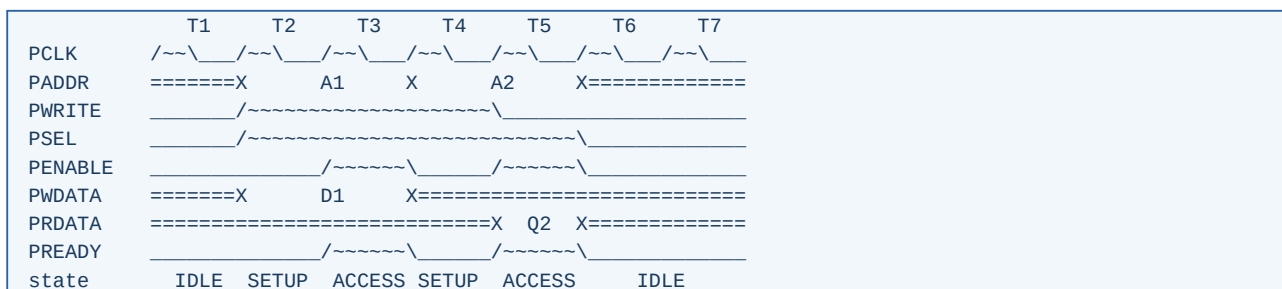
- **IDLE:** PSELx low. The bus parks here when there is nothing to do.
- **SETUP:** PSELx rises, PADDR/PWRITE/PWDATA/PSTRB/PPROT become valid, PENABLE is low. Lasts exactly one PCLK cycle.
- **ACCESS:** PENABLE rises. The completer samples the transfer; if PREADY is low the requester stays in ACCESS with everything held stable. When PREADY is high the transfer completes on that rising edge; the requester returns to IDLE or goes straight to SETUP for a back-to-back transfer (PSEL may stay high, PENABLE must drop).

Two consequences follow. First, the best-case throughput is **one transfer per two PCLK cycles**: at 100 MHz with 32-bit data that is 200 MB/s peak, plenty for registers but nothing like a data path. Second, because PENABLE must go low between transfers, a completer can always recognize the start of a new transfer as the **PSEL & ~PENABLE** cycle.

5.5 Timing diagrams

In the diagrams below each column is one PCLK cycle. Signals change just after a rising edge and are sampled by the completer (or requester) on the next rising edge; '=' marks a bus with no valid value.

Write and read without wait states



Write to A1 followed immediately by a read of A2. PSEL stays high between the transfers; PENABLE drops for the second SETUP. The write completes at the edge ending T3, the read data Q2 is sampled at the edge ending T5.

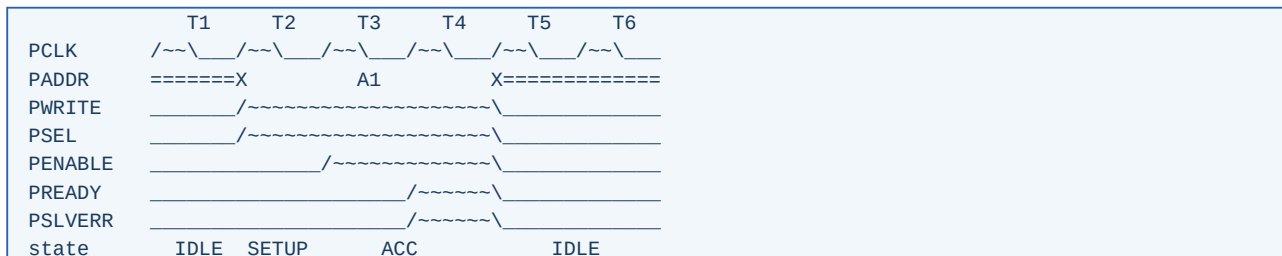
Transfers with wait states



Read with two wait states. The completer holds PREADY low in T3 and T4; the requester keeps PADDR, PWRITE, PSEL and PENABLE stable. PRDATA is only required to be valid in the final ACCESS cycle (T5).

PREADY is **don't care** outside ACCESS: many completers simply drive it high all the time except when they need to stall, and a completer that never stalls it to 1. The requester, however, must only look at PREADY when PSEL and PENABLE are both high.

Error response



A write that fails: PSLVERR is high in the last ACCESS cycle, together with PREADY. PSLVERR is only sampled in that cycle.

The APB specification is deliberately loose about what an error means. A failed write **may or may not** have changed the completer's state, and a failed read may return any data. It is the bridge's job to convert PSLVERR into the upstream error (SLVERR on AXI, ERROR on AHB) so that software sees a bus fault. Typical reasons for asserting PSLVERR are: an access to an unmapped offset inside the peripheral, a write to a read-only register, a non-secure or unprivileged access to a protected register (PPROT), a FIFO overflow, or a timeout in a clock-domain-crossing wrapper.

TIE OFF UNUSED COMPLETER OUTPUTS

An APB2 peripheral on an APB3/4 bus has no PREADY/PSLVERR. Tie PREADY = 1 and PSLVERR = 0 **inside the decoder**, per port. Leaving them floating in RTL gives X in simulation and random values in silicon, and the lint tool will only warn.

5.6 PSTRB and PPROT in practice

PSTRB lets software do byte and halfword writes to a 32-bit APB register, which matters for C code that writes `uint8_t` fields in a packed register struct. The completer ANDs each byte lane's write enable with its strobe bit. Two rules: PSTRB must be all zero for reads, and an APB4 completer is allowed to ignore PSTRB (treating every write as full-word) only if the register specification says so. Sparse strobes (for example 4'b1001) are legal and must be handled or explicitly flagged as an error.

PPROT carries the same three attributes as AXI's AxPROT: privilege, security and instruction/data. In a TrustZone system a secure peripheral (a key store, the secure watchdog, a TZPC configuration block) checks PPROT[1] and answers non-secure accesses with PSLVERR, or with read-as-zero/write-ignored (RAZ/WI) if the security architecture prefers not to reveal the register's existence. Whichever policy is chosen must be consistent across the SoC and written down for the firmware team.

5.7 Bridges: how APB is fed

An APB bus has exactly one requester, and it is almost always a bridge from a faster protocol. The bridge is an AHB or AXI completer on one side and the APB requester on the other.

AHB-to-APB bridge

The AHB side captures the address phase (HADDR, HWRITE, HSIZE, HPROT) when HSEL and HREADY are high and the transfer is NONSEQ or SEQ. For a read it can start the APB SETUP immediately; for a write it must wait one cycle because HWDATA only arrives in the AHB data phase. While the APB transfer runs the bridge holds HREADYOUT low. PSLVERR becomes the two-cycle AHB ERROR response (Chapter 6). A typical non-buffered bridge therefore costs about three AHB cycles per write and per read at equal clocks, more when PCLK is a divided clock. Some bridges **post** writes (return OKAY on AHB before the APB write finishes) to hide the latency, which means PSLVERR on a posted write can no longer be reported to the requester that caused it.

AXI-to-APB bridge

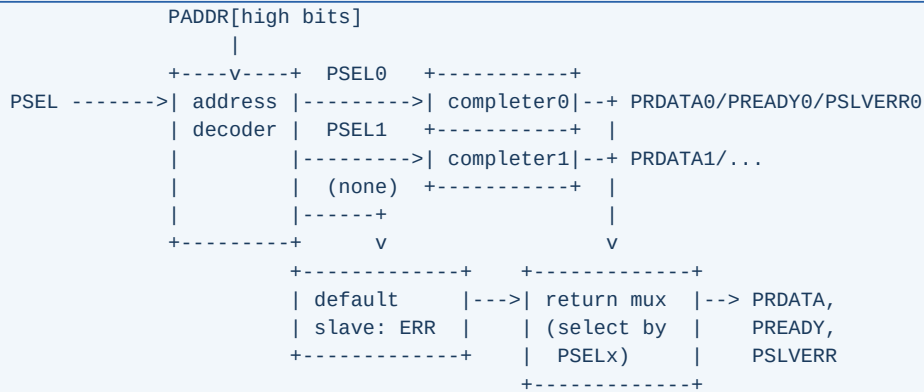
An AXI bridge must also serialize: it accepts one AW+W or one AR at a time (arbitrating between reads and writes), splits any burst into individual APB transfers, generates PSTRB from WSTRB, maps AxPROT to PPROT, and returns B or R with SLVERR if any beat saw PSLVERR. Because APB has no IDs, the bridge echoes the stored AXI ID. The Arm CoreLink bridges also support **multiple APB ports**, i.e. the decoder of Section 5.8 is built into the bridge.

CLOCK DOMAINS AND APB

The APB clock is frequently a synchronous divided clock ($PCLK = HCLK/2, /4$) with a clock-enable in the bridge, so no CDC is needed. When a peripheral lives in a truly asynchronous domain (for example an always-on block), put an **APB asynchronous bridge** (a request/acknowledge handshake, Chapter 4) in front of it: the APB transfer on the fast side waits with PREADY low until the slow side acknowledges. Budget several slow-clock cycles of latency per access and never poll such a register in a tight loop.

5.8 Decoding and multiplexing multiple completers

PADDR, PWRITE, PENABLE, PWDATA, PSTRB and PPROT are broadcast to all completers. The decoder turns the address into one-hot PSELx and multiplexes PRDATA, PREADY and PSLVERR back from the selected completer:



APB decoder: one-hot PSELx, a return multiplexer, and a default slave that answers unmapped addresses with PSLVERR.

- **Decode on PADDR only while PSEL is high.** When the requester is idle, all PSELx must be low.
- **Default slave.** An address that hits nothing must still complete, otherwise the bus hangs forever with PREADY low. The decoder answers it with PREADY = 1 and PSLVERR = 1 in ACCESS.
- **Mux PRDATA with PSELx, not with PADDR alone,** and drive zero when nothing is selected; this avoids X propagation in simulation and saves power in silicon.
- Use **AND-OR multiplexing** (each completer drives zero when not selected, then OR everything) when there are many completers; it synthesizes to a shallow tree and is the style used in many commercial APB subsystems.

5.9 Verification: the protocol rules as assertions

Almost the whole APB protocol can be checked with a handful of SystemVerilog assertions bound to the interface. A checker like this belongs on every APB port in the SoC-level testbench:

```
// APB protocol checker (bind to every APB port)
property p_setup_then_access;          // SETUP is followed by ACCESS
  @(posedge pclk) disable iff (!presetn)
    psel && !penable | => psel && penable;
endproperty

property p_stable_in_access;           // everything stable until PREADY
  @(posedge pclk) disable iff (!presetn)
    psel && penable && !pready | => $stable({paddr, pwrite, pwrite, pstrb, pprot})
    && psel && penable;
endproperty

property p_enable_drops;               // PENABLE low after completion
  @(posedge pclk) disable iff (!presetn)
    psel && penable && pready | => !penable;
endproperty

property p_read_strobes_zero;         // APB4: PSTRB = 0 on reads
  @(posedge pclk) disable iff (!presetn) psel && !pwrite | -> pstrb == '0;
endproperty

a1: assert property (p_setup_then_access);
a2: assert property (p_stable_in_access);
a3: assert property (p_enable_drops);
a4: assert property (p_read_strobes_zero);
c1: cover property (@(posedge pclk) psel && penable && !pready); // saw a wait
c2: cover property (@(posedge pclk) psel && penable && pready && pslverr);
```

Four assertions and two covers capture nearly all of APB. The covers prove that the test actually exercised wait states and errors.

Add a **timeout** assertion (PREADY must rise within N cycles, where N comes from the peripheral's specification) and an X-check on PRDATA when PREADY is high for a read. For register-level verification the UVM register layer (Chapter 25) drives this interface through an adapter; the APB agent is the simplest VIP most DV engineers ever write, which is why it is the classic first UVM project.

5.10 Hands-on: APB4 slave, decoder and a testbench master

The example has three pieces. `apb4_regs` is an APB4 completer with five registers: CTRL and SCRATCH (read/write), KEY (secure only: a non-secure access, `PPROT[1] = 1`, gets `PSLVERR`), ID (read-only: a write gets `PSLVERR`) and SLOW, which inserts a programmable number of wait states. Writes honour `PSTRB` and commit exactly once, in the final `ACCESS` cycle. `apb_mux` decodes two completers at `0x00xx` and `0x10xx` and contains a default slave for everything else.

```
// APB4 register slave: PSTRB byte writes, PPROT secure check, programmable
// wait states on one "slow" register, PSLVERR for bad accesses.
module apb4_regs #(parameter int WS = 2) (      // wait states on SLOW register
    input  logic      pclk, presetn,
    input  logic      psel, penable, pwrite,
    input  logic [7:0] paddr,
    input  logic [2:0] pprot,                  // [0] priv, [1] NON-secure, [2] instr
    input  logic [31:0] pwrdata,
    input  logic [3:0] pstrb,
    output logic [31:0] prdata,
    output logic      pready, pslverr
);
    localparam logic [7:0] CTRL = 8'h00, SCRATCH = 8'h04, KEY = 8'h08,
                        ID   = 8'h0C, SLOW   = 8'h10;
    logic [31:0] ctrl, scratch, key, slow;
    logic [3:0] wcnt;
    wire setup = psel & ~penable;
    wire access = psel & penable;
    wire mapped = (paddr == CTRL) || (paddr == SCRATCH) || (paddr == KEY)
                || (paddr == ID)   || (paddr == SLOW);
    wire err = !mapped
              || (pwrite && paddr == ID)           // read-only register
              || (paddr == KEY && pprot[1]);       // non-secure access to KEY

    // wait-state counter: loaded in SETUP, counts down during ACCESS
    always_ff @(posedge pclk or negedge presetn)
        if (!presetn) wcnt <= '0;
        else if (setup) wcnt <= (paddr == SLOW) ? 4'(WS) : 4'd0;
        else if (access && wcnt != 0) wcnt <= wcnt - 4'd1;

    assign pready = !(access && wcnt != 0);
    assign pslverr = access && pready && err;      // valid only in last cycle

    wire wr = access && pready && pwrite && !err;  // commit exactly once
    always_ff @(posedge pclk or negedge presetn)
        if (!presetn) begin ctrl <= '0; scratch <= '0; key <= '0; slow <= '0; end
        else if (wr)
            for (int b = 0; b < 4; b++) if (pstrb[b])
                case (paddr)
                    CTRL: ctrl  [8*b+: 8] <= pwrdata[8*b+: 8];
                    SCRATCH: scratch[8*b+: 8] <= pwrdata[8*b+: 8];
                    KEY: key   [8*b+: 8] <= pwrdata[8*b+: 8];
                    SLOW: slow  [8*b+: 8] <= pwrdata[8*b+: 8];
                    default: ;
                endcase
            endcase

    always_comb
        if (!(access && !pwrite && !err)) prdata = '0; // drive zero when not reading
        else case (paddr)
            CTRL: prdata = ctrl;
            SCRATCH: prdata = scratch;
            KEY: prdata = key;
        endcase
endmodule
```

```

        ID:      prdata = 32'h0A9B_0004;
        SLOW:    prdata = slow;
        default: prdata = '0;
    endcase
endmodule

// APB decoder + return mux: slave 0 at 0x00xx, slave 1 at 0x10xx,
// anything else is answered by a built-in default slave with PSLVERR.
module apb_mux (
    input  logic      psel, penable,
    input  logic [15:0] paddr,
    output logic      psel0, psel1,
    input  logic [31:0] prdata0, prdata1,
    input  logic      pready0, pready1, pslverr0, pslverr1,
    output logic [31:0] prdata,
    output logic      pready, pslverr
);
    assign psel0 = psel && paddr[15:8] == 8'h00;
    assign psel1 = psel && paddr[15:8] == 8'h10;
    wire  nomap = psel && !psel0 && !psel1;
    assign prdata  = psel0 ? prdata0 : psel1 ? prdata1 : 32'h0;
    assign pready  = psel0 ? pready0 : psel1 ? pready1 : 1'b1;
    assign pslverr = psel0 ? pslverr0 : psel1 ? pslverr1 : (nomap && penable);
endmodule

```

apb4_regs (APB4 completer) and apb_mux (decoder, return mux and default slave).

The testbench master is a single task that performs SETUP for one cycle and then ACCESS until PREADY is sampled high, exactly the state machine of Section 5.4. A monitor prints every cycle in which PSEL is high and checks that address, direction and write data do not change while the completer is stalling.

```

module tb;
    logic pclk = 0, presetn = 0;
    always #5 pclk = ~pclk;
    logic psel = 0, penable = 0, pwrite = 0;
    logic [15:0] paddr = 0; logic [31:0] pwrite = 0; logic [3:0] pstrb = 0;
    logic [2:0] pprot = 0;
    logic psel0, psel1, pready0, pready1, pslverr0, pslverr1, pready, pslverr;
    logic [31:0] prdata0, prdata1, prdata;
    int cyc = 0, viol = 0;

    apb_mux mux (.*);
    apb4_regs #(.WS(2)) s0 (.pclk, .presetn, .psel(psel0), .penable, .pwrite,
        .paddr(paddr[7:0]), .pprot, .pwrite, .pstrb, .prdata(prdata0),
        .pready(pready0), .pslverr(pslverr0));
    apb4_regs #(.WS(0)) s1 (.pclk, .presetn, .psel(psel1), .penable, .pwrite,
        .paddr(paddr[7:0]), .pprot, .pwrite, .pstrb, .prdata(prdata1),
        .pready(pready1), .pslverr(pslverr1));

    // ---- master tasks: SETUP for one cycle, then ACCESS until PREADY ----
    task automatic apb_xfer(input logic wr, input logic [15:0] a, input logic [31:0] d,
        input logic [3:0] s, input logic [2:0] pr,
        output logic [31:0] rd, output logic err);
        psel <= 1; penable <= 0; pwrite <= wr; paddr <= a; pprot <= pr;
        pwrite <= wr ? d : 32'h0; pstrb <= wr ? s : 4'h0; // PSTRB=0 on reads
        @(posedge pclk); penable <= 1;
        do @(posedge pclk); while (!pready);
        rd = prdata; err = pslverr;
        psel <= 0; penable <= 0;
    endtask

```



```
// ---- cycle trace + simple protocol checker ----
logic [15:0] a_q; logic w_q; logic [31:0] d_q; logic stall_q = 0;
always @(posedge pclk) begin
    cyc++;
    if (psel)
        $display("%3d %-6s %04h %s %08h %b %b %1b %1b %08h", cyc,
            penable ? (pready ? "ACCESS" : "ACC/w") : "SETUP", paddr, pwrite ? "w" : "R",
            pwdata, pstrb, pprot, pready, pslverr, prdata);
    if (stall_q && (paddr != a_q || pwrite != w_q || pwdata != d_q || !psel)) viol++;
    stall_q <= psel && penable && !pready;
    a_q <= paddr; w_q <= pwrite; d_q <= pwdata;
end

logic [31:0] rd; logic err;
initial begin
    repeat (2) @(posedge pclk); presetn <= 1; @(posedge pclk);
    $display("cyc phase ADDR D PWDATA STRB PROT RDY ERR PRDATA");
    apb_xfer(1, 16'h0004, 32'h1122_3344, 4'b1111, 3'b000, rd, err); // scratch
    apb_xfer(1, 16'h0004, 32'hAAAA_BBCC, 4'b0001, 3'b000, rd, err); // byte 0 only
    apb_xfer(0, 16'h0004, 0, 0, 3'b000, rd, err);
    apb_xfer(1, 16'h0010, 32'hCAFE_F00D, 4'b1111, 3'b000, rd, err); // 2 wait states
    apb_xfer(0, 16'h0010, 0, 0, 3'b000, rd, err);
    apb_xfer(1, 16'h0008, 32'h5EC0_0001, 4'b1111, 3'b010, rd, err); // non-secure->KEY
    apb_xfer(1, 16'h000C, 32'h0, 4'b1111, 3'b000, rd, err); // write R0 ID
    apb_xfer(0, 16'h100C, 0, 0, 3'b000, rd, err); // slave 1 ID
    apb_xfer(0, 16'h2000, 0, 0, 3'b000, rd, err); // unmapped
    @(posedge pclk);
    $display("protocol violations = %0d", viol);
    $finish;
end
endmodule
```

Testbench: APB master task, cycle trace and a stability checker. Compiled with iverilog -g2012 -Wall.

cyc	phase	ADDR	D	PWDATA	STRB	PROT	RDY	ERR	PRDATA
4	SETUP	0004	W	11223344	1111	000	1	0	00000000
5	ACCESS	0004	W	11223344	1111	000	1	0	00000000
6	SETUP	0004	W	aaaabbcc	0001	000	1	0	00000000
7	ACCESS	0004	W	aaaabbcc	0001	000	1	0	00000000
8	SETUP	0004	R	00000000	0000	000	1	0	00000000
9	ACCESS	0004	R	00000000	0000	000	1	0	112233cc
10	SETUP	0010	W	cafef00d	1111	000	1	0	00000000
11	ACC/w	0010	W	cafef00d	1111	000	0	0	00000000
12	ACC/w	0010	W	cafef00d	1111	000	0	0	00000000
13	ACCESS	0010	W	cafef00d	1111	000	1	0	00000000
14	SETUP	0010	R	00000000	0000	000	1	0	00000000
15	ACC/w	0010	R	00000000	0000	000	0	0	cafef00d
16	ACC/w	0010	R	00000000	0000	000	0	0	cafef00d
17	ACCESS	0010	R	00000000	0000	000	1	0	cafef00d
18	SETUP	0008	W	5ec00001	1111	010	1	0	00000000
19	ACCESS	0008	W	5ec00001	1111	010	1	1	00000000
20	SETUP	000c	W	00000000	1111	000	1	0	00000000
21	ACCESS	000c	W	00000000	1111	000	1	1	00000000
22	SETUP	100c	R	00000000	0000	000	1	0	00000000
23	ACCESS	100c	R	00000000	0000	000	1	0	0a9b0004
24	SETUP	2000	R	00000000	0000	000	1	0	00000000
25	ACCESS	2000	R	00000000	0000	000	1	1	00000000

protocol violations = 0

Real Icarus Verilog output. ACC/w marks an ACCESS cycle with PREADY low (a wait state).

Read the trace from top to bottom:

- Cycles 4-5: a full-word write to SCRATCH, two cycles, no wait.

- Cycles 6-9: a write with PSTRB = 0001 changes only byte 0 (0xCC), so the read-back is 0x112233CC. Note PSTRB = 0000 on the read.
- Cycles 10-17: SLOW inserts two wait states on both the write and the read. The completer drives PRDATA early (cycle 15), but only the value in cycle 17, when PREADY is high, counts.
- Cycle 19: a non-secure write (PPROT = 010) to KEY returns PSLVERR. Cycle 21: a write to the read-only ID register returns PSLVERR.
- Cycle 23: the read of 0x100C reaches the second completer and returns its ID. Cycle 25: 0x2000 matches nothing, and the default slave completes the transfer with PSLVERR instead of hanging the bus.
- Transfers run back to back: SETUP follows ACCESS directly, with PSEL staying high, which the protocol allows.

PITFALL: COMMITTING A WRITE MORE THAN ONCE

A completer that writes its register whenever `p_sel && p_enable && p_write` is true commits the write in **every** wait-state cycle. That is harmless for a plain register but fatal for a FIFO push, a write-1-to-clear status bit or a counter increment. Qualify every side effect with PREADY, as `wr` does in `apb4_regs`. The same rule applies to read side effects (clear-on-read registers, FIFO pops): they must happen exactly once, in the completing cycle.

PITFALL: COMBINATIONAL PREADY PATHS

PREADY often comes from deep inside a peripheral (a FIFO full flag, a CDC handshake). If it is combinational from PADDR through the decoder, the peripheral and the return mux back into the bridge, it can become the critical path of the whole APB subsystem. Keep PREADY registered where possible, as the wait counter in the example does.

5.11 Who owns what, and interview insights

Team	APB responsibilities
RTL (IP)	Completer logic, register map, PSLVERR policy, exactly-once side effects, PSTRB/PPROT handling
RTL (integration)	Bridge and decoder configuration, address map, tie-offs for APB2/APB3 IP, clock ratio between HCLK and PCLK
DV	APB VIP and assertions, register-model tests (reset values, access types), error and wait-state coverage
PD / STA	Usually easy; watch long PRDATA/PREADY return paths on a large die, and the multicyle constraints of a divided PCLK
Firmware	Register headers, byte-access rules, handling of bus faults raised by PSLVERR

INTERVIEW QUESTIONS YOU SHOULD BE ABLE TO ANSWER

Why does APB need two cycles minimum? SETUP gives the completer one full cycle of stable address/control to decode before PENABLE says 'do it now'; this is what makes the protocol timing-friendly and decode-friendly without pipelining.

Can PSEL stay high between two transfers? Yes, when the second transfer is to the same completer; PENABLE must still go low for one SETUP cycle.

What happens if a slave never asserts PREADY? The bus and the CPU hang. Real SoCs add a timeout monitor in the bridge that forces an error response after N cycles.

When is PRDATA valid? Only in the last ACCESS cycle (`PSEL & PENABLE & PREADY & !PWRITE`).

5.12 Summary

- APB is the non-pipelined, single-outstanding leaf bus for registers and low-bandwidth peripherals; its only requester is a bridge.
- APB3 added PREADY and PSLVERR, APB4 added PPROT and PSTRB, APB5 adds PWAKEUP, user and parity signals.
- Every transfer is SETUP (one cycle) plus ACCESS (one or more cycles, extended by PREADY low); all requester signals are stable until the completing edge.
- PSLVERR and PRDATA are only meaningful in the final ACCESS cycle; side effects must happen exactly once, in that cycle.
- A decoder produces one-hot PSELx, multiplexes the return signals and needs a default slave so unmapped accesses complete with an error instead of hanging.
- The verified example showed wait states, PSTRB byte writes, PPROT-based security errors and a default-slave error.

5.13 Exercises

- Draw the timing diagram for a write with one wait state followed by a read with zero wait states to a different completer. Mark the edges where each completer samples its inputs.
- Modify `apb4_regs` so that KEY is read-as-zero/write-ignored (RAZ/WI) for non-secure accesses instead of returning PSLVERR. What does firmware lose and gain with each policy?
- An APB peripheral runs at $PCLK = HCLK/4$ with a clock-enable in the AHB-to-APB bridge. Count the HCLK cycles for a zero-wait APB read, from the AHB address phase to the AHB data phase completing.
- Write an SVA property that fails if PSLVERR is ever high while PREADY is low during ACCESS, and explain why a completer might legitimately do that (hint: what does the spec require?).
- Add a timeout to `apb_mux`: if PREADY stays low for 64 cycles, force PREADY = 1 and PSLVERR = 1. What must the completer do when its late PREADY finally arrives?
- Estimate the maximum register-write rate (writes per second) of a CPU at 2 GHz writing an APB register at $PCLK = 50$ MHz through a non-posted bridge with 20 ns of interconnect latency each way.

6. AMBA AHB and AHB-Lite

The **Advanced High-performance Bus (AHB)** sits between APB and AXI. It introduced the idea that dominates every fast on-chip bus since: **pipelining** the address of the next transfer with the data of the current one, so that a stream of transfers moves one word per clock. AHB is still everywhere in microcontrollers (every Arm Cortex-M0/M3/M4 core exposes AHB-Lite, Cortex-M23/M33 expose AHB5), in low-power subsystems of large SoCs, in on-chip flash controllers and in countless third-party IP blocks. Understanding its address/data pipeline is also the best preparation for AXI.

6.1 Three generations: AHB, AHB-Lite and AHB5

Variant	Spec	Key features	Status
AHB (multi-master)	AMBA 2 (1999)	Shared bus with a central arbiter (HBUSREQx, HGRANTx, HLOCKx, HMASTER, HMASTLOCK); 2-bit HRESP with OKAY, ERROR, RETRY and SPLIT ; HSPLITx for split transactions	Obsolete for new design; complex, poor for multi-master throughput
AHB-Lite	AMBA 3 (2006)	Single master per bus; 1-bit HRESP (OKAY/ERROR); no arbitration signals, no RETRY/SPLIT. Multiple masters are handled by a multi-layer interconnect instead	The workhorse of MCUs and peripheral subsystems
AHB5	AMBA 5 (2015)	AHB-Lite plus: HNONSEC (TrustZone), HEXCL/HEXOKAY (exclusive access), HMASTER, extended HPROT[6:0] memory types, optional user signals (HAUSER, HWUSER, HRUSER, HBUSER), a secure/non-secure split, and (in later issues) parity check signals; formally renamed master/slave to manager/subordinate	Cortex-M23/M33/M55/M85 and new IP

The rest of this chapter describes AHB-Lite (and AHB5, which is a superset) in detail, then shows how the multi-master problem is solved with a multi-layer interconnect, and finally what the old AMBA 2 arbitration looked like, because legacy IP and interview questions still use it.

6.2 Signal reference (AHB-Lite / AHB5)

Signal	Dir (from master)	Meaning
HCLK, HRESETn	global	Clock (all signals timed to its rising edge) and active-low reset
HADDR[31:0]	out	Byte address of the transfer (address phase)
HTRANS[1:0]	out	00 IDLE , 01 BUSY , 10 NONSEQ , 11 SEQ
HWRITE	out	1 = write, 0 = read
HSIZE[2:0]	out	Bytes per beat = 2^{HSIZE} : 000 byte, 001 half, 010 word, 011 dword, up to 111 = 1024 bits; must not exceed the bus width
HBURST[2:0]	out	000 SINGLE, 001 INCR (undefined length), 010 WRAP4, 011 INCR4, 100 WRAP8, 101 INCR8, 110 WRAP16, 111 INCR16
HPROT[3:0] / [6:0]	out	[0] 1 = data (0 = opcode fetch), [1] 1 = privileged, [2] bufferable, [3] cacheable. AHB5 extended memory types: [3] becomes modifiable, plus [4] lookup, [5] allocate, [6] shareable
HMASTLOCK	out	Current transfer is part of a locked sequence
HWDATA	out	Write data (data phase , one cycle after the address)
HNONSEC	out	AHB5: 1 = non-secure transfer
HEXCL, HMASTER	out	AHB5: exclusive transfer; master ID for the monitor

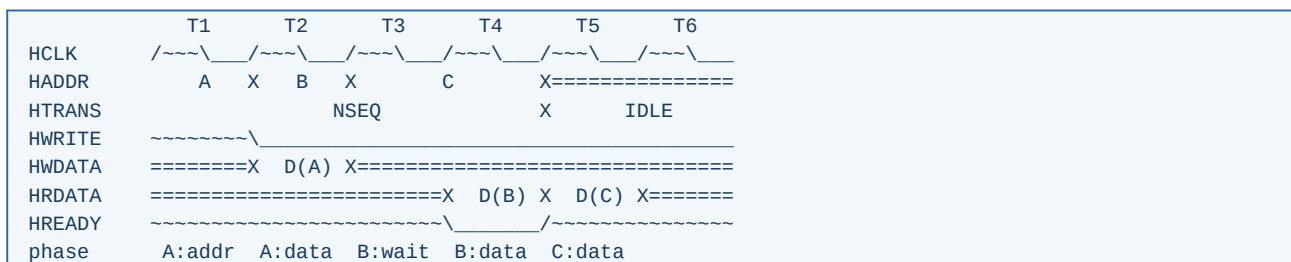
Signal	Dir (from master)	Meaning
HSELx	decoder	Selects slave x (combinational decode of HADDR)
HRDATA	in	Read data from the slave, through the read mux
HREADYOUT	slave out	Slave: 0 = extend the data phase (wait state)
HREADY	in (to all)	Multiplexed HREADYOUT of the slave in the data phase; tells every slave and the master that the bus moved
HRESP	in	0 OKAY, 1 ERROR (AHB-Lite/AHB5)
HEXOKAY	in	AHB5: exclusive transfer succeeded

HREADY VERSUS HREADYOUT

Each slave **drives** HREADYOUT and **receives** HREADY. HREADY is the HREADYOUT of whichever slave currently owns the data phase, selected by the slave multiplexer. A slave must only sample the address phase (HSEL, HADDR, HTRANS...) when HREADY is high, because while another slave is stalling, the address on the bus is not yet accepted. Forgetting this is the number-one AHB slave bug.

6.3 The pipeline: address phase and data phase

Every AHB transfer has an **address phase** (one cycle, or longer if the previous transfer is stalled) and a **data phase** (one cycle plus wait states). The data phase of transfer N overlaps the address phase of transfer N+1:



Three transfers: write A, read B (one wait state), read C. The address phase of C is extended in T3 because the data phase of B is stalled (HREADY low); the master must hold HADDR/HTRANS until HREADY is high.

Rules that follow from the pipeline:

- The slave captures the address/control into registers at the end of the address phase (HSEL & HREADY & HTRANS[1]) and uses the **registered** copy during the data phase.
- Write data arrives one cycle after its address, so a write to an SRAM cannot happen in the address phase; read data, in contrast, can be fetched from a synchronous SRAM by presenting the address in the address phase so that data is ready in the data phase with zero wait states.
- When the slave inserts wait states (HREADYOUT low), the master holds the **next** address stable. The spec allows only a few HTRANS changes during waits (for example IDLE to NONSEQ, or BUSY to SEQ); an address, once presented with NONSEQ or SEQ, must be held until accepted.
- Peak throughput is **one beat per cycle**, twice APB's best case.

6.4 Transfer types: HTRANS

HTRANS	Name	Meaning	Slave response
00	IDLE	No transfer. Used when the master has nothing to do, and after a locked sequence	OKAY, zero wait states
01	BUSY	Inside a burst: the master needs a cycle (for example, its data is not ready) but the burst continues. HADDR shows the next address of the burst	OKAY, zero wait; ignore the transfer

HTRANS	Name	Meaning	Slave response
10	NONSEQ	First beat of a burst, or a single transfer; address unrelated to the previous one	Normal transfer
11	SEQ	Remaining beats of a burst; address = previous + size (wrapped for WRAPx)	Normal transfer

A useful mental model: HTRANS[1] means **there is a real transfer**, HTRANS[0] means **this continues a burst**. A slave that never needs burst information can decode only HTRANS[1], exactly as the SRAM slave below does.

6.5 Bursts, wrapping and the 1KB boundary

HBURST tells the slave how many beats follow and how the address advances. Fixed-length bursts (INCR4/8/16, WRAP4/8/16) allow a memory controller to prefetch; INCR is an undefined-length burst that ends when the master issues NONSEQ or IDLE.

HBURST	Type	Beats	Address sequence (word size, start 0x38)
000	SINGLE	1	0x38
001	INCR	any	0x38, 0x3C, 0x40, ... until NONSEQ/IDLE
010	WRAP4	4	0x38, 0x3C, 0x30 , 0x34 (wraps at 16 bytes)
011	INCR4	4	0x38, 0x3C, 0x40, 0x44
100	WRAP8	8	0x38, 0x3C, 0x20 , 0x24, 0x28, 0x2C, 0x30, 0x34
101	INCR8	8	0x38 ... 0x54
110 / 111	WRAP16 / INCR16	16	wrap at 64 bytes / linear

Wrapping bursts exist for **cache line fills**: the CPU asks first for the word it missed on (the critical word) and the burst wraps around the line boundary to fetch the rest. The wrap boundary is beats x bytes-per-beat, which for WRAP4 of words is 16 bytes.

PITFALL: THE 1KB BOUNDARY

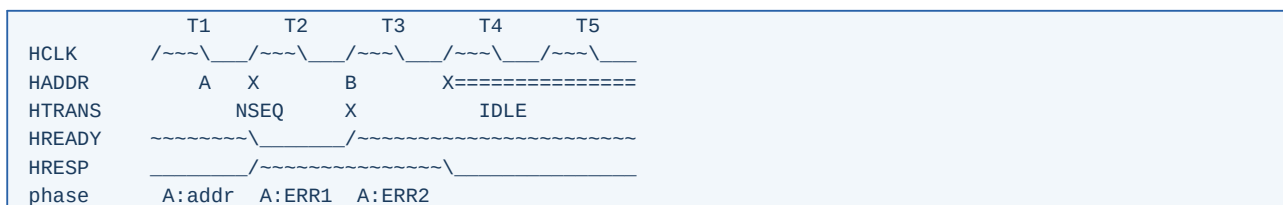
An AHB burst must **never cross a 1KB address boundary**. Slaves are decoded on at least 1KB granularity, so crossing would silently move the middle of a burst into a different slave that never saw the NONSEQ. Masters must split INCR bursts at 1KB (DMA engines forget this, typically when the transfer length is not a multiple of the burst size). Compare with AXI's 4KB rule in Chapter 7.

Bursts can be **terminated early**. In multi-master AMBA 2 AHB a master that lost the bus stopped mid-burst; in AHB-Lite the master may abandon a burst after an ERROR response; in a multi-layer system a burst can be broken by the interconnect. Slaves therefore must never rely on seeing all the SEQ beats promised by HBURST; they may use HBURST only as a prefetch hint.

6.6 Wait states and the two-cycle error response

A slave extends the data phase by driving HREADYOUT low. There is no protocol limit, but the specification recommends that slaves do not insert more than about 16 wait states so that the bus cannot be blocked for long; slow slaves should instead be moved behind a bridge or use a split/retry strategy (AMBA 2).

An **ERROR** response always takes **two cycles**:



Two-cycle ERROR: cycle 1 has HRESP = 1 with HREADY = 0, cycle 2 has HRESP = 1 with HREADY = 1. The master used cycle 1 to cancel the pipelined transfer B by changing HTRANS to IDLE.

Why two cycles? Because of the pipeline: when the error for A is signalled, the address of the next transfer B is already on the bus. The first cycle (HREADY low) holds B off and gives the master time to decide whether to **cancel** it (drive IDLE) or continue. Without that extra cycle B would already be accepted by the time the master saw the error. The master is allowed, but not required, to cancel the rest of a burst after an ERROR.

THE DEFAULT SLAVE

Every AHB decoder needs a **default slave** that is selected for unmapped addresses. It answers IDLE/BUSY with a zero-wait OKAY and NONSEQ/SEQ with the two-cycle ERROR. Without it an access to a hole in the memory map selects nothing, HREADY is undefined and the system hangs.

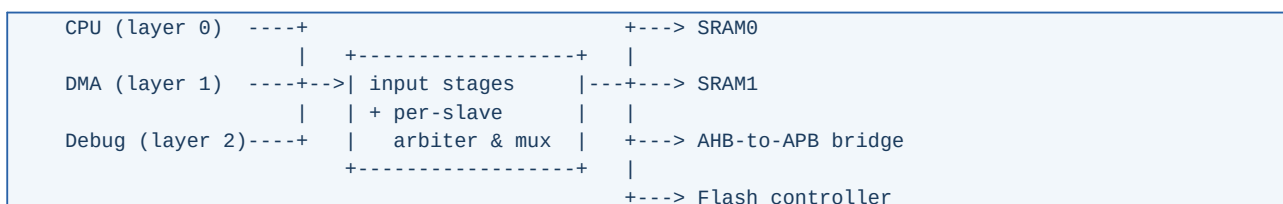
6.7 Multiple masters: arbitration, multi-layer AHB and HMASTLOCK

AMBA 2 AHB: shared bus with an arbiter

In the original AHB, several masters shared one set of address/data wires. Each master raised HBUSREQx, the arbiter chose one by priority and raised HGRANTx; the granted master took ownership of the address bus when HREADY was high, and one cycle later of the data bus. HMASTER told split-capable slaves which master was on the bus. A slow slave could answer **SPLIT** (the arbiter masks that master until the slave raises its HSPLITx bit) or **RETRY** (the master tries again), freeing the bus for others. The protocol worked, but only one transfer could be in flight in the whole system, and verifying SPLIT/RETRY corner cases was notoriously hard.

Multi-layer AHB (AHB-Lite interconnect matrix)

AHB-Lite removed arbitration from the protocol. Each master gets its own **layer**, i.e. its own AHB-Lite bus. An **interconnect matrix** sits between masters and slaves and contains, per slave port, an arbiter and a multiplexer, and per master port, an input stage that can hold (buffer) one address phase while the target slave is busy with another master:



Multi-layer AHB: masters talk to different slaves in parallel; only masters that want the same slave in the same cycle are arbitrated.

From the master's point of view, losing arbitration just looks like wait states (the input stage holds HREADY low). CPU instruction fetches from flash and DMA transfers to SRAM therefore proceed simultaneously, which is why Cortex-M based MCUs call this a **bus matrix**. Arbitration is usually fixed priority or round-robin per slave port, and the matrix is generated by a configuration tool.

Locked transfers: HMASTLOCK

HMASTLOCK marks a sequence (typically a read-modify-write for a semaphore, as produced by an older CPU's SWP instruction) that must be **indivisible**. In a multi-layer interconnect, once a master starts a locked sequence to a slave, the arbiter for that slave must not grant any other master until the lock is released. The

specification recommends that a master insert an IDLE transfer after a locked sequence so the interconnect can release the lock cleanly. Modern systems replace locking with **exclusive access** (AHB5 HEXCL, AXI exclusives, Chapter 7), which does not block other masters.

6.8 The AHB-to-APB bridge

The bridge is an AHB-Lite slave and the single APB requester (Chapter 5). Its FSM is a good exercise in pipeline thinking:

AHB	:		addr A (read)		data A	
bridge:			capture A	->	APB SETUP	->
HREADYOUT:	1		0		0	->
						1
AHB	:		addr B (write)		data B (HWDATA valid)	
bridge:			capture B	->	wait for HWDATA	->
HREADYOUT:	1		0		0	->
						1

Read: APB SETUP can start right after the address phase. Write: PWDATA only exists in the AHB data phase, so SETUP starts one cycle later (unless the bridge posts the write).

- HSIZE and HADDR[1:0] become PSTRB for APB4 completers; HPROT[1] and HNONSEC map to PPROT[0] and PPROT[1]. HPROT[0] = 0 (opcode fetch) maps to PPROT[2] = 1 (instruction).
- PSLVERR becomes the two-cycle HRESP ERROR.
- Bursts are split into single APB transfers; HTRANS = BUSY cycles are answered OKAY and ignored.
- If PCLK is HCLK divided by N, the bridge advances its APB FSM only on the PCLK enable, and the AHB wait-state count scales with N.

6.9 Hands-on: AHB-Lite SRAM slave with wait states and ERROR

The slave below is a behavioural 1KB SRAM with the features that matter for protocol learning: it samples the address phase only when HREADY is high, supports byte and halfword writes from HSIZE and HADDR[1:0], inserts a per-transfer number of wait states (a demo input *ws*, standing in for, say, a flash or a slow memory), and answers out-of-range addresses with the two-cycle ERROR. A real SRAM slave would present the read address to a synchronous macro in the address phase; here an array read keeps the focus on the protocol.

```
// AHB-Lite SRAM slave (behavioural array): wait states, byte/halfword writes,
// and the two-cycle ERROR response for out-of-range addresses.
module ahb_sram #(parameter int BYTES = 1024) (
    input logic hclk, hresetn,
    input logic hsel, hready, hwrite,
    input logic [31:0] haddr,
    input logic [1:0] htrans, // 00 IDLE 01 BUSY 10 NONSEQ 11 SEQ
    input logic [2:0] hsize, hburst,
    input logic [3:0] hprot,
    input logic [31:0] hwdata,
    input logic [1:0] ws, // wait states for this transfer (demo knob)
    output logic [31:0] hrdata,
    output logic hreadyout, hresp
);
    logic [31:0] mem [BYTES/4];
    // ---- address-phase register: captured only when HREADY is high ----
    logic dp_valid, dp_write;
    logic [31:0] dp_addr;
    logic [2:0] dp_size;
    logic [1:0] cnt; // remaining wait states
    logic [1:0] eph; // 0 none, 1 first ERROR cycle, 2 second

    wire xfer = hsel && htrans[1]; // NONSEQ or SEQ (IDLE/BUSY: no transfer)
    always_ff @(posedge hclk or negedge hresetn)
        if (!hresetn) begin dp_valid <= 0; cnt <= 0; eph <= 0; end
```



```

else if (hready) begin // previous data phase done: sample address
    dp_valid <= xfer; dp_write <= hwrite; dp_addr <= haddr; dp_size <= hsize;
    eph <= (xfer && haddr >= BYTES) ? 2'd1 : 2'd0;
    cnt <= (xfer && haddr < BYTES) ? ws : 2'd0;
end else if (eph == 1) eph <= 2; // ERROR cycle 1 -> cycle 2
else if (cnt != 0) cnt <= cnt - 1;

assign hreadyout = !dp_valid || (eph == 1 ? 1'b0 : eph == 2 ? 1'b1 : cnt == 0);
assign hresp = dp_valid && eph != 0; // 1 = ERROR, held for both cycles

// byte-lane enables from HSIZE and the low address bits (little-endian)
logic [3:0] be;
wire [1:0] lo = dp_addr[1:0];
always_comb case (dp_size)
    3'd0: be = 4'b0001 << lo;
    3'd1: be = (lo >= 2) ? 4'b1100 : 4'b0011;
    default: be = 4'b1111;
endcase

wire [7:0] widx = dp_addr[$clog2(BYTES)-1:2];
always_ff @(posedge hclk) // write in the last data-phase cycle
    if (dp_valid && dp_write && eph == 0 && hreadyout)
        for (int b = 0; b < 4; b++) if (be[b]) mem[widx][8*b +: 8] <= hwddata[8*b +: 8];

assign hrdata = (dp_valid && !dp_write && eph == 0) ? mem[widx] : 32'h0;
endmodule

```

ahb_sram: AHB-Lite slave with a registered address phase, wait-state counter, byte lanes and two-cycle ERROR.

The testbench is a **pipelined** master: a list of transfers is walked by a clocked process that, whenever HREADY is high, moves the current address-phase transfer into the data phase (driving its HWDATA) and drives the next address. This is the minimum correct structure for any AHB master model.

```

module tb;
    logic hclk = 0, hresetn = 0;
    always #5 hclk = ~hclk;
    logic hsel = 1, hwrite = 0; logic [31:0] haddr = 0, hwddata = 0;
    logic [1:0] htrans = 0, ws = 0; logic [2:0] hsize = 2, hburst = 0;
    logic [3:0] hprot = 4'b0011;
    logic [31:0] hrdata; logic hreadyout, hresp, hready;
    assign hready = hreadyout; // single slave: HREADY = HREADYOUT
    ahb_sram dut (.*)

    // transfer list (parallel arrays): htrans, hburst, hsize, hwrite, ws, haddr, wdata
    logic [1:0] qt [32], qws [32]; logic [2:0] qb [32], qs [32]; logic qw [32];
    logic [31:0] qa [32], qd [32];
    localparam IDLE = 2'b00, BUSY = 2'b01, NSEQ = 2'b10, SEQ = 2'b11;
    localparam SINGLE = 3'b000, INCR4 = 3'b011;
    int nq = 0, ai = 0, cyc = 0, n_err = 0;
    logic go = 0;
    logic dp_rd = 0; logic [31:0] dp_a; // TB copy of the data phase

    // pipelined driver: when HREADY is high the address phase on the bus is
    // accepted, its write data moves to HWDATA and the next address is driven
    always @(posedge hclk) if (go && hready && ai < nq) begin
        hwddata <= qw[ai] ? qd[ai] : 32'h0;
        dp_rd <= (qt[ai] >= NSEQ) && !qw[ai]; dp_a <= qa[ai];
        ai = ai + 1;
        if (ai < nq) begin
            htrans <= qt[ai]; hburst <= qb[ai]; hsize <= qs[ai];
            hwrite <= qw[ai]; ws <= qws[ai]; haddr <= qa[ai];

```

```

    end else begin htrans <= IDLE; hwrite <= 0; end
end

function string tn(input logic [1:0] t);
    return t == 0 ? "IDLE " : t == 1 ? "BUSY " : t == 2 ? "NONSEQ" : "SEQ ";
endfunction

always @(posedge hclk) if (go) begin
    cyc++;
    $write("%3d %s %03h %s %s %0d | %08h %1b %s", cyc, tn(htrans), haddr[11:0],
        hwrite ? "W" : "R", hburst == INCR4 ? "INCR4 " : "SINGLE", hsize, hwddata,
        hreadyout, hresp ? "ERR " : "OKAY");
    if (hready && dp_rd && !hresp) $display(" HRDATA=%08h", hrdata); else $display("");
    if (hresp) n_err++;
end

task automatic add(input logic [1:0] t, input logic [2:0] b, s, input logic w,
    input logic [1:0] w_s, input logic [31:0] a, d);
    qt[nq] = t; qb[nq] = b; qs[nq] = s; qw[nq] = w; qws[nq] = w_s; qa[nq] = a; qd[nq] = d;
    nq++;
endtask

initial begin
    add(IDLE, SINGLE, 2, 0, 0, 32'h0, 32'h0);
    add(NSEQ, SINGLE, 2, 1, 0, 32'h00, 32'hDEADBEEF);
    add(NSEQ, INCR4, 2, 1, 0, 32'h10, 32'hA0A0A0A0);
    add(SEQ, INCR4, 2, 1, 0, 32'h14, 32'hA1A1A1A1);
    add(SEQ, INCR4, 2, 1, 0, 32'h18, 32'hA2A2A2A2);
    add(SEQ, INCR4, 2, 1, 0, 32'h1C, 32'hA3A3A3A3);
    add(NSEQ, INCR4, 2, 0, 1, 32'h10, 32'h0); // read burst, 1 wait/beat
    add(SEQ, INCR4, 2, 0, 1, 32'h14, 32'h0);
    add(BUSY, INCR4, 2, 0, 1, 32'h18, 32'h0); // master inserts BUSY
    add(SEQ, INCR4, 2, 0, 1, 32'h18, 32'h0);
    add(SEQ, INCR4, 2, 0, 1, 32'h1C, 32'h0);
    add(NSEQ, SINGLE, 0, 1, 0, 32'h01, 32'h00005500); // byte write, lane 1
    add(NSEQ, SINGLE, 2, 0, 0, 32'h00, 32'h0);
    add(NSEQ, SINGLE, 2, 0, 0, 32'h800, 32'h0); // out of range
    add(IDLE, SINGLE, 2, 0, 0, 32'h0, 32'h0);
    repeat (2) @(posedge hclk); hresetn <= 1;
    $display("cyc HTRANS ADR D HBURST SZ | HWDATA RDY RESP");
    @(posedge hclk); go <= 1;
    wait (ai >= nq); repeat (2) @(posedge hclk);
    $display("error-response cycles seen = %0d", n_err);
    $finish;
end
endmodule

```

Testbench: pipelined AHB-Lite master replaying a transfer list (single write, INCR4 write, INCR4 read with BUSY, byte write, error).

```

cyc HTRANS ADR D HBURST SZ | HWDATA RDY RESP
1 IDLE 000 R SINGLE 2 | 00000000 1 OKAY
2 NONSEQ 000 W SINGLE 2 | 00000000 1 OKAY
3 NONSEQ 010 W INCR4 2 | deadbeef 1 OKAY
4 SEQ 014 W INCR4 2 | a0a0a0a0 1 OKAY
5 SEQ 018 W INCR4 2 | a1a1a1a1 1 OKAY
6 SEQ 01c W INCR4 2 | a2a2a2a2 1 OKAY
7 NONSEQ 010 R INCR4 2 | a3a3a3a3 1 OKAY
8 SEQ 014 R INCR4 2 | 00000000 0 OKAY
9 SEQ 014 R INCR4 2 | 00000000 1 OKAY HRDATA=a0a0a0a0
10 BUSY 018 R INCR4 2 | 00000000 0 OKAY
11 BUSY 018 R INCR4 2 | 00000000 1 OKAY HRDATA=a1a1a1a1
12 SEQ 018 R INCR4 2 | 00000000 1 OKAY

```

```

13 SEQ    01c R INCR4  2 | 00000000 0 OKAY
14 SEQ    01c R INCR4  2 | 00000000 1 OKAY HRDATA=a2a2a2a2
15 NONSEQ 001 W SINGLE 0 | 00000000 0 OKAY
16 NONSEQ 001 W SINGLE 0 | 00000000 1 OKAY HRDATA=a3a3a3a3
17 NONSEQ 000 R SINGLE 2 | 00005500 1 OKAY
18 NONSEQ 800 R SINGLE 2 | 00000000 1 OKAY HRDATA=dead55ef
19 IDLE    000 R SINGLE 2 | 00000000 0 ERR
20 IDLE    000 R SINGLE 2 | 00000000 1 ERR
21 IDLE    000 R SINGLE 2 | 00000000 1 OKAY
22 IDLE    000 R SINGLE 2 | 00000000 1 OKAY
error-response cycles seen = 2

```

Real Icarus Verilog output. Each line is one HCLK cycle: address-phase signals on the left, data-phase signals on the right, and read data printed when a read data phase completes.

What to look for in the trace:

- **Pipelining** (cycles 2-7): the address of each INCR4 beat is on HADDR while the previous beat's data is on HWDATA, e.g. in cycle 4 the SEQ address 0x014 appears with data 0xA0A0A0A0 of beat 0x010.
- **Wait states** (cycles 8-14): the read burst was issued with one wait state per beat. HREADY goes low every other cycle and the master holds the next address (SEQ 0x014 appears twice).
- **BUSY** (cycles 10-11): the master inserted a BUSY in the middle of the read burst. It carries the next address (0x018) but no transfer; in cycle 12 the SEQ 0x018 follows with zero waits for the BUSY 'data phase'.
- **Byte write** (cycles 15-17): HSIZE = 0 at address 0x001 writes lane 1 only (HWDATA = 0x00005500), and the read of 0x000 in cycle 18 returns 0xDEAD55EF: the other three bytes are untouched.
- **ERROR** (cycles 18-20): 0x800 is outside the 1KB SRAM. The data phase shows HRESP = ERR with HREADY = 0, then ERR with HREADY = 1, exactly the two-cycle response.

PITFALL: SAMPLING THE ADDRESS WHEN HREADY IS LOW

If `ahb_sram` captured the address phase with `if (hsel)` instead of `if (hready)`, then during the wait states of cycle 8 it would re-capture SEQ 0x014 every cycle and restart its wait counter forever, or, in a multi-slave system, capture an address that belongs to a transfer still waiting for a different slave. Every AHB slave must qualify its address register with HREADY.

PITFALL: READ-AFTER-WRITE HAZARDS IN ZERO-WAIT SRAM SLAVES

A zero-wait SRAM slave presents read addresses to the macro in the address phase but can only write in the data phase. A write to X immediately followed by a read of X therefore reads the macro before the write lands. Real designs keep a one-entry **write buffer** and forward its data on an address match, or insert one wait state on the conflicting read. Directed tests for write-then-read to the same address catch this; random tests rarely do.

6.10 Verification and integration notes

- **Protocol assertions:** address/control stable while HREADY is low (except the permitted HTRANS changes); SEQ addresses follow the burst rules; no 1KB crossing; HRESP two-cycle shape; BUSY only inside bursts; slaves answer IDLE/BUSY with zero-wait OKAY.
- **Coverage:** all HBURST x HSIZE combinations the IP supports, BUSY inside bursts, wait states on the first and last beat, early burst termination, back-to-back reads and writes to the same address.
- **Endianness:** AHB is little-endian in practice; big-endian legacy systems used BE-32 or BE-8 byte-lane mapping, which the AHB5 spec lists as an interface property. Check the lane mapping of every bridge.
- **Timing:** HREADY is a global, combinational signal that returns from the selected slave to all slaves and the master; on a large multi-slave bus it is often the critical path. Keep HREADYOUT registered in slaves and limit the number of slaves per layer.

Team	AHB responsibilities
RTL	Slave pipeline registers, HREADY qualification, wait-state and error logic, bus matrix configuration

Team	AHB responsibilities
DV	AHB VIP (master, slave, monitor), protocol assertions, burst and error coverage, multi-master contention tests
PD/STA	HREADY and HRDATA mux paths, fan-out of HADDR/HWDATA
Firmware	Memory map, handling of bus faults, exclusive/lock usage

6.11 Summary

- AHB pipelines the address phase of one transfer with the data phase of the previous one, reaching one beat per cycle.
- AHB-Lite is single-master with 1-bit HRESP; multiple masters use a multi-layer bus matrix; AHB5 adds security (HNONSEC), exclusives and extended memory attributes.
- HTRANS (IDLE/BUSY/NONSEQ/SEQ), HBURST (SINGLE, INCR, INCRx, WRAPx) and HSIZE describe every transfer; bursts must not cross 1KB.
- Slaves stall with HREADYOUT low, must sample address phases only when HREADY is high, and signal ERROR over two cycles so the master can cancel the pipelined next transfer.
- The verified SRAM slave showed pipelined bursts, BUSY, wait states, byte writes and the two-cycle ERROR.

6.12 Exercises

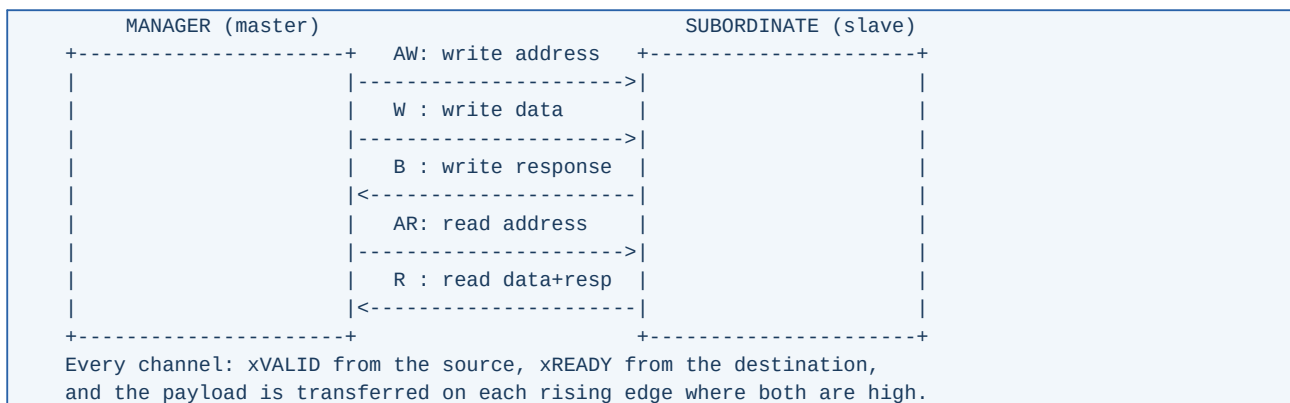
- Draw the timing diagram of a WRAP4 read of words starting at 0x2C with one wait state on the second beat. List the addresses and the cycle in which each HRDATA is sampled.
- Extend `ahb_sram` with a one-entry write buffer so that reads can be served with zero wait states from a synchronous SRAM model, and write a test that proves the read-after-write hazard is handled.
- A DMA engine must copy 3000 bytes starting at address 0x0000_03F0 using INCR16 bursts of words. How many bursts, of what types, does it issue so that no burst crosses 1KB?
- Why is the ERROR response two cycles long but the OKAY response only one? What would break if ERROR were a single cycle?
- Sketch the input stage of a multi-layer AHB interconnect: what must it store, and what does it drive on HREADYOUT toward its master when the target slave is busy with another layer?
- Compute the maximum sustained bandwidth of a 32-bit AHB-Lite bus at 150 MHz reading from a flash that needs 3 wait states per NONSEQ and 0 per SEQ, using INCR8 bursts.

7. AMBA AXI4 in Depth

AXI (Advanced eXtensible Interface) is the protocol of the SoC backbone. CPU clusters, GPUs, NPU, DMA engines, PCIe and USB controllers, display engines and DDR controllers all speak AXI or one of its descendants, and every major interconnect (Arm CoreLink NIC/NI, Arteris FlexNoC, Synopsys DesignWare, Xilinx SmartConnect) is built around it. AXI replaces AHB's single shared pipeline with **five independent channels**, each with its own VALID/READY handshake, and adds **transaction IDs** so that many transactions can be in flight and complete out of order. That is what lets a single AXI port keep a DDR controller with 100+ ns latency busy.

This chapter is deliberately thorough, because AXI is the protocol you will debug most often: channels and every signal, the handshake dependency rules, burst address arithmetic (verified against a Python model), the 4KB rule, narrow and unaligned transfers, ordering and IDs, exclusive access, memory types, responses, the differences between AXI3, AXI4 and AXI5, interconnect components and performance math. It ends with an AXI4 memory slave that handles FIXED, INCR and WRAP bursts under a randomized testbench.

7.1 Five channels



The five AXI channels. Writes use AW, W and B; reads use AR and R. The channels are independent: no channel shares wires with another.

A write is one AW transfer, one or more W transfers (the beats, the last marked WLAST) and one B transfer. A read is one AR transfer and one or more R transfers (the last marked RLAST), each R beat carrying its own response. Because the address and data of a write travel on different channels, write data may even arrive **before** its address.

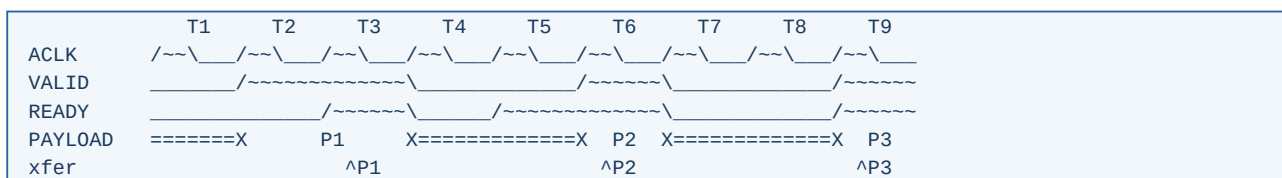
7.2 Signal reference

Channel	Signals (AXI4)	Notes
AW	AWID, AWADDR, AWLEN[7:0], AWSIZE[2:0], AWBURST[1:0], AWLOCK, AWCACHE[3:0], AWPROT[2:0], AWQOS[3:0], AWREGION[3:0], AWUSER, AWVALID, AWREADY	One transfer per write burst
W	WDATA,WSTRB, WLAST, WUSER, WVALID, WREADY	No WID in AXI4: write data must be in AW order
B	BID, BRESP[1:0], BUSER, BVALID, BREADY	One response per burst, after the last beat
AR	ARID, ARADDR, ARLEN, ARSIZE, ARBURST, ARLOCK, ARCACHE, ARPROT, ARQOS, ARREGION, ARUSER, ARVALID, ARREADY	One transfer per read burst
R	RID, RDATA, RRESP[1:0], RLAST, RUSER, RVALID, RREADY	One response per beat
global	ACLK, ARESETn	All signals sampled on ACLK rising edge; VALIDs low during reset

Field	Encoding
AxLEN	Beats - 1: 0..255 for INCR (1-256 beats); FIXED max 16 beats; WRAP 2, 4, 8 or 16 beats
AxSIZE	Bytes per beat = 2^{AxSIZE} (1 to 128 bytes); must not exceed the data bus width
AxBURST	00 FIXED, 01 INCR, 10 WRAP, 11 reserved
AxLOCK	AXI4: 0 normal, 1 exclusive (AXI3 had 2 bits incl. locked)
AxCACHE	[0] Bufferable, [1] Modifiable, [2]/[3] allocate hints (Section 7.9)
AxPROT	[0] privileged, [1] non-secure, [2] instruction
AxQOS	4-bit priority hint; higher value = more important (by convention)
AxREGION	Up to 16 address regions for one physical interface, so a slave can avoid its own decoder
xRESP	00 OKAY, 01 EXOKAY, 10 SLVERR, 11 DECERR
WSTRB	One bit per byte lane of WDATA; 1 = lane holds valid data

7.3 The handshake and its dependency rules

Every channel uses the same two-wire handshake (Chapter 3): the source asserts **VALID** when payload is available, the destination asserts **READY** when it can accept, and the transfer happens on the rising edge where both are high. Three orderings are legal:



P1: VALID before READY (source waits, payload held). P2: READY before VALID (destination waiting, transfer as soon as VALID rises). P3: both in the same cycle. The transfer happens at the end of the cycle marked ^.

THE TWO RULES THAT PREVENT DEADLOCK

- 1. A source must not wait for READY before asserting VALID.** (Otherwise two blocks that each wait for the other deadlock.)
- 2. Once VALID is asserted it must stay asserted, with stable payload, until the handshake completes.** READY, in contrast, may be asserted and deasserted freely before VALID arrives, and a destination **may** wait for VALID before asserting READY.

On top of the per-channel rule, the specification defines **dependencies between channels**. The spec draws them as arrows; the ASCII version below uses a single arrow for 'must wait for' and a double arrow for 'must wait for both':

READ TRANSACTION		
ARVALID --+		(slave may wait for ARVALID before ARREADY)
ARREADY --+==> RVALID		RVALID only after the AR handshake
RREADY		(master may wait for RVALID before RREADY)
WRITE TRANSACTION (AXI4)		
AWVALID WVALID		master must NOT wait for AWREADY or WREADY before asserting AWVALID/WVALID
AWREADY WREADY		slave MAY wait for AWVALID and/or WVALID before asserting AWREADY or WREADY
AWVALID --+		
AWREADY --+		
WVALID --+==> BVALID		BVALID only after the AW handshake AND the handshake of the LAST W beat (WLAST)
WREADY --+		(master may wait for BVALID before BREADY)
BREADY		

Channel dependencies. The AW handshake as a condition for BVALID was added in AXI4; AXI3 required only the last W beat.

PITFALL: THE CLASSIC AXI DEADLOCKS

A master that waits for AWREADY before driving WVALID, attached to a slave that waits for WVALID before driving AWREADY: legal for the slave, illegal for the master; the system hangs on the first write.

A slave (or interconnect) that combinational makes READY depend on VALID and a master that makes VALID depend on READY: a combinational loop, not just a deadlock.

A bridge that accepts AW for write A, then waits for W of write B because it arbitrates channels independently: W data must follow AW order, and the bridge must never reorder them.

A practical consequence: an AXI slave must be able to accept W beats that arrive before their AW (or apply backpressure with WREADY low until AW arrives, which is what the example slave in Section 7.14 does), and a master must be ready to present all W beats regardless of when AWREADY rises.

7.4 Burst types and address calculation

AXI bursts come in three flavours, set by AxBURST:

- **FIXED**: every beat uses the same address. Used for FIFOs and peripheral data registers (a UART TX register written 16 times).
- **INCR**: the address increments by the transfer size each beat. The normal type for memory.
- **WRAP**: incrementing, but wrapping at an aligned boundary of (beats x size) bytes. Used for critical-word-first cache line fills. Start address must be aligned to AxSIZE and the length must be 2, 4, 8 or 16.

The specification gives the address of every beat with a small set of formulas. With beats numbered $N = 1, 2, \dots$ and Data_Bus_Bytes the width of the data bus in bytes:

```

Number_Bytes    = 2 ^ AxSIZE
Burst_Length    = AxLEN + 1

Aligned_Address = INT(Start_Address / Number_Bytes) x Number_Bytes
Address_1      = Start_Address                               (first beat)
Address_N      = Aligned_Address + (N - 1) x Number_Bytes   (INCR, N >= 2)
Wrap_Boundary  = INT(Start_Address / (Number_Bytes x Burst_Length))
                x (Number_Bytes x Burst_Length)

WRAP: if Address_N = Wrap_Boundary + Number_Bytes x Burst_Length
    then Address_N = Wrap_Boundary, and subsequent beats continue from it

Lower_Byte_Lane = Address_N - INT(Address_N / Data_Bus_Bytes) x Data_Bus_Bytes
Upper_Byte_Lane = Lower_Byte_Lane + Number_Bytes - 1        (aligned beats)

first beat:      Aligned_Address + Number_Bytes - 1
                - INT(Start_Address / Data_Bus_Bytes) x Data_Bus_Bytes

```

AXI beat address and byte-lane equations (paraphrased from the AXI specification). FIXED bursts repeat Address_1 and its lanes.

A worked example: a WRAP4 burst of 4-byte beats starting at 0x38. Number_Bytes = 4, Burst_Length = 4, so the wrap container is 16 bytes and Wrap_Boundary = 0x30. The addresses are 0x38, 0x3C, then 0x40 = 0x30 + 16 wraps to **0x30**, then 0x34. The critical word 0x38 comes first, and the whole 16-byte line 0x30-0x3F is fetched.

Verified: a burst-address generator against a Python model

The combinational module below computes, for beat n of a burst, the beat address, the active byte lanes on a 64-bit bus and a flag for an illegal 4KB crossing. Note how simple the hardware form of WRAP is: since the container is a power of two, the wrapped address is the boundary OR'ed with the low bits of the linear address.


```

// AXI4 beat-address and byte-lane generator (combinational).
// For beat n (0-based) of a burst: address, active byte lanes, 4KB-crossing flag.
module axi_addr_gen #(parameter int DB = 8) (
    input  logic [31:0]  start,          // AxADDR
    input  logic [7:0]   len,            // AxLEN  (beats - 1)
    input  logic [2:0]   size,           // AxSIZE (bytes = 2**size <= DB)
    input  logic [1:0]   burst,          // 0 FIXED, 1 INCR, 2 WRAP
    input  logic [7:0]   n,              // beat index
    output logic [31:0]  addr,
    output logic [DB-1:0] lanes,         // byte lanes that carry data
    output logic         cross4k        // INCR burst crosses 4KB: illegal
);
    logic [31:0] nbytes, aligned, total, wbound, linear, last, lo, hi;
    always_comb begin
        nbytes = 32'd1 << size;
        aligned = start & ~(nbytes - 1);          // start rounded down to size
        total   = nbytes * (len + 1);              // bytes in burst (wrap container)
        wbound  = start & ~(total - 1);           // wrap boundary (WRAP only)
        linear  = aligned + n * nbytes;
        case (burst)
            2'd0:   addr = start;                  // FIXED
            2'd2:   addr = wbound | (linear & (total - 1)); // WRAP
            default: addr = (n == 0) ? start : linear; // INCR
        endcase
        lo = addr % DB;                            // first valid lane
        hi = (addr & ~(nbytes - 1)) % DB + nbytes - 1; // end of size-aligned container
        for (int b = 0; b < DB; b++) lanes[b] = (b >= lo) && (b <= hi);
        last   = aligned + len * nbytes;
        cross4k = (burst == 2'd1) && ((start >> 12) != (last >> 12));
    end
endmodule

```

axi_addr_gen: beat address, byte lanes and 4KB check. The same module is reused inside the AXI slave of Section 7.14.

The testbench sweeps FIXED, INCR and WRAP bursts, all sizes from 1 to 8 bytes, lengths from 1 to 256 beats and aligned/unaligned start addresses, writing every beat to a file. The Python model was written **independently from the specification's pseudo-code** (beat numbering from 1, the explicit 'wrap when Address_N reaches the boundary' test and the separate first-beat lane formula) and compares every line.

```

module tb;
    logic [31:0] start, addr; logic [7:0] len, n; logic [2:0] size; logic [1:0] burst;
    logic [7:0] lanes; logic cross4k;
    axi_addr_gen #(.DB(8)) dut (.*);          // 64-bit data bus
    int fd, lines = 0; bit ok;
    int starts [8], lens [6];
    initial begin
        starts[0] = 'h0; starts[1] = 'h3; starts[2] = 'h6; starts[3] = 'h14;
        starts[4] = 'h38; starts[5] = 'h3E; starts[6] = 'hFF9; starts[7] = 'h1234;
        lens[0] = 0; lens[1] = 1; lens[2] = 3; lens[3] = 7; lens[4] = 15; lens[5] = 255;
        fd = $fopen("axi_addr_rtl.txt", "w");
        for (int bu = 0; bu < 3; bu++)
            for (int sz = 0; sz < 4; sz++)
                for (int li = 0; li < 6; li++)
                    for (int si = 0; si < 8; si++) begin
                        burst = bu; size = sz; len = lens[li]; start = starts[si];
                        ok = 1;
                        if (bu == 2) begin          // WRAP: len 2/4/8/16, aligned start
                            ok = (len == 1 || len == 3 || len == 7 || len == 15);
                            start = start & ~((1 << sz) - 1);
                        end
                    end
            end
    end
endmodule

```



```

        if (bu == 0 && len > 15) ok = 0;          // FIXED: at most 16 beats
        if (ok) for (int k = 0; k <= len; k++) begin
            n = k; #1;
            $fdisplay(fd, "%0d %0d %0d %0h %0d %0h %02h %0d",
                        bu, sz, len, start, k, addr, lanes, cross4k);
            lines++;
        end
    end
endmodule

$fclose(fd);
$display("RTL beats written: %0d", lines);
// two worked examples for the text
burst = 2; size = 2; len = 3; start = 32'h38;
$write("WRAP4 x 4B @0x38 :");
for (int k = 0; k < 4; k++) begin n = k; #1 $write(" %02h/%08b", addr[7:0], lanes); end
$display("");
burst = 1; size = 2; len = 3; start = 32'h3E;
$write("INCR4 x 4B @0x3E :");
for (int k = 0; k < 4; k++) begin n = k; #1 $write(" %02h/%08b", addr[7:0], lanes); end
$display("");
burst = 1; size = 3; len = 15; start = 32'hFF8; n = 0; #1;
$display("INCR16 x 8B @0xFF8: crosses 4KB = %0d", cross4k);
end
endmodule

```

Sweep testbench and three worked examples.

```

# Python reference written directly from the AXI spec pseudo-code
# (Start_Address, Aligned_Address, Wrap_Boundary, Lower/Upper_Byte_Lane).
DB = 8
def beats(burst, size, length, start):
    nb, bl = 1 << size, length + 1
    aligned = (start // nb) * nb
    wrap_b = (start // (nb * bl)) * (nb * bl)
    wrapped = False
    for n in range(1, bl + 1):
        # spec numbers beats from 1
        if n == 1:
            a = start
            lower = start - (start // DB) * DB
            upper = aligned + (nb - 1) - (start // DB) * DB
        else:
            if burst == 0: a = start
            else:
                a = aligned + (n - 1) * nb
                if burst == 2:
                    if wrapped or a >= wrap_b + nb * bl:
                        wrapped = True; a = start + (n - 1) * nb - nb * bl
            if burst == 0:
                lower = start - (start // DB) * DB
                upper = aligned + (nb - 1) - (start // DB) * DB
            else:
                lower = a - (a // DB) * DB
                upper = lower + nb - 1
            lanes = sum(1 << b for b in range(lower, upper + 1))
            last = aligned + length * nb
            cross = int(burst == 1 and (start >> 12) != (last >> 12))
            yield n - 1, a, lanes, cross

bad = total = 0
for line in open("axi_addr_rtl.txt"):
    bu, sz, ln, st, k, a, lanes, cr = line.split()
    bu, sz, ln, k, cr = int(bu), int(sz), int(ln), int(k), int(cr)

```

```

st, a, lanes = int(st, 16), int(a, 16), int(lanes, 16)
exp = list(beats(bu, sz, ln, st))[k]
total += 1
if exp != (k, a, lanes, cr):
    bad += 1
    if bad < 5: print("MISMATCH", line.strip(), exp)
print("python model compared %d beats, mismatches = %d" % (total, bad))

```

Independent Python reference model that checks the RTL output file.

```

RTL beats written: 11136
WRAP4  x 4B @0x38 : 38/00001111 3c/11110000 30/00001111 34/11110000
INCR4  x 4B @0x3E : 3e/11000000 40/00001111 44/11110000 48/00001111
INCR16 x 8B @0xFF8: crosses 4KB = 1

```

Icarus Verilog output: worked examples as address/lanes (lane 0 is the rightmost bit).

```
python model compared 11136 beats, mismatches = 0
```

Python cross-check of all 11136 beats: zero mismatches.

The worked examples read as follows. WRAP4 at 0x38 on a 64-bit bus gives 38, 3C, 30, 34, alternating between the lower and upper word lanes. The unaligned INCR4 at 0x3E with 4-byte beats has a first beat that only uses lanes 6 and 7 (bytes 0x3E-0x3F): the rest of that 4-byte container, 0x3C-0x3D, lies before the start address. Subsequent beats are aligned. The INCR16 of 8-byte beats at 0xFF8 would cover 0xFF8-0x1077, crossing 0x1000, so it is flagged.

7.5 The 4KB boundary rule

A burst must not cross a 4KB address boundary. The reason is the same as AHB's 1KB rule, scaled up: the smallest region an interconnect decodes to a slave, and the smallest MMU page, is 4KB. A burst that crossed a boundary could start in one slave and end in another, or straddle two pages with different physical mappings. Because of the rule, an interconnect can route an entire burst using only its first address.

- For INCR the check is: `page(Start_Address) == page(Aligned_Address + AxLEN x Number_Bytes)`. With 256 beats x 16 bytes = 4096 bytes, only a perfectly 4KB-aligned maximal burst fits.
- WRAP bursts never cross, because their container is aligned and at most $16 \times 128 = 2048$ bytes.
- Masters (DMA engines, bridges from PCIe or USB) must split transfers at 4KB; a protocol checker should flag violations, and some interconnects split or reject them.

7.6 Narrow, unaligned and sparse transfers

A **narrow transfer** has `AxSIZE` smaller than the data bus: for example 2-byte beats on a 64-bit bus. The active lanes then move from beat to beat (0-1, 2-3, 4-5, 6-7, 0-1, ...), and the master must put each beat's data on the right lanes and set `WSTRB` accordingly. An **unaligned** transfer has a start address that is not a multiple of `AxSIZE`; only the first beat is affected, and `WSTRB` (writes) or the master's own lane selection (reads) discards the bytes below the start address. **Sparse** writes simply clear `WSTRB` bits anywhere; this is how a master writes a partial word, and how it 'cancels' the remainder of a burst, since AXI has no early termination: all `AxLEN + 1` beats must always be transferred.

PITFALL: WSTRB OUTSIDE THE LEGAL LANES

`WSTRB` may only be high for lanes inside the beat's legal byte lanes (the equations above). A slave that blindly writes every lane with `WSTRB = 1` corrupts neighbouring bytes when a buggy master drives strobes outside the transfer; robust slaves AND `WSTRB` with the computed lanes, as `axi4_mem` does.

7.7 IDs, ordering and outstanding transactions

Every AW and AR carries an ID (AWID, ARID), echoed on the corresponding B (BID) and R (RID). The ordering rules are:

- Transactions with the **same ID in the same direction** complete in the order they were issued. Read data for the same ARID returns in order; write responses for the same AWID return in order.
- Transactions with **different IDs** have no ordering relationship; the slave or interconnect may complete them in any order.
- **Reads and writes are never ordered with respect to each other**, even with the same ID. A master that must read what it just wrote waits for the B response first (or relies on a guarantee documented for a specific slave).
- In AXI4 the read data of different IDs may be **interleaved** beat by beat (RID changes between beats of different bursts); a slave declares its read data reordering depth.
- **Write data interleaving was removed in AXI4**: WID no longer exists, and W beats must arrive in the same order as their AW transfers. AXI3 allowed interleaving W beats of different AWIDs, which almost no slave supported.

IDs are also how the interconnect routes responses: when a crossbar merges several masters onto one slave port it **extends** the ID with the master's port number, and uses those extra bits to steer B and R back. A slave port behind a 4-master crossbar therefore sees IDs two bits wider than any master produces, which is why slave IP has a configurable ID width.

ORDERING MODELS IN PRACTICE

Many masters use a single ID (a simple DMA channel) and therefore get strict in-order completion for free. CPUs use several IDs so that a slow device read does not block a fast cache refill. The interconnect must keep the per-ID order even when the two transactions go to different slaves; the usual solution is to stall a new transaction whose ID already has outstanding transactions to a **different** slave (a 'single-slave-per-ID' rule), because responses from two slaves could otherwise return out of order.

7.8 Exclusive access

Exclusive access implements load-linked/store-conditional semantics (Arm LDREX/STREX, LDXR/STXR; RISC-V LR/SC maps onto the same mechanism) without locking the bus:

- The master issues an **exclusive read** (AxLOCK = 1). An **exclusive monitor** at the slave (or in the interconnect) records the address and the master's ID and returns EXOKAY.
- The master computes, then issues an **exclusive write** to the same address with the same ID, size and length.
- If no other master wrote the monitored location in between, the write happens and returns **EXOKAY**. Otherwise the write is **not** performed and returns **OKAY**, meaning 'exclusive failed'; software retries the sequence.
- Constraints: the burst must be at most 16 beats, the total size a power of two up to 128 bytes, and the address aligned to it.
- A slave without exclusive support answers the exclusive read with OKAY (not EXOKAY), telling the master exclusives are not supported there; its exclusive writes then always update memory and return OKAY, so software must not rely on them.

In coherent systems the monitor for cacheable memory lives in the CPU cluster (the local monitor plus the coherency protocol); the AXI exclusive monitor matters for non-cacheable shared memory, for example a mailbox between a CPU and a DSP.

7.9 Memory attributes: AxCACHE and AxPROT

AxCACHE tells the interconnect and slaves how a transaction may be handled. **Bufferable** (bit 0) allows an early write response from an intermediate point. **Modifiable** (bit 1, called Cacheable in AXI3) allows the transaction to be split, merged, widened or have its data prefetched; non-modifiable transactions must reach the destination exactly as issued, which is what device registers need. Bits 2 and 3 are allocation hints for caches. The AXI4 memory types are:

Memory type	ARCACHE	AWCACHE
Device Non-bufferable	0000	0000
Device Bufferable	0001	0001
Normal Non-cacheable Non-bufferable	0010	0010
Normal Non-cacheable Bufferable	0011	0011
Write-through No-allocate	1010	0110
Write-through Read-allocate	1110	0110
Write-through Write-allocate	1010	1110
Write-through Read and Write-allocate	1110	1110
Write-back No-allocate	1011	0111
Write-back Read-allocate	1111	0111
Write-back Write-allocate	1011	1111
Write-back Read and Write-allocate	1111	1111

AXI4 memory types (AxCACHE encodings from the AXI4 specification). For reads bit 2 is Allocate and bit 3 Other Allocate; for writes the roles are swapped.

AxPROT carries privilege (bit 0), **non-secure** (bit 1) and instruction (bit 2). TrustZone firewalls (for example a TZC-400 in front of DDR) check AxPROT[1] against per-region permissions and respond with an error (or read-as-zero) to violations.

7.10 Responses

xRESP	Name	Meaning
00	OKAY	Normal success; or failure of an exclusive access
01	EXOKAY	Exclusive access succeeded
10	SLVERR	The slave was reached but reports an error: unsupported size, FIFO overflow, read-only register, ECC uncorrectable error, firewall violation
11	DECERR	Decode error, normally generated by the interconnect's default slave: no slave at this address

Reads have a response per beat (a burst can mix OKAY and SLVERR), writes one response for the whole burst. Note that every beat of an erroring burst must still be transferred: a slave returning SLVERR on the first read beat still returns all AxLEN + 1 beats with RLAST on the last.

7.11 AXI3, AXI4 and AXI5

Feature	AXI3 (2003)	AXI4 (2010)	AXI5 (AMBA 5)
Max INCR burst	16 beats	256 beats	256 beats
Write interleaving	Allowed (WID)	Removed (no WID)	Removed
AxLOCK	2 bits (locked + exclusive)	1 bit (exclusive)	1 bit

Feature	AXI3 (2003)	AXI4 (2010)	AXI5 (AMBA 5)
QoS / REGION / USER	No	Yes	Yes
BVALID dependency	After last W	After last W and AW	Same as AXI4
AxCACHE[1]	Cacheable	Modifiable	Modifiable
New in AXI5	-	-	Atomic transactions (AWATOP), parity/check signals, poison, cache stashing, persistent CMOs, memory tagging (MTE), MPAM, wakeup signals, unique-ID indication, untranslated transactions, trace and loopback signals (most optional)

AXI5 features are enabled per interface through **interface properties** (for example Atomic_Transactions, Poison, Check_Type), so an 'AXI5' port may be almost identical to AXI4. Always read the interface property list of an IP, not just its protocol name. AMBA 5 also contains ACE5 and CHI, covered in Chapter 9.

7.12 Interconnect building blocks

Component	What it does	Design notes
Crossbar / switch	Connects M masters to S slaves; per-slave arbitration, per-master decode; extends IDs	Sparse connectivity saves area; QoS arbitration uses AxQOS
Default slave	Answers unmapped addresses with DECERR	Must still consume all W beats and return all R beats
Width converter (upsizer/downsizer)	Changes data width; splits or packs beats, rewrites AxLEN/AxSIZE and WSTRB	Downsizing a 256-beat burst may need splitting into several bursts; must respect modifiable bit
Clock converter	Crosses clock domains with async FIFOs on all five channels	Adds latency of a few cycles of each clock (Chapter 4)
Register slice	Pipeline stage on a channel (full or skid buffer) to break long paths	Adds one cycle latency; a full-throughput slice needs two entries
Protocol converter	AXI4 to AXI3/AXI4-Lite/AHB/APB	Splits bursts, serializes, handles ID mapping
Firewall / TZC	Security filtering by address and AxPROT	Returns SLVERR or DECERR or RAZ/WI
Exclusive monitor	Tracks exclusive reads per ID	Needed in front of non-coherent shared memory

7.13 Performance: bandwidth, latency and outstanding transactions

The raw bandwidth of one AXI data channel is width x frequency. A 128-bit bus at 1 GHz moves 16 bytes per cycle, 16 GB/s per direction; AXI's separate R and W channels mean reads and writes can each use that simultaneously. Achieved bandwidth is lower because of three effects:

- **Bubbles** between bursts or inside them (VALID or READY low).
- **Latency** not covered by enough outstanding transactions.
- **Narrow or short bursts**: a 4-byte access on a 16-byte bus uses 25% of the lanes; a single-beat burst pays the full address and response overhead for one beat.

The key sizing rule is **Little's law**: to sustain bandwidth BW with round-trip latency L, the number of bytes in flight must be BW x L, so the required number of outstanding transactions is:

$$\text{Outstanding} = \text{ceil}(\text{Bandwidth} \times \text{Latency} / \text{Bytes_per_transaction})$$

Example: 128-bit @ 1 GHz = 16 GB/s, read latency 150 ns, 64-byte bursts

$$16\text{e9 B/s} \times 150\text{e-9 s} = 2400 \text{ bytes in flight} \rightarrow 2400 / 64 = 37.5 \rightarrow 38$$

Example: 64-bit @ 800 MHz = 6.4 GB/s, latency 100 ns, 64-byte bursts

$$6.4\text{e9} \times 100\text{e-9} = 640 \text{ bytes} \rightarrow 10 \text{ outstanding transactions}$$

Outstanding depth = bandwidth-delay product / transaction size.

This is why a DMA engine with only 4 outstanding reads cannot saturate DDR no matter how wide its bus, and why every interconnect component on the path (crossbar, clock converter, width converter) must also be configured for enough **acceptance capability**. The number of IDs matters too: if all 38 transactions share one ID, the slave must return them in order, which prevents a DDR controller from reordering for page hits.

PERFORMANCE DEBUGGING CHECKLIST

Measure per channel: cycles with VALID & READY, VALID & !READY (slave backpressure) and !VALID (master starvation).

Plot outstanding transactions over time; a flat line at the configured limit means you are latency-bound.

Check burst length and size distributions: a 'bandwidth problem' is often a master issuing single-beat or narrow transfers.

Look for write-response stalls: a master that waits for B before the next AW has an effective outstanding depth of one.

7.14 Hands-on: an AXI4 memory slave with FIXED, INCR and WRAP bursts

`axi4_mem` is a 4KB, 32-bit AXI4 memory slave. The write and read paths are independent state machines, so a read can proceed while a write is in progress, but each path handles one burst at a time (no outstanding transactions, the simplest legal choice). Both paths reuse `axi_addr_gen` for beat addresses and byte lanes. Writes honour WSTRB AND the legal lanes, check that WLAST arrives exactly on the last beat (answering SLVERR otherwise) and echo AWID on BID. Reads echo ARID and generate RLAST.

```
// AXI4 memory slave, 32-bit data, 4KB. One write and one read burst in flight
// (independent channels); FIXED/INCR/WRAP via axi_addr_gen; IDs echoed on B/R.
module axi4_mem #(parameter int IDW = 4) (
    input logic          aclk, aresetn,
    // write address
    input logic [IDW-1:0] awid,   input logic [11:0] awaddr, input logic [7:0] awlen,
    input logic [2:0]      awsize, input logic [1:0]  awburst,
    input logic            awvalid, output logic awready,
    // write data
    input logic [31:0]      wdata, input logic [3:0]  wstrb,   input logic wlast,
    input logic            wvalid, output logic wready,
    // write response
    output logic [IDW-1:0] bid,   output logic [1:0]  bresp,
    output logic            bvalid, input logic bready,
    // read address
    input logic [IDW-1:0] arid,   input logic [11:0] araddr, input logic [7:0] arlen,
    input logic [2:0]      arsize, input logic [1:0]  arbust,
    input logic            arvalid, output logic arready,
    // read data
    output logic [IDW-1:0] rid,   output logic [31:0] rdata,  output logic [1:0] rresp,
    output logic            rlast, output logic rvalid, input logic rready
);
    logic [31:0] mem [1024];

    // ----- write channel FSM -----
```

```

typedef enum logic [1:0] {W_ADDR, W_DATA, W_RESP} wst_t;
wst_t ws;
logic [IDW-1:0] w_id; logic [31:0] w_start, w_addr; logic [7:0] w_len, w_n;
logic [2:0] w_size; logic [1:0] w_burst; logic w_err; logic [3:0] w_lanes;
logic w_x4k;
axi_addr_gen #(.DB(4)) wag (.start(w_start), .len(w_len), .size(w_size),
    .burst(w_burst), .n(w_n), .addr(w_addr), .lanes(w_lanes), .cross4k(w_x4k));

assign awready = (ws == W_ADDR);
assign wready = (ws == W_DATA);
assign bvalid = (ws == W_RESP);
assign bid = w_id;
assign bresp = w_err ? 2'b10 : 2'b00; // SLVERR : OKAY

always_ff @(posedge aclk or negedge aresetn)
    if (!aresetn) ws <= W_ADDR;
    else case (ws)
        W_ADDR: if (awvalid) begin // AW handshake
            w_id <= awid; w_start <= {20'h0, awaddr}; w_len <= awlen;
            w_size <= awsize; w_burst <= awburst; w_n <= 0; w_err <= 0; ws <= W_DATA;
        end
        W_DATA: if (wvalid) begin // W handshake: one beat
            for (int b = 0; b < 4; b++) // strobe AND legal lanes
                if (wstrb[b] && w_lanes[b]) mem[w_addr[11:2]][8*b +: 8] <= wdata[8*b +: 8];
            if (wlast != (w_n == w_len)) w_err <= 1; // WLAST protocol check
            w_n <= w_n + 1;
            if (w_n == w_len) ws <= W_RESP;
        end
        W_RESP: if (bready) ws <= W_ADDR; // B handshake
        default: ws <= W_ADDR;
    endcase

// ----- read channel FSM -----
logic r_busy; logic [IDW-1:0] r_id; logic [31:0] r_start, r_addr;
logic [7:0] r_len, r_n; logic [2:0] r_size; logic [1:0] r_burst;
logic [3:0] r_lanes; logic r_x4k;
axi_addr_gen #(.DB(4)) rag (.start(r_start), .len(r_len), .size(r_size),
    .burst(r_burst), .n(r_n), .addr(r_addr), .lanes(r_lanes), .cross4k(r_x4k));

assign arready = !r_busy;
assign rvalid = r_busy;
assign rid = r_id;
assign rdata = mem[r_addr[11:2]]; // whole word; master picks lanes
assign rresp = 2'b00;
assign rlast = r_busy && (r_n == r_len);

always_ff @(posedge aclk or negedge aresetn)
    if (!aresetn) r_busy <= 0;
    else if (!r_busy) begin
        if (arvalid) begin // AR handshake
            r_busy <= 1; r_id <= arid; r_start <= {20'h0, araddr}; r_len <= arlen;
            r_size <= arsize; r_burst <= arbust; r_n <= 0;
        end
        end else if (rready) begin // R handshake
            r_n <= r_n + 1;
            if (rlast) r_busy <= 0;
        end
    end
endmodule

```

axi4_mem: AXI4 slave with independent write/read FSMs, burst address generation and WLAST checking.

Look at the handshake choices: AWREADY is high only in W_ADDR, so the slave waits for the address before accepting data (WREADY is low until then). That is allowed: the **slave** may wait for AWVALID before asserting WREADY; only the **master** may not wait. BVALID rises after both the AW and the last W handshake, satisfying the AXI4 dependency.

The testbench master drives AW and W from two parallel threads (fork/join) with random gaps on WVALID, applies random backpressure on RREADY, keeps a byte-accurate shadow memory using its **own** beat-address function, and checks every byte of every read beat, RID and RLAST. After nine directed transactions it runs 300 random bursts of all types, sizes and lengths.

```
module tb;
  logic aclk = 0, aresetn = 0;
  always #5 aclk = ~aclk;
  logic [3:0] awid = 0, arid = 0, bid, rid; logic [11:0] awaddr = 0, araddr = 0;
  logic [7:0] awlen = 0, arlen = 0; logic [2:0] awsize = 0, arsize = 0;
  logic [1:0] awburst = 0, arbust = 0, bresp, rresp;
  logic awvalid = 0, awready, wvalid = 0, wready, wlast = 0, bvalid, bready = 0;
  logic arvalid = 0, arready, rvalid, rready = 0, rlast;
  logic [31:0] wdata = 0, rdata; logic [3:0] wstrb = 0;
  axi4_mem dut (.*));

  byte unsigned shadow [4096]; // reference memory
  int beats_chk = 0, errs = 0;
  logic verbose = 1;

  // TB's own beat-address model (independent of the DUT's generator)
  function automatic int beat_addr(int st, int len, int sz, int bu, int n);
    int nb = 1 << sz, tot = nb * (len + 1), al = st / nb * nb, wb = st / tot * tot;
    if (bu == 0) return st;
    if (bu == 1) return n == 0 ? st : al + n * nb;
    return wb + (st - wb + n * nb) % tot;
  endfunction

  string bn [3];
  initial begin bn[0] = "FIXED"; bn[1] = "INCR"; bn[2] = "WRAP"; end

  task automatic axi_write(int id, int a, int len, int sz, int bu, int seed,
    int bad_last = -1);
    int x = 0;
    fork
      begin // AW channel
        @(posedge aclk); awvalid <= 1; awid <= id; awaddr <= a; awlen <= len;
        awsize <= sz; awburst <= bu;
        do @(posedge aclk); while (!awready);
        awvalid <= 0;
      end
      for (int n = 0; n <= len; n++) begin // W channel (random gaps)
        int ba = beat_addr(a, len, sz, bu, n), nb = 1 << sz;
        logic [31:0] d = (seed + n + 1) * 32'h9E37_79B1;
        logic [3:0] s = 0;
        for (int b = ba % 4; b < (ba / nb * nb) % 4 + nb; b++) s[b] = 1;
        while ($urandom % 3 == 0) @(posedge aclk);
        wvalid <= 1; wdata <= d; wstrb <= s;
        wlast <= (bad_last >= 0) ? (n == bad_last) : (n == len);
        do @(posedge aclk); while (!wready);
        wvalid <= 0; wlast <= 0;
        for (int b = 0; b < 4; b++) if (s[b]) shadow[(ba & ~3) + b] = d[8*b+: 8];
      end
    join
    bready <= 1; do @(posedge aclk); while (!bvalid); bready <= 0;
  endtask
endmodule
```



```

    if (verbose) $display("WRITE id=%0d %-5s %2d x %0dB @%03h -> BID=%0d BRESP=%0s", id,
        bn[bu], len + 1, 1 << sz, 12'(a), bid, (bresp == 0) ? "OKAY" : "SLVERR");
endtask

task automatic axi_read(int id, int a, int len, int sz, int bu);
    @(posedge aclk); arvalid <= 1; arid <= id; araddr <= a; arlen <= len;
    arsize <= sz; arbust <= bu;
    do @(posedge aclk); while (!arready);
    arvalid <= 0;
    if (verbose) $write("READ id=%0d %-5s %2d x %0dB @%03h ->", id, bn[bu],
        len + 1, 1 << sz, 12'(a));
    for (int n = 0; n <= len; n++) begin
        int ba = beat_addr(a, len, sz, bu, n), nb = 1 << sz;
        rready <= ($random % 4 != 0); // random backpressure
        do begin @(posedge aclk); if (!rready) rready <= 1; end
        while (!(rvalid && rready));
        for (int b = ba % 4; b < (ba / nb * nb) % 4 + nb; b++)
            if (rdata[8*b+: 8] != shadow[(ba & ~3) + b]) errs++;
        if (rid != 4'(id) || rlast != (n == len)) errs++;
        beats_chk++;
        if (verbose) $write(" %03h:%08h", 12'(ba), rdata);
    end
    rready <= 0;
    if (verbose) $display(" RID=%0d", rid);
endtask

int bu, sz, len, a;
initial begin
    for (int i = 0; i < 1024; i++) dut.mem[i] = 0; // SRAM model starts zeroed
    repeat (3) @(posedge aclk); aresetn <= 1;
    axi_write(1, 'h100, 3, 2, 1, 10); // INCR4, 4-byte beats
    axi_read (2, 'h108, 3, 2, 2); // WRAP4 starting mid-block
    axi_write(3, 'h202, 3, 1, 1, 20); // narrow: 2-byte beats
    axi_read (4, 'h200, 1, 2, 1);
    axi_write(5, 'h3F2, 2, 2, 1, 30); // unaligned start
    axi_read (6, 'h3F0, 3, 2, 1);
    axi_write(7, 'h300, 3, 2, 0, 40); // FIXED: same address 4 times
    axi_read (8, 'h300, 0, 2, 1);
    axi_write(9, 'h400, 3, 2, 1, 50, 1); // WLAST on wrong beat
    verbose = 0;
    for (int i = 0; i < 300; i++) begin // random regression
        bu = $random % 3; sz = $random % 3;
        len = (bu == 2) ? (2 << ($random % 4)) - 1 : $random % 16;
        a = ($random % 3584) & ~((bu == 2) ? (1 << sz) - 1 : 0);
        if ($random % 2) axi_write(i % 16, a, len, sz, bu, i * 7);
        else
            axi_read (i % 16, a, len, sz, bu);
    end
    $display("read beats checked = %0d, mismatches = %0d", beats_chk, errs);
    $finish;
end
endmodule

```

AXI4 testbench master with shadow memory, random backpressure and a random regression.

```

WRITE id=1 INCR 4 x 4B @100 -> BID=1 BRESP=OKAY
READ id=2 WRAP 4 x 4B @108 -> 108:08d12dfd 10c:a708a7ae 100:cc623a9b 104:6a99b44c RID=2
WRITE id=3 INCR 4 x 2B @202 -> BID=3 BRESP=OKAY
READ id=4 INCR 2 x 4B @200 -> 200:fa8c0000 204:36fb7536 RID=4
WRITE id=5 INCR 3 x 4B @3f2 -> BID=5 BRESP=OKAY
READ id=6 INCR 4 x 4B @3f0 -> 3f0:28b70000 3f4:c6ef3620 3f8:6526afd1 3fc:00000000 RID=6
WRITE id=7 FIXED 4 x 4B @300 -> BID=7 BRESP=OKAY
READ id=8 INCR 1 x 4B @300 -> 300:3188ea6c RID=8
WRITE id=9 INCR 4 x 4B @400 -> BID=9 BRESP=SLVERR
read beats checked = 1197, mismatches = 0

```

Real Icarus Verilog output: directed transactions, then the random-regression summary.

- **WRAP4 at 0x108** returns 108, 10C, 100, 104: the critical word first, wrapping inside the 16-byte container 0x100-0x10F that the INCR4 write filled.
- **Narrow writes:** four 2-byte beats starting at 0x202 land in 0x202-0x209; the following read of 0x200 shows the low half-word of 0x200 untouched (0000) and the next word fully written.
- **Unaligned INCR at 0x3F2:** the first beat only writes bytes 0x3F2-0x3F3 (read back as 28b7 in the upper half of 0x3F0); the burst of 3 beats ends at 0x3FB, so 0x3FC stays zero.
- **FIXED** writes 4 beats to 0x300; the last beat's data survives.
- **Bad WLAST** (asserted on beat 2 of 4) returns BRESP = SLVERR.
- 1197 read beats of random traffic matched the shadow memory byte for byte.

PITFALL: RLAST, BID AND RID IN MULTI-OUTSTANDING SLAVES

This slave has one burst in flight per direction, so RID and BID are simply the stored IDs. As soon as a slave accepts several transactions it must keep a queue of {ID, length} per direction and must never return responses of the same ID out of order. The two most common bugs in hand-written AXI slaves are an RLAST that is off by one on single-beat bursts (AxLEN = 0) and a B response issued before the AW handshake when W arrives first.

7.15 Verification and ownership

- **Protocol checkers:** Arm provides AXI protocol assertions (historically the 'AXI protocol checker' SVA); commercial VIP from Cadence, Synopsys and Siemens include AXI monitors with hundreds of checks (stability, dependencies, 4KB, WRAP rules, exclusive rules, ID ordering).
- **Scoreboards** must model ordering per ID and handle out-of-order completion across IDs.
- **Performance verification** (latency histograms, bandwidth per master, QoS behaviour) is a separate activity, often done with traffic generators on an emulator or FPGA prototype.
- **Ownership:** the interconnect team (RTL + configuration) owns crossbars, converters and QoS; IP teams own their AXI ports; DV owns VIP integration and system traffic tests; the performance architecture team owns outstanding-depth and bandwidth budgets; PD owns register-slice placement on long AXI routes.

7.16 Summary

- AXI has five independent channels (AW, W, B, AR, R), each with a VALID/READY handshake: VALID must not wait for READY and must hold until the handshake.
- BVALID waits for AW and the last W; RVALID waits for AR. A slave may wait for VALID before READY, a master may not wait for READY before VALID.
- FIXED, INCR (up to 256 beats) and WRAP (2/4/8/16 beats) bursts follow exact address and lane equations; bursts never cross 4KB and cannot be terminated early.
- IDs order same-ID transactions per direction; different IDs may complete out of order; reads and writes are unordered; AXI4 has no write interleaving.
- Exclusives, AxCACHE memory types, AxPROT security and the four responses complete the protocol; AXI5 adds optional atomics, parity, poison, MTE and more.
- Outstanding transactions = bandwidth x latency / transaction size.

- The burst generator matched a Python model on 11136 beats and the AXI4 slave passed directed and random tests.

7.17 Exercises

- List the addresses and WSTRB values (64-bit bus) of an INCR burst with AWADDR = 0x1003, AWSIZE = 2, AWLEN = 3. Check your answer with the Python model.
- A master issues AR with ARID = 3 to a slow peripheral, then AR with ARID = 3 to DDR. What can the interconnect do, and what must it not do? What changes if the second read uses ARID = 4?
- Extend `axi4_mem` to accept up to four outstanding read bursts (an AR queue) and return them in order. Which testbench checks prove that ID order is preserved?
- A 256-bit AXI port at 1.2 GHz reads DDR with a 120 ns average latency using 128-byte bursts. How many outstanding reads are needed for 80% of peak bandwidth?
- Explain why a WRAP burst of 16 beats x 128 bytes can never cross a 4KB boundary, and write an SVA property that checks the 4KB rule for INCR bursts.
- Design (block diagram and FSM) a downsizer from a 128-bit AXI4 slave port to a 32-bit master port. How are AxLEN, AxSIZE, WSTRB and RLAST transformed, and what happens to a 256-beat INCR burst?

8. AXI4-Lite and AXI4-Stream

Full AXI4 is powerful but heavy. Two lighter members of the family cover the two most common special cases. **AXI4-Lite** is AXI4 stripped to single-beat register accesses: the standard control interface of IP blocks, especially in FPGA designs, where it plays the role that APB plays in ASICs. **AXI4-Stream** drops addresses altogether and becomes a point-to-point, flow-controlled data pipe with packet framing: the backbone of video, networking, DSP and DMA data paths. Both reuse AXI's VALID/READY handshake, so everything you learned in Chapter 7 about the handshake carries over.

8.1 AXI4-Lite: what is removed

Aspect	AXI4	AXI4-Lite
Burst length	1-256 beats	Always 1 (no AxCLEN, no xLAST)
Transfer size	Any AxCLEN up to the bus width	Always the full bus width (no AxCLEN); partial writes use WSTRB
Data width	8 to 1024 bits	32 or 64 bits
IDs	AWID/ARID, reordering	None: all transactions in order per direction (AXI5-Lite later allows optional IDs)
Exclusive access	AxLOCK, EXOKAY	Not supported
Memory attributes	AxCACHE, AxQOS, AxREGION	Not present: treated as Device Non-bufferable
Protection	AxPROT	AxPROT (kept, needed for security)
Responses	OKAY, EXOKAY, SLVERR, DECERR	OKAY, SLVERR, DECERR

The remaining signals are AWADDR, AWPROT, AWVALID, AWREADY; WDATA, WSTRB, WVALID, WREADY; BRESP, BVALID, BREADY; ARADDR, ARPROT, ARVALID, ARREADY; RDATA, RRESP, RVALID, RREADY: 19 signals plus clock and reset. An AXI4-Lite slave can be connected to an AXI4 master only through a **protocol converter** (or a master known to issue single-beat, full-width transactions), because the converter must split bursts and reflect IDs, which the Lite slave does not return.

AXI4-LITE VERSUS APB

Both are register buses. APB has a two-cycle minimum per access, one requester and no pipelining, and is the natural ASIC choice behind a bridge. AXI4-Lite keeps independent read and write channels and can overlap transactions, so it can sustain one access per cycle per direction, but costs more flops per port. FPGA ecosystems standardize on AXI4-Lite because their interconnect generators and IP catalogues are AXI-based; ASIC peripheral subsystems usually stay on APB.

8.2 Designing an AXI4-Lite register slave

The address and data of a write arrive on independent channels, in either order and possibly in different cycles. The slave has three legal strategies:

- **Wait for both.** Keep AWREADY and WREADY low until AWVALID and WVALID are both high, then raise both READYs for one cycle. Simple and legal (a slave may wait for VALID before READY), at the cost of one extra cycle of latency.
- **Capture independently.** Accept AW into an address register and W into a data register whenever each arrives, then perform the write when both registers are full. Higher throughput, more flops.
- **Skid buffers on every channel.** The high-performance version: READY is registered and never depends combinationaly on VALID, at the cost of two-entry buffers per channel.

After the write, BVALID must be raised and **held until BREADY**. While a B response is pending the slave must not accept another write unless it can queue its response. The read side is the same pattern: accept AR, raise

RVALID with RDATA/RRESP, hold both until RREADY, and only then accept the next AR (or buffer).

```
// Write path core of a 'capture independently' AXI4-Lite slave
assign awready = !aw_full && !bvalid; // no new write while B is pending
assign wready = !w_full && !bvalid;
always_ff @(posedge aclk) if (!aresetn) begin
    aw_full <= 0; w_full <= 0; bvalid <= 0;
end else begin
    if (awvalid && awready) begin aw_full <= 1; aw_q <= awaddr; end
    if (wvalid && wready) begin w_full <= 1; w_q <= wdata; s_q <= wstrb; end
    if (aw_full && w_full && !bvalid) begin // both halves present: do it
        reg_write(aw_q, w_q, s_q); // honourWSTRB byte lanes
        bresp <= decode_ok(aw_q) ? 2'b00 : 2'b10;
        bvalid <= 1; aw_full <= 0; w_full <= 0;
    end
    if (bvalid && bready) bvalid <= 0; // hold B until accepted
end
```

Structure of a robust AXI4-Lite write path (fragment). The companion RTL Design guide contains a complete, simulated AXI4-Lite register block.

PITFALL: THE MOST COMMON AXI4-LITE SLAVE BUGS

Dropping BVALID or RVALID before the READY handshake because the slave assumes the master is always ready. Many masters (and all good VIP) apply backpressure.

Accepting a new AR while RVALID is stalled and overwriting RDATA, or accepting a new write while BVALID is stalled and losing a response. Several widely copied vendor templates were shown (by formal verification, notably in public write-ups from the open-source community) to fail exactly this way under backpressure.

Requiring AWVALID and WVALID in the same cycle, which only works with masters that happen to drive them together.

IgnoringWSTRB, so that a byte store from the CPU clobbers the other three bytes of the register.

Decoding the full address including AWADDR[1:0] or the upper bits, so that an aliasing or unaligned address hits nothing.

Read side effects on the AR handshake executed twice (clear-on-read, FIFO pop) when the design re-evaluates the read while RVALID is stalled.

Formal verification is exceptionally effective here: a property set of about twenty assertions (stability, response counting, no response without request) proves an AXI4-Lite slave correct for all backpressure patterns in minutes. SymbiYosys with an open AXI-Lite property file, or a commercial formal tool with AXI assertion IP, should be part of every register-block sign-off (Chapter 25).

8.3 AXI4-Stream: signals

An AXI4-Stream interface has one direction: a **transmitter** (master) sends transfers to a **receiver** (slave). There are no addresses, responses or IDs in the AXI sense; TID and TDEST are routing fields.

Signal	Width	Meaning
ACLK, ARESETn	1	Clock and active-low reset
TVALID	1	Transmitter has a valid transfer
TREADY	1	Receiver can accept (optional: if absent, treated as always 1)
TDATA	8n	Payload, an integer number of bytes
TSTRB	n	Per byte: 1 = data byte, 0 = position byte
TKEEP	n	Per byte: 0 = null byte (not part of the stream)
TLAST	1	Marks the last transfer of a packet
TID	i	Stream identifier, for interleaving several streams (recommended at most 8 bits)

Signal	Width	Meaning
TDEST	d	Routing destination (recommended at most 4 bits)
TUSER	u	Sideband: user-defined, typically a multiple of the number of data bytes
TWAKEUP	1	AXI5-Stream: activity hint for power management

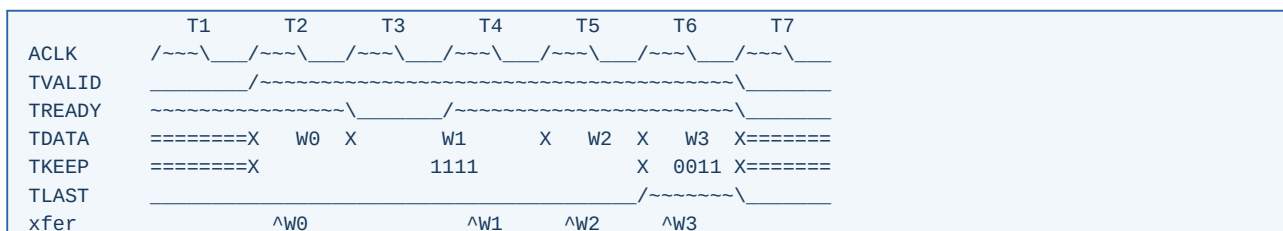
TKEEP	TSTRB	Byte type	Meaning
1	1	Data byte	Valid data, transmitted
1	0	Position byte	Placeholder whose position matters (for example unwritten bytes in a frame buffer), value undefined
0	0	Null byte	Not part of the stream; may be removed or inserted freely
0	1	Reserved	Must not be used

Most streams need neither: a **continuous aligned stream** has TKEEP = TSTRB = all ones on every transfer, and designs omit the signals. The common exception is the last transfer of a packet whose byte length is not a multiple of the bus width: TKEEP marks the valid bytes of that final beat (a 'continuous unaligned stream' in spec terms).

8.4 Packets, framing and backpressure

TLAST groups transfers into **packets**: an Ethernet frame, a video line, a DMA buffer, a DSP block. The protocol does not define a maximum packet length; the application does. Every component in a stream path makes one of three choices about packets:

- **Transparent**: pass TLAST along with the data (register slices, FIFOs, width converters).
- **Packet-aware**: switches and arbiters must not interleave two packets on one output, so they lock the output from the first transfer to the transfer with TLAST.
- **Store-and-forward**: a **packet FIFO** holds a whole packet and releases it only when TLAST arrived (so the downstream never stalls mid-packet), or drops a packet flagged bad in TUSER at its end (Ethernet FCS errors).



A 14-byte packet on a 32-bit stream: three full words and a last word with TKEEP = 0011. The receiver stalls in T3; the transmitter holds W1 until the handshake in T4.

8.5 Data width conversion

Width converters appear wherever a 512-bit DMA meets a 64-bit MAC or an 8-bit UART. Their behaviour is fixed by the byte-stream view of AXI4-Stream: the sequence of data and position bytes must be preserved, null bytes may be dropped or added, and TLAST must stay on the last byte of the packet.

- **Downsizing** (wide to narrow): each wide transfer becomes up to N narrow transfers, lowest byte lanes first (little-endian byte order), skipping null bytes. TLAST goes on the last emitted transfer of a wide transfer that had TLAST.
- **Upsizing** (narrow to wide): accumulate narrow transfers until the wide word is full **or TLAST arrives**; a packet ending early is flushed with the unused lanes marked null (TKEEP = 0). TUSER is typically concatenated per byte or ORed, depending on meaning.
- Non-integer ratios (for example 24 to 32 bits for RGB pixels) need a gearbox with a byte shift register; they are a common source of off-by-one bugs at packet ends.

8.6 Switching and routing streams

An **AXI4-Stream switch** (or interconnect) routes transfers by TDEST from S inputs to M outputs. Arbitration happens at packet boundaries: once an input wins an output, it keeps it until TLAST (or, in designs that allow it, a TID-based interleave). A **broadcaster** copies a stream to several outputs and must combine their TREADYs (a transfer completes only when all outputs accepted it, which needs care to avoid duplicating data to a fast output while a slow one stalls). A **combiner** merges several narrow streams into one wide one. TID lets several logical streams share one physical interface with transfers interleaved; a receiver that cannot handle interleaving needs one reassembly buffer per TID.

8.7 Video over AXI4-Stream

The dominant convention, defined by Xilinx (AMD) in its video IP user guides and followed by many others, carries one video frame as a sequence of lines:

- **TUSER[0] = SOF** (start of frame) on the first pixel of each frame.
- **TLAST = EOL** (end of line) on the last pixel of each line.
- TDATA holds one or more pixels per clock (1, 2, 4 or 8 pixels per clock for 4K and above), with components packed in a defined order; there is no blanking: TVALID is simply low between lines.
- A video timing controller and a 'stream to video out' block re-create HSYNC/VSYNC/blanking at the display interface (Chapter 21); the reverse block converts camera timing to a stream.

```
frame: line 0: [SOF]p0 p1 p2 ... p1919[EOL]   line 1: p0 ... p1919[EOL]   ...
TUSER0:      1  0  0           0           0           0
TLAST :      0  0  0           1           0           1
1080 lines x 1920 pixels, 1 pixel/clock at 60 fps -> about 124 Mpixel/s active
(the 148.5 MHz 1080p60 pixel clock includes blanking; the stream does not)
```

Video over AXI4-Stream: SOF in TUSER[0], EOL in TLAST.

PITFALL: LOST FRAME SYNCHRONIZATION

If a block drops or duplicates a transfer, every following line is shifted and the image tears until the next SOF. Robust video receivers therefore resynchronize on SOF, count pixels per line and flag 'early EOL' and 'late EOL' errors, which is the first thing to check in a video pipeline hang.

8.8 DMA: memory-mapped to stream and back

A **DMA engine** is the bridge between the memory-mapped world (AXI4) and the stream world (AXI4-Stream). By convention the two directions are named from the memory's point of view:

Channel	Direction	What it does
MM2S	memory -> stream	Reads a buffer with AXI4 read bursts and emits it as a stream packet, TLAST on the last byte, TKEEP on a partial last beat
S2MM	stream -> memory	Writes an incoming stream packet to a buffer with AXI4 write bursts; the packet length is only known at TLAST, so the engine reports the received byte count in a status register or descriptor

Simple DMAs are programmed with one address/length per transfer; **scatter-gather** DMAs fetch chains of descriptors from memory, each pointing to a buffer, so that a stream of packets lands in a ring of buffers without CPU involvement per packet. Design points that matter: burst splitting at 4KB (Chapter 7), enough outstanding reads for DDR latency, what S2MM does when a packet is longer than the buffer (truncate and flag, or continue into the next descriptor), and cache coherency of the buffers (Chapter 9).

8.9 Hands-on: a 32-to-8 bit stream downsizer with TKEEP and TLAST

`axis_32to8` converts a 32-bit stream into bytes. It holds one input word and its TKEEP; each output handshake clears the lowest set TKEEP bit ($k \& (k - 1)$), so null bytes are skipped automatically. TLAST is asserted on the last kept byte of a word that carried TLAST. The input is accepted when the register is empty or when its last byte leaves in the same cycle, so the converter sustains one byte per clock with no bubble between words.

```
// AXI4-Stream 32 -> 8 bit width converter. Null bytes (TKEEP=0) are dropped,
// TLAST moves to the last kept byte of the packet.
module axis_32to8 (
    input  logic      aclk, aresetn,
    input  logic      s_tvalid, input logic [31:0] s_tdata, input logic [3:0] s_tkeep,
    input  logic      s_tlast, output logic s_tready,
    output logic      m_tvalid, output logic [7:0] m_tdata, output logic m_tlast,
    input  logic      m_tready
);
    logic [31:0] d; logic [3:0] k; logic l;           // holding register
    wire [3:0] k_nxt = k & (k - 4'd1);              // clear lowest set keep bit
    wire      lastbyte = (k_nxt == 4'd0);
    wire      take      = m_tvalid && m_tready;

    assign m_tvalid = |k;
    assign m_tdata  = k[0] ? d[7:0] : k[1] ? d[15:8] : k[2] ? d[23:16] : d[31:24];
    assign m_tlast  = l && lastbyte;
    // accept a new word when empty, or when the last byte leaves this cycle
    assign s_tready = !m_tvalid || (take && lastbyte);

    always_ff @(posedge aclk or negedge aresetn)
        if (!aresetn) k <= '0;
        else if (s_tvalid && s_tready) begin d <= s_tdata; k <= s_tkeep; l <= s_tlast; end
        else if (take) k <= k_nxt;
endmodule
```

`axis_32to8`: AXI4-Stream downsizer that removes null bytes and moves TLAST to the last valid byte.

The testbench sends one directed packet (7 bytes, with a null byte in the middle word) and then 500 random packets of 1 to 40 bytes in which about one lane in eight is a null byte. TVALID has random gaps and TREADY random backpressure. A scoreboard compares every output byte and TLAST with the expected byte stream, and a checker verifies the AXI4-Stream rule that the output's TVALID, TDATA and TLAST stay stable while TREADY is low.

```
module tb;
    logic aclk = 0, aresetn = 0;
    always #5 aclk = ~aclk;
    logic s_tvalid = 0, s_tlast = 0, s_tready, m_tvalid, m_tlast, m_tready = 0;
    logic [31:0] s_tdata = 0; logic [3:0] s_tkeep = 0; logic [7:0] m_tdata;
    axis_32to8 dut (.*);

    byte unsigned exp_q [$]; bit exp_last [$];           // scoreboard
    int npk = 0, nbytes = 0, errs = 0, viol = 0, cyc = 0, trace = 1;

    // ---- source: random packets, random TVALID gaps, occasional null bytes ----
    task automatic send_packet(int len);
        int i = 0;
        while (i < len) begin
            logic [31:0] w = 0; logic [3:0] k = 0; logic last = 0;
            for (int b = 0; b < 4 && i < len; b++)
                if (($urandom % 8 != 0) || b == 3) begin // ~1/8 of lanes left null
                    w[8*b +: 8] = 8'($urandom); k[b] = 1;
                end
            exp_q[i] = w[7:0]; exp_last[i] = last;
            i++;
        end
    endtask
```



```

        exp_q.push_back(w[8*b +: 8]); exp_last.push_back(i == len - 1); i++;
    end
    last = (i == len);
    while ($urandom % 4 == 0) @(posedge aclk);
    s_tvalid <= 1; s_tdata <= w; s_tkeep <= k; s_tlast <= last;
    do @(posedge aclk); while (!s_tready);
    s_tvalid <= 0;
end
endtask

task automatic send_word(logic [31:0] w, logic [3:0] k, logic last); // directed
    int top = 0;
    for (int b = 0; b < 4; b++) if (k[b]) top = b;
    for (int b = 0; b < 4; b++) if (k[b]) begin
        exp_q.push_back(w[8*b +: 8]); exp_last.push_back(last && b == top);
    end
    s_tvalid <= 1; s_tdata <= w; s_tkeep <= k; s_tlast <= last;
    do @(posedge aclk); while (!s_tready);
    s_tvalid <= 0;
endtask

// ---- sink: random TREADY, scoreboard, AXIS stability rule checker ----
logic pend = 0; logic [8:0] held;
always @(posedge aclk) begin
    cyc++;
    if (trace && (s_tvalid || m_tvalid))
        $display("%3d S: v=%b r=%b keep=%b data=%08h last=%b | M: v=%b r=%b data=%02h last=%b",
            cyc, s_tvalid, s_tready, s_tkeep, s_tdata, s_tlast, m_tvalid, m_tready,
            m_tdata, m_tlast);
    if (pend && (!m_tvalid || {m_tlast, m_tdata} != held)) viol++; // no change while stalled
    pend <= m_tvalid && !m_tready; held <= {m_tlast, m_tdata};
    if (m_tvalid && m_tready) begin
        if (m_tdata != exp_q[0] || m_tlast != exp_last[0]) errs++;
        void'(exp_q.pop_front()); void'(exp_last.pop_front());
        nbytes++; if (m_tlast) npk++;
    end
    m_tready <= ($urandom % 3 != 0);
end

initial begin
    repeat (2) @(posedge aclk); aresetn <= 1;
    send_word(32'h44332211, 4'b1111, 0); // traced example: 7 bytes,
    send_word(32'h88776655, 4'b1101, 0); // byte 0x66 is a null byte
    send_word(32'h0000AA99, 4'b0011, 1);
    repeat (6) @(posedge aclk); trace = 0;
    for (int p = 0; p < 500; p++) send_packet(1 + $urandom % 40);
    repeat (50) @(posedge aclk);
    $display("packets=%0d bytes=%0d data/TLAST errors=%0d stability violations=%0d left=%0d",
        npk, nbytes, errs, viol, exp_q.size());
    $finish;
end
endmodule

```

Testbench: directed + random packets, random backpressure, scoreboard and stability checker.

```

 3 S: v=1 r=1 keep=1111 data=44332211 last=0 | M: v=0 r=1 data=xx last=x
 4 S: v=1 r=0 keep=1101 data=88776655 last=0 | M: v=1 r=1 data=11 last=0
 5 S: v=1 r=0 keep=1101 data=88776655 last=0 | M: v=1 r=0 data=22 last=0
 6 S: v=1 r=0 keep=1101 data=88776655 last=0 | M: v=1 r=1 data=22 last=0
 7 S: v=1 r=0 keep=1101 data=88776655 last=0 | M: v=1 r=1 data=33 last=0
 8 S: v=1 r=0 keep=1101 data=88776655 last=0 | M: v=1 r=0 data=44 last=0
 9 S: v=1 r=1 keep=1101 data=88776655 last=0 | M: v=1 r=1 data=44 last=0
10 S: v=1 r=0 keep=0011 data=0000aa99 last=1 | M: v=1 r=1 data=55 last=0
11 S: v=1 r=0 keep=0011 data=0000aa99 last=1 | M: v=1 r=0 data=77 last=0
12 S: v=1 r=0 keep=0011 data=0000aa99 last=1 | M: v=1 r=1 data=77 last=0
13 S: v=1 r=1 keep=0011 data=0000aa99 last=1 | M: v=1 r=1 data=88 last=0
14 S: v=0 r=0 keep=0011 data=0000aa99 last=1 | M: v=1 r=1 data=99 last=0
15 S: v=0 r=0 keep=0011 data=0000aa99 last=1 | M: v=1 r=0 data=aa last=1
16 S: v=0 r=0 keep=0011 data=0000aa99 last=1 | M: v=1 r=0 data=aa last=1
17 S: v=0 r=1 keep=0011 data=0000aa99 last=1 | M: v=1 r=1 data=aa last=1
packets=501 bytes=10184 data/TLAST errors=0 stability violations=0 left=0

```

Real Icarus Verilog output: cycle trace of the directed packet, then the summary of 501 packets.

- Cycle 3: word 0x44332211 is accepted into the empty converter.
- Cycles 4-9: bytes 11, 22, 33, 44 leave; in cycle 5 and 8 the sink is not ready and the byte is held (same data, TVALID high).
- The second word 0x88776655 has TKEEP = 1101: byte 0x66 is a null byte and never appears on the output (55 is followed by 77).
- The last word has TKEEP = 0011 and TLAST, so TLAST appears on 0xAA, the last kept byte, and is held through two stall cycles (15-16).
- Note the input side: TREADY rises exactly in the cycle where the last byte of the held word is taken (cycles 9 and 13): no bubble.

PITFALL: COMBINATIONAL TREADY AND ALL-NUL TLAST BEATS

In this design `s_tready` depends combinational on `m_tready`. That is legal, but chaining many such blocks creates a long combinational ready path through the whole pipeline. Insert a **register slice** (a two-entry skid buffer) every few stages.

A transfer with TKEEP = 0000 and TLAST = 1 can occur in AXI4-Stream (it ends a packet with no data). This converter would load it, see no valid byte and lose the TLAST. A production converter must either forbid such transfers (and assert on them) or hold back each output byte until the next input transfer shows whether a TLAST follows, so that TLAST can still be attached to the previous valid byte. Many IP cores simply document that TLAST must accompany at least one valid byte.

8.10 Summary

- AXI4-Lite keeps AXI's five channels but only single-beat, full-width, in-order transactions without IDs, exclusives or cache attributes; it is the FPGA-world register bus.
- AXI4-Lite slaves must handle AW and W in any order, hold BVALID and RVALID until accepted, honour WSTRB and never lose a response under backpressure; formal verification catches these bugs quickly.
- AXI4-Stream is an addressless VALID/READY pipe with TDATA, TKEEP/TSTRB byte qualifiers, TLAST packet framing and TID/TDEST/TUSER sideband.
- Width converters, switches (packet-locked on TLAST), packet FIFOs and DMA MM2S/S2MM engines are the standard stream components; video uses TUSER[0] = SOF and TLAST = EOL.
- The verified downsizer handled null bytes, partial last words and random backpressure on 501 packets without error.

8.11 Exercises

- Write the complete read path of the AXI4-Lite slave whose write path is shown in Section 8.2, including a clear-on-read status register whose side effect happens exactly once.
- Design the matching 8-to-32 bit **upsizer**: how does it handle a packet of 6 bytes, and what TKEEP does its last output word carry?
- A stream switch has two inputs sending 1500-byte packets to the same output. Why must it lock the output until TLAST, and what is the worst-case latency for the second input at 16 bytes per cycle?
- An S2MM DMA receives a 9000-byte jumbo frame into 2048-byte buffers. Describe the descriptor and status sequence you would implement.
- For 4K (3840 x 2160) at 60 frames per second with 2 pixels per clock, compute the minimum stream clock frequency (active pixels only) and the TDATA width for 10-bit YUV 4:2:2.
- Add an SVA property set to `axis_tb` that checks TKEEP/TSTRB reserved combinations and that TLAST is never asserted on a transfer whose TKEEP is all zero.

9. Cache Coherency Protocols: MESI/MOESI, ACE and CHI

A modern application processor has 8 to 200+ cores, each with private L1 and L2 caches, plus GPUs, NPUs and I/O masters that read and write the same DRAM. As soon as two caches can hold a copy of the same memory location, the hardware must make sure nobody reads a stale value. That is the job of a **cache coherency protocol**. On the wires it becomes the most complex protocol in the SoC: ACE extends AXI with snoop channels, and CHI replaces buses altogether with a packetized, credit-based protocol over a mesh network.

This chapter builds the concepts from the ground up: the coherence invariants, the MSI/MESI/MOESI state machines, snooping versus directories, AMBA ACE and ACE-Lite, AMBA CHI (layers, channels, nodes, flows, credits and retry), memory consistency models and barriers, and a preview of CXL.cache and CXL.mem. The hands-on part is a MESI model with three caches whose invariant checker catches an injected protocol bug within a few transactions.

9.1 Why coherence, and what exactly it guarantees

Consider two cores that both cached address $X = 0$. Core 0 writes $X = 1$ into its own cache (write-back, so memory still holds 0). If core 1 now reads X from its cache it gets 0, forever. Coherence protocols forbid this with two invariants, per memory location:

- **SWMR (single-writer, multiple-reader)**: at any time either one cache may write the location (and nobody else holds a copy), or any number of caches may read it (and nobody writes).
- **Data-value invariant**: a read returns the value of the most recent write in the coherence order of that location.

COHERENCE VERSUS CONSISTENCY

Coherence is about a single address: all agents agree on the order of writes to that address. **Consistency** (the memory model) is about the order in which accesses to **different** addresses become visible. Coherence is implemented by the protocol in this chapter; consistency is defined by the ISA (x86-TSO, Arm, RISC-V RVWMO) and implemented by the cores and interconnect, using barriers where the programmer needs ordering (Section 9.6).

The granularity of coherence is the **cache line** (64 bytes on almost all current Arm, x86 and RISC-V application processors). Two variables in the same line written by different cores bounce the line back and forth: **false sharing**, a classic performance bug that the protocol handles correctly but slowly.

9.2 MSI, MESI and MOESI

Each cache keeps a small state per line. The simplest useful protocol has three states: **Modified** (only copy, dirty), **Shared** (clean, maybe other copies) and **Invalid**. MESI adds **Exclusive** (only copy, clean): a line read when no other cache has it becomes E, and a later write upgrades it to M **silently**, without any bus transaction, which removes most upgrade traffic for private data. MOESI adds **Owned** (dirty but shared): the owner supplies data to readers and remains responsible for the eventual write-back, so a dirty line can be shared without first writing memory.

MESI transitions on processor requests

State	Processor read	Processor write	Eviction
I	BusRd; -> E if no other copy, else -> S	BusRdX (read for ownership); -> M	-
S	Hit, stay S	BusUpgr (invalidate others); -> M	Silent -> I (or Evict notification)
E	Hit, stay E	Hit, silent -> M	Silent -> I
M	Hit, stay M	Hit, stay M	Write back; -> I

MESI transitions on snooped bus transactions

State	Snooped BusRd	Snooped BusRdX	Snooped BusUpgr
I	-	-	-
S	Stay S	-> I	-> I
E	-> S (assert 'shared')	-> I	(cannot happen)
M	Supply data (flush), memory updated; -> S	Supply data; -> I	(cannot happen)

MOESI differs only where M or O see a BusRd: M supplies the data and goes to **O** instead of S, without updating memory; O supplies data on later BusRds and writes back on eviction. Intel's **MESIF** adds **Forward**, a designated clean sharer that answers requests so that exactly one cache responds. The ACE and CHI state names map onto these letters:

MOESI	ACE / CHI name	Unique?	Dirty?
M	UniqueDirty (UD)	yes	yes
E	UniqueClean (UC)	yes	no
O	SharedDirty (SD)	no	yes
S	SharedClean (SC)	no	no
I	Invalid (I)	-	-
-	CHI only: UniqueCleanEmpty (UCE), UniqueDirtyPartial (UDP)	yes	no / partial

9.3 Snooping versus directories

A **snooping** protocol broadcasts every coherent request to all caches, which check their tags and respond. It is simple and fast for a handful of agents, which is why ACE-based clusters of up to about eight cores use it. Broadcast traffic grows with the number of agents squared, so large systems use a **directory**: a home node per address range records which caches may hold each line (a presence bit-vector, or a limited pointer list) and sends snoops only to them.

Aspect	Snooping	Directory / snoop filter
Request path	Broadcast to all caches	To the home node, which snoops only recorded sharers
Scalability	Poor beyond ~8-16 agents	Hundreds of agents
Latency	One broadcast, fast for small systems	Extra hop to the home (indirection)
Storage	None	Directory or snoop-filter SRAM, sized for the total private-cache capacity
Examples	Classic SMP buses, ACE clusters	Arm CMN (CHI) snoop filters, AMD/Intel server fabrics

In practice most SoCs use a **snoop filter**: an inclusive directory of the lines held in the private caches, located at the home node. When the snoop filter runs out of entries it must **back-invalidate** a line from the caches, a performance effect worth checking in system benchmarks.

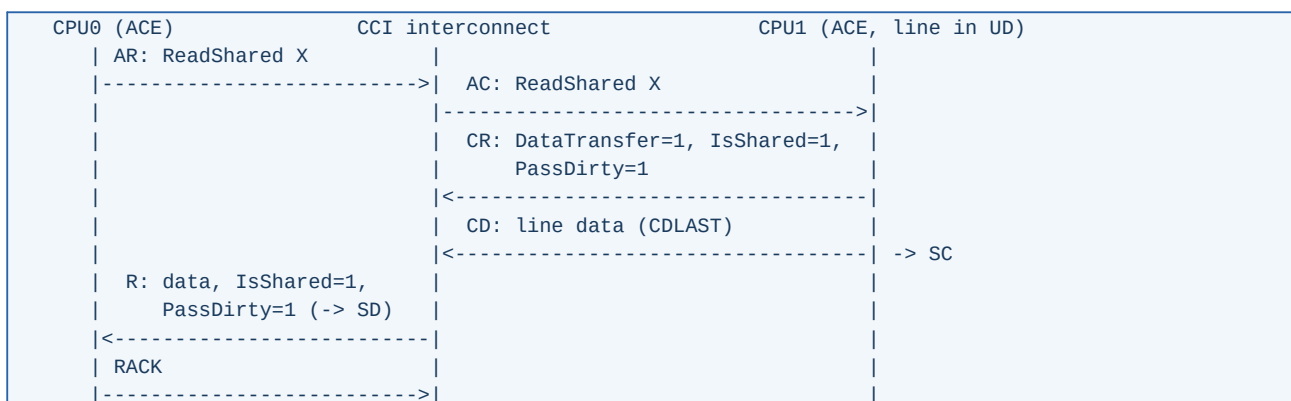
9.4 AMBA ACE: AXI Coherency Extensions

ACE (AMBA 4, 2011) turns AXI into a snooping protocol for clusters such as Cortex-A15/A7 big.LITTLE systems connected by a CCI interconnect. It adds signals to the existing channels and three new channels for snoops:

Where	Added signals	Purpose
AR	ARSNOOP[3:0], ARDOMAIN[1:0], ARBAR[1:0]	Coherent read type, shareability domain, barrier
AW	AWSNOOP[2:0], AWDOMAIN[1:0], AWBAR[1:0], AWUNIQUE	Coherent write type, domain, barrier, 'line held unique'
R	RRESP[3:2] = {IsShared, PassDirty}	Tells the master the final state: shared or unique, and whether it inherits dirtiness
RACK, WACK	1 bit each, master to interconnect	Acknowledge that the R/B response was consumed (ordering of snoops against responses)
AC (snoop address)	ACVALID, ACREADY, ACADDR, ACSNOOP[3:0], ACPROT	Interconnect to master: snoop request
CR (snoop response)	CRVALID, CRREADY, CRRESP[4:0]	Master to interconnect: DataTransfer, Error, PassDirty, IsShared, WasUnique
CD (snoop data)	CDVALID, CDREADY, CDDATA, CDLAST	Master to interconnect: the line's data when DataTransfer = 1

Domains (AxDOMAIN) limit the scope of coherence: Non-shareable, Inner Shareable (typically the CPU clusters), Outer Shareable (adds, for example, a GPU) and System. The main transaction groups are:

Group	Transactions	Use
Non-snooping	ReadNoSnoop, WriteNoSnoop	Non-shareable memory
Coherent reads	ReadOnce, ReadClean, ReadNotSharedDirty, ReadShared , ReadUnique	ReadShared for loads, ReadUnique for a store miss (full line with ownership), ReadOnce for a one-time snapshot not kept in cache
Dataless ownership	CleanUnique , MakeUnique	CleanUnique: upgrade a Shared line to Unique (invalidate others, others' dirty data is written back); MakeUnique: get ownership of a line that will be fully overwritten, without fetching data
Cache maintenance	CleanShared, CleanInvalid, MakeInvalid	Software cache maintenance (clean/invalidate by address)
Coherent writes	WriteUnique, WriteLineUnique	Write into shareable memory without holding the line (I/O masters)
Memory update	WriteBack , WriteClean, WriteEvict, Evict	Eviction of dirty (WriteBack), cleaning (WriteClean), clean eviction to a lower cache (WriteEvict), eviction notice for snoop filters (Evict)
Other	Barriers, DVM Operation / DVM Sync / DVM Complete	Ordering barriers; distributed virtual memory messages (TLB and branch-predictor invalidation broadcast)



An ACE ReadShared that hits a dirty line in another CPU: the snooped cache supplies the data on CD and passes the dirty responsibility to the requester (it ends in SharedDirty, i.e. MOESI's O state).

ACE-Lite

ACE-Lite has the extra AR/AW signals but **no snoop channels**: the master has no coherent cache of its own, so it never needs to be snooped, but its reads and writes snoop the CPU caches. This is **I/O coherency**: a DMA engine, GPU or network controller behind ACE-Lite can read data a CPU just wrote into its cache without software cache cleaning. ACE-Lite supports ReadOnce, WriteUnique, WriteLineUnique, the non-snooping transactions, cache maintenance and barriers; **ACE-Lite + DVM** adds DVM messages for SMMUs.

PITFALL: 'COHERENT' DMA THAT IS NOT

An I/O master is only coherent if its transactions are marked shareable (AxDOMAIN) with the right AxCACHE attributes **and** the interconnect port is configured to snoop. A driver that maps a buffer as non-cacheable on one side and cacheable on the other, or an SoC integration that ties AxDOMAIN to non-shareable, produces rare stale-data bugs that look like software races. Put a directed coherency test (CPU writes, DMA reads immediately) into the SoC bring-up suite.

9.5 AMBA CHI: the Coherent Hub Interface

CHI (AMBA 5, first issue 2014) was designed for scalable systems: tens to hundreds of requesters on a mesh network, such as Arm's CMN-600/CMN-700 based server and infrastructure SoCs. Instead of wires per transaction field it sends **flits** (packets) over **channels**, with explicit source and target node IDs, and it separates concerns into layers:

Layer	Responsibility
Protocol	Transactions and their message flows, cache states, ordering, retry and protocol credits
Network	Packetizing messages, node IDs (SrcID, TgtID), routing across the interconnect topology
Link	Flow control between two adjacent components with link-layer credits, link activation/deactivation handshake, flit transfer per channel
Physical	Wires, clocking, possibly die-to-die links; not defined by the CHI specification itself (implementation-specific)

Channels and node types

Each node has a transmit and a receive side of four channels: **REQ** (requests), **RSP** (responses without data, e.g. CompAck, RetryAck, Comp), **SNP** (snoops, home to requester) and **DAT** (data, e.g. CompData, SnpRespData, write data).

Node	Role
RN-F	Fully coherent requester with a cache (a CPU cluster)
RN-D	I/O coherent requester that also accepts DVM snoops (for example an SMMU)
RN-I	I/O coherent requester without a cache (typically the bridge from an ACE-Lite/AXI I/O subsystem)
HN-F	Fully coherent home node: point of coherence for its address slice, holds the snoop filter and often a slice of the system level cache
HN-I	Home node for I/O (non-coherent) address space
SN-F	Subordinate node for normal memory (a DDR controller)
SN-I	Subordinate node for I/O peripherals
MN	Miscellaneous node: DVM and barrier handling

A CHI read flow

RN-F0 (requester)	HN-F (home, snoop filter)	RN-F1 (holds UD)	SN-F (DDR)
-- REQ ReadShared ----->			
	-- SNP SnpShared ----->		
<- DAT CompData (DCT: direct cache transfer) -----			
	<- RSP SnpResp -----		
-- RSP CompAck ----->			
on a miss in all caches:			
-- REQ ReadShared ----->	-- REQ ReadNoSnp ----->		
<- DAT CompData (DMT: direct memory transfer) -----			
-- RSP CompAck ----->			

CHI ReadShared: with a snoop hit, the snooped RN-F can send data straight to the requester (DCT); on a miss the memory node can do so (DMT). The requester's CompAck lets the home close the transaction.

Credits and retry

CHI has two independent flow-control mechanisms:

- **Link-layer credits (L-credits):** per channel and per link, the receiver grants credits (one per free flit buffer, signalled on a credit-valid wire) and the transmitter may only send a flit when it holds a credit. This is the credit-based flow control of Chapter 3 and Section 10.6, and it means a link never drops a flit.
- **Protocol retry:** a home node may not have a free tracker for a new request. It then answers **RetryAck** and later sends **PCrdGrant** (a protocol credit of a given type) when resources free up; the requester resends the request, now marked as using that credit (AllowRetry = 0), and it is guaranteed to be accepted. This prevents a request from occupying network buffers while waiting, which is essential for deadlock freedom.

WHY CHI SCALES WHERE ACE DOES NOT

Snoops go only where the snoop filter says the line lives.

Home nodes are distributed: each HN-F owns an address hash slice, so coherence bandwidth grows with the mesh.

Direct transfers (DCT, DMT, and DWT for writes) remove the home from the data path.

Everything is a message with IDs and credits, so the same protocol runs across chiplets (CHI over a die-to-die link such as UCIe, Chapter 22) with only the link and physical layers changing.

9.6 Memory consistency models and barriers

Coherence alone does not say when a write to X becomes visible relative to a write to Y. The ISA's memory model does:

Model	Key property	Used by
Sequential consistency (SC)	All accesses appear in one global order consistent with each program's order	Reference model; too restrictive for fast hardware
TSO (total store order)	Like SC except a load may pass an earlier store to a different address (the store buffer)	x86 (x86-TSO), SPARC TSO, RISC-V Ztso option
Weak / relaxed	Loads and stores to different addresses may be reordered unless ordered by a dependency or barrier	Arm (v8/v9), RISC-V RVWMO, POWER
Release consistency	Ordering only at acquire (after a lock/flag read) and release (before a lock/flag write)	Arm LDAR/STLR, RISC-V .aq/.rl, C/C++ atomics


```
// Message-passing litmus test (MP).      x = data, f = flag, both 0 initially
// Core 0                                Core 1
x = 42;                                while (f == 0) ;
f = 1;                                r = x;          // SC, TSO: r == 42 guaranteed

// On Arm / RISC-V the stores or loads may be reordered, so r == 0 is allowed
// unless ordered, for example:
x = 42;                                while (f == 0) ;
DMB ISHST (or STLR)    DMB ISHLD (or LDAR f)    // Arm
f = 1;                r = x;

// RISC-V: fence w,w between the stores; fence r,r between the loads
```

The message-passing litmus test and the barriers that make it work on weakly ordered machines.

Barriers reach the interconnect too. A **DMB** only orders the core's own accesses; a **DSB** additionally waits until earlier accesses have **completed**, which for writes means the B response (or the CHI Comp) has returned from the point of coherence or the device. This is why device drivers use a DSB before triggering a DMA engine through a register write: the descriptor writes must be visible to the DMA first. ACE had explicit barrier transactions (AxBAR); later AMBA 5 protocols removed barrier transactions and rely on response semantics instead.

9.7 Preview: CXL.cache and CXL.mem

Compute Express Link (Chapter 18) carries coherency **off chip**, over the PCIe physical layer. It multiplexes three protocols on one link:

Protocol	Direction and purpose	Channels
CXL.io	PCIe-equivalent: discovery, configuration, DMA, interrupts	PCIe TLPs
CXL.cache	A device (accelerator) caches host memory coherently; the host snoops the device	D2H Req/Resp/Data, H2D Req/Resp/Data
CXL.mem	The host reads and writes device-attached memory with load/store semantics (memory expansion, pooling)	M2S Req and RwD; S2M NDR and DRS (plus back-invalidate channels from CXL 3.0)

Device types: **Type 1** (cache, no memory: smart NIC) uses .io + .cache; **Type 2** (accelerator with its own memory) uses all three; **Type 3** (memory expander) uses .io + .mem. Versions 1.1 and 2.0 run on the PCIe 5.0 PHY at 32 GT/s (2.0 adds switching and memory pooling); 3.x moves to PCIe 6.x at 64 GT/s with PAM4 and FLIT mode and adds fabrics, memory sharing and back-invalidate snoops for device-managed coherence. Inside the SoC, a CXL controller is typically an RN (for .cache) or an SN/HN (for .mem) on the CHI mesh.

9.8 Hands-on: a MESI model with invariant checking

The model below keeps one cache line in three caches on an **atomic snooping bus**: each cycle one processor request is served together with its bus transaction, so there are no transient states. It implements the tables of Section 9.2 directly: requester actions in one block, snoop reactions in a loop over the other caches, and wired-OR 'shared' and 'dirty' snoop results. A parameter **BUG** injects a realistic error: snoopers ignore BusUpgr, so a write hit on a Shared line does not invalidate the other copies.

```
// One cache line shared by N snooping caches on an atomic bus (one bus
// transaction per cycle). MESI protocol with write-back memory.
// BUG=1 injects a classic error: snoopers ignore BusUpgr (no invalidation).
module mesi_sys #(parameter int N = 3, parameter bit BUG = 0) (
    input logic      clk, rst_n,
    input logic      req,                // one processor request per cycle
    input logic [1:0] cpu,              // which processor
    input logic [1:0] op,               // 0 read, 1 write, 2 evict
    input logic [15:0] wval,            // value to write
```

```

output logic [15:0] rval,          // value returned to a read
output logic [2:0] bus,           // bus transaction issued this cycle
output logic [N-1:0][1:0] st,     // line state in each cache
output logic [N-1:0][15:0] val,   // line data in each cache
output logic [15:0] mem           // memory copy
);
localparam logic [1:0] I = 0, S = 1, E = 2, M = 3;
localparam logic [2:0] NONE = 0, BUSRD = 1, BUSRDX = 2, BUSUPGR = 3, WB = 4;

// --- snoop results (wired-OR "shared" and "dirty" lines of a snooping bus) ---
logic shared, dirty; logic [15:0] owner_val;
always_comb begin
    shared = 0; dirty = 0; owner_val = mem;
    for (int j = 0; j < N; j++) if (j != cpu && st[j] != I) begin
        shared = 1;
        if (st[j] == M) begin dirty = 1; owner_val = val[j]; end
    end
end

// --- requester side: which bus transaction does this request need? ---
always_comb begin
    bus = NONE;
    if (req) case (op)
        0: if (st[cpu] == I) bus = BUSRD;           // read miss
        1: if (st[cpu] == I) bus = BUSRDX;         // write miss
           else if (st[cpu] == S) bus = BUSUPGR;    // write hit on S
        2: if (st[cpu] == M) bus = WB;             // dirty eviction
        default: ;
    endcase
    rval = (st[cpu] != I) ? val[cpu] : owner_val;
end

always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin
        mem <= 0;
        for (int j = 0; j < N; j++) begin st[j] <= I; val[j] <= 0; end
    end else if (req) begin
        // snooping caches react to the bus transaction
        for (int j = 0; j < N; j++) if (j != cpu)
            case (bus)
                BUSRD:   if (st[j] == M || st[j] == E) st[j] <= S; // M also flushes
                BUSRDX:  st[j] <= I;
                BUSUPGR: if (!BUG) st[j] <= I;
                default: ;
            endcase
        if (dirty && (bus == BUSRD || bus == BUSRDX)) mem <= owner_val; // flush
        // requesting cache
        case (op)
            0: if (st[cpu] == I) begin st[cpu] <= shared ? S : E; val[cpu] <= owner_val; end
            1: begin st[cpu] <= M; val[cpu] <= wval; end // E->M is silent
            2: begin if (st[cpu] == M) mem <= val[cpu]; st[cpu] <= I; end
            default: ;
        endcase
    end
endmodule

```

mesi_sys: one line, N snooping caches, MESI with write-back memory and an optional injected bug.

The testbench instantiates a correct and a buggy system side by side and feeds both the same request stream: twelve directed operations, then 20000 random reads, writes and evictions. After every request it checks **SWMR** (at most one M/E copy, and an M/E copy excludes all others) and the **data-value invariant**: every value written is a new number, and every read must return the latest one.

```

module tb;
  localparam int N = 3;
  logic clk = 0, rst_n = 0; always #5 clk = ~clk;
  logic req = 0; logic [1:0] cpu = 0, op = 0; logic [15:0] wval = 0, rval, mem;
  logic [2:0] bus;
  logic [N-1:0][1:0] st0, st1; logic [N-1:0][15:0] v0, v1;
  logic [15:0] rv0, rv1, m0, m1; logic [2:0] b0, b1;
  mesi_sys #(.N(N), .BUG(0)) good (.clk, .rst_n, .req, .cpu, .op, .wval,
    .rval(rv0), .bus(b0), .st(st0), .val(v0), .mem(m0));
  mesi_sys #(.N(N), .BUG(1)) bad (.clk, .rst_n, .req, .cpu, .op, .wval,
    .rval(rv1), .bus(b1), .st(st1), .val(v1), .mem(m1));

  function automatic string sn(logic [1:0] s); return s == 0 ? "I" : s == 1 ? "S" :
    s == 2 ? "E" : "M"; endfunction
  function automatic string bn(logic [2:0] b); return b == 0 ? "-" :
    b == 1 ? "BusRd" : b == 2 ? "BusRdX" : b == 3 ? "BusUpgr" : "WrBack "; endfunction

  // invariant: at most one M/E copy, and an M/E copy excludes all other copies
  function automatic bit swmr_ok(logic [N-1:0][1:0] s);
    int own = 0, valid = 0;
    for (int j = 0; j < N; j++) begin own += (s[j] >= 2); valid += (s[j] != 0); end
    return own == 0 || (own == 1 && valid == 1);
  endfunction

  logic [15:0] golden = 0; // last value written by anyone
  int inv_err [2], stale [2], reads = 0;
  task automatic do_op(int c, int o, int show);
    string on = o == 0 ? "RD" : o == 1 ? "WR" : "EV";
    cpu <= c; op <= o; wval <= golden + 1; req <= 1;
    @(negedge clk); // combinational results
    if (o == 0) begin
      reads++; if (rv0 != golden) stale[0]++; if (rv1 != golden) stale[1]++;
    end
    if (show) $write("P%0d %s bus=%s rd=%4h |", c, on, bn(b0), o == 0 ? rv0 : 16'h0);
    @(posedge clk); #1;
    if (o == 1) golden = golden + 1;
    if (!swmr_ok(st0)) inv_err[0]++;
    if (!swmr_ok(st1)) inv_err[1]++;
    if (show) $display(" good: %s %s %s mem=%4h | BUG: %s %s %s", sn(st0[0]), sn(st0[1]),
      sn(st0[2]), m0, sn(st1[0]), sn(st1[1]), sn(st1[2]));
    req <= 0;
  endtask

  initial begin
    inv_err[0] = 0; inv_err[1] = 0; stale[0] = 0; stale[1] = 0;
    repeat (2) @(posedge clk); rst_n <= 1; @(posedge clk); #1;
    do_op(0, 0, 1); do_op(0, 1, 1); do_op(1, 0, 1); do_op(2, 0, 1);
    do_op(1, 1, 1); do_op(0, 0, 1); do_op(2, 2, 1); do_op(0, 1, 1);
    do_op(2, 1, 1); do_op(1, 0, 1); do_op(2, 2, 1); do_op(1, 1, 1);
    for (int i = 0; i < 20000; i++) do_op($urandom % N, $urandom % 3, 0);
    $display("reads=%0d | good: stale=%0d SWMR violations=%0d | BUG=1: stale=%0d viol=%0d",
      reads, stale[0], inv_err[0], stale[1], inv_err[1]);
    $finish;
  end
endmodule

```

Testbench: shared stimulus, SWMR check and a golden latest-value model for stale-read detection.

```

P0 RD bus=BusRd rd=0000 | good: E I I mem=0000 | BUG: E I I
P0 WR bus=-      rd=0000 | good: M I I mem=0000 | BUG: M I I
P1 RD bus=BusRd rd=0001 | good: S S I mem=0001 | BUG: S S I
P2 RD bus=BusRd rd=0001 | good: S S S mem=0001 | BUG: S S S
P1 WR bus=BusUpgr rd=0000 | good: I M I mem=0001 | BUG: S M S
P0 RD bus=BusRd rd=0002 | good: S S I mem=0002 | BUG: S M S
P2 EV bus=-      rd=0000 | good: S S I mem=0002 | BUG: S M I
P0 WR bus=BusUpgr rd=0000 | good: M I I mem=0002 | BUG: M M I
P2 WR bus=BusRdX rd=0000 | good: I I M mem=0003 | BUG: I I M
P1 RD bus=BusRd rd=0004 | good: I S S mem=0004 | BUG: I S S
P2 EV bus=-      rd=0000 | good: I S I mem=0004 | BUG: I S I
P1 WR bus=BusUpgr rd=0000 | good: I M I mem=0004 | BUG: I M I
reads=6649 | good: stale=0 SWMR violations=0 | BUG=1: stale=859 viol=3570

```

Real Icarus Verilog output. The bus column and read data refer to the correct system; the right-hand states are the buggy system's.

- P0 reads (miss, no sharers): **E**. P0 writes: E -> M **silently** (bus = '-').
- P1 reads: BusRd; P0 flushes (memory becomes 0001) and both end in S. P2 reads: all three S.
- P1 writes: **BusUpgr**. Correct system: I M I. Buggy system: **S M S**, an SWMR violation visible immediately.
- P0 reads next: in the correct system it misses and gets 0002 from P1 (who flushes). In the buggy system P0 still holds a Shared copy and hits on stale data 0001; the counters at the end show 859 stale reads and 3570 SWMR violations in the buggy model, zero in the correct one.

FROM THIS MODEL TO REAL PROTOCOL VERIFICATION

Real protocols are not atomic: a request, its snoops and responses are separate messages, so caches have **transient states** (for example IS^D: 'was I, sent a read, waiting for data'). Most bugs live in races between a snoop and an in-flight request to the same line.

Industry practice is to model-check the protocol at the architectural level (Murphi, TLA+, or formal tools on an abstract RTL model) and then run SWMR/data-value scoreboards in RTL simulation and emulation with heavy same-line contention.

A scoreboard like the one above, attached to an SoC simulation, is the most valuable coherency check you can write: it is independent of the protocol details.

9.9 Ownership and integration

Team	Coherency responsibilities
Architecture	Protocol choice (ACE vs CHI), snoop filter sizing, home-node count, SLC policy, coherency domains
RTL	Cache controllers and their transient states, home nodes, ACE/CHI bridges, DVM handling
DV / formal	Protocol VIP (ACE/CHI), coherency scoreboards, litmus tests on the full SoC, formal checks of cache FSMs
Firmware / OS	Memory attributes in page tables, cache maintenance for non-coherent masters, barriers in drivers
Performance	Snoop latency and bandwidth, false-sharing analysis, snoop-filter back-invalidation rates

9.10 Summary

- Coherence enforces SWMR and the data-value invariant per cache line; consistency (TSO, weak, release) orders different addresses and is enforced with barriers.
- MSI, MESI (silent E -> M upgrade) and MOESI (dirty sharing with an owner) are the core state machines; ACE/CHI name the states UC, UD, SC, SD, I (CHI adds UCE and UDP).
- Snooping broadcasts and suits small clusters; directories and snoop filters scale to large meshes.
- ACE adds snoop channels AC/CR/CD and coherent transaction types to AXI; ACE-Lite gives I/O coherency without snoop channels.

- CHI is a layered, packetized protocol (REQ/RSP/SNP/DAT) with RN, HN and SN nodes, link credits, protocol retry and direct transfers.
- CXL.cache and CXL.mem extend coherency and memory semantics over the PCIe PHY.
- The MESI model's invariant checker found the injected BusUpgr bug at the first upgrade and reported hundreds of stale reads.

9.11 Exercises

- Extend `mes_i_sys` to MOESI: add the O state, make M → O on a snooped BusRd without a memory update, and adapt the invariants. How much memory write traffic does the random test save?
- Draw the message sequence for a store miss (ReadUnique) in CHI when two other RN-Fs hold the line in SC. Which snoops are sent, and when may the requester write?
- Explain why the E state saves bus traffic for private data, and construct an access sequence where MESI and MSI produce a different number of bus transactions.
- Write the store-buffering (SB) litmus test and explain why `r1 == r2 == 0` is allowed under TSO but not under SC. Which barrier forbids it on Arm?
- A DMA engine writes a buffer through ACE-Lite with WriteUnique while a CPU holds some of those lines in UD. Describe what the interconnect must do for each line.
- Estimate the snoop-filter size (entries) needed for 64 cores with 1 MB private L2 each and 64-byte lines, and discuss what happens when it is under-provisioned.

10. Other On-Chip Protocols: Wishbone, Avalon, TileLink, OCP and NoCs

AMBA dominates commercial SoCs, but it is not the only on-chip protocol you will meet. Open-source cores and FPGA soft-SoCs use **Wishbone**; Intel (Altera) FPGA designs use **Avalon**; RISC-V SoCs from SiFive, the Rocket Chip generator and OpenTitan use **TileLink**; older consumer SoCs used **OCP** and IBM **CoreConnect**. And inside every large SoC, all of these ride on a **network-on-chip (NoC)** that packetizes transactions and routes them across the die. This chapter surveys the alternatives at the level needed to integrate, bridge and debug them, then dives into NoC fundamentals with a working 3x3 mesh of wormhole routers.

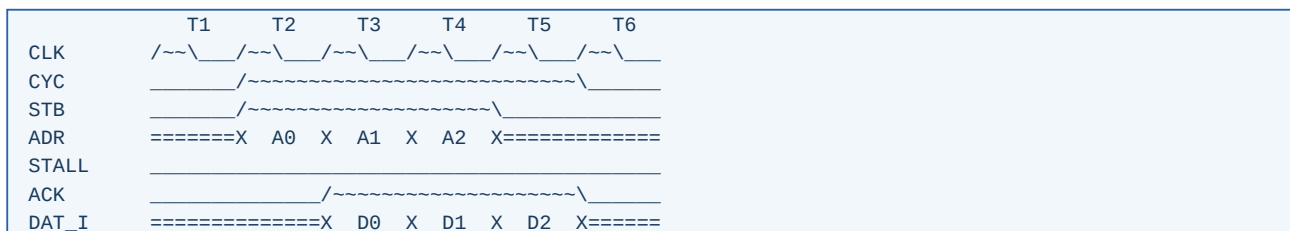
10.1 Wishbone B4

Wishbone is an open, royalty-free bus defined by Silicore and maintained by the OpenCores community; revision **B4** (2010) is the current one. Its philosophy is minimalism: a handful of signals, named from the point of view of the module (suffix **_O** for outputs, **_I** for inputs), an active-high synchronous reset and a single clock.

Signal (master side)	Meaning
CLK_I, RST_I	Clock and active-high reset
CYC_O	A bus cycle is in progress (held for a whole block of transfers; also used as the arbitration request)
STB_O	Strobe: this cycle carries a valid transfer request
WE_O	1 = write, 0 = read
ADR_O, DAT_O, DAT_I	Address, write data, read data
SEL_O	Byte select, one bit per byte lane (like WSTRB)
ACK_I, ERR_I, RTY_I	Normal termination, error, retry
STALL_I	Pipelined mode: slave cannot accept a new request
LOCK_O	Indivisible sequence
CTI_O, BTE_O	Registered-feedback bursts: cycle type (classic, constant-address, incrementing, end-of-burst) and burst type (linear, 4/8/16-beat wrap)
TGA/TGC/TGD	User tags on address, cycle and data

Classic and pipelined cycles

In the **classic** cycle, the master asserts CYC and STB with address and data and holds them until the slave terminates the transfer with ACK (or ERR/RTY). A slave with a registered ACK therefore costs at least two clocks per transfer, similar to APB. In **pipelined** mode (introduced in B4), STB means 'new request this cycle'; the master may issue one request per clock while STALL is low, and ACKs come back later, one per request and in order, while CYC stays high until the last ACK. That turns Wishbone into a single-outstanding-queue protocol with AHB-like throughput.



Pipelined Wishbone read of three words: requests in T2-T4, acknowledgements with data one cycle later in T3-T5; CYC drops after the last ACK.

Hands-on: a Wishbone classic slave

A register slave with a registered ACK (one wait state), SEL byte enables and ERR for unmapped addresses. The `!ack_o && !err_o` term in `req` is the important detail: in a classic cycle the master is still asserting STB in the cycle where ACK is high, and without that term the slave would see a second request and acknowledge twice.

```
// Wishbone B4 classic slave: 8 x 32-bit registers at 0x00-0x1C, registered
// ACK (one wait state per access), SEL byte enables, ERR above 0x1C.
module wb_regs (
    input logic      clk_i, rst_i,          // Wishbone uses an active-high reset
    input logic      cyc_i, stb_i, we_i,
    input logic [5:0] adr_i,                // byte address, word aligned
    input logic [31:0] dat_i,
    input logic [3:0] sel_i,
    output logic [31:0] dat_o,
    output logic      ack_o, err_o
);
    logic [31:0] r [8];
    wire req = cyc_i && stb_i && !ack_o && !err_o; // new request (not the ack cycle)
    wire bad = adr_i[5];
    always_ff @(posedge clk_i)
        if (rst_i) begin ack_o <= 0; err_o <= 0; for (int i = 0; i < 8; i++) r[i] <= 0; end
        else begin
            ack_o <= req && !bad; // ACK or ERR terminates the cycle
            err_o <= req && bad;
            if (req && !bad) begin
                if (we_i) for (int b = 0; b < 4; b++)
                    if (sel_i[b]) r[adr_i[4:2]][8*b +: 8] <= dat_i[8*b +: 8];
                dat_o <= r[adr_i[4:2]];
            end
        end
    end
endmodule
```

wb_regs: Wishbone B4 classic slave.

```
module tb;
    logic clk = 0, rst = 1; always #5 clk = ~clk;
    logic cyc = 0, stb = 0, we = 0, ack, err; logic [5:0] adr = 0;
    logic [31:0] dw = 0, dr; logic [3:0] sel = 0; int t = 0;
    wb_regs dut (.clk_i(clk), .rst_i(rst), .cyc_i(cyc), .stb_i(stb), .we_i(we),
                .adr_i(adr), .dat_i(dw), .sel_i(sel), .dat_o(dr), .ack_o(ack), .err_o(err));

    always @(posedge clk) if (!rst) begin
        t++;
        if (cyc) $display("%2d  %b  %b  %b  %02h  %b  %08h |  %b  %b  %08h",
                        t, cyc, stb, we, adr, sel, dw, ack, err, ack && !we ? dr : 32'h0);
    end

    // classic single cycle: assert CYC+STB, hold everything until ACK or ERR
    task automatic wb_xfer(input logic w, input logic [5:0] a, input logic [31:0] d,
                          input logic [3:0] s, input logic keep_cyc = 0);
        cyc <= 1; stb <= 1; we <= w; adr <= a; dw <= d; sel <= s;
        do @(posedge clk); while (!(ack || err));
        stb <= 0; if (!keep_cyc) begin cyc <= 0; @(posedge clk); end // bus released
    endtask

    initial begin
        repeat (2) @(posedge clk); rst <= 0; @(posedge clk);
        $display("clk CYC STB WE ADR SEL  DAT_W    | ACK ERR DAT_R");
        wb_xfer(1, 6'h04, 32'h1234_5678, 4'b1111);
    end
endmodule
```

```

wb_xfer(1, 6'h04, 32'h0000_AB00, 4'b0010);      // byte lane 1 only
wb_xfer(0, 6'h04, 0, 4'b1111, 1);              // block read: CYC stays high
wb_xfer(0, 6'h08, 0, 4'b1111);
wb_xfer(0, 6'h24, 0, 4'b1111);                  // unmapped -> ERR
@(posedge clk); $finish;
end
endmodule

```

Testbench with a classic-cycle master task; a block read keeps CYC asserted between two transfers.

clk	CYC	STB	WE	ADR	SEL	DAT_W	ACK	ERR	DAT_R
2	1	1	1	04	1111	12345678	0	0	00000000
3	1	1	1	04	1111	12345678	1	0	00000000
5	1	1	1	04	0010	0000ab00	0	0	00000000
6	1	1	1	04	0010	0000ab00	1	0	00000000
8	1	1	0	04	1111	00000000	0	0	00000000
9	1	1	0	04	1111	00000000	1	0	1234ab78
10	1	1	0	08	1111	00000000	0	0	00000000
11	1	1	0	08	1111	00000000	1	0	00000000
13	1	1	0	24	1111	00000000	0	0	00000000
14	1	1	0	24	1111	00000000	0	1	00000000

Real Icarus Verilog output: every transfer takes two clocks (request, then ACK); SEL = 0010 writes only byte 1; the block read in cycles 8-11 keeps CYC high; 0x24 terminates with ERR.

PITFALL: BRIDGING WISHBONE TO AXI

Wishbone has no IDs, no separate read/write channels and (in classic mode) at most one transfer in flight, so a Wishbone-to-AXI bridge is easy but slow. The reverse direction must serialize AXI bursts and, for pipelined Wishbone, count outstanding requests to match ACKs with the right AXI beat. The most frequent bug is forgetting RTY_I and ERR_I termination paths, which hang the bridge when a slave answers with them.

10.2 Intel Avalon: Avalon-MM and Avalon-ST

Avalon is the interface family of Intel (formerly Altera) FPGA Platform Designer (formerly Qsys). Recent documentation calls the roles **host** and **agent**.

Avalon-MM signal	Meaning
address, byteenable	Address and byte lanes. Hosts use byte addresses; agents by default see word addresses, and the interconnect converts
read, write, writedata, readdata	Command strobes and data
waitrequest	Agent cannot accept the command this cycle (the host holds it), i.e. the inverse of READY
readdatavalid	Pipelined reads: data returns later, in order, one pulse per read beat
burstcount	Burst length in beats (bursts are always incrementing)
response, writeresponsevalid	OKAY, SLVERR/SLAVEERROR, DECODEERROR; optional write responses
lock, debugaccess	Locked sequences; debugger access

Avalon-ST is the streaming counterpart: valid, ready, data, channel, error, startofpacket, endofpacket and empty (number of unused symbols in the last beat, where AXI4-Stream uses TKEEP). Its notable difference from AXI4-Stream is the **readyLatency** parameter: with readyLatency = N the source may send data N cycles after ready was asserted, which allows ready to be pipelined but means an adapter (a small FIFO) is needed to connect to an AXI4-Stream interface, whose behaviour corresponds to readyLatency = 0.

10.3 TileLink

TileLink was created at UC Berkeley and SiFive for the Rocket Chip generator and is now maintained under the Chips Alliance. It is a chip-scale, **coherent-capable** protocol with a clean layered design and three conformance levels:

Level	Operations	Typical use
TL-UL (Uncached Lightweight)	Get, PutFullData, PutPartialData; single-beat	Peripherals; OpenTitan uses TL-UL as its whole on-chip bus
TL-UH (Uncached Heavyweight)	Adds multi-beat bursts, atomics (ArithmeticData, LogicalData) and hints (Intent)	Memory and accelerators
TL-C (Cached)	Adds Acquire/Probe/Release/Grant for coherent caches	Coherent CPU clusters, L2 caches

TileLink messages travel on up to **five channels**, each with its own valid/ready handshake:

master (client)	slave (manager)	
A: requests (Get, Put, Acquire, atomics)	----->	
B: probes (Probe, forwarded ops)	<-----	TL-C
C: releases (ProbeAck[Data], Release[Data])	----->	TL-C
D: responses (AccessAck[Data], Grant[Data], ReleaseAck)	<-----	
E: GrantAck (completes an Acquire)	----->	TL-C

fields: opcode, param, size, source (ID), address, mask, data, corrupt;
D also has sink (ID for E) and denied

TileLink channels. TL-UL and TL-UH use only A and D; TL-C adds B, C and E.

Deadlock freedom is built into the rules: channels have a priority order (A lowest, then B, C, D, E highest), and a message on a channel may never wait for a message on a lower-priority channel. Coherence uses **permissions** (None, Branch = read-only, Trunk/Tip = read-write) rather than MESI letters, but the concepts map directly onto Chapter 9: Acquire is ReadShared/ReadUnique, Probe is a snoop, Release is a write-back.

TILELINK IN PRACTICE

The Rocket Chip and Chipyard generators describe TileLink networks in Chisel with 'diplomacy', which negotiates parameters (widths, supported sizes and operations) between every client and manager at elaboration time. When bridging to AXI (TLToAXI4, AXI4ToTL), the source field maps to AXI IDs and TL's per-message size maps to AxLEN/AxSIZE.

10.4 OCP: the Open Core Protocol

OCP was promoted from 2001 by the OCP-IP consortium (Sonics, TI, Nokia and others) as a **configurable socket**: instead of fixing a bus, it defines a large menu of optional features from which each core picks a profile, and an interconnect (often a Sonics NoC) adapts. Its basic signals are MCmd (IDLE, WR, RD, RDEX, RDL, WRNP, WRC, BCST), MAddr, MData, MByteEn, SCmdAccept, SResp (NULL, DVA, FAIL, ERR) and SData, with optional bursts (MBurstLength, MBurstSeq), threads (MThreadID) and tags (MTagID) for out-of-order completion. TI's OMAP phones used it extensively. The OCP-IP assets were transferred to **Accellera** in 2013; new designs rarely choose OCP, but it remains in legacy IP and its thread concept influenced later NoCs.

10.5 IBM CoreConnect

CoreConnect (1999) was IBM's bus architecture for PowerPC 4xx embedded SoCs and, for a decade, Xilinx's embedded-processor platform (PowerPC 405/440 hard cores, early MicroBlaze). It had three buses: **PLB** (Processor Local Bus, 32 to 128 bits, pipelined and split read/write), **OPB** (On-chip Peripheral Bus, simpler) and **DCR** (Device Control Register bus, a daisy-chained ring for configuration registers). Xilinx moved to AXI around 2009-2010, and CoreConnect now matters mostly for maintaining old designs.

10.6 Networks-on-chip: packets, flits, routing and flow control

A crossbar connecting M masters to S slaves costs $O(M \times S)$ wires and cannot be placed and routed across a 400 mm² die at GHz speeds. A **network-on-chip** replaces it with routers connected by short, registered links, and transports each transaction as a **packet**. The masters and slaves keep their native protocols (AXI, AHB, APB, CHI); **network interface units (NIUs)** at the edge convert transactions to packets and back.

```

AXI master --[NIU]--(R)----(R)----(R)--[NIU]-- DDR ctrl (AXI slave)
              |           |           |
AHB master --[NIU]--(R)----(R)----(R)--[NIU]-- APB subsystem
              |           |           |
CHI RN-F  -----(R)----(R)----(R)----- HN-F / SLC slice

packet = head flit [route | dest | VC | type | txn id] + body flits + tail
flit   = flow-control unit (one buffer slot); phit = what one link moves per cycle

```

A NoC: NIUs packetize protocol transactions, routers (R) forward flits.

Topologies

Topology	Properties	Where used
Crossbar / partial crossbar	Single hop, $O(n^2)$ wiring	Small subsystems, inside routers
Ring	Simple, low area, latency grows with n	Older Arm CCN, earlier Intel client ring bus
2D mesh	Regular, scalable, easy to floorplan	Arm CMN, many-core server CPUs, AI accelerators
Torus	Mesh plus wrap-around links, lower diameter, needs deadlock care	Some HPC/AI chips
Tree / irregular	Matches the floorplan and traffic	Commercial NoC generators (FlexNoC, NI-700)

Switching and routing

- **Store-and-forward:** a router receives the whole packet before forwarding; latency grows with packet length times hops.
- **Virtual cut-through:** forwarding starts as soon as the head arrives, but a router must have room for the whole packet.
- **Wormhole:** forwarding starts with the head flit and buffers are per flit; a packet stretches across several routers like a worm. Minimal buffering, the default for on-chip networks, but a blocked head stalls every link the worm occupies.
- **Dimension-order (XY) routing:** travel in X until the column matches, then in Y. Deterministic, trivially simple, in-order per source/destination pair, and deadlock-free on a mesh because no packet ever turns from Y back into X.
- **Adaptive routing** (west-first, odd-even turn models) routes around congestion but may reorder packets, which the NIU must then fix for protocols like AXI that require per-ID ordering.

Credit-based flow control

Flits must never be dropped, so each link uses **credit-based flow control**: the sender keeps a counter of free buffer slots in the receiver, decrements it for every flit sent and increments it when the receiver returns a credit (one per freed slot). For full throughput the buffer depth must cover the **credit round trip**: link latency forward, processing, and credit latency back. With a one-cycle link each way and one cycle of processing, a router needs at least three buffers per input (per virtual channel) to sustain one flit per cycle. A valid/ready link with a registered 'not full' READY, as in the example below, is the degenerate case of credits with zero return latency.

10.7 Virtual channels and deadlock

A **virtual channel (VC)** is a separate buffer queue sharing one physical link, with flits of different VCs interleaved on the wires. VCs serve three purposes:

- **Head-of-line blocking relief:** a blocked packet in one VC no longer blocks packets behind it in another VC.
- **Routing deadlock avoidance:** on rings and tori a cyclic dependency between buffers can deadlock even with minimal routing; a 'dateline' that moves packets to a second VC when they cross it breaks the cycle.
- **Protocol (message-dependent) deadlock avoidance:** a slave that cannot send a response because the network is full of requests that wait for that slave deadlocks the system. Requests and responses must use separate VCs or physically separate networks. That is why AXI NoCs have separate request and response networks, and why CHI defines four channel classes (REQ, RSP, SNP, DAT).

TWO KINDS OF DEADLOCK

Routing deadlock is a cycle in the channel-dependency graph of the network itself; it is removed by the routing function (XY, turn models) or by VCs with a dateline. **Protocol deadlock** is a cycle through the endpoints (request waits for response waits for request); it is removed by separating message classes and by guaranteeing that endpoints always sink responses. A NoC can be free of the first and still suffer the second.

Router microarchitecture

A classic input-queued VC router processes a head flit in stages: **RC** (route computation), **VA** (virtual-channel allocation at the output), **SA** (switch allocation, a separable arbiter per input and output), **ST** (switch traversal) and **LT** (link traversal). Body flits skip RC and VA. Production routers use lookahead routing and speculation to cut this to one or two cycles per hop at the target frequency; the physical-design team decides how many pipeline stages the links need across the floorplan.

10.8 Commercial NoCs and AMBA over a NoC

- **Arteris FlexNoC** (non-coherent) and **Ncore** (coherent): NIUs for AXI, AHB, APB, OCP and others; a configurable transport with its own packet format; widely used in mobile and automotive SoCs.
- **Arm CoreLink NIC-400 / NI-700:** AMBA network interconnects for AXI/AHB/APB; **CMN-600/CMN-700** are CHI coherent mesh networks with crosspoints (XPs), HN-F slices and SN-F ports.
- In-house NoCs at large SoC vendors, often generated from a topology description, plus chiplet extensions over UCIe (Chapter 22).

Transporting AXI over a NoC raises questions the NIU must answer: **ordering** (per-ID order across different destinations: the initiator NIU either stalls or reorders responses), **ID compression** (map wide AXI IDs to small NoC transaction tags), **4KB and burst splitting** (large bursts may be chopped into several packets), **QoS** (AxQoS to packet priority, with bandwidth regulators and pressure propagation), **clock and power domains** (asynchronous links and power-down handshakes inside the NoC), **security** (firewalls in the target NIU) and **safety** (ECC or parity on flits, duplicated routers for ASIL D).

10.9 Protocol comparison

Protocol	Topology	Outstanding / ordering	Bursts	Coherent	Typical use
APB	Bridge + decoder	1, in order	No	No	Registers
AHB-Lite / AHB5	Bus, multi-layer matrix	1 pipelined	INCR/WRAP, 1KB	No	MCUs, peripheral subsystems
AXI4	Crossbar / NoC	Many, per-ID order	Up to 256, 4KB	No (ACE: yes)	SoC backbone
AXI4-Lite	Crossbar	In order	No	No	FPGA register access

Protocol	Topology	Outstanding / ordering	Bursts	Coherent	Typical use
AXI4-Stream	Point to point	N/A (stream)	Packets (TLAST)	No	Data paths, video, DMA
ACE / ACE-Lite	Snooping interconnect	Many	Cache lines	Yes / I/O	CPU clusters (AMBA 4)
CHI	Mesh NoC	Many, retry and credits	Cache lines	Yes	Servers, high-end mobile
Wishbone B4	Shared bus / crossbar	1 (classic), queue (pipelined)	Via CTI/BTE	No	Open-source IP, LiteX
Avalon-MM	Platform Designer fabric	Pipelined reads, in order	burstcount	No	Intel FPGA
TileLink	Crossbar / NoC	Many (source IDs)	Power-of-2 sizes	TL-C: yes	RISC-V SoCs, OpenTitan
OCP	Sonics NoC etc.	Threads/tags	Configurable	Optional	Legacy SoCs

On-chip protocols at a glance (see Appendix A for the full quick-reference).

10.10 Hands-on: a 3x3 mesh of XY wormhole routers

`xy_router` has five ports (local, north, east, south, west), a 2-flit input buffer per port, XY route computation on the head flit and a per-output **lock**: when a head flit wins round-robin arbitration for an output, that output stays allocated to the same input until the tail flit has passed, which is wormhole switching. Links use valid/ready with `READY = 'buffer not full'`. `mesh3x3` instantiates nine routers with generate loops and ties off the mesh edges. Flits are 24 bits: head/tail flags, destination (x, y) and, for checking, a source/sequence tag in every flit.

```
// 5-port wormhole router with XY routing, 2-flit input buffers, valid/ready
// links (equivalent to 2 credits) and round-robin output allocation.
// Flit: [23] head [22] tail [21:20] dst x [19:18] dst y [17:0] tag + payload
module xy_router #(parameter int X = 0, parameter int Y = 0) (
    input logic          clk, rst_n,
    input logic [4:0]    in_valid, output logic [4:0] in_ready,
    input logic [119:0]  in_data,                    // 5 ports x 24 bits
    output logic [4:0]   out_valid, input logic [4:0] out_ready,
    output logic [119:0] out_data
);
    localparam int L = 0, N = 1, E = 2, S = 3, W = 4;        // port numbers

    function automatic int route(logic [23:0] f);            // dimension-order XY
        if (f[21:20] > X) return E;
        if (f[21:20] < X) return W;
        if (f[19:18] > Y) return N;
        if (f[19:18] < Y) return S;
        return L;
    endfunction

    // ----- input buffers (2 entries each) -----
    logic [23:0] head [5], tail [5]; logic [1:0] cnt [5]; logic [4:0] pop;
    always_comb for (int i = 0; i < 5; i++) in_ready[i] = (cnt[i] != 2);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) for (int i = 0; i < 5; i++) cnt[i] <= 0;
        else for (int i = 0; i < 5; i++) begin
            logic push; push = in_valid[i] && in_ready[i];
            if (pop[i]) head[i] <= tail[i];                // shift
            if (push) begin
                if (cnt[i] - pop[i] == 0) head[i] <= in_data[24*i +: 24];
            end
        end
endmodule
```

```

        else                                tail[i] <= in_data[24*i +: 24];
    end
    cnt[i] <= cnt[i] + push - pop[i];
end

// ----- output allocation (per-packet lock) -----
logic [4:0] lock; logic [2:0] owner [5], rr [5];
always_comb begin
    pop = '0;
    for (int o = 0; o < 5; o++) begin
        out_valid[o] = lock[o] && cnt[owner[o]] != 0;
        out_data[24*o +: 24] = head[owner[o]];
        if (out_valid[o] && out_ready[o]) pop[owner[o]] = 1'b1;
    end
end

always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin lock <= '0; for (int o = 0; o < 5; o++) rr[o] <= 0; end
    else for (int o = 0; o < 5; o++)
        if (lock[o]) begin // release after the tail
            if (out_valid[o] && out_ready[o] && out_data[24*o + 22]) lock[o] <= 1'b0;
        end else begin // round-robin grant
            logic done; done = 0;
            for (int k = 0; k < 5; k++) begin
                int i; logic busy; i = (rr[o] + k) % 5; busy = 0;
                for (int q = 0; q < 5; q++) if (lock[q] && owner[q] == i) busy = 1;
                if (!done && !busy && cnt[i] != 0 && head[i][23] && route(head[i]) == o) begin
                    lock[o] <= 1'b1; owner[o] <= i; rr[o] <= (i + 1) % 5; done = 1;
                end
            end
        end
    end
endmodule

// 3x3 mesh. Node k = 3*y + x. Port 1 = north (y+1), 2 = east (x+1),
// 3 = south, 4 = west, 0 = local. Link (k,p) = bit 5*k+p, data 24*(5*k+p).
module mesh3x3 (
    input logic clk, rst_n,
    input logic [8:0] inj_valid, output logic [8:0] inj_ready, input logic [215:0] inj_data,
    output logic [8:0] ej_valid, output logic [215:0] ej_data // ejection always ready
);
wire [44:0] iv, ir, ov, orr; wire [1079:0] id, od;
for (genvar y = 0; y < 3; y++) for (genvar x = 0; x < 3; x++) begin : node
    localparam int K = 3 * y + x;
    xy_router #(.X(x), .Y(y)) r (.clk, .rst_n,
        .in_valid(iv[5*K +: 5]), .in_ready(ir[5*K +: 5]), .in_data(id[120*K +: 120]),
        .out_valid(ov[5*K +: 5]), .out_ready(orr[5*K +: 5]), .out_data(od[120*K +: 120]));
    `define LINK(p, K2, p2) \
        assign iv[5*K+p] = ov[5*(K2)+p2]; \
        assign id[24*(5*K+p) +: 24] = od[24*(5*(K2)+p2) +: 24]; \
        assign orr[5*K+p] = ir[5*(K2)+p2];
    `define EDGE(p) \
        assign iv[5*K+p] = 1'b0; assign id[24*(5*K+p) +: 24] = '0; assign orr[5*K+p] = 1'b0;
    assign iv[5*K] = inj_valid[K]; assign id[24*5*K +: 24] = inj_data[24*K +: 24];
    assign inj_ready[K] = ir[5*K]; assign orr[5*K] = 1'b1;
    assign ej_valid[K] = ov[5*K]; assign ej_data[24*K +: 24] = od[24*5*K +: 24];
    if (y < 2) begin `LINK(1, K+3, 3) end else begin `EDGE(1) end // north
    if (x < 2) begin `LINK(2, K+1, 4) end else begin `EDGE(2) end // east
    if (y > 0) begin `LINK(3, K-3, 1) end else begin `EDGE(3) end // south
    if (x > 0) begin `LINK(4, K-1, 2) end else begin `EDGE(4) end // west
end
endmodule

```

xy_router (5-port wormhole router, XY routing) and mesh3x3.

Every node injects 40 packets of 1 to 4 flits to random destinations with random gaps, all nodes concurrently. The ejection side checks that each packet arrives whole, in order, at the right node, with the right payload, and measures the latency from head injection to tail ejection as a function of hop count. A monitor traces the links used by one 3-flit packet from node (0,0) to node (2,1).

```

module tb;
  logic clk = 0, rst_n = 1;  always #5 clk = ~clk;
  logic [8:0] inj_valid = 0, inj_ready, ej_valid;
  logic [215:0] inj_data = 0, ej_data;
  mesh3x3 dut (.*);

  // flit: [23]H [22]T [21:20]dx [19:18]dy [17:14]src [13:8]seq [7:0]data
  localparam int NP = 40; // packets per source
  logic [23:0] txf [9*256];  int ntx [9], ptr [9];
  int plen [9*64], pdst [9*64], t_inj [9*64];
  int cyc = 0, got = 0, errs = 0, lat_sum [5], lat_n [5];
  int exp_idx [9], cur_src [9], cur_seq [9]; // receiver state per node
  logic go = 0, trace = 1;

  function automatic logic [23:0] flit(bit h, bit t, int d, int s, int q, int b);
    return {h, t, 2'(d % 3), 2'(d / 3), 4'(s), 6'(q), 8'(b)};
  endfunction

  task automatic mk_pkt(int s, int q, int d, int len);
    plen[64*s + q] = len; pdst[64*s + q] = d;
    for (int b = 0; b < len; b++) begin
      txf[256*s + ntx[s]] = flit(b == 0, b == len - 1, d, s, q, (q * 16 + b) & 8'hFF);
      ntx[s] = ntx[s] + 1;
    end
  endtask

  // ---- injection: each node streams its flit list, random gaps between packets ----
  always @(posedge clk) if (go) begin
    cyc++;
    for (int k = 0; k < 9; k++) begin
      if (inj_valid[k] && inj_ready[k]) ptr[k]++;
      if (ptr[k] < ntx[k] && (!inj_valid[k] || inj_ready[k])) begin
        logic [23:0] f;  f = txf[256*k + ptr[k]];
        if (!f[23] || $urandom % 4 == 0) begin // heads wait a random time
          inj_valid[k] <= 1; inj_data[24*k +: 24] <= f;
          if (f[23]) t_inj[64*k + f[13:8]] = cyc;
        end else inj_valid[k] <= 0;
      end else if (ptr[k] >= ntx[k]) inj_valid[k] <= 0;
    end
  end

  // ---- ejection: check every packet arrives whole, in order, at the right node ----
  always @(posedge clk) if (go) for (int k = 0; k < 9; k++) if (ej_valid[k]) begin
    logic [23:0] f;  int s, q;
    f = ej_data[24*k +: 24];  s = f[17:14];  q = f[13:8];
    if (f[23]) begin cur_src[k] = s; cur_seq[k] = q; exp_idx[k] = 0; end
    if (s != cur_src[k] || q != cur_seq[k] || pdst[64*s + q] != k ||
        f[7:0] != ((q * 16 + exp_idx[k]) & 8'hFF) ||
        f[22] != (exp_idx[k] == plen[64*s + q] - 1))
      errs++;
    exp_idx[k]++;
    if (f[22]) begin
      int h;  h = (s % 3 > k % 3 ? s % 3 - k % 3 : k % 3 - s % 3)
        + (s / 3 > k / 3 ? s / 3 - k / 3 : k / 3 - s / 3);
    end
  end

```

```

        lat_sum[h] += cyc - t_inj[64*s + q]; lat_n[h]++; got++;
    end
end

// ---- trace the links used by packet (src 0, seq 0) ----
string pn [5];
always @(posedge clk) if (go && trace)
    for (int k = 0; k < 9; k++) for (int p = 0; p < 5; p++)
        if (dut.ov[5*k+p] && dut.orr[5*k+p] && dut.od[24*(5*k+p) + 17 -: 10] == 0)
            $display("cyc %2d router(%0d,%0d) -> %-5s %s flit data=%02h", cyc, k % 3, k / 3,
                pn[p], dut.od[24*(5*k+p)+23] ? "HEAD" : dut.od[24*(5*k+p)+22] ? "TAIL" : "BODY",
                dut.od[24*(5*k+p) +: 8]);

initial begin
    pn[0] = "LOCAL"; pn[1] = "NORTH"; pn[2] = "EAST"; pn[3] = "SOUTH"; pn[4] = "WEST";
    for (int k = 0; k < 9; k++) begin ntx[k] = 0; ptr[k] = 0; end
    for (int h = 0; h < 5; h++) begin lat_sum[h] = 0; lat_n[h] = 0; end
    mk_pkt(0, 0, 5, 3); // traced: (0,0) -> (2,1), 3 flits
    for (int k = 0; k < 9; k++)
        for (int q = (k == 0); q < NP; q++) begin
            int d; d = $urandom % 9; if (d == k) d = (d + 1) % 9;
            mk_pkt(k, q, d, 1 + $urandom % 4);
        end
    rst_n <= 0; repeat (2) @(posedge clk); rst_n <= 1; @(posedge clk); go <= 1;
    repeat (25) @(posedge clk); trace = 0;
    repeat (4000) @(posedge clk);
    $display("packets delivered=%0d/%0d errors=%0d", got, 9 * NP, errs);
    for (int h = 1; h < 5; h++)
        $display("hops=%0d packets=%3d avg latency (head inject -> tail eject) = %0.1f cyc",
            h, lat_n[h], real'(lat_sum[h]) / lat_n[h]);
    $finish;
end
endmodule

```

Testbench: concurrent random traffic on all nine nodes, packet integrity checks, latency statistics and a link trace.

```

cyc 4 router(0,0) -> EAST HEAD flit data=00
cyc 5 router(0,0) -> EAST BODY flit data=01
cyc 6 router(1,0) -> EAST HEAD flit data=00
cyc 7 router(0,0) -> EAST TAIL flit data=02
cyc 7 router(1,0) -> EAST BODY flit data=01
cyc 8 router(2,0) -> NORTH HEAD flit data=00
cyc 9 router(1,0) -> EAST TAIL flit data=02
cyc 9 router(2,0) -> NORTH BODY flit data=01
cyc 13 router(2,1) -> LOCAL HEAD flit data=00
cyc 14 router(2,0) -> NORTH TAIL flit data=02
cyc 14 router(2,1) -> LOCAL BODY flit data=01
cyc 15 router(2,1) -> LOCAL TAIL flit data=02
packets delivered=360/360 errors=0
hops=1 packets=130 avg latency (head inject -> tail eject) = 9.4 cyc
hops=2 packets=125 avg latency (head inject -> tail eject) = 12.7 cyc
hops=3 packets= 79 avg latency (head inject -> tail eject) = 15.5 cyc
hops=4 packets= 26 avg latency (head inject -> tail eject) = 18.4 cyc

```

Real Icarus Verilog output.

- The traced packet goes **east, east, then north**: XY routing finishes the X dimension first. Each hop costs 2 cycles here (one to buffer the flit, one to win the output), and the body and tail follow the head one cycle apart: a worm spanning three routers at cycle 7.
- Between cycles 9 and 13 the head waits inside router (2,1), whose local output is held by another packet; the tail waits one router behind and only crosses (2,0) -> north at cycle 14. The whole worm stalls in place without losing a flit.

- All 360 packets arrived intact with zero errors: no deadlock, no interleaving of packets on an output, correct payload order.
- Average latency rises by about 3 cycles per hop (9.4, 12.7, 15.5, 18.4 cycles for 1-4 hops): 2 cycles of router pipeline plus queueing under load, plus the serialization of up to 4 flits.

PITFALL: RELEASING THE LOCK ON THE WRONG FLIT

A wormhole router that releases its output allocation on a flit count instead of the tail flag, or that re-arbitrates an output while the current packet's input buffer is momentarily empty, interleaves flits of two packets on the same link. Downstream the packet structure is destroyed and the error surfaces far away, as a corrupted AXI burst at some target. Protocol checkers on every NoC link (head...tail contiguity per VC) find this in the first minutes of simulation.

10.11 Summary

- Wishbone B4 is the minimal open bus (CYC/STB/ACK), with classic and pipelined (STALL) modes and registered-feedback bursts.
- Avalon-MM (waitrequest, readdatavalid, burstcount) and Avalon-ST (readyLatency, empty, SOP/EOP) are Intel FPGA's equivalents of AXI4 and AXI4-Stream.
- TileLink has five channels (A-E), three conformance levels (TL-UL, TL-UH, TL-C) and a channel-priority rule for deadlock freedom; OCP and CoreConnect are legacy but still met in old IP.
- NoCs packetize transactions into flits, route them (XY on a mesh), use wormhole switching and credit-based flow control, and use virtual channels and separate message classes against routing and protocol deadlock.
- The 3x3 mesh delivered 360 random packets intact, showing wormhole stalls and latency growing with hop count.

10.12 Exercises

- Convert `wb_regs` to Wishbone pipelined mode: STALL low always, one request per cycle, ACK one cycle later. Write a testbench that issues four back-to-back reads with CYC held high.
- Map an AXI4 INCR8 read burst onto Avalon-MM and onto TileLink TL-UH. Which fields carry the length and the ID in each?
- Draw the channel-dependency graph of a 2x2 mesh with XY routing and show it is acyclic. Then add a single Y-then-X route and find the cycle.
- A router input has a 1-cycle link, 1-cycle credit return and 1-cycle internal pipeline. How many flit buffers per VC are needed for full throughput? What happens to throughput with 2 buffers?
- Modify `noc_tb` so that every node sends to node (2,2) (a hotspot). Predict and then measure the latency and the throughput of the ejection port.
- Explain why AXI over a NoC with adaptive routing needs a reorder buffer in the initiator NIU, and size it for 16 outstanding 64-byte reads.

PART III

Low-Speed Peripheral Protocols

Every SoC, from a 50-cent microcontroller to a data-centre CPU, is ringed with slow serial buses: a UART console, SPI boot flash, I2C sensors and power regulators, I3C on modern phones and DDR5 DIMMs, CAN and LIN in cars, I2S to the audio codec, MDIO to the Ethernet PHY. They run at kilobits to a few hundred megabits per second, but they are where bring-up starts, where many field bugs live, and where interview questions love to probe. This part takes each one down to the bit: frame formats, timing, electrical behaviour, error handling, controller RTL that was actually simulated, and the integration traps that catch real chips.

11. UART, RS-232 and RS-485

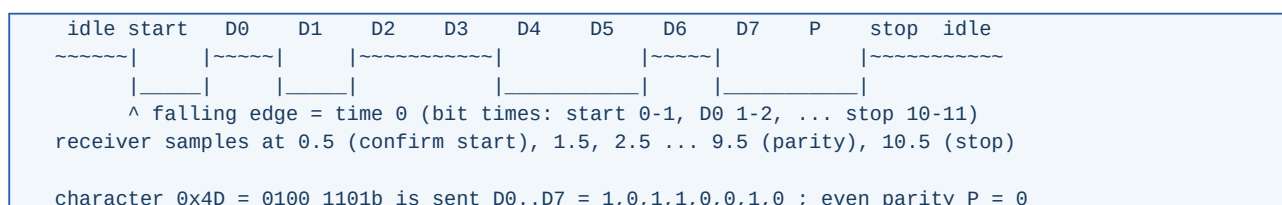
The Universal Asynchronous Receiver/Transmitter is the oldest serial link still found on every SoC. It has no clock wire: transmitter and receiver each run from their own clock and merely **agree** on a bit rate. Every character re-synchronizes the receiver with its start-bit edge, so the two clocks only need to stay aligned for about ten bit times. That single idea - resynchronize on every character, sample in the middle of each bit - explains the frame format, the oversampling receiver, the clock-tolerance budget and nearly every UART bug. The UART itself is only the logic layer; the voltages on the wire are set by separate standards (TTL/CMOS levels on the board, RS-232 to a PC, RS-422/RS-485 across a factory).

Where you find it	Typical use	Who owns what
Debug console	Boot ROM messages, bootloader, Linux console, firmware logs; often the first thing alive after reset	RTL: UART IP + pad mux; FW: divisor, driver; board: level shifter
Companion chips	Bluetooth/Wi-Fi combo (HCI UART with RTS/CTS at 3-4 Mbit/s), GNSS receivers, cellular modems	RTL: flow control, FIFOs, DMA; DV: long-burst and flow tests
Industrial / automotive	RS-485 Modbus, DMX512, LIN (a UART with a special header, Chapter 15)	Board: transceiver and termination; RTL: driver-enable timing, break detection
Manufacturing / security	Test-mode access, secure-boot recovery console	Security: console must be lockable (fuses, debug authentication)

11.1 The asynchronous frame

The line **idles high** (logic 1, historically called **mark**). A character is sent as:

- one **start bit** (logic 0, **space**) - its falling edge is the timing reference for the entire character;
- **5 to 9 data bits, least-significant bit first** (8 is by far the most common; 9-bit mode is used for multiprocessor addressing, the ninth bit marking an address byte);
- an optional **parity bit**: even, odd, or the rarely used mark (always 1) / space (always 0) parity;
- **1, 1.5 or 2 stop bits** (logic 1). The stop bit guarantees an idle-high period so the next start bit produces a falling edge.



An 8E1 frame (8 data bits, even parity, 1 stop): 11 bit times per character. 8N1 (no parity) is 10 bit times, so 115200 baud carries at most 11520 bytes/s.

The shorthand **8N1**, **7E1**, **8E1**, **8N2** gives data bits, parity (None/Even/Odd/Mark/Space) and stop bits. Efficiency is $8/10 = 80\%$ for 8N1 and $8/11 = 73\%$ for 8E1. **Baud** is symbols per second; on a UART one symbol is one bit, so baud = bit/s here, but the distinction matters once multi-level line codes appear (Chapter 2).

WHY LSB FIRST?

The earliest teleprinters and UART chips shifted characters out of a shift register toward bit 0, and every later standard kept it. Almost everything else in this book (SPI, I2C, CAN, Ethernet headers aside) is MSB first, so LSB-first order is a classic source of bit-reversed data when a UART frame is decoded by hand or with a logic analyser configured for the wrong order.

11.2 Baud-rate generation and divisor error

A UART needs a tick at a multiple of the baud rate - normally 16x, sometimes 8x or 4x at high rates. The classic 16550 divides its input clock by a 16-bit integer divisor (DLL/DLM registers):

```
divisor = round( f_clk / (16 * baud) )
actual  = f_clk / (16 * divisor)
error % = 100 * (actual - baud) / baud
```

Integer divisor arithmetic of a 16x-oversampling UART.

The 1.8432 MHz crystal (and its multiples 3.6864, 7.3728, 14.7456, 18.432 MHz) exists precisely because 1.8432 MHz / 16 = 115200 divides evenly into every standard rate. An SoC rarely has such a clock; it has 24, 48, 50 or 100 MHz, and the integer divisor error can be large at high baud rates. Modern UARTs add a **fractional divisor** (a 4-bit or larger fraction, dithered by an accumulator) or a fully programmable **NCO** (numerically controlled oscillator), as the RTL below does. The Python model shows why:

```
# Integer 16550-style divisor vs a divisor with a 4-bit fraction (1/16 steps)
print("f_clk(MHz)  baud  int_div  err_int  frac_div  err_frac")
for fclk in (1.8432e6, 48e6, 50e6):
    for baud in (9600, 115200, 921600, 3000000):
        if baud > fclk / 16:
            continue
        ideal = fclk / (16 * baud)
        di = max(1, round(ideal))
        df = max(1, round(ideal * 16) / 16)
        ei = 100 * (fclk / (16 * di) - baud) / baud
        ef = 100 * (fclk / (16 * df) - baud) / baud
        print("%8.4f  %7d  %7d  %+6.2f%%  %9.4f  %+6.2f%%" % (fclk / 1e6, baud, di, ei, df, ef))
```

Divisor error for integer and 1/16-fractional dividers (Python 3).

f_clk(MHz)	baud	int_div	err_int	frac_div	err_frac
1.8432	9600	12	+0.00%	12.0000	+0.00%
1.8432	115200	1	+0.00%	1.0000	+0.00%
48.0000	9600	312	+0.16%	312.5000	+0.00%
48.0000	115200	26	+0.16%	26.0625	-0.08%
48.0000	921600	3	+8.51%	3.2500	+0.16%
48.0000	3000000	1	+0.00%	1.0000	+0.00%
50.0000	9600	326	-0.15%	325.5000	+0.01%
50.0000	115200	27	+0.47%	27.1250	+0.01%
50.0000	921600	3	+13.03%	3.3750	+0.47%
50.0000	3000000	1	+4.17%	1.0625	-1.96%

With a 50 MHz clock an integer divisor gives +13 % at 921600 baud - a guaranteed failure - while a 1/16 fraction brings it to 0.47 %. Note also 3 Mbaud from 50 MHz: the ideal divisor is 1.04, so even the fraction leaves -1.96 %, which eats most of the budget computed next. That is why high-speed UARTs support 8x or 4x oversampling (more divisor resolution per bit) or a dedicated baud clock.

11.3 Receiver: 16x oversampling and mid-bit sampling

The receiver knows nothing about the transmitter's clock. It oversamples the (synchronized) RX pin at 16 ticks per bit and runs this algorithm:

- **Idle:** wait for a 1 -> 0 transition (the start edge). It is observed with up to one tick (1/16 bit) of uncertainty.
- **Confirm start:** count 8 ticks to the middle of the start bit and check the line is still 0. If it is 1, it was a glitch (false start) - go back to idle. This is a free noise filter.
- **Data:** every 16 ticks thereafter the counter is at the middle of the next bit; sample it. Better receivers take three samples (ticks 7, 8, 9) and use a **majority vote**, and flag noise if they disagree.

- **Parity / stop:** sample parity and stop the same way. A 0 where the stop bit should be is a **framing error**.
- Return to idle **at the middle of the stop bit** - not the end - so that the next start edge, which may come as early as the end of the stop bit from a slightly fast transmitter, is not missed.

RX (after 2-FF sync)	~~~~~_____ start _____/~~~~ D0 ~~~~
16x ticks	
counter	0 1 2 3 4 5 6 7 8 0 1 2 ... 15
	^ mid-start: still 0? ^ mid-D0: sample
edge detected up to 1 tick late -> every sample point may be up to 1/16 bit late	

The 16x receiver. After confirming the start bit, every subsequent sample is exactly 16 ticks later.

11.4 The clock tolerance budget

Because the receiver resynchronizes only at the start edge, the timing error accumulates across the character. The last sample - the middle of the stop bit - is 9.5 bit times after the edge for 8N1 (10.5 for 8E1 or 9-bit data). For that sample to land inside the correct bit, the accumulated drift must stay below half a bit, minus the edge-detection uncertainty:

$$|df/f| \times 9.5 \text{ bits} < 0.5 \text{ bit} - 1/16 \text{ bit (edge quantization, 16x)}$$

$$|df/f| < (0.5 - 0.0625) / 9.5 = 4.6 \% \quad (8N1, \text{ total of both ends})$$

$$|df/f| < (0.5 - 0.0625) / 10.5 = 4.2 \% \quad (8E1 / 9\text{-bit data})$$

Total allowable baud mismatch between transmitter and receiver.

That total is shared by **both** ends, and it must also absorb asymmetric rise/fall times, transceiver skew and noise, which shift the effective edge. The practical engineering rule is therefore **about 2 % per side, preferably under 1-1.5 %** - a crystal-based clock (tens of ppm) is effortless, an on-chip RC oscillator (often +/-1 % to +/-5 % uncalibrated) is not. This is the single number to remember: when a customer reports 'garbled characters at high baud rates', compute the divisor error first.

The testbench below proves the budget with real RTL. The transmitter and receiver each have their own NCO baud generator; the receiver is exact (115200 baud at 100 MHz) and the transmitter's increment is offset in 0.5 % steps. 64 back-to-back characters are sent - the worst case, because there is no idle time to absorb drift - alternating random data with 0x00 (whose stop bit follows eight zeros).

```
// Fractional (NCO) baud-tick generator: tick rate = f_clk * INC / 2^24
module baud_nco (
    input logic      clk, rst_n,
    input logic [23:0] inc,           // = 2^24 * 16 * baud / f_clk
    output logic     tick16           // one pulse per 1/16 bit
);
    logic [24:0] acc;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin acc <= '0; tick16 <= 1'b0; end
        else begin
            acc <= {1'b0, acc[23:0]} + inc;
            tick16 <= acc[24];           // carry out = one tick
        end
endmodule

module uart_tx (                // 8N1 transmitter
    input logic      clk, rst_n, tick16,
    input logic      start, input logic [7:0] data,
    output logic     txd, busy
);
    logic [9:0] sh; logic [3:0] nbit, ph;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin sh <= '1; busy <= 0; nbit <= 0; ph <= 0; end
        else if (!busy && start) begin
```

```

        sh <= {1'b1, data, 1'b0};          // stop, D7..D0, start (LSB goes first)
        busy <= 1; nbit <= 0; ph <= 0;
    end else if (busy && tick16) begin
        ph <= ph + 1;
        if (ph == 15) begin                // 16 ticks = one bit time
            sh <= {1'b1, sh[9:1]};
            nbit <= nbit + 1;
            if (nbit == 9) busy <= 0;      // start + 8 data + stop sent
        end
    end
    assign txd = busy ? sh[0] : 1'b1;
endmodule

module uart_rx (                          // 8N1 receiver, 16x oversampling
    input  logic      clk, rst_n, tick16, rxd_async,
    output logic      valid, ferr,        // one-cycle pulses
    output logic [7:0] data
);
    typedef enum logic [1:0] {IDLE, START, DATA, STOP} st_t;
    st_t st; logic [3:0] ph; logic [2:0] nbit; logic r1, rxd;
    always_ff @(posedge clk or negedge rst_n) // 2-flop synchronizer
        if (!rst_n) {rxd, r1} <= 2'b11; else {rxd, r1} <= {r1, rxd_async};
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin st <= IDLE; ph <= 0; nbit <= 0; valid <= 0; ferr <= 0; data <= 0; end
        else begin
            valid <= 0; ferr <= 0;
            if (tick16) case (st)
                IDLE: if (!rxd) begin st <= START; ph <= 0; end // falling edge seen
                START: if (ph == 7) begin // middle of the start bit
                    st <= rxd ? IDLE : DATA; // high again = glitch, not a start
                    ph <= 0; nbit <= 0;
                end else ph <= ph + 1;
                DATA: if (ph == 15) begin // 16 ticks later = middle of next bit
                    data <= {rxd, data[7:1]}; // LSB first
                    ph <= 0; nbit <= nbit + 1;
                    if (nbit == 7) st <= STOP;
                end else ph <= ph + 1;
                STOP: if (ph == 15) begin
                    valid <= rxd; ferr <= !rxd; // stop bit must be 1
                    st <= IDLE; ph <= 0;
                end else ph <= ph + 1;
            endcase
        end
endmodule

```

An NCO baud-tick generator plus 8N1 transmitter and 16x oversampling receiver with false-start rejection and framing-error detection. The receiver samples once per bit at tick 7/15 (a production design would take a 3-sample majority vote and add parity and break detection).

```

module tb;
    logic clk = 0, rst_n = 0; always #5 clk = ~clk; // 100 MHz
    localparam real NOM = 16.0 * 115200.0 * 16777216.0 / 100.0e6;
    logic [23:0] inc_tx, inc_rx; logic t_tx, t_rx;
    logic start = 0, busy, txd, valid, ferr; logic [7:0] d, q;
    baud_nco gtx (.clk, .rst_n, .inc(inc_tx), .tick16(t_tx));
    baud_nco grx (.clk, .rst_n, .inc(inc_rx), .tick16(t_rx));
    uart_tx tx (.clk, .rst_n, .tick16(t_tx), .start, .data(d), .txd, .busy);
    uart_rx rx (.clk, .rst_n, .tick16(t_rx), .rxd_async(txd), .valid, .ferr, .data(q));
    int ERR[] = {-60, -55, -50, -45, -30, 0, 30, 45, 50, 55, 60};
    byte exp_q[$]; int good, bad, fe;
    always @(posedge clk) if (valid || ferr) begin

```

```

    if (valid && q == exp_q[0]) good++; else bad++;
    if (ferr) fe++;
    void'(exp_q.pop_front());
end
initial begin
    inc_rx = $rtoi(NOM);
    $display("TX baud error sent rx_ok rx_bad framing_err");
    foreach (ERR[k]) begin int e; e = ERR[k]; // TX error in 0.1 % steps
        inc_tx = $rtoi(NOM * (1.0 + e / 1000.0));
        rst_n = 0; good = 0; bad = 0; fe = 0; exp_q.delete();
        repeat (4) @(posedge clk); rst_n = 1;
        for (int i = 0; i < 64; i++) begin // back-to-back frames
            d = (i % 2) ? 8'h00 : 8'($urandom);
            exp_q.push_back(d);
            @(posedge clk) start = 1; @(posedge clk) start = 0;
            wait (busy == 0);
        end
        repeat (3000) @(posedge clk);
        $display(" %5.1f %6d %2d %2d %2d", e / 10.0, good, bad, fe);
    end
    $finish;
end
endmodule

```

Testbench: sweep the transmitter's baud error and count good characters, bad characters and framing errors.

TX baud error	sent	rx_ok	rx_bad	framing_err
-6.0 %	64	15	49	49
-5.5 %	64	48	16	16
-5.0 %	64	64	0	0
-4.5 %	64	64	0	0
-3.0 %	64	64	0	0
0.0 %	64	64	0	0
3.0 %	64	64	0	0
4.5 %	64	64	0	0
5.0 %	64	1	61	54
5.5 %	64	0	61	51
6.0 %	64	0	61	53

Real Icarus Verilog output. Error-free from -5.0 % to +4.5 %; at +5 % (transmitter fast) almost everything is lost, at -5.5 % (transmitter slow) framing errors appear.

The measured window, about -5.0 % to +4.5 % with zero errors, sits right around the 4.6 % theory; it is slightly asymmetric because the 2-flop synchronizer and the tick phase delay every sample point by a fraction of a tick, which helps a slow transmitter and hurts a fast one. On a real board that margin is consumed by the other end's error and by edge distortion - never design to the simulated edge.

PITFALL: THE FAST TRANSMITTER AND THE MISSING START EDGE

When the transmitter is fast, its next start bit begins before the receiver has finished the stop bit. A receiver that waits until the **end** of the stop bit before looking for the next falling edge will miss it (or sample it late) on back-to-back characters. Leave the stop state at mid-bit, as the RTL above does. Similarly, a transmitter should honour the programmed stop-bit count exactly - some FIFO-less designs 'save a cycle' and send short stop bits.

11.5 Line conditions: errors and break

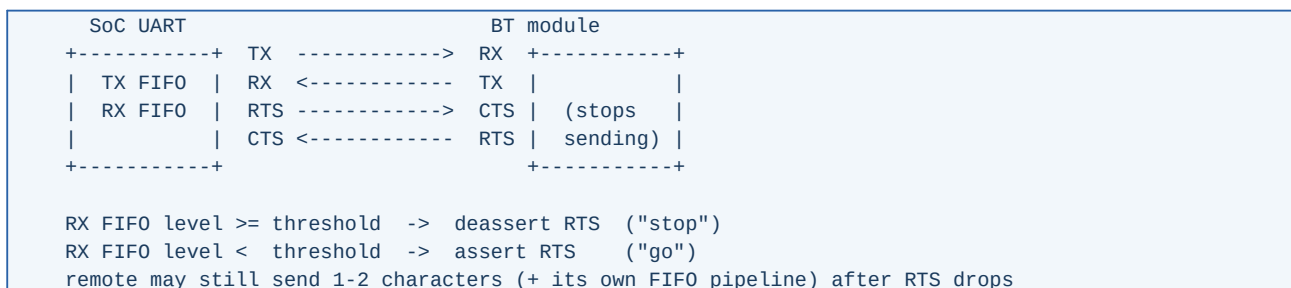
Condition	Detected when	Typical cause	Reported as
Framing error	Stop bit sampled as 0	Baud mismatch, noise, wrong frame format, a break	LSR.FE, per character in the RX FIFO
Parity error	Received parity does not match	Noise, wrong parity setting	LSR.PE, per character

Condition	Detected when	Typical cause	Reported as
Overrun error	A character completes while the receive buffer/FIFO is full; it (or the oldest) is lost	Software or DMA too slow; no flow control	LSR.OE
Break	Line held at 0 for longer than a full character (start + data + parity + stop)	Deliberate signal (SysRq, LIN header, bootloader entry) or a disconnected RS-232 cable	LSR.BI with a 0x00 character
Noise / glitch	Majority-vote samples disagree, or start bit not confirmed	EMI, slow edges	Some UARTs flag it; others silently filter
RX timeout	Characters in the FIFO but no new one for about 4 character times	End of a message shorter than the FIFO trigger level	Timeout interrupt (not an error)

A **break** is sent by forcing TX low (LCR bit 6 on a 16550) for at least one character time, usually longer. Receivers detect it as a 0x00 character with a framing error and all-zero bits, and then must wait for the line to return high before looking for a new start bit - otherwise the long low is decoded as a stream of garbage characters.

11.6 Hardware flow control: RTS/CTS

A UART has no acknowledgement, so the receiver throttles the sender with two extra wires. Each side drives **RTS** (request to send - in modern use 'ready to receive') and reads the other's RTS on its **CTS** input. The transmitter only starts a new character while CTS is asserted (active low on the wire).



RTS/CTS crossed between two UARTs (null-modem wiring).

- **Auto flow control** (16750/16950-class UARTs and most SoC UARTs): hardware drives RTS from the RX FIFO level and gates the TX engine with CTS, with no software in the loop. Software-driven RTS cannot react within one character at 3 Mbit/s (a character is 3.3 us).
- Set the RTS threshold with **headroom**: the remote will finish the character in flight and may already have committed the next one. Leave at least 2-4 free FIFO entries.
- CTS is asynchronous - synchronize it, and sample it only at a character boundary; stopping mid-character corrupts the frame.
- **Software flow control** (XON = 0x11, XOFF = 0x13) sends control characters in-band; it is slow and cannot carry binary data without escaping.
- RS-232 also defines DTR/DSR/DCD/RI modem-control lines; on SoCs they are usually GPIOs or absent.

11.7 The 16550 register model

The National Semiconductor 16550 (and its 8250 ancestor) defined a register interface that is still emulated by countless SoC UARTs, virtual machines and the Linux [8250](#) driver. Eight byte-wide registers occupy eight addresses (often at a 4-byte stride on a 32-bit APB bus); the **DLAB** bit in LCR re-maps offsets 0 and 1 to the divisor latch.

Off	Read	Write	Key bits
0	RBR receive buffer (DLAB=0) / DLL	THR transmit holding (DLAB=0) / DLL	DLL/DLM = 16-bit divisor, low/high byte
1	IER / DLM (DLAB=1)	IER / DLM	IER: RX data, THR empty, line status, modem status interrupt enables
2	IIR interrupt identification	FCR FIFO control	FCR: FIFO enable, reset RX/TX FIFO, RX trigger 1/4/8/14; IIR: pending interrupt by priority
3	LCR line control	LCR	Word length 5-8, stop bits, parity enable/even/stick, break (bit 6), DLAB (bit 7)
4	MCR modem control	MCR	DTR, RTS, OUT1/OUT2, loopback (bit 4); auto-flow enable on 16750-class
5	LSR line status	-	DR, OE, PE, FE, BI, THRE, TEMT, RX FIFO error (bit 7)
6	MSR modem status	-	CTS, DSR, RI, DCD and their delta (changed) bits
7	SCR scratch	SCR	No function; used to probe presence

IIR reports the highest-priority pending interrupt: receiver line status (errors/break) first, then received data available (FIFO at trigger level), then **character timeout**, then THR empty, then modem status. Two behaviours trip up RTL re-implementations: reading IIR clears a pending THR-empty interrupt, and reading LSR clears the error bits - so these reads have **side effects** that a register model and a DV scoreboard must replicate exactly (and that a debugger reading registers can disturb).

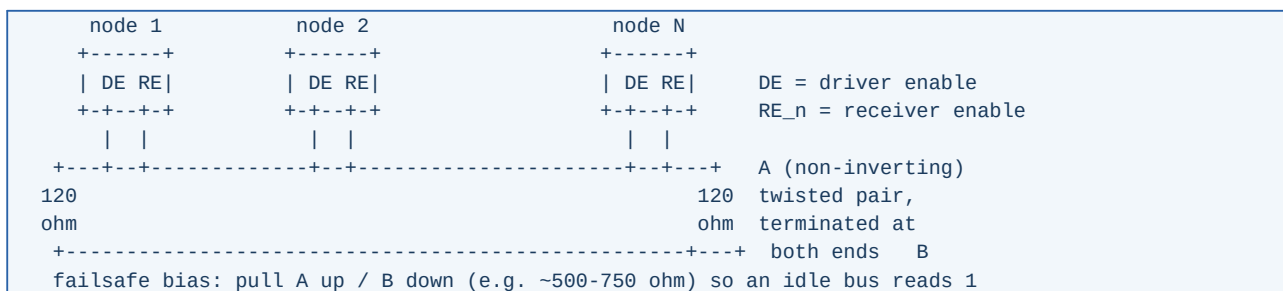
INTERVIEW INSIGHT: WHY A TEMT BIT AS WELL AS THRE?

THRE says the transmit holding register (or FIFO) is empty - software can write more. TEMT says the **shift register** is empty too, i.e. the last stop bit has left the pin. Anything that must wait for the line to be truly idle - turning an RS-485 driver off, changing baud rate, entering sleep - must wait for TEMT, not THRE. Waiting for THRE and then disabling the RS-485 driver cuts off the last character, a bug found in many drivers.

11.8 Electrical layers: TTL, RS-232, RS-422 and RS-485

The SoC pad speaks CMOS levels (idle high = 1.8 V or 3.3 V). Everything else is a transceiver on the board:

Standard	Signalling	Logic 1 / 0	Topology	Rate x distance (typical)
TTL/CMOS UART	Single-ended, ground referenced	VDD / 0 V	Point-to-point, same board	Mbit/s over cm
RS-232 (TIA-232)	Single-ended, inverted , bipolar	Mark (1) = -3 to -15 V; space (0) = +3 to +15 V; drivers typically +/-5 to +/-12 V	Point-to-point	Specified to 20 kbit/s over ~15 m; in practice 115-1000 kbit/s over short cables
RS-422 (TIA-422)	Differential, one driver	A-B > +200 mV = 1, < -200 mV = 0	1 driver, up to 10 receivers	10 Mbit/s over ~12 m; ~100 kbit/s over 1200 m
RS-485 (TIA-485)	Differential, multi-driver , tri-state	Same thresholds; driver >= 1.5 V into 54 ohm	Multi-drop bus, 32 unit loads (more with 1/4-1/8 UL receivers)	Same rate-distance family as RS-422; common-mode -7 to +12 V



RS-485 two-wire half-duplex bus. Only one node drives at a time; termination matches the ~120 ohm cable impedance.

RS-485 two-wire is **half duplex**: every node's transmitter shares the pair, so the UART must switch its transceiver's **driver enable (DE)** on only while it transmits. The timing is critical:

- Assert DE a little before the start bit (a 'pre-delay' of a fraction of a bit) so the driver's output has settled.
- Deassert DE only after the **last stop bit has left** (TEMT, not THRE), optionally plus a post-delay; too early truncates the stop bit, too late collides with a fast responder.
- Many SoC UARTs have an **RS-485 mode** that drives DE from the transmitter FSM in hardware with programmable assertion/de-assertion times - far more reliable than toggling a GPIO from an interrupt handler.
- When the node's own receiver is enabled while it transmits it hears its own echo; either disable RE or discard the echo (it is also a handy collision check).
- An idle, undriven bus floats near 0 V differential, which is inside the +/-200 mV undefined band: **failsafe biasing** resistors or failsafe receivers make idle read as 1.

11.9 Modbus RTU: a protocol on top of RS-485

Modbus RTU is the lingua franca of industrial RS-485. A master polls slaves (addresses 1-247; 0 is broadcast) with frames delimited by **silence**: a gap of at least **3.5 character times** starts a new frame, and a gap of more than 1.5 character times inside a frame is an error. The frame is **address, function code, data, CRC-16** (the Modbus CRC uses polynomial 0x8005 in reflected form, 0xA001, initial value 0xFFFF, sent low byte first). Standard characters are 11 bits (8E1, or 8N2 when parity is off). Common function codes: 03 read holding registers, 04 read input registers, 06 write single register, 16 (0x10) write multiple registers; an exception response sets bit 7 of the function code.

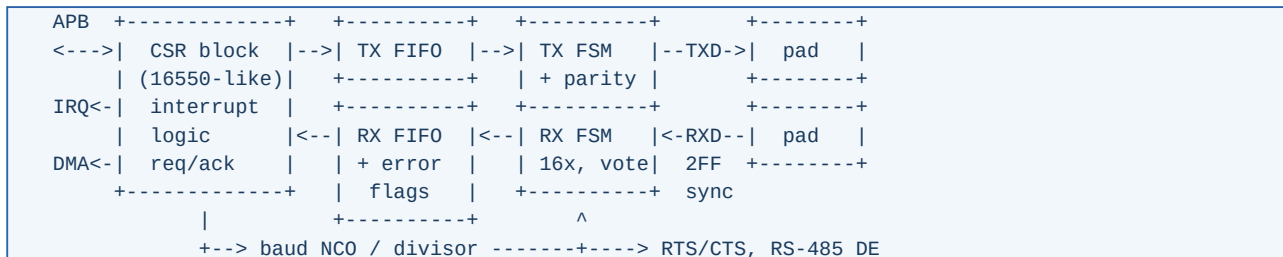
HARDWARE HELP FOR MODBUS

The 3.5-character silence is awkward in software at high baud rates (at 115200 baud it is only ~330 us; the Modbus spec fixes it at 1.75 ms above 19200 baud for that reason). SoC UARTs aimed at industrial use provide a programmable **receiver idle timeout** interrupt and sometimes an address-match mode, so software sees whole frames. That is the same RX-timeout logic as the 16550's character timeout, with a programmable length.

11.10 Auto-baud detection

A receiver that does not know the rate can measure it. The simplest method times the **start bit** (the first low period) of a known character with a fast counter; better methods time several edges of a known pattern. The pattern must begin with a single 0 bit followed by a 1: 0x55 ('U') alternates perfectly (0 1 0 1 0 1 0 1 0 1 on the wire), and the 'A'/'a' of the Hayes 'AT' command also starts with a single-bit low. LIN's sync byte is exactly 0x55 for this reason (Chapter 15). In RTL, auto-baud is a counter that runs from the falling edge to the rising edge, divides by the number of bits measured, and loads the result into the NCO or divisor - with a guard against glitches and a timeout.

11.11 Building and verifying a UART in an SoC



Block diagram of a typical SoC UART.

- **Clocking:** the UART usually has its own functional clock (for an exact baud reference) separate from the APB bus clock; the CSR block then crosses domains (Chapter 4). Keeping the baud clock alive in low-power modes allows wake-on-RX.
- **FIFOs** store per-character error flags alongside the data so software knows which byte had the parity error.
- **DMA:** RX request at the FIFO trigger level, plus the character-timeout interrupt to flush short messages - the classic bug is a DMA RX path that stalls with the last few bytes of a message stuck below the trigger level.
- **DV:** a UART VIP (a behavioural transmitter/receiver with programmable baud error, glitch and break injection), a scoreboard that models the 16550 side effects, coverage on every data-bits/parity/stop combination and on each error type, and **loopback mode** tests (MCR bit 4) that firmware reuses on silicon.
- **Silicon bring-up:** a UART is often driven by the boot ROM before PLLs are configured, so its divisor must be correct for the reset-default reference clock (for example 24 MHz or 38.4 MHz) - check the ROM's baud table against the actual crystal.

UART integration checklist

- ☐ Divisor/NCO error computed for every supported baud rate and every reference clock the product uses ($\leq \sim 1.5\%$).
- ☐ RX pin 2-FF synchronized; start confirmed at mid-bit; stop state exits at mid-stop-bit.
- ☐ Break detection waits for the line to return high.
- ☐ TEmT (not THRE) used for RS-485 DE and for baud-rate changes.
- ☐ RTS threshold leaves headroom; CTS sampled only between characters.
- ☐ RX timeout interrupt works with DMA; per-character error flags stored in the FIFO.
- ☐ Pad has a pull-up on RX so an unconnected console does not produce break storms; console can be disabled for secure products.

11.12 Summary

- A UART frame is idle-high, start bit (0), 5-9 data bits LSB first, optional parity, 1-2 stop bits (1); every character resynchronizes the receiver on its start edge.
- Receivers oversample (usually 16x), confirm the start bit at mid-bit, then sample every 16 ticks; the last sample is 9.5 bits (8N1) after the edge.
- Total baud mismatch must stay below about 4.6 % (8N1, 16x); budget about 2 % per side. The simulated RTL worked from -5.0 % to +4.5 % and failed beyond.
- Integer divisors give large errors at high baud rates on non-UART-friendly clocks; use fractional dividers or an NCO.
- Errors: framing, parity, overrun, break; plus the RX character timeout that makes FIFOs and DMA usable.
- RTS/CTS provides back-pressure; RS-232 is inverted bipolar single-ended; RS-422/485 are differential, and RS-485 is a half-duplex multi-drop bus that needs precise driver-enable timing and failsafe biasing.

11.13 Exercises

- 1 A UART runs from a 26 MHz clock. Compute the integer divisor and the error for 115200 and 460800 baud with 16x oversampling. Would 8x oversampling help at 460800?
- 2 Derive the maximum tolerable total baud mismatch for 7E2 frames with 8x oversampling. Which term grew and which shrank?
- 3 Modify the `uart_rx` in this chapter to take a 3-sample majority vote at ticks 7, 8 and 9 and to report a noise flag. Re-run the baud sweep; does the window change?
- 4 Design the RS-485 DE control for a UART: DE rises one-half bit before the start bit and falls one bit after TEMT. Draw the FSM and list the corner cases (back-to-back writes, FIFO refilled during the post-delay).
- 5 A customer sees one corrupted byte every few thousand when a Bluetooth module streams at 3 Mbaud with RTS/CTS enabled. List three possible root causes and how you would distinguish them on silicon.
- 6 Write the Modbus RTU CRC-16 in Python and compute it for the request `01 03 00 00 00 0A` (read 10 holding registers from slave 1). Which byte is transmitted first?

12. SPI, Quad/Octal SPI and xSPI

The Serial Peripheral Interface is the opposite of the UART: it carries its own clock, has no framing, no addressing, no acknowledgement and no formal standard - just a controller that toggles a clock and two shift registers that exchange bits. That simplicity lets it scale from a 1 MHz ADC link to a 200 MHz double-data-rate octal flash that an application processor boots and executes from. This chapter starts with the four-wire bus and its four clock modes, then follows the flash market's evolution: Dual and Quad I/O, QPI, Octal DDR with a data strobe (JEDEC xSPI) and HyperBus, and the execute-in-place controllers that make serial flash look like memory.

Where you find it	Examples
Boot and code storage	SPI NOR / Quad / Octal flash holding the boot loader, firmware or the whole MCU program (XIP)
Data converters and sensors	ADCs, DACs, IMUs, touch controllers - streams with tight timing, often 10-50 MHz
Displays and radios	Small LCD/OLED panels, Wi-Fi/BLE and sub-GHz radio chips, Ethernet controllers
Inside the SoC ecosystem	PMIC configuration, secure elements, TPMs (TPM over SPI), FPGA configuration, DDR5-era sideband components use I3C instead (Chapter 14)

12.1 Four wires, two shift registers

SPI signals (the industry has moved from master/slave to controller/peripheral naming; both appear in datasheets):

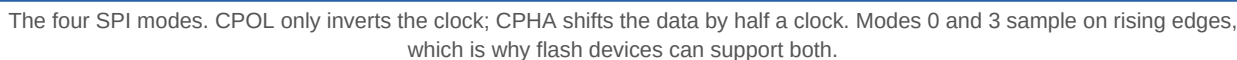
Signal	Also called	Driven by	Function
SCLK	SCK, CLK	Controller	Serial clock; only toggles during a transfer
MOSI	SDO (at controller), SDI (at peripheral), PICO, COPI	Controller	Data from controller to peripheral
MISO	SDI (at controller), SDO (at peripheral), POCI, CIPO	Selected peripheral	Data to the controller; tri-stated when the peripheral is not selected
CS_n	SS_n, CE_n, nCS	Controller	Active-low chip select, one per peripheral; frames the transaction



SPI is an exchange: every clock moves one bit each way, so reading requires clocking out (dummy) bits and writing always receives bits.

SPI is inherently **full duplex**, but most protocols built on it use it half-duplex in practice: a command goes out, then data comes back while the controller sends don't-care bits. Word length is whatever the peripheral expects (8, 12, 16, 24 or 32 bits); CS_n delimits the transaction and, for many devices, the falling edge of CS_n resets the peripheral's bit counter - which is why SPI is fairly robust to a missed clock: the next CS_n cycle resynchronizes.

Mode	CPO L	CPH A	SCLK idle	Sample on	Shift out on	Common for
0	0	0	low	rising (leading)	falling; first bit at CS_n fall	Most flash, most peripherals
1	0	1	low	falling (trailing)	rising	Some ADCs and sensors
2	1	0	high	falling (leading)	rising; first bit at CS_n fall	Rare
3	1	1	high	rising (trailing)	falling	Flash (modes 0 and 3 both supported), SD in SPI mode



Treat `sc1k XOR CP0L` as the internal clock. Then there are only two cases: `CPHA=0` - data must be ready **before** the first edge and is sampled on every leading edge; `CPHA=1` - data is driven on every leading edge and sampled on every trailing edge. The RTL below and its slave model are written exactly this way.

```

module spi_master #(parameter int HALF = 2) (    // SCLK half-period in clk cycles
    input  logic      clk, rst_n,
    input  logic      cpol, cpha,                // mode = {cpol, cpha}
    input  logic      start, input logic [7:0] tx,
    output logic      done,  output logic [7:0] rx,
    output logic      sclk, mosi, cs_n,
    input  logic      miso
);
    logic [7:0] sh;  logic [4:0] edges;  logic [$clog2(HALF+1)-1:0] cnt;
    logic      lvl, busy;                // lvl: SCLK before CPOL inversion
    wire      tick    = busy && (cnt == HALF-1);
    wire      leading = (lvl == 1'b0);    // edge about to be made is leading
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            busy <= 0; cs_n <= 1; lvl <= 0; cnt <= 0; edges <= 0; done <= 0;
            sh <= 0; rx <= 0; mosi <= 0;
        end else begin
            done <= 0;
            cnt  <= tick ? '0 : (busy ? cnt + 1 : '0);
            if (!busy && start) begin
                busy <= 1; cs_n <= 0; edges <= 0; lvl <= 0;
                sh <= tx;  mosi <= cpha ? 1'b0 : tx[7]; // CPHA=0: first bit valid at CS fall
            end else if (tick && edges == 16) begin // half-period CS hold, then finish
                busy <= 0; cs_n <= 1; done <= 1;
            end else if (tick) begin

```

```

        lvl    <= ~lvl; edges <= edges + 1;
        if (leading ^ cpha) rx <= {rx[6:0], miso};           // sample edge
        else if (cpha)      begin mosi <= sh[7]; sh <= {sh[6:0], 1'b0}; end
        else if (edges != 15) begin mosi <= sh[6]; sh <= {sh[6:0], 1'b0}; end
    end
end
assign sclk = lvl ^ cpol;                                // CPOL just inverts the idle level
endmodule

```

An SPI controller supporting all four modes. A half-period counter generates edges; the edge type (leading/trailing) and CPHA decide whether each edge samples MISO or shifts MOSI.

```

module tb;
    logic clk = 0, rst_n = 0; always #5 clk = ~clk;
    logic cpol, cpha, start = 0, done, sclk, mosi, cs_n, miso;
    logic [7:0] tx, rx;
    spi_master #(.HALF(2)) m (.*) ;
    // ---- behavioural slave for any mode: leading edge = posedge of (sclk ^ cpol) ----
    logic [7:0] s_sh, s_tx, s_rx; int nb;
    wire sck_l = sclk ^ cpol;
    always @(negedge cs_n) begin s_sh = s_tx; nb = 0; if (!cpha) miso = s_sh[7]; end
    always @(posedge sck_l) if (!cs_n) begin                // leading edge
        if (!cpha) begin s_rx = {s_rx[6:0], mosi}; nb++; end
        else            miso = s_sh[7];
    end
    always @(negedge sck_l) if (!cs_n) begin                // trailing edge
        if (cpha) begin s_rx = {s_rx[6:0], mosi}; nb++; s_sh = {s_sh[6:0], 1'b0}; end
        else            begin s_sh = {s_sh[6:0], 1'b0}; miso = s_sh[7]; end
    end
    // ---- simple waveform printer: one character per clk ----
    string w_cs, w_sck, w_mosi;
    always @(posedge clk) if (cpol == cpha) begin
        w_cs  = {w_cs, cs_n ? "~" : "_"};
        w_sck = {w_sck, sclk ? "~" : "_"};
        w_mosi = {w_mosi, cs_n ? "-" : (mosi ? "1" : "0")};
    end
    initial begin
        repeat (2) @(posedge clk); rst_n = 1;
        for (int mode = 0; mode < 4; mode++) begin
            {cpol, cpha} = mode[1:0];
            tx = 8'hA0 | mode; s_tx = 8'h5C ^ mode;
            repeat (2) @(posedge clk); w_cs = ""; w_sck = ""; w_mosi = "";
            @(posedge clk) start <= 1; @(posedge clk) start <= 0;
            @(posedge done); repeat (3) @(posedge clk);
            $display("mode %0d (CPOL=%0d CPHA=%0d): M->S %h got %h | S->M %h got %h %s",
                mode, cpol, cpha, tx, s_rx, s_tx, rx,
                (s_rx == tx && rx == s_tx && nb == 8) ? "PASS" : "FAIL");
            if (mode == 0 || mode == 3) begin
                $display("  CS_n %s", w_cs); $display("  SCLK %s", w_sck);
                $display("  MOSI %s", w_mosi);
            end
        end
    end
    $finish;
end
endmodule

```

Testbench with a behavioural peripheral that works in any mode, plus a one-character-per-clock waveform printer.

```

mode 0 (CPOL=0 CPHA=0): M->S a0 got a0 | S->M 5c got 5c PASS
CS_n  ~~_____~~
SCLK  ____~__~__~__~__~__~__~__~__~__
MOSI  --11110000111100000000000000000000--
mode 1 (CPOL=0 CPHA=1): M->S a1 got a1 | S->M 5d got 5d PASS
mode 2 (CPOL=1 CPHA=0): M->S a2 got a2 | S->M 5e got 5e PASS
mode 3 (CPOL=1 CPHA=1): M->S a3 got a3 | S->M 5f got 5f PASS
CS_n  ~~~_____~~
SCLK  ~~~~~~__~__~__~__~__~__~__~__~__
MOSI  ---00111100001111000000000000111111--

```

Real Icarus Verilog output: every mode exchanges a byte in both directions correctly. In mode 0 MOSI carries bit 7 of A0h before the first rising edge; in mode 3 the controller drives bit 7 of A3h at the first (falling) edge.

PITFALL: MODE MISMATCH THAT ALMOST WORKS

Using mode 0 against a mode-1 device shifts all data by one bit: reads come back doubled or halved and the top bit is lost - which looks like a data or register-map bug rather than a clocking one. When SPI data is 'off by a factor of two', check CPHA first.

12.3 Multiple peripherals: chip selects and daisy chains

independent chip selects ctrl SCLK ---+-----+-----+ MOSI ---+-----+-----+ MISO <--+-----+-----+ (tri-state) CS0_n -> dev A CS1_n -----> dev B CS2_n -----> dev C	daisy chain (shared CS_n) ctrl MOSI -> [dev A] -> [dev B] -> [dev C] -+ ctrl MISO <-----+-----+ all devices share SCLK and one CS_n; 3 devices x 8 bits = 24 clocks per update
---	--

Two ways to connect several peripherals.

- **One CS_n per device** is the norm: simple, any device can be addressed alone, but costs a pin per device and the MISO line is shared, so every unselected device must tri-state its output.
- **Daisy chain:** each device's output feeds the next device's input, forming one long shift register (common for LED drivers, shift registers, some ADC/DAC arrays and JTAG-like chains). One CS_n, but every update must shift the whole chain.
- A CS_n decoder or a GPIO can extend chip selects, but hardware CS timing (setup before the first edge, hold after the last, minimum high time between transactions) is then software's problem - a frequent source of flaky peripherals.

12.4 How fast can SPI go? The round-trip limit

Writes are easy: the controller launches both SCLK and MOSI, so they travel together and skew is small. **Reads are the problem.** In mode 0 the peripheral launches MISO on the falling SCLK edge and the controller samples it on the next rising edge - half a period later. But the falling edge seen by the peripheral is late (output pad and board delay), the peripheral adds its clock-to-output time (t_V / t_{CLQV} in flash datasheets, typically 5-8 ns), and MISO then travels back through the board and the input pad:

$$t_{rt} = t_{pad_out}(SCLK) + t_{board} + t_{CO}(peripheral) + t_{board} + t_{pad_in}(MISO)$$

$$requirement: t_{rt} + t_{setup} < T_{SCLK} / 2$$

e.g. $t_{rt} = 12\text{ ns}$, $t_{setup} = 2\text{ ns}$ -> $T_{SCLK} > 28\text{ ns}$ -> $f_{SCLK} < \sim 35\text{ MHz}$

The read round-trip budget for sampling on the opposite edge.

The standard fixes, in increasing sophistication:

- **Delayed (late) sampling:** sample MISO one or more system-clock cycles after the nominal edge, or on the **next falling edge** (a full period after launch). Controllers expose this as a 'sample delay' or 'RX capture edge' register.

- **Feedback / loopback clock:** route SCLK out through a pad and back in (or use a dummy pad), so the capture clock experiences the same pad delay as the data.
- **Programmable delay lines (DLL)** that place the capture point in the middle of the data eye; the delay is trained at boot by reading a known pattern.
- **A data strobe (DQS)** driven by the memory alongside the data, as in Octal xSPI and HyperBus: the strobe has the same round trip as the data, so the controller captures with it (source synchronous, like DDR DRAM).

The QSPI testbench below reproduces the problem and the first fix. SCLK runs at 100 MHz (10 ns period, from a 200 MHz controller clock), and the flash model's total round-trip delay is 3, 6 or 9 ns. With `rx_dly = 0` the controller captures at the rising edge 5 ns after launch; with `rx_dly = 1` it captures one controller clock later, at the falling edge 10 ns after launch.

12.5 SPI NOR flash: the command set

Serial NOR flash is the dominant SPI device, and its command set is a de-facto standard shared by most vendors (Winbond, Macronix, Micron, Infineon, GigaDevice, ISSI...), with the **SFDP** table (Serial Flash Discoverable Parameters, JEDEC JESD216) letting a controller discover sizes, erase types and fast-read opcodes at runtime. A transaction is `CS_n low, opcode, [address], [mode/dummy clocks], [data], CS_n high`.

Opcode	Command	Format	Notes
03h	READ	cmd + 3-byte addr + data	No dummy cycles; limited to a lower clock (often ~50 MHz)
0Bh	FAST READ	cmd + addr + 8 dummy clocks + data	Dummy clocks give the array access time; full speed
3Bh / BBh	Dual Output / Dual I/O read	1-1-2 / 1-2-2	Two data lines
6Bh / EBh	Quad Output / Quad I/O read	1-1-4 / 1-4-4	Four data lines; EBh sends the address on 4 lines too
06h / 04h	WREN / WRDI	cmd only	Write enable latch must be set before every program/erase
02h	PAGE PROGRAM	cmd + addr + 1-256 bytes	Within one 256-byte page; wraps at the page end
20h / 52h / D8h	Erase 4 KB / 32 KB / 64 KB	cmd + addr	Milliseconds to hundreds of ms
60h or C7h	Chip erase	cmd	Seconds to minutes
05h	READ STATUS REGISTER	cmd + data	Bit 0 = WIP/BUSY, bit 1 = WEL; poll until WIP = 0
9Fh	READ JEDEC ID	cmd + 3 bytes	Manufacturer, type, capacity
5Ah	READ SFDP	cmd + addr + 8 dummy + data	Parameter tables (JESD216)
66h + 99h	Reset enable + reset	cmd, cmd	Return to a known state (e.g. after a warm reset mid-command)
B7h / E9h	Enter / exit 4-byte address mode	cmd	For devices > 16 MB; dedicated 4-byte opcodes (13h, 0Ch, 12h, 21h, DCh...) avoid mode state

```

CS_n  ~~\
SCLK  ___/~/~/~\ ... _/~/~/~/~\ ... _/~/~/~/~/~\ ... _/~/~/~/~/~\~/~\ ...
IO0   --< 0 1 1 0 1 0 1 1 > A23 ... A0 >----- dummy -----< 4 > 0 > 4 > 0 >
IO1   -----< 5 > 1 > 5 > 1 >
IO2   -----< 6 > 2 > 6 > 2 >
IO3   -----< 7 > 3 > 7 > 3 >
      opcode 6Bh (8)      address (24)      8 clocks      byte 0      byte 1
IO3..0 carry bits 7..4 of each byte, then bits 3..0 (high nibble first)

```

Fast Read Quad Output (6Bh, 1-1-4): command and address on IO0, then two clocks per byte on four lines. IO2/IO3 are WP_n/HOLD_n in single-bit modes, which is why the Quad Enable (QE) status bit must be set first.

A controller for this sequence is a small FSM - command/address shifting on IO0, a dummy counter, then nibble capture - plus the tri-state control that hands IO0 from the controller to the flash after the address. The version below also has the programmable capture delay discussed above.

```
// Fast Read Quad Output (6Bh), 1-1-4: command and address on IO0, 8 dummy clocks,
// data on IO[3:0]. SCLK = clk/2 (mode 0). rx_dly moves the capture point by 1 clk.
module qspi_rd (
    input logic      clk, rst_n, start, rx_dly,
    input logic [23:0] addr, input logic [3:0] nbytes,
    output logic      sclk, cs_n, done,
    output logic [3:0] io_o, io_oe, input logic [3:0] io_i,
    output logic [63:0] rdata // up to 8 bytes, first byte in [63:56]
);
typedef enum logic [1:0] {IDLE, CMDADR, DUMMY, DATA} st_t;
st_t st; logic [31:0] sh; logic [5:0] n; logic [4:0] nib, ncap; logic cap_d;
wire rise = (st != IDLE) && !sclk; // this clk edge makes SCLK rise
wire fall = (st != IDLE) && sclk;
wire cap = rx_dly ? cap_d : (rise && st == DATA); // capture strobe
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin st <= IDLE; sclk <= 0; cs_n <= 1; done <= 0; io_oe <= 0; io_o <= 0;
        sh <= 0; n <= 0; nib <= 0; ncap <= 0; cap_d <= 0; rdata <= 0; end
    else begin
        done <= 0;
        cap_d <= rise && st == DATA;
        if (cap) begin rdata <= {rdata[59:0], io_i}; ncap <= ncap + 1; end
        case (st)
            IDLE: if (start) begin
                st <= CMDADR; cs_n <= 0; sh <= {8'h6B, addr}; n <= 0; ncap <= 0;
                io_oe <= 4'b0001; io_o <= 4'b0; // bit 7 of 6Bh (=0) valid at CS fall
            end else if (cs_n == 0 && ncap == 2*nbytes) begin
                cs_n <= 1; done <= 1; // last nibble captured
            end
            default: begin
                sclk <= ~sclk;
                if (fall) begin // launch on falling edge
                    n <= n + 1;
                    case (st)
                        CMDADR: if (n == 31) begin st <= DUMMY; n <= 0; io_oe <= 0; end
                                else begin sh <= {sh[30:0], 1'b0}; io_o[0] <= sh[30]; end
                        DUMMY: if (n == 7) begin st <= DATA; n <= 0; nib <= 0; end
                        DATA: if (nib == 2*nbytes - 1) st <= IDLE;
                                else nib <= nib + 1;
                        default: ;
                    endcase
                end
            end
        endcase
    end
endmodule
```

A Fast Read Quad Output (6Bh) controller: SCLK = clk/2, launch on the falling edge, capture on the rising edge or one controller clock later (*rx_dly*).

```
module tb;
    logic clk = 0, rst_n = 0; always #2.5 clk = ~clk; // 200 MHz -> SCLK 100 MHz
    logic start = 0, rx_dly, sclk, cs_n, done; logic [3:0] io_o, io_oe;
    logic [63:0] rdata; real t_rt; // board + flash round trip
    wire [3:0] io;
    qspi_rd dut (.clk, .rst_n, .start, .rx_dly, .addr(24'h000100), .nbytes(4'd8),
        .sclk, .cs_n, .done, .io_o, .io_oe, .io_i(io), .rdata);
endmodule
```

```

for (genvar i = 0; i < 4; i++) assign io[i] = io_oe[i] ? io_o[i] : 1'bz;
// ----- flash model: 32 clocks of cmd+addr, 8 dummy, then nibbles -----
logic [7:0] mem [0:511]; logic [3:0] f_out; logic f_oe; int clks; logic [31:0] ca;
logic [23:0] fa; int nibc;
assign io = f_oe ? f_out : 4'bz;
initial for (int i = 0; i < 512; i++) mem[i] = 8'(i * 7 + 8'h31);
always @(negedge cs_n) begin clks = 0; f_oe = 0; ca = 0; nibc = 0; end
always @(posedge cs_n) f_oe <= #(t_rt) 0;
always @(posedge sclk) if (!cs_n) begin
    if (clks < 32) ca = {ca[30:0], io[0]};
    clks++;
end
always @(negedge sclk) if (!cs_n && clks >= 40) begin // after 32 + 8 dummy clocks
    fa = ca[23:0] + nibc / 2;
    f_oe <= #(t_rt) 1;
    f_out <= #(t_rt) (nibc % 2) ? mem[fa][3:0] : mem[fa][7:4];
    nibc++;
end
logic [63:0] exp_d;
initial begin
    for (int i = 0; i < 8; i++) exp_d = {exp_d[55:0], 8'((256 + i) * 7 + 8'h31)};
    repeat (2) @(posedge clk); rst_n = 1;
    $display("round trip rx_dly opcode addr data read result");
    for (int k = 0; k < 6; k++) begin
        t_rt = 3.0 + 3.0 * (k % 3); // 3, 6, 9 ns
        rx_dly = (k >= 3);
        @(posedge clk) start <= 1; @(posedge clk) start <= 0;
        @(posedge done); repeat (4) @(posedge clk);
        $display(" %4.1f ns %0d %h %h %s", t_rt, rx_dly, ca[31:24], ca[23:0],
            rdata, rdata == exp_d ? "PASS" : "FAIL");
    end
    $display("SCLK cycles per 8-byte read: 8 cmd + 24 addr + 8 dummy + 16 data = %0d", clks);
    $finish;
end
endmodule

```

Testbench: a flash model that decodes the opcode and address and returns data after 8 dummy clocks with a configurable round-trip delay; the read is repeated for three delays with and without late sampling.

round trip	rx_dly	opcode	addr	data read	result
3.0 ns	0	6b	000100	31383f464d545b62	PASS
6.0 ns	0	6b	000100	z31383f464d545b6	FAIL
9.0 ns	0	6b	000100	z31383f464d545b6	FAIL
3.0 ns	1	6b	000100	31383f464d545b62	PASS
6.0 ns	1	6b	000100	31383f464d545b62	PASS
9.0 ns	1	6b	000100	31383f464d545b62	PASS

SCLK cycles per 8-byte read: 8 cmd + 24 addr + 8 dummy + 16 data = 56

Real Icarus Verilog output. Nominal-edge capture fails once the round trip exceeds half an SCLK period (5 ns): the first nibble is captured as z and the data slips by one nibble. Capturing one clock later passes for all three delays.

Note the throughput arithmetic in the last line: the 8-byte quad read costs 56 SCLK cycles, of which only 16 move data. The same read with 03h would cost $8 + 24 + 64 = 96$ cycles, with 1-4-4 (EBh) it would be $8 + 6 + (\text{mode} + \text{dummy, typically } 6) + 16 = 36$. For short, random reads - exactly what an instruction cache miss produces - **command and address overhead dominate**, which is what drove the industry to 4-4-4 and 8-8-8 modes and continuous-read tricks.

12.6 Dual, Quad and QPI modes

Multi-line modes are written **x-y-z**: the number of lines used for command, address and data.

Mode	Command	Address	Data	Example opcode	Remarks
1-1-1	1	1	1	03h, 0Bh	Classic SPI
1-1-2	1	1	2	3Bh	Dual output
1-2-2	1	2	2	BBh	Dual I/O
1-1-4	1	1	4	6Bh	Quad output
1-4-4	1	4	4	EBh	Quad I/O; with 'mode bits' (continuous read) the next read skips the opcode
4-4-4	4	4	4	EBh in QPI	QPI : entered with a vendor-specific command (for example 38h or 35h); everything on 4 lines
1S-8S-8S / 8S-8S-8S	8	8	8	vendor	Octal SDR
8D-8D-8D	8 (DDR)	8 (DDR)	8 (DDR)	vendor/xSPI	Octal DDR with DQS: 2 bytes per clock

PITFALL: MODE STATE THAT SURVIVES YOUR RESET

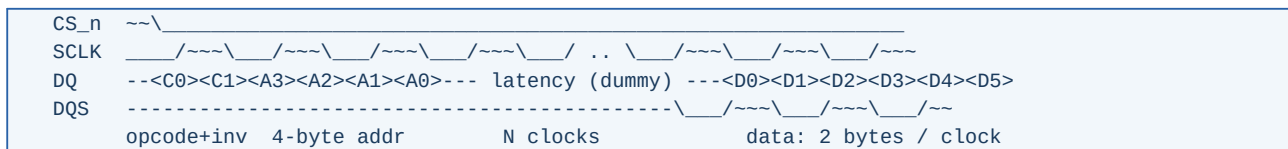
QPI mode, 4-byte address mode, continuous-read (XIP) mode and volatile dummy-cycle settings live **inside the flash** and survive an SoC reset that does not power-cycle the flash. The boot ROM then sends a 1-1-1 03h read to a device that expects 4-4-4, reads garbage and bricks the board after a watchdog or warm reset. Robust designs route the SoC reset to the flash RESET_n pin, or have the boot ROM issue the reset sequence (66h/99h, sent in every possible mode) before the first read.

12.7 Octal SPI, xSPI (JEDEC JESD251) and HyperBus

For execute-in-place application processors and automotive cluster displays, quad SPI ran out of bandwidth. Two related solutions appeared, both with eight data lines, double data rate and a strobe for read capture:

	Octal xSPI (JEDEC JESD251)	HyperBus (Infineon/Cypress)
Data	DQ[7:0], SDR or DDR (8D-8D-8D)	DQ[7:0], DDR
Clock	SCLK (optionally differential)	CK/CK_n (differential at 1.8 V)
Strobe	DQS - driven by the memory on reads	RWDS - read strobe on reads, write mask / latency indicator otherwise
Header	Opcode (often 2 bytes in DDR: code + inverse) + 4-byte address + latency	48-bit command-address (CA) word
Typical peak	200 MHz DDR = 400 MB/s	200 MHz DDR = 400 MB/s (HyperBus); 16-bit HyperBus variants exist
Devices	Octal NOR flash, some PSRAM	HyperFlash (NOR), HyperRAM (self-refresh DRAM)

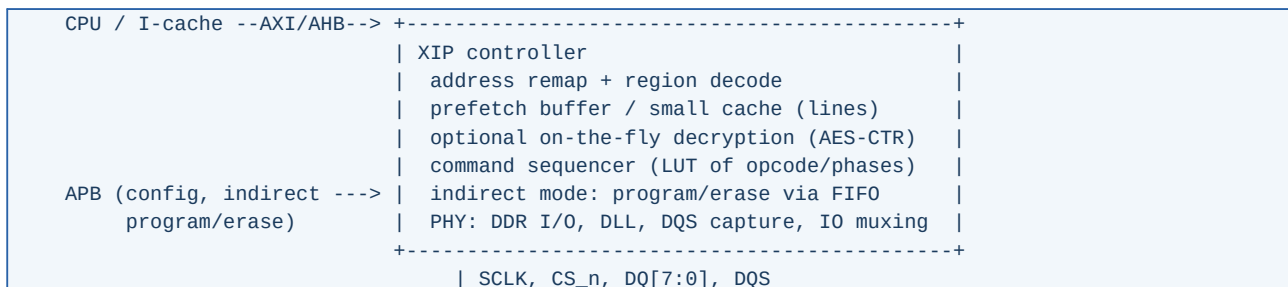
JESD251 standardized the Octal/xSPI electrical interface, command structure and profiles (profile 1.0 is the command-based SPI evolution; profile 2.0 is HyperBus-like), and JESD251-1 adds a 4-line variant. The controller side looks much like a small DDR memory controller: a DQS-based capture path with a delay line, DQS preamble handling, variable latency, and training at boot.



Octal DDR (8D-8D-8D) read: two bytes per SCLK cycle, and the memory drives DQS edge-aligned with the data; the controller delays DQS by about a quarter period (DLL) to capture in the middle of each byte.

12.8 Execute in place (XIP) controllers

An XIP controller maps the flash into the processor's address space: an AXI or AHB read to, say, 0x6000_0000 + offset becomes a flash read command with that offset. It is one of the most performance-critical peripherals in a microcontroller or cluster SoC.



Anatomy of a serial-flash XIP controller.

- **Sequence LUT:** firmware programs the opcode, address width, dummy count, lane count and DDR/SDR for each phase, so one controller supports every vendor's flash.
- **Continuous read / mode bits:** after the first 1-4-4 or 8D read, the flash stays in read mode and subsequent reads send only the address, saving the opcode.
- **Prefetch and wrap reads:** fetch a whole cache line, critical word first, using the flash's wrap-around burst mode.
- **Writes** are not memory-mapped on NOR (program/erase takes milliseconds and needs WREN); software uses an indirect mode. While an erase runs, XIP reads of the same device stall or fault - code that erases flash must run from RAM.
- **On-the-fly decryption** (AES in counter mode keyed per region) protects code at rest; the counter is derived from the address so random access works.

INTERVIEW INSIGHT: WHY XIP PERFORMANCE IS ABOUT LATENCY, NOT BANDWIDTH

A 200 MHz Octal DDR flash streams 400 MB/s, but a random 32-byte cache-line read still pays CS_n high time + opcode + address + latency (often 10-20 clocks) before the first byte - on the order of 100-150 ns, comparable to DRAM. XIP-heavy systems therefore live or die by the I-cache hit rate, prefetching and placing hot code in on-chip SRAM.

12.9 Verification and integration notes

- **VIP:** a flash model (vendors ship Verilog models for their parts) is the gold reference; also test against a generic SPI VIP in all four modes with random word lengths and CS_n timing.
- **Timing sign-off:** SPI I/O constraints use a generated clock on the SCLK output pin, with MOSI as output delay and MISO as input delay relative to it; for DDR/DQS paths the constraints resemble a DDR PHY's (Chapter 20). Many SPI read failures on silicon are simply missing or wrong I/O constraints.
- **Boot ROM:** the SPI/QSPI boot path is usually hard-coded in ROM at a conservative clock and 1-1-1 mode, then the second stage switches to quad/octal after reading SFDP. The ROM must survive every flash state (see the mode-state pitfall).
- **Pads:** IO2/IO3 double as WP_n/HOLD_n - pull-ups are required, and pad slew/drive strength directly affects signal integrity at 100+ MHz.

- **Ownership:** RTL owns the controller and sequencer; the PHY/IO team owns the DLL and pads; firmware owns the LUT, SFDP parsing and flash-state recovery; board designers own trace lengths.

12.10 Summary

- SPI is a clocked exchange of two shift registers framed by CS_n; there is no addressing, ACK, flow control or error detection at the bus level.
- CPOL sets the SCLK idle level and CPHA chooses leading- or trailing-edge sampling; modes 0 and 3 (sampling on rising edges) are the most common. The RTL in this chapter handles all four.
- Read speed is limited by the round trip SCLK-out -> peripheral tCO -> MISO-in; late sampling, feedback clocks, DLLs and data strobes extend it (the simulation failed at 6 ns round trip with nominal capture and passed with one clock of capture delay).
- SPI NOR flash commands: 03h/0Bh reads, 3Bh/BBh/6Bh/EBh multi-I/O reads, 06h WREN, 02h page program, 20h/52h/D8h erases, 05h status (WIP), 9Fh ID, 5Ah SFDP.
- x-y-z notation gives the lanes for command, address and data; QPI is 4-4-4, Octal DDR is 8D-8D-8D with DQS; xSPI (JESD251) and HyperBus reach about 400 MB/s.
- XIP controllers turn flash into memory: sequence LUTs, continuous read, prefetch, decryption, and careful handling of flash internal mode state across resets.

12.11 Exercises

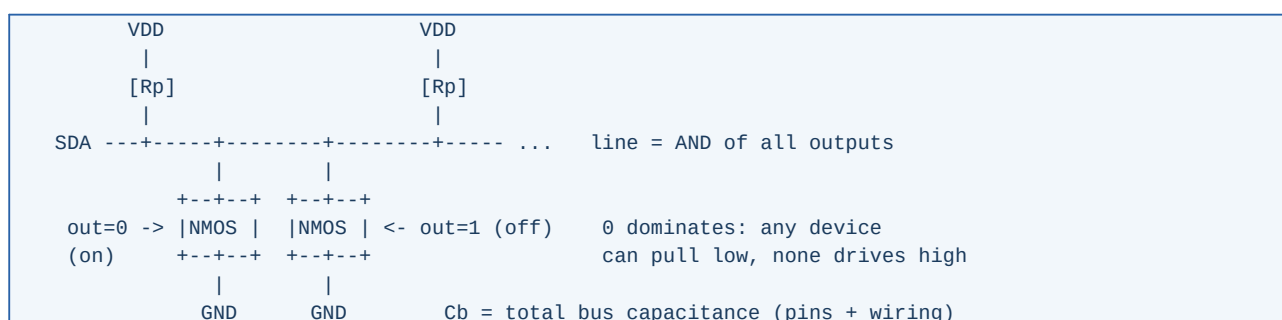
- 1 A peripheral samples on the falling edge and changes data on the rising edge, with SCLK idle low. Which mode is it? Draw the first two bits.
- 2 Pad delays are 3 ns out and 2.5 ns in, board flight time is 1 ns each way, the flash tCLQV is 6 ns and the controller needs 1.5 ns setup. What is the highest SCLK for mode-0 nominal sampling? With sampling moved a full SCLK period after launch?
- 3 Count SCLK cycles for a 32-byte cache-line fill using 03h, 0Bh, 6Bh, EBh (with 2 mode + 4 dummy clocks) and 8D-8D-8D (2-byte opcode, 4-byte address, 16 latency clocks). At 100 MHz SDR / DDR, what fraction of each is data?
- 4 Extend `qspi_rd` to the 1-4-4 EBh command: the address and mode bits go out on four lines. What changes in the IO tri-state control?
- 5 Write the recovery sequence a boot ROM should send to a flash that may be in QPI, 4-byte or continuous-read mode before issuing 9Fh.
- 6 Three 12-bit DACs are daisy-chained. How many clocks per update? How would you update just one DAC?

13. I2C, SMBus and PMBus

I2C (Inter-Integrated Circuit), created by Philips in 1982 and now specified in NXP's UM10204, connects dozens of low-speed devices with just two wires, and does it with **addressing, acknowledgement, multi-master arbitration and flow control** (clock stretching) - all by exploiting one electrical trick: **open-drain outputs with pull-up resistors**. SMBus (System Management Bus) is a stricter profile of I2C for PC and server management, and PMBus layers a power-supply command set on top of SMBus. On a typical SoC board the I2C buses reach the PMIC and voltage regulators, temperature sensors, EEPROMs, camera sensors (control path), touch and audio codecs, battery gauges, HDMI DDC, and DIMM SPD EEPROMs.

13.1 Open drain and the wired-AND bus

Neither SCL nor SDA is ever driven high. Each device can only pull a line **low** (open-drain NMOS) or let go; a **pull-up resistor** returns the line high. The line is therefore the logical AND of all devices' outputs: any device writing 0 wins.



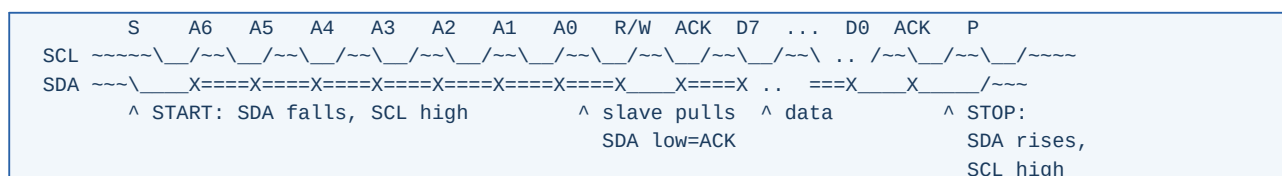
The wired-AND I2C bus. The rising edge is an RC charge through R_p into C_b , which is what limits speed.

Three features fall directly out of wired-AND: the **ACK** (a receiver pulls SDA low while the transmitter releases it), **clock stretching** (a slow slave holds SCL low and the master waits) and **arbitration** (two masters talking at once detect the collision without damaging anything). In RTL, an I2C pin is modelled as an output-enable that pulls low: `assign sda = sda_oe ? 1'b0 : 1'bz;` with a `pullup` on the net in simulation and an open-drain pad on silicon.

13.2 START, STOP and the bit rules

Normally SDA may change only while SCL is **low**, and is stable while SCL is high. Two exceptions are the bus's framing signals:

- **START (S)**: SDA falls while SCL is high. The bus is busy from here.
- **STOP (P)**: SDA rises while SCL is high. The bus is free after t_{BUF} .
- **Repeated START (Sr)**: a START without a preceding STOP; it keeps the bus while changing direction or target, which is how a 'write register pointer then read' is done atomically.



An I2C write: START, 7-bit address MSB first, R/W = 0, ACK from the slave, data bytes each followed by ACK, STOP. Every byte is 9 clocks.

13.3 Addressing, ACK/NACK and transactions

The first byte after START is the **7-bit address** plus the R/W bit (0 = write, 1 = read), so device 0x50 appears on the wire as 0xA0 (write) or 0xA1 (read) - a permanent source of confusion between 7-bit and 8-bit address conventions in datasheets and driver APIs. The ninth clock of every byte is the **acknowledge**: the receiver pulls SDA low (ACK) or leaves it high (NACK).

NACK means...	Situation
No device has this address	After the address byte: absent, unpowered, or busy device (EEPROMs NACK during their internal write cycle - 'ACK polling')
The receiver cannot accept more	Slave rejects a data byte (invalid register, buffer full)
The master wants no more data	On a read , the master NACKs the last byte so the slave releases SDA and the master can generate STOP or Sr

Address (7-bit)	R/W	Meaning
0000 000	0	General call (e.g. 0x06 = reset and re-read address pins)
0000 000	1	START byte (for slow, polling receivers)
0000 001	x	Reserved for CBUS
0000 010	x	Reserved for a different bus format
0000 011	x	Reserved for future use
0000 1xx	x	Hs-mode master code
1111 1xx	1/0	Device ID
1111 0xx	x	10-bit slave addressing (first byte)

Reserved 7-bit addresses (UM10204). Usable addresses are 0x08-0x77.

10-bit addressing sends **11110 A9 A8 R/W** followed by the lower eight address bits as a second byte (each ACKed). For a 10-bit read, the master writes both address bytes, then sends Sr and only the first byte again with R/W = 1. It is rarely used in practice.

```

write N bytes: S | ADR,W | A | reg | A | d0 | A | d1 | A | ... | P
random read:   S | ADR,W | A | reg | A | Sr | ADR,R | A | d0 | A | d1 | N | P
current read:  S | ADR,R | A | d0 | A | ... | dn | N | P
               (master drives S, Sr, address, reg, write data, final N, P;
               slave drives A after address/writes and the read data)

```

The three transaction shapes every I2C driver implements. A = ACK, N = NACK. Register-based devices auto-increment their pointer.

13.4 Clock stretching and arbitration

SCL is also wired-AND. After driving SCL low, a master releases it and must then **wait until SCL actually reads high** before timing the high period. A slave that needs time (to fetch data, finish an ADC conversion, or because its firmware is servicing an interrupt) simply holds SCL low - **clock stretching**. Masters with multiple devices also synchronize their clocks this way: the low period is set by the slowest device, the high period by the fastest.

Arbitration: two masters may start at the same time. Each compares what it sends with what it reads on SDA while SCL is high. A master that sends 1 (releases) but reads 0 has lost: it stops driving immediately and becomes a receiver; the winner's message is uncorrupted. The lowest address (more leading zeros) wins. The same principle is used by CAN (Chapter 15) and I3C dynamic address assignment (Chapter 14).

PITFALL: MASTERS THAT IGNORE SCL

A bit-banged or simplistic master that times SCL purely from its own counter, without reading SCL back, breaks with any stretching slave: it shortens the high period or misses the ACK. Many microcontroller peripherals (and some I2C slaves implemented in firmware) stretch, so a master must always synchronize on the actual SCL level. Conversely, an SoC's **slave** controller should stretch whenever software has not yet supplied read data rather than sending stale bytes.

13.5 An I2C master in RTL, against an open-drain slave

A practical master is split into a **bit controller** (generates START, STOP, and one data bit as four quarter-periods of SCL) and a **byte controller** (sequences 8 data bits + ACK). The version below combines them into one FSM driven by byte commands: START (also used for Sr), WRITE, READ (with ACK or NACK) and STOP. Pins are pure open-drain enables; SCL and SDA inputs are synchronized, and quarter 2 (SCL released) does not advance until SCL really is high - that is the clock-stretching support.

	q0	q1	q2	q3
data bit:	SCL low, set SDA	release SCL (rise)	SCL high: wait for SCL=1, then sample SDA	pull SCL low (fall)
START:	release SDA	release SCL	pull SDA	pull SCL
STOP:	SDA, SCL low	release SCL	release SDA	done

Quarter-period phases of the bit controller.

```
// Byte-command I2C master. Pads are open-drain: *_oe = 1 pulls the line low.
module i2c_master #(parameter int QTR = 25) ( // clk cycles per quarter of an SCL period
    input logic      clk, rst_n,
    input logic      go, input logic [1:0] cmd, // 0 START/rep.START 1 WRITE 2 READ 3 STOP
    input logic [7:0] wdata, input logic nack_send, // for READ: 1 = answer with NACK
    output logic     done, output logic [7:0] rdata, output logic nack_rcvd,
    output logic     scl_oe, sda_oe,
    input logic      scl_i, sda_i
);
typedef enum logic [1:0] {IDLE, START, BITS, STOP} st_t;
st_t st; logic [1:0] q, c; logic [3:0] bitn; logic [8:0] sh; logic rd;
logic [$clog2(QTR)-1:0] t; logic [1:0] scl_s, sda_s;
always_ff @(posedge clk) begin scl_s <= {scl_s[0], scl_i}; sda_s <= {sda_s[0], sda_i}; end
wire scl = scl_s[1], sda = sda_s[1];
// a quarter ends after QTR cycles; SCL is released at the end of quarter 1, and in
// quarter 2 time only runs once SCL is really high: clock stretching pauses us here
wire qend = (t == QTR-1);
wire hold = (q == 2) && !scl;
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin
        st <= IDLE; q <= 0; t <= 0; scl_oe <= 0; sda_oe <= 0; done <= 0;
        bitn <= 0; sh <= 0; rd <= 0; rdata <= 0; nack_rcvd <= 0; c <= 0;
    end else begin
        done <= 0;
        if (st == IDLE) begin
            if (go) begin
                c <= cmd; q <= 0; t <= 0; bitn <= 0; rd <= (cmd == 2);
                sh <= (cmd == 2) ? {8'hFF, nack_send} : {wdata, 1'b1}; // 1 = release SDA
                st <= (cmd == 0) ? START : (cmd == 3) ? STOP : BITS;
            end
        end else if (!hold) begin
            t <= qend ? '0 : t + 1;
            if (qend) begin
                q <= q + 1;
            end
        end
    end
end
```



```

    case (st)
    START: case (q) // SDA falls while SCL is high
        0: sda_oe <= 0; // release SDA (repeated START case)
        1: scl_oe <= 0; // release SCL, wait until high
        2: sda_oe <= 1; // START condition
        3: begin scl_oe <= 1; st <= IDLE; done <= 1; end
    endcase
    STOP: case (q) // SDA rises while SCL is high
        0: begin scl_oe <= 1; sda_oe <= 1; end
        1: scl_oe <= 0;
        2: sda_oe <= 0; // STOP condition
        3: begin st <= IDLE; done <= 1; end
    endcase
    BITS: case (q) // 9 bits: 8 data + ACK
        0: begin scl_oe <= 1; sda_oe <= ~sh[8]; end // change SDA, SCL low
        1: scl_oe <= 0; // rising edge
        2: sh <= {sh[7:0], sda}; // sample while high
        3: begin
            scl_oe <= 1; // falling edge
            bitn <= bitn + 1;
            if (bitn == 8) begin
                st <= IDLE; done <= 1; sda_oe <= 0;
                if (rd) rdata <= sh[8:1]; else nack_rcvd <= sh[0];
            end
        end
    endcase
    default: ;
endcase
end
end
end
endmodule

```

Byte-command I2C master (open-drain pins, input synchronizers, clock-stretching aware).

```

module tb;
    logic clk = 0, rst_n = 0; always #10 clk = ~clk; // 50 MHz, QTR=125 -> 100 kHz
    wire scl, sda; pullup(scl); pullup(sda); // wired-AND bus + pull-ups
    logic go = 0, done, nack_rcvd, m_scl_oe, m_sda_oe, nack_send = 0;
    logic [1:0] cmd; logic [7:0] wdata, rdata;
    i2c_master #(.QTR(125)) m (.clk, .rst_n, .go, .cmd, .wdata, .nack_send, .done, .rdata,
        .nack_rcvd, .scl_oe(m_scl_oe), .sda_oe(m_sda_oe), .scl_i(scl), .sda_i(sda));
    assign scl = m_scl_oe ? 1'b0 : 1'bz;
    assign sda = m_sda_oe ? 1'b0 : 1'bz;
    // ----- behavioural slave at address 0x50 with 256 byte registers -----
    localparam logic [6:0] MY = 7'h50;
    typedef enum {S_IDLE, S_ADDR, S_WR, S_RD} sst_t;
    sst_t ss; int bitc; bit in_ack, first, m_nack; logic [7:0] ssh, ptr, regs [256];
    logic s_sda_low = 0, s_scl_low = 0;
    assign sda = s_sda_low ? 1'b0 : 1'bz;
    assign scl = s_scl_low ? 1'b0 : 1'bz;
    always @(negedge sda) if (scl === 1'b1) begin ss = S_ADDR; bitc = 0; in_ack = 0; end
    always @(posedge sda) if (scl === 1'b1) begin ss = S_IDLE; s_sda_low = 0; end
    always @(posedge scl) if (ss != S_IDLE) begin
        if (!in_ack && bitc < 8) begin if (ss != S_RD) ssh = {ssh[6:0], sda}; bitc++; end
        else if (in_ack && ss == S_RD) m_nack = sda; // master's ACK/NACK
    end
    always @(negedge scl) if (ss != S_IDLE) begin
        if (!in_ack && bitc == 8) begin // 8 bits done -> ACK slot
            in_ack = 1;
            case (ss)

```

```

        S_ADDR: if (ssh[7:1] == MY) s_sda_low = 1; else ss = S_IDLE;    // not me: NACK
        S_WR:   begin s_sda_low = 1; if (first) ptr = ssh; else regs[ptr++] = ssh;
                first = 0; end
        S_RD:   s_sda_low = 0;                                         // master drives ACK
        default: ;
    endcase
end else if (in_ack) begin                                           // ACK slot finished
    in_ack = 0; bitc = 0; s_sda_low = 0;
    if (ss == S_ADDR) begin
        if (ssh[0]) begin                                           // read: stretch, then data
            s_scl_low = 1; ss = S_RD; ssh = regs[ptr++];
            #20000 s_sda_low = ~ssh[7];                             // data ready after 20 us
            #500 s_scl_low = 0;                                       // ... then release SCL
        end else begin ss = S_WR; first = 1; end
    end else if (ss == S_RD) begin
        if (m_nack) ss = S_IDLE; else begin ssh = regs[ptr++]; s_sda_low = ~ssh[7]; end
    end
    end else if (ss == S_RD && bitc < 8) s_sda_low = ~ssh[7 - bitc];
end
// ----- command helpers -----
task automatic op(input logic [1:0] c, input logic [7:0] d = 0, input logic nk = 0);
    @(posedge clk) begin go <= 1; cmd <= c; wdata <= d; nack_send <= nk; end
    @(posedge clk) go <= 0;
    @(posedge done);
    case (c)
        0: $display("%8.2f us  START", $realtime / 1000);
        1: $display("%8.2f us  WRITE %h -> %0s", $realtime / 1000, d,
            nack_rcvd ? "NACK" : "ACK");
        2: $display("%8.2f us  READ  %h <- master answers %0s", $realtime / 1000, rdata,
            nk ? "NACK" : "ACK");
        3: $display("%8.2f us  STOP", $realtime / 1000);
    endcase
endtask
initial begin
    repeat (3) @(posedge clk); rst_n = 1;
    $display("-- write 5A, C3 to registers 10h, 11h of device 50h");
    op(0); op(1, 8'hA0); op(1, 8'h10); op(1, 8'h5A); op(1, 8'hC3); op(3);
    $display("-- random read from 10h: write pointer, repeated START, read 2 bytes");
    op(0); op(1, 8'hA0); op(1, 8'h10); op(0); op(1, 8'hA1);
    op(2, 0, 0); op(2, 0, 1); op(3);
    $display("-- address 51h: nobody answers");
    trace = 1; op(0); op(1, 8'hA2); op(3); #5000 trace = 0;
    $display("  SCL %s", w_scl); $display("  SDA %s", w_sda);
    $finish;
end
// one character per 2.5 us (a quarter SCL period) while trace = 1
bit trace = 0; string w_scl = "", w_sda = "";
always #2500 if (trace) begin
    w_scl = {w_scl, scl ? "~" : "_"}; w_sda = {w_sda, sda ? "~" : "_"};
end
// measure the stretched low time of SCL
realtime tfall;
always @(negedge scl) tfall = $realtime;
always @(posedge scl) if ($realtime - tfall > 6000)
    $display("          (SCL held low %0.2f us by the slave: clock stretching)",
        ($realtime - tfall) / 1000);
endmodule

```

Testbench: the bus is two **wires** with **pullups**; the slave (address 50h, 256 registers with an auto-incrementing pointer) is behavioural, stretches SCL for 20 us before its first read byte, and the third transaction addresses a device that does not exist.

```
-- write 5A, C3 to registers 10h, 11h of device 50h
```

```

10.09 us  START
100.49 us WRITE a0 -> ACK
190.89 us WRITE 10 -> ACK
281.29 us WRITE 5a -> ACK
371.69 us WRITE c3 -> ACK
381.77 us  STOP
-- random read from 10h: write pointer, repeated START, read 2 bytes
391.81 us  START
482.21 us WRITE a0 -> ACK
572.61 us WRITE 10 -> ACK
582.69 us  START
673.09 us WRITE a1 -> ACK
      (SCL held low 20.50 us by the slave: clock stretching)
778.93 us READ 5a <- master answers ACK
869.33 us READ c3 <- master answers NACK
879.41 us  STOP
-- address 51h: nobody answers
889.45 us  START
979.85 us WRITE a2 -> NACK
989.93 us  STOP
SCL ~~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_
SDA ~~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~_~~~

```

Real Icarus Verilog output at 100 kHz (50 MHz clock, QTR = 125): register write, random read with repeated START and clock stretching, and a NACKed address. The waveform is the NACK transaction sampled every quarter period: START, A2h = 1010 0010, SDA left high in the ninth clock (NACK), STOP.

Each byte takes 90.4 us: 9 SCL periods of 10 us plus synchronizer latency - which is why 100 kHz I2C moves at most about 11 kB/s. The stretched byte took 20.5 us longer, and the master's quarter-2 wait absorbed it without any special case.

13.6 Speed modes, rise time and pull-up sizing

Mode	Max rate	Bus cap. max	tr max	Drivers	Notes
Standard (Sm)	100 kbit/s	400 pF	1000 ns	open drain	Original mode
Fast (Fm)	400 kbit/s	400 pF	300 ns	open drain	Most common today; 50 ns spike filter
Fast-mode Plus (Fm+)	1 Mbit/s	550 pF	120 ns	open drain, 20 mA sink	Stronger drivers allow lower Rp
High-speed (Hs)	3.4 Mbit/s	100 pF (400 pF at 1.7 Mbit/s)	tens of ns	current-source pull-up on master SCL	Entered with master code 00001xxx at Fm speed
Ultra-fast (UFm)	5 Mbit/s	-	-	push-pull	Unidirectional (write-only), no ACK; USCL/USDA; not compatible with other modes

Rise time is the RC charge from 30 % to 70 % of VDD through the pull-up: $t = R_p \times C_b \times \ln(0.7/0.3) = 0.8473 R_p C_b$. That gives a **maximum Rp** for a given capacitance, while the driver's sink current at $V_{OL}(\max) = 0.4 V$ gives a **minimum Rp**:

```
# I2C pull-up window (UM10204 method): Rp_max = tr / (0.8473 * Cb),
# Rp_min = (VDD - VOL_max) / IOL. 0.8473 = ln(0.7/0.3): time from 30 % to 70 % VDD.
VDD, VOL = 3.3, 0.4
for mode, tr, iol in (("Sm 100k", 1000e-9, 3e-3), ("Fm 400k", 300e-9, 3e-3),
                     ("Fm+ 1M ", 120e-9, 20e-3)):
    for cb in (50e-12, 200e-12, 400e-12):
        rmax = tr / (0.8473 * cb)
        rmin = (VDD - VOL) / iol
        ok = "ok" if rmin <= rmax else "no solution with a 3 mA sink"
        print("%s Cb=%3.0f pF Rp_min=%5.0f ohm Rp_max=%6.0f ohm %s"
              % (mode, cb * 1e12, rmin, rmax, ok))
```

Pull-up resistor window for a 3.3 V bus (Python 3).

```
Sm 100k Cb= 50 pF Rp_min= 967 ohm Rp_max= 23604 ohm ok
Sm 100k Cb=200 pF Rp_min= 967 ohm Rp_max= 5901 ohm ok
Sm 100k Cb=400 pF Rp_min= 967 ohm Rp_max= 2951 ohm ok
Fm 400k Cb= 50 pF Rp_min= 967 ohm Rp_max= 7081 ohm ok
Fm 400k Cb=200 pF Rp_min= 967 ohm Rp_max= 1770 ohm ok
Fm 400k Cb=400 pF Rp_min= 967 ohm Rp_max= 885 ohm no solution with a 3 mA sink
Fm+ 1M Cb= 50 pF Rp_min= 145 ohm Rp_max= 2833 ohm ok
Fm+ 1M Cb=200 pF Rp_min= 145 ohm Rp_max= 708 ohm ok
Fm+ 1M Cb=400 pF Rp_min= 145 ohm Rp_max= 354 ohm ok
```

At Fm with 400 pF there is no valid resistor for a 3 mA driver - you must reduce capacitance (split the bus with a buffer/mux), use Fm+ drivers or an active pull-up.

CHOOSING RP IN PRACTICE

Pick a value near the low end of the window for speed margin but not at it (sink current, VOL and power when the line is held low). Common results: 4.7 kohm for 100 kHz, 2.2 kohm for 400 kHz on short board buses, ~1 kohm or less for Fm+. Remember every device on the bus, the connector and each board trace adds capacitance - on the order of 10 pF per device pin.

13.7 Timing parameters

UM10204 specifies a set of named timing parameters. Their meaning matters more than memorizing values; the key ones for Sm/Fm/Fm+:

Parameter	Meaning	Sm	Fm	Fm+
fSCL	SCL clock frequency (max)	100 kHz	400 kHz	1 MHz
tLOW	SCL low period (min)	4.7 us	1.3 us	0.5 us
tHIGH	SCL high period (min)	4.0 us	0.6 us	0.26 us
tHD;STA	Hold time after (repeated) START before first SCL fall	4.0 us	0.6 us	0.26 us
tSU;STA	Setup time for a repeated START (SCL high to SDA fall)	4.7 us	0.6 us	0.26 us
tSU;DAT	Data setup before SCL rises	250 ns	100 ns	50 ns
tHD;DAT	Data hold after SCL falls (min 0; devices provide internal hold to bridge the SCL fall time)	0 ns	0 ns	0 ns
tSU;STO	Setup time for STOP (SCL high to SDA rise)	4.0 us	0.6 us	0.26 us
tBUF	Bus free time between STOP and next START	4.7 us	1.3 us	0.5 us
tSP	Spikes shorter than this must be suppressed	-	50 ns	50 ns

Notice that tLOW(min) + tHIGH(min) at Fm is 1.9 us, less than the 2.5 us period, but only barely: with realistic rise and fall times, 400 kHz is only achievable if the controller's low phase is longer than half the period. Controllers therefore program SCL low and high counts separately (for example 60:40), and the rise time adds

to the high count when the controller waits for SCL to go high, as ours does.

PITFALL: THD;DAT = 0 AND THE SCL FALL TIME

Because a device may change SDA immediately after SCL falls, a receiver seeing a slow SCL fall (or a synchronizer-delayed one) can sample SDA that has already changed - interpreted as a spurious START or STOP. The fix, mandatory for slaves, is an internal SDA hold delay (the spec requires devices to provide at least 300 ns internally to bridge SCL's undefined region) and filtering SCL/SDA before edge detection.

13.8 Glitch filters and bus recovery

- **Glitch (spike) filters:** Fm and Fm+ inputs must reject pulses shorter than 50 ns. In RTL this is a digital filter: a signal must be stable for N samples of a fast clock before it is accepted ($N \times T_{\text{clk}} \geq 50 \text{ ns}$). It is applied before START/STOP detection.
- **Bus recovery:** if a master is reset mid-transaction while a slave is driving a 0 (an ACK, or a data bit of a read), the slave holds SDA low forever waiting for clocks. The standard remedy is to send **up to nine SCL pulses** with SDA released until the slave finishes its byte and releases SDA, then generate STOP. Good controllers do this in hardware and report 'bus stuck'. If SCL itself is stuck low, only a reset of the offending device (or a power cycle) helps.
- **Software reset:** general call address 0x00 followed by 0x06 resets devices that support it.

```

master reset here --v
SCL ~~~\___/~~~\___/~~~\___ (stopped)          ___/~~~\___/~~~\___/~~~\___/~~~ P
SDA =====X_____ (slave holds SDA low until it gets clocks)~~~/_~~~
recovery: clock SCL up to 9 times with SDA
released, stop as soon as SDA reads 1, then STOP

```

Nine-clock bus recovery.

13.9 SMBus: I2C with rules

The System Management Bus (SMBus, originally Intel/Duracell, now maintained by the SBS Implementers Forum) uses I2C signalling but tightens it for reliable system management in PCs, servers and batteries:

Topic	I2C	SMBus
Clock range	0 to 100/400/1000 kHz	10 kHz minimum to 100 kHz (SMBus 3.x also defines 400 kHz and 1 MHz classes)
Timeout	None - a slave may stretch forever	tTIMEOUT 25-35 ms : a device seeing SCL low > 25 ms must reset its interface; limits on cumulative stretching per byte and per message
Logic levels	Relative to VDD	Fixed thresholds (so 1.8 V-3.3 V devices mix)
ACK	Optional semantics	A device must ACK its own address at all times (so presence can be detected)
Error check	None	PEC : optional CRC-8 ($x^8 + x^2 + x + 1$) over every byte of the message, including addresses
Protocols	Free-form	Defined: Quick Command, Send/Receive Byte, Write/Read Byte/Word, Process Call, Block Write/Read, Block Process Call
Extras	-	SMBALERT_n interrupt line with Alert Response Address 0x0C; Host Notify; ARP (Address Resolution Protocol) assigns addresses using a 128-bit UDID via the default address 0x61

The timeout is the key difference for RTL: an SMBus controller or target needs a counter that detects SCL low for longer than 25 ms and resets the interface, which also makes SMBus self-recovering from the stuck-bus scenario that plagues plain I2C. PEC generation is a serial CRC-8 identical in structure to the CAN CRC shown in Chapter 15, run over the address byte(s), command and data.

13.10 PMBus: power management on SMBus

PMBus (Power Management Bus) standardizes the **commands** for digital power supplies, regulators and PMICs on top of SMBus transport, so one driver can configure, monitor and log faults for any compliant device. Each command is an SMBus command code:

Code	Command	Purpose
00h	PAGE	Select the output rail on a multi-rail device
01h	OPERATION	On/off, margin high/low
03h	CLEAR_FAULTS	Clear latched status bits
20h / 21h	VOUT_MODE / VOUT_COMMAND	Output-voltage data format and set-point
78h / 79h	STATUS_BYTE / STATUS_WORD	Summary fault status
88h / 8Bh / 8Ch	READ_VIN / READ_VOUT / READ_IOUT	Telemetry
8Dh	READ_TEMPERATURE_1	Telemetry

Values use compact formats: **LINEAR11** (a 5-bit two's-complement exponent and an 11-bit mantissa, value = $m \times 2^e$), **LINEAR16** for output voltage (16-bit mantissa with the exponent in VOUT_MODE) and **DIRECT** (coefficients m, b, R). PMBus 1.3 also added AVSBus, a fast point-to-point interface for adaptive voltage scaling from the SoC to its regulator. In an SoC this is a firmware-heavy protocol: the RTL is a standard I2C/SMBus controller with PEC and timeouts, and a power-management microcontroller runs the PMBus stack.

13.11 I2C in an SoC: controllers, slaves and verification

- **Controller features:** separate SCL low/high counters, FIFOs for TX/RX, command queue (so DMA can run whole transactions), 10-bit addressing, repeated START, clock-stretch timeout, bus recovery, arbitration-loss interrupt, SMBus PEC/timeouts.
- **Target (slave) mode:** address match (often two addresses plus general call), stretching until software/DMA supplies data, wake-up on address match from low-power states.
- **Pads:** true open-drain (or a push-pull pad with the output tied to 0 and the enable toggled), 5 V-tolerant or fail-safe when the SoC is unpowered (a powered-off SoC must not clamp the bus through its ESD diodes).
- **DV:** an I2C VIP with stretching, arbitration and NACK injection; checkers for tHD;STA/tSU;STO/tBUF; tests of recovery from a stuck SDA; a real pull-up model (**pullup** or **tri1**) - simulations that drive 1 instead of Z hide wired-AND bugs.
- **Ownership:** RTL owns controller/target logic and filters; PD and IO own open-drain pads; board designers own Rp and capacitance; firmware owns addresses, retries and recovery.

13.12 Summary

- I2C uses two open-drain lines with pull-ups; wired-AND gives ACK, clock stretching and lossless arbitration.
- START/STOP are SDA edges while SCL is high; otherwise SDA changes only while SCL is low. Every byte is 8 bits + ACK/NACK.
- 7-bit addresses are sent as address<<1 | R/W; 10-bit addressing uses the 11110xx prefix; several address ranges are reserved.
- Speed modes: Sm 100k, Fm 400k, Fm+ 1M, Hs 3.4M (open drain with current-source pull-up), UfM 5M (push-pull, write-only). Rise time $0.8473 R_p C_b$ bounds the pull-up; bus capacitance limits speed.
- Our RTL master performed writes, a repeated-START read with clock stretching and detected a NACK against an open-drain slave.
- Recovery: up to nine SCL pulses then STOP; glitch filters reject < 50 ns spikes; SMBus adds 25-35 ms timeouts, PEC CRC-8, ARP and SMBALERT_n; PMBus defines power-management commands and data formats.

13.13 Exercises

- 1 Draw SCL and SDA for a random read of register 0x2C from device 0x68 returning 0x5A, marking which device drives SDA in every bit.
- 2 A bus has 6 devices at 10 pF each plus 60 pF of wiring. Compute the Rp window at 400 kHz and 3.3 V. Would 1.8 V change it?
- 3 Add arbitration-loss detection to the `i2c_master` in this chapter: in quarter 2 of a data bit, if SDA reads 0 while the master released it, abort and report `arb_lost`. Where else can arbitration be lost?
- 4 Implement a 50 ns glitch filter for a 100 MHz sampling clock and explain why it must precede START/STOP detection.
- 5 Compute the SMBus PEC for a Write Byte of 0x55 to command 0x10 at 7-bit address 0x16 (hint: the CRC covers 0x2C, 0x10, 0x55).
- 6 Why must an SMBus device ACK its own address even when busy, whereas an I2C EEPROM may NACK during a write cycle?

14. MIPI I3C

By the mid-2010s a flagship phone carried a dozen or more sensors (accelerometer, gyroscope, magnetometer, pressure, proximity, ambient light, fingerprint, touch, haptics...), each on I2C or SPI plus a separate interrupt GPIO. I2C was too slow and too power-hungry (every 0 bit burns current through a pull-up), SPI needed too many pins, and the GPIO count kept growing. The MIPI Alliance's answer, **I3C** (Improved Inter-Integrated Circuit, first released in 2016/2017), keeps the two-wire bus and I2C compatibility but adds push-pull signalling at up to 12.5 MHz, in-band interrupts, dynamic addressing and optional high-data-rate modes. JEDEC adopted the royalty-free **I3C Basic** subset for the DDR5 DIMM sideband bus (SPD hub, PMIC and temperature sensors), which has brought I3C into servers and PCs as well.

Problem with I2C	I3C answer
Open-drain rising edges limit speed to 0.4-1 Mbit/s and waste power	Push-pull data at 12.5 MHz SDR (open drain only where arbitration or ACK needs it)
Every sensor needs an interrupt pin	In-band interrupts (IBI) on SDA, with priority by address
Static addresses collide; address pins cost pins	Dynamic address assignment (ENTDAA) using a 48-bit provisioned ID
No standard management commands	Common Command Codes (CCCs) for enable/disable events, reset, max data length, status, timing
Devices cannot join a running bus cleanly	Hot-join
No path beyond ~1 Mbit/s on two wires	HDR modes (DDR, ternary, bulk transport)
Clock stretching makes timing unbounded	Targets may not stretch SCL; flow control is explicit (T-bit, IBI)

14.1 Bus roles and backward compatibility

An I3C bus has one **active controller** (optionally secondary controllers that can take over), I3C **targets**, and optionally **legacy I2C targets**. The controller always drives SCL - in I3C SCL is effectively push-pull - and SDA is shared.

- I2C targets on an I3C bus must be Fm/Fm+ devices with the **50 ns spike filter** and must not stretch the clock. The controller keeps SCL high pulses short (well under 50 ns) during I3C-only traffic, so the legacy devices' filters simply never see a clock - they cannot misinterpret I3C traffic as their address.
- I2C traffic to legacy devices is still sent at Fm/Fm+ timing, open drain, on the same wires.
- The controller provides SDA's pull-up (often switchable and a weak 'high-keeper'), so boards do not need per-bus resistors tuned for speed.
- An I3C target can also have a **static I2C address** and behave as an I2C device until it is given a dynamic address.

14.2 Open drain where needed, push-pull everywhere else

I3C switches SDA's driver type within a single message:

Phase	SDA driver	Why
START + address header 7Eh (or a target address) after START	Open drain	Targets may be arbitrating in parallel (IBI, hot-join, controller request)
ACK/NACK after the address	Open drain	Target pulls low to ACK; nobody is harmed if several ACK
Data bytes, CCC codes, T-bits	Push-pull	One driver at a time; fast edges independent of Rp
Address after a Repeated START	Push-pull	No arbitration is possible after Sr
Handoff (T-bit in reads)	Target drives, then releases	Allows the controller to abort a read (see below)

	S	7E / W (open drain)								ACK	CCC code (push-pull)								T	Sr	DA / R ...					
SCL	---	~	~	~	~	~	~	~	~	~	~	~	~	~	~	~	~	..	~	~	~	~	~	~	..	
SDA	~	~	X	==	X	==	X	==	X	==	X	==	X	==	X	==	X	..	==	X	==	X	~	~	X	=
		1	1	1	1	1	1	1	0		^ target ACK										^ odd parity					
		open drain + pull-up (slow edges)									push-pull, SCL up to 12.5 MHz															

An SDR message: broadcast address 7Eh in open drain, then push-pull data with a T-bit in place of ACK. Repeated START lets the controller address a target without a STOP.

14.3 SDR mode, the T-bit and the 9th bit

In **SDR** (single data rate) mode an I3C message looks like I2C: START, address + R/W, ACK, 8-bit bytes. SCL runs at up to **12.5 MHz**, giving a raw 12.5 Mbit/s and about 11 Mbit/s of payload after the ninth bits. The difference is the ninth bit of each data byte:

- **Write data:** the ninth bit is a **T-bit = odd parity** of the byte, driven by the controller. The target checks it; there is no per-byte ACK. (A parity error is a detectable target error.)
- **Read data:** the ninth bit is an **end-of-data** signal driven by the target: T = 1 means 'more data follows', T = 0 means 'this was the last byte'. When the target signals more data, it hands SDA back after driving the high level, and the controller may still abort by generating a Repeated START during that bit.
- Because targets cannot stretch SCL, a target that is not ready must NACK its address, or use its IBI to tell the controller when data is available.

WHY ODD PARITY?

With odd parity the T-bit is 1 whenever the byte has an even number of ones - in particular for 00h. A stuck-low SDA (all zeros) therefore always fails parity, and a byte plus T-bit always contains at least one 1, so the line keeps toggling. The run below shows T = 1 for 00h, A5h and FFh and T = 0 for 01h.

14.4 Dynamic address assignment (ENTDAA)

Every I3C target carries a **48-bit Provisioned ID (PID)**: a 15-bit MIPI manufacturer ID, a type bit (random or vendor-fixed part value), a part ID, an instance ID and extra bits. It is followed by the **BCR** (Bus Characteristics Register: role, IBI capability, whether IBIs carry payload, speed limits, offline capability) and the **DCR** (Device Characteristics Register: device type code). After reset, targets without a static address have no address at all; the controller runs ENTDAAs:

```

S 7E/W A  ENTDA0(07h) T
Sr 7E/R A   <- every unaddressed target ACKs (open drain)
        targets send PID[47:0], BCR[7:0], DCR[7:0] MSB first, OPEN DRAIN:
        a target that writes 1 but reads 0 has lost and drops out
        controller sends dynamic address DA[6:0] + parity bit          target A
Sr 7E/R A   <- remaining targets arbitrate again ...
Sr 7E/R N   <- nobody ACKs: all targets have addresses -> P

```

ENTDAA flow. The 64-bit arbitration uses the same wired-AND principle as I2C multi-master arbitration and CAN; the lowest {PID, BCR, DCR} value wins each round.

The RTL below models the arbitration core: three targets shift out their 64-bit IDs onto a wired-AND SDA, each drops out as soon as it sends a 1 but sees a 0, and the survivor accepts the next dynamic address. The controller keeps running rounds until the bus reads all ones (nobody arbitrating, i.e. the NACK case). The address is sent with an odd-parity bit computed by the same function as the SDR T-bit. (The surrounding framing - 7Eh header, ACKs, Sr - is omitted to keep the example short.)

```
// ENTDAAs arbitration: each unaddressed target shifts out {PID[47:0], BCR, DCR} MSB first
// in open-drain mode. A target that sends 1 but sees 0 has lost and waits for next round.
module daa_target #(parameter logic [63:0] ID = '0) (
    input logic      clk, rst_n,
    input logic      rnd_start,          // controller starts an arbitration round
    input logic      bit_en, input logic [5:0] bitn, input logic sda,
```

```

input logic      assign_en, input logic [6:0] new_addr,
output logic     sda_low,                // open-drain drive
output logic     has_addr, output logic [6:0] dyn_addr
);
logic lost;
wire my_bit = ID[63 - bitn];
assign sda_low = bit_en && !lost && !has_addr && !my_bit;
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin lost <= 0; has_addr <= 0; dyn_addr <= 0; end
    else if (rnd_start) lost <= has_addr;    // addressed targets stay silent
    else if (bit_en && !lost && my_bit && !sda) lost <= 1;    // sent 1, saw 0
    else if (assign_en && !lost && !has_addr) begin has_addr <= 1; dyn_addr <= new_addr; end
endmodule

// T-bit for SDR write data and the parity bit after a DAA address: both are ODD parity
function automatic logic odd_par(input logic [7:0] d);
    return ~^d;    // makes the count of ones (data + bit) odd
endfunction

```

Target-side DAA arbitration logic and the I3C odd-parity function.

```

module tb;
    logic clk = 0, rst_n = 0; always #5 clk = ~clk;
    // PID = MIPI mfr ID(15) | type(1) | part ID(16) | instance(4) | extra(12); then BCR, DCR
    localparam logic [63:0] ID_A = {15'h011A, 1'b0, 16'h4A10, 4'h2, 12'h000, 8'h06, 8'h44};
    localparam logic [63:0] ID_B = {15'h011A, 1'b0, 16'h4A10, 4'h1, 12'h000, 8'h06, 8'h44};
    localparam logic [63:0] ID_C = {15'h0105, 1'b0, 16'h7702, 4'h0, 12'h001, 8'h27, 8'h63};
    logic rnd_start = 0, bit_en = 0, assign_en = 0; logic [5:0] bitn = 0; logic [6:0] na;
    wire [2:0] low; wire [2:0] has; wire [6:0] da [3];
    wire sda = ~|low;    // wired-AND with the pull-up
    daa_target #(ID_A) ta (.clk, .rst_n, .rnd_start, .bit_en, .bitn, .sda, .assign_en,
        .new_addr(na), .sda_low(low[0]), .has_addr(has[0]), .dyn_addr(da[0]));
    daa_target #(ID_B) tb_ (.clk, .rst_n, .rnd_start, .bit_en, .bitn, .sda, .assign_en,
        .new_addr(na), .sda_low(low[1]), .has_addr(has[1]), .dyn_addr(da[1]));
    daa_target #(ID_C) tc (.clk, .rst_n, .rnd_start, .bit_en, .bitn, .sda, .assign_en,
        .new_addr(na), .sda_low(low[2]), .has_addr(has[2]), .dyn_addr(da[2]));
    logic [63:0] seen; bit fin = 0;
    initial begin
        repeat (2) @(posedge clk); rst_n = 1;
        na = 7'h08;
        for (int r = 0; r < 4 && !fin; r++) begin
            @(posedge clk) rnd_start <= 1; @(posedge clk) rnd_start <= 0;
            seen = '1;
            for (int b = 0; b < 64; b++) begin    // 64 open-drain bits
                @(posedge clk) begin bit_en <= 1; bitn <= b[5:0]; end
                @(negedge clk) seen[63 - b] = sda; // controller reads the bus
            end
            @(posedge clk) bit_en <= 0;
            if (seen == '1) begin    // nobody pulled low: no target left
                $display("round %0d: all ones -> no target left, controller ends ENTDA", r);
                fin = 1;
            end else begin
                @(posedge clk) assign_en <= 1; @(posedge clk) assign_en <= 0;
                $display("round %0d: PID=%h BCR=%h DCR=%h wins -> dyn addr %h, sent as %b_%b",
                    r, seen[63:16], seen[15:8], seen[7:0], na, na, odd_par({1'b0, na}));
                @(negedge clk) na = na + 1;
            end
        end
        end
        $display("final: A=%h B=%h C=%h", da[0], da[1], da[2]);
        $display("SDR T-bit (odd parity): 00->%b 01->%b A5->%b FF->%b",
            odd_par(8'h00), odd_par(8'h01), odd_par(8'hA5), odd_par(8'hFF));
    end
endmodule

```

```
$finish;
end
endmodule
```

Three targets: two from the same manufacturer differing only in instance ID, and one from a manufacturer with a lower ID.

```
round 0: PID=020a77020001 BCR=27 DCR=63 wins -> dyn addr 08, sent as 0001000_0
round 1: PID=02344a101000 BCR=06 DCR=44 wins -> dyn addr 09, sent as 0001001_1
round 2: PID=02344a102000 BCR=06 DCR=44 wins -> dyn addr 0a, sent as 0001010_1
round 3: all ones -> no target left, controller ends ENTDA
final: A=0a B=09 C=08
SDR T-bit (odd parity): 00->1 01->0 A5->1 FF->1
```

Real Icarus Verilog output. Target C wins first (lower manufacturer ID), then B (instance 1) before A (instance 2); the fourth round reads all ones and ends DAA.

Other ways to assign addresses: **SETDASA** (direct CCC: give a dynamic address to a target that has a known static I2C address), **SETAASA** (broadcast: all targets use their static address as dynamic address - quick, used by the DDR5 sideband bus), and **SETNEWDA** to change an address later. **RSTDAA** removes all dynamic addresses.

INTERVIEW INSIGHT: PRIORITY IS BAKED INTO THE ADDRESS

In IBI arbitration the lower dynamic address wins, exactly like in DAA. A controller therefore assigns the **lowest** dynamic addresses to the targets whose interrupts are most urgent - address assignment is also interrupt-priority assignment.

14.5 Common Command Codes (CCCs)

CCCs are the bus-management language. A **broadcast CCC** (codes 00h-7Fh) is sent after **S 7Eh/w** and applies to all targets; a **direct CCC** (80h-FEh) is followed by **Sr** and one or more target addresses, each with its own data. Some examples:

Code (bcast / direct)	Name	Purpose
00h / 80h	ENEC	Enable target events (IBI, controller request, hot-join)
01h / 81h	DISEC	Disable target events
06h	RSTDAA	Reset all dynamic addresses
07h	ENTDAA	Enter dynamic address assignment
08h	DEFTGTS (DEFSLVS)	Tell secondary controllers the list of targets
09h / 89h, 0Ah / 8Ah	SETMWL, SETMRL	Set max write / read length
20h-27h	ENTHDR0-7	Enter an HDR mode (0 = HDR-DDR)
29h	SETAASA	Use static addresses as dynamic addresses
87h, 88h	SETDASA, SETNEWDA	Assign / change a dynamic address
8Dh, 8Eh, 8Fh	GETPID, GETBCR, GETDCR	Read identification
90h	GETSTATUS	Read target status (pending interrupt, errors)
94h, 95h	GETMXDS, GETCAPS	Max data speed and turnaround; capabilities (HDR modes supported)

Selected CCCs (names from I3C v1.1; older documents use master/slave names such as DEFSLVS and GETHRCAP).

14.6 In-band interrupts and hot-join

With the bus idle (after a bus-available time), a target that needs attention **pulls SDA low** - a START request. The controller responds by clocking SCL, and the target sends its own **dynamic address with R/W = 1** in the open-drain arbitration header. If several targets request at once, the lowest address wins; the others retry later. The controller ACKs to accept (or NACKs to refuse and later disable with DISEC); if the target's BCR says its IBIs carry payload, it then sends a **Mandatory Data Byte** (and optionally more) in push-pull mode.

```

bus idle ~~~~~ SDA pulled low by target (START request)
SCL ~~~~~ \_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_ ...
SDA ~~~~~ \_____X DA6 .. DA0 X R=1 X ACK X MDB ... T X P
              target arbitration (open drain) ctrl push-pull

```

In-band interrupt: no extra wire, and the interrupt identifies its source and can carry a payload byte (e.g. 'FIFO watermark').

Hot-join: a target that powers up on a running bus waits for bus idle, then requests with the reserved address **02h** and R/W = 0. The controller ACKs and later runs ENTDAAs to give it an address. Similarly, a secondary controller asks for the bus with its own address and R/W = 0 (controller role request).

14.7 HDR modes

After the controller enters an HDR mode with ENTHDRx, the bus uses a different encoding until the controller sends the **HDR Exit Pattern** (SDA toggling while SCL is held low), which every I3C device - including ones that do not implement that HDR mode - detects. Legacy I2C devices see no valid clock pulses throughout.

Mode	Idea	Raw rate at 12.5 MHz (typical figures quoted)
SDR	One bit per SCL cycle	12.5 Mbit/s (about 11 Mbit/s effective)
HDR-DDR	Data on both SCL edges; 16-bit words with a 2-bit preamble and parity; command/data/CRC words	25 Mbit/s raw (about 20 Mbit/s effective)
HDR-TSP / HDR-TSL	Ternary symbols: both SCL and SDA carry information (transitions on either line); TSP for pure I3C buses, TSL when legacy I2C devices are present	about 33 Mbit/s raw
HDR-BT (v1.1)	Bulk transport with larger blocks and optional multi-lane data (additional data lines)	Highest; scales with lanes

HDR-DDR is by far the most implemented HDR mode. Its framing (preamble bits that distinguish command, data and CRC words, a 5-bit CRC at the end, and parity per word) is a good example of how a protocol adds robustness when it removes per-byte ACKs.

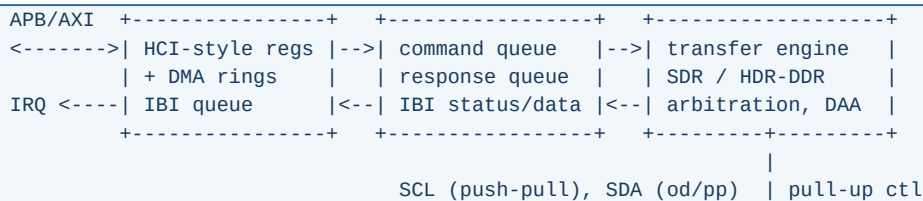
14.8 Timing and electrical overview

- SDR SCL up to 12.5 MHz (80 ns period); in mixed buses with legacy I2C devices, the SCL high time is kept short enough (well below the 50 ns spike filter) that I2C devices ignore it.
- Open-drain phases are slower (on the order of a few MHz) because they rely on the pull-up; the controller may use a faster open-drain timing when no legacy devices are present.
- Targets sample on SCL edges and the controller must allow for the target's clock-to-data turnaround on reads (GETMXDS reports it) - the same round-trip problem as SPI, solved by slowing reads or delaying capture.
- Voltage levels are typically 1.2 V or 1.8 V on phones and around 1.0-1.2 V on the DDR5 sideband bus (varies by implementation).
- Energy per bit is far lower than I2C: push-pull edges do not dump current through a pull-up for every 0.

14.9 I3C Basic versus full I3C

I3C Basic is a subset of the full specification that MIPI made available publicly and royalty-free (I3C Basic v1.0 in 2018, updated with later versions). It keeps what most systems need - SDR mode, dynamic addressing, IBIs, hot-join, the common CCCs and (depending on version) HDR-DDR - and omits some advanced features such as the ternary HDR modes. JEDEC's DDR5 sideband bus (JESD403) and SPD5 hub devices are built on I3C Basic, and MIPI's **I3C HCI** (Host Controller Interface) specification standardizes the controller's register/queue interface so one Linux/firmware driver can drive controllers from different vendors.

14.10 Building an I3C controller



Typical controller structure. Targets are simpler: address match, CCC decoder, IBI request logic, and a small register/FIFO interface.

- The **SDA pad** must switch between open-drain and push-pull per bit, with a controllable pull-up; this is the main PHY-level difference from an I2C pad.
- An internal **clock** of 4x-8x the SCL rate (for example 100 MHz for 12.5 MHz SCL) lets the engine place edges precisely and sample SDA with programmable delay.
- **DV**: an I3C VIP is essential (DAA with colliding IDs, IBI races with controller-initiated messages, parity errors, HDR entry/exit, legacy devices on the bus); compliance testing uses the MIPI conformance test suites.
- **Firmware**: bus enumeration (DAA, GETPID/BCR/DCR), IBI handling, per-target max read/write lengths - most integration bugs are in this sequencing rather than in bit timing.

14.11 Summary

- I3C keeps two wires and I2C compatibility but uses push-pull data at up to 12.5 MHz SDR, falling back to open drain only for arbitration and ACK phases; targets cannot stretch SCL.
- Write data carries an odd-parity T-bit instead of ACK; read data uses the ninth bit as an end-of-data indicator.
- ENTDAAs assigns 7-bit dynamic addresses by open-drain arbitration on the 48-bit PID + BCR + DCR (lowest wins); the simulated three targets were addressed in exactly that order.
- CCCs (broadcast 00h-7Fh, direct 80h-FEh) manage the bus; IBIs replace interrupt wires with address-prioritized in-band requests; hot-join uses address 02h.
- HDR modes (DDR, TSP/TSL, BT) raise throughput; all devices recognize the HDR exit pattern.
- I3C Basic is the public, royalty-free subset used by JEDEC DDR5 sideband devices; I3C HCI standardizes the controller software interface.

14.12 Exercises

- 1 Compute the T-bits for the bytes 3Ch, 7Fh and 80h. Which single stuck-at fault of SDA does odd parity always detect?
- 2 Four targets have PIDs whose first differing bit is bit 20. Explain why the order of DAA does not depend on the order in which they were powered up, and what happens if two targets have identical PID, BCR and DCR.
- 3 An accelerometer must interrupt with priority over a temperature sensor. Which dynamic addresses would you give them and why?
- 4 Estimate the time to read a 32-byte FIFO in SDR at 12.5 MHz (including S, header, address, T-bits, P) and compare with I2C Fm+ at 1 MHz.
- 5 Why can an I3C controller use push-pull after a Repeated START but not after a START?

- 6 Extend the DAA testbench so that a fourth target joins the bus after the first round (hot-join) and is addressed in the next ENTDAAs.

15. Automotive and Industrial Buses: CAN, CAN FD, CAN XL, LIN, FlexRay and Automotive Ethernet

A modern car contains somewhere between tens and over a hundred electronic control units (ECUs) - engine, brakes, airbags, doors, seats, infotainment, cameras, radar - connected by a hierarchy of networks. The requirements are unlike those inside an SoC: long cables in an electrically hostile environment, dozens of nodes on one bus, hard real-time deadlines, and functional safety (ISO 26262). Factories have similar needs. This chapter covers the bit-level design of CAN in depth, because its ideas - dominant/recessive signalling, bitwise arbitration, bit stuffing, CRC and fault confinement - are elegant and are asked about constantly, and then surveys CAN FD, CAN XL, LIN, FlexRay, automotive Ethernet and industrial fieldbuses.

Network	Rate	Typical use	Medium
LIN	up to 20 kbit/s	Seats, mirrors, window lifts, switches, sensors (cheap sub-networks)	1 wire, 12 V
CAN (classic)	up to 1 Mbit/s (125-500k typical)	Body, powertrain, chassis; diagnostics (OBD-II)	Twisted pair
CAN FD	~2-5 Mbit/s data phase (up to 8)	Powertrain, ADAS, software updates	Twisted pair
CAN XL	~10-20 Mbit/s data phase	Zonal architectures, bridge to Ethernet	Twisted pair
FlexRay	10 Mbit/s per channel	Chassis/x-by-wire (legacy, declining)	1-2 twisted pairs
Automotive Ethernet	10 Mbit/s - 10 Gbit/s	Backbone, cameras, ADAS, infotainment, zonal	Single twisted pair

15.1 CAN physical layer: dominant and recessive

Classic CAN (Bosch, 1986; ISO 11898-1 for the data link, 11898-2 for the high-speed physical layer) uses a differential pair, CAN_H and CAN_L, terminated with 120 ohm at each end. A transceiver translates between the pair and the controller's logic-level TXD/RXD pins.

```

bit value      recessive (1)      dominant (0)      recessive (1)
CAN_H  3.5 V -      +-----+
        2.5 V -----+              +-----+
CAN_L  2.5 V -----+              +-----+
        1.5 V -      +-----+
Vdiff = CAN_H - CAN_L:  ~0 V      ~2 V      ~0 V
receiver: Vdiff > 0.9 V -> dominant ; Vdiff < 0.5 V -> recessive

node: MCU CAN controller --TXD/RXD--> transceiver --CAN_H/CAN_L--> bus
bus:  [120 ohm]====+=====+=====+====[120 ohm]  (60 ohm total)

```

High-speed CAN levels (nominal values).

Dominant (logic 0) is actively driven; recessive (logic 1) is the undriven state where the termination pulls the lines together. Electrically this is a **wired-AND** exactly like I2C: if any node transmits 0, the bus is 0. Every node always reads back the bus (RXD) while transmitting, which is the basis of arbitration and error detection. The maximum bus length shrinks with bit rate because each bit must propagate to the far end and back within the bit time (rule of thumb: ~40 m at 1 Mbit/s, ~100 m at 500 kbit/s, ~500 m at 125 kbit/s).

15.2 Frame formats

```

classic base (standard) data frame, 44 + 8N bits before stuffing
+-----+-----+-----+-----+-----+-----+-----+-----+
|SOF| ID[10:0] |RTR|IDE|r0| DLC | DATA | CRC[14:0] |CD|ACK|AD| EOF |IFS|
| 1 | 11      | 1 | 1 | 1 | 4 | 0..64 | 15      |1 | 1 |1 | 7 | 3 |
+-----+-----+-----+-----+-----+-----+-----+
|<----- arbitration ----->|<- control ->|          CD = CRC delimiter (1)
|<----- bit stuffing applies ----->| AD = ACK delimiter (1)

extended data frame (29-bit ID):
|SOF| ID[28:18] |SRR|IDE| ID[17:0] |RTR|r1|r0| DLC | DATA | CRC | ...
| 1 | 11      | 1 | 1 | 1 | 18 | 1 | 1 | 1 | 4 |
          ^ recessive ^ recessive = extended

```

Classic CAN frames. ACK: the transmitter sends a recessive ACK slot; every receiver that got a correct CRC overwrites it with dominant.

- **SOF**: one dominant bit; all nodes hard-synchronize to its edge.
- **Identifier**: 11 bits (CAN 2.0A base format) or 29 bits (2.0B extended). It identifies the **message content and priority**, not a node - any node may receive any message (acceptance filtering is local).
- **RTR**: dominant for data frames, recessive for **remote frames** (a request for a message with that ID; no data field). **IDE**: dominant for base format. **SRR** in extended frames is a recessive placeholder at the RTR position.
- **DLC**: 0-8 data bytes (codes 9-15 also mean 8 in classic CAN).
- **CRC**: 15-bit sequence + recessive delimiter. **ACK**: slot + delimiter. **EOF**: 7 recessive bits; then 3 bits of intermission before the next frame may start.
- **Error frames** (error flag of 6 bits + 8-bit delimiter) and **overload frames** are not data - they are signals, discussed below.

15.3 Bitwise arbitration

CAN is multi-master: any node can start transmitting after the bus has been idle. If several start in the same bit time (all synchronized to the same SOF), they arbitrate **during the identifier, bit by bit**. A node sending recessive (1) that reads dominant (0) has lost: it stops transmitting at once and becomes a receiver of the winning frame. The winner never notices the contention - **non-destructive arbitration** - so no bandwidth is wasted and the frame with the **lowest numerical ID** (highest priority) always gets through with bounded latency.

```

module can_arb_node (
    // transmits SOF + 11-bit ID + RTR, watches the bus
    input logic      clk, rst_n, go, input logic [11:0] id_rtr, // {ID, RTR}
    input logic      bus, // wired-AND of all transmitters (0 = dominant)
    output logic      tx, lost
);
    logic [12:0] sh; logic active;
    assign tx = (active && !lost) ? sh[12] : 1'b1; // recessive once it has lost
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin sh <= '1; active <= 0; lost <= 0; end
        else if (go) begin sh <= {1'b0, id_rtr}; active <= 1; lost <= 0; end
        else if (active) begin
            if (!lost && tx && !bus) lost <= 1; // sent recessive, read dominant: back off
            sh <= {sh[11:0], 1'b1};
        end
endmodule

```

A minimal arbitration node: it shifts out SOF, 11-bit ID and RTR, reads back the wired-AND bus, and drops to recessive after losing.

```

module tb;
    logic clk = 0, rst_n = 0, go = 0; always #5 clk = ~clk;
    logic [11:0] a_id, b_id; logic ta, tb_, la, lb;
    wire bus = ta & tb_; // open-collector transceivers: wired-AND
    can_arb_node na (.clk, .rst_n, .go, .id_rtr(a_id), .bus, .tx(ta), .lost(la));

```



```

can_arb_node nb (.clk, .rst_n, .go, .id_rtr(b_id), .bus, .tx(tb_), .lost(lb));
string sa, sb, sbus;
task automatic run(input logic [11:0] a, input logic [11:0] b);
  a_id = a; b_id = b; sa = ""; sb = ""; sbus = "";
  @(negedge clk) go = 1; @(negedge clk) go = 0;
  repeat (13) begin
    sa = {sa, ta ? "1 " : "0 "}; sb = {sb, tb_ ? "1 " : "0 "};
    sbus = {sbus, bus ? "1 " : "0 "};
    @(negedge clk);
  end
  $display("      S 1 9 8 7 6 5 4 3 2 1 0 R");
  $display("      0 0                                T");
  $display("      F                                R");
  $display("node A    %s ID=%h RTR=%0d %0s", sa, a[11:1], a[0], la ? "LOST" : "WINS");
  $display("node B    %s ID=%h RTR=%0d %0s", sb, b[11:1], b[0], lb ? "LOST" : "WINS");
  $display("bus      %s", sbus);
endtask
initial begin
  repeat (2) @(posedge clk); rst_n = 1;
  run({11'h123, 1'b0}, {11'h12B, 1'b0});           // different IDs
  run({11'h123, 1'b0}, {11'h123, 1'b1});           // same ID: data frame vs remote frame
  $finish;
end
endmodule

```

Two nodes starting simultaneously: first with different IDs, then with the same ID as data frame vs. remote frame.

```

      S 1 9 8 7 6 5 4 3 2 1 0 R
      0 0                                T
      F                                R
node A   0 0 0 1 0 0 1 0 0 0 1 1 0   ID=123 RTR=0 WINS
node B   0 0 0 1 0 0 1 0 1 1 1 1 1   ID=12b RTR=0 LOST
bus      0 0 0 1 0 0 1 0 0 0 1 1 0
      S 1 9 8 7 6 5 4 3 2 1 0 R
      0 0                                T
      F                                R
node A   0 0 0 1 0 0 1 0 0 0 1 1 0   ID=123 RTR=0 WINS
node B   0 0 0 1 0 0 1 0 0 0 1 1 1   ID=123 RTR=1 LOST
bus      0 0 0 1 0 0 1 0 0 0 1 1 0

```

Real Icarus Verilog output. 123h beats 12Bh at ID bit 3; with identical IDs the data frame (RTR dominant) beats the remote frame at the RTR bit.

Two more consequences: two nodes must **never** transmit the same ID with different data (arbitration would not resolve and a bit error would follow), and a base-format frame beats an extended frame with the same first 11 ID bits, because the extended frame sends a recessive SRR where the base frame has its dominant RTR.

INTERVIEW INSIGHT: WHY CAN LATENCY IS ANALYSABLE

Because arbitration is priority-based and non-destructive, worst-case response times can be computed with fixed-priority scheduling analysis (like tasks on an RTOS): a message waits at most for one lower-priority frame already on the bus (no pre-emption mid-frame) plus all higher-priority traffic. Classic Ethernet with random back-off had no such bound, which is why cars used CAN for control loops and why automotive Ethernet needed TSN before it could replace it.

15.4 Bit stuffing and the CRC-15

CAN is NRZ-coded and self-clocked: receivers resynchronize on recessive-to-dominant edges. A long run of identical bits would give no edges, so the transmitter inserts a **stuff bit** of the opposite value after **five consecutive identical bits**, from SOF through the end of the CRC sequence. The receiver removes it. Six identical bits in that region is therefore illegal - a **stuff error** - which is exactly what an error flag (six equal bits) exploits to get everyone's attention.

The CRC is computed over the **unstuffed** bits from SOF through the data field with generator $x^{15} + x^{14} + x^{10} + x^8 + x^7 + x^4 + x^3 + 1$ (0x4599, initial value 0), giving a Hamming distance of 6 for classic frame lengths. The stuffing is then applied to the whole bit string including the CRC. The RTL below implements all three blocks as serial one-bit-per-clock engines, exactly as a CAN controller does:

```

module can_crc15 (                                // serial CRC-15:  $x^{15}+x^{14}+x^{10}+x^8+x^7+x^4+x^3+1$ 
    input  logic clk, clr, en, din,
    output logic [14:0] crc
);
    wire fb = din ^ crc[14];
    always_ff @(posedge clk)
        if (clr)      crc <= '0;
        else if (en)  crc <= {crc[13:0], 1'b0} ^ (fb ? 15'h4599 : 15'h0);
endmodule

module can_stuff (                                // inserts a complement bit after 5 equal bits
    input  logic clk, clr, en, din,
    output logic dout, take                        // take = 0: this cycle is a stuff bit, hold din
);
    logic [2:0] run; logic last;
    assign take = (run != 5);
    assign dout = take ? din : ~last;
    always_ff @(posedge clk)
        if (clr) begin run <= 0; last <= 1'b1; end
        else if (en) begin
            run <= (!take || dout != last) ? 3'd1 : run + 1; // stuff bit starts a new run
            last <= dout;
        end
endmodule

module can_destuff (                              // drops stuff bits; flags 6 equal bits
    input  logic clk, clr, en, din,
    output logic keep, stuff_err                  // keep = 0: din was a stuff bit
);
    logic [2:0] run; logic last;
    assign keep = (run != 5);
    assign stuff_err = en && (run == 5) && (din == last);
    always_ff @(posedge clk)
        if (clr) begin run <= 0; last <= 1'b1; end
        else if (en) begin
            run <= (din != last || run == 5) ? 3'd1 : run + 1;
            last <= din;
        end
endmodule

```

CAN CRC-15 generator, bit stuffer and destuffer with stuff-error detection.

```

module tb;
    logic clk = 0; always #5 clk = ~clk;
    // SOF, ID=123h, RTR, IDE, r0, DLC=2, data 00 FF (35 bits, sent MSB first)
    localparam logic [34:0] HDR = {1'b0, 11'h123, 3'b000, 4'd2, 8'h00, 8'hFF};
    logic clr = 1, c_en = 0, c_din = 0, s_en = 0, s_din, d_en = 0, d_din;
    logic [14:0] crc; logic s_out, take, keep, serr;
    logic [49:0] frame; logic [99:0] stuffed; logic [49:0] unst; int ns, nu, errs;
    string s_str = "", m_str = "";
    can_crc15  c (.clk, .clr, .en(c_en), .din(c_din), .crc);
    can_stuff  s (.clk, .clr, .en(s_en), .din(s_din), .dout(s_out), .take);
    can_destuff d (.clk, .clr, .en(d_en), .din(d_din), .keep, .stuff_err(serr));
    initial begin
        @(posedge clk) clr <= 0;
    end
endmodule

```

```

for (int i = 34; i >= 0; i--) begin @(negedge clk) begin c_en = 1; c_din = HDR[i]; end end
@(negedge clk) c_en = 0;
frame = {HDR, crc};
$display("verilog: CRC-15 = %h", crc);
// ---- stuff SOF .. CRC sequence ----
ns = 0; clr = 1; @(negedge clk) clr = 0;
for (int i = 49; i >= 0; ) begin
    @(negedge clk) begin s_en = 1; s_din = frame[i]; end
    #1 s_str = {s_str, s_out ? "1" : "0"}; m_str = {m_str, take ? " " : "^"};
    stuffed[99 - ns] = s_out; ns++;
    if (take) i--;
end
@(negedge clk) s_en = 0;
$display("verilog: %0d stuffed bits", ns);
$display("verilog: %s", s_str);
$display("      %s (^ = stuff bit)", m_str);
// ---- destuff and compare, then corrupt one stuff bit ----
for (int pass = 0; pass < 2; pass++) begin
    clr = 1; @(negedge clk) clr = 0; nu = 0; errs = 0;
    for (int i = 0; i < ns; i++) begin
        @(negedge clk) begin
            d_en = 1; d_din = stuffed[99 - i];
            if (pass == 1 && i == 17) d_din = ~d_din;          // bit 17 is the first stuff bit
        end
        #1 if (serr) errs++;
        if (keep) begin unst[49 - nu] = d_din; nu++; end
    end
    @(negedge clk) d_en = 0;
    if (pass == 0) $display("destuffed %0d bits, equal to original: %s", nu,
                           unst == frame ? "yes" : "NO");
    else          $display("stuff bit corrupted -> stuff errors detected: %0d", errs);
end
$finish;
end
endmodule

```

Testbench: a base data frame with ID 123h, DLC 2, data 00 FFh. The CRC is computed, the frame stuffed, destuffed and compared; then one stuff bit is corrupted.

```

verilog: CRC-15 = 63e6
verilog: 55 stuffed bits
verilog: 0001001000110000011000001000011111011111000011111000110
           ^      ^      ^      ^      ^      (^ = stuff bit)
destuffed 50 bits, equal to original: yes
stuff bit corrupted -> stuff errors detected: 1

```

Real Icarus Verilog output: CRC-15 = 63E6h; 50 bits become 55 on the wire; destuffing restores the original; a corrupted stuff bit produces a stuff error.

```

# Python reference: classic CAN base frame, CRC-15 (poly 0x4599) and bit stuffing
def bits(v, n): return [(v >> (n - 1 - i)) & 1 for i in range(n)]
ID, DLC, DATA = 0x123, 2, [0x00, 0xFF]
f = [0] + bits(ID, 11) + [0, 0, 0] + bits(DLC, 4)          # SOF, ID, RTR, IDE, r0, DLC
for d in DATA: f += bits(d, 8)
crc = 0
for b in f:                                                 # CRC over SOF .. data field
    fb = b ^ ((crc >> 14) & 1)
    crc = (crc << 1) & 0x7FFF
    if fb: crc ^= 0x4599
f += bits(crc, 15)
out, mark, run, last = [], [], 0, None
for b in f:                                                 # stuff SOF .. CRC sequence
    out.append(b); mark.append(' ')

```

```

run = run + 1 if b == last else 1; last = b
if run == 5:
    out.append(1 - b); mark.append('^'); last = 1 - b; run = 1
print("python: %d unstuffed bits, CRC-15 = %04x, %d stuffed bits (%d stuff bits)"
      % (len(f), crc, len(out), len(out) - len(f)))
print("python: " + "".join(map(str, out)))
# independent check: remainder of (message * x^15) mod generator by long division
m = int("".join(map(str, f[:-15])), 2) << 15
g = 0xC599                                     # x^15 + 0x4599 terms
for s in range(m.bit_length() - 1, 14, -1):
    if (m >> s) & 1: m ^= g << (s - 15)
print("python: long-division remainder = %04x" % m)

```

An independent Python reference: the same frame, CRC by shift register and by polynomial long division, and the stuffed bit string.

```

python: 50 unstuffed bits, CRC-15 = 63e6, 55 stuffed bits (5 stuff bits)
python: 0001001000110000011000001000011111011111000011111000110
python: long-division remainder = 63e6

```

Python output - identical CRC (by two methods) and identical stuffed bit string to the Verilog run.

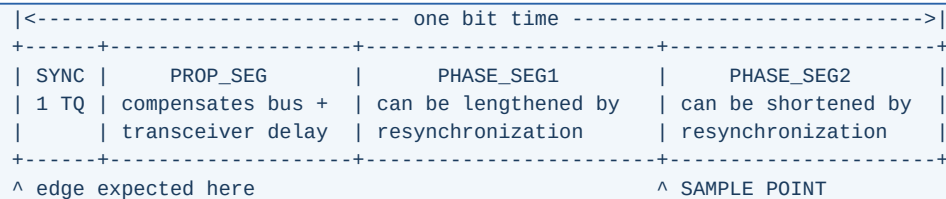
Look at where the stuff bits fell: after the five zeros of RTR, IDE, r0 and the first two DLC bits; in the 00h data byte; inside the FFh byte; across the boundary between the FFh byte and the CRC; and inside the CRC itself. Worst-case stuffing adds one bit per four after the first five, so frame length - and therefore worst-case latency - depends on the data, a subtlety that schedule analyses must include.

PITFALL: THE STUFFING BOUNDARY AND THE CRC DELIMITER

Stuffing covers SOF through the **CRC sequence** only; the CRC delimiter, ACK field and EOF are fixed-form and never stuffed. A controller that keeps stuffing into the delimiter, or whose destuffer still counts the last CRC bits into the delimiter, fails interoperability tests in a data-dependent way - exactly the kind of bug that random-data simulation finds and directed tests miss.

15.5 Bit timing

Each bit is divided into **time quanta** (TQ) derived from the controller clock by a baud-rate prescaler: $TQ = BRP / f_{clk}$. A bit time consists of four segments:



CAN bit timing. The sample point sits between PHASE_SEG1 and PHASE_SEG2, typically at 75-87.5 % of the bit.

- **Hard synchronization** on the SOF edge restarts the bit timing.
- **Resynchronization** on every recessive-to-dominant edge: if the edge falls in PHASE_SEG1 it is lengthened, if in PHASE_SEG2 it is shortened, by at most the **SJW** (synchronization jump width, 1-4 TQ classically).
- **PROP_SEG** must cover twice the one-way delay (bus plus transceiver), because during arbitration and ACK a node must see the far node's dominant bit before its own sample point.
- A later sample point tolerates more propagation delay; an earlier one tolerates more oscillator error. Automotive practice (CiA recommendations) favours about 87.5 %.

f_clk = 80 MHz, target 500 kbit/s -> 160 clocks per bit

BRP = 10 -> TQ = 125 ns ; 16 TQ per bit

SYNC = 1, PROP_SEG + PHASE_SEG1 = 13, PHASE_SEG2 = 2 -> sample at 14/16 = 87.5 %

SJW = 2 TQ

A worked bit-timing configuration.

15.6 Error detection and fault confinement

Error	Detected by	How
Bit error	Transmitter	Reads back a different value than it sent (outside arbitration and the ACK slot)
Stuff error	Receivers	Six equal bits in a stuffed field
CRC error	Receivers	Computed CRC differs from received
Form error	Receivers	A fixed-form bit (delimiter, EOF) has the wrong value
ACK error	Transmitter	Nobody pulled the ACK slot dominant

A node that detects an error transmits an **error flag**. In the normal **error-active** state this is six **dominant** bits, which deliberately violates stuffing so every other node also detects an error and discards the frame - a global, consistent rejection; the transmitter then retries automatically. To stop a faulty node from destroying all traffic, each node keeps two counters:

TEC (transmit error counter): +8 per transmit error, -1 per successful frame
 REC (receive error counter): +1 per receive error (+8 in some cases), -1 per good

TEC or REC > 127		TEC > 255	
+-----+ ERROR ACTIVE flags: 6 dom <----- +-----+ both<=127	+-----+ ERROR PASSIVE flags: 6 rec, extra 8-bit suspend TX +-----+	+-----+ BUS-OFF no TX/RX +-----+	
^ +----- 128 x 11 consecutive recessive bits (+ SW request) +			

CAN fault confinement.

An **error-passive** node's flag is six **recessive** bits, which cannot disturb other nodes' frames, and it must wait an extra suspend-transmission time before retrying. A node whose TEC exceeds 255 goes **bus-off** and stops participating until it has seen 128 occurrences of 11 recessive bits (and, in most controllers, software has requested recovery). The asymmetric +8/-1 weighting means a node with a broken transmitter removes itself quickly while occasional noise is forgiven.

15.7 CAN FD

CAN FD (flexible data rate; Bosch 2012, in ISO 11898-1:2015) keeps arbitration at the classic nominal rate but can switch to a faster **data phase** and carries up to **64 bytes**:

|SOF|ID..|RRS|IDE|FDF|res|BRS|ESI|DLC| DATA 0-64 B |SC| CRC-17/21 |CD|ACK|..
 |<--- nominal bit rate (e.g. 500k) --->|<--- data bit rate (e.g. 2-5M) --->|<-nominal
 switch at BRS sample point ^ switch back at CRC delimiter ^

CAN FD base frame. FDF (recessive) marks an FD frame at the old r0 position; BRS selects the fast data phase; ESI reports the transmitter's error-passive state; SC is the stuff count.

DLC	0-8	9	10	11	12	13	14	15
Bytes (CAN FD)	0-8	12	16	20	24	32	48	64

- **Longer CRCs:** CRC-17 for up to 16 data bytes, CRC-21 for more, to keep the Hamming distance at the longer lengths.
- **Stuff count and fixed stuff bits (ISO CAN FD):** the original non-ISO FD had a flaw - a stuff bit corrupted in a way that shifted the frame could escape the CRC. ISO CAN FD adds a 3-bit Gray-coded stuff-bit count with parity before the CRC, and uses **fixed stuff bits** (one every four bits) inside the CRC field. Non-ISO and ISO FD are not interoperable - a configuration bit in controllers.
- **No remote frames** in FD (the RTR position becomes RRS, always dominant).
- **Transmitter delay compensation (TDC):** at several Mbit/s the transceiver loop delay exceeds a data-phase bit, so the transmitter checks its own bits at a **secondary sample point** delayed by the measured loop delay.
- The data-phase rate is limited in practice by ringing on the bus topology; 2 Mbit/s is common, 5 Mbit/s with careful design; 'SIC' (signal improvement capability) transceivers extend this.

15.8 CAN XL

CAN XL (CiA 610-1, being standardized in ISO 11898-1/-2 revisions) is the third generation: data fields of **1 to 2048 bytes**, data phase rates of roughly 10-20 Mbit/s, an 11-bit priority ID for arbitration (kept separate from a 32-bit **acceptance field**), an SDU-type field that says what the payload carries (for example a tunnelled Ethernet frame), a virtual CAN network ID, and stronger CRCs (a header CRC and a 32-bit frame CRC). New transceivers switch to a different signalling mode during the data phase. Its purpose is to bridge the gap between CAN FD and 10BASE-T1S Ethernet in zonal vehicle architectures while keeping CAN's deterministic arbitration.

15.9 LIN

LIN (Local Interconnect Network; LIN 2.x, standardized as ISO 17987) is a cheap single-wire, 12 V, UART-based sub-bus for up to about 16 nodes at up to 20 kbit/s (19.2 kbit/s is typical). One **commander** (master) node schedules all traffic; **responders** (slaves) never talk unsolicited. That determinism lets responders be tiny state machines, often with an RC oscillator instead of a crystal.

<----- header (commander) -----> <- response (a responder) ->										
+-----+-----+-----+-----+-----+-----+ .. +-----+										
BREAK >= 13 bits	d	SYNC = 55h		PID = ID + P0P1		D0		D1		checksum
dominant (0)	1	(UART 8N1)		(UART 8N1)		8N1		8N1		8N1
+-----+-----+-----+-----+-----+-----+ .. +-----+										
^ break delimiter >= 1 recessive bit; 1-8 data bytes										

A LIN frame. Every byte is a normal 8N1 UART character.

- **Break:** at least 13 dominant bit times - longer than any legal character - so it cannot be confused with data; responders detect it as a UART break / framing error.
- **Sync byte 55h:** alternating bits let a responder measure the commander's bit time and trim its baud generator (auto-baud, Chapter 11): responders may be about +/-14 % off before sync and must be within about +/-2 % after.
- **Protected identifier:** 6-bit frame ID (0-59 signal frames, 60 = 3Ch commander request and 61 = 3Dh responder response for diagnostics, 62-63 reserved) plus two parity bits: P0 = ID0 ^ ID1 ^ ID2 ^ ID4 and P1 = NOT(ID1 ^ ID3 ^ ID4 ^ ID5).
- **Checksum:** inverted 8-bit sum with end-around carry, over the data (**classic**, LIN 1.x and diagnostic frames) or over PID + data (**enhanced**, LIN 2.x).

```
# LIN protected identifier and checksums (LIN 2.x / ISO 17987)
def pid(i):
    b = [(i >> k) & 1 for k in range(6)]
    p0 = b[0] ^ b[1] ^ b[2] ^ b[4]
    p1 = 1 - (b[1] ^ b[3] ^ b[4] ^ b[5])
    return i | (p0 << 6) | (p1 << 7)
def checksum(bytes_):
    # 8-bit sum with end-around carry, inverted
    s = 0
```

```
for b in bytes_:
    s += b
    if s > 0xFF: s -= 0xFF
return (~s) & 0xFF
for i in (0x00, 0x10, 0x3C, 0x3D):
    print("ID %02x -> PID %02x" % (i, pid(i)))
data = [0x4A, 0x55, 0x93, 0xE5]
print("data", " ".join("%02x" % d for d in data))
print("classic checksum (data only)      = %02x" % checksum(data))
print("enhanced checksum (PID %02x + data) = %02x" % (pid(0x10), checksum([pid(0x10)] + data)))
```

LIN PID parity and checksums (Python 3).

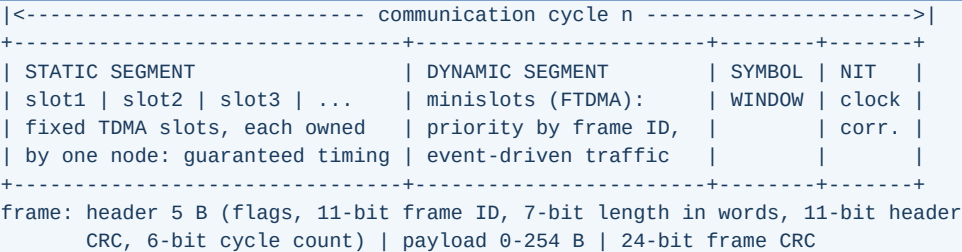
```
ID 00 -> PID 80
ID 10 -> PID 50
ID 3c -> PID 3c
ID 3d -> PID 7d
data 4a 55 93 e5
classic checksum (data only)      = e6
enhanced checksum (PID 50 + data) = 96
```

Real Python output. The classic checksum E6h for data 4A 55 93 E5 matches the worked example in the LIN specification; PID 3Ch/7Dh are the familiar diagnostic identifiers.

In an SoC, LIN is almost always a mode of the UART (Chapter 11): break generation and detection, sync-field measurement for auto-baud, header/response timeouts and checksum hardware are added to a normal UART, with a LIN transceiver on the board.

15.10 FlexRay

FlexRay (FlexRay Consortium, 2000s; ISO 17458) was designed for deterministic, fault-tolerant x-by-wire systems: 10 Mbit/s on each of **two channels** (A and B), which carry either redundant copies or different data. Communication is **time-triggered**: a global time base is maintained by clock synchronization, and a repeating communication cycle (typically 1-5 ms, counted 0-63) is divided into:



The FlexRay cycle and frame.

FlexRay's cost and complexity (bus guardians, star couplers, precise configuration of the whole cluster) limited it mostly to premium chassis applications; new designs use CAN FD/XL and time-sensitive Ethernet instead. It remains an excellent example of a TDMA protocol and of time-triggered architecture.

15.11 Automotive Ethernet

Standard	Rate	Encoding / medium	Notes
100BASE-T1 (IEEE 802.3bw)	100 Mbit/s full duplex	PAM3 at 66.7 MBd, one unshielded twisted pair, ~15 m	Origin: BroadR-Reach; the workhorse of in-vehicle Ethernet
1000BASE-T1 (802.3bp)	1 Gbit/s full duplex	PAM3 at 750 MBd, one pair, 15 m (type A) / 40 m (type B)	Cameras, ADAS, backbone
10BASE-T1S (802.3cg)	10 Mbit/s half duplex	Differential Manchester, multidrop (at least 8 nodes over 25 m)	PLCA (physical layer collision avoidance) gives each node a transmit opportunity in turn: deterministic, CAN-like buses

Standard	Rate	Encoding / medium	Notes
10BASE-T1L (802.3cg)	10 Mbit/s	PAM3, point-to-point, up to 1 km	Industrial / process automation (Ethernet-APL)
2.5/5/10GBASE-T1 (802.3ch)	multi-gigabit	PAM4, one pair	Sensor and backbone links

Full-duplex twisted-pair PHYs use echo cancellation to send and receive on one pair simultaneously (Chapter 19 covers Ethernet PHYs and the xMII interfaces to the MAC). Vehicles add **TSN** (IEEE 802.1AS time synchronization, 802.1Qbv scheduled traffic, and related standards) for bounded latency, and MACsec (802.1AE) for security. The trend is **zonal architectures**: a few zone controllers on an Ethernet backbone, each bridging to local CAN FD/XL, LIN and 10BASE-T1S segments.

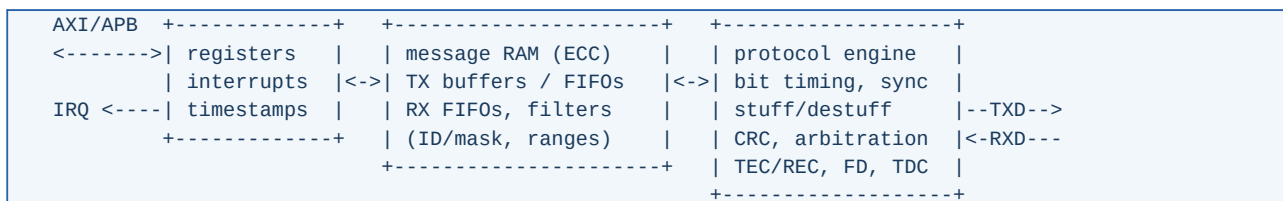
15.12 Industrial buses in brief

Protocol	Layer	Key idea
Modbus RTU / TCP	RS-485 / TCP port 502	Simple register read/write polling (Chapter 11)
PROFIBUS DP	RS-485, up to 12 Mbit/s	Cyclic master-slave token-passing fieldbus
PROFINET RT / IRT	Ethernet	RT: prioritized frames; IRT: scheduled, time-synchronized cut-through switching in dedicated ASICs for sub-millisecond cycles
EtherCAT	Ethernet	One frame visits every slave in a logical ring; each slave controller (ESC) reads and inserts its data on the fly as the frame passes, with distributed clocks for sub-microsecond synchronization
EtherNet/IP	Ethernet, TCP/UDP	CIP object model over standard Ethernet
IO-Link	3-wire 24 V point-to-point	Smart sensor/actuator link to a master (4.8/38.4/230.4 kbit/s)

WHY THESE MATTER TO SOC DESIGNERS

Industrial and automotive SoCs integrate these controllers directly: CAN FD controllers with message RAM, EtherCAT slave controllers, TSN-capable Ethernet MACs, LIN-capable UARTs. The protocol engines are often licensed IP (for example the widely used Bosch M_CAN CAN FD IP), and the integrator's work is the bus interface, message RAM with ECC, interrupts, timestamps and safety mechanisms.

15.13 A CAN controller in an SoC



Structure of a CAN FD controller (transceiver external).

- **Acceptance filtering** in hardware (ID/mask pairs, ranges) keeps the CPU from seeing every frame on a busy bus.
- **TX prioritization**: the controller must always offer the highest-priority pending message to arbitration, not the oldest - a FIFO-only TX queue causes **priority inversion**.
- **Timestamps** at SOF or EOF for time synchronization and diagnostics.
- **Safety** (Chapter 26): ECC on message RAM, loopback and bus monitoring self-tests, protected configuration registers.
- **DV**: a CAN VIP that injects every error type at every bit position, random stuffing-heavy data, arbitration with many nodes, and bit-timing tolerance (oscillator offsets, propagation delay); ISO 16845 defines conformance test plans for CAN controllers.

15.14 Summary

- CAN uses a differential wired-AND bus: dominant 0 overrides recessive 1, enabling non-destructive bitwise arbitration in which the lowest ID wins (demonstrated in RTL).
- Classic frames: SOF, 11/29-bit ID, RTR, IDE, DLC, 0-8 bytes, CRC-15, ACK, EOF; stuffing after five equal bits from SOF to the end of the CRC. The Verilog CRC-15 (63E6h) and stuffed stream matched a Python reference exactly.
- Bit timing: TQ from a prescaler; SYNC + PROP + PHASE1 + PHASE2; sample point ~75-87.5 %; resynchronization limited by SJW.
- Fault confinement: TEC/REC counters, error-active, error-passive (> 127) and bus-off (TEC > 255) states.
- CAN FD: up to 64 bytes, bit-rate switch, CRC-17/21, stuff count; CAN XL: up to 2048 bytes and ~10-20 Mbit/s.
- LIN: commander-scheduled UART frames with break, 55h sync, parity-protected ID and checksum; FlexRay: dual-channel TDMA; automotive Ethernet: 100BASE-T1, 1000BASE-T1, 10BASE-T1S (PLCA multidrop); industrial: Modbus, PROFINET, EtherCAT and others.

15.15 Exercises

- 1 Three nodes start simultaneously with IDs 0x3A5, 0x3A4 and 0x1FF. Which wins, and at which ID bit does each loser drop out?
- 2 Compute the worst-case number of stuff bits for a classic base frame with 8 data bytes, and the resulting worst-case frame time at 500 kbit/s including the 3-bit intermission.
- 3 Choose BRP and segment lengths for 1 Mbit/s from a 40 MHz clock with a sample point near 80 %. What SJW would you pick?
- 4 Modify `can_stuff/can_destuff` for the ISO CAN FD CRC field (fixed stuff bit every 4 bits, always the complement of the previous bit). Why can a fixed-position stuff bit not be lost undetected?
- 5 Starting from error-active, how many consecutive transmit errors drive a node bus-off? How many to become error-passive?
- 6 Compute the LIN PID and enhanced checksum for frame ID 0x22 with data 01 02 03 using the Python model, then verify by hand.

16. Audio and Miscellaneous Links: I2S, TDM, PDM, S/PDIF, 1-Wire, MDIO and SWD

The last low-speed chapter collects a family of small but ubiquitous serial links. Digital audio uses its own clocked serial formats (I2S, TDM, PDM) inside the box and a self-clocked one (S/PDIF) between boxes. 1-Wire reaches ID chips and temperature sensors with a single wire, MDIO/MDC configures Ethernet PHYs, and SWD gives a debugger a two-wire path into an Arm SoC. Each teaches something distinct: continuous streaming with frame alignment, decimation of a 1-bit stream, biphase-mark self-clocking, time-slot signalling, turnaround cycles and packet-based debug access.

16.1 I2S: the inter-IC sound bus

I2S (Philips, 1986) carries PCM audio between an SoC's audio interface and codecs, DACs, ADCs and amplifiers. It is a continuous stream, not a transaction bus - there is no address and no idle time. Three (or four) signals:

Signal	Also called	Function
SCK	BCLK, bit clock	One cycle per data bit: 64 x fs for two 32-bit slots, e.g. 3.072 MHz at 48 kHz
WS	LRCLK, frame clock	Word select: 0 = left, 1 = right; runs at the sample rate fs
SD	SDATA, DIN/DOUT	Two's-complement samples, MSB first; changes on the falling SCK edge, sampled on the rising edge
MCLK	system/master clock	Optional oversampling clock for the codec's converters, typically 256 x fs (12.288 MHz at 48 kHz, 11.2896 MHz at 44.1 kHz)

```
SCK  _/_/_/_/_/_/_/_/_/_ .. _/_/_/_/_/_/_/_/_/_/_ .. _/_/_/_/_/_/_/_/_/_/_
WS   ~~~~\_____ .. _____/~~~~~ .. ~~~~~\_____
SD   ==X 0 X MSB X B14 X .. X B0 X 0 X .. X 0 X MSB X B14 .. X 0 X MSB
      last <-- left: 16 data bits, 16 zeros --> <-- right word ...
           bit of the previous right slot
WS changes one SCK before the MSB: the MSB is delayed by one bit clock
```

Philips I2S format (16-bit samples in 32-bit slots). The receiver does not need to know the word length: it takes the MSB after each WS edge and ignores or zero-fills extra bits.

Two common variants differ only in alignment, and a mismatch produces loud, distorted audio (samples shifted by one bit, or left/right swapped) rather than silence, which makes it easy to misdiagnose:

Format	MSB position	WS polarity	Notes
I2S (Philips)	One SCK after the WS edge	0 = left	Default for most codecs
Left-justified	Coincident with the WS edge	Usually 1 = left	No one-bit delay; word length need not be known
Right-justified	LSB coincides with the end of the slot	Usually 1 = left	Receiver must know the word length (16/20/24) to find the MSB

The transmitter and receiver below implement the Philips format with 32-bit slots and a width parameter W. The transmitter is the clock master (it generates $SCK = clk/2$ and WS); the receiver lives entirely in the SCK domain, as codec receivers do, and aligns itself on WS transitions.

```
// Philips I2S, 64 SCK per frame (32-bit slots), W-bit samples MSB first.
// WS: 0 = left, 1 = right; it changes one SCK before the MSB of the new channel.
module i2s_tx #(parameter int W = 16) (
    input  logic          clk, rst_n,          // clk = 2 x SCK
    input  logic [W-1:0] left, right,
    output logic          load,                // pulse: left/right captured, give next pair
```

```

    output logic      sck, ws, sd
);
    logic [5:0] pos;  logic [W-1:0] l_q, r_q;
    wire  [5:0] nxt = pos + 1;
    wire  [4:0] idx = nxt[4:0];          // bit position inside the next slot
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin sck <= 0; pos <= 6'd63; ws <= 0; sd <= 0; load <= 0;
                           l_q <= 0; r_q <= 0; end
        else begin
            sck <= ~sck;
            load <= 0;
            if (sck) begin                // falling SCK: transmitter changes outputs
                pos <= nxt;
                if (nxt == 0) begin l_q <= left; r_q <= right; load <= 1; end
                ws <= (6'(nxt + 1) >= 32); // leads the slot by one bit
                if (idx < W) sd <= nxt[5] ? r_q[W-1-idx] : ((nxt == 0) ? left[W-1] : l_q[W-1-idx]);
                else sd <= 1'b0;          // unused slot bits are zero
            end
        end
    end
endmodule

module i2s_rx #(parameter int W = 16) ( // lives in the SCK domain
    input logic      sck, rst_n, ws, sd,
    output logic      valid, right, // valid for one SCK: word just completed
    output logic [W-1:0] data
);
    logic ws_d; logic [5:0] cnt; logic [W-1:0] sh;
    wire  [W-1:0] sh_n = (cnt < W) ? {sh[W-2:0], sd} : sh;
    always_ff @(posedge sck or negedge rst_n)
        if (!rst_n) begin ws_d <= 0; cnt <= 6'd63; sh <= 0; valid <= 0; data <= 0; right <= 0; end
        else begin
            ws_d <= ws;
            valid <= 0;
            if (ws != ws_d) begin // this is the last bit of the old slot
                if (cnt != 63) begin data <= sh_n; right <= ws_d; valid <= 1; end
                cnt <= 0;
            end else begin
                sh <= sh_n;
                if (cnt != 63) cnt <= cnt + 1;
            end
        end
    end
endmodule

```

Parameterized I2S transmitter (clock master) and receiver. The WS edge is issued one bit before each slot; the receiver treats the bit sampled with a new WS value as the last bit of the previous word.

```

module pair #(parameter int W = 16) (input logic clk, rst_n);
    logic [W-1:0] L [4], R [4], l, r, rx; logic load, sck, ws, sd, valid, right;
    int k = 0, rk = 0, errs = 0, words = 0; string w_ws = "", w_sd = "";
    i2s_tx #(W) tx (.clk, .rst_n, .left(l), .right(r), .load, .sck, .ws, .sd);
    i2s_rx #(W) rxu (.sck, .rst_n, .ws, .sd, .valid, .right, .data(rx));
    initial for (int i = 0; i < 4; i++) begin
        L[i] = W'(64'hA5C3_F10E_9B27_D4E8 >> (8 * i));
        R[i] = ~W'(64'h1234_5678_9ABC_DEF0 >> (4 * i));
    end
    assign l = L[k % 4]; assign r = R[k % 4];
    always @(posedge clk) if (load) k++;
    always @(posedge sck) if (valid) begin
        // the receiver needs one WS edge to lock, so the first full left word is frame 1
        if (!right && rk == 0) rk = 1;
        if (rk > 0) begin

```

```

        if (rx !== (right ? R[rk % 4] : L[rk % 4])) errs++;
        words++;
        if (words <= 2) $display("W=%0d  %s word %h", W, right ? "R" : "L", rx);
        if (right) rk++;
    end
end
always @(posedge sck) if (k == 3 && w_ws.len() < 64) begin          // frame 2 as RX sees it
    w_ws = {w_ws, ws ? "1" : "0"};  w_sd = {w_sd, sd ? "1" : "0"};
end
endmodule

module tb;
    logic clk = 0, rst_n = 0;  always #5 clk = ~clk;
    pair #(16) p16 (.clk, .rst_n);
    pair #(24) p24 (.clk, .rst_n);
    initial begin
        repeat (3) @(posedge clk); rst_n = 1;
        repeat (64 * 2 * 10) @(posedge clk);
        $display("W=16: %0d words checked, %0d errors; W=24: %0d words checked, %0d errors",
            p16.words, p16.errs, p24.words, p24.errs);
        $display("W=16 frame 2 (L=%h R=%h), one char per SCK rising edge:", p16.L[2], p16.R[2]);
        $display("  WS %s", p16.w_ws);  $display("  SD %s", p16.w_sd);
        $finish;
    end
endmodule

```

Testbench: 16-bit and 24-bit transmitter/receiver pairs streaming stereo samples, a self-checking scoreboard and a trace of WS/SD for one frame.

```
W=24 L word 9b27d4
W=16 L word 27d4
W=24 R word 543210
W=16 R word 3210
W=16: 17 words checked, 0 errors; W=24: 17 words checked, 0 errors
W=16 frame 2 (L=9b27 R=4321), one char per SCK rising edge:
WS 0000000000000000000000000000000011111111111111111111111111111110
SD 1001101100100111000000000000000000100001100100001000000000000000
```

Real Icarus Verilog output: both widths stream without error. In the trace, WS is 0 for 31 bits, 1 for 32 bits and 0 again one bit before the next left word: SD carries 9B27h (left) from the bit after the WS edge, then 4321h (right) from bit 32.

PITFALL: THE FIRST FRAME AND WORD ALIGNMENT

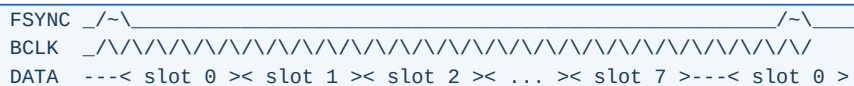
The receiver discards everything before the first WS edge it sees (it cannot know where a word began), which is why the testbench starts scoring at frame 1. Real systems see the same effect after every stream start or clock switch: DMA must be started so that channel 0 lands in the correct buffer position, or left and right are swapped for the whole session - a very common audio driver bug.

- **Clocking** is the hard part of audio SoC design: 48 kHz and 44.1 kHz families need two MCLK frequencies (or a fractional audio PLL), and jitter on MCLK/SCK directly degrades converter SNR.
- **Master/slave roles**: either the SoC or the codec can generate SCK/WS; when the codec is master, the SoC's I2S block runs in an external clock domain and its FIFOs cross to the bus clock.
- **Asynchronous sample-rate conversion (ASRC)** hardware bridges two audio domains whose clocks are not locked (e.g. Bluetooth audio to a local DAC).

16.2 TDM: many channels on one data line

TDM (time-division multiplexed) audio, sometimes called DSP mode, puts N channel slots into each frame. A **frame sync** (FSYNC) marks the start of the frame - either a one-bit pulse or a 50 % duty clock - and slots follow back to back. Eight 32-bit slots at 48 kHz need a 12.288 MHz bit clock. DSP mode A starts the first slot one BCLK after the FSYNC edge (like I2S); mode B starts it on the same edge (like left-justified). TDM is

standard for microphone arrays, multi-channel codecs, smart amplifiers (which also return current/voltage sense data in spare slots) and automotive audio.



TDM framing (8 slots per frame, DSP mode A).

16.3 PDM microphones and decimation

Almost every phone and laptop microphone is a MEMS device with a **PDM** (pulse-density modulation) output: an on-chip sigma-delta modulator produces a **1-bit stream** at a high rate (typically 1 to 3.2 MHz; 3.072 MHz = 64 x 48 kHz), in which the density of 1s follows the audio waveform and the quantization noise is pushed to high frequencies. Two microphones share one data line: a SEL pin makes one drive data after the rising clock edge and the other after the falling edge, so the SoC samples both on alternate edges.

The SoC must **decimate**: low-pass filter away the shaped noise and reduce the rate by 64. The first stage is almost always a **CIC** (cascaded integrator-comb, Hogenauer) filter, because it needs no multipliers: N integrators at the input rate, a rate change by R, then N combs (differentiators) at the output rate. Its response is sinc^N , with droop in the passband that a following FIR compensates; half-band FIR stages complete the decimation.

$$H(z) = \left[\frac{(1 - z^{-RM})}{(1 - z^{-1})} \right]^N \quad (\text{CIC, } R = \text{decimation, } M = \text{diff. delay})$$

gain = $(R \times M)^N \rightarrow$ output width = input width + $N \times \log_2(R \times M)$

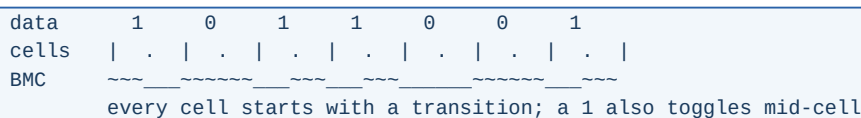
example: 1-bit PDM, $N = 5$, $R = 32$, $M = 1 \rightarrow 1 + 5 \times 5 = 26$ -bit integrators

PDM 3.072 MHz --CIC /32--> 96 kHz --half-band FIR /2--> 48 kHz PCM --> I2S/DMA

CIC decimator sizing. Integrators rely on two's-complement wrap-around: overflow in them is harmless as long as the registers are as wide as the output requires.

16.4 S/PDIF and AES3: self-clocked audio between boxes

S/PDIF (consumer, IEC 60958 Type II, over 75 ohm coax or optical TOSLINK) and AES3 (professional, 110 ohm balanced) carry stereo PCM (or compressed IEC 61937 streams) over **one** signal with no separate clock. They use **biphase mark coding (BMC)**: every bit cell begins with a transition, and a 1 has an extra transition in the middle. The signal is DC-free, polarity-insensitive, and the receiver recovers the clock with a PLL.



```
subframe (32 time slots):
```

0-3	4-7	8	27	28	29	30	31
preamble	aux	audio sample, LSB first (20 b)	V	U	C	P	

preamble B/Z = channel A + block start, M/X = channel A, W/Y = channel B
 (preambles deliberately violate BMC so they can never occur in data)

```
frame = 2 subframes; block = 192 frames -> 192-bit channel-status word from C
```

Biphase-mark coding and the IEC 60958 subframe. Aux bits extend the sample to 24 bits. P gives even parity over slots 4-31.

The bit rate is $64 \times f_s$ (3.072 Mbit/s at 48 kHz), so the BMC cell rate is $128 \times f_s$. A receiver in RTL oversamples the input, measures pulse widths (one, two or three half-cells - three only occurs in preambles), locks to the preambles, and extracts the samples; the channel-status block carries the sample rate, copy protection and whether the payload is PCM or compressed.

16.5 1-Wire

Maxim's (now Analog Devices') 1-Wire bus uses **one** open-drain data line plus ground; devices can even be **parasitically powered** from the pull-up. Timing is everything: the master starts every slot by pulling the line low, and the **duration** of the low pulse encodes the information.

```

reset/presence:
master  ~~~\_____ >= 480 us _____/~~~~~
slave                                     15-60 us \_ 60-240 us _/

write 1:  ~~~\_/~~~~~ low 1-15 us, then release
write 0:  ~~~\_____/~~~~ low 60-120 us
read:     ~~~\_/  slave holds low (0) or not (1); master samples within 15 us
           |<----- time slot 60-120 us ----->| recovery >= 1 us

```

1-Wire standard-speed signalling (overdrive speed is roughly ten times faster).

- **Reset/presence:** every transaction starts with a reset pulse; any device answers with a presence pulse.
- **ROM commands** select devices: Read ROM (33h, single device), Match ROM (55h + 64-bit ROM), Skip ROM (CCh, all devices), **Search ROM (F0h)** and Alarm Search (ECh). Search ROM walks the binary tree of all ROM codes: for each bit, all devices send the bit and its complement (wired-AND reveals whether both values exist) and the master chooses a branch - the same wired-AND trick as I2C and CAN arbitration.
- **Function commands** are device-specific (a DS18B20 thermometer uses 44h Convert T and BEh Read Scratchpad).
- Every device has a unique **64-bit ROM**: 8-bit family code, 48-bit serial number and an 8-bit **CRC**, all sent LSB first.

The 1-Wire CRC-8 (often called 'Dallas/Maxim CRC') uses the polynomial $x^8 + x^5 + x^4 + 1$ with the register shifting right (LSB first, reflected polynomial 8Ch) and initial value 0. Running the CRC over all 8 ROM bytes, including the stored CRC, gives 0 for a valid code.

```

// Maxim/Dallas 1-Wire CRC-8: x^8 + x^5 + x^4 + 1, bits processed LSB first
// (the order they appear on the wire), register shifts right, reflected poly 8Ch.
module ow_crc8 (
    input  logic      clk, clr, en, din,
    output logic [7:0] crc
);
    wire fb = din ^ crc[0];
    always_ff @(posedge clk)
        if (clr)      crc <= 8'h00;
        else if (en)  crc <= {1'b0, crc[7:1]} ^ (fb ? 8'h8C : 8'h00);
endmodule

```

Serial 1-Wire CRC-8, one bit per clock, LSB first.

```

module tb;
    logic clk = 0, clr = 1, en = 0, din = 0;  logic [7:0] crc;
    always #5 clk = ~clk;
    ow_crc8 u (.clk, .clr, .en, .din, .crc);
    // 64-bit ROM codes; byte 0 (family code) is sent first, byte 7 is the stored CRC.
    // The third code has a corrupted CRC byte (2c instead of 2b).
    logic [63:0] rom [3];
    initial begin
        rom[0] = 64'hA2_000000_01B81C_02;  rom[1] = 64'h30_031691_624CFF_28;
        rom[2] = 64'h2C_000802_0D078A_10;
    end
    task automatic run(input int r, input int nbytes, output logic [7:0] c);
        @(negedge clk) clr = 1; @(negedge clk) clr = 0;
        for (int b = 0; b < nbytes; b++)
            for (int i = 0; i < 8; i++) begin                // LSB of each byte first
                @(negedge clk) begin en = 1; din = rom[r][8*b + i]; end
            end
    endtask
endmodule

```

```

        end
        @(negedge clk) en = 0; c = crc;
    endtask
    logic [7:0] c7, c8;
    initial begin
        for (int r = 0; r < 3; r++) begin
            run(r, 7, c7); run(r, 8, c8);
            $display("ROM (wire order) %h %h %h %h %h %h | stored %h  crc(7)=%h  crc(8)=%h %0s",
                    rom[r][7:0], rom[r][15:8], rom[r][23:16], rom[r][31:24], rom[r][39:32],
                    rom[r][47:40], rom[r][55:48], rom[r][63:56], c7, c8, c8 == 0 ? "OK" : "BAD");
        end
        $finish;
    end
endmodule

```

Testbench: the example ROM code from Maxim's application note on 1-Wire CRCs, a DS18B20-style code, and a code whose CRC byte has been corrupted.

```

ROM (wire order) 02 1c b8 01 00 00 00 | stored a2  crc(7)=a2  crc(8)=00 OK
ROM (wire order) 28 ff 4c 62 91 16 03 | stored 30  crc(7)=30  crc(8)=00 OK
ROM (wire order) 10 8a 07 0d 02 08 00 | stored 2c  crc(7)=2b  crc(8)=83 BAD

```

Real Icarus Verilog output: CRC(7 bytes) equals the stored byte for the valid codes and CRC(8 bytes) = 00h; the corrupted code is flagged.

```

def crc8_maxim(data):                                # reflected x^8+x^5+x^4+1, init 0
    c = 0
    for b in data:
        for _ in range(8):
            mix = (c ^ b) & 1
            c >>= 1
            if mix: c ^= 0x8C
            b >>= 1
    return c
for rom in ([0x02, 0x1C, 0xB8, 0x01, 0x00, 0x00, 0x00],
            [0x28, 0xFF, 0x4C, 0x62, 0x91, 0x16, 0x03],
            [0x10, 0x8A, 0x07, 0x0D, 0x02, 0x08, 0x00]):
    print("python: " + " ".join("%02x" % b for b in rom) + "  crc8 = %02x" % crc8_maxim(rom))

```

Python reference for the same CRC.

```

python: 02 1c b8 01 00 00 00  crc8 = a2
python: 28 ff 4c 62 91 16 03  crc8 = 30
python: 10 8a 07 0d 02 08 00  crc8 = 2b

```

Python output: A2h, 30h and 2Bh - identical to the Verilog results (A2h is also the value published in Maxim's application note for that ROM code).

16.6 MDIO / MDC: managing Ethernet PHYs

Every Ethernet MAC configures and monitors its PHY over the **management interface** defined in IEEE 802.3: MDC (clock, driven by the MAC-side station management entity, STA) and MDIO (bidirectional data, with a pull-up). It is also called MIIM or **SMI** (serial management interface). Up to 32 PHY addresses share one bus. 802.3 specifies MDC up to 2.5 MHz; many PHYs accept faster clocks. Data is sampled on the rising edge of MDC.

```

Clause 22:
| PRE (32 x 1) | ST 01 | OP | PHYAD[4:0] | REGAD[4:0] | TA | DATA[15:0] | idle Z |
OP: 01 = write, 10 = read      TA: write = 10 (STA drives)
                                read  = Z0 (PHY drives 0 in 2nd bit)

Clause 45 (indirect, 16-bit register space per device):
| PRE | ST 00 | OP | PRTAD[4:0] | DEVAD[4:0] | TA | ADDRESS or DATA[15:0] |
OP: 00 = address, 01 = write, 11 = read, 10 = read + post-increment

```

MDIO frame formats. The turnaround (TA) gives the bus one bit time to change driver on reads.

- Clause 22 has 32 registers per PHY: 0 control (reset, loopback, speed, auto-negotiation enable), 1 status (link, AN complete), 2-3 PHY identifier, 4-5 auto-negotiation advertisement and link-partner ability, 9-10 1000BASE-T control/status, 15 extended status.
- Clause 45 addresses a port, an **MMD** (MDIO manageable device, e.g. 1 = PMA/PMD, 3 = PCS, 7 = auto-negotiation) and a 16-bit register address - needed for 10G and multi-gig PHYs. A write or read is two frames: an address frame, then a data frame.
- RTL is simple: a shift register and a small FSM. The traps are the tri-state turnaround, sampling MDIO read data late enough after the MDC rising edge (the PHY may drive up to ~300 ns after it), and exclusive access when several software agents poll the PHY.

16.7 SWD as a two-wire serial link

Arm's **Serial Wire Debug** replaces the four or five JTAG pins with SWCLK and a bidirectional SWDIO. It is a packet protocol with turnaround cycles, much like MDIO:

```
| Start=1 | APnDP | RnW | A[2:3] | Parity | Stop=0 | Park=1 | Trn | ACK[0:2] | Trn |
|<----- 8-bit request (host) ----->|      |<- target->|
then 32 data bits LSB first + 1 parity bit (from host on writes, target on reads)
ACK: OK = 001b, WAIT = 010b, FAULT = 100b (bit 0 first on the wire)
line reset: >= 50 SWCLK cycles with SWDIO high
```

An SWD transaction. Chapter 23 covers the Debug Access Port behind it, JTAG-to-SWD switching and CoreSight.

The protocol shows the recurring ideas of this part: a request protected by parity, an explicit turnaround when the direction changes, a three-bit acknowledgement that can ask the host to retry (WAIT - flow control) and a line reset of many 1s that recovers from any state. Its details, the debug port register model and the SoC debug architecture are in Chapter 23.

16.8 GPIO bit-banging

Any of the protocols in this part can be generated by software toggling GPIOs - useful for bring-up, for a protocol no controller supports, or for bus recovery (the nine I2C clocks of Chapter 13 are often bit-banged). The limitations are timing and CPU cost:

- Interrupts, cache misses and bus contention add jitter of microseconds; fine for I2C at 100 kHz or 1-Wire (with interrupts masked during a slot), hopeless for I2S or SPI at tens of MHz.
- GPIO registers are usually behind a slow peripheral bus; set/clear or bit-band registers avoid read-modify-write races between contexts.
- Open-drain protocols need the pin configured as open-drain (or emulated by toggling the output enable with the data fixed at 0).
- Many SoCs therefore include **programmable I/O engines** - small state machines with shift registers and cycle-exact timing (RP2040-style PIO, or timer-triggered DMA to GPIO) - which implement custom or legacy protocols in 'software' without CPU jitter.

16.9 Where these blocks sit in an SoC

Block	Typical integration	Owner concerns
I2S/TDM/PDM	Audio subsystem with its own DMA, audio PLL, often a DSP; always-on for voice wake-up	Clock domains, jitter, FIFO depth vs. latency, low-power listening
S/PDIF	TV/set-top/AV-receiver SoCs, HDMI audio paths	Clock recovery PLL, channel-status handling
1-Wire	Accessory/battery authentication, board ID; often a small controller or bit-banged	Precise slot timing; strong pull-up for parasitic power
MDIO	Part of the Ethernet MAC subsystem	Shared bus arbitration between MAC instances and software agents
SWD	Debug subsystem (DAP)	Security: debug authentication and lock-out in production parts

16.10 Summary

- I2S streams PCM on SCK/WS/SD (plus MCLK); Philips format delays the MSB one bit after WS, left-justified does not, and right-justified needs the word length. Our 16- and 24-bit RTL pairs streamed error-free.
- TDM packs N slots per frame behind a frame sync; PDM carries 1-bit sigma-delta audio that the SoC decimates with CIC and FIR filters.
- S/PDIF/AES3 are self-clocked with biphasic mark coding; 32-slot subframes with BMC-violating preambles; 192-frame blocks carry channel status.
- 1-Wire encodes bits in low-pulse durations on one open-drain wire; ROM commands and the Search ROM algorithm use wired-AND; its CRC-8 ($x^8+x^5+x^4+1$, LSB first) was verified in RTL against Python and the published example.
- MDIO/MDC manages Ethernet PHYs with clause 22 (ST=01) and clause 45 (ST=00) frames and a turnaround; SWD is a two-wire packet protocol with parity, turnaround, ACK/WAIT/FAULT and line reset.
- Bit-banging is fine for slow, jitter-tolerant links; programmable I/O engines give software flexibility with hardware timing.

16.11 Exercises

- 1 A codec expects left-justified 24-bit data but the SoC sends Philips I2S. Describe exactly what the listener hears and why.
- 2 Compute BCLK for 8-slot, 32-bit TDM at 96 kHz, and MCLK = 256 x fs for 44.1 kHz. Why do products often need two audio PLL settings?
- 3 Size a CIC decimator ($N = 4$) that takes a 2.4 MHz PDM stream to 75 kHz. What integrator width is needed, and where is the first passband droop compensated?
- 4 Encode the byte 0xA5 (LSB first) in biphasic-mark code starting from a high level, and show why the encoding is independent of wire polarity.
- 5 Using the Python 1-Wire CRC model, find a single-bit error in the 48-bit serial number that the CRC would not detect, or argue why none exists.
- 6 Write the MDIO bit sequence (including preamble and turnaround) to read clause 22 register 1 of PHY address 3, and mark which side drives each bit.

PART IV

High-Speed and Memory Interfaces

Part III dealt with links measured in kilobits and megabits. This part climbs to the interfaces that set an SoC's pin budget, power and performance: USB from 1.5 Mb/s to USB4's 80 Gb/s, PCI Express and CXL, Ethernet from the MAC down to the xMII pins, the DRAM and flash interfaces that feed every processor, then display and camera links, chip-to-chip and die-to-die interconnect, and finally the debug, test and trace ports every chip must carry. These are layered protocols with a PHY (usually a bought hard macro), a controller in RTL, and a large firmware stack on top; each chapter shows where that split falls and what the RTL, DV, PHY and firmware teams each own.

17. USB 2.0, USB 3.x and USB4

The Universal Serial Bus is the most widely deployed wired peripheral link in the world, and almost every application processor, microcontroller and PC chipset carries at least one USB controller. Its history also explains its complexity: USB 1.x and 2.0 are a half-duplex, host-pollled bus on one differential pair; USB 3.x added a completely separate, full-duplex SerDes link beside the 2.0 pair; USB Type-C changed the connector and brought power negotiation; and USB4 turned the port into a tunnelling fabric that also carries PCI Express and DisplayPort. An SoC team that 'adds USB' therefore integrates two or three different protocol stacks, an analog PHY, a Type-C port controller and a large software stack.

17.1 Versions, speeds and names

Mode / marketing name	Signalling rate	Encoding	Spec / year	Wires used
Low Speed (LS)	1.5 Mb/s	NRZI + bit stuffing	USB 1.0, 1996	D+/D- half duplex
Full Speed (FS)	12 Mb/s	NRZI + bit stuffing	USB 1.0/1.1	D+/D- half duplex
High Speed (HS)	480 Mb/s	NRZI + bit stuffing	USB 2.0, 2000	D+/D- half duplex
SuperSpeed, USB 3.2 Gen 1	5 Gb/s	8b/10b	USB 3.0, 2008	1 TX + 1 RX pair
SuperSpeedPlus, USB 3.2 Gen 2	10 Gb/s	128b/132b	USB 3.1, 2013	1 TX + 1 RX pair
USB 3.2 Gen 2x2	2 x 10 Gb/s	128b/132b	USB 3.2, 2017	2 lanes (Type-C only)
USB4 Gen 2x2 / Gen 3x2	20 / 40 Gb/s	64b/66b / 128b/132b	USB4 v1, 2019	2 lanes, Type-C
USB4 Gen 4 (USB4 v2)	80 Gb/s (120/40 asym.)	PAM3	USB4 v2, 2022	2 lanes, Type-C

USB generations. Data rates are raw line rates; usable throughput is lower (encoding, protocol and software overhead). USB4 always also carries a USB 2.0 link on D+/D-.

The naming has been re-branded several times: 'USB 3.0' SuperSpeed became 'USB 3.1 Gen 1' and then 'USB 3.2 Gen 1', all meaning the same 5 Gb/s link. In a specification or datasheet, trust the signalling rate, not the version number. A 'USB 3.2' device may legally be only a 5 Gb/s device.

17.2 Topology: host, hubs, devices and tiers

tier 1	tier 2	tier 3	...	tier 7
+-----+ HOST (xHCI)	+-----+ root hub (ports)	+-----+ hub +-----+ +-- device (compound: hub + functions)	...	+-----+ device (function) +-- hub -- hub -- ... -- device
- one host per bus; all traffic is scheduled by the host - 7-bit device address: up to 127 devices (address 0 = default, unconfigured) - at most 5 external hubs between host and a device (7 tiers incl. root) - up to 16 endpoints per direction per device; EP0 is the control endpoint				

USB is a tiered star, not a shared multi-drop bus. Hubs repeat traffic downstream and aggregate it upstream.

USB 2.0 is **host-centric**: a device never speaks unless the host addresses it with a token. Interrupt endpoints are therefore *polled* at an interval the device requests. USB 3.x keeps the host in charge of scheduling but lets a device send an asynchronous **ERDY** notification when it becomes ready, so the host does not waste link time polling. A hub is itself a device with a control endpoint and an interrupt endpoint that reports port status changes. An HS hub also contains one or more **Transaction Translators (TT)** that convert HS **split transactions** into FS/LS traffic for slow devices, so a keyboard does not throttle a 480 Mb/s bus.

17.3 USB 2.0 electrical layer: D+, D-, J, K and SE0

LS and FS use single-ended 3.3 V CMOS-like drivers on the two wires of a twisted pair, received both differentially (for data) and single-ended (for SE0 and line state). The host's downstream port has a 15 kohm pull-down on each wire; a device announces itself with a 1.5 kohm pull-up to about 3.3 V: on **D+ for full speed**, on **D- for low speed**. That pull-up is how attach and speed are detected.

Line state	D+ / D-	Meaning
Differential 1	D+ > D-	J for FS/HS; K for LS
Differential 0	D+ < D-	K for FS/HS; J for LS
SE0	both low	End of packet (2 bit times), reset (≥ 10 ms from the host), disconnect (long SE0 with no pull-up)
SE1	both high	Illegal - must never be driven
Idle	J (held by pull-up)	Bus idle between packets
Resume	K for ≥ 20 ms	Wake a suspended bus (host drives it downstream)

High speed is different electrically: it is a **current-mode** driver that steers about 17.78 mA into 45 ohm terminations at each end of the line (both D+ and D- terminated to ground), giving a differential swing of about 400 mV. HS capability is discovered after reset by the **chirp** handshake: an HS-capable device drives a 'chirp K' for about 1-7 ms; an HS-capable host answers with alternating chirp K/J pairs; the device then removes its 1.5 kohm pull-up, turns on its 45 ohm terminations and both sides are in HS mode. A legacy host simply ignores the chirp and the device stays at FS - backward compatibility is built into the reset sequence.

FS attach and HS chirp (not to scale)

```

D+  ____/~~~~~\_____/~\~/~\~/~\  ... HS idle (SE0-like,
D-  _____/~~~~~\_____/~\~/~\~/~\  ~0 V, terminated)
      ^ pull-up  ^ host  ^ device  ^ host chirps K J K J K J
      (FS device) | reset | chirp K | (>= 3 KJ pairs seen: device
                  | SE0  | 1..7 ms | switches to HS terminations)

```

Speed negotiation happens inside the bus reset. The PHY does the analog work; the link controller (and firmware timers) sequence it.

NRZI and bit stuffing

Data is sent LSB first and **NRZI** coded: a 0 bit is sent as a **transition** (J->K or K->J), a 1 bit as **no transition**. NRZI makes the data polarity-independent, but a long run of 1s would give the receiver's clock-recovery DPLL no edges. So the transmitter **stuffs** a 0 after every six consecutive 1s (before NRZI), which forces a transition at least every seven bit times. The receiver removes the stuffed bit; if it sees seven 1s in a row it declares a **bit-stuff error**, which is also how an HS **EOP** is signalled - deliberately.

Field	FS / LS	HS
SYNC	8 bits 00000001 (wire: KJKJKJKK)	32 bits (15 KJ pairs then KK)
EOP	SE0 for 2 bit times, then J	Deliberate bit-stuff error: 8 bits 01111111 NRZI (40 bits after an SOF)
Bit stuffing	after six 1s	after six 1s
Inter-packet / turnaround	on the order of 2-7.5 bit times gap, 16-18 bit-time response timeout	much longer in bit times (hundreds), same order in ns

The line coder below is the part of a USB 2.0 PHY (or of the controller's FS/LS path) that implements this. The transmitter drops **in_ready** for one bit time whenever it inserts a stuff bit, which is exactly why UTMI has a **TxReady** handshake on its parallel interface. The receiver side takes one recovered sample per bit (in a real PHY a DPLL or oversampler produces them, see Chapter 4).

```

// USB 2.0 FS/LS line coding: bit stuffing + NRZI (transmit) and the inverse (receive).
// line = 1 means J (idle), line = 0 means K (full-speed polarity).
module usb_nrzi_tx (
    input  logic clk, rst_n,
    input  logic in_valid, in_bit,    // raw bits, LSB of each byte first
    output logic in_ready,            // low for one cycle while a stuff bit is sent
    output logic line                 // 1 = J, 0 = K
);
    logic [2:0] ones;                // consecutive 1s sent so far
    wire stuff = (ones == 3'd6);     // six 1s in a row: next bit on the wire is a 0
    assign in_ready = !stuff;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin ones <= '0; line <= 1'b1; end          // bus idles in J
        else if (stuff) begin ones <= '0; line <= ~line; end     // stuffed 0 = transition
        else if (in_valid) begin
            if (in_bit) ones <= ones + 3'd1;                    // 1 = no transition
            else begin ones <= '0; line <= ~line; end           // 0 = transition
        end
endmodule

module usb_nrzi_rx (
    input  logic clk, rst_n,
    input  logic line_valid, line,    // one recovered sample per bit time
    output logic out_valid, out_bit,
    output logic stuff_err            // seventh consecutive 1 seen
);
    logic prev;
    logic [2:0] ones;
    wire bit_now = (line == prev);   // NRZI: no change = 1
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            prev <= 1'b1; ones <= '0; out_valid <= 0; out_bit <= 0; stuff_err <= 0;
        end
        else begin
            out_valid <= 1'b0; stuff_err <= 1'b0;
            if (line_valid) begin
                prev <= line;
                if (ones == 3'd6) begin                          // this bit must be a stuffed 0
                    ones <= '0;
                    stuff_err <= bit_now;                        // a 1 here is a bit-stuff error
                end else begin
                    out_valid <= 1'b1; out_bit <= bit_now;
                    ones <= bit_now ? ones + 3'd1 : 3'd0;
                end
            end
        end
endmodule

```

usb_line.sv - FS/LS bit stuffing + NRZI encoder and decoder.

```

module tb;
    logic clk = 0, rst_n = 0;
    always #5 clk = ~clk;
    // SYNC, PID DATA0, then data with long runs of 1s
    localparam int N = 6;
    logic [8*N-1:0] tx = 48'h00_3F_7F_FF_C3_80;    // sent from the low byte up
    logic in_valid, in_bit, in_ready, line;
    logic out_valid, out_bit, stuff_err;
    usb_nrzi_tx utx (.*);
    logic lv = 0;
    always @(posedge clk) lv <= in_valid;
endmodule

```

```

usb_nrzi_rx urx (.clk, .rst_n, .line_valid(lv), .line, .out_valid, .out_bit, .stuff_err);
string wire_s = "";
int nbits = 0, rx_n = 0, stuffed = 0, errs = 0;
logic [7:0] rx_sr, rx_bytes [N];
// receive side: rebuild bytes (LSB first)
always @(posedge clk) if (rst_n) begin
    if (lv) wire_s = {wire_s, line ? "J" : "K"};
    if (out_valid) begin
        rx_sr = {out_bit, rx_sr[7:1]};
        if (++nbits % 8 == 0) begin rx_bytes[rx_n] = rx_sr; rx_n++; end
    end
    if (stuff_err) errs++;
end
initial begin
    in_valid = 0; in_bit = 0;
    repeat (2) @(posedge clk); rst_n = 1;
    for (int b = 0; b < N; b++) // drive on the falling edge
        for (int i = 0; i < 8; i++) begin
            @(negedge clk);
            while (!in_ready) begin stuffed++; @(negedge clk); end
            in_valid = 1; in_bit = tx[8*b+i];
        end
        @(negedge clk); in_valid = 0;
    repeat (3) @(posedge clk);
    $display("wire: %s", wire_s);
    $display("payload bits=%0d stuffed bits=%0d stuff errors=%0d", N*8, stuffed, errs);
    $write("decoded bytes:");
    for (int b = 0; b < rx_n; b++) $write(" %02h", rx_bytes[b]);
    $display("");
    $finish;
end
endmodule

```

tb_usb_line.sv - sends SYNC, DATA0 PID and bytes with long runs of 1s through the encoder into the decoder.

```

wire: KJKJKJKKKKJKJKKKKKKKJJJJJJKKKKKKKJJJJJJJKJKJKJKJKJK
payload bits=48 stuffed bits=3 stuff errors=0
decoded bytes: 80 c3 ff 7f 3f 00

```

Real vvp output. The SYNC byte appears as the expected KJKJKJKK, three stuff bits were inserted (after the 0xFF, and in 0x7F/0x3F runs), and the decoder returns the original bytes.

The same stream was run through an independent Python model; its wire string is identical, character for character, to the RTL's:

```

def stuff_nrzi(data):
    bits = [(b >> i) & 1 for b in data for i in range(8)] # LSB first
    out, ones, level = [], 0, 1 # idle J (=1)
    for bit in bits:
        if bit: ones += 1
        else: ones, level = 0, level ^ 1
        out.append(level)
        if ones == 6: # stuff a 0
            ones, level = 0, level ^ 1
            out.append(level)
    return "".join("J" if v else "K" for v in out), len(out) - len(bits)

def crc5(value, nbits=11): # token: 7-bit addr + 4-bit endp, LSB first
    crc = 0x1F
    for i in range(nbits):
        fb = ((value >> i) & 1) ^ (crc >> 4)
        crc = ((crc << 1) & 0x1F) ^ (0x05 if fb else 0)
    return crc ^ 0x1F # complemented; sent MSB (c4) first

```

```
def crc16(data):
    crc = 0xFFFF
    for b in data:
        for i in range(8):
            fb = ((b >> i) & 1) ^ (crc >> 15)
            crc = ((crc << 1) & 0xFFFF) ^ (0x8005 if fb else 0)
    return crc ^ 0xFFFF

w, n = stuff_nrzi([0x80, 0xC3, 0xFF, 0x7F, 0x3F, 0x00])
print("wire:", w, " stuffed:", n)
print("crc5(SETUP addr 0 ep 0) = %02x" % crc5(0))
print("crc16(GET_DESCRIPTOR) = %04x" % crc16([0x80, 6, 0, 1, 0, 0, 0x40, 0]))
```

ref_usb.py - Python golden model for NRZI/bit stuffing and the two USB CRCs.

```
wire: KJKJKJKKKKJKJKKKKKKKJJJJJJJJKKKKKKJJJJJJJKJKJKJKJKJK stuffed: 3
crc5(SETUP addr 0 ep 0) = 08
crc16(GET_DESCRIPTOR) = bb29
```

Real Python output. The wire string matches the vvp run exactly; the CRC values are used in the next section.

PITFALL: WHERE STUFFING ENDS

Bit stuffing applies to everything from SYNC to the end of the CRC, including the CRC bits. A classic bug is to stop counting ones at a byte boundary, or to reset the ones counter at the start of the CRC field. Another is forgetting that a stuff bit is required even when the next data bit would have been 0 anyway - look at the 0x3F byte above. On receive, the ones counter must survive across bytes, and the EOP detector must run before the stuff-error detector so that a normal HS EOP is not logged as an error.

17.4 Packets: PIDs, tokens, data, handshakes and CRCs

Every USB 2.0 packet starts with SYNC and a **PID** byte: a 4-bit packet identifier followed by its one's complement, so a single-bit error in the PID is always detected. PIDs fall into four groups.

Group	PID name	PID[3:0]	Byte on bus	Use
Token	OUT	0001	E1	host -> device data follows
	IN	1001	69	device -> host data requested
	SOF	0101	A5	start of (micro)frame + 11-bit frame number
	SETUP	1101	2D	control transfer setup stage
Data	DATA0 / DATA1	0011 / 1011	C3 / 4B	data with toggle bit
	DATA2 / MDATA	0111 / 1111	87 / 0F	HS high-bandwidth isochronous
Handshake	ACK	0010	D2	received without error
	NAK	1010	5A	not ready now - retry later
	STALL	1110	1E	endpoint halted / request unsupported
	NYET	0110	96	HS: accepted, but not ready for next
Special	PRE / ERR	1100	3C	LS preamble (FS hub) / split error
	SPLIT	1000	78	HS split transaction to a TT
	PING	0100	B4	HS: flow-control probe for OUT/control

USB 2.0 PIDs. Byte on bus = {~PID, PID}, sent LSB first.

Token	SYNC	PID(8)	ADDR(7)	ENDP(4)	CRC5	EOP
SOF	SYNC	PID(8)	Frame number(11)		CRC5	EOP
Data	SYNC	PID(8)	DATA (0..1024 bytes)		CRC16	EOP
Handsh.	SYNC	PID(8)	EOP			

A bulk IN transaction: host: IN token dev: DATA0/1 host: ACK

A bulk OUT transaction: host: OUT token host: DATA0/1 dev: ACK / NAK / STALL / NYET

USB 2.0 packet formats. Every field is transmitted LSB first; CRCs are transmitted MSB first.

CRC5 (generator $x^5 + x^2 + 1$) protects the 11 bits of a token; **CRC16** ($x^{16} + x^{15} + x^2 + 1$) protects the data field. Both registers are seeded with all ones, the complement of the remainder is transmitted, and a receiver that runs the same LFSR over data and CRC finds a fixed **residual**: 01100 for CRC5 and 0x800D for CRC16. The serial implementation is a few flops:

```
// Bit-serial USB CRC5 (tokens) and CRC16 (data), fed with the bits in wire order
// (LSB of every byte first). Seed all ones; the transmitter sends the complement,
// MSB first. A receiver that runs the CRC across data + CRC checks for the residual.
module usb_crc (
    input logic      clk, init, en, din,
    output logic [4:0] crc5, output logic [15:0] crc16,    // raw LFSR state
    output logic [4:0] crc5_tx, output logic [15:0] crc16_tx,
    output logic      crc5_ok, crc16_ok
);
    wire fb5 = din ^ crc5[4];
    wire fb16 = din ^ crc16[15];
    always_ff @(posedge clk)
        if (init) begin crc5 <= '1; crc16 <= '1; end
        else if (en) begin
            crc5 <= {crc5[3:0], 1'b0} ^ (fb5 ? 5'h05 : 5'h00);    // x^5 + x^2 + 1
            crc16 <= {crc16[14:0], 1'b0} ^ (fb16 ? 16'h8005 : 16'h0000); // x^16+x^15+x^2+1
        end
    assign crc5_tx = ~crc5;
    assign crc16_tx = ~crc16;
    assign crc5_ok = (crc5 == 5'b01100);    // residual after a good token
    assign crc16_ok = (crc16 == 16'h800D);    // residual after a good data packet
endmodule
```

usb_crc.sv - serial CRC5 and CRC16 with receiver residual checks.

```
module tb;
    logic clk = 0, init, en, din;
    logic [4:0] crc5, crc5_tx; logic [15:0] crc16, crc16_tx; logic crc5_ok, crc16_ok;
    usb_crc dut (.*);
    always #5 clk = ~clk;
    task automatic send(input logic [63:0] v, input int n);    // n bits, LSB first
        for (int i = 0; i < n; i++) begin
            @(negedge clk); en = 1; din = v[i];
        end
        @(negedge clk); en = 0;
    endtask
    task automatic reset_crc; @(negedge clk); init = 1; @(negedge clk); init = 0; endtask
    function automatic logic [15:0] rev(input logic [15:0] v, input int n);
        rev = 0; for (int i = 0; i < n; i++) rev[i] = v[n-1-i];
    endfunction
    logic [54:0] toks = {11'h001, 4'h4, 7'h70, 4'hA, 7'h3A, 4'hE, 7'h15, 11'h000};
    logic [10:0] tok;
    logic [63:0] setup = 64'h0040_0000_0100_0680;    // bytes 80 06 00 01 00 00 40 00
    logic [15:0] field;
    initial begin
        init = 0; en = 0; din = 0;
        for (int k = 0; k < 5; k++) begin
```



```

    tok = toks[11*k +: 11];
    reset_crc(); send(tok, 11);
    field = {rev(crc5_tx, 5), tok}; // CRC5 goes out MSB first
    $write("token addr=%02h endp=%h  crc5=%02h  bytes after PID: %02h %02h",
           tok[6:0], tok[10:7], crc5_tx, field[7:0], field[15:8]);
    send(rev(crc5_tx, 5), 5); // receiver view: data + crc
    $display("  rx residual ok=%0d", crc5_ok);
end
reset_crc(); send(setup, 64);
field = rev(crc16_tx, 16);
$write("DATA0 GET_DESCRIPTOR  crc16=%04h  CRC bytes on the wire: %02h %02h",
       crc16_tx, field[7:0], field[15:8]);
send(field, 16);
$display("  rx residual ok=%0d", crc16_ok);
reset_crc(); send(setup ^ 64'h1_0000, 64); send(field, 16); // one data bit flipped
$display("same packet, one bit flipped:  rx residual ok=%0d", crc16_ok);
$finish;
end
endmodule

```

tb_usb_crc.sv - five tokens and the standard GET_DESCRIPTOR(Device) setup packet, plus a corrupted copy.

```

token addr=00 endp=0  crc5=08  bytes after PID: 00 10  rx residual ok=1
token addr=15 endp=e  crc5=17  bytes after PID: 15 ef  rx residual ok=1
token addr=3a endp=a  crc5=1c  bytes after PID: 3a 3d  rx residual ok=1
token addr=70 endp=4  crc5=0e  bytes after PID: 70 72  rx residual ok=1
token addr=01 endp=0  crc5=17  bytes after PID: 01 e8  rx residual ok=1
DATA0 GET_DESCRIPTOR  crc16=bb29  CRC bytes on the wire: dd 94  rx residual ok=1
same packet, one bit flipped:  rx residual ok=0

```

Real vvp output. The SETUP token to address 0, endpoint 0 is the well-known byte sequence 2D 00 10, and the 8-byte GET_DESCRIPTOR setup data carries the CRC bytes DD 94 seen on every USB analyzer trace. The Python model above gives the same values (08 and bb29).

WHY THE CRC 'LOOKS BACKWARDS' IN A TRACE

The data bits go out LSB first but the CRC goes out MSB first (c4 first for CRC5). A protocol analyzer that shows the packet as LSB-first bytes therefore displays the bit-reversed CRC: CRC5 08 (00010 reversed is 01000) shows up as the top five bits of the second address byte, 0x10. Getting this bit order wrong is the most common reason a first-silicon SIE sees every token as corrupt.

17.5 Transfer types and bandwidth

A **transfer** is made of **transactions**, and a transaction is token + data + handshake. Four transfer types exist; the endpoint descriptor fixes the type of each endpoint.

Type	Guarantees	Max packet (FS / HS / SS)	Typical use
Control	delivery, retries; 10-20% of bus reserved	8-64 / 64 / 512	enumeration, configuration, class requests
Bulk	delivery, no latency guarantee; uses leftover bandwidth	8-64 / 512 / 1024 (bursts up to 16)	mass storage, printers, network
Interrupt	bounded polling interval, retries	64 / 1024 x up to 3 per uframe / 1024 x up to 3	HID (keyboards, mice), status
Isochronous	reserved bandwidth, bounded latency, NO retry	1023 / 1024 x up to 3 per uframe / 1024 x 16 x 3	audio, video, webcams

Time is divided into 1 ms **frames** at FS (each starting with an SOF token carrying an 11-bit frame number) and 125 us **microframes** at HS (eight per frame, same frame number repeated). The host may reserve at most **90%** of an FS frame or **80%** of an HS microframe for periodic (isochronous + interrupt) transfers; the rest is shared by control and bulk. The best-case HS bulk rate is 13 packets of 512 bytes per microframe:

13 x 512 B / 125 us = 53.2 MB/s (theoretical HS bulk ceiling)

typical achieved: ~35-45 MB/s (host scheduling, NAKs, software)

Why a 480 Mb/s port rarely exceeds ~40 MB/s in practice.

Reliable transfer types use the **data toggle**: the sender alternates DATA0 and DATA1, and the receiver only advances its expected toggle when it accepts a packet. If an ACK is lost, the sender retransmits with the old toggle, the receiver sees a repeat, ACKs it again and discards the duplicate. The host retries a failed transaction up to three times ('three strikes') before reporting an error to software.

17.6 Control transfers, enumeration and descriptors

SETUP stage: SETUP token DATA0 (8-byte setup packet) ACK
 DATA stage: IN/OUT token DATA1, DATA0, DATA1 ... ACK (optional)
 STATUS stage: opposite-direction token, zero-length DATA1, ACK

setup packet: bmRequestType(1) bRequest(1) wValue(2) wIndex(2) wLength(2)
 e.g. 80 06 00 01 00 00 40 00 = device-to-host, standard, device;
 GET_DESCRIPTOR (06), type DEVICE (01) index 0, length 64

The three stages of a control transfer. The status stage always uses DATA1 and runs in the opposite direction to the data stage.

When a device is plugged in, the host **enumerates** it. A typical sequence (details vary by operating system) is:

- 1 Hub reports a connect (pull-up seen); host waits for debounce (about 100 ms) and issues a **bus reset** (SE0 >= 10 ms), during which the HS chirp happens.
- 2 Host sends GET_DESCRIPTOR(Device) to **address 0** to learn **bMaxPacketSize0** (8, 16, 32 or 64 at FS; 64 at HS; 512 at SS, encoded as 2⁹).
- 3 Host sends **SET_ADDRESS** (bRequest 05). The device must switch to the new address only *after* the status stage completes, and has 2 ms of recovery time.
- 4 Host reads the full 18-byte device descriptor, then the 9-byte configuration descriptor, then the whole configuration hierarchy (**wTotalLength** bytes), and string descriptors.
- 5 The driver is chosen from VID/PID or class codes; it sends **SET_CONFIGURATION** (09) and class-specific requests. Only now may the device draw its configured current (up to 500 mA at USB 2.0, 900 mA at USB 3.x; unconfigured it is limited to 100 mA / 150 mA).

Descriptor	Type	Length	Key fields
Device	0x01	18	bcdUSB, class, bMaxPacketSize0, idVendor, idProduct, bNumConfigurations
Configuration	0x02	9	wTotalLength, bNumInterfaces, bmAttributes (self-powered, remote wakeup), bMaxPower
String	0x03	var.	UTF-16LE text; index 0 lists language IDs
Interface	0x04	9	bInterfaceNumber, bAlternateSetting, class/subclass/protocol, bNumEndpoints
Endpoint	0x05	7	bEndpointAddress (bit 7 = IN), bmAttributes (type), wMaxPacketSize, bInterval
Device qualifier	0x06	10	how the device would look at the other speed
Interface association	0x0B	8	groups interfaces of one function (e.g. UVC, CDC)
BOS	0x0F	var.	USB 2.0 LPM / SuperSpeed / USB4 capabilities
SS endpoint companion	0x30	6	bMaxBurst, streams / Mult, wBytesPerInterval

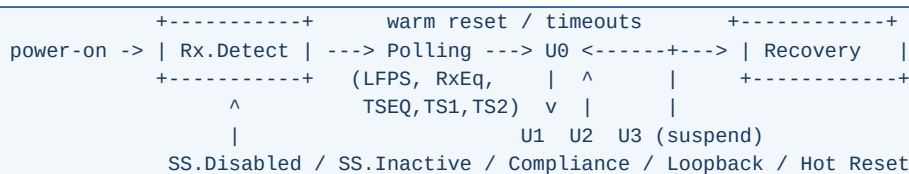
INTERVIEW INSIGHT: ENDPOINTS ARE THE UNIT OF HARDWARE

An **endpoint** is a buffer plus state (type, max packet size, toggle, halt bit) inside the device controller, addressed by number and direction. Interfaces and configurations are software groupings described in descriptors. When sizing a device controller, count endpoints and their FIFO depth; when writing firmware, count interfaces and alternate settings.

17.7 USB 3.x SuperSpeed

SuperSpeed is effectively a different protocol that happens to share the connector and the device framework (descriptors, requests, transfer types). It adds two differential pairs (SSTX+/- and SSRX+/-) for **dual-simplex** full-duplex operation, while the USB 2.0 pair stays alive in parallel: every SuperSpeed device must also work at USB 2.0.

- **Physical layer:** 5 GT/s with **8b/10b** (Gen 1) or 10 GT/s with **128b/132b** (Gen 2), scrambled (Gen 1 uses $x^{16} + x^5 + x^4 + x^3 + 1$), AC-coupled, spread-spectrum clocking allowed, receiver equalization. **LFPS** (low-frequency periodic signalling, bursts in the tens of MHz) is used out of band for polling, U1/U2 exit and warm reset because the high-speed receivers may be off.
- **Link layer:** a header packet is 12 bytes of header, a CRC-16 and a 2-byte link control word with its own CRC-5 (16 bytes, after a 4-symbol start-of-packet framing); data payloads carry a CRC-32. Reliable delivery uses header sequence numbers and **link commands**: LGOOD_n (ACK of header n), LBAD (request retry), LRTY (retry follows), LCRD_A..D (return of one of four header buffer credits), LGO_U1/U2/U3, LAU/LXU (accept/reject), LUP/LDN (keep-alive).
- **Protocol layer:** transaction packets (ACK, NRDY, ERDY, STATUS, PING), data packets, ITP (isochronous timestamp) and LMPs. Packets are **unicast** with a 20-bit route string instead of broadcast; bulk endpoints may use **bursts** (up to 16 packets without waiting for each ACK) and **streams** (used by UASP storage).



Simplified USB 3.x LTSSM. U0 is active; U1 and U2 are link low-power states with progressively longer exit latency; U3 is suspend.

The **link training and status state machine** (LTSSM) - the same idea as PCIe's, Chapter 18 - detects a far-end receiver by measuring the line's charge time (Rx.Detect), exchanges LFPS, trains the receiver equalizer with a TSEQ pattern and exchanges TS1/TS2 ordered sets in Polling, then enters **U0**. Idle links drop into U1/U2 under timer control (negotiated with LGO_Ux / LAU / LXU) and into U3 when the host suspends the device.

17.8 USB Type-C and USB Power Delivery

The **Type-C** connector (24 pins, reversible) is defined by its own specification. Its **CC1/CC2** configuration-channel pins do three jobs: detect attach, detect the plug **orientation** (which of CC1 or CC2 sees the sink's pull-down), and advertise current. The source presents a pull-up **Rp** whose value signals Default USB power, 1.5 A or 3.0 A at 5 V; the sink presents a 5.1 kohm pull-down **Rd**; an electronically marked cable presents **Ra** and draws **VCONN** on the unused CC pin. There are two SuperSpeed lane pairs (TX1/RX1, TX2/RX2), so an orientation mux (or a two-lane USB 3.2 Gen 2x2 / USB4 PHY) is needed behind the connector.

- **USB Power Delivery (PD)** runs a half-duplex packet protocol on the CC wire: **BMC** (biphase mark) coding at about 300 kb/s, 4b5b symbols, CRC-32, GoodCRC acknowledgements. Source and sink negotiate a contract (Source_Capabilities -> Request -> Accept -> PS_RDY). Standard Power Range reaches 100 W (20 V, 5 A with a 5 A cable); PD 3.1 **Extended Power Range** reaches 240 W (48 V, 5 A). **PPS** allows fine-grained programmable voltage for direct battery charging.

- **Alternate modes** reuse the high-speed pairs for another protocol after a PD structured-VDM exchange (Discover Identity, Discover SVIDs, Discover Modes, Enter Mode). **DisplayPort Alt Mode** (Chapter 21) is the common one; Thunderbolt 3 was another.
- SoC split: the CC logic and PD PHY usually live in a small **Type-C port controller** (TCPC, separate chip or integrated), with the policy engine in firmware (TCPM) talking over I2C (Chapter 13); the SoC provides the orientation mux control and role (host/device, DRP) switching.

PITFALL: TYPE-C INTEGRATION BUGS

Common field failures: the orientation mux driven from a stale CC reading; dead-battery mode not supported (the sink's Rd must be present with no power); VBUS applied before the contract; USB 2.0 D+/D- stubs from the two connector rows left unshorted or routed as a long stub; data-role swap not propagated to the dual-role controller. These are system bugs that span PHY, board, firmware and RTL - put them in the integration test plan explicitly.

17.9 USB4 and Thunderbolt tunnelling

USB4 is based on the Thunderbolt 3 protocol contributed by Intel. A USB4 port is a **router**: the link runs on both Type-C lane pairs (two lanes, 10 or 20 Gb/s each in USB4 v1; 40 Gb/s per lane with PAM3 in USB4 v2), and packets of different protocols are **tunnelled** through it. Adapters at each router convert native protocol traffic into tunnelled packets:

host router	USB4 link (2 lanes)	device router
+-----+ USB3 adapter <-- xHCI PCIe adapter <-- RC port DP IN adapter <-- display host interface (DMA) +-----+	===== transport layer: paths, hop IDs, credits, priority =====	+-----+ USB3 adapter -> hub PCIe adapter -> sw. DP OUT -> monitor host iface (P2P net) +-----+
+ a native USB 2.0 link always runs in parallel on D+/D-		

USB4 multiplexes tunnelled USB 3.x, PCIe and DisplayPort traffic over one link, with bandwidth managed by a connection manager in software.

The USB4 **transport layer** carries fixed-size transport-layer packets with a hop ID that selects a **path** through the routers, per-path credit-based flow control and scheduling priorities. A **connection manager** (software in the host OS, or firmware) discovers routers, allocates bandwidth - for example reserving isochronous DisplayPort bandwidth before PCIe gets the rest - and sets up the paths. For an SoC this means a USB4 subsystem contains a USB4 router, an xHCI controller, a PCIe root port and a DisplayPort source, all behind one PHY.

17.10 Controller architectures

Host controllers: xHCI

The **eXtensible Host Controller Interface** (xHCI, from Intel) replaced the older OHCI/UHCI (USB 1.1) and EHCI (USB 2.0) register models and handles every speed with one controller. It is a descriptor-ring DMA engine: software writes **Transfer Request Blocks** (TRBs, 16 bytes each) into per-endpoint **transfer rings** in system memory and rings a **doorbell** register; the controller fetches TRBs, schedules transactions on the bus and writes completion TRBs to an **event ring**, raising an MSI/MSI-X or wired interrupt through one of its interrupters. A **device context** per attached device (slot context + up to 31 endpoint contexts) lives in memory and is pointed to by the Device Context Base Address Array.

CPU / driver	system memory	xHCI (AXI master)
write TRBs ----->	transfer ring <---- fetch ----	scheduler / DMA
ring doorbell ----->	event ring <----- write -----	event generator
interrupt <-----		interrupter

The xHCI programming model: rings in memory, doorbells in MMIO.

Device controllers and dual role

A **device controller** contains the serial interface engine (SIE: PID decode, CRC, toggle, handshake generation), per-endpoint state, endpoint FIFOs (often a shared RAM partitioned in software), and a DMA engine with its own descriptor format. Phones and embedded SoCs almost always use a **dual-role** controller (historically USB OTG with the ID pin, today Type-C DRP) that is an xHCI host in one mode and a device in the other. Licensed IP from a few vendors dominates; in-house designs are rare because of the compliance effort.

Team	Owns
PHY / analog	HS transceiver, chirp, squelch/disconnect detectors, SuperSpeed SerDes, LFPS, Rx.Detect, eye compliance
RTL / integration	controller configuration, AXI and interrupt hookup, UTM/PIPE wiring, clocks/resets/power domains, Type-C mux control
DV	USB VIP (host and device models), protocol checkers, enumeration and power-state tests, xHCI ring tests
Firmware / software	xHCI and gadget drivers, class drivers, PD policy engine, USB4 connection manager
Validation / compliance	USB-IF electrical, link-layer and USBCV tests, interoperability plugfests

17.11 PHY interfaces: UTM, UTM+, ULPI and PIPE

USB PHYs are almost always bought or reused as hard macros, so the controller-to-PHY interface is standardized:

Interface	Width / clock	Key signals	Notes
UTMI	8 bit @ 60 MHz or 16 bit @ 30 MHz	DataIn/DataOut, TxValid, TxReady, RxValid, RxActive, RxError, XcvtSelect, TermSelect, OpMode, LineState[1:0], SuspendM	USB 2.0 device PHY: PHY does SYNC/EOP, NRZI, bit stuffing, serialization and clock recovery
UTMI+	same	adds DpPulldown/DmPulldown, VBUS valid/session, ID, host-side signals	levels 0-3 add host and OTG support
ULPI	8 data + CLK, DIR, STP, NXT (12 pins), 60 MHz	register reads/writes and packets over one bus	low pin count for an external PHY chip
PIPE	8/16/32/(64)-bit per lane, PCLK	TxData/TxDataK, RxData/RxDataK, RxValid, RxStatus, PowerDown, Rate, PhyStatus, TxDetectRx, TxElecIdle, RxPolarity	shared with PCIe and SATA (Chapter 18); PIPE 5+ adds a low-pin-count message bus and a thinner SerDes architecture
eUSB2	1.2 V signalling repeater	eD+/eD-	lets advanced-node SoCs avoid 3.3 V I/O; a repeater chip restores USB 2.0 levels

EXPERT NOTE: THE 3.3 V PROBLEM

FS/LS signalling needs about 3.3 V swing and tolerance to 5 V shorts on the connector, which is increasingly expensive in 5 nm-class processes whose thick-oxide devices top out near 1.8 V. That is why eUSB2 and repeaters exist, and why many SoCs place the USB 2.0 PHY on a companion chip or use special I/O devices. Plan the process option early.

17.12 Verification and bring-up

- Use a commercial USB VIP for the host and device side; write your own scoreboard for the SoC data path (DMA to memory).
- Test **reset during every state** (mid-enumeration, mid-transfer, during U1/U2 entry), unplug during DMA, and suspend/resume with remote wakeup - these are where controllers hang.

- Check toggle recovery (drop an ACK), NAK storms, STALL and clear-halt, and zero-length packet termination of transfers that are an exact multiple of the max packet size.
- In emulation/FPGA, boot the real Linux xHCI driver early: most integration bugs (cache coherency of rings, interrupt routing, 64-bit addressing) only appear with real software.

17.13 Summary

- USB 2.0 is a host-pollled, half-duplex bus: NRZI with bit stuffing after six 1s, SYNC/PID/data/CRC/EOP packets, CRC5 on tokens and CRC16 on data (both seeded with ones, complemented, sent MSB first).
- FS/LS speed is signalled by a 1.5 kohm pull-up; HS is negotiated by the chirp during reset and uses 400 mV current-mode signalling.
- Four transfer types: control (enumeration), bulk (throughput), interrupt (bounded polling), isochronous (reserved bandwidth, no retry); the data toggle gives exactly-once delivery.
- USB 3.x adds a full-duplex SerDes link with 8b/10b or 128b/132b, an LTSSM, link-level retry with credits, and U1/U2/U3 power states.
- Type-C adds CC-based orientation/current detection and PD negotiation; USB4 tunnels USB3, PCIe and DisplayPort over one link.
- Hosts use xHCI (TRB rings + doorbells); device controllers use endpoint FIFOs + DMA; PHYs attach through UTMI/ULPI (USB 2.0) and PIPE (SuperSpeed).

17.14 Exercises

- 1 Hand-encode the SOF packet for frame number 0x2A5: give the 11-bit field, the CRC5 (use `ref_usb.py`), and the NRZI line states from SYNC to EOP. How many stuff bits are inserted?
- 2 Extend `usb_nrzi_rx` with an FS EOP detector (SE0 for 2 bit times followed by J) and a SYNC detector that aligns the byte counter. Add them to the testbench.
- 3 A full-speed isochronous audio endpoint carries 48 kHz, 24-bit stereo. What `wMaxPacketSize` does it need, and what fraction of the FS frame does it reserve?
- 4 Explain step by step what happens if the ACK for a bulk OUT DATA1 packet is corrupted. Which side detects the duplicate and how?
- 5 Compute the maximum theoretical throughput of a USB 3.2 Gen 1 link after 8b/10b encoding, then estimate the protocol overhead per 1024-byte data packet (header packet, CRC-32, framing). Compare with the ~400-450 MB/s seen by real storage devices.
- 6 List the signals you would need to add to a UTMI-based device controller to make it work as a host (UTMI+ level 2).

18. PCI Express and CXL

PCI Express (PCIe) is the backbone of every PC, server, GPU, NIC and SSD, and the physical basis of CXL. It kept the software model of the old parallel PCI bus - memory, I/O and configuration address spaces, Bus/Device/Function numbering, BARs and capability lists - so that a 2025 operating system still enumerates a Gen6 device with code written for PCI. Underneath, the hardware is a packet-switched, point-to-point, credit-flow-controlled serial network with a three-layer stack. An SoC engineer meets PCIe as a **root port** (the SoC is the host), as an **endpoint** (the SoC is an accelerator or SSD controller on a card), or as both.

18.1 Generations, encoding and bandwidth

Gen	Year	Rate / lane	Encoding	Per lane, per direction	x16, per direction
1.x	2003	2.5 GT/s NRZ	8b/10b	250 MB/s	4 GB/s
2.0	2007	5.0 GT/s NRZ	8b/10b	500 MB/s	8 GB/s
3.0	2010	8.0 GT/s NRZ	128b/130b	~985 MB/s	~15.8 GB/s
4.0	2017	16 GT/s NRZ	128b/130b	~1.97 GB/s	~31.5 GB/s
5.0	2019	32 GT/s NRZ	128b/130b	~3.94 GB/s	~63 GB/s
6.0	2022	64 GT/s PAM4	1b/1b, 256 B FLIT + FEC + CRC	~7.5 GB/s	~121 GB/s
7.0	2025	128 GT/s PAM4	FLIT (as 6.0)	~15 GB/s	~242 GB/s

PCIe generations. Gen6/7 figures use the 242/256 FLIT payload efficiency; TLP/DLLP overhead reduces all of these further.

Each doubling was achieved differently. Gen3 kept NRZ but dropped the 25% overhead of 8b/10b for 128b/130b with scrambling, so 8 GT/s gives nearly double Gen2's bandwidth. Gen6 kept the Nyquist frequency of Gen5 (16 GHz) but sends two bits per symbol with **PAM4**; the higher raw bit-error rate (on the order of 1e-6 before correction) forced a lightweight **FEC** and a fixed 256-byte **FLIT** (flow-control unit) containing 236 bytes of TLP, 6 bytes of DLLP, 8 bytes of CRC and 6 bytes of FEC. A link is 1, 2, 4, 8 or 16 lanes wide in practice (x12 and x32 appear in older revisions but are essentially unused), each lane a TX and an RX differential pair.

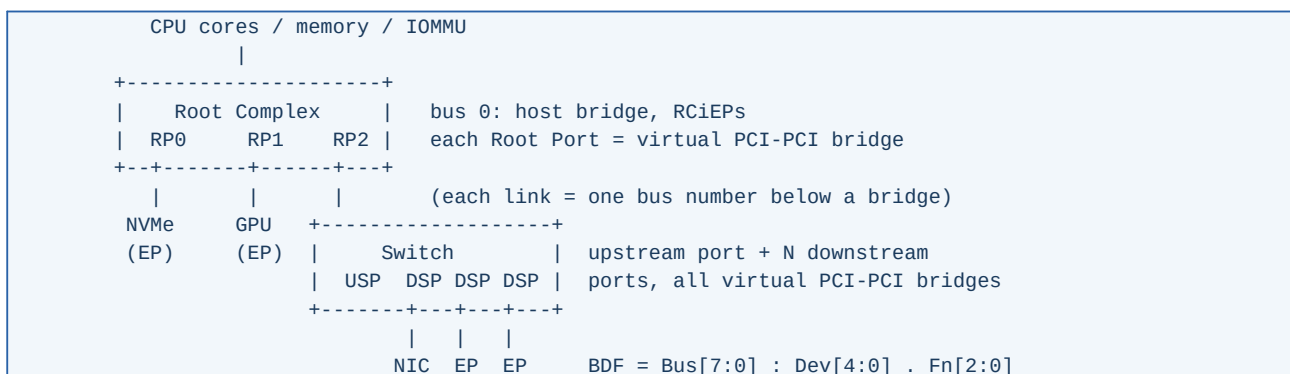
BW(Gen3+) per lane = rate x 128/130 / 8 e.g. 16 GT/s -> 1.969 GB/s

BW(Gen6) per lane = 64 GT/s x 242/256 / 8 = 7.56 GB/s

x16 Gen5 bidirectional = 2 x 63 GB/s = 126 GB/s (before TLP overhead)

Raw per-direction bandwidth. Real DMA throughput is typically 80-90% of this with 256-byte payloads.

18.2 Topology: root complex, switches, endpoints and BDF



PCIe topology. Software sees a tree of PCI-PCI bridges and functions; hardware sees point-to-point links.

- The **Root Complex** connects the CPU/memory system to the PCIe hierarchy. Each **Root Port** is its own hierarchy domain root.
- A **switch** has one upstream port and several downstream ports; internally it looks to software like a set of PCI-PCI bridges on an internal bus. It routes TLPs by address, by ID or implicitly.
- An **endpoint** has up to 8 functions (up to 256 with **ARI**, which reuses the 5 device bits because a link has only device 0 below it). Each function is identified by **BDF** = Bus:Device.Function, which is also its **Requester ID**.
- Configuration space is reached through **ECAM**: a memory window of 4 KB per function, 1 MB per bus, 256 MB for 256 buses.

18.3 The layered stack

Transmit	Receive
+----- Transaction Layer -----+	
build TLP header (+ data, + optional ECRC)	check ECRC, route,
FC credit check (P / NP / Cpl) ordering, VCs	ordering, return credits
+----- Data Link Layer -----+	
+ 12-bit sequence number (2 B), + LCRC (4 B)	check LCRC + seq,
replay buffer, ACK/NAK, DLLPs (UpdateFC, PM, ...)	send ACK/NAK
+----- Physical Layer -----+	
framing (STP/SDP/END or tokens), byte striping,	deskew, descramble,
scrambling, 8b/10b or 128b/130b or FLIT+FEC, SerDes	decode, CDR, LTSSM
+-----	
TLP on the wire (Gen1-5):	
STP Seq# (2B) Header 12/16 B Data 0-4096 B ECRC 4B opt. LCRC 4B END	

Each layer adds its own fields. In Gen6 FLIT mode the sequence number and CRC move from per-TLP to per-FLIT.

18.4 Transaction layer

TLP types

TLP	Fmt	Type	Class	Notes
MRd (Memory Read)	000 / 001	0 0000	non-posted	3DW (32-bit address) / 4DW (64-bit) header; answered by CplD
MWrr (Memory Write)	010 / 011	0 0000	posted	no completion; the most common TLP (DMA, doorbells, MSI)
IORd / IOWr	000 / 010	0 0010	non-posted	legacy I/O space
CfgRd0 / CfgWr0	000 / 010	0 0100	non-posted	type 0: to a function on this bus
CfgRd1 / CfgWr1	000 / 010	0 0101	non-posted	type 1: forwarded by bridges, converted to type 0
Msg / MsgD	001 / 011	1 0rrr	posted	rrr = routing (to RC, by address, by ID, broadcast, local); INTx emulation, PM, errors, vendor
Cpl / CplD	000 / 010	0 1010	completion	status SC, UR, CA, CRS; carries read data
AtomicOp	010 / 011	0 1100-0 1110	non-posted	FetchAdd, Swap, CAS; returns CplD

Main TLP types (non-FLIT header encoding). Fmt[0] selects a 4DW header, Fmt[1] means the TLP carries data.

	byte 0	byte 1	byte 2	byte 3
DW0	Fmt[2:0] Type[4:0] T9 TC[2:0] T8 A2 LN TH	TD EP Attr[1:0] AT[1:0] L[9:8]	L[7:0]	
DW1	Requester ID (BDF, 16 bits)	Tag (8)	LastBE 1stBE	
DW2	Address[31:2] (Address[63:32] in a 4DW header)		PH[1:0]	
DW3	Address[31:2] (4DW header only)		PH[1:0]	
Completion:	DW1 = Completer ID Status(3) BCM Byte Count(12)			
	DW2 = Requester ID Tag R Lower Address(7)			
Config:	DW2 = Bus(8) Dev(5) Fn(3) Rsvd ExtReg(4) Reg(6) R			

TLP header layouts (bytes transmitted left to right, DW0 first). Length is in DWs, 0 meaning 1024; TD marks an ECRC; EP marks poisoned data.

Building these headers is the first job of a transmit-side transaction layer. The subtle part is converting a byte address and byte count into a DW length and first/last byte enables, and choosing a 3DW or 4DW header (a 4DW header must not be used for an address below 4 GB). The package below does exactly that:

```
// PCIe TLP header builder (non-flit mode, as used through Gen5).
// A header is returned as 4 DWs packed {DW0, DW1, DW2, DW3}; DW0 goes first on the
// link, most-significant byte first. Fmt[0] = 1 marks a 4DW header.
package tlp_pkg;
    typedef logic [127:0] hdr_t;
    // byte0 = {Fmt, Type} byte1 = {T9, TC, T8, Attr2, LN, TH} byte2 = {TD, EP, Attr, AT, Len[9:8]}
    function automatic logic [31:0] dw0(input logic [2:0] fmt, input logic [4:0] typ,
        input logic [2:0] tc, input logic [9:0] len);
        return {fmt, typ, 1'b0, tc, 4'b0000, 1'b0, 1'b0, 2'b00, 2'b00, len};
    endfunction

    // MRd / MWr from a byte address and byte count: DW length and byte enables.
    // 3DW header below 4 GB, 4DW above (a 4DW header below 4 GB is not allowed).
    function automatic hdr_t mem_req(input bit wr, input logic [63:0] addr,
        input logic [12:0] nbytes, input logic [15:0] rid, input logic [7:0] tag);
        logic [63:0] last; logic [9:0] len; logic [3:0] fbe, lbe; bit four;
        last = addr + nbytes - 1;
        len = 10'(last[63:2] - addr[63:2] + 1); // DWs touched (1024 encodes as 0)
        fbe = 4'b1111 << addr[1:0];
        lbe = 4'b1111 >> (2'd3 - last[1:0]);
        four = (addr[63:32] != 0);
        if (len == 1) begin fbe = fbe & lbe; lbe = 4'b0000; end // single-DW request
        return {dw0({1'b0, wr, four}, 5'b00000, 3'd0, len), rid, tag, lbe, fbe,
            four ? {addr[63:32], addr[31:2], 2'b00} : {addr[31:2], 2'b00, 32'h0}};
    endfunction

    // Type 0 configuration read (CfgRd0: Fmt 000, Type 00100), always 1 DW
    function automatic hdr_t cfg_rd0(input logic [7:0] bus, input logic [4:0] dev,
        input logic [2:0] fn, input logic [11:0] reg_byte, input logic [15:0] rid,
        input logic [7:0] tag);
        return {dw0(3'b000, 5'b00100, 3'd0, 10'd1), rid, tag, 4'b0000, 4'b1111,
            bus, dev, fn, 4'b0000, reg_byte[11:2], 2'b00, 32'h0};
    endfunction

    // Completion with data (CplD: Fmt 010, Type 01010), status SC = 000, BCM = 0
    function automatic hdr_t cpld(input logic [15:0] cid, input logic [15:0] rid,
        input logic [7:0] tag, input logic [11:0] byte_cnt, input logic [6:0] lower_addr,
        input logic [9:0] len);
        return {dw0(3'b010, 5'b01010, 3'd0, len), cid, 3'b000, 1'b0, byte_cnt,
            rid, tag, 1'b0, lower_addr, 32'h0};
    endfunction
endpackage
```

tlp_pkg.sv - header builders for MRd/MWr, CfgRd0 and CplD.

```

module tb;
  import tlp_pkg::*;
  task automatic show(input string name, input hdr_t h);
    int ndw = h[125] ? 4 : 3; // Fmt[0]: 4DW header
    $write("%-32s", name);
    for (int i = 0; i < ndw; i++) $write(" %08h", h[127 - 32*i -: 32]);
    $display("");
  endtask
  initial begin
    show("MWr32 F0000104, 8 B, tag 05", mem_req(1, 64'hF000_0104, 8, 16'h0100, 8'h05));
    show("MRd32 80000002, 2 B, tag 06", mem_req(0, 64'h8000_0002, 2, 16'h0100, 8'h06));
    show("MRd64 1234567801, 256 B", mem_req(0, 64'h12_3456_7801, 256, 16'h0100, 8'h07));
    show("MRd32 00001000, 4096 B", mem_req(0, 64'h1000, 4096, 16'h0100, 8'h08));
    show("CfgRd0 01:00.0 reg 010 (BAR0)", cfg_rd0(8'h01, 5'd0, 3'd0, 12'h010, 16'h0, 8'h09));
    show("CplD 4 B for tag 09", cpld(16'h0100, 16'h0, 8'h09, 12'd4, 7'h10, 10'd1));
  end
endmodule

```

tb_tlp.sv - builds six headers and prints them as DWs.

```

MWr32 F0000104, 8 B, tag 05      40000002 010005ff f0000104
MRd32 80000002, 2 B, tag 06      00000001 0100060c 80000000
MRd64 1234567801, 256 B          20000041 0100071e 00000012 34567800
MRd32 00001000, 4096 B           00000000 010008ff 00001000
CfgRd0 01:00.0 reg 010 (BAR0)    04000001 0000090f 01000010
CplD 4 B for tag 09              4a000001 01000004 00000910

```

Real vvp output: DW0, DW1, DW2 (and DW3 for the 64-bit read).

```

def dw0(fmt, typ, tc, length):
    return (fmt << 29) | (typ << 24) | (tc << 20) | (length & 0x3FF)

def mem_req(wr, addr, nbytes, rid, tag):
    last = addr + nbytes - 1
    length = (last >> 2) - (addr >> 2) + 1
    fbe = (0xF << (addr & 3)) & 0xF
    lbe = 0xF >> (3 - (last & 3))
    if length == 1:
        fbe, lbe = fbe & lbe, 0
    four = addr >= 1 << 32
    h = [dw0((wr << 1) | four, 0, 0, length),
         (rid << 16) | (tag << 8) | (lbe << 4) | fbe]
    h += [addr >> 32, addr & ~3 & 0xFFFFFFFF] if four else [addr & ~3]
    return h

def cfg_rd0(bus, dev, fn, reg, rid, tag):
    return [dw0(0, 0b00100, 0, 1), (rid << 16) | (tag << 8) | 0xF,
           (bus << 24) | (dev << 19) | (fn << 16) | (reg & 0xFFC)]

def cpld(cid, rid, tag, bc, la, length):
    return [dw0(0b010, 0b01010, 0, length), (cid << 16) | bc, (rid << 16) | (tag << 8) | la]

rows = [mem_req(1, 0xF0000104, 8, 0x100, 5), mem_req(0, 0x80000002, 2, 0x100, 6),
        mem_req(0, 0x1234567801, 256, 0x100, 7), mem_req(0, 0x1000, 4096, 0x100, 8),
        cfg_rd0(1, 0, 0, 0x10, 0, 9), cpld(0x100, 0, 9, 4, 0x10, 1)]
for r in rows:
    print(" ".join("%08x" % d for d in r))

```

ref_tlp.py - an independent Python packer for the same six requests.

```

40000002 010005ff f0000104
00000001 0100060c 80000000
20000041 0100071e 00000012 34567800
00000000 010008ff 00001000
04000001 0000090f 01000010
4a000001 01000004 00000910

```

Real Python output - identical to the RTL, DW for DW (checked with diff).

Read the results: the 2-byte read at 0x80000002 is a single-DW request with First BE = 1100 and Last BE = 0000; the 256-byte read starting at byte 1 touches 65 DWs (0x41) with First BE 1110 and Last BE 0001; the 4096-byte read encodes Length as 0. The CfgRd0 targets bus 01, device 0, function 0, register 0x10 (BAR0).

Traffic classes, virtual channels and ordering

Each TLP carries a 3-bit **Traffic Class** (TC). A port maps TCs to **Virtual Channels** (VC0 is mandatory; most devices implement only VC0), each with its own buffers and credits so that one class cannot block another. Within a VC, TLPs are split into three flow-control classes with ordering rules designed for the producer-consumer model and deadlock freedom:

Row passes column?	Posted (MWr, Msg)	Non-posted (MRd, Cfg, IO)	Completion
Posted	No (Yes if RO / IDO)	Yes - must be able to	Yes (may)
Non-posted	No (Yes if RO / IDO)	Yes (may)	Yes (may)
Completion	No (Yes if RO / IDO)	Yes - must be able to	Yes, except same-tag completions of one request

Simplified ordering table. 'No' guarantees that a flag written after data is never seen before the data. Posted requests must be able to pass stalled non-posted ones to avoid deadlock.

Relaxed Ordering (Attr bit RO) allows a TLP to pass posted writes when software knows ordering is not needed (for example data buffers, but not the completion flag); **ID-Based Ordering** (IDO) relaxes ordering between different requesters; **No Snoop** tells the root complex that the access does not need a cache snoop.

Flow control credits

The receiver advertises buffer space in **credits** for six types: PH, PD, NPH, NPD, CplH, CplD (header and data credits for posted, non-posted and completion). One header credit holds one TLP header; one data credit is **16 bytes** (4 DW). Initial values are exchanged with **InitFC1/InitFC2** DLLPs during link initialization; an advertisement of 0 means infinite (endpoints normally advertise infinite completion credits because they only receive completions for reads they issued and sized for). As buffers drain, the receiver sends **UpdateFC** DLLPs with its new cumulative CREDITS_ALLOCATED.

```

TX: CREDIT_LIMIT (CL)    <- value from InitFC / UpdateFC

    CREDITS_CONSUMED (CC) += credits of every TLP sent

    may send TLP needing n if (CL - (CC + n)) mod 2^F <= 2^F / 2

    F = 8 for header, 12 for data credits (larger with scaled FC in Gen4+)

```

The transmitter gate. Both counters wrap; the modulo compare works as long as fewer than half the counter range is ever outstanding.

```

// PCIe flow-control gate for one credit type (e.g. Posted Header or Posted Data).
// Counters are modulo 2^F; F = 8 for header and 12 for data credits (no scaling).
// A TLP needing `need` credits may go if
// (CREDIT_LIMIT - (CREDITS_CONSUMED + need)) mod 2^F <= 2^F / 2
module fc_gate #(parameter int F = 8) (
    input logic      clk, rst_n,
    input logic      init_valid,    input logic [F-1:0] init_credits, // InitFC
    input logic      upd_valid,     input logic [F-1:0] upd_limit,    // UpdateFC
    input logic [F-1:0] need,

```

```

input logic      consume,                                // TLP sent
output logic     ok,
output logic [F-1:0] limit, consumed
);
logic [F-1:0] diff;
assign diff = limit - (consumed + need);
assign ok    = (diff <= F'(1 << (F-1)));
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n)        begin limit <= '0; consumed <= '0; end
    else begin
        if (init_valid)    limit <= init_credits;        // CL starts at the advertised value
        else if (upd_valid) limit <= upd_limit;          // receiver's new CREDITS_ALLOCATED
        if (consume)       consumed <= consumed + need;
    end
end
endmodule

```

fc.sv - one credit-type gate with modulo-2^F counters.

```

// Posted writes from a transmitter to a receiver with 4 header / 32 data credits of
// buffer (32 x 16 B = 512 B). The receiver drains one TLP every 12 cycles and returns
// the freed credits with an UpdateFC that arrives 8 cycles later.
module tb;
    localparam int NTLP = 300;
    logic clk = 0, rst_n = 0; always #5 clk = ~clk;
    logic init = 0, h_ok, d_ok, send; logic [7:0] h_lim, h_cc; logic [11:0] d_lim, d_cc;
    logic upd = 0; logic [7:0] h_alloc = 4, h_upd; logic [11:0] d_alloc = 32, d_upd;
    logic [11:0] dneed;
    fc_gate #(8) gh (.clk, .rst_n, .init_valid(init), .init_credits(8'd4),
        .upd_valid(upd), .upd_limit(h_upd), .need(8'd1), .consume(send),
        .ok(h_ok), .limit(h_lim), .consumed(h_cc));
    fc_gate #(12) gd (.clk, .rst_n, .init_valid(init), .init_credits(12'd32),
        .upd_valid(upd), .upd_limit(d_upd), .need(dneed), .consume(send),
        .ok(d_ok), .limit(d_lim), .consumed(d_cc));
    function automatic int size(input int i); // payload bytes, cycled
        return (i % 4 == 0) ? 64 : (i % 4 == 1) ? 256 : (i % 4 == 2) ? 128 : 512;
    endfunction
    int dsum = 0, k = 0, sent = 0, stall = 0, cyc = 0, drained = 0;
    int q_hdr = 0, q_dat = 0, max_hdr = 0, max_dat = 0; // receiver buffer occupancy
    int pend [$]; // data credits of queued TLPs
    int upd_at [$], d; // UpdateFC arrival times
    logic [19:0] upd_v [$]; // {hdr, data} it carries
    assign dneed = 12'((size(k) + 15) / 16); // 1 data credit = 16 bytes
    assign send = rst_n && !init && h_ok && d_ok && sent < NTLP;
    always @(posedge clk) if (rst_n && !init) begin
        cyc++;
        if (send) begin
            q_hdr++; q_dat += dneed; dsum += dneed; pend.push_back(dneed);
            if (sent < 6) $display("cyc %3d send MWr %3d B CL/CC hdr %3d/%3d data %4d/%4d",
                cyc, size(k), h_lim, h_cc + 8'd1, d_lim, d_cc + dneed);
            sent++; k++;
        end else if (sent < NTLP) stall++;
        max_hdr = (q_hdr > max_hdr) ? q_hdr : max_hdr;
        max_dat = (q_dat > max_dat) ? q_dat : max_dat;
        if (cyc % 12 == 0 && pend.size() > 0) begin // receiver frees one TLP
            d = pend.pop_front();
            q_hdr--; q_dat -= d; drained++;
            h_alloc = h_alloc + 8'd1; d_alloc = d_alloc + 12'(d);
            upd_at.push_back(cyc + 8); upd_v.push_back({h_alloc, d_alloc});
        end
        upd <= 1'b0;
        if (upd_at.size() > 0 && upd_at[0] == cyc) begin // DLLP arrives: CL updates

```

```

        void'(upd_at.pop_front());
        {h_upd, d_upd} <= upd_v.pop_front(); upd <= 1'b1;
    end
end
initial begin
    repeat (2) @(posedge clk); rst_n = 1;
    @(negedge clk) init = 1; @(negedge clk) init = 0;
    wait (sent == NTLP); repeat (2) @(posedge clk);
    $display("sent=%0d TLPs in %0d cycles, stalled %0d cycles for credits", sent, cyc, stall);
    $display("receiver peak occupancy: %0d/4 headers, %0d/32 data credits (never exceeded)",
        max_hdr, max_dat);
    $display("counters mod 2^F: hdr CL=%0d CC=%0d, data CL=%0d CC=%0d (%0d data credits used)",
        h_lim, h_cc, d_lim, d_cc, dsum);
    $finish;
end
endmodule

```

tb_fc.sv - 300 posted writes of 64-512 bytes into a receiver with 4 PH / 32 PD credits of buffer that drains slowly.

```

cyc  1  send MWr  64 B  CL/CC hdr  4/  1 data  32/  4
cyc  2  send MWr 256 B  CL/CC hdr  4/  2 data  32/ 20
cyc  3  send MWr 128 B  CL/CC hdr  4/  3 data  32/ 28
cyc 46  send MWr 512 B  CL/CC hdr  7/  4 data  60/ 60
cyc 58  send MWr  64 B  CL/CC hdr  8/  5 data  92/ 64
cyc 59  send MWr 256 B  CL/CC hdr  8/  6 data  92/ 80
sent=300 TLPs in 3600 cycles, stalled 3298 cycles for credits
receiver peak occupancy: 3/4 headers, 32/32 data credits (never exceeded)
counters mod 2^F: hdr CL=47 CC=44, data CL=404 CC=404 (4500 data credits used)

```

Real vvp output. The 512-byte write needs all 32 data credits and waits until the receiver has drained everything (cycle 46). Both counters wrapped (300 mod 256 = 44, 4500 mod 4096 = 404) and the receiver buffer was never overrun.

CREDITS SIZE THE RECEIVE BUFFER, AND VICE VERSA

Throughput is capped at (credits advertised) / (credit round-trip time). The round trip includes the TLP transmission, receiver pipeline, UpdateFC scheduling, DLLP transmission and TX pipeline - typically hundreds of ns to a few us. A Gen5 x16 port moving about 63 GB/s with a 1 us credit loop needs on the order of 64 KB of posted-data buffer to keep the link full. Undersized receive buffers are the number-one cause of 'PCIe link at full width but half speed' performance bugs.

18.5 Data link layer: sequence numbers, LCRC and replay

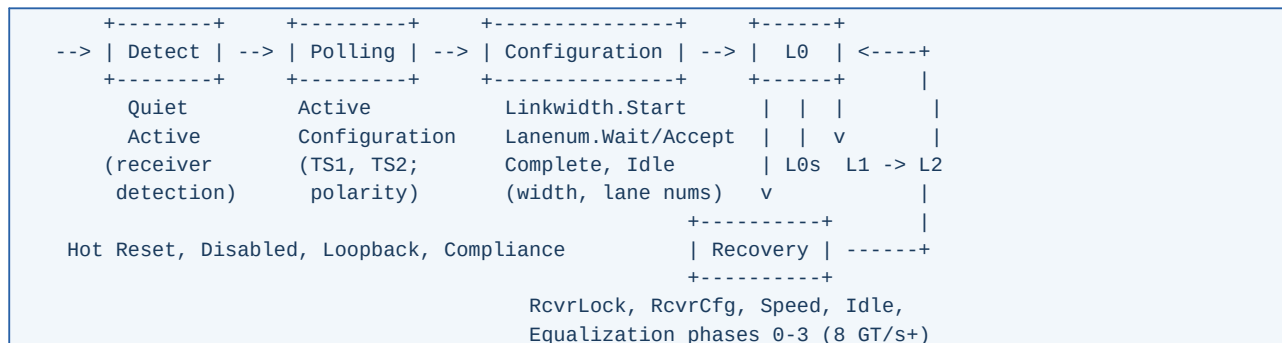
The data link layer makes each link reliable. The transmitter prepends a 12-bit **sequence number**, appends a 32-bit **LCRC** and keeps a copy in the **replay buffer**. The receiver checks LCRC and sequence; it returns an **Ack** DLLP (acknowledging everything up to a sequence number, coalesced by an AckNak latency timer) or a **Nak** on error. On Nak or when the **REPLAY_TIMER** expires, the transmitter replays everything unacknowledged from the buffer. After four replays of the same TLP (REPLAY_NUM rollover) the link is retrained through Recovery.

DLLP	Purpose
Ack / Nak	acknowledge / negatively acknowledge up to a sequence number
InitFC1 / InitFC2 (P, NP, Cpl)	advertise initial credits at link-up
UpdateFC (P, NP, Cpl)	return credits as receive buffers drain
PM_Enter_L1, PM_Enter_L23, PM_Active_State_Request_L1, PM_Request_Ack	power-management handshakes
Feature (Gen4+)	negotiate Data Link features such as scaled flow control
Vendor-specific	implementation defined

DLLPs are 6 bytes on Gen1-5 links: 4 bytes of content plus a 16-bit CRC; they are not sequence-numbered or replayed.

The replay buffer must hold every TLP sent during one Ack round trip, so its size scales with bandwidth x latency, exactly like the credit buffers. In Gen6 **FLIT mode** the unit of retry becomes the FLIT, and most errors are corrected by FEC before retry is needed, keeping the added latency to a few ns.

18.6 Physical layer: LTSSM, training sets and equalization



The PCIe LTSSM (simplified). Every link comes up at 2.5 GT/s and then changes speed through Recovery.

- **Detect**: the transmitter measures the charge time of the AC-coupled line to find a receiver termination on each lane.
- **Polling**: lanes exchange **TS1** and **TS2** training ordered sets to achieve bit and symbol (or block) lock. A lane that receives inverted TS symbols sets **polarity inversion** - D+ and D- swapped on the board are legal.
- **Configuration**: link width and lane numbers are negotiated; **lane reversal** (lane 0 wired to lane 15) is detected and handled here.
- **L0** is the normal state. Speed changes go through **Recovery**: at 8 GT/s and above, a four-phase **equalization** procedure tunes the transmitter FIR (pre-cursor, main, post-cursor coefficients; presets P0-P10) using feedback from the far receiver's eye measurement.
- Ordered sets: TS1, TS2, **SKP** (clock compensation: inserted periodically so an elastic buffer can absorb up to +/-300 ppm reference difference, or up to 5600 ppm with SRIS), EIOS/EIEOS (electrical idle), FTS (L0s exit), SDS (start of data stream).

PITFALL: 'THE LINK TRAINED AT GEN1 X4 INSTEAD OF GEN4 X8'

A degraded link almost never shows up as a failure: the LTSSM happily settles for fewer lanes or a lower rate. Always read the negotiated width and speed from the Link Status register in bring-up scripts and fail the test if they differ from expectations. Common causes: a lane map mismatch between package and board, reference-clock (100 MHz, SSC or not) configuration, an equalization preset table left at defaults, or a PHY firmware version mismatch.

18.7 Configuration space

Type 0 header (endpoint)	Type 1 header (bridge / port)
00 Device ID Vendor ID	00 Device ID Vendor ID
04 Status Command	04 Status Command
08 Class Code (24) Rev ID	08 Class Code (24) Rev ID
0C BIST HdrType LatTmr CacheLn	0C BIST HdrType(01) LatTmr CacheLn
10 BAR0	10 BAR0
14 BAR1	14 BAR1
18 BAR2	18 SecLat Subord Second. Prim. Bus
1C BAR3	1C Sec.Status IO Limit IO Base
20 BAR4	20 Memory Limit Memory Base
24 BAR5	24 Pref. Mem Limit Pref. Mem Base
28 CardBus CIS ptr	28..30 Pref. upper 32 / IO upper 16
2C Subsys ID Subsys Vendor ID	34 Capabilities Pointer
30 Expansion ROM base	38 Expansion ROM base
34 Capabilities Pointer	3C BridgeCtl IntPin IntLine
3C MaxLat MinGnt IntPin IntLine	
40..FF capability list (PM, MSI, MSI-X, PCI Express capability ...)	
100..FFF extended capabilities (AER, SR-IOV, ATS, L1 PM Substates ...)	

The 64-byte PCI-compatible headers, then 192 bytes of capabilities, then 3840 bytes of PCIe extended configuration space per function.

A **BAR** tells software how much address space a function needs and where it was placed. Software writes all ones, reads back, and the number of low-order zeros (in the writable bits) gives the size; bit 0 marks I/O vs memory, bits 2:1 mark 32/64-bit, bit 3 marks prefetchable. In RTL a BAR is simply a register whose low bits are hard-wired to 0 - an easy place to create a bug by making them writable. Capabilities are a linked list: each has an ID and a next pointer; common IDs are 0x01 PM, 0x05 MSI, 0x10 PCI Express, 0x11 MSI-X, and extended IDs 0x0001 AER, 0x000D ACS, 0x000E ARI, 0x000F ATS, 0x0010 SR-IOV, 0x001E L1 PM Substates.

Interrupts: INTx, MSI and MSI-X

- **INTx** emulation: Assert_INTx/Deassert_INTx messages mimic the four legacy level-triggered wires. Avoid for new designs.
- **MSI**: the function performs a posted **memory write** of a 16-bit data value to an address programmed by software; up to 32 vectors (a power of two, contiguous data values).
- **MSI-X**: up to 2048 vectors, each with its own address, data and mask bit, in a **table** located in a BAR, plus a Pending Bit Array. This is what NICs and NVMe use (one vector per queue).
- Because an MSI is a posted write, it is ordered behind the DMA writes that preceded it: the interrupt can never overtake its data. That property (and not a sideband wire) is what makes MSI safe.

18.8 Power management

State	Link	Exit latency (typical)	How entered
L0	active	-	normal operation
L0s	TX side in electrical idle	tens of ns to ~1 us (FTS)	ASPM, per direction, hardware idle timer (removed in FLIT mode)
L1	both directions idle, PLL may stay on	~2-10 us	ASPM (idle timer) or PCI-PM D-state change
L1.1 / L1.2	L1 + CLKREQ# deasserted; L1.2 also turns off common-mode keepers	~tens of us (L1.2)	L1 PM Substates capability
L2 / L3	main power off (L2 keeps aux power for wake)	ms	D3cold, system sleep

Device power states (D0, D3hot, D3cold) are separate from link states but interact with them. **LTR** (Latency Tolerance Reporting) lets a device tell the platform how much latency it can tolerate so that deep states are used safely. L1.2 matters most for battery devices: an NVMe SSD in a laptop spends most of its life there, and it drops PHY power to a few mW or less.

18.9 Errors, AER and advanced features

Class	Examples	Handling
Correctable	Receiver Error, Bad TLP, Bad DLLP, Replay Timer Timeout, REPLAY_NUM Rollover	fixed by hardware (retry); logged and counted
Uncorrectable non-fatal	Poisoned TLP, Completion Timeout, Unexpected Completion, Unsupported Request, Completer Abort, ECRC Error	transaction lost, link fine; driver recovery
Uncorrectable fatal	Data Link Protocol Error, Surprise Down, Flow Control Protocol Error, Receiver Overflow, Malformed TLP	link unreliable; reset (or DPC contains it)

Error classes reported through the Advanced Error Reporting (AER) extended capability, which also logs the offending TLP header.

- **Completion timeout:** every non-posted request needs a timer; the requester must release the tag and report an error if no completion arrives (ranges from 50 us to seconds are programmable).
- **DPC** (Downstream Port Containment) stops traffic below a port on a fatal error so that one bad device cannot crash the system.
- **SR-IOV:** a Physical Function (PF) exposes many **Virtual Functions** (VFs), each with its own Requester ID, BARs (carved from VF BARs) and MSI-X, so a VM can own a VF directly.
- **ATS/PRI/PASID:** a device caches address translations in an **ATC** by sending Translation Requests to the IOMMU (ATS), asks for pages to be made resident (PRI), and tags requests with a process address space ID (PASID prefix) for shared virtual memory.
- **IDE** (Integrity and Data Encryption) secures TLPs with AES-GCM per stream (Chapter 26).

18.10 PIPE: between controller and PHY

The **PIPE** specification (PHY Interface for PCI Express, SATA, USB and USB4 architectures, from Intel) standardizes the boundary between the controller's MAC (LTSSM, framing) and the PHY (SerDes). It is what lets an SoC team buy a PHY from one vendor and a controller from another.

Signal group	Examples	Direction / notes
Transmit data	TxData[31:0], TxDataK / TxDataValid, TxStartBlock, TxSyncHeader	MAC -> PHY, at PCLK, width 8-64 bits
Receive data	RxData, RxDataK, RxValid, RxStartBlock, RxSyncHeader, RxStatus[2:0]	PHY -> MAC; RxStatus reports decode/disparity/elastic-buffer events
Control	PowerDown[3:0], Rate, Width, TxDetectRx/Loopback, TxElecIdle, RxPolarity, TxDeemph / coefficients	MAC -> PHY
Status	PhyStatus, RxElecIdle, RxEqEval feedback	PHY -> MAC; PhyStatus acknowledges rate and power changes
Message bus (PIPE 5+)	M2P_MessageBus[7:0], P2M_MessageBus[7:0]	register access replacing many sideband wires

Two PIPE architectures exist. In the original one the PHY contains 8b/10b or 128b/130b coding, the elastic buffer and receiver detection; in the newer **SerDes architecture** the PHY is little more than a serializer/deserializer with CDR, and the MAC does encoding, block alignment and clock compensation. The latter needs more controller RTL but makes PHYs simpler and more reusable across protocols.

18.11 Controller design considerations

- **Datapath width vs clock:** a Gen5 x16 port carries about 64 GB/s per direction, i.e. 512 bits at 1 GHz or 1024 bits at 500 MHz. At these widths several TLPs can start and end in the same cycle; TLP straddling logic (multiple SOP/EOP per beat) dominates RTL complexity.

- **Buffers:** replay buffer, posted/non-posted/completion receive queues sized from the credit round trip; completion reassembly for split read completions (Read Completion Boundary 64/128 B, Max_Read_Request_Size up to 4 KB, Max_Payload_Size 128 B-4 KB).
- **Tags:** 8-bit, 10-bit (Gen4+) or 14-bit (FLIT mode) tags; the number of outstanding reads x read size must cover memory latency x bandwidth, or reads will not saturate the link.
- **Bridging to the SoC:** an AXI bridge with inbound and outbound address translation windows (iATU), MSI capture in the root complex, DMA engines, and error/interrupt reporting to the GIC. Ordering rules must be preserved across the AXI boundary, which has different rules (Chapter 7).
- **Clocks and resets:** PIPE PCLK (changes with rate), core clock, AXI clock and aux clock (for L1.2/L2); PERST# fundamental reset; hot reset and FLR (function level reset) of individual functions.
- **Verification:** PCIe VIP, compliance test suites (the PCI-SIG Compliance Workshop), LTSSM coverage, error injection (LCRC, sequence, credit overflow, completion timeout) and long interoperability runs on FPGA or emulation with real drivers.

18.12 CXL: Compute Express Link

CXL reuses the PCIe physical layer and electricals but adds coherent memory semantics. After link training in PCIe mode, the link negotiates CXL through **Alternate Protocol Negotiation** and then carries three multiplexed protocols in fixed-size flits:

Protocol	Direction	What it does
CXL.io	both	PCIe-equivalent: discovery, configuration, interrupts, DMA, register access
CXL.cache	device -> host memory	lets a device cache host memory coherently (the host is the home; channels H2D/D2H request, response, data)
CXL.mem	host -> device memory	lets the host load/store device-attached memory with low latency (M2S Req/RwD, S2M NDR/DRS)

Device type	Protocols	Examples
Type 1	CXL.io + CXL.cache	SmartNIC or accelerator without its own memory, caching host data
Type 2	CXL.io + CXL.cache + CXL.mem	GPU/accelerator with local memory (HDM) shared coherently with the host
Type 3	CXL.io + CXL.mem	memory expander: DRAM behind a CXL controller, pooled memory

Version	Year	PHY	Key additions
1.0 / 1.1	2019	PCIe 5.0, 32 GT/s	direct attach, 68-byte flits, Types 1-3
2.0	2020	PCIe 5.0	single-level switching, memory pooling with multi-logical devices (up to 16 LDs), hot plug, IDE link security
3.0	2022	PCIe 6.0, 64 GT/s	256-byte flits, multi-level switching and fabrics, memory sharing, back-invalidate for coherent device memory
3.1 / 3.2	2023 / 2024	PCIe 6.x	fabric and security extensions, improved RAS and monitoring

Why CXL matters: the **CXL.cache/CXL.mem** paths bypass the PCIe transaction layer and its ordering machinery, so a load to CXL-attached memory costs roughly one extra NUMA hop (on the order of 100+ ns over local DRAM) rather than the microseconds of a PCIe DMA round trip. Type 3 memory expanders let servers add DRAM capacity and bandwidth without more DDR channels, and pooling lets that memory be assigned to hosts dynamically. The coherence protocol is a MESI-style host-managed scheme (Chapter 9); the device-side **DCOH** (device coherency engine) and the host's **home agent** are the new RTL blocks.

EXPERT NOTE: LATENCY, NOT BANDWIDTH, IS THE CXL DESIGN GOAL

A CXL controller's RTL budget is dominated by latency: flit packing without waiting for a full flit, bypass paths around the CXL.io stack, and tight ARB/MUX logic. A PCIe endpoint that adds 100 ns of pipeline is fine; a CXL.mem device that does the same doubles its added latency. Measure load-to-use latency in simulation from the first design review.

18.13 Summary

- PCIe is a point-to-point, packet-switched link that preserves the PCI software model (BDF, config space, BARs, capabilities).
- Rates double each generation: 8b/10b to Gen2, 128b/130b for Gen3-5, PAM4 + FLIT + FEC for Gen6/7.
- Transaction layer: TLPs (MRd/MWr/Cfg/Cpl/Msg/AtomicOp), three flow-control classes with credit gating (modulo compare) and ordering rules that keep producer-consumer correct and deadlock-free.
- Data link layer: sequence numbers, LCRC, replay buffer, Ack/Nak and flow-control DLLPs. Physical layer: LTSSM, TS1/TS2, equalization, polarity inversion and lane reversal.
- MSI/MSI-X are posted writes; power states L0s/L1/L1.x; AER, DPC, SR-IOV and ATS are standard server features.
- PIPE separates controller and PHY; CXL adds CXL.cache and CXL.mem for coherent, low-latency device and memory attach.

18.14 Exercises

- 1 Use `tlp_pkg` to build the headers of: a 64-byte MWr to 0x1_0000_0000; a 1-byte MRd at 0x1003; a CplD returning 128 of 256 requested bytes (what is Byte Count?). Check them with `ref_tlp.py`.
- 2 A 4096-byte MRd is issued with a Read Completion Boundary of 64 B and Max_Payload_Size 256 B. List the possible completion sizes and the Lower Address / Byte Count fields of each.
- 3 Modify `tb_fc.sv` so the receiver drains every 4 cycles and the UpdateFC latency is 40 cycles. How many data credits are needed to keep the transmitter from stalling? Verify by simulation.
- 4 Explain why posted writes must be allowed to pass non-posted requests. Construct the deadlock that occurs otherwise with two devices doing peer-to-peer traffic.
- 5 Estimate the replay buffer size for a Gen4 x8 port whose Ack round-trip is 700 ns.
- 6 Compare MSI and a sideband interrupt wire from an endpoint's DMA engine: which ordering hazard does MSI remove?

19. Ethernet: MAC, PHY and the xMII Family

Ethernet (IEEE 802.3) has run from 10 Mb/s coax in the 1980s to 800 Gb/s optics today while keeping one thing constant: the **MAC frame**. Everything below the MAC - line codes, FEC, lanes, copper or fiber - has been replaced many times, but a frame captured on a 400G data-center link has the same header and the same CRC-32 FCS as one on a 1983 thick-coax network. For an SoC this layering is the key to the design: the MAC is digital RTL you write or license; the PHY is a separate chip or a hard macro; and a standard **media-independent interface** (the xMII family) joins them. Automotive Ethernet PHYs (100BASE-T1, 1000BASE-T1) are covered in Chapter 15; MDIO electrical details in Chapter 16.

19.1 IEEE 802.3 and the layer model

OSI	IEEE 802.3 sublayers	where it lives
+-----+	+-----+	
Data	MAC client (LLC / bridge / IP stack)	software, switch logic
link	MAC control (PAUSE, PFC) - optional	RTL (MAC IP)
	MAC (framing, FCS, addressing)	RTL (MAC IP)
+-----+	Reconciliation Sublayer (RS)	RTL
	+----- xMII: MII/RMII/GMII/RGMII/ -----+	<- pins or on-die bus
Phys-	PCS (4B5B, 8b/10b, 64b/66b, lanes)	PHY chip or SerDes RTL
ical	FEC (RS-FEC, BASE-R FEC) - by speed	PHY / SerDes RTL
	PMA (serialize, CDR, bit mux)	analog SerDes
	PMD (optics / copper driver)	module / analog
	AN (auto-negotiation)	PHY
+-----+	+-----+	
MDI: connector / medium		

The 802.3 layering. The xMII is the boundary most SoC designers integrate; the PCS/PMA may be on-chip (SerDes-based SoCs) or in an external PHY (BASE-T copper).

19.2 The frame

Preamble	SFD	Dest MAC	Src MAC	[802.1Q tag]	Type/Len	Payload	FCS	
7 x 55	D5	6	6	81 00 + TCI	2	46..1500	4	
<-- 8 bytes ---> <----- 64 .. 1518 bytes (1522 with one VLAN tag) ----->								
then IFG >= 12 byte times								
TCI (16 bits) = PCP[2:0] priority DEI VID[11:0] VLAN id								
Type/Len >= 0x0600: EtherType (0x0800 IPv4, 0x86DD IPv6, 0x0806 ARP, 0x8100 VLAN, 0x88A8 S-tag, 0x8808 MAC control, 0x88F7 PTP)								
Type/Len <= 1500: length of an 802.3 / LLC frame								

Ethernet frame format. The minimum frame (64 bytes from destination to FCS) came from CSMA/CD collision detection and was kept forever.

- **Preamble + SFD:** 7 bytes of 0x55 and 0xD5. On the wire, LSB first, this is 62 alternating bits ending in '11' - originally for receiver clock sync, today mainly a delimiter (10G+ PCS encodes the start as a control character in lane 0).
- **Addresses:** 48-bit MACs. Bit 0 of the first byte is the I/G bit (1 = multicast/broadcast; FF:FF:FF:FF:FF:FF is broadcast), bit 1 is U/L (1 = locally administered).
- **Payload** 46-1500 bytes; shorter payloads are **padded** with zeros so the frame reaches 64 bytes. **Jumbo frames** (typically up to ~9000-9216 bytes) are a widely used non-standard extension.
- **FCS:** CRC-32 (polynomial 0x04C11DB7, reflected, initial value all ones, final complement) over destination address through padding - the same CRC as zlib/PNG. It is transmitted so that its least-significant byte in the 'reflected' convention goes first, which is why `struct.pack('<I', zlib.crc32(frame))` gives the on-wire FCS bytes.
- **IFG:** at least 96 bit times (12 bytes) of idle between frames. Together with preamble/SFD this adds 20 bytes of overhead to every frame.

$$\text{wire bytes per minimum frame} = 8 + 64 + 12 = 84 \text{ bytes} = 672 \text{ bits}$$

$$\text{max frame rate at 10 Gb/s} = 10\text{e9} / 672 = 14.88 \text{ Mframes/s}$$

$$\text{payload efficiency at 1500 B} = 1500 / (1500 + 18 + 20) = 97.5\%$$

The famous line-rate numbers. A 10G MAC must accept a new frame every 67.2 ns; at 100G every 6.7 ns - i.e. every few clock cycles.

19.3 Speeds and PHY types

Speed	Example PHY	Line code / modulation	Standard
10 Mb/s	10BASE-T	Manchester	802.3i (1990)
100 Mb/s	100BASE-TX	4B5B + MLT-3, 125 MBd	802.3u (1995)
1 Gb/s	1000BASE-T / -X (SX, LX)	4 pairs PAM5 / 8b/10b at 1.25 GBd	802.3ab / 802.3z
2.5 / 5 Gb/s	2.5G/5GBASE-T	reduced-rate 10GBASE-T coding	802.3bz (2016)
10 Gb/s	10GBASE-R (SR/LR/KR), 10GBASE-T	64b/66b at 10.3125 GBd; PAM16 (DSQ128) on copper	802.3ae / 802.3an
25 Gb/s	25GBASE-R	64b/66b at 25.78125 GBd, optional RS-FEC	802.3by (2016)
40 / 100 Gb/s	40GBASE-R4, 100GBASE-R4	4 x 10G / 4 x 25G lanes, RS(528,514) FEC	802.3ba / 802.3bm
200 / 400 Gb/s	400GBASE-SR8 / -FR8	PAM4, 26.5625 GBd per 50G lane, RS(544,514) FEC	802.3bs (2017)
100G per lane	100/200/400GBASE-KR1/2/4...	PAM4 at 53.125 GBd	802.3ck (2022)
800 Gb/s	800GBASE-R8	8 x 100G PAM4 lanes	802.3df (2024)
1.6 Tb/s	200G-per-lane PAM4	~106 GBd PAM4	802.3dj (in progress)

From 10G upward, the MAC and PCS talk through a wide internal interface, the PCS uses **64b/66b** blocks (2-bit sync header + 64 scrambled bits, self-synchronizing scrambler $x^{58} + x^{39} + 1$) and multi-lane speeds insert **alignment markers** so the receiver can deskew lanes. From 50G-per-lane PAM4 onwards, **RS-FEC** is mandatory and adds on the order of 100 ns of latency - a real concern for low-latency trading and AI clusters.

19.4 MAC functions

- **Transmit:** prepend preamble/SFD, insert source address (optional), pad short frames, compute and append the FCS, enforce IFG, handle PAUSE (stop transmitting when paused), and, in the rare half-duplex 10/100 mode, CSMA/CD with backoff.
- **Receive:** detect SFD, check FCS, check length (runts < 64 bytes, oversize > max), filter destination address (exact match, multicast hash, promiscuous), strip FCS/padding if configured, recognize MAC control frames, and count everything in **statistics counters** (the RMON / 802.3 MIB).
- **Management:** MDIO master to read PHY registers, link state tracking, speed/duplex configuration from auto-negotiation results.

19.5 The xMII family

Interface	Speed	Data path	Clock	Signals (approx.)
MII	10/100M	4-bit TXD/RXD	25 / 2.5 MHz, from PHY (TX_CLK, RX_CLK)	16 + MDC/MDIO
RMII	10/100M	2-bit	50 MHz REF_CLK shared	7-8 + MDIO
GMII	1G	8-bit	125 MHz; GTX_CLK from MAC	24-25 + MDIO

Interface	Speed	Data path	Clock	Signals (approx.)
RGMI	10M-1G	4-bit DDR	125 MHz DDR (25 / 2.5 MHz at 100/10)	12 + MDIO
SGMI	10M-1G	serial 8b/10b	1.25 GBd LVDS pair each way	4 (8 with clocks)
XGMI	10G	32-bit + 4 ctrl, DDR	156.25 MHz	74
XAUI	10G	4 lanes 8b/10b	3.125 GBd per lane	16
XFI / SFI	10G	1 serial lane 64b/66b	10.3125 GBd	4
USXGMI	10M-10G	1 serial lane, rate replication	10.3125 GBd	4
CAUI-4 / 100GAUI-2	100G	4 x 25G NRZ / 2 x 50G PAM4	25.78 / 26.56 GBd	16 / 8

Media-independent interfaces. Pin counts exclude power and the two MDIO wires.

RGMI transmit (1 Gb/s), one byte per 8 ns TxC period

```
TxC      ____/~~~~~\____/~~~~~\____/~~~~~\____
TXD[3:0] ==X= b[3:0] =X= b[7:4] =X= b[3:0] =X= b[7:4] =X==
TX_CTL  ==X= TX_EN  =X= EN^ER  =X= TX_EN  =X= EN^ER  =X==
           rising edge   falling edge
```

Data changes with the clock edge at the MAC pins (RGMI 1.x) -> a delay of about 1.5-2 ns must be added (in the PHY: 'RGMI-ID', in the MAC, or on the board) so the receiver samples in the middle of the data eye.

RGMI halves GMII's pin count by using both clock edges, and multiplexes TX_EN and TX_ER onto one control pin.

PITFALL: RGMI DELAY APPLIED TWICE (OR NOT AT ALL)

The single most common Ethernet bring-up bug on SoC boards: the 2 ns RGMI clock delay must be inserted exactly once per direction - by the PHY (rgmii-id / rgmii-txid / rgmii-rxid in the Linux device tree), by the MAC's I/O delay lines, or by PCB trace length. Zero or double delay gives a link that negotiates fine (MDIO and auto-negotiation work) but drops or corrupts frames, often only at one temperature. Put the delay configuration in the SoC integration checklist and test at 10, 100 and 1000 Mb/s.

SGMI deserves a note: it carries the GMII byte stream over one 1.25 GBd 8b/10b SerDes lane per direction (1000BASE-X coding), and supports 100M and 10M by repeating each byte 10 or 100 times. Its in-band auto-negotiation word (a Cisco-defined variant of Clause 37) tells the MAC the PHY's speed and link state. **USXGMI** extends the same idea to 10M-10G on a 10.3125 GBd lane, and multi-port variants carry several ports on one lane for switch-to-PHY links.

19.6 Auto-negotiation and MDIO management

Twisted-pair PHYs negotiate speed and duplex with **Clause 28** auto-negotiation: each side sends **Fast Link Pulse** bursts encoding a 16-bit link code word (selector field, technology ability, remote fault, next page). Both sides pick the highest common ability by a fixed priority; 1000BASE-T adds next pages for master/slave resolution (who provides the clock). Backplane (KR) links use **Clause 73**, and 1000BASE-X uses **Clause 37**. If one side has auto-negotiation disabled, 'parallel detection' guesses the speed but must assume half duplex - the source of the classic duplex mismatch that gives a working but terribly slow link.

```
Clause 22 MDIO frame (MDC <= 2.5 MHz, MDIO open-drain, sampled on MDC rising)
| PRE (32 x 1) | ST 01 | OP 10=rd 01=wr | PHYAD 5 | REGAD 5 | TA 2 | DATA 16 | idle
```

```
Clause 45: ST 00, 'address' frame then read/write frame;
           PRTAD 5 | DEVAD 5 (MMD: PMA, PCS, AN ...) | 16-bit register address
```

MDIO management frames (details in Chapter 16). Registers 0 (control) and 1 (status) of every Clause 22 PHY are standardized; bit 2 of register 1 is Link Status (latching low).

19.7 Flow control: PAUSE and PFC

PAUSE (802.3x, Annex 31B) is a MAC control frame: destination 01-80-C2-00-00-01, EtherType **0x8808**, opcode **0x0001** and a 16-bit pause time in **quanta of 512 bit times**. A receiver whose buffers are filling sends PAUSE; the link partner's MAC stops transmitting (after finishing the current frame) for that time, or until a PAUSE with time 0. PAUSE stops the whole link, which causes head-of-line blocking, so data centers use **Priority Flow Control** (PFC, 802.1Qbb): opcode **0x0101**, an 8-bit class-enable vector and eight timers, pausing only selected priorities. PFC is what makes Ethernet 'lossless' enough for RoCE (RDMA over Converged Ethernet).

one quantum = 512 bit times = 51.2 ns at 10 Gb/s

pause 0xFFFF quanta at 10 Gb/s = 65535 x 51.2 ns = 3.36 ms

headroom needed $\geq (\text{cable} + \text{PHY} + \text{MAC delay}) \times 2 \times \text{rate} + 1 \text{ max frame}$

PAUSE arithmetic. The receiver must still absorb what is in flight after it sends PAUSE: buffer headroom grows with speed and cable length.

19.8 Precision time: IEEE 1588 PTP

The **Precision Time Protocol** synchronizes clocks across a network to sub-microsecond (with hardware support, nanosecond-level) accuracy. A master sends **Sync** (and, in two-step mode, a **Follow_Up** carrying the precise transmit time t_1); the slave records receive time t_2 , sends **Delay_Req** at t_3 and gets t_4 back in **Delay_Resp**.

mean path delay = $((t_2 - t_1) + (t_4 - t_3)) / 2$

offset = $((t_2 - t_1) - (t_4 - t_3)) / 2$ (assumes symmetric path)

The PTP servo equations.

- Accuracy comes from **hardware timestamping**: the MAC (or PHY) latches a free-running, frequency-adjustable time counter when the SFD passes the xMII, so software and queueing delays are excluded.
- One-step** mode writes t_1 into the Sync frame on the fly (the MAC must patch the field and fix the UDP checksum / FCS in flight); **two-step** sends it in Follow_Up.
- Transparent clocks in switches add their residence time to a correction field. **gPTP** (IEEE 802.1AS) is the profile used by TSN and automotive.
- RTL owns: the timer with fractional increment (e.g. 32.32 fixed-point nanoseconds per clock) for frequency trimming, timestamp capture at a fixed pipeline point with known latency, PPS output and external timestamp inputs.

19.9 Time-Sensitive Networking (TSN)

Standard	Mechanism	Purpose
802.1AS	gPTP time synchronization	common time base for all nodes
802.1Qav	credit-based shaper	smooth audio/video (AVB) streams, bounded latency
802.1Qbv	time-aware shaper: gate control list opens/closes each queue on a time schedule	hard real-time slots (control loops)
802.1Qbu / 802.3br	frame preemption	express frames interrupt long frames to cut latency
802.1CB	frame replication and elimination	seamless redundancy
802.1Qci	per-stream filtering and policing	protect the schedule from misbehaving talkers

TSN turns best-effort Ethernet into a deterministic network for industrial control and in-vehicle backbones. In the MAC this means multiple transmit queues, per-queue gates driven by the PTP time, shapers, and a preemptible/express MAC pair.

19.10 NIC offloads

- **Checksum offload:** compute IPv4 header and TCP/UDP one's-complement checksums in the MAC (store-and-forward, because the checksum field precedes the data it covers) or verify them on receive.
- **TSO/LSO:** software hands one large buffer (up to 64 KB) and a header template; hardware cuts MSS-sized segments, updates IP ID, TCP sequence numbers, flags and checksums. **LRO/GRO** does the reverse on receive.
- **RSS:** a Toeplitz hash of the 5-tuple indexes an indirection table that selects a receive queue (and so a CPU core), spreading load while keeping each flow in order.
- **VLAN tag insertion/stripping,** multiple DMA rings with interrupt moderation, SR-IOV virtual functions (Chapter 18), and in data-center NICs, RDMA and crypto offload (MACsec, Chapter 26).

19.11 A MAC in RTL: TX framer and RX FCS checker

A MAC's transmit path is a small state machine around a CRC. Below is a complete GMII-style (8-bit, 125 MHz) framer: it takes a frame (destination MAC to end of payload) on a ready/valid stream, sends preamble and SFD, forwards the data, **pads** to 60 bytes, appends the complemented CRC LSB-first and enforces a 12-byte IFG. The receive checker runs the same CRC over data **and** FCS and compares with the constant residue 0xDEBB20E3 - no need to buffer the last four bytes to know where the FCS starts.

```
// Byte-wide CRC-32 as used by the Ethernet FCS (reflected form of 0x04C11DB7).
package eth_crc_pkg;

function automatic logic [31:0] crc32_byte(input logic [31:0] crc, input logic [7:0] d);
    crc = crc ^ {24'h0, d};
    for (int i = 0; i < 8; i++)
        crc = crc[0] ? (crc >> 1) ^ 32'hEDB8_8320 : (crc >> 1);
    return crc;
endfunction
endpackage

// MAC transmit framer, GMII-style byte output (txd/tx_en), one byte per clock.
// Input: frame from destination MAC to end of payload, AXI-Stream-like handshake.
// Output: 7 x 55 preamble, D5 SFD, frame, zero pad to 60 bytes, FCS, 12-byte IFG.
module eth_tx_framer (
    input logic      clk, rst_n,
    input logic [7:0] s_data, input logic s_valid, s_last, output logic s_ready,
    output logic [7:0] txd,      output logic tx_en
);
    import eth_crc_pkg::*;
    typedef enum logic [2:0] {IDLE, PRE, SFD, DATA, PAD, FCS, IFG} st_t;
    st_t st;
    logic [31:0] crc;
    logic [5:0] n; // byte counter within a state (or of frame)
    assign s_ready = (st == DATA);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin st <= IDLE; tx_en <= 0; txd <= '0; n <= '0; crc <= '1; end
        else case (st)
            IDLE: begin tx_en <= 0; if (s_valid) begin st <= PRE; n <= '0; end end
            PRE:  begin tx_en <= 1; txd <= 8'h55; n <= n + 1'b1; if (n == 6) st <= SFD; end
            SFD:  begin txd <= 8'hD5; crc <= '1; n <= '0; st <= DATA; end
            DATA: if (s_valid) begin
                        txd <= s_data; crc <= crc32_byte(crc, s_data);
                        if (n != 6'd63) n <= n + 1'b1; // saturate: only "< 60" matters
                    end
        endcase
end
```



```

        if (s_last) st <= (n < 6'd59) ? PAD : FCS;
        if (s_last) n <= (n < 6'd59) ? n + 1'b1 : 6'd0;
    end
    PAD: begin txd <= 8'h00; crc <= crc32_byte(crc, 8'h00); n <= n + 1'b1;
        if (n == 6'd59) begin st <= FCS; n <= '0; end end
    FCS: begin txd <= ~crc[8*n[1:0] +: 8]; // complement, LSB byte first
        n <= n + 1'b1; if (n == 3) begin st <= IFG; n <= '0; end end
    IFG: begin tx_en <= 0; n <= n + 1'b1; if (n == 11) st <= IDLE; end
    default: st <= IDLE;
endcase
endmodule

// Receive FCS check: run the CRC over everything after the SFD, FCS included.
// For a good frame the (uncomplemented, reflected) register ends at 0xDEBB20E3.
module eth_rx_fcs (
    input logic clk, rst_n,
    input logic [7:0] rxd, input logic rx_dv,
    output logic done, fcs_ok, output logic [10:0] len
);
    import eth_crc_pkg::*;
    logic [31:0] crc; logic in_frame, dv_q;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            crc <= '1; in_frame <= 0; dv_q <= 0; done <= 0; fcs_ok <= 0; len <= 0;
        end
        else begin
            dv_q <= rx_dv; done <= 1'b0;
            if (rx_dv && !in_frame && rxd == 8'hD5) begin in_frame <= 1; crc <= '1; len <= 0; end
            else if (rx_dv && in_frame) begin crc <= crc32_byte(crc, rxd); len <= len + 1'b1; end
            if (!rx_dv && dv_q && in_frame) begin // end of carrier
                in_frame <= 0; done <= 1'b1; fcs_ok <= (crc == 32'hDEBB_20E3);
            end
        end
end
endmodule

```

eth.sv - byte-wide CRC-32, MAC TX framer and RX FCS checker.

```

module tb;
    logic clk = 0, rst_n = 0; always #4 clk = ~clk; // 125 MHz GMII clock
    logic [7:0] s_data, txd, rxd; logic s_valid = 0, s_last = 0, s_ready, tx_en, rx_dv;
    logic done, fcs_ok; logic [10:0] len;
    logic flip = 0; // corrupt one bit on the "wire"
    eth_tx_framer tx (.*);
    assign rxd = txd ^ {7'b0, flip}; assign rx_dv = tx_en;
    eth_rx_fcs rx (.clk, .rst_n, .rxd, .rx_dv, .done, .fcs_ok, .len);
    // ARP request: broadcast dst, src 02:00:00:00:00:01, EtherType 0806, 28-byte body
    logic [7:0] frm [42];
    int nb = 0; string line = ""; bit dump = 1;
    always @(posedge clk) if (tx_en && dump) begin
        nb++; line = {line, $sformatf("%02h", txd)};
        if (nb % 36 == 0) begin $display("%s", line); line = ""; end
    end
    always @(posedge clk) if (done)
        if (fcs_ok) $display("RX: %0d bytes after SFD, FCS OK", len);
        else $display("RX: %0d bytes after SFD, FCS BAD -> frame dropped", len);
    task automatic send_frame;
        for (int i = 0; i < 42; i++) begin
            s_data = frm[i]; s_valid = 1; s_last = (i == 41);
            @(posedge clk); while (!s_ready) @(posedge clk);
            #1;
        end
    end
end

```



```

    s_valid = 0; s_last = 0;
endtask
initial begin
    for (int i = 0; i < 6; i++) frm[i] = 8'hFF;
    frm[6] = 8'h02; for (int i = 7; i < 11; i++) frm[i] = 8'h00; frm[11] = 8'h01;
    frm[12] = 8'h08; frm[13] = 8'h06;
    for (int i = 14; i < 42; i++) frm[i] = 8'(i - 14);    // stand-in ARP body 00..1B
    repeat (2) @(posedge clk); rst_n = 1; #1;
    send_frame();
    wait (done); @(posedge clk);
    if (line.len()) $display("%s", line);
    $display("TX: %0d bytes on GMII incl. preamble/SFD", nb);
    dump = 0;
    fork
        // second frame: flip one bit
        send_frame();
    begin repeat (30) @(posedge clk); #1 flip = 1; @(posedge clk); #1 flip = 0; end
    join
    wait (done); repeat (2) @(posedge clk);
    $finish;
end
endmodule

```

tb_eth.sv - sends a 42-byte ARP-style frame (padded to 60), loops it into the checker, then repeats it with one bit flipped on the wire.

```

5555555555555555d5ffffffffffff0200000000010806000102030405060708090a0b0c0d
0e0f101112131415161718191a1b0000000000000000000000000000000000000000a95ab799
TX: 72 bytes on GMII incl. preamble/SFD
RX: 64 bytes after SFD, FCS OK
RX: 64 bytes after SFD, FCS BAD -> frame dropped

```

Real vvp output: 72 bytes on the GMII (8 preamble/SFD + 60 frame + 4 FCS); FCS bytes a9 5a b7 99; the corrupted copy is rejected.

```

import zlib, struct
frame = bytes([0xFF]*6 + [0x02, 0, 0, 0, 0, 0x01] + [0x08, 0x06] + list(range(28)))
frame += bytes(60 - len(frame))                # pad to 60 bytes
fcs = struct.pack("<I", zlib.crc32(frame))      # FCS sent LSB byte first
print("python zlib FCS bytes:", fcs.hex())
def crc_reg(data):                             # register without final xor
    c = 0xFFFFFFFF
    for b in data:
        c ^= b
        for _ in range(8):
            c = (c >> 1) ^ (0xEDB88320 if c & 1 else 0)
    return c
print("residue over frame+FCS: %08x" % crc_reg(frame + fcs))

```

ref_eth.py - reference FCS with Python's zlib.crc32 and the residue check.

```

python zlib FCS bytes: a95ab799
residue over frame+FCS: debb20e3

```

Real Python output. zlib.crc32 over the padded 60 bytes gives exactly the FCS the RTL transmitted, and the residue over frame + FCS is 0xDEBB20E3, the constant the RX checker compares to.

WHY A RESIDUE CHECK WORKS

Appending the complemented CRC makes the whole codeword (data + FCS) satisfy a fixed polynomial identity, so the LFSR state after the last FCS byte is the same constant for every good frame. The receiver just keeps clocking the CRC to the end of the frame and compares once, when RX_DV falls. USB's CRC16 residual 0x800D in Chapter 17 is the same trick.

From 1G to 100G: what changes in the RTL

- **Width:** 8 bits at 125 MHz (1G) becomes 64 bits at 156.25 MHz (10G), 64 bits at 390.625 MHz (25G) and 256-512 bits at a few hundred MHz for 100G-400G. The CRC must then process 8-64 bytes per clock: a parallel CRC (XOR tree derived from the matrix form), with a separate final stage for frames that end mid-word.
- **Alignment:** on XGMII a frame may start in lane 0 or lane 4; wider buses let two frames share a beat, so start/end detection, padding and FCS insertion become multi-lane problems (the same straddling problem as PCIe TLPs).
- **FIFOs:** TX FIFO for store-and-forward (checksum insertion, no underrun mid-frame - an underrun must be signalled with TX_ER or a bad FCS so the frame is dropped, never truncated silently); RX FIFO with drop-on-overflow at frame granularity and a status word per frame.
- **Clock domains:** the xMII/PCS clock (from the PHY or SerDes) is not the system clock; async FIFOs sit between MAC and DMA. The clock-compensation 'IFG stretching/shrinking' happens in the PCS.

19.12 Where Ethernet sits in an SoC

Block	Typical owner	Notes
MAC + DMA (GMAC, XGMAC)	RTL (licensed IP), integration	AXI master for descriptors/data, APB/AXI-Lite for registers, interrupts
PCS/PMA (SerDes-based)	PHY team / vendor hard macro	shared SerDes with PCIe/USB in many SoCs (combo PHY)
External PHY (BASE-T)	board / PHY vendor	RGMII or SGMII to the SoC; MDIO for control
Switch (multi-port)	RTL	lookup tables, queues, TSN, often integrated in automotive SoCs
Drivers, PTP stack, TSN config	software	Linux stmmac/fec-style drivers, linuxptp

19.13 Summary

- The 802.3 MAC frame is fixed across all speeds: preamble, SFD, addresses, optional VLAN tag, EtherType/length, 46-1500 bytes payload, CRC-32 FCS, then at least 12 bytes of IFG.
- The PHY is split into PCS/FEC/PMA/PMD; the MAC-PHY boundary is an xMII: MII/RMII/GMII/RGMII in parallel, SGMII/USXGMII/XFI/CAUI in serial form.
- Auto-negotiation (Clause 28/37/73) picks speed and duplex; MDIO (Clause 22/45) manages PHY registers.
- PAUSE and PFC provide flow control; PTP with hardware timestamps gives ns-level time; TSN adds scheduled, shaped, preemptible traffic.
- The RTL MAC is a framer + CRC + FIFOs; wide MACs add parallel CRC and multi-frame-per-beat alignment. The FCS equals zlib.crc32 of the frame, and a receiver can check a fixed residue.

19.14 Exercises

- 1 Modify `eth_tx_framer` to insert an 802.1Q VLAN tag (0x8100 + TCI) after the source address. What happens to the minimum payload? Check the new FCS with `zlib`.
- 2 Add a destination-address filter to `eth_rx_fcs` that accepts only broadcast and one programmable unicast address.
- 3 Compute the maximum frame rate and payload throughput at 25 Gb/s for 64-, 512- and 1518-byte frames.
- 4 A 100 m 10GBASE-T link is paused with PFC. Estimate the buffer headroom needed per priority (propagation about 5 ns/m, PHY latency on the order of 2-3 us each way).
- 5 Derive the RGMII setup/hold budget at 125 MHz DDR: the bit period is 4 ns; with 2 ns of added delay and +/-500 ps of skew, what margin remains?

- 6 Write the equations for a two-step PTP exchange with an asymmetric path (forward delay d_1 , reverse d_2). What offset error results?

20. Memory Interfaces: SDRAM, DDR4/DDR5, LPDDR, HBM, DFI and Flash

Every processor, GPU and AI accelerator is ultimately limited by how fast it can move data to and from memory, so the DRAM interface is usually the widest, fastest, most power-hungry and most timing-critical interface on an SoC. It is also unlike the other links in this part: DRAM is not a packet protocol with a smart partner on the other end but a **command-driven analog array** that does exactly what it is told, when it is told, and relies on the controller to respect dozens of timing rules. This chapter builds that model from the bank upward, then covers the DDR, LPDDR, GDDR and HBM families, the controller and PHY (joined by **DFI**), and finally the flash interfaces: raw NAND (ONFI/Toggle), eMMC, SD and UFS.

20.1 SDRAM basics: banks, rows, columns

```

one DRAM device (e.g. DDR4 x8, 8 Gb)
+-----+
| bank group 0      bank group 1    ...    bank group 3      |
| +-----+ +-----+ +-----+ +-----+ |
| |bank 0| |bank 1|... |bank 4| ...    |bank15| |
| | rows | |      | |      | |      | |
| | 64K  | |      | |      | |      | |
| |-----| |-----| |-----| |-----| |
| |sense | |sense | |sense | <- row buffer (1 KB page) |
| |amps  | |amps  | |amps  | one open row per bank    |
| +-----+ +-----+ +-----+ |
| +---- column mux / prefetch 8n ---- I/O (DQ[7:0], DQS) ----|--> pins
+-----+

```

A DRAM device is several independent banks. Each bank is a 2-D array of 1T1C cells with one row buffer (the sense amplifiers).

A cell is one transistor and one capacitor; reading it is destructive and slow, so DRAM works a whole **row** (a 'page', typically 1-2 KB per device) at a time. **ACTIVATE** (ACT) copies a row into the bank's sense amplifiers; **READ/WRITE** then transfer bursts of **columns** from/to that open row; **PRECHARGE** (PRE) writes the row back and prepares the bank for another row. Because the capacitors leak, every row must be **REFRESHED** within a retention period (64 ms at normal temperature, 32 ms when hot), so the controller issues a REF command on average every **tREFI** (7.8 us for DDR4, 3.9 us for DDR5).

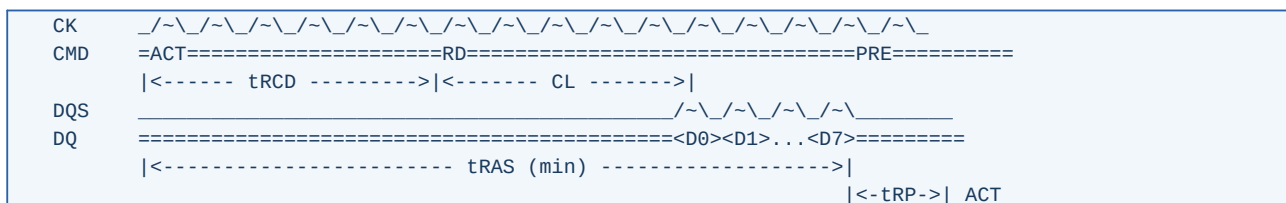
Command	Meaning
ACT (bank, row)	open a row: copy it into the bank's sense amplifiers
RD / WR (bank, column)	burst read or write from the open row (BL8 or BL16); RDA/WRA add auto-precharge
PRE / PREA	close the open row of one bank / all banks
REF (all-bank / same-bank / per-bank)	refresh a group of rows; banks must be precharged
MRS / MRW / MRR	write or read mode registers (latencies, ODT, drive, training modes)
ZQCL / ZQCS (DDR4), ZQ start/latch (DDR5, LPDDR)	calibrate driver and termination impedance
SRE / SRX, PDE / PDX	enter/exit self-refresh and power-down
DES / NOP	no command this cycle

Latencies and the key timing parameters

CAS latency CL (or RL, read latency) is the number of clocks from a READ command to the first data; **CWL** (write latency) the same for writes. A datasheet line such as 'DDR4-3200 22-22-22' means CL-tRCD-tRP = 22 clocks of 0.625 ns = 13.75 ns each. That absolute value has stayed around 13-15 ns for twenty years - DRAM got wider and faster at the pins, not faster in the array.

Parameter	Between	Why it exists	DDR4-3200 order of magnitude
tRCD	ACT -> RD/WR, same bank	sense amplifiers must develop the row	~13.75 ns (22 clk)
tRP	PRE -> ACT, same bank	bitlines must be precharged	~13.75 ns
tRAS	ACT -> PRE, same bank (min)	row must be fully restored	~32 ns
tRC	ACT -> ACT, same bank	= tRAS + tRP	~46 ns
tRRD_S / _L	ACT -> ACT, other bank (other / same bank group)	power delivery / charge pump	~2.5-5 ns / ~5-6.4 ns
tFAW	any 4 ACTs in a window	peak current limit	~20-30 ns
tWR	end of write data -> PRE	write recovery into cells	15 ns
tWTR_S / _L	end of write data -> RD	bus turnaround inside DRAM	~2.5 / 7.5 ns
tRTP	RD -> PRE	read must finish before precharge	7.5 ns
tCCD_S / _L	RD->RD or WR->WR (other / same bank group)	column path cycle time	4 clk / ~5-8 clk
tRFC	REF -> next valid command	refresh takes many rows at once	350 ns (8 Gb), 550 ns (16 Gb)
tREFI	average REF interval	retention	7.8 us (3.9 us hot)

The core DRAM timing parameters. Exact values depend on device density, speed bin and vendor; always use the JEDEC speed-bin table or the datasheet.



A closed-page read: ACT, wait tRCD, RD, data after CL (BL8 = 4 clocks of DDR data), PRE no earlier than tRAS after ACT, next ACT to that bank after tRP.

The timing checker below is the kind of passive monitor a DV team binds to a memory controller's DFI or DRAM command bus (and that memory models contain internally). It keeps per-bank state and timestamps and flags every rule it knows. Timing values are small cycle counts for readability, not a real speed bin.

```
// SDRAM command/timing checker: a passive monitor as used in DV or on a controller
// bench. Times in memory-clock cycles. Per bank: ACT->RD/WR >= tRCD, ACT->PRE >= tRAS,
// PRE->ACT >= tRP, ACT->ACT >= tRC, WR->PRE >= WL + BL/2 + tWR, and state legality.
// Across banks: ACT->ACT >= tRRD; REF needs every bank idle and precharged.
module sdram_chk #(parameter int NB = 4,
                    parameter int tRCD = 4, tRP = 4, tRAS = 10, tRC = 14,
                    parameter int tRRD = 2, tWR = 5, tCWL = 3, BL = 8, tRFC = 20) (
    input  logic clk,
    input  logic [2:0] cmd, // NOP, ACT, RD, WR, PRE, REF
    input  logic [$clog2(NB)-1:0] bank,
    input  logic [15:0] row,
    output int  errors
);
    localparam logic [2:0] NOP = 0, ACT = 1, RD = 2, WR = 3, PRE = 4, REF = 5;
    localparam int WREC = tCWL + BL/2 + tWR; // last write data -> precharge
    longint cyc = 0, t_act [NB], t_pre [NB], t_wr [NB], t_any_act = -99, t_ref = -99;
    bit is_open [NB];
    logic [15:0] open_row [NB];
    initial begin
        errors = 0;
        for (int b = 0; b < NB; b++) begin t_act[b] = -99; t_pre[b] = -99; t_wr[b] = -99; end
    end
```

```

end
function automatic void need(input bit ok, input string msg);
    if (!ok) begin $display(" cyc %3d VIOLATION: %s", cyc, msg); errors++; end
endfunction
always @(posedge clk) begin
    longint dA, dP, dW;
    cyc++;
    dA = cyc - t_act[bank]; dP = cyc - t_pre[bank]; dW = cyc - t_wr[bank];
    if (cmd != NOP) need(cyc - t_ref >= tRFC, "command inside tRFC after REF");
    case (cmd)
        ACT: begin
            need(!is_open[bank], $sformatf("ACT to open bank %0d", bank));
            need(dP >= tRP, $sformatf("ACT %0d cycles after PRE < tRP=%0d", dP, tRP));
            need(dA >= tRC, $sformatf("ACT->ACT same bank %0d < tRC=%0d", dA, tRC));
            need(cyc - t_any_act >= tRRD,
                $sformatf("ACT->ACT other bank %0d < tRRD=%0d", cyc - t_any_act, tRRD));
            is_open[bank] = 1; open_row[bank] = row; t_act[bank] = cyc; t_any_act = cyc;
        end
        RD, WR: begin
            need(is_open[bank], $sformatf("column command to idle bank %0d", bank));
            if (is_open[bank])
                need(dA >= tRCD, $sformatf("RD/WR %0d cycles after ACT < tRCD=%0d", dA, tRCD));
            if (cmd == WR) t_wr[bank] = cyc;
        end
        PRE: begin
            if (is_open[bank])
                need(dA >= tRAS, $sformatf("PRE %0d cycles after ACT < tRAS=%0d", dA, tRAS));
            need(dW >= WREC, $sformatf("PRE %0d cycles after WR < WL+BL/2+tWR=%0d", dW, WREC));
            is_open[bank] = 0; t_pre[bank] = cyc;
        end
        REF: begin
            for (int b = 0; b < NB; b++) begin
                need(!is_open[b], $sformatf("REF with bank %0d open", b));
                need(cyc - t_pre[b] >= tRP, "REF inside tRP of a PRE");
            end
            t_ref = cyc;
        end
        default: ;
    endcase
end
endmodule

```

sdrām_chk.sv - a per-bank SDRAM command-timing checker.

```

module tb;
    logic clk = 1; always #5 clk = ~clk;
    logic [2:0] cmd = 0; logic [1:0] bank = 0; logic [15:0] row = 0; int errors;
    sdrām_chk #(.NB(4)) chk (.*);
    function automatic string name(input logic [2:0] c);
        case (c) 1: return "ACT"; 2: return "RD"; 3: return "WR"; 4: return "PRE";
                5: return "REF"; default: return "NOP"; endcase
    endfunction
    int t = 0;
    task automatic issue(input int at, input logic [2:0] c, input logic [1:0] b,
        input int r = 0);
        while (t < at - 1) begin @(negedge clk); cmd = 0; t++; end
        @(negedge clk); cmd = c; bank = b; row = 16'(r); t++;
        if (c == 1) $display("cyc %3d %-3s bank %0d row %0h", t, name(c), b, r);
        else if (c == 5) $display("cyc %3d REF (all banks)", t);
        else $display("cyc %3d %-3s bank %0d", t, name(c), b);
    endtask

```

```

initial begin
    $display("-- legal sequence --");
    issue( 1, 1, 0, 'h12); // ACT b0
    issue( 3, 1, 1, 'h40); // ACT b1 (tRRD = 2 later)
    issue( 5, 2, 0); // RD b0 (tRCD = 4)
    issue( 7, 3, 1); // WR b1
    issue(11, 4, 0); // PRE b0 (tRAS = 10)
    issue(19, 4, 1); // PRE b1 (WR + 3 + 4 + 5 = 19)
    issue(24, 5, 0); // REF (all banks idle)
    $display("-- buggy sequence --");
    issue(45, 1, 2, 'h7); // ACT b2
    issue(46, 1, 3, 'h8); // ACT b3 one cycle later: tRRD
    issue(47, 2, 2); // RD b2 too early: tRCD
    issue(50, 4, 2); // PRE b2 too early: tRAS
    issue(52, 1, 2, 'h9); // ACT b2: tRP and tRC
    issue(53, 2, 0); // RD to idle bank 0
    issue(56, 1, 3, 'h1); // ACT to already-open bank 3
    @(negedge clk); cmd = 0; @(negedge clk);
    $display("violations = %0d", errors);
    $finish;
end
endmodule

```

tb_sdram.sv - a legal command sequence followed by a sequence of six illegal commands.

```

-- legal sequence --
cyc  1  ACT bank 0 row 12
cyc  3  ACT bank 1 row 40
cyc  5  RD  bank 0
cyc  7  WR  bank 1
cyc 11  PRE bank 0
cyc 19  PRE bank 1
cyc 24  REF (all banks)
-- buggy sequence --
cyc 45  ACT bank 2 row 7
cyc 46  ACT bank 3 row 8
    cyc 46  VIOLATION: ACT->ACT other bank 1 < tRRD=2
cyc 47  RD  bank 2
    cyc 47  VIOLATION: RD/WR 2 cycles after ACT < tRCD=4
cyc 50  PRE bank 2
    cyc 50  VIOLATION: PRE 5 cycles after ACT < tRAS=10
cyc 52  ACT bank 2 row 9
    cyc 52  VIOLATION: ACT 2 cycles after PRE < tRP=4
    cyc 52  VIOLATION: ACT->ACT same bank 7 < tRC=14
cyc 53  RD  bank 0
    cyc 53  VIOLATION: column command to idle bank 0
cyc 56  ACT bank 3 row 1
    cyc 56  VIOLATION: ACT to open bank 3
    cyc 56  VIOLATION: ACT->ACT same bank 10 < tRC=14
violations = 8

```

Real vvp output. The legal sequence passes silently; each illegal command is flagged in the cycle it is issued, some with two violations (the ACT at cycle 52 breaks both tRP and tRC).

INTERVIEW INSIGHT: WHY TRC MATTERS MORE THAN TRCD

Random accesses to the **same bank** are limited by tRC (~46 ns), i.e. about 22 million row activations per second per bank - hopeless for a 25 GB/s channel. Bandwidth comes from **bank parallelism** (16-32 banks, several ranks and channels) and **row-buffer locality** (many column bursts per activate). Every memory-controller feature - address mapping, reordering, page policy - exists to exploit those two.

20.2 The DDR family

Standard	Data rate (MT/s)	VDD / VDDQ	Organization highlights
DDR3	800-2133	1.5 V (DDR3L 1.35 V)	8 banks, 8n prefetch, BL8, fly-by command/clock routing on DIMMs, 64-bit channel (72 with ECC)
DDR4	1600-3200	1.2 V, VPP 2.5 V	16 banks in 4 bank groups (x4/x8), POD I/O, DBI, CA parity, write CRC, internal VrefDQ
DDR5	3200-6400 (to 8800 in later bins)	1.1 V	32 banks in 8 bank groups (x4/x8), BL16, two independent 32-bit subchannels per DIMM (40 with ECC), on-die ECC, PMIC on DIMM, DFE receivers, same-bank refresh
LPDDR4 / 4X	up to 4266	VDDQ 1.1 V / 0.6 V	two x16 channels per die, 8 banks, 6-bit multi-cycle CA bus, BL16/32, CA training
LPDDR5 / 5X	up to 6400 / 8533 (and beyond)	VDDQ 0.5 V	16 banks (bank-group mode), WCK data clock separate from CK, DVFS, link ECC (5X)
GDDR6 / 6X	~12-24 Gb/s per pin	1.35 V / 1.25 V	two independent x16 channels per device, 16 banks; 6X (Micron) uses PAM4
GDDR7	~28-32+ Gb/s per pin	lower than GDDR6	PAM3 signalling, four independent channels per device
HBM2E	~3.2-3.6 Gb/s per pin	1.2 V	1024-bit stack, 8 channels x 128 (16 pseudo-channels x 64), ~410-460 GB/s per stack
HBM3 / 3E	6.4 / ~9.2-9.8 Gb/s per pin	1.1 V core, low-swing I/O	16 channels x 64 (32 pseudo-channels), ~819 GB/s / ~1.2 TB/s per stack
HBM4	~6.4-8 Gb/s per pin	-	2048-bit stack, 32 channels, ~2 TB/s per stack

DRAM families. Speeds are JEDEC (or announced) ranges; vendors ship faster non-JEDEC parts, especially for GDDR/HBM.

- **DDR** (DIMMs) optimizes capacity and cost per bit: servers and PCs. **DDR5** splits each DIMM into two 32-bit **subchannels** so that BL16 still delivers a 64-byte cache line per burst, and moves voltage regulation onto the module.
- **LPDDR** is soldered next to (or stacked on) the SoC: narrow 16-bit channels, very low I/O voltage, deep power-down states and frequency scaling for phones, laptops and automotive.
- **GDDR** optimizes bandwidth per pin for GPUs on a PCB: very high per-pin rates, point-to-point, short traces.
- **HBM** optimizes bandwidth per watt: a **stack** of DRAM dies on a base logic die, connected by thousands of **TSVs**, placed on a **silicon interposer** (or bridge) next to the SoC so that 1024 (or 2048) data wires can run at modest per-pin rates over millimeter distances.

ON-DIE ECC IS NOT SYSTEM ECC

DDR5 (and LPDDR5) devices correct single-bit errors **inside** the chip (for example 8 check bits per 128 data bits) to cope with shrinking cells, but they do not report those corrections or protect the bus. Servers still need **side-band ECC** (the 40-bit subchannel: 32 data + 8 ECC) computed by the controller; mobile SoCs use **in-line ECC** that stores check bits in a reserved part of the address space, or link ECC on the interface.

20.3 Source-synchronous capture: DQS and training

At several Gb/s per pin, no global clock can capture data across a board, so DDR is **source-synchronous**: every byte lane (8 DQ bits, a data mask/DBI bit) travels with its own differential **strobe DQS** generated by whoever drives the data. On **reads** the DRAM sends DQS **edge-aligned** with DQ and the controller PHY delays it by about a quarter clock to sample in the middle of the eye; on **writes** the controller sends DQS **center-aligned**.

READ (from DRAM)	WRITE (to DRAM)
DQS	DQS
DQ ==X=D0=X=D1=X=D2=X=D3=X== ^ edge aligned: PHY delays DQS by ~1/4 tCK (DLL/DQS delay line) before sampling	DQ ==X=D0==X=D1==X=D2==X=D3= edges fall in the middle of each bit: the controller shifts DQS by 90 deg before sending

DQS/DQ alignment. The PHY's per-lane delay lines are set by training.

Because trace lengths, package skews and voltage/temperature differ per lane, a DDR PHY **trains** its delays at boot (and re-trains or tracks periodically):

- **Write leveling** (DDR3 onward, needed by fly-by routing where CK reaches each DRAM at a different time): the DRAM samples CK with DQS and reports the value on DQ; the PHY sweeps the DQS delay until it finds the 0→1 transition, aligning DQS to CK at every device.
- **Read gate training**: find when the DQS preamble arrives, so the PHY enables its DQS input only during the burst (DQS is undriven, and noisy, between bursts).
- **Read and write data eye training**: sweep per-bit DQ delays (and VREF) with known patterns (MPR or a DRAM-internal pattern buffer) and center each bit in its eye.
- **CA training** (LPDDR4/5, DDR5) and **Vref training** for the command/address bus and the internal receivers.
- Firmware (often a microcontroller inside the PHY running vendor code) performs the sequence; results can be saved and restored to speed up resume from suspend.

ODT (on-die termination) terminates the bus inside the receiving DRAM instead of on the board, with separate values for the target and non-target ranks during writes (RTT_WR, RTT_NOM, RTT_PARK in DDR4). **ZQ calibration** trims output drivers and ODT against an external precision 240 ohm resistor, at initialization and periodically as temperature and voltage drift.

PITFALL: TRAINING PASSES AT ROOM TEMPERATURE

DRAM interfaces that train at 25 C and fail at 85 C (or after a voltage droop) are common. Run memory tests across temperature and voltage corners, enable the PHY's periodic tracking (DQS drift compensation), schedule periodic ZQ calibration, and check that the controller's refresh rate doubles above 85 C (2x refresh / tREFI halved). 'Works on the bench' is not a sign-off criterion.

20.4 Bank groups and refresh modes

DDR4 introduced **bank groups** because the internal column path could no longer cycle at the pin data rate. Back-to-back accesses to the **same** bank group must wait tCCD_L (and tRRD_L, tWTR_L); to **different** bank groups only tCCD_S = 4 clocks, which exactly matches a BL8 burst. A controller that does not alternate bank groups loses a large fraction of peak bandwidth at high data rates.

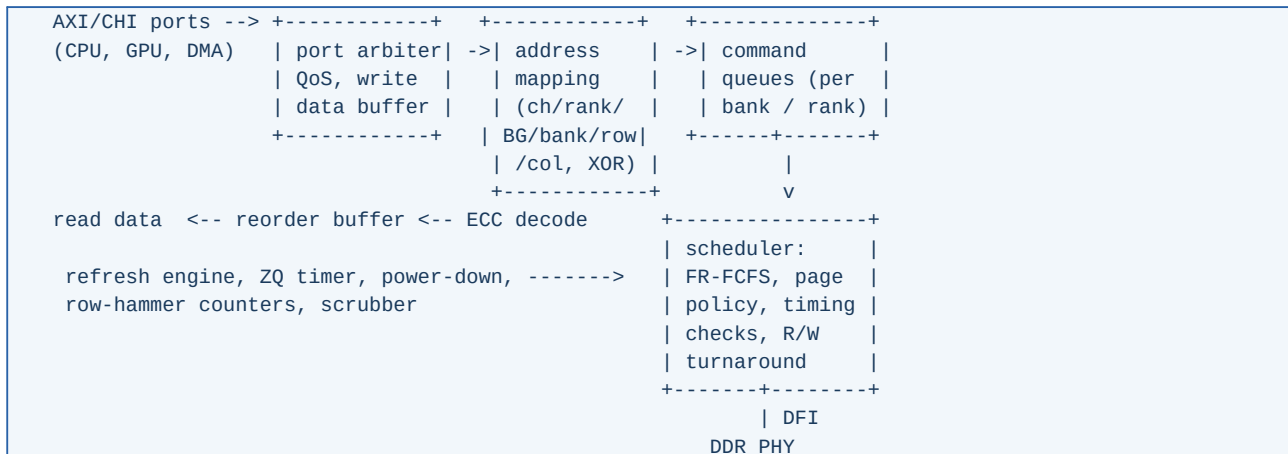
- **All-bank refresh**: every bank closed, the whole device busy for tRFC. Up to 8 REF commands may be postponed or pulled in (DDR4) to fit around traffic.
- **Fine-granularity refresh** (DDR4 2x/4x) and **same-bank refresh** (DDR5 REFsb) / **per-bank refresh** (LPDDR) refresh a subset while other banks keep working - much lower refresh stall.
- **Self-refresh** keeps data during idle with the controller and PHY powered down; the DRAM refreshes itself from an internal oscillator.
- **Row hammer** mitigation: repeated activation of one row disturbs its neighbours; DDR5 adds **RFM** (refresh management) commands and vendors add target-row refresh (TRR). Controllers count activations per bank.

refresh overhead ~ tRFC / tREFI = 350 ns / 7.8 us = 4.5% (DDR4 8 Gb)

550 ns / 3.9 us = 14% (16 Gb, hot, all-bank)

Why refresh modes matter more with every density generation.

20.5 Memory controller architecture



Main blocks of a DRAM controller. The scheduler is where performance is won or lost; the protocol engine behind it must never violate a timing rule.

- **Address mapping** decides which address bits select channel, rank, bank group, bank, row and column. Putting bank/bank-group bits low (and XOR-ing them with row bits) spreads sequential and strided streams across banks.
- **Scheduling**: the classic policy is **FR-FCFS** (first ready, first come first served): prefer commands that hit an open row, then the oldest. Real controllers add QoS classes, starvation limits and read priority.
- **Page policy**: **open page** leaves the row open hoping for another hit (good for streaming); **closed page** auto-precharges (good for random server traffic); adaptive policies predict per bank.
- **Read/write grouping**: each bus turnaround costs tens of clocks (tWTR, read-to-write gaps), so writes are buffered and drained in batches between high and low watermarks.
- **ECC and scrubbing**: SECDED over 64+8 bits (or symbol-based Chipkill-class codes on servers), background scrubbing, error logging and poisoning.
- **Low power**: power-down after idle timeouts, self-refresh entry, frequency change (LPDDR DVFS) coordinated with the PHY.

20.6 DFI: the controller-PHY interface

The **DDR PHY Interface** (DFI) is an industry specification that separates the controller from the PHY so they can come from different vendors. The controller drives DRAM commands and write data as if it were at the pins; the PHY handles the analog I/O, delay lines, DLLs, training and clock-domain crossing to the DRAM clock.

Interface group	Representative signals	Purpose
Command	dfi_address, dfi_bank / dfi_bg, dfi_cs, dfi_act_n, dfi_cke, dfi_odt, dfi_reset_n	DRAM commands, one set per phase
Write data	dfi_wrdata_en, dfi_wrdata, dfi_wrdata_mask	timed by tphy_wrlat / tphy_wrdata after the WR command
Read data	dfi_rddata_en, dfi_rddata, dfi_rddata_valid	rddata_en timed by trddata_en; PHY returns data with a valid flag
Update	dfi_ctrlupd_req/ack, dfi_phyupd_req/ack	windows where the PHY may retune delays with no traffic
Status	dfi_init_start, dfi_init_complete, dfi_freq_change / dfi_frequency	initialization and frequency change
Training	dfi_rdlvl_*, dfi_wrlvl_*, or PHY-independent training mode	controller- or PHY-driven training steps
Low power	dfi_lp_ctrl_req, dfi_lp_data_req, dfi_lp_ack	PHY power-down handshakes

DFI supports **frequency ratios** of 1:1, 1:2 and 1:4 between the controller clock and the DRAM clock: at 1:4 the controller runs at a quarter of the DRAM clock and drives four **phases** (p0-p3) of command and data per controller cycle. A DDR5-6400 PHY (3.2 GHz CK) therefore talks to an 800 MHz controller that handles four command slots per cycle - the scheduler must place commands in the right phase to respect single-clock timing rules.

20.7 Bandwidth and efficiency

$$\text{peak BW} = \text{data rate} \times \text{bus width} / 8$$

$$\text{DDR4-3200 x64:} \quad 3200\text{e6} \times 64 / 8 = 25.6 \text{ GB/s per channel}$$

$$\text{DDR5-6400 DIMM (2 x 32):} \quad 6400\text{e6} \times 64 / 8 = 51.2 \text{ GB/s per DIMM}$$

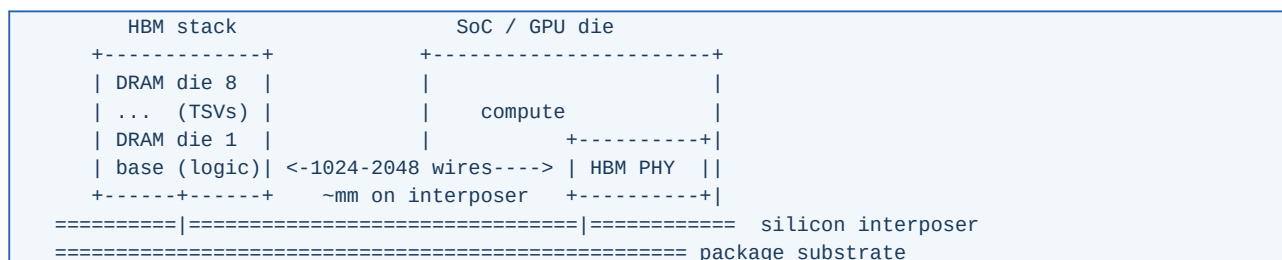
$$\text{LPDDR5X-8533 x64:} \quad 8533\text{e6} \times 64 / 8 = 68.3 \text{ GB/s}$$

$$\text{HBM3 stack (1024 x 6.4):} \quad 6.4\text{e9} \times 1024 / 8 = 819 \text{ GB/s}$$

Peak bandwidth. Sustained efficiency is typically 60-90% of peak.

Efficiency losses come from refresh (a few % to over 10%), row misses (tRP + tRCD per miss when a row must be changed), bank conflicts (tRC), tFAW limits for random traffic, read-write turnarounds, and rank switching. Streaming reads with good locality reach 85-90%; random 64-byte accesses across a large footprint may fall well below 50%. Architecture teams model this with cycle-accurate DRAM simulators (for example DRAMSim-style or Ramulator-style models) before choosing the number of channels.

20.8 HBM: stacks, TSVs and interposers



HBM is a 2.5D package: the stack and the SoC sit side by side on an interposer (CoWoS-style) or are joined by an embedded bridge.

- Each stack has 4 to 16 DRAM dies connected by TSVs to a base die that contains the PHY-side I/O, test logic and (in HBM4) potentially customer logic.
- The interface is divided into independent channels (and pseudo-channels) - essentially many narrow DRAMs in parallel - so the controller is an array of channel controllers.
- Microbump connections are too fine to probe, so HBM includes an IEEE 1500 test port (Chapter 23), lane repair (spare data lanes) and, in HBM3, ECC options.
- Ownership: the PHY is a hard macro co-designed with the package team; the interposer routing is a signal-integrity sign-off item; the RTL team owns the controller array and its NoC connection.

20.9 Flash interfaces

Raw NAND: ONFI and Toggle

NAND flash is organized as **pages** (typically 4-16 KB plus a spare area for ECC) grouped into **blocks** (hundreds of pages). Reads and programs are per page, erases per block, and a page can only be programmed after its block is erased. The interface is an 8-bit bidirectional bus, standardized by **ONFI** (Open NAND Flash Interface) and, in a compatible variant, by **Toggle DDR**.

Signal	Function
CE#	chip enable (one per die/target)
CLE / ALE	command latch enable / address latch enable
WE# / RE# (RE_t/RE_c)	write / read strobes (differential RE in fast modes)
DQ[7:0], DQS (DQS_t/c)	data bus and source-synchronous strobe (NV-DDR modes)
R/B#	ready/busy (open drain)
WP#	write protect

Operation	Command cycles	Typical time
Page read	00h, 5 address cycles, 30h ... wait tR ... data out	tR ~ 25-100 us (SLC fastest)
Page program	80h, address, data in, 10h ... wait tPROG	tPROG ~ 0.2-2 ms
Block erase	60h, 3 row-address cycles, D0h ... wait tBERS	a few ms
Read status	70h -> status byte (ready, fail)	-
Reset / Read ID / features	FFh / 90h / EFh (set) EEh (get)	-

Interface speed grew from asynchronous SDR (tens of MT/s) through **NV-DDR**, **NV-DDR2** and **NV-DDR3** to more than 2000 MT/s in recent ONFI 5.x / Toggle generations, with ODT, ZQ and training - the same techniques as DRAM. Error correction moved from **BCH** codes (tens of bits correctable per 1 KB) to **LDPC** with soft-decision reads for TLC and QLC. A NAND controller (inside an SSD, eMMC or UFS device) therefore contains a multi-channel ONFI PHY, a strong ECC engine and a **flash translation layer** (FTL) in firmware for wear-leveling, garbage collection and bad-block management.

Managed flash: eMMC, SD and UFS

Interface	Bus	Modes / speed	Protocol notes
eMMC 5.1	CLK, CMD, DAT[7:0], Data Strobe	legacy 26 MHz, HS 52 MHz SDR/DDR, HS200 (200 MB/s), HS400 (400 MB/s)	command/response on CMD with CRC7, data blocks with CRC16; tuning for HS200
SD	CLK, CMD, DAT[3:0]	Default 12.5 MB/s, High Speed 25 MB/s, UHS-I SDR104 104 MB/s (1.8 V), UHS-II/III LVDS lanes	same command set as MMC in spirit; SPI mode for simple hosts
SD Express	PCIe lanes on extra pins	PCIe 3.0 x1 (~985 MB/s) to PCIe 4.0 x2	NVMe protocol
UFS 3.x / 4.x	MIPI M-PHY, 2 lanes each way	HS-Gear 4 (11.6 Gb/s/lane), Gear 5 (23.3 Gb/s/lane): UFS 4.0 ~4 GB/s class	MIPI UniPro link layer; SCSI commands in UPIUs; command queue
NVMe	PCIe (Chapter 18)	Gen3-Gen5 x4 typical	submission/completion queues in host memory, doorbells, up to 64K queues

An SoC normally contains an **SD/eMMC host controller** (the SD Association's SDHCI register model is standard, with ADMA descriptor DMA) and, in phones, a **UFS host controller** (UFSHCI) with an M-PHY and UniPro stack. The storage device itself contains its own controller, NAND and FTL; the SoC only speaks the command protocol. Tuning (finding the sampling phase for HS200/SDR104) and voltage switching (3.3 V to 1.8 V signalling) are the usual bring-up pain points.

20.10 Who owns what

Area	Owner	Typical deliverables
DDR PHY, I/O, DLLs, training firmware	PHY vendor / analog team	hard macro, timing models, training code, SI guidelines
Memory controller, DFI integration, NoC ports	RTL team	configuration (ranks, mapping, QoS), ECC, performance counters
Package, board, interposer	package / SI / PI teams	channel simulation, power integrity, HBM interposer routing
DV	DV team	JEDEC memory models with timing checks, controller traffic tests, DFI protocol checkers
Boot firmware	firmware team	DRAM init and training sequence, margining tools, SPD read over I2C

20.11 Summary

- DRAM is a command-driven array: ACT opens a row, RD/WR move bursts of columns, PRE closes it, REF keeps it alive. tRCD, tRP, tRAS, tRC, tRRD, tFAW, tWR and tRFC govern what may happen when.
- DDR4/DDR5 target capacity, LPDDR power, GDDR per-pin speed, HBM bandwidth per watt via wide stacked interfaces; DDR5 uses two 32-bit subchannels, BL16 and on-die ECC.
- Data is source-synchronous with DQS per byte lane; write leveling, read gate and eye training, ODT and ZQ calibration make multi-Gb/s capture work.
- Controllers win bandwidth with bank parallelism and row locality: address mapping, FR-FCFS scheduling, page policy, write batching, bank-group alternation and smart refresh.
- DFI separates controller and PHY, with phase-based 1:2/1:4 frequency ratios.
- Flash: ONFI/Toggle raw NAND with BCH/LDPC ECC and an FTL; eMMC/SD for simple managed storage, UFS (M-PHY + UniPro) for phones, NVMe over PCIe for SSDs.

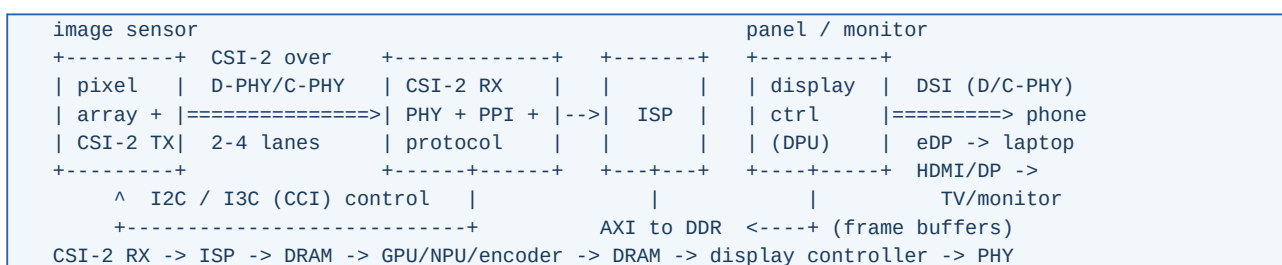
20.12 Exercises

- 1 Extend `sdram_chk` with tFAW (no more than four ACTs in any window of tFAW cycles) and tWTR; add a test sequence that violates each.
- 2 For DDR4-3200 (tCK 0.625 ns) with tRCD = tRP = CL = 22, compute the latency of a row hit, a closed-bank access and a row conflict, in ns, from command to first data.
- 3 A DDR5-4800 subchannel runs a random 64-byte read stream to one bank group with tRC = 48 ns. How many banks must be in use to reach 80% of peak bandwidth?
- 4 Design an address mapping for a 2-channel, 2-rank, 32-bank DDR5 system that spreads a 4 KB-strided stream across all banks. Show which address bits you XOR.
- 5 Compute the refresh overhead of a 16 Gb DDR4 device at 95 C with all-bank refresh, and explain how same-bank refresh changes the picture in DDR5.
- 6 An LDPC-protected TLC NAND page is 16 KB with a 2 KB spare area. What code rate does that imply, and why do QLC devices need soft-decision reads?

21. Display and Camera: MIPI D-PHY/C-PHY, CSI-2, DSI, HDMI and DisplayPort

Pixels are the largest continuous data streams most SoCs handle. A phone application processor ingests several camera streams at once, each several gigabits per second, and pushes a high-refresh panel at similar rates, all while the rest of the chip sleeps as much as it can. Two families of interfaces dominate. For the short, power-critical links inside a device, the MIPI Alliance defines the physical layers **D-PHY** and **C-PHY** and the protocols **CSI-2** (camera) and **DSI/DSI-2** (display). For links that leave the box, or that must interoperate with any monitor or TV, **HDMI** and **DisplayPort** (plus its embedded variant **eDP**) rule. This chapter starts from the one concept they all share - video timing - then works up the MIPI stack and finishes with the consumer display links. Along the way we build and run a video timing generator, a CSI-2 packet-header ECC encoder/corrector and an HDMI TMDS encoder, each cross-checked against a Python model.

21.1 Where display and camera links sit in an SoC



The camera-to-display data path of a typical application processor.

The camera side is **receive**: a CSI-2 receiver deserializes lanes, reassembles packets, checks ECC and CRC, and writes pixel lines either to an image signal processor (ISP) or straight to memory. The sensor itself is configured over a separate low-speed bus - I2C or I3C, called CCI (Camera Control Interface) in the MIPI documents (see Chapters 13 and 14). The display side is **transmit**: a display processing unit (DPU) fetches frame buffers over AXI, composes layers, and a timing engine feeds a DSI, eDP, HDMI or DP transmitter.

Interface	Owner body	Typical use	PHY	Rough peak rate
CSI-2	MIPI	Camera sensor -> SoC	D-PHY, C-PHY, A-PHY	Several Gb/s per lane or trio; 4 lanes/3 trios typical
DSI / DSI-2	MIPI	SoC -> phone/tablet panel	D-PHY, C-PHY	Same PHYs as CSI-2
eDP	VESA	SoC -> laptop panel	DP main link + AUX	Up to 4 lanes at HBR3 (8.1 Gb/s) in eDP 1.4/1.5
DisplayPort	VESA	PC/monitor, USB-C Alt Mode	Main link + AUX	Up to 4 x 20 Gb/s (UHBR20) in DP 2.1
HDMI	HDMI Forum / LA	TVs, set-top boxes, consoles	TMDS or FRL	18 Gb/s (2.0), 48 Gb/s (2.1)
MIPI A-PHY	MIPI	Automotive camera/display, up to ~15 m	Asymmetric SerDes	Up to ~16 Gb/s downlink in v1.x

The display and camera interface landscape.

21.2 Video timing: the raster every link carries

Every display interface, whether it sends pixels as a continuous stream (HDMI TMDS, DSI video mode) or as packets (DisplayPort, CSI-2), is describing a **raster**: a frame is a set of lines, a line is a set of pixels, and between them there are **blanking intervals** inherited from CRT displays that needed time for the beam to fly back. The horizontal line is divided into **active** pixels, a **front porch**, a **sync pulse** and a **back porch**; the frame is divided the same way in lines. A **data-enable** (DE) signal is high only during active pixels.

one line, counted in pixel clocks (x = 0 at the first active pixel)

|<----- active ----->|<---HFP--->|<--- HSYNC --->|<---HBP--->|

DE _____

HSYNC_n _____

_____ (active-low)

|<----- H total ----->|

x: 0 HA HA+HFP +HS HT

one frame: the same structure counted in lines (V active, VFP, VSYNC, VBP)

Horizontal timing. The vertical axis has the same structure, counted in lines.

$$\text{pixel clock} = H_total \times V_total \times \text{frame rate}$$

$$640 \times 480 @ 60 : 800 \times 525 \times 59.94 \text{ Hz} = 25.175 \text{ MHz}$$

$$1080 \text{p}60 : 2200 \times 1125 \times 60 \text{ Hz} = 148.5 \text{ MHz}$$

$$2160 \text{p}60 : 4400 \times 2250 \times 60 \text{ Hz} = 594 \text{ MHz}$$

The pixel-clock equation; the totals include blanking.

Mode	H act/FP/sync/BP	V act/FP/sync/BP	Totals	Pixel clock	Sync pol.
640x480@60 (VGA)	640/16/96/48	480/10/2/33	800 x 525	25.175 MHz	-/-
1280x720@60 (CEA-861)	1280/110/40/220	720/5/5/20	1650 x 750	74.25 MHz	+/+
1920x1080@60 (CEA-861)	1920/88/44/148	1080/4/5/36	2200 x 1125	148.5 MHz	+/+
3840x2160@60 (CTA-861)	3840/176/88/296	2160/8/10/72	4400 x 2250	594 MHz	+/+

Common timings (VESA DMT / CTA-861). Reduced-blanking (CVT-RB) modes shrink the porches to save bandwidth on digital links.

The timing generator is the heartbeat of every display controller: two counters and a few comparators. Its outputs drive the pixel fetch pipeline and the transmitter. Here is a parameterized version set up for 640x480@60, with a testbench that counts two full frames and checks every number against the table.

```

module vtg #(parameter int HA = 640, HFP = 16, HS = 96, HBP = 48, // horizontal, in pixels
               parameter int VA = 480, VFP = 10, VS = 2, VBP = 33) // vertical, in lines
( input logic      pclk, rst_n,
  output logic     hsync_n, vsync_n, de, // 640x480: both syncs active-low
  output logic [11:0] x, y ); // position of the current pixel
  localparam int HT = HA + HFP + HS + HBP; // 800 pixels per line
  localparam int VT = VA + VFP + VS + VBP; // 525 lines per frame
  logic [11:0] hc, vc;
  always_ff @(posedge pclk or negedge rst_n)
    if (!rst_n) begin hc <= '0; vc <= '0; end
    else if (hc == HT-1) begin
      hc <= '0;
      vc <= (vc == VT-1) ? '0 : vc + 1'b1;
    end else hc <= hc + 1'b1;
  // active region first, then front porch, sync, back porch
  assign de = (hc < HA) && (vc < VA);
  assign hsync_n = !((hc >= HA+HFP) && (hc < HA+HFP+HS));
  assign vsync_n = !((vc >= VA+VFP) && (vc < VA+VFP+VS));
  assign x = hc; assign y = vc;
endmodule

```

A parameterized video timing generator (active region first).

```

module tb;
  logic pclk = 0, rst_n = 0; always #20 pclk = ~pclk; // 25 MHz stand-in for 25.175
  logic hs_n, vs_n, de; logic [11:0] x, y;
  vtg dut (.pclk, .rst_n, .hsync_n(hs_n), .vsync_n(vs_n), .de, .x, .y);
  int clocks, de_cnt, hs_pulses, vs_pulses, hs_width, max_hs_width, de_lines;
  logic hs_d = 1, vs_d = 1, de_d = 0;

```



```

initial begin
  repeat (2) @(negedge pclk); rst_n = 1;    // release: first sample sees x=y=0
  // count exactly two frames starting at the first pixel of frame 0
  repeat (2 * 800 * 525) begin
    clocks++; de_cnt += de;
    if (!hs_n && hs_d) hs_pulses++;
    if (!vs_n && vs_d) vs_pulses++;
    if (de && !de_d) de_lines++;
    hs_width = hs_n ? 0 : hs_width + 1;
    if (hs_width > max_hs_width) max_hs_width = hs_width;
    hs_d = hs_n; vs_d = vs_n; de_d = de;
    @(negedge pclk); // sample mid-pixel
  end
  $display("clocks=%0d (2 frames x 800 x 525)", clocks);
  $display("active pixels=%0d (2 x 640 x 480 = %0d)", de_cnt, 2*640*480);
  $display("active lines=%0d hsync pulses=%0d vsync pulses=%0d", de_lines, hs_pulses,
    vs_pulses);
  $display("hsync width=%0d pclk frame rate at 25.175 MHz = %.3f Hz", max_hs_width,
    25.175e6 / (800.0*525.0));
  $finish;
end
endmodule

```

Testbench: count clocks, DE pixels, lines and sync pulses over two frames.

```

clocks=840000 (2 frames x 800 x 525)
active pixels=614400 (2 x 640 x 480 = 614400)
active lines=960 hsync pulses=1050 vsync pulses=2
hsync width=96 pclk frame rate at 25.175 MHz = 59.940 Hz

```

Icarus Verilog output: 800 x 525 clocks per frame, 307200 active pixels, 525 HSYNC pulses and one VSYNC per frame, 96-clock HSYNC.

PITFALL: OFF-BY-ONE PORCHES AND THE WRONG SYNC POLARITY

The two classic bring-up bugs are (1) counting porches from the wrong reference - e.g. placing sync immediately after active video and forgetting the front porch - which shifts the picture by a few pixels or makes a monitor reject the mode, and (2) wrong HSYNC/VSYNC polarity. VGA uses negative syncs, the CEA/CTA HD modes positive. Always self-check a timing generator by counting, exactly as the testbench above does, and compare with the EDID detailed timing descriptor the monitor reports.

EXPERT CORNER: THE PIXEL CLOCK IS A PLL PROBLEM

25.175 MHz and 148.5 MHz (and the NTSC-derived 1000/1001 variants such as 148.35 MHz for 59.94 Hz) are not integer ratios of a typical 19.2 or 24 MHz reference. Display PLLs therefore use fractional-N synthesis, and a DPU usually has its own PLL per output so an external monitor and the internal panel can run unrelated rates. Packetized links (DP, CSI-2, DSI burst mode) decouple the link clock from the pixel clock with FIFOs, but the sink must still regenerate the pixel clock - DisplayPort sends M/N values for exactly that purpose.

21.3 MIPI D-PHY: the physical layer

D-PHY is a **source-synchronous** PHY: one **clock lane** plus one to four **data lanes** (implementations sometimes go beyond four), each a pair of wires. Its trick is that the same two wires operate in two completely different electrical modes:

- **High-Speed (HS) mode:** low-swing differential signalling, about 200 mV differential swing around a ~200 mV common mode (SLVS-400 style), terminated with 100 ohm differential at the receiver. Data is sent as bursts; the clock lane carries a **DDR clock** (clock frequency = bit rate / 2), so a 2.5 Gb/s lane uses a 1.25 GHz clock, which the transmitter places in quadrature (90 degrees) with the data.
- **Low-Power (LP) mode:** each wire is an independent, unterminated 1.2 V LVCMOS-like single-ended signal. The pair's two bits form **line states** named LP-ab, e.g. LP-11 (both high) is the **Stop** state, LP-01 and LP-10

and LP-00 are used in sequences. LP mode signals at a few tens of Mb/s at most and costs almost no static power.

A lane rests in LP-11. To send an HS burst the transmitter walks a fixed sequence of LP states, turns on HS drivers, sends a known **sync** pattern (Start-of-Transmission, SoT), the payload, and a trailer (End-of-Transmission, EoT), then returns to LP-11.

phase	Stop	HS-Rqst	HS-Prepare	HS-Zero	SoT sync	payload	HS-Trail	Stop
line	LP-11	LP-01	LP-00	HS-0	00011101	bytes	~last bit	LP-11
Dp / Dn	1.2/1.2	0/1.2	0/0	<----	HS differential,	~200 mV swing	---->	1.2/1.2
RX term.	off	off	turns on	on	on	on	on	off
timing	>TLPX	TLPX	THS-PREPARE	THS-ZERO	8 UI	8 UI/byte	THS-TRAIL	THS-EXIT
(LP-ab: a = Dp, b = Dn. The sync byte is 0xB8 sent LSB first.)								

D-PHY data-lane HS burst: LP-11 -> LP-01 -> LP-00 -> HS-0 -> sync -> data -> trail -> LP-11.

The receiver enables its termination during HS-Prepare, waits out HS-Zero (long enough for its HS receiver and deserializer to settle), and then hunts for the sync sequence **00011101** in bit-transmission order (the byte 0xB8 sent LSB first). That hunt gives it **byte alignment**: everything after the sync is a byte stream on that lane. At the end the transmitter holds the inverse of the last data bit for **HS-Trail**, then drives LP-11; the receiver discards the trailing bits it may have sampled after the true payload. The PPI (PHY-Protocol Interface) presents this to the controller as an 8- (or 16/32-) bit word per lane with **RxActiveHS**, **RxValidHS**, **RxSyncHS**-style signals, so the protocol layer never sees raw bits.

Clock lane, escape mode and bus turnaround

- **Clock lane**: continuous (always in HS) or **non-continuous** (enters LP between bursts to save power). A non-continuous clock has its own HS entry/exit sequence with parameters such as TCLK-PREPARE, TCLK-ZERO, TCLK-PRE (clock must run before data starts) and TCLK-POST (after data ends).
- **Escape mode**: an LP-mode signalling scheme entered by LP-11 -> LP-10 -> LP-00 -> LP-01 -> LP-00 followed by an 8-bit entry command sent as spaced-one-hot LP symbols. Commands select **Low-Power Data Transmission (LPDT)** - used by DSI to send commands to a panel without starting HS - **Ultra-Low Power State (ULPS)** or a remote **trigger** (e.g. a reset trigger).
- **ULPS**: the deepest lane state (LP-00 held). The PHY can switch off its PLL; waking takes a TWAKEUP of about a millisecond, so power management firmware must budget it.
- **Bus turnaround (BTA)**: DSI needs the panel to answer reads and acknowledgements, so data lane 0 is **bidirectional in LP mode**. A BTA sequence hands control of the lane to the other side. CSI-2 is unidirectional (sensor to SoC) and does not need it.
- **Deskew**: above 1.5 Gb/s (D-PHY v1.2 and later) the transmitter sends a periodic **deskew calibration** burst so the receiver can trim each lane's sampling point against the clock. Lane-to-lane skew is the D-PHY PHY team's number one closure item at high rates.

D-PHY version	Year (approx.)	Max rate per lane	Notable additions
v1.0 / v1.1	2009 / 2011	1.0 / 1.5 Gb/s	Base spec; the mobile workhorse
v1.2	2014	2.5 Gb/s	Deskew calibration
v2.0 / v2.1	2016 / 2017	4.5 Gb/s	Equalization, spread-spectrum clocking; 2.1 better channel/ESD models
v2.5	2019	4.5 Gb/s	Alternate Low Power (ALP): LP signalling replaced by HS-side signals for longer channels
v3.0 / v3.5	2021 / 2023	around 9 Gb/s / around 11 Gb/s (check the release notes)	Stronger TX/RX equalization, embedded clock options for longer reach

D-PHY generations. Rates are the maximum; most products run far below them and a channel sets the real limit.

PITFALL: TIMING PARAMETERS IN THE WRONG UNIT

D-PHY HS entry/exit parameters are specified as a mix of absolute nanoseconds and multiples of the unit interval UI (e.g. THS-PREPARE = 40 ns + 4 UI to 85 ns + 6 UI in v1.x). Driver code that programs them in byte-clock cycles must recompute them whenever the lane rate changes. A too-short HS-Zero or HS-Prepare typically shows up as occasional SoT sync errors - intermittent, rate dependent, and blamed on signal integrity for weeks.

21.4 MIPI C-PHY: three wires, three levels, embedded clock

C-PHY attacks two D-PHY limits: the dedicated clock lane (wires and skew) and the 1 bit per wire-pair efficiency. A C-PHY **lane** is a **trio** of three wires A, B, C. At every symbol each wire is driven to one of three levels (high, mid, low) with exactly one wire at each level, giving **six wire states** named +x, -x, +y, -y, +z, -z. The receiver uses three differential comparators (A-B, B-C, C-A).

wire state	A	B	C	receivers A-B	B-C	C-A
+x	high	low	mid	+	-	...
-x	low	high	mid	a symbol is the TRANSITION from the		
+y	mid	high	low	current state to one of the 5 others:		
-y	mid	low	high	3 bits = Flip, Rotation, Polarity		
+z	low	mid	high	(the 'no change' case is illegal,		
-z	high	mid	low	so every symbol carries a clock edge)		

16-bit word -> mapper -> 7 symbols ($5^7 = 78125 \geq 2^{16} = 65536$)
 efficiency = $16 / 7 = 2.28$ bits/symbol ($\log_2(5) = 2.32$ is the ceiling)

C-PHY states and symbol coding (level assignment shown schematically).

Because every symbol is a state change, the receiver recovers the clock from the data itself: **no clock lane, no lane-to-lane deskew**. The cost is analog complexity (three-level drivers, multi-comparator clock recovery) and a more intricate PCS: each burst starts with a preamble and a sync word, and the mapper converts 16-bit words into 7-symbol groups. Speeds are quoted in symbols per second: roughly 2.5 Gsym/s in v1.0, 3.5 Gsym/s in v1.2, and around 6 Gsym/s and beyond in the v2.x generation. At 2.28 bits/symbol, a 3-trio (9-wire) link at 6 Gsym/s carries about 41 Gb/s, where a 4-lane D-PHY needs 10 wires. Many SoCs ship a **combo PHY** that can be configured as D-PHY (4 data + clock = 10 wires) or C-PHY (3 trios = 9 wires) on the same pads.

KEY IDEA: D-PHY VS C-PHY IN ONE SENTENCE

D-PHY forwards a clock and sends 1 bit per pair per UI; C-PHY embeds the clock in guaranteed transitions and sends 16 bits per 7 symbols on each 3-wire trio - fewer wires, no skew budget, more analog complexity. The CSI-2/DSI protocol layers sit on either.

21.5 CSI-2: the camera protocol

CSI-2 is a layered protocol. From the bottom: the PHY; **lane management** (byte striping over lanes); the **low-level protocol** (packets, ECC, CRC); **pixel/byte packing**; and the application layer (frames, lines, virtual channels). Lane management is simple round-robin: byte 0 on lane 0, byte 1 on lane 1, and so on; the last bytes of a packet may leave some lanes idle, and each lane ends its burst independently (and the receiver merges them).

SHORT PACKET (4 bytes)	LONG PACKET (6 + WC bytes)
+-----+-----+-----+-----+	+-----+-----+-----+-----+//-----+
DataID Data field 16 b ECC	DataID WC lo WC hi ECC data CRC16
+-----+-----+-----+-----+	+-----+-----+-----+-----+//-----+
DI byte = [7:6] VC [5:0] DT	<----- packet header (PH) -----> footer
frame/line number in the data field	WC = payload byte count; CRC over payload only
one frame: FS LS line0 LE LS line1 LE ... LS lineN LE FE	
(FS/FE/LS/LE are short packets; LS/LE are optional)	
LP-11 between packets is allowed; each packet is typically one HS burst	

CSI-2 packet formats (D-PHY) and frame structure.

Every packet begins with a 32-bit **packet header**. The Data Identifier byte carries a **virtual channel** (VC) number in its two top bits and a 6-bit **data type** (DT). For short packets, the next 16 bits are data (for Frame Start/End the frame number, for Line Start/End the line number); for long packets they are the **word count** (WC), the payload length in bytes. The fourth byte is an **ECC** that protects the 24 header bits. Long packets carry a **CRC-16** footer over the payload.

Data type (hex)	Meaning	Notes
0x00 / 0x01	Frame Start / Frame End (short)	Data field = frame number
0x02 / 0x03	Line Start / Line End (short)	Optional; data = line number
0x08 - 0x0F	Generic short packets	Application specific (e.g. shutter sync)
0x10 / 0x11 / 0x12	Null / blanking / embedded 8-bit non-image data	Embedded data = sensor register dump per frame
0x18 - 0x1F	YUV formats	0x1E = YUV422 8-bit, 0x18 = YUV420 8-bit
0x20 - 0x24	RGB formats	0x22 = RGB565, 0x24 = RGB888
0x28 - 0x2F	RAW formats	0x2A = RAW8, 0x2B = RAW10, 0x2C = RAW12, 0x2D = RAW14, 0x2E = RAW16, 0x2F = RAW20
0x30 - 0x37	User-defined 8-bit data	e.g. JPEG or metadata

Selected CSI-2 data types.

Pixel packing is part of the spec. **RAW10** packs 4 pixels into 5 bytes: the first four bytes hold bits [9:2] of pixels 0..3, the fifth holds their 2 LSBs. A 3840-pixel RAW10 line is therefore 4800 bytes (WC = 4800). **RAW12** packs 2 pixels into 3 bytes. The receiver's unpacker (bytes -> pixels, often into 16-bit containers in memory) is a classic RTL block and a classic source of bugs when the line length is not a multiple of the packing group.

Virtual channels

With 2 VC bits a single link carries up to 4 independent streams - e.g. long and short HDR exposures, or embedded data and image. CSI-2 v2.0 added **VC extension** bits (on D-PHY stored in the top two ECC bits, which were previously reserved zeros) for 16 channels, and more on C-PHY, which is how aggregator/serializer chips funnel several sensors into one SoC port. The receiver demultiplexes on (VC, DT) into separate DMA contexts.

The packet-header ECC

The header ECC is a (30,24) Hamming-style code with 6 parity bits (P0..P5; P6 and P7 are zero or, from v2.0, carry VC extension). Each parity bit is the XOR of a fixed subset of the 24 header bits. The 24 columns of the parity-check matrix are distinct and all have odd weight (3 or 5), so the code is **SEC-DED**: a single bit error in the 24 data bits produces a syndrome equal to that bit's column and is corrected, a single error in a parity bit produces a weight-1 syndrome, and any double error produces a non-zero even-weight syndrome that matches no column - detected, not corrected. The spec's worked example is DI = 0x37, WC = 0x01F0 -> ECC = 0x3F.

```
// MIPI CSI-2 / DSI packet-header ECC (6 parity bits over the 24-bit header; P7=P6=0)
function automatic logic [5:0] hdr_ecc(input logic [23:0] d);
    hdr_ecc[0] = ^(d & 24'hF12CB7); // D0 1 2 4 5 7 10 11 13 16 20 21 22 23
    hdr_ecc[1] = ^(d & 24'hF2555B); // D0 1 3 4 6 8 10 12 14 17 20 21 22 23
    hdr_ecc[2] = ^(d & 24'h749A6D); // D0 2 3 5 6 9 11 12 15 18 20 21 22
    hdr_ecc[3] = ^(d & 24'hB8E38E); // D1 2 3 7 8 9 13 14 15 19 20 21 23
    hdr_ecc[4] = ^(d & 24'hDF03F0); // D4..9 16..20 22 23
    hdr_ecc[5] = ^(d & 24'hEFC000); // D10..19 21 22 23
endfunction

module csi_hdr_rx (
    input logic [31:0] hdr_in, // receive side: correct 1 error, flag 2
    output logic [23:0] hdr_out, // {ECC byte, WC[15:8], WC[7:0], DataID}
```

```

output logic      corrected, uncorrectable );
logic [5:0] syn;  logic [23:0] fix;
assign syn = hdr_ecc(hdr_in[23:0]) ^ hdr_in[29:24];
always_comb begin
    fix = '0;
    for (int b = 0; b < 24; b++)                // syndrome equals the column of bit b
        if (syn == hdr_ecc(24'(1) << b)) fix[b] = 1'b1;
end
assign hdr_out      = hdr_in[23:0] ^ fix;
// a data-bit error matches a column; an ECC-bit error gives a weight-1 syndrome
assign corrected    = (syn != 0) && ((fix != 0) || ($countones(syn) == 1));
assign uncorrectable = (syn != 0) && !corrected;
endmodule

```

Header ECC function and a receive-side single-error corrector / double-error detector.

```

P = [
    [0,1,2,4,5,7,10,11,13,16,20,21,22,23],
    [0,1,3,4,6,8,10,12,14,17,20,21,22,23],
    [0,2,3,5,6,9,11,12,15,18,20,21,22],
    [1,2,3,7,8,9,13,14,15,19,20,21,23],
    [4,5,6,7,8,9,16,17,18,19,20,22,23],
    [10,11,12,13,14,15,16,17,18,19,21,22,23]]
def ecc(h):
    e=0
    for i,bits in enumerate(P):
        v=0
        for b in bits: v^=(h>>b)&1
        e|=v<<i
    return e
cols=[sum((b in P[i])<<i for i in range(6)) for b in range(24)]
print("cols unique",len(set(cols))==24, "min weight",min(bin(c).count('1') for c in cols))
print(hex(ecc(0x01F037)))
for h in [0x000000,0x01F037,0x0A0000|0x2B, 0x780012,0x000001]:
    print(hex(h),hex(ecc(h)))

```

Python reference: the parity-bit subsets, a check that all columns are unique, and 1000 random golden vectors written for the testbench.

```

module tb;
    logic [31:0] gold [1000];                // {hdr[23:0], ecc} from the Python model
    logic [31:0] rx_in;  logic [23:0] rx_hdr;  logic corr, unc;
    csi_hdr_rx rx (.hdr_in(rx_in), .hdr_out(rx_hdr), .corrected(corr), .uncorrectable(unc));
    int mism, fixed1, bad1, det2, bad2;
    initial begin
        $readmemh("golden.hex", gold);
        $display("spec example DI=37 WC=01F0 -> ECC=%h", hdr_ecc(24'h01F037));
        foreach (gold[i]) if (hdr_ecc(gold[i][31:8]) != gold[i][5:0]) mism++;
        $display("encoder vs Python on 1000 random headers: mismatches=%0d", mism);
        foreach (gold[i]) begin
            logic [29:0] cw;  int a, b;
            cw = {gold[i][5:0], gold[i][31:8]};                // 30-bit code word
            a = $urandom_range(29);  b = (a + 1 + $urandom_range(28)) % 30;
            rx_in = {2'b00, cw ^ (30'(1) << a)};  #1;        // single-bit error
            if (corr && !unc && rx_hdr == gold[i][31:8]) fixed1++; else bad1++;
            rx_in = {2'b00, cw ^ (30'(1) << a) ^ (30'(1) << b)};  #1;  // double error
            if (unc) det2++; else bad2++;
        end
        $display("single-bit errors corrected=%0d failed=%0d", fixed1, bad1);
        $display("double-bit errors flagged  =%0d missed=%0d", det2, bad2);
        $finish;
    end
endmodule

```

Testbench: encoder vs Python, then 1000 single- and 1000 double-bit error injections.

```
spec example DI=37 WC=01F0 -> ECC=3f
encoder vs Python on 1000 random headers: mismatches=0
single-bit errors corrected=1000 failed=0
double-bit errors flagged =1000 missed=0
```

Icarus Verilog output: the spec example matches (0x3F), the encoder agrees with Python on all 1000 headers, all single errors are corrected and all double errors flagged.

The payload CRC

The long-packet footer is a CRC-16 with generator $x^{16} + x^{12} + x^5 + 1$ (the CCITT polynomial), seeded with 0xFFFF, processed LSB-first (so in software it is the reflected polynomial 0x8408) with no final inversion. The spec includes a 24-byte worked example, which our Python reference reproduces; the RTL implementation is the parallel CRC of Chapter 3, usually 8, 16 or 32 bits per clock to match the PPI width times lanes.

```
def csi2_crc16(data):
    # x^16+x^12+x^5+1, seed 0xFFFF, LSB first
    c = 0xFFFF
    for byte in data:
        c ^= byte
        for _ in range(8):
            c = (c >> 1) ^ 0x8408 if c & 1 else c >> 1
    return c
pay = bytes.fromhex("FF000002B9DCF372BBD4B85AC875C27C81F805DFFF000001")
print("payload of %d bytes -> CRC-16 = 0x%04X" % (len(pay), csi2_crc16(pay)))
pay2 = bytes.fromhex("FF000001B9DCF372BBD4B85AC875C27C81F805DFFF000001")
print("byte 3 changed 02 -> 01 -> CRC-16 = 0x%04X" % csi2_crc16(pay2))
```

Python reference for the CSI-2 payload CRC.

```
payload of 24 bytes -> CRC-16 = 0x00F0
byte 3 changed 02 -> 01 -> CRC-16 = 0x5944
```

Python output: 0x00F0 matches the worked example; a changed byte gives a completely different CRC.

PITFALL: ECC IS NOT OPTIONAL ON THE RECEIVE SIDE

A receiver that only checks ECC for errors but does not correct silently drops packets on a single-bit error; worse, one that ignores it trusts a corrupted WC and then consumes the next packet's header as payload - a whole frame lost from one flipped bit. Correct single errors, drop the packet on double errors, count both in status registers, and resynchronise at the next SoT. The CRC error, by contrast, is normally reported but the line is still delivered, because dropping a line would shift the whole image.

21.6 Bandwidth math: a 4K60 camera over CSI-2

Sizing the link is the first architecture question for any camera port. The steps are: active pixel payload, then add horizontal and vertical blanking (sensors need line time for readout; typically 5-20% overhead), packet overhead (6 bytes per line for header and footer - negligible), LP transitions between packets if the transmitter uses them, and margin. Then divide by lanes.

```
w, h, fps = 3840, 2160, 60
for name, bpp in [("RAW10", 10), ("RAW12", 12), ("YUV422-8", 16)]:
    act = w * h * fps * bpp / 1e9
    print("%-9s active payload %.2f Gb/s" % (name, act))
raw10 = w * h * fps * 10
link = raw10 * 1.20 # ~20% for blanking, packet overhead, margin
for lanes in (2, 4):
    print("D-PHY %d lanes: %.2f Gb/s per lane" % (lanes, link / lanes / 1e9))
print("C-PHY 3 trios: %.2f Gsym/s per trio (16 bits / 7 symbols)"
      % (link / 3 / (16 / 7) / 1e9))
```

Python: link sizing for 3840 x 2160 at 60 fps.

```

RAW10      active payload  4.98 Gb/s
RAW12      active payload  5.97 Gb/s
YUV422-8   active payload  7.96 Gb/s
D-PHY 2 lanes: 2.99 Gb/s per lane
D-PHY 4 lanes: 1.49 Gb/s per lane
C-PHY 3 trios: 0.87 Gsym/s per trio (16 bits / 7 symbols)

```

Python output. 20% covers blanking, SoT/EoT and margin.

A 4K60 RAW10 sensor needs about 6 Gb/s of link, i.e. about 1.5 Gb/s per lane on 4 D-PHY lanes - comfortably inside D-PHY v1.2 (2.5 Gb/s per lane) - or 3 Gb/s per lane on 2 lanes, which needs v1.2 at full rate with deskew. On C-PHY, three trios at under 1 Gsym/s suffice. The same arithmetic flags HDR: a sensor sending two exposures per frame on two VCs doubles the load. And the receiver side must be sized too: 6 Gb/s is 750 MB/s of DRAM write bandwidth per camera, which the NoC and DDR controller (Chapter 20) have to absorb alongside everything else.

PRACTICAL TIP: SIZE THE RECEIVER'S LINE BUFFER FOR THE WORST BURST

The CSI-2 receiver writes to memory through AXI whose latency can spike when the DDR is busy, while the sensor cannot be paused. The receive FIFO must absorb the longest AXI stall at the full link rate: at 750 MB/s a 10 us stall is 7.5 KB. Size it from measured worst-case interconnect latency with QoS enabled, and give the camera path real-time QoS priority - overflow here means corrupt frames, not just slow ones.

21.7 DSI and DSI-2: driving a panel

The Display Serial Interface reuses the D-PHY (DSI) or C-PHY/D-PHY (DSI-2) and a packet format nearly identical to CSI-2 (same header, ECC and CRC), but in the other direction and with a command set for the panel. Its central choice is the **operating mode**:

	Video mode	Command mode
Panel has frame memory?	No (or minimal)	Yes (a GRAM in the driver IC)
Host sends	A continuous raster: sync events/pulses, blanking and pixel packets every frame	Only updated regions, via DCS <code>write_memory_start</code> / <code>write_memory_continue</code>
Timing master	Host (like HDMI)	Panel; it reports its scan position through a tearing-effect (TE) signal or DSI packet
Power	Link active every frame	Link idle (ULPS) while the picture is static
Typical	Large/high-refresh panels	Smart watches, small or always-on displays

DSI video mode vs command mode.

Video mode has three sub-flavours: **non-burst with sync pulses** (sync start and end packets mirror HSYNC/VSNC edges exactly), **non-burst with sync events** (only sync-start packets), and **burst mode**, where pixel data is sent at a higher link rate than needed and the lanes drop to LP during the saved time. Command mode relies on the **Display Command Set (DCS)**, a standard command vocabulary: `exit_sleep_mode` (0x11), `set_display_on` (0x29), `set_column_address` (0x2A), `set_page_address` (0x2B), `write_memory_start` (0x2C), `set_tear_on` (0x35), `set_pixel_format` (0x3A). They travel in DCS short-write (DT 0x05 or 0x15 with one parameter), DCS long-write (0x39) or DCS read (0x06) packets; reads use a bus turnaround so the panel can answer on lane 0 in LP mode. Panel initialisation sequences are long lists of such commands, often sent in LPDT before HS video starts.

PITFALL: TEARING IN COMMAND MODE

If the host writes the panel's frame memory while the panel is scanning the same region, the user sees a tear line. The fix is to synchronise updates to the TE signal (start writing just after the panel's scan passes the top line and write faster than it scans). Forgetting to enable TE (`set_tear_on`) or mis-routing the TE GPIO is one of the most common display bring-up bugs.

High-resolution panels increasingly compress the stream with VESA **Display Stream Compression (DSC)** - visually lossless at around 3:1 - which DSI carries in its own packet type. The DSC encoder sits in the DPU just before the DSI controller.

21.8 HDMI: TMDS, FRL and HDCP

HDMI grew out of DVI and keeps DVI's **TMDS** (Transition-Minimized Differential Signalling) link: three data channels (blue, green, red) plus a clock channel at the character rate. Each channel sends one 10-bit character per pixel clock (for 8-bit colour), so at 1080p60 each channel runs at 1.485 Gb/s. The link is DC-coupled current-mode logic pulled up to 3.3 V at the sink. Three kinds of period alternate on the wire:

- **Video data period:** pixels, each byte TMDS-encoded 8b -> 10b.
- **Control period:** during blanking, 2 bits per channel (HSYNC and VSYNC on channel 0) sent as four special high-transition 10-bit characters that the receiver uses for character alignment.
- **Data island period** (HDMI addition): audio samples and InfoFrames (AVI, audio, vendor-specific) in packets using **TERC4** (4 bits -> 10) with BCH error correction, bracketed by guard bands.

TMDS 8b/10b encoding

TMDS 8b/10b is **not** the IBM 8b/10b of PCIe/USB (Chapter 2). It works in two stages. Stage 1 **minimises transitions**: it builds 8 bits by chaining XOR (or XNOR, chosen when the byte has many ones) of each data bit with the previous encoded bit, and records the choice in bit 8. Stage 2 **balances DC**: it keeps a running disparity counter and, if the new word would push the line further from balance, inverts bits [7:0] and sets bit 9. The receiver undoes both from bits 9 and 8. The four control characters have many transitions, so they cannot be confused with data and give the receiver an alignment reference.

```
module tmds_enc (
    // DVI 1.0 / HDMI TMDS 8b/10b encoder, one channel
    input logic clk, rst_n,
    input logic de, // 1: video data, 0: control period
    input logic [7:0] d,
    input logic [1:0] c, // {C1,C0}: HSYNC/VSYNC on the blue channel
    output logic [9:0] q );
    logic [8:0] qm; logic [3:0] n1q; logic signed [4:0] cnt, diff;
    function automatic logic [8:0] stage1(input logic [7:0] v); // transition minimisation
        logic xn; logic [7:0] m;
        xn = ($countones(v) > 4) || ($countones(v) == 4 && !v[0]);
        m = v; // m[0] = v[0]
        for (int i = 1; i < 8; i++) m[i] = xn ? ~(m[i-1] ^ v[i]) : (m[i-1] ^ v[i]);
        return {!xn, m}; // bit 8: 1 = XOR used, 0 = XNOR used
    endfunction
    assign qm = stage1(d);
    assign n1q = 4'($countones(qm[7:0]));
    assign diff = 5'(n1q) - 5'(8 - n1q); // N1 - N0 of qm[7:0]
    always_ff @(posedge clk or negedge rst_n) // stage 2: DC balance
        if (!rst_n) begin q <= '0; cnt <= '0; end
        else if (!de) begin
            cnt <= '0;
            case (c)
                2'b00: q <= 10'b1101010100; 2'b01: q <= 10'b0010101011;
                2'b10: q <= 10'b0101010100; default: q <= 10'b1010101011;
            endcase
        end else if (cnt == 0 || diff == 0) begin
```



```

    q <= {~qm[8], qm[8], qm[8] ? qm[7:0] : ~qm[7:0]};
    cnt <= qm[8] ? cnt + diff : cnt - diff;
end else if ((cnt > 0 && diff > 0) || (cnt < 0 && diff < 0)) begin
    q <= {1'b1, qm[8], ~qm[7:0]}; // invert to pull cnt back
    cnt <= cnt + (qm[8] ? 5'sd2 : 5'sd0) - diff;
end else begin
    q <= {1'b0, qm[8], qm[7:0]};
    cnt <= cnt - (qm[8] ? 5'sd0 : 5'sd2) + diff;
end
endmodule

```

A TMDS encoder following the DVI 1.0 / HDMI algorithm.

```

import random
def tmds(d, cnt):
    n1 = bin(d).count('1')
    xnor = n1 > 4 or (n1 == 4 and not (d & 1))
    qm = d & 1
    for i in range(1, 8):
        b = ((qm >> (i-1)) ^ (d >> i)) & 1
        qm |= ((b ^ 1) if xnor else b) << i
    qm8 = 0 if xnor else 1
    n1q = bin(qm).count('1'); diff = n1q - (8 - n1q)
    if cnt == 0 or diff == 0:
        q = ((qm8 ^ 1) << 9) | (qm8 << 8) | (qm if qm8 else qm ^ 0xFF)
        cnt = cnt + diff if qm8 else cnt - diff
    elif (cnt > 0 and diff > 0) or (cnt < 0 and diff < 0):
        q = (1 << 9) | (qm8 << 8) | (qm ^ 0xFF); cnt = cnt + 2*qm8 - diff
    else:
        q = (qm8 << 8) | qm; cnt = cnt - 2*(1-qm8) + diff
    return q, cnt
random.seed(7)
data = [random.getrandbits(8) for _ in range(4000)] + [0x00]*500 + [0xFF]*500 \
    + list(range(256))*4
cnt = 0
with open('tmds_gold.hex', 'w') as f:
    for d in data:
        q, cnt = tmds(d, cnt); f.write('%02x %03x\n' % (d, q))
print(len(data))

```

Python reference model of the same algorithm; it generates 6024 golden input/output pairs (random bytes, long runs of 0x00 and 0xFF, and every byte value).

```

module tb;
    localparam int N = 6024;
    logic [9:0] mem [2*N]; // pairs: data byte, expected 10-bit symbol
    logic clk = 0, rst_n = 0, de = 0; always #5 clk = ~clk;
    logic [7:0] d; logic [1:0] c = 2'b00; logic [9:0] q;
    tmds_enc dut (.clk, .rst_n, .de, .d, .c, .q);
    int mism, rd, rd_min, rd_max, run, max_run, tr, max_tr; logic last;
    initial begin
        $readmemh("tmds_gold.hex", mem);
        @(negedge clk) rst_n = 1;
        @(negedge clk) c = 2'b01; // HSYNC=1 control period
        @(negedge clk) $display("control C1C0=01 -> %b (expect 0010101011)", q);
        for (int i = 0; i < N; i++) begin
            de = 1; d = mem[2*i][7:0];
            @(negedge clk);
            if (q !== mem[2*i+1]) mism++;
            tr = 0;
            for (int b = 0; b < 10; b++) begin // LSB is sent first
                rd += q[b] ? 1 : -1;
                if (rd < rd_min) rd_min = rd; if (rd > rd_max) rd_max = rd;
                if (b > 0 && q[b] != q[b-1]) tr++;
            end
        end
    end
endmodule

```



```

        run = (i + b > 0 && q[b] == last) ? run + 1 : 1;  last = q[b];
        if (run > max_run) max_run = run;
    end
    if (tr > max_tr) max_tr = tr;
end
$display("symbols=%0d  mismatches vs Python=%0d", N, mism);
$display("running disparity of the line: min=%0d max=%0d final=%0d", rd_min, rd_max, rd);
$display("longest run of equal bits=%0d  max transitions in a symbol=%0d",
        max_run, max_tr);
$finish;
end
endmodule

```

Testbench: compare every symbol against Python and measure the line's running disparity bit by bit.

```

control C1C0=01 -> 0010101011 (expect 0010101011)
symbols=6024  mismatches vs Python=0
running disparity of the line: min=-10 max=10 final=0
longest run of equal bits=13  max transitions in a symbol=5

```

Icarus Verilog output: bit-exact against Python, and the running disparity of the serial stream stays within +/-10 and returns to 0 - DC balanced.

Note what TMDS does **not** guarantee: the longest run of equal bits (13 here) is much longer than IBM 8b/10b's 5, because stage 1 minimises transitions on purpose (fewer transitions = less EMI on a cable). The receiver's clock comes from the forwarded clock channel, so it does not need data transitions for CDR - a key difference from embedded-clock SerDes.

HDMI versions, FRL and the side channels

Version	Signalling	Max rate	Highlights
1.4	TMDS	340 MHz TMDS clock, 10.2 Gb/s total	4K30, ARC, HDMI Ethernet channel
2.0	TMDS + scrambling	600 Mchar/s, 18 Gb/s total	4K60; clock channel at 1/40 of the character rate above 340 MHz; SCDC registers
2.1	FRL (Fixed Rate Link)	Up to 4 lanes x 12 Gb/s = 48 Gb/s	3 or 4 lanes at 3/6/8/10/12 Gb/s, 16b/18b coding, RS FEC, the clock channel becomes a 4th data lane; 4K120, 8K60 (with DSC)
2.2	FRL	Up to 96 Gb/s (announced 2025)	Higher-bandwidth cables

HDMI generations.

Besides the fast lanes the connector carries: **DDC**, an I2C bus (100 kHz) used to read the sink's **EDID** at address 0x50 and, from HDMI 2.0, the **SCDC** status/control registers at 0x54; **HPD** (hot-plug detect); **+5 V**; and **CEC**, a slow single-wire bus for remote-control commands. **HDCP** encrypts the content: HDCP 1.4 uses a legacy 56-bit key scheme, HDCP 2.2/2.3 uses RSA-based authentication, a locality check (round-trip time) and AES-128 in counter mode. HDCP keys live in secure storage (OTP/secure enclave) and the cipher sits between the DPU and the TMDS/FRL encoder (Chapter 26 covers link encryption more broadly).

21.9 DisplayPort and eDP

DisplayPort was designed as a **packetized** link from the start. The **main link** has 1, 2 or 4 AC-coupled differential lanes with an **embedded clock** (no clock lane). Pixel data, audio and secondary packets are mapped into **transfer units (TUs)** and striped across lanes; the source also sends **M/N** values from which the sink regenerates the pixel clock from the link clock. Beside the main link run the **AUX channel** and **HPD**.

Link rate	Per-lane rate	Coding	4-lane payload	Spec
RBR	1.62 Gb/s	8b/10b	5.18 Gb/s	DP 1.0
HBR	2.7 Gb/s	8b/10b	8.64 Gb/s	DP 1.0/1.1

Link rate	Per-lane rate	Coding	4-lane payload	Spec
HBR2	5.4 Gb/s	8b/10b	17.28 Gb/s	DP 1.2
HBR3	8.1 Gb/s	8b/10b	25.92 Gb/s	DP 1.3/1.4
UHBR10 / 13.5 / 20	10 / 13.5 / 20 Gb/s	128b/132b	about 38.7 / 52.4 / 77.6 Gb/s	DP 2.0/2.1

DisplayPort link rates (payload excludes coding overhead only).

- **AUX channel:** a half-duplex, AC-coupled differential pair at about 1 Mb/s using Manchester-II coding. The source is the master; it reads/writes the sink's **DPCD** register space (capabilities, link status, training requests) with native AUX transactions and tunnels I2C (for EDID and MCCS) through I2C-over-AUX. Its 20-bit address and burst-of-16-bytes limit are familiar to anyone who has written a DP driver.
- **Link training:** the source cannot know the channel, so it negotiates. Phase 1, **clock recovery**, sends training pattern TPS1 (a D10.2 clock-like pattern) while the sink reports lock per lane and requests voltage-swing and pre-emphasis levels through DPCD. Phase 2, **channel equalization**, sends TPS2/TPS3/TPS4 (TPS4 is scrambled, required at HBR3) until symbol lock, inter-lane alignment and EQ are done. On failure the source lowers the link rate or lane count and retries. UHBR (128b/132b) has its own LANEx_EQ / CDS sequence.
- **MST (Multi-Stream Transport)**, DP 1.2: the link is divided into a Multi-Stream Transport Packet of 64 time slots; each stream gets slots, and branch devices (hubs, daisy-chained monitors) route them. A sideband message protocol over AUX discovers the topology.
- **eDP (embedded DP):** the laptop panel variant. It drops hot-plug assumptions and adds power features: **Panel Self Refresh (PSR/PSR2)** - the panel keeps a copy of the frame and the source shuts down the main link while the image is static - **ALPM** and fast link training shortcuts, plus backlight control through DPCD.
- **DSC:** DP 1.4 and later carry VESA DSC-compressed streams, which is how 8K60 or 4K240 fit through HBR3. **USB-C Alt Mode** carries DP main-link lanes over the USB-C high-speed pairs (Chapter 17), and USB4 tunnels DP packets.

```

source                                AUX (native / I2C-over-AUX)                                sink
1. HPD rises <----- HPD
2. read DPCD caps (max rate, lanes) ----->
3. write LINK_BW_SET, LANE_COUNT_SET, TRAINING_PATTERN_SET = TPS1 ---->
   main link: TPS1 on all lanes =====> CDR locks?
4. poll LANE_STATUS (CR_DONE) <----- adjust request
   apply requested swing / pre-emphasis, repeat until CR_DONE or give up
5. TPS2/3/4 =====> poll EQ_DONE, SYMBOL_LOCKED, ALIGNED
6. TRAINING_PATTERN_SET = 0 ; start video (on failure: lower rate/lanes, retry)

```

DisplayPort 8b/10b link training in outline.

EXPERT CORNER: WHO OWNS THE DISPLAY LINK

The **PHY/analog team** owns the SerDes, swing/pre-emphasis tables and the AUX transceiver. The **RTL team** owns the timing engine, the DP/HDMI link layer (TU packing, secondary-data packets, scrambler, encoder), the CSI-2/DSI protocol cores and the DSC codec. **Firmware/driver** owns link training, EDID/DPCD parsing, mode selection, HDCP and PSR policy - link training is software on almost every SoC. **DV** buys VIP for DP/HDMI/CSI-2/DSI (Chapter 25) because the protocols are large and compliance (VESA CTS, HDMI CTS, MIPI conformance) is gated on those corner cases, and runs pixel-accurate end-to-end tests comparing captured frames with a reference model.

21.10 Integration and verification checklist

Display / camera port bring-up checklist

- ☐ Link budget computed for the worst mode (resolution x fps x bpp + blanking + margin) and the matching DRAM bandwidth reserved with real-time QoS.
- ☐ D-PHY/C-PHY timing parameters recomputed per lane rate; deskew enabled above 1.5 Gb/s; lane swap/polarity-swap muxes match the board.
- ☐ CSI-2 receiver corrects single-bit header errors, drops on double, counts CRC errors, and recovers at the next Frame Start after any error (test by injection).

- ☐ Unpacker verified for line lengths that are not multiples of the packing group (RAW10: 4 pixels, RAW12: 2 pixels).
- ☐ Timing generator self-checked by counting; sync polarities match EDID.
- ☐ TMDS/8b10b/128b132b encoders compared bit-exactly with a reference model; DC balance measured.
- ☐ DSI: TE signal wired and enabled; panel init sequence sent in the correct mode (LP vs HS); BTA tested for reads.
- ☐ DP: link training retried at lower rates; HPD IRQ pulses (short pulses signalling sink events) handled separately from unplug.
- ☐ HDCP keys provisioned securely; HDCP failure does not hang the pipeline (the sink sees blank/snow content instead).

21.11 Summary

- Every display link carries a **raster**: active pixels plus front porch, sync and back porch in both directions; pixel clock = $H_total \times V_total \times fps$. A timing generator is two counters and comparators - verify it by counting.
- **D-PHY** switches each lane between low-swing HS differential bursts and 1.2 V LP single-ended states; HS bursts start LP-11 -> LP-01 -> LP-00 -> HS-zero -> sync 0xB8 and end with HS-trail. Escape mode, ULPS and BTA use LP signalling.
- **C-PHY** uses 3-wire trios, six wire states and state transitions as symbols: 16 bits per 7 symbols (2.28 bits/symbol) with an embedded clock.
- **CSI-2** sends short packets (FS/FE/LS/LE) and long packets (DI + WC + ECC header, payload, CRC-16 footer); VC/DT in the DI byte; the header ECC is SEC-DED, the payload CRC is CCITT $x^{16}+x^{12}+x^5+1$ seeded 0xFFFF.
- A 4K60 RAW10 camera needs about 5 Gb/s of payload, about 1.5 Gb/s per lane on a 4-lane D-PHY with margin.
- **DSI** is CSI-2's sibling towards the panel: video mode streams a raster; command mode writes a panel frame buffer with DCS commands and synchronises with TE.
- **HDMI** uses TMDS (8b/10b transition-minimised, DC-balanced) up to 18 Gb/s and FRL (16b/18b, FEC) up to 48 Gb/s, with DDC/EDID, SCDC, CEC, HPD and HDCP around it.
- **DisplayPort** is packetized with an embedded clock, AUX-based training, MST, eDP power features and DSC; UHBR rates use 128b/132b.

21.12 Exercises

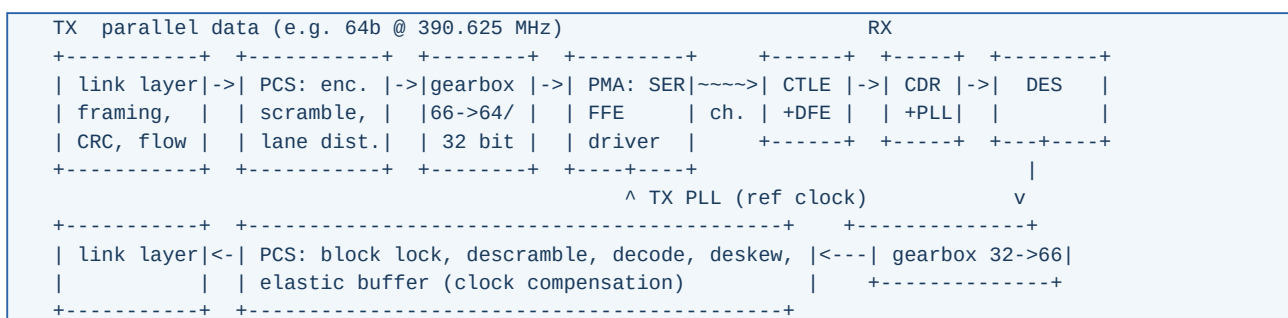
- 1 Compute the pixel clock for 2560x1440@144 with CVT reduced blanking v2 (H blank 80 pixels, V blank at least 460 us). How many HDMI 2.0 TMDS channels' worth of bandwidth is that at 8 bits per colour?
- 2 Extend the timing generator with programmable registers (APB) and a double-buffered update that only takes effect at the start of vertical blanking. Why is the double-buffering necessary?
- 3 Using the ECC function, show by exhaustive simulation that every double-bit error in the 30-bit code word is detected. What happens with a triple-bit error? Explain from the column weights.
- 4 Write the RAW10 unpacker (5 bytes in, 4 x 10-bit pixels out) for a 32-bit PPI word stream, and test it with line lengths of 3840, 3842 and 3844 pixels.
- 5 A 12-bit HDR sensor sends two exposures of 4000x3000 at 30 fps on two virtual channels. Size a 4-lane D-PHY link and the DRAM write bandwidth, and choose the minimum D-PHY version.
- 6 Modify the TMDS testbench to count the maximum |cnt| reached by the encoder's internal disparity counter. Is a 5-bit signed counter sufficient? Prove it.

22. Chip-to-Chip and Die-to-Die: SerDes Links, JESD204B/C, Aurora, Interlaken, UCIe and BoW

PCIe, Ethernet and USB (Chapters 17-19) are general-purpose, standardised and heavy: they bring enumeration, configuration space, transaction layers and big compliance suites. Many links in a system do not want any of that. An RF transceiver needs to stream ADC samples into an FPGA with **known latency**; two FPGAs need a lightweight pipe; a network processor needs a packet interface to a traffic manager with **per-channel flow control**; and a modern processor made of **chipselets** needs die-to-die links with the bandwidth of on-chip wires and the energy of a few tenths of a picojoule per bit. This chapter covers those links, from the common SerDes and PCS building blocks, through JESD204, Aurora and Interlaken, to UCIe, BoW and the advanced packages that make chipselets possible. The run example is the heart of most modern PCS layers: a 64b/66b encoder with the self-synchronous scrambler $x^{58} + x^{39} + 1$ and a block-lock state machine.

22.1 The SerDes link, reviewed

Chapter 2 introduced SerDes; here is the block diagram a link designer keeps in mind, split the way responsibilities are split in an SoC team. The **PMA** (physical medium attachment) is analog/mixed signal and belongs to the PHY team; the **PCS** (physical coding sublayer) is digital RTL; above it the protocol-specific link layer.



A SerDes link split into link layer, PCS and PMA.

- **TX equalization (FFE)** pre-distorts the signal (pre/post-cursor taps); **RX CTLE** boosts high frequencies; **DFE** cancels post-cursor ISI using past decisions.
- **CDR** recovers a sampling clock from data transitions - so the line code must guarantee transitions (8b/10b, scrambling).
- The **gearbox** adapts a 66-bit (or 67-bit) block to the PMA's 32/40/64-bit parallel width: 33 PMA words carry 32 blocks of 66 bits, so the PCS must stall one cycle in 33.
- The **elastic buffer** absorbs the ppm difference between the transmitter's and receiver's reference clocks by inserting or deleting idle/skip symbols - unless the system distributes a common reference clock, as JESD204 subclass 1 and many die-to-die links do.
- **Lane deskew**: multi-lane links send alignment markers (8b/10b /A/ or ||A|| characters, 64b/66b alignment blocks, Interlaken metaframe sync words) so the receiver can line lanes up again.

22.2 A protocol-agnostic PCS

FPGA transceivers and many ASIC SerDes IPs expose a **raw** or **protocol-agnostic PCS**: configurable encoders, scramblers, comma/block aligners and elastic buffers from which several protocols can be assembled. The main choices are summarized below.

Line code	Overhead	DC balance / transitions	Alignment	Used by
8b/10b	25%	Guaranteed: disparity control, max run 5	Comma K28.5	PCIe 1-2, USB 3.0, SATA, JESD204B, Aurora 8B/10B, DP 1.x
64b/66b	3.125%	Statistical: scrambler $x^{58}+x^{39}+1$	2-bit sync header	10/25/40/100G Ethernet, JESD204C, Aurora 64B/66B
64b/67b	4.7%	Scrambler + inversion bit bounds disparity	2-bit sync + metaframe	Interlaken
128b/130b / 128b/132b	1.5% / 3%	Scrambler	Sync header + ordered sets	PCIe 3-5, USB 3.2 (128b/132b), DP 2.x
PAM4 + FEC flits	FEC-dependent	Scrambler, Gray coding	Flit/FEC framing	PCIe 6/7, 200G/400G Ethernet, JESD204D

Line codes available in a typical configurable PCS.

22.3 64b/66b in depth: blocks, scrambler and block lock

A 64b/66b **block** is a 2-bit **sync header** followed by a 64-bit payload. Sync header **01** (in transmission order) marks a **data block** - all 8 bytes are data; **10** marks a **control block** whose first payload byte is a **block type** field (in 802.3 Clause 49 e.g. 0x1E = all idle/control characters, 0x78 = start of packet in lane 0, 0x87..0xFF = terminate variants). **00** and **11** are illegal, and that is the key to alignment: at the correct boundary every block has a sync header with exactly one transition.

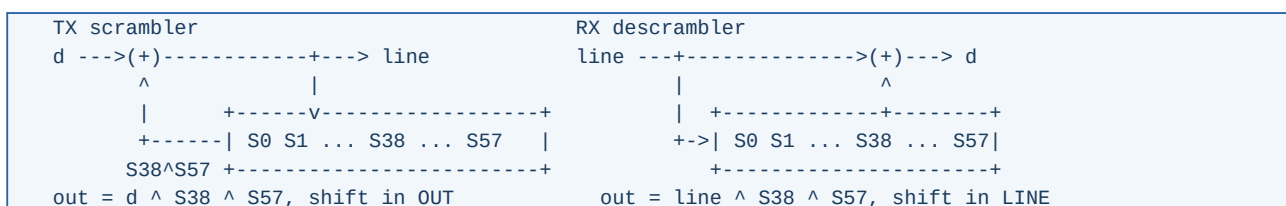
```

bit 0 first on the wire
+-----+-----+-----+-----+-----+-----+-----+-----+
| 01 | D0 D1 D2 D3 D4 D5 D6 D7          (data block)          |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 10 | type | 56 bits of control codes / data (control block) |
+-----+-----+-----+-----+-----+-----+-----+-----+
sync <----- 64-bit payload, SCRAMBLED ----->
header
(not scrambled: it must stay a guaranteed 0->1 or 1->0 transition)

```

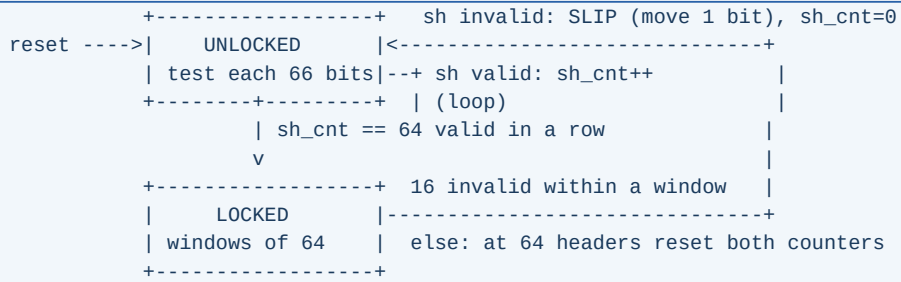
The 64b/66b block.

The payload is scrambled with the **self-synchronous** scrambler $G(x) = 1 + x^{39} + x^{58}$: each transmitted bit is the data bit XOR the line bits sent 39 and 58 bit-times earlier. The descrambler applies the same XOR using **received** line bits, so after 58 bits it is automatically in step with the transmitter - no seed exchange, no reset alignment. The price is **error multiplication**: one line-bit error corrupts the output three times (at the bit itself and 39 and 58 bits later), which CRCs such as Ethernet's CRC-32 are designed to tolerate.



Self-synchronous scrambler and descrambler (S0 = most recent line bit).

The receiver finds block boundaries with the **block lock** state machine of IEEE 802.3 Clause 49 (JESD204C, Aurora 64B/66B and others use the same idea). It tests the two bits at the candidate boundary of each 66-bit window. While unlocked, an invalid header causes a **slip** - the boundary moves by one bit - and the valid-header count restarts; 64 consecutive valid headers declare lock. While locked, the machine counts headers in windows of 64 and drops lock (and slips) if 16 of them are invalid, which tolerates the occasional bit error but reacts quickly to a real loss of alignment.



Block-lock state machine (simplified from IEEE 802.3 Clause 49).

The RTL below implements the scrambler (parameterised to be the descrambler as well) and a serial-input block-lock unit. Real designs work on 32- or 64-bit words from the PMA with a barrel-shifter gearbox, but the one-bit-per-clock version shows the algorithm plainly.

```

// 64b/66b PCS pieces (IEEE 802.3 Clause 49 style). Bit 0 is transmitted first.
module scr58 #(parameter bit DESCR = 0) ( // self-synchronous  $x^{58} + x^{39} + 1$ 
  input logic clk, rst_n, en,
  input logic [63:0] din,
  output logic [63:0] dout );
  logic [57:0] s; // s[0] = most recent line bit
  function automatic logic [121:0] step(input logic [57:0] st, input logic [63:0] d);
    logic [63:0] o;
    for (int i = 0; i < 64; i++) begin
      o[i] = d[i] ^ st[38] ^ st[57];
      st = {st[56:0], DESCR ? d[i] : o[i]}; // always shift in the LINE bit
    end
    return {st, o};
  endfunction
  logic [57:0] s_n;
  assign {s_n, dout} = step(s, din);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) s <= DESCR ? '0 : '1; // TX seed all-ones; RX needs no seed
    else if (en) s <= s_n;
endmodule

module block_lock ( // 1 bit in per clock, finds the 66-bit boundary
  input logic clk, rst_n,
  input logic bit_in,
  output logic blk_valid, // pulses when a 66-bit block is assembled
  output logic [65:0] blk, // blk[1:0] = sync header, blk[65:2] = payload
  output logic block_lock,
  output logic [15:0] slips );
  logic [65:0] sr; logic [6:0] bitcnt; logic [6:0] sh_cnt; logic [4:0] bad_cnt;
  wire [65:0] win = {bit_in, sr[65:1]}; // newest bit enters at the top
  wire sh_ok = win[0] ^ win[1]; // 01 or 10 is a legal sync header
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin
      sr <= '0; bitcnt <= '0; sh_cnt <= '0; bad_cnt <= '0;
      block_lock <= 1'b0; slips <= '0; blk_valid <= 1'b0; blk <= '0;
    end else begin
      sr <= win; blk_valid <= 1'b0;
      if (bitcnt != 65) bitcnt <= bitcnt + 1'b1;
      else begin // a candidate block boundary
        bitcnt <= '0; blk <= win; blk_valid <= 1'b1;
        if (!block_lock) begin
          if (!sh_ok) begin // SLIP: discard one bit, restart counting
            bitcnt <= 7'd127; // wraps to 0 on the next clock -> window moves by 1 bit
            sh_cnt <= '0; slips <= slips + 1'b1;
          end else if (sh_cnt == 63) begin block_lock <= 1'b1; sh_cnt <= '0; end
          else sh_cnt <= sh_cnt + 1'b1;
        end
      end
    end
endmodule

```

```

        end else begin                                // locked: 16 bad headers in 64 -> lose lock
            if (!sh_ok && bad_cnt == 15) begin
                block_lock <= 1'b0; sh_cnt <= '0; bad_cnt <= '0;
                bitcnt <= 7'd127; slips <= slips + 1'b1;
            end else if (sh_cnt == 63) begin sh_cnt <= '0; bad_cnt <= '0; end
            else begin sh_cnt <= sh_cnt + 1'b1; bad_cnt <= bad_cnt + !sh_ok; end
        end
    end
end
endmodule

```

64b/66b PCS pieces: a 64-bit-parallel self-synchronous scrambler/descrambler and a block-lock state machine.

```

M = (1 << 64) - 1
def payload(i): return (i << 32) | ((i * 0x9E3779B9) & 0xFFFFFFFF)
s = [1] * 58                                # s[0] = most recent bit; seed all ones
def scramble(d):
    global s
    o = 0
    for i in range(64):
        b = ((d >> i) & 1) ^ s[38] ^ s[57]
        o |= b << i
        s = [b] + s[:57]
    return o
with open('scr_gold.hex', 'w') as f:
    for i in range(700):
        f.write('%016x\n' % scramble(payload(i)))
print(open('scr_gold.hex').read().split()[:3])

```

Python reference: the same scrambler, bit by bit, producing golden scrambled payloads.

The testbench scrambles 700 blocks (every 16th a control block) and compares each against Python, prepends 37 junk bits so the receiver starts at an unknown offset, corrupts the sync headers of blocks 250-289 to force loss of lock, flips one payload bit of block 640, and checks every descrambled payload while locked.

```

module tb;
    localparam int NB = 700, JUNK = 37;
    logic clk = 0, rst_n = 0, rx_rst_n = 0; always #1 clk = ~clk;
    function automatic logic [63:0] payload(int i);
        return {32'(i), 32'(i) * 32'h9E3779B9};
    endfunction
    // ----- transmitter: scramble payloads, prepend sync header -----
    logic [63:0] gold[NB]; logic [63:0] tx_in, tx_out; logic tx_en = 0;
    scr58 #(.DESCR(0)) tx (.clk, .rst_n, .en(tx_en), .din(tx_in), .dout(tx_out));
    logic [65:0] line[NB];
    bit q [$];                                // serial line, bit 0 first
    // ----- receiver: block lock + descrambler -----
    logic rx_bit = 0, bv, lock; logic [65:0] blk; logic [15:0] slips; logic [63:0] rx_pl;
    block_lock bl (.clk, .rst_n(rx_rst_n), .bit_in(rx_bit), .blk_valid(bv), .blk,
        .block_lock(lock), .slips);
    scr58 #(.DESCR(1)) rx (.clk, .rst_n(rx_rst_n), .en(bv), .din(blk[65:2]), .dout(rx_pl));
    int mism, nbit, exp_i = -1, errs, good_blks; logic lock_d = 0;
    initial begin
        $readmemh("scr_gold.hex", gold);
        @(negedge clk) rst_n = 1;
        for (int i = 0; i < NB; i++) begin
            tx_in = payload(i); #0;
            if (tx_out !== gold[i]) mism++;
            line[i] = {tx_out, (i % 16 == 0) ? 2'b01 : 2'b10}; // line order: 01 data, 10 ctrl
            tx_en = 1; @(negedge clk); tx_en = 0;
        end
        $display("TX scrambler vs Python, %0d blocks: mismatches=%0d", NB, mism);
        for (int i = 0; i < JUNK; i++) q.push_back(1'($urandom)); // unknown bit offset
        for (int i = 0; i < NB; i++) begin

```



```

    if (i >= 250 && i < 290) line[i][1:0] = 2'b11;          // burst of bad headers
    if (i == 640) line[i] = line[i] ^ (66'(1) << 12);      // one line bit error
    for (int b = 0; b < 66; b++) q.push_back(line[i][b]);
end
nbit = q.size(); rx_rst_n = 1;                             // start the receiver
for (int n = 0; n < nbit; n++) begin
    rx_bit = q.pop_front(); @(negedge clk);
    if (lock != lock_d)
        $display("bit %5d: block_lock=%0d slips so far=%0d", n, lock, slips);
    lock_d = lock;
    if (bv && lock) begin                                     // aligned block, descrambled
        if (exp_i < 0) exp_i = int'(rx_pl[63:32]);          // first block after lock
        else exp_i++;
        if (rx_pl == payload(exp_i)) good_blks++;
    else begin
        errs = $countones(rx_pl ^ payload(exp_i));
        $display("block %0d: %0d bit errors after descrambling", exp_i, errs);
    end
end
    if (bv && !lock) exp_i = -1;
end
$display("blocks descrambled correctly while locked: %0d", good_blks);
$finish;
end
endmodule

```

Testbench: TX vs Python, then serial RX with offset, a header-error burst and a single line-bit error.

```

TX scrambler vs Python, 700 blocks: mismatches=0
bit 8946: block_lock=1 slips so far=37
bit 18450: block_lock=0 slips so far=38
bit 30726: block_lock=1 slips so far=103
block 640: 2 bit errors after descrambling
block 641: 1 bit errors after descrambling
blocks descrambled correctly while locked: 378

```

Icarus Verilog output.

Read the output carefully, because every number is explainable:

- The TX scrambler matches Python on all 700 blocks.
- Lock is found after exactly **37 slips** - the 37-bit junk prefix. It took about 8900 bits because each slip requires an invalid header to be observed, and a wrong boundary in scrambled data still shows a 'valid' header half the time.
- The 40 corrupted headers produce lock loss (one more slip). The machine then has to walk all the way round: 65 further slips (38 -> 103) bring it back to the same boundary, 66 positions later.
- The single line-bit error at payload bit 10 of block 640 appears as **three** output errors - bits 10 and 49 of block 640 and bit 4 of block 641 ($10 + 58 = 68 = 64 + 4$): error multiplication, exactly as predicted.
- No other block is wrong: the descrambler, fed with aligned received bits, was synchronised long before lock was declared.

PITFALL: SCRAMBLING THE SYNC HEADER OR THE WRONG BIT ORDER

The sync header must bypass the scrambler, and the scrambler must run in transmission-bit order (bit 0 first). A parallel implementation that loops over bits in the wrong direction still round-trips against its own descrambler - both ends are wrong the same way - and only fails against a third-party device. Always compare against an independent reference model or a known test vector, as the Python cross-check does here.

22.4 JESD204B and JESD204C: data converters

JEDEC JESD204 connects high-speed **ADCs and DACs** to FPGAs and ASICs (baseband processors, radar front-ends, test equipment). Before it, converters used wide parallel LVDS buses; JESD204 replaces them with a few SerDes lanes. JESD204A (2008) was capped at 3.125 Gb/s; **JESD204B** (2011) added up to 12.5 Gb/s and, above all, **deterministic latency**; **JESD204C** (2017) reaches 32 Gb/s and adds 64b/66b coding; **JESD204D** moves to PAM4 with FEC for lane rates on the order of 100 Gb/s.

Parameters and framing

Parameter	Meaning
L	Lanes per converter device (link)
M	Converters per device (an I/Q pair counts as 2)
N / N'	Converter resolution / bits per sample on the link incl. control and tail bits (e.g. N = 14, N' = 16)
S	Samples per converter per frame cycle
F	Octets per frame per lane: $F = M \times S \times N' / (8 \times L)$
K	Frames per multiframe (204B): $F \times K$ between 17 and 1024 octets, K at most 32
CS / CF / HD	Control bits per sample / control words per frame / high-density mode (samples may span lanes)
E (204C)	Multiblocks per extended multiblock

JESD204 link parameters.

$$\text{lane rate} = M \times N' \times S \times (\text{frame clock}) \times (\text{encoding}) / L = M \times N' \times F_s \times \text{enc} / L$$

example: $M = 2$, $N' = 16$, $F_s = 1 \text{ GS/s}$, $L = 4$, **8b/10b**:

$$2 \times 16 \times 1\text{e}9 \times 10/8 / 4 = 10 \text{ Gb/s per lane}, \quad F = 2 \times 1 \times 16 / 32 = 1$$

The fundamental JESD204 sizing equation.

Samples are packed into **frames** (F octets per lane per frame clock) and frames into **multiframes** (K frames) in 204B. The multiframe is the unit of alignment and of deterministic latency.

JESD204B link bring-up: CGS, ILAS, data

RX (FPGA)	SYNC~ ~~~~~ ~~~~~ ~~~~~
	(low = request sync) (high from next LMFC edge)
TX (ADC) lane	... /K/ /K/ /K/ /K/ /K/ /K/ /K/ /R/ ... /A/ /R/ /Q/ cfg /A/ ...
	Code Group Synchronization <-- ILAS: 4 multiframees ----->
	(K28.5 commas) then user data, with /F/ and /A/
	character replacement at frame and
	multiframe ends

JESD204B link start-up (8b/10b).

- **CGS (code group synchronization)**: the receiver pulls the active-low **SYNC~** signal low; the transmitter sends K28.5 commas; each lane receiver aligns its 8b/10b decoder; after the required number of good commas on all lanes, SYNC~ is released.
- **ILAS (initial lane alignment sequence)**: four multiframees, each starting with /R/ (K28.0) and ending with /A/ (K28.3); the second multiframe carries /Q/ (K28.4) and the **link configuration octets** (L, M, F, K, N, N', S, lane IDs, checksum) so the receiver can verify its settings match the transmitter's. The /A/ positions let the receiver align lanes through per-lane elastic buffers.
- **User data**: optionally scrambled with $1 + x^{14} + x^{15}$. In place of repeated octets, the transmitter substitutes /F/ (K28.7) at frame ends and /A/ at multiframe ends; the receiver uses them to monitor alignment continuously.

Deterministic latency, subclasses and SYSREF

For beamforming, phased-array radar or multi-channel instruments, the latency from antenna to logic must be the **same on every power cycle and on every device**, otherwise channels cannot be combined coherently. JESD204B achieves this by referencing everything to a **local multiframe clock (LMFC)** in each device and aligning all LMFCs:

Subclass	LMFC alignment	Deterministic latency?
0	None	No (compatible with JESD204A behaviour)
1	SYSREF : a signal distributed with the device clock, sampled by the device clock in every converter and FPGA, resets the LMFC phase	Yes - the common choice; SYSREF must meet setup/hold to the device clock at every device
2	The SYNC~ deassertion itself, sampled by the device clock	Yes, but only at lower rates because SYNC~ timing is hard to control

JESD204B subclasses.

The receiver buffers each lane's data in an elastic buffer and releases all lanes together at an LMFC edge (optionally plus a programmable **RBD**, release buffer delay). Since lane skew is absorbed inside one multiframe period, the total latency is fixed regardless of when each lane happened to lock.

JESD204C: 64b/66b and multiblocks

JESD204C keeps 8b/10b as an option and adds **64b/66b** (plus a 64b/80b mode for rate-plan flexibility). With 64b/66b there are no commas and no SYNC~ handshake for alignment: each lane uses the sync-header block lock of the previous section. 32 blocks form a **multiblock**; E multiblocks form an **extended multiblock**, and the **LEMC** replaces the LMFC for subclass 1. The sequence of sync headers across a multiblock forms a low-rate **sync header stream** carrying a pilot for extended-multiblock alignment plus, selectably, a CRC-12, a CRC-3, or FEC bits, and a command channel. The scrambler is the same $x^{58} + x^{39} + 1$ as Ethernet - the RTL above is directly reusable.

PITFALL: SYSREF IS A BOARD-LEVEL TIMING PATH

Subclass-1 determinism only holds if SYSREF is captured on the same device-clock edge at every converter and in the FPGA/ASIC. Trace-length mismatches, a clock chip with wrong SYSREF delay settings, or an ASIC capturing SYSREF on a CDC-synchronised path (adding one cycle of uncertainty) break determinism - and the symptom is a latency that differs by one LMFC period on some power cycles only. Capture SYSREF directly with the device clock, check its phase with a status register, and test across hundreds of power cycles.

22.5 Aurora 8B/10B and 64B/66B

Aurora is a lightweight, openly published link-layer protocol from Xilinx (now AMD) for point-to-point links between FPGAs or between an FPGA and an ASIC. It gives you lane bonding, framing and flow control on top of a SerDes, with none of PCIe's complexity:

- Two flavours: **Aurora 8B/10B** (8b/10b, comma alignment, K-codes for idles, start/end of frame and clock compensation) and **Aurora 64B/66B** (64b/66b blocks, scrambling, sync-header block lock).
- Single lane or multi-lane **channel bonding**, full-duplex or simplex.
- **Framing** interface (frames with start/end marks, like AXI4-Stream with TLAST) or **streaming** interface (an endless word stream).
- **Flow control**: native flow control (NFC) lets the receiver pause the far transmitter; user flow control (UFC) sends high-priority short messages that bypass data. Aurora 64B/66B optionally adds a CRC.
- Clock compensation sequences at a fixed interval handle the ppm difference between reference clocks.

Because the spec is public, Aurora is a common choice when an ASIC must talk to an FPGA prototype or test board: the ASIC team implements the (small) link layer in RTL and the FPGA side uses the vendor's IP core.

22.6 Interlaken

Interlaken (Cortina Systems and Cisco, 2006; now maintained by an industry alliance) is the standard chip-to-chip **packet** interface of networking silicon: NPUs, traffic managers, switches, FPGAs. It replaced the parallel SPI-4.2 and scales by adding lanes (1 to dozens, each typically 6-25 Gb/s or more).

- **Channels**: up to 256 logical channels (more with extensions), each with its own **flow control**, multiplexed over one link.
- **Bursts**: packets are segmented into bursts up to **BurstMax** (e.g. 256 bytes), with a **BurstMin** rule preventing tiny bursts from killing efficiency. Each burst is delimited by a **control word** carrying the channel number, SOP/EOP flags, the byte count of the last word, in-band flow control bits and a **CRC-24** covering the preceding burst.
- **64b/67b** coding: a 2-bit framing header as in 64b/66b plus a third bit that says whether the payload was **inverted** to bound the running disparity - a cheap DC-balance guarantee on top of scrambling.
- **Metaframes**: each lane periodically sends a metaframe framing layer - a **synchronization word**, a **scrambler state word** (the scrambler $x^{58} + x^{39} + 1$ is run as a synchronous, not self-synchronous, scrambler, so its state is transmitted), a **skip word** for clock compensation and a **diagnostic word** with a per-lane CRC-32 and status. Metaframe sync words also deskew the lanes.
- **Flow control**: in-band (XON/XOFF bits for a calendar of channels in each control word) or out-of-band (a separate slow interface). **Interlaken Look-Aside** is a variant for lookup coprocessors (e.g. TCAMs).

INTUITION: WHY INTERLAKEN USES A SYNCHRONOUS SCRAMBLER

A self-synchronous scrambler multiplies errors by three, which would break the per-burst CRC-24's error-detection guarantees. A synchronous (additive) scrambler does not multiply errors - one line error is one data error - but both ends must share the scrambler state. Interlaken solves that by sending the state in every metaframe. Ethernet chose the opposite trade-off because CRC-32 over a whole frame is robust to the tripled errors.

SerialLite (Altera, now Intel) is a comparable vendor protocol for FPGA-to-FPGA streaming - lightweight framing, lane bonding and optional CRC/retry over transceivers - in the same design space as Aurora.

22.7 Chiplets and die-to-die interfaces

Reticle limits, yield and cost push large designs to split into **chiplets**: several dies in one package, often in different process nodes (compute on the leading node, I/O and analog on an older one). The links between them are unlike board-level SerDes: channels are millimetres long, bumps are tens of microns apart, and the goals are **bandwidth per millimetre of die edge** (shoreline), **energy per bit** and **latency** close to on-die wires. Most die-to-die (D2D) PHYs therefore use many **parallel, single-ended, clock-forwarded** wires at moderate rates rather than a few long-reach SerDes lanes.

UCIe

UCIe (Universal Chiplet Interconnect Express), launched in 2022 by a consortium including Intel, AMD, Arm, TSMC, Samsung and others, is the leading open D2D standard. It is layered:

```
+-----+
| Protocol layer: PCIe | CXL | Streaming (user/raw protocol) |
+-----+----- FDI (flit-aware D2D interface) -----+
| Die-to-Die Adapter: link state management, parameter |
| negotiation, CRC + retry (when needed), ARB/MUX of protocols |
+-----+----- RDI (raw D2D interface) -----+
| Physical layer: logical PHY (link training, lane repair, |
| (de)scrambling) + electrical AFE; clock-forwarded, sideband |
+-----+
module = N data lanes + forwarded clock + valid + track + sideband
```

UCIe layers and interfaces.

	Standard package	Advanced package
Substrate	Organic substrate (2D)	Silicon interposer, bridge (EMIB-like), or fan-out RDL (2.5D)
Data lanes per module	16	64
Bump pitch	about 100-130 μm	about 25-55 μm
Reach	up to about 25 mm	up to about 2 mm
Data rates	4 - 32 GT/s per lane (UCle 1.x/2.0); higher rates in later revisions	same
Lane repair	Not required (width degrade instead)	Spare lanes for repair
Rough energy target	under 1 pJ/bit	around 0.5 pJ/bit or less

UCle standard vs advanced package (UCle 1.x figures; check the current spec for exact limits).

- The protocol layer maps **PCIe** and **CXL** (Chapter 18) flits directly - 68-byte and 256-byte flit formats - so a chiplet can appear as a PCIe/CXL device to software, or uses **streaming** mode for a vendor's own coherent fabric (e.g. CHI-based, Chapter 9).
- The **D2D adapter** adds CRC and retry when the raw bit error rate requires it, negotiates parameters, manages link states (active, low power L1/L2-like states) and arbitrates between protocol stacks.
- A low-speed **sideband** carries link training, register access and management messages; the main band is forwarded-clock, single-ended, DDR data plus a **valid** and a **track** signal.
- UCle 2.0 (2024) added a manageability architecture and **UCle-3D** for hybrid-bonded stacks with very fine bump pitch; later revisions push per-lane data rates further (check the consortium's current releases).

BoW, AIB, OpenHBI and others

Interface	Origin	Character
AIB (Advanced Interface Bus)	Intel; released via CHIPS Alliance	Wide parallel, clock-forwarded, single-ended; around 1-2 Gb/s per wire in AIB 1.0, higher in AIB 2.0; used in Intel FPGA tiles on EMIB
BoW (Bunch of Wires)	OCP Open Domain-Specific Architecture (ODSA)	Deliberately simple: slices of 16 data wires with forwarded clock, unterminated at low rates, a few Gb/s up to around 16 Gb/s per wire; targets standard organic packages
OpenHBI	OCP ODSA	Derived from the HBM3 PHY and its interposer channel; reuses a proven HBM-style wide parallel interface for logic-to-logic
XSR / USR SerDes	OIF (CEI-112G-XSR etc.)	Short-reach SerDes, few wires at very high rates; good for organic packages where bumps are scarce
Proprietary	e.g. AMD Infinity Fabric on-package, Apple UltraFusion, NVIDIA NV-HBI	Tuned to one product family; UCle aims to replace these at the ecosystem level

Other die-to-die interfaces.

KEY IDEA: THE THREE D2D FIGURES OF MERIT

Shoreline bandwidth density (Gb/s per mm of die edge) - determined by bump pitch, wires per module and rate. **Energy efficiency** (pJ/bit) - determined by swing, termination and SerDes vs parallel choice. **Latency** (ns) - parallel clock-forwarded links add only a few ns, far less than a PCIe SerDes hop. Advanced packaging buys density; simplicity buys energy and latency.

22.8 Advanced packaging: 2.5D and 3D

Technology	What it is	Examples
2.5D silicon interposer	Dies side by side on a passive silicon die with fine-pitch wiring and TSVs down to the substrate	TSMC CoWoS-S; GPU + HBM stacks
Embedded bridge	Small silicon bridges embedded in the organic substrate only under die edges	Intel EMIB; TSMC CoWoS-L (local silicon interconnect)
Fan-out / RDL interposer	Redistribution layers built on a moulded wafer instead of silicon	TSMC InFO, CoWoS-R
3D stacking (micro-bump)	Dies stacked face-to-face or face-to-back with micro-bumps and TSVs	HBM DRAM stacks, Intel Foveros
3D hybrid bonding	Direct copper-to-copper bonding at pitches of ~10 um and below: thousands of connections per mm ²	TSMC SoIC, AMD 3D V-Cache, Intel Foveros Direct

Packaging options for chiplets.

Packaging changes the test and integration story as much as the electrical one. Each chiplet must be **known good die (KGD)** before assembly, so D2D PHYs include loopback, pattern generators and checkers; die-to-die lanes have **repair** muxes controlled by fuses or training; and the whole package needs a test access architecture reaching every die (IEEE 1838 for 3D stacks builds on the 1149.1 and 1500 infrastructure of Chapter 23).

EXPERT CORNER: WHO OWNS A CHIPLET LINK

The **PHY/analog team** owns the AFE, clocking and bump map; the **package team** owns the channel (interposer/bridge routing, SI/PI, thermal and warpage); **RTL** owns the adapter, protocol layer and the logical PHY state machines; **DV** runs multi-die simulations with both dies' RTL (or a VIP on one side) and UCIe compliance; **DFT** owns KGD tests, lane repair and the package-level test network; **firmware** owns training, margining and repair policy. The biggest integration risk is the **bump map / lane order contract** between two dies designed by different teams (or companies) - freeze it early and check it automatically.

22.9 Summary

- A SerDes link = link layer + PCS (coding, scrambling, alignment, deskew, clock compensation) + PMA (serializer, equalization, CDR, PLL); a protocol-agnostic PCS supplies the reusable pieces.
- 64b/66b**: 2-bit unscrambled sync header (01 data, 10 control) + 64-bit payload scrambled by self-synchronous $x^{58} + x^{39} + 1$; block lock slips one bit per invalid header, locks after 64 valid, unlocks on 16 invalid in 64; one line error becomes three data errors.
- JESD204B**: 8b/10b, CGS with K28.5 and SYNC~, ILAS with configuration data, frames and multiframe, deterministic latency via LMFC alignment - SYSREF in subclass 1. **JESD204C**: 64b/66b, multiblocks, sync-header stream with CRC/FEC, up to 32 Gb/s.
- Aurora** is a light, public link layer (8B/10B or 64B/66B, framing/streaming, NFC/UFC); **Interlaken** is the packet interface of networking chips (channels, bursts, control words with CRC-24, 64b/67b, metaframes, per-channel flow control).
- UCIe** layers PCIe/CXL/streaming over a D2D adapter and a clock-forwarded parallel PHY; standard (16 lanes, ~100+ um bumps) and advanced (64 lanes, ~25-55 um) packages; BoW, AIB and OpenHBI are simpler alternatives.
- Advanced packaging (interposers, bridges, RDL, 3D micro-bump and hybrid bonding) sets bump density and therefore D2D bandwidth.

22.10 Exercises

- 1 A 16-bit DAC pair ($M = 2$, $N' = 16$, $S = 1$) runs at 3 GS/s over JESD204C with 64b/66b. Compute the lane rate for $L = 4$ and $L = 8$ and decide which fits under 24.75 Gb/s.
- 2 Modify the block-lock testbench so the junk prefix is 65 bits. How many slips occur and why? What is the worst-case lock time in blocks for a clean link?
- 3 Prove that a self-synchronous descrambler resynchronises after 58 error-free bits regardless of its initial state, and use the testbench to measure it by starting the RX descrambler with a random seed.
- 4 Add a gearbox to the block-lock design so it accepts 32 bits per clock instead of 1 (hint: keep a 97-bit history and a 6-bit offset; a slip increments the offset).
- 5 An Interlaken link has 8 lanes at 12.5 Gb/s, BurstMax = 256 bytes, and a metaframe of 2048 words. Estimate the efficiency for 64-byte and 1500-byte packets, counting 64b/67b, control words and metaframe overhead.
- 6 Compare the shoreline bandwidth density (Gb/s per mm) of a UCle advanced-package module (64 lanes, 32 GT/s, module width about 0.4 mm assumed) with a x16 PCIe Gen5 port occupying about 6 mm of die edge.

23. Debug, Test and Trace: JTAG, IEEE 1500, IJTAG, CoreSight, RISC-V Debug and Trace

Every protocol so far moves data for the application. The protocols in this chapter move data **about the chip itself**: they test the board connections and the silicon, let a debugger halt a core and read its registers, and stream a record of what the processors did. They are slow, old and unglamorous, and they are on every chip ever taped out - a device you cannot test or debug is a device you cannot ship. We start with IEEE 1149.1 **JTAG** and its descendants (1149.6, 1500, 1687), move to the processor-debug world of **Arm CoreSight** and **SWD** and the **RISC-V debug specification**, and finish with **trace** and cross-triggering. The run examples are a complete JTAG TAP controller with IDCODE, BYPASS and boundary scan, and an SWD request-packet builder cross-checked against Python.

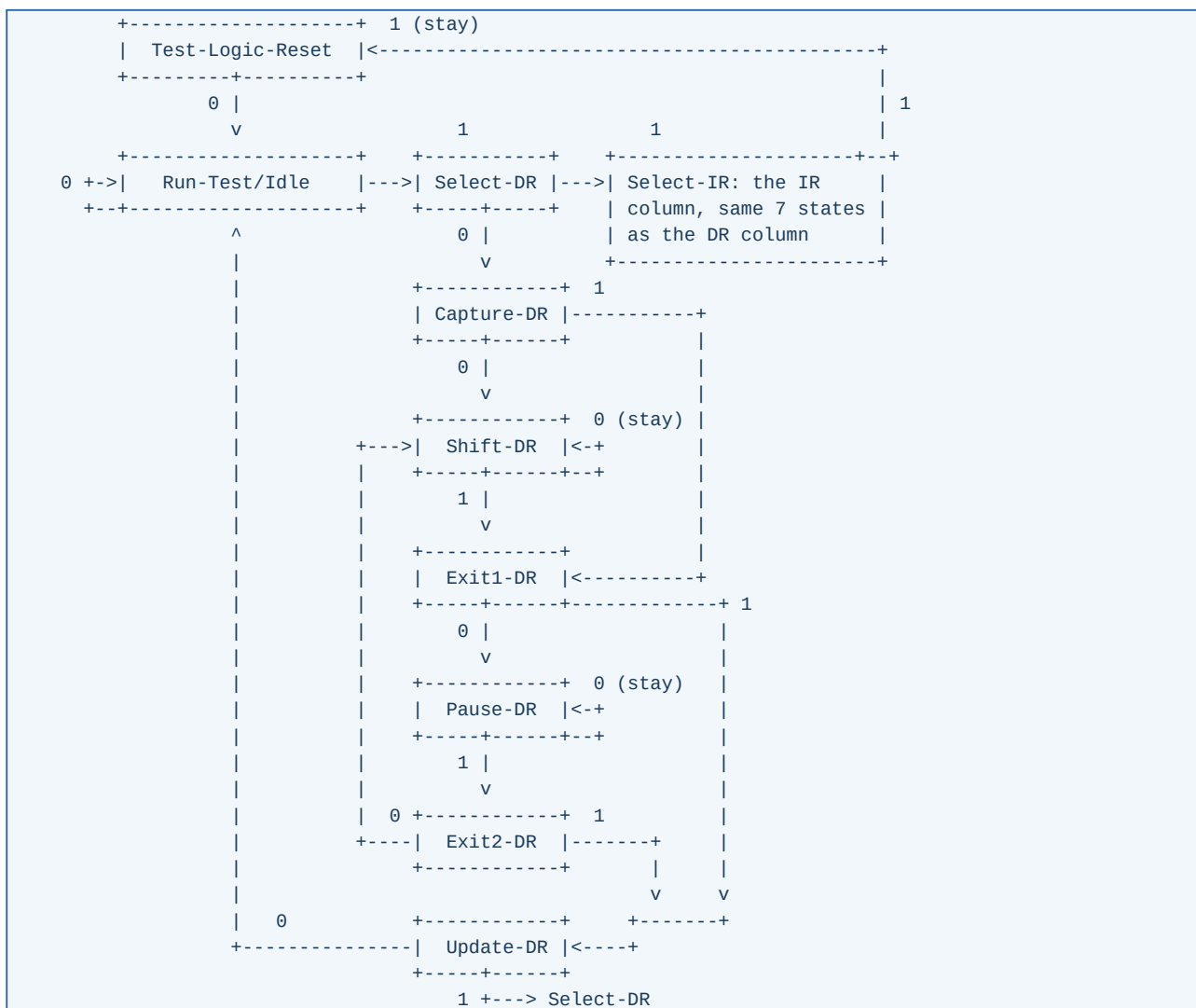
23.1 JTAG: IEEE 1149.1 boundary scan

The Joint Test Action Group standardised boundary scan in 1990 as IEEE 1149.1 (revised 2001 and 2013) to test solder joints on densely populated boards without a bed-of-nails tester. Its **Test Access Port (TAP)** turned out to be the perfect generic back door into a chip, so it now also carries processor debug, FPGA configuration, memory BIST control and fuse programming.

Pin	Direction	Function
TCK	in	Test clock. Inputs sampled on the rising edge; TDO changes on the falling edge.
TMS	in	Test mode select; steers the TAP state machine. Pulled up internally so an undriven TMS resets the TAP.
TDI	in	Serial data into the selected register (LSB first). Pulled up.
TDO	out	Serial data out; tri-stated except in Shift-IR/Shift-DR.
TRST*	in	Optional asynchronous reset of the TAP (active low). Without it, 5 TCKs with TMS = 1 reset the TAP.

The TAP pins.

The 16-state TAP controller



The TAP controller. The IR column mirrors the DR column; from any state, five TCKs with TMS = 1 reach Test-Logic-Reset.

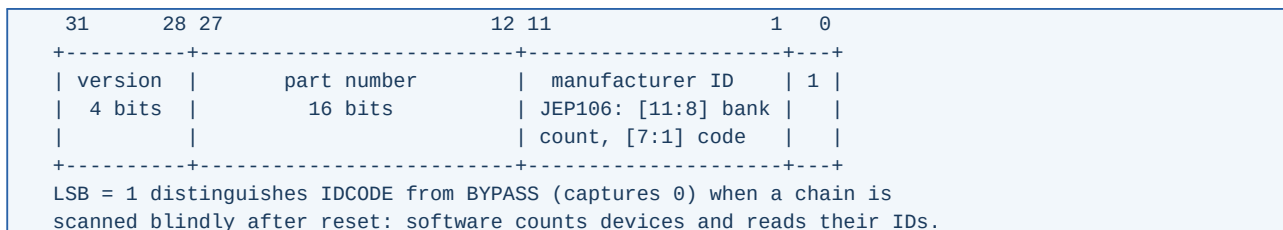
The state machine has two symmetric columns: one for the **instruction register (IR)** and one for the currently selected **data register (DR)**. **Capture** loads the register's parallel value into its shift stage, **Shift** moves bits from TDI towards TDO, **Pause** lets a slow tester stop mid-shift, and **Update** transfers the shift stage to the parallel output latch on the falling edge of TCK - so the chip's behaviour changes only at Update, never while bits ripple through. Its key property: **five TCKs with TMS high reach Test-Logic-Reset from any state**, which is why an unpowered or disconnected debugger (TMS pulled high) keeps the TAP harmlessly in reset.

Instructions and data registers

Instruction	Status in 1149.1	Selects	Effect
BYPASS	Mandatory; opcode all ones	1-bit bypass register (captures 0)	Shortens a daisy chain to one bit per inactive device
SAMPLE / PRELOAD	Mandatory (separate or shared opcodes since 2001)	Boundary-scan register	Snapshot the pins without disturbing them / load values for a later EXTEST
EXTEST	Mandatory (all-zeros opcode required in 1990, no longer in 2001)	Boundary-scan register	Pins driven from the boundary cells: test the board interconnect
IDCODE	Optional but near-universal; selected at reset when present	32-bit device ID register	Identify each device in the chain

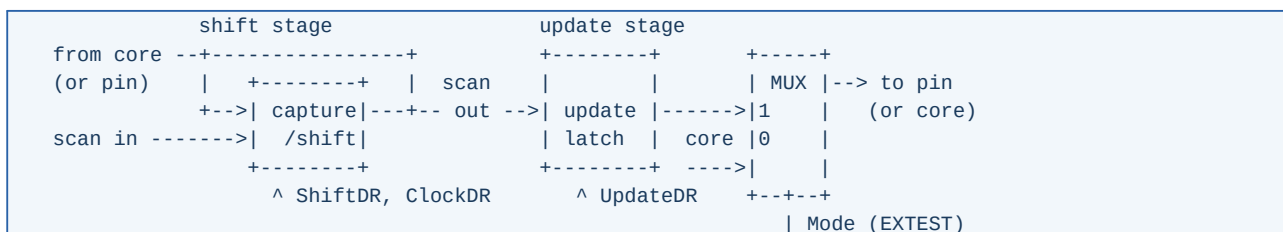
Instruction	Status in 1149.1	Selects	Effect
USERCODE, INTEST, CLAMP, HIGHZ, RUNBIST	Optional	Various	User ID, core test, safe pin states, BIST
Private (e.g. DEBUG, DTM)	Vendor defined	Vendor DRs	Debug port access, fuses, DFT controls

1149.1 instructions. The IR is at least 2 bits; its captured value must end in binary 01, which lets software measure IR lengths in a chain.



The IDCODE register format.

Boundary-scan cells and BSDL



A classic output boundary-scan cell (BC 1 style).

Each I/O has one or more cells (input, output, output-enable). A device's register layout, opcodes, IR length and cell types are published in a **BSDL** file (Boundary Scan Description Language, a VHDL subset) that board-test tools read to generate interconnect tests automatically. Getting the BSDL to match the silicon is a sign-off item: tools compare it with the netlist, and a wrong cell order breaks every customer's board test.

```
-- BSDL excerpt (illustrative)
attribute INSTRUCTION_LENGTH of demo_soc : entity is 4;
attribute INSTRUCTION_OPCODE of demo_soc : entity is
    "EXTEST (0000), IDCODE (0001), SAMPLE (0010), PRELOAD (0010), BYPASS (1111)";
attribute INSTRUCTION_CAPTURE of demo_soc : entity is "0101";
attribute IDCODE_REGISTER of demo_soc : entity is
    "0001" & "1010010110110011" & "00001010101" & "1";
attribute BOUNDARY_LENGTH of demo_soc : entity is 8;
```

What the TAP below would publish in its BSDL file.

A TAP controller in RTL

The design below implements the full 16-state controller, a 4-bit IR (capture value 0101), IDCODE, BYPASS, SAMPLE/PRELOAD and EXTEST over an 8-cell boundary register. Capture and shift happen on the rising edge of TCK; IR/boundary updates and TDO on the falling edge, as 1149.1 requires.

```

module jtag_tap #(parameter logic [31:0] IDCODE = {4'h1, 16'hA5B3, 11'h0AB, 1'b1},
                  parameter int NBSC = 8)                                // boundary-scan cells (= pins)
( input  logic tck, tms, tdi, trst_n,
  output logic tdo, tdo_en,
  input  logic [NBSC-1:0] pin_in,                                     // what the pads see
  output logic [NBSC-1:0] pin_out, input logic [NBSC-1:0] core_out,
  output logic [3:0] state_o );
localparam logic [3:0] TLR = 0, RTI = 1, SEL_DR = 2, CAP_DR = 3, SH_DR = 4, EX1_DR = 5,
  PA_DR = 6, EX2_DR = 7, UPD_DR = 8, SEL_IR = 9, CAP_IR = 10, SH_IR = 11, EX1_IR = 12,
  PA_IR = 13, EX2_IR = 14, UPD_IR = 15;

```

```

localparam logic [3:0] I_EXTEST = 4'b0000, I_IDCODE = 4'b0001, I_SAMPLE = 4'b0010,
                      I_BYPASS = 4'b1111;

logic [3:0] st, nx;
always_comb case (st)
    // the 16-state TAP controller
    TLR:    nx = tms ? TLR    : RTI;    RTI:    nx = tms ? SEL_DR : RTI;
    SEL_DR: nx = tms ? SEL_IR : CAP_DR;  CAP_DR: nx = tms ? EX1_DR : SH_DR;
    SH_DR:  nx = tms ? EX1_DR : SH_DR;  EX1_DR: nx = tms ? UPD_DR : PA_DR;
    PA_DR:  nx = tms ? EX2_DR : PA_DR;  EX2_DR: nx = tms ? UPD_DR : SH_DR;
    UPD_DR: nx = tms ? SEL_DR : RTI;    SEL_IR: nx = tms ? TLR    : CAP_IR;
    CAP_IR: nx = tms ? EX1_IR : SH_IR;  SH_IR:  nx = tms ? EX1_IR : SH_IR;
    EX1_IR: nx = tms ? UPD_IR : PA_IR;  PA_IR:  nx = tms ? EX2_IR : PA_IR;
    EX2_IR: nx = tms ? UPD_IR : SH_IR;  default: nx = tms ? SEL_DR : RTI; // UPD_IR
endcase
always_ff @(posedge tck or negedge trst_n) st <= !trst_n ? TLR : nx;
assign state_o = st;

logic [3:0] ir_sr, ir; logic [31:0] id_sr; logic by_sr;
logic [NBSC-1:0] bs_sr, bs_upd;
always_ff @(posedge tck) begin
    // capture / shift on rising TCK
    if (st == CAP_IR) ir_sr <= 4'b0101; // 1149.1: two LSBs must be 01
    if (st == SH_IR) ir_sr <= {tdi, ir_sr[3:1]};
    case (st)
        CAP_DR: begin id_sr <= IDCODE; by_sr <= 1'b0; bs_sr <= pin_in; end
        SH_DR: case (ir)
            I_IDCODE:    id_sr <= {tdi, id_sr[31:1]};
            I_SAMPLE, I_EXTEST: bs_sr <= {tdi, bs_sr[NBSC-1:1]};
            default:      by_sr <= tdi; // BYPASS and unused codes
        endcase
    default: ;
endcase
end

always_ff @(negedge tck or negedge trst_n) // update + TDO on falling TCK
    if (!trst_n) begin ir <= I_IDCODE; bs_upd <= '0; tdo <= 1'b0; tdo_en <= 1'b0; end
    else begin
        if (st == TLR)    ir <= I_IDCODE; // IDCODE is selected after reset
        if (st == UPD_IR) ir <= ir_sr;
        if (st == UPD_DR && (ir == I_SAMPLE || ir == I_EXTEST)) bs_upd <= bs_sr;
        tdo_en <= (st == SH_IR) || (st == SH_DR);
        tdo <= (st == SH_IR) ? ir_sr[0] :
            (ir == I_IDCODE) ? id_sr[0] :
            (ir == I_SAMPLE || ir == I_EXTEST) ? bs_sr[0] : by_sr;
    end
assign pin_out = (ir == I_EXTEST) ? bs_upd : core_out; // EXTEST drives the pads
endmodule

```

An IEEE 1149.1 TAP controller with IDCODE, BYPASS, SAMPLE/PRELOAD and EXTEST.

```

module tb;
    logic tck = 0, tms = 1, tdi = 0, trst_n = 0, tdo, tdo_en;
    logic [7:0] pin_in = 8'hC3, pin_out, core_out = 8'h00; logic [3:0] st;
    jtag_tap dut (.*, .state_o(st));
    logic tdo_s; // TDO sampled at rising TCK
    task automatic clk(input logic m, input logic d = 0); // one TCK period
        tms = m; tdi = d; #5 tdo_s = tdo; tck = 1; #5 tck = 0;
    endtask
    task automatic shift(input int n, input logic [63:0] din, output logic [63:0] dout);
        dout = '0; // enter from Capture-xR; leave in Exit1-xR
        for (int i = 0; i < n; i++) begin
            clk(i == n-1, din[i]);
            dout[i] = tdo_s;
        end
    endtask
endmodule

```

```

logic [63:0] r; int bad;
initial begin
    #1 trst_n = 1;
    repeat (5) clk(1); clk(0); // Test-Logic-Reset -> RTI
    clk(1); clk(0); clk(0); // Select-DR, Capture-DR, Shift-DR
    shift(32, '0, r); clk(1); clk(0); // Update-DR, RTI
    $display("IDCODE after reset = %h (LSB=%0d, manuf=%h, part=%h, ver=%h)",
            r[31:0], r[0], r[11:1], r[27:12], r[31:28]);
    clk(1); clk(1); clk(0); clk(0); // Sel-DR, Sel-IR, Cap-IR, Sh-IR
    shift(4, 64'hF, r); clk(1); clk(0); // load BYPASS
    $display("IR capture value = %b (1149.1 requires xx01)", r[3:0]);
    clk(1); clk(0); clk(0);
    shift(8, 64'hB5, r); clk(1); clk(0);
    $display("BYPASS: in 10110101 -> out %b (in << 1: one-bit delay)", r[7:0]);
    clk(1); clk(1); clk(0); clk(0); shift(4, 64'h2, r); clk(1); clk(0); // SAMPLE/PRELOAD
    clk(1); clk(0); clk(0); shift(8, 64'h5A, r); clk(1); clk(0); // capture + preload
    $display("SAMPLE: pins=%b captured=%b, preloaded 01011010", pin_in, r[7:0]);
    clk(1); clk(1); clk(0); clk(0); shift(4, 64'h0, r); clk(1); clk(0); // EXTEST
    $display("EXTEST: pads now driven with %b", pin_out);
    for (int t = 0; t < 2000; t++) begin // any state + 5 x TMS=1 -> TLR
        repeat ($urandom_range(1, 12)) clk(1'($urandom));
        repeat (5) clk(1);
        if (st != 4'd0) bad++;
    end
    $display("random walks ending in 5 x TMS=1 that missed Test-Logic-Reset: %0d/2000", bad);
    $finish;
end
endmodule

```

Testbench: read IDCODE after reset, read IR capture, test BYPASS, SAMPLE and EXTEST, and 2000 random TMS walks that must end in Test-Logic-Reset.

```

IDCODE after reset = 1a5b3157 (LSB=1, manuf=0ab, part=a5b3, ver=1)
IR capture value = 0101 (1149.1 requires xx01)
BYPASS: in 10110101 -> out 01101010 (in << 1: one-bit delay)
SAMPLE: pins=11000011 captured=11000011, preloaded 01011010
EXTEST: pads now driven with 01011010
random walks ending in 5 x TMS=1 that missed Test-Logic-Reset: 0/2000

```

Icarus Verilog output: IDCODE 0x1A5B3157 shifts out with LSB = 1, IR captures 0101, BYPASS delays by one bit, SAMPLE captures the pins, EXTEST drives the preloaded pattern, and all 2000 random walks reset.

PITFALL: TDO TIMING, CHAINS AND JTAG CLOCKS

TDO must change on the **falling** edge so the next device (or the probe) samples a stable value on the rising edge; changing it on the rising edge creates a hold race in daisy chains. TCK is an external, often slow and noisy clock: treat it as its own clock domain in CDC analysis, constrain it in STA, and synchronise every signal crossing into the functional domain. Finally, remember our own testbench bug: sampling TDO in the same time step as the falling edge read the old value - a race that real probes avoid by sampling at the rising edge, exactly as the fixed task does.

23.2 IEEE 1149.6: AC-coupled and differential nets

1149.1 EXTEST assumes a DC path: the receiver sees the level the driver holds. High-speed SerDes lanes are **AC-coupled** through series capacitors, which block DC, so a static EXTEST value decays away. IEEE 1149.6 (2003) adds **EXTEST_PULSE** and **EXTEST_TRAIN** instructions that make drivers produce edges, and special **test receivers** with hysteresis that detect those edges through the capacitor. It lets board test find opens and shorts on PCIe, Ethernet or DP lanes. The test receiver sits in the PHY - a PHY/analog-team deliverable that is easy to forget until board-test engineers ask for the BSDI extension.

23.3 IEEE 1500: embedded core test wrappers

1149.1 tests a chip's boundary; IEEE 1500 (2005) applies the same idea to **cores inside the chip**, so an IP block can be tested in isolation from the logic around it. Each core gets a **wrapper**:

- **WSP (Wrapper Serial Port)**: WSI, WSO, WRCK, WRSTN and control signals SelectWIR, CaptureWR, ShiftWR, UpdateWR - the equivalent of a TAP's internal signals, but driven by a chip-level controller (usually the TAP) rather than a state machine in the core.
- **WIR (Wrapper Instruction Register)** selects the wrapper mode.
- **WBY (Wrapper Bypass)**: 1-bit bypass, like BYPASS.
- **WBR (Wrapper Boundary Register)**: cells on every core terminal, used to isolate the core (INTEST, testing the core from its wrapper) or its surroundings (EXTEST, testing the logic between cores).
- Optional **WPP (Wrapper Parallel Port)** for high-bandwidth scan test, and **WDRs** for core-specific data registers. Mandatory instructions include WS_BYPASS and WS_EXTEST plus at least one core-test instruction. The wrapper is described for tools in CTL (Core Test Language, IEEE 1450.6).

In practice DFT tools insert 1500-style wrappers automatically as part of **hierarchical test** so that each core's scan patterns are generated once and reused wherever the core is instantiated.

23.4 IEEE 1687 IJTAG: instrument access networks

A modern SoC contains thousands of **embedded instruments**: memory BIST controllers, PLL and SerDes test registers, temperature and voltage sensors, fuse controllers, ring oscillators. Putting each on its own JTAG instruction does not scale. IEEE 1687 (2014), **IJTAG**, defines a **reconfigurable scan network** behind the TAP and languages to describe and use it:

```

TDI --> [SIB A] -----> [SIB B] -----> [TDR] --> TDO
      |   ^               |   ^
      v   | (segment      v   |
open:  [ MBIST ctrl TDR ]   [ SIB C ]-->[ PLL test TDR ]
                        |
                        [ thermal sensor TDR ]
SIB = Segment Insertion Bit: a 1-bit register; when set (and updated)
it splices its sub-network into the scan path, when clear it bypasses it

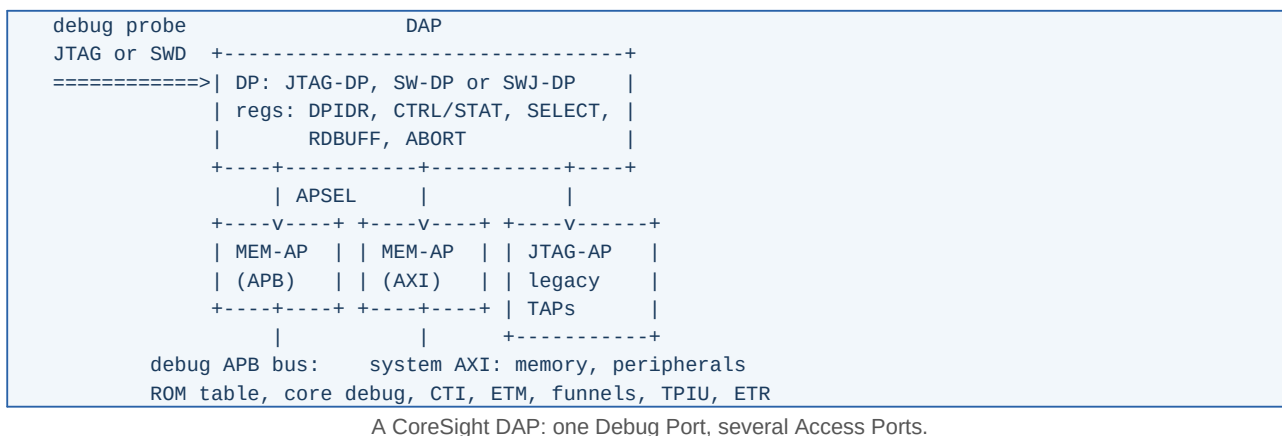
```

An IJTAG network: SIBs open and close hierarchical segments.

- **SIB (Segment Insertion Bit)**: a one-bit shift/update cell that either bypasses or inserts a sub-segment. The active scan path is only as long as the instruments you are using.
- **ICL (Instrument Connectivity Language)**: describes the network - registers, SIBs, muxes and their connections.
- **PDL (Procedure Description Language)**: describes how to operate an instrument at its own terminals: `iWrite`, `iRead`, `iApply`, `iCall`. The instrument vendor writes PDL once; a **retargeting** tool uses the ICL to translate it into TAP-level shift sequences for whatever network the SoC integrator built.

23.5 Arm CoreSight: the Debug Access Port

Processor debug needs far more than boundary scan: halting cores, reading registers and memory, setting breakpoints, controlling trace. Arm's **CoreSight** architecture standardises it. The entry point is the **Debug Access Port (DAP)** defined by the Arm Debug Interface architecture (**ADiv5**, and ADiv6 which allows APs to live in a memory map and to be nested).



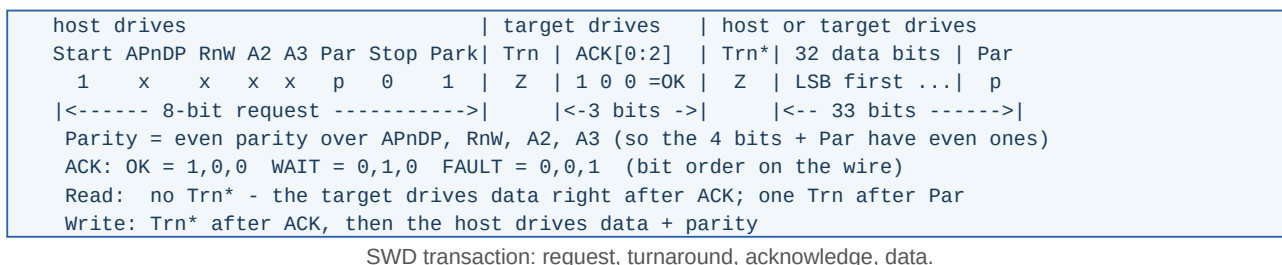
- The **Debug Port (DP)** terminates the external protocol: a **JTAG-DP** (a JTAG TAP with DPACC/APACC instructions), a **SW-DP** (Serial Wire Debug) or an **SWJ-DP** that supports both on shared pins (TMS/SWDIO and TCK/SWCLK) and switches with a magic sequence.
- DP registers: **DPIDR** (identification, e.g. 0x2BA01477 for a common SW-DP), **CTRL/STAT** (power-up requests CDBGPRUPREQ and CSYSPWRUPREQ with their ACKs, sticky error flags), **SELECT** (which AP and which 16-byte register bank), **RDBUFF** (result of the last posted AP read) and **ABORT** (clear sticky errors, abort a stuck transaction).
- An **Access Port (AP)** turns DP accesses into something useful. A **MEM-AP** masters a bus: the debugger writes **TAR** (transfer address), sets size and auto-increment in **CSW**, and reads or writes **DRW** (data read/write); **BD0-BD3** access four words around TAR without rewriting it. An APB-AP typically reaches the debug components, an AXI-AP reaches system memory.
- Discovery: every CoreSight system has **ROM tables** listing the components (cores, CTIs, ETMs, funnels) with their component and peripheral ID registers, so a debugger can find everything starting from the AP's BASE register.

PITFALL: POSTED AP READS AND THE DEBUG POWER DOMAIN

AP reads through a DP are **posted**: the value returned by an AP read is the result of the **previous** AP read, and the last one must be collected from RDBUFF. Hand-written probe scripts that forget this are off by one. Also, the debug logic usually sits in its own power domain: until the debugger sets CDBGPWRUPREQ and sees CDBGPWRUPACK, AP accesses fault. SoC integrators must wire those handshakes to the power controller - a missing connection makes the chip undebuggable exactly when a core is in a low-power state.

23.6 Serial Wire Debug (SWD)

SWD replaces the four or five JTAG wires with two: **SWCLK** (driven by the host) and bidirectional **SWDIO**. It carries exactly the DP and AP register accesses of ADIV5, nothing else, which makes it faster per pin and ideal for small microcontrollers. Each transaction has three phases:



- **Request:** 8 bits LSB first - Start (1), APnDP (0 = DP, 1 = AP), RnW (1 = read), A[2:3] (register address bits, word-aligned), Parity, Stop (0), Park (1: the host drives SWDIO high before releasing it).
- **Turnaround (Trn):** one or more clock periods with neither side driving, whenever SWDIO changes direction.

- **ACK:** OK, WAIT (target busy: the host must retry - typically the AP bus transfer has not finished) or FAULT (a sticky error is set: the host clears it through ABORT). No response (all ones or all zeros) means the target is not listening - protocol error.
- **Data:** 32 bits LSB first plus one even-parity bit.
- **Line reset:** at least 50 SWCLK cycles with SWDIO high, followed by idle cycles, then a read of DPIDR (required after reset).
- **JTAG-to-SWD switching** on SWJ-DP: line reset, the 16-bit sequence 0xE79E sent LSB first, line reset again. Newer parts (ADiv5.2 and later) also support a **dormant** state entered and left with a 128-bit selection alert and an activation code, which lets several debug protocols share pins safely.
- **Multi-drop SWD (SWDv2):** several targets on one SWD bus, selected by writing a TARGETSEL value right after a line reset.

The request byte is small enough to verify exhaustively. The builder below is a pair of SystemVerilog functions; the testbench compares all 16 combinations against a Python model and prints the DP request table that every SWD probe firmware writer ends up memorising (0xA5 = read DPIDR, 0x81 = write ABORT, 0xB1 = write SELECT, 0xBD = read RDBUFF).

```
// Arm SWD (ADiv5) request packet: 8 bits, sent LSB first on SWDIO
// bit: 0 Start=1 | 1 APnDP | 2 RnW | 3 A[2] | 4 A[3] | 5 Parity | 6 Stop=0 | 7 Park=1
function automatic logic [7:0] swd_req(input logic apndp, rnw, input logic [3:2] a);
    logic par;
    par = apndp ^ rnw ^ a[2] ^ a[3];          // even parity over bits 1..4
    return {1'b1, 1'b0, par, a[3], a[2], rnw, apndp, 1'b1};
endfunction
// data phase: 32 data bits LSB first, then one even-parity bit
function automatic logic [32:0] swd_data(input logic [31:0] d);
    return {^d, d};
endfunction
```

SWD request packet and data-parity builders.

```
# Python reference used to generate swd_gold.hex
def req(ap, rnw, a2, a3):
    par = ap ^ rnw ^ a2 ^ a3
    bits = [1, ap, rnw, a2, a3, par, 0, 1]      # wire order, bit 0 first
    return sum(b << i for i, b in enumerate(bits))
with open('swd_gold.hex', 'w') as f:
    for n in range(16):
        # n = {APnDP, RnW, A2, A3}
        ap, rnw, a2, a3 = (n >> 3) & 1, (n >> 2) & 1, (n >> 1) & 1, n & 1
        f.write('%02x\n' % req(ap, rnw, a2, a3))
```

Python reference for the SWD request byte.

```
module tb;
    logic [7:0] gold [16]; int mism; logic [7:0] r; logic [32:0] dp;
    initial begin
        $readmemh("swd_gold.hex", gold);          // generated by the Python reference
        for (int n = 0; n < 16; n++) begin        // n = {APnDP, RnW, A2, A3}
            r = swd_req(n[3], n[2], {n[0], n[1]});
            if (r != gold[n]) mism++;
        end
        $display("16 request packets vs Python: mismatches=%0d", mism);
        $display("APnDP RnW A[3:2] packet on the wire (first bit left)");
        for (int k = 0; k < 4; k++) for (int rw = 0; rw < 2; rw++) begin // DP 0x0..0xC
            r = swd_req(1'b0, rw[0], k[1:0]);
            $display(" 0      %0d  %b      0x%h  %b%b%b%b%b%b%b%b", rw, k[1:0], r,
                    r[0], r[1], r[2], r[3], r[4], r[5], r[6], r[7]);
        end
        r = swd_req(1'b1, 1'b1, 2'b11);
        $display("AP read of 0xC (e.g. MEM-AP DRW) = 0x%h", r);
        dp = swd_data(32'h2BA01477);
        $display("data 0x2BA01477 -> parity bit %0d (ones in data = %0d)", dp[32],
```

```

        $countones(dp[31:0]));
    $finish;
end
endmodule

```

Testbench: all 16 requests vs Python, the DP request table, and a data-phase parity example.

```

16 request packets vs Python: mismatches=0
APnDP RnW A[3:2] packet on the wire (first bit left)
0 0 00 0x81 10000001
0 1 00 0xa5 10100101
0 0 01 0xa9 10010101
0 1 01 0x8d 10110001
0 0 10 0xb1 10001101
0 1 10 0x95 10101001
0 0 11 0x99 10011001
0 1 11 0xbd 10111101
AP read of 0xC (e.g. MEM-AP DRW) = 0x9f
data 0x2BA01477 -> parity bit 0 (ones in data = 14)

```

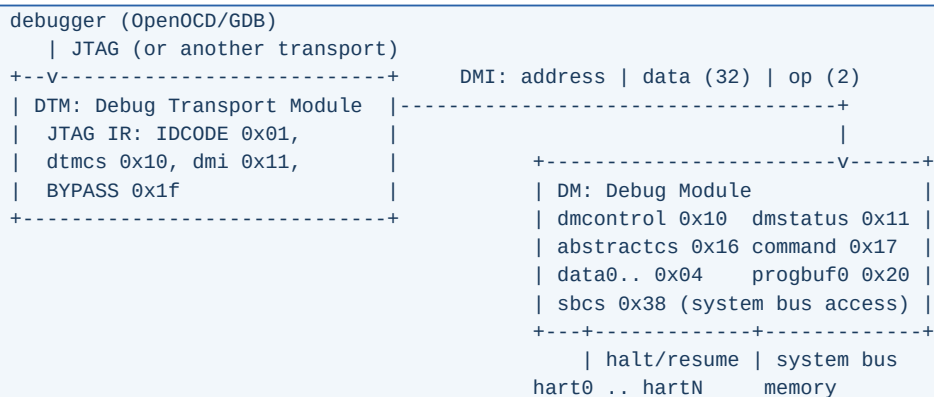
Icarus Verilog output. The 'on the wire' column shows the bit sequence in transmission order (Start first, Park last).

PRACTICAL TIP: THE SWD BRING-UP SEQUENCE

A robust probe does: line reset -> JTAG-to-SWD sequence -> line reset -> 2+ idle cycles -> read DPIDR (0xA5) -> write ABORT (0x81) to clear sticky errors -> write CTRL/STAT to request debug and system power-up -> poll the ACK bits -> write SELECT (0xB1) to pick an AP -> read the AP's IDR -> program CSW/TAR -> DRW accesses. On WAIT, retry; on FAULT, read CTRL/STAT, clear through ABORT and investigate. If DPIDR never answers, suspect pin muxing, a debug-disable fuse, or a core that disabled the SWD pins in firmware.

23.7 The RISC-V debug specification

RISC-V's **External Debug Support** specification (0.13 was the first widely implemented version, 1.0 ratified later) defines a layered, implementation-independent debug architecture:



RISC-V debug: DTM -> DMI -> Debug Module -> harts.

- The **DTM** turns the external transport into **DMI** accesses. The standard JTAG DTM has a 5-bit IR; **dtmcs** reports the DMI address width and idle-cycle requirement, and **dmi** is a shift register of {address, 32-bit data, 2-bit op} where op = 1 is read, 2 is write, and on capture the op field returns status (0 success, 2 failed, 3 busy - retry with more idle cycles).
- The **Debug Module (DM)** controls harts: **dmcontrol** selects harts and requests halt/resume/reset; **dmstatus** reports state (allhalted, anyrunning, authenticated...).
- Abstract commands** (written to **command**, arguments in **data0..**): Access Register (cmdtype 0; regno 0x1000+n for GPR xn, CSR numbers below 0x1000; aarsize 2 for 32-bit, 3 for 64-bit), Quick Access and Access Memory. **abstractcs** reports busy and an error code (cmderr) that must be cleared by writing ones.

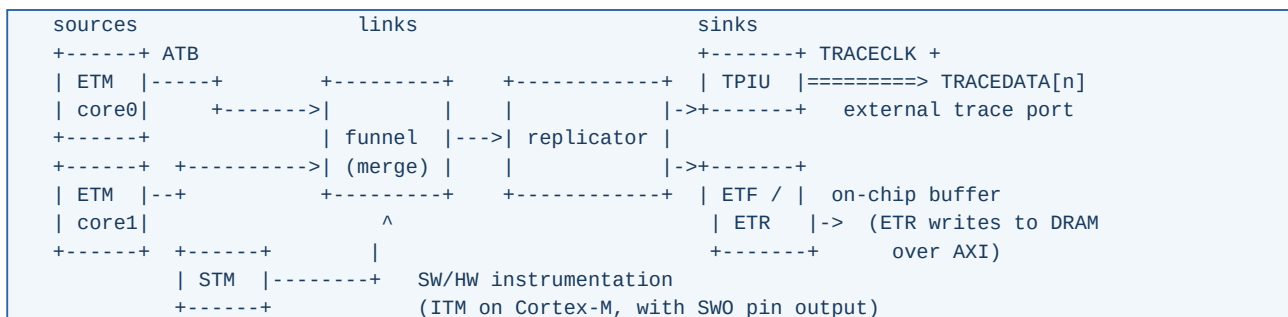
- A **program buffer** lets the debugger run short instruction sequences on a halted hart (e.g. fence.i or CSR moves the abstract commands do not cover); **system bus access** (sbcs/sbaddress/sbdata) reads memory without halting any hart, much like a MEM-AP.
- In the hart, **debug mode** is entered on halt, with **dcsr** (cause, step, ebreak behaviour) and **dpc** (resume PC); the **Sdtrig** extension defines hardware triggers (breakpoints and watchpoints).

INTUITION: CORESIGHT AND RISC-V DEBUG ARE THE SAME SHAPE

Probe protocol (JTAG/SWD vs JTAG DTM) -> a small register-access transport (DP/AP accesses vs DMI) -> a debug block that halts cores and a bus master that reads memory (core debug + MEM-AP vs DM abstract commands + system bus access). Once you know one, the other is a matter of register names - and SoCs mixing Arm and RISC-V cores often bridge a RISC-V DM behind a CoreSight APB-AP.

23.8 Trace: recording what happened

Halting a core changes the timing of the bug you are chasing. **Trace** records execution non-intrusively and streams it off-chip or into a buffer. CoreSight trace is a pipeline of **sources**, **links** and **sinks**, all joined by the **ATB** (AMBA Trace Bus):



A CoreSight trace topology.

Component	Role
ETM / PTM	Embedded Trace Macrocell (PTM: Program Trace Macrocell, an older program-flow-only variant): compresses a core's instruction stream into packets - mainly taken/not-taken atoms and indirect branch targets; a decoder reconstructs the full flow using the program image. Optional data trace and cycle counts.
ITM	Instrumentation Trace Macrocell (Cortex-M): software writes to stimulus ports (printf-style logging without a UART), plus hardware event and PC-sampling packets from the DWT. Often output over the single-pin SWO (Manchester or UART-style NRZ).
STM	System Trace Macrocell: many masters (cores, DMA, hardware events) write to memory-mapped stimulus channels; output uses the MIPI STP protocol with timestamps.
Funnel / replicator	Merge several ATB streams (each tagged with a trace ID) / duplicate a stream to two sinks.
TPIU	Trace Port Interface Unit: formats the ATB stream into 16-byte frames with IDs and drives a parallel trace port (TRACECLK + 1-32 TRACEDATA pins) to an external trace probe.
ETB / ETF / ETR	On-chip sinks: ETB is a dedicated SRAM; the Trace Memory Controller configured as ETF (FIFO/buffer) or ETR (router) - the ETR writes trace to system DRAM through AXI, the usual choice on large SoCs since it needs no pins.
RISC-V N-trace / E-trace	RISC-V trace encoders: E-trace (Efficient Trace) is a branch-trace format fed from a standard ingress port on the hart; N-trace uses the IEEE-ISTO 5001 Nexus message format. Both feed similar funnels and sinks.

Trace components.

MATH: TRACE BANDWIDTH

Program-flow trace of a core at 2 GHz typically needs on the order of 1 bit or less per instruction after compression, so a few cores produce several Gb/s. A 16-pin TPIU at 300 MHz DDR gives about 9.6 Gb/s; an ETR into DRAM can sustain much more but eats memory bandwidth. Size trace sinks for the number of cores traced at once, and expect **overflow** packets when a burst of indirect branches exceeds the budget.

23.9 Cross-triggering: CTI and CTM

To debug a multi-core or heterogeneous system, you want a breakpoint on one core to **halt all of them** within a few cycles, or a trace capture to start when a peripheral raises an event. CoreSight provides a **Cross Trigger Interface (CTI)** next to each core or trace component and a **Cross Trigger Matrix (CTM)** that broadcasts events on a small number of **channels** (typically 4) between CTIs. Each CTI maps its local trigger inputs (e.g. 'core halted', 'trace buffer full') onto channels and channels onto its trigger outputs (e.g. 'debug request', 'restart', 'trace start'). The CTIs are programmed by the debugger through the APB-AP; the handshakes cross clock and power domains, so trigger outputs are usually level signals with acknowledges rather than single pulses.

23.10 Post-silicon debug and debug security

- **Scan dump:** stop the clocks and shift out every scan flop through the test infrastructure - a full-state snapshot after a hang, mapped back to RTL signal names with the scan-chain map.
- **Embedded logic analyzers (ELA) and debug buses:** selectable internal signals muxed into a trace buffer with trigger logic (CoreSight ELA-500/600 is one product); chosen at design time, so the architecture team must predict which signals will matter.
- **On-chip performance monitors** and bus trace (e.g. CoreSight AXI trace / Catu, NoC probes) for interconnect deadlocks.
- **Security:** debug is the most powerful attack surface on the chip. Arm's authentication signals DBGEN, NIDEN, SPIDEN and SPNIDEN (invasive/non-invasive, non-secure/secure) are driven by a secure life-cycle controller; production parts lock or authenticate debug (challenge-response), keep TAP access to secure fuses and keys disabled, and allow re-opening only for authenticated failure analysis (RMA). The RISC-V DM has an 'authenticated' bit for the same purpose.
- **Ownership:** DFT owns 1149.1/1500/1687 and scan; the debug/SoC architecture team owns CoreSight topology, ROM tables and CTI wiring; RTL integrates; DV verifies with a JTAG/SWD VIP and by running real debugger scripts on the emulator; security reviews the life-cycle gating; the tools/firmware team delivers the debugger configuration files that describe it all.

PITFALL: DEBUG THAT ONLY WORKS WHEN NOTHING IS WRONG

Debug paths that depend on the functional system - a debug APB clocked by a gated clock, a MEM-AP behind an interconnect that deadlocked, trace written to DRAM that is not initialised yet - fail exactly when you need them. Keep the DAP, the ROM table and at least one path to core debug registers on always-on clocks and power, give the MEM-AP a path that bypasses congested NoC queues, and make sure JTAG/SWD can reset the system (a debug-requested reset) without software help.

23.11 Summary

- **1149.1 JTAG:** TCK/TMS/TDI/TDO (+ optional TRST*), a 16-state TAP controller with IR and DR columns, capture/shift on rising TCK and update/TDO on falling TCK, five TMS = 1 clocks reset from anywhere.
- Mandatory instructions BYPASS (all ones), SAMPLE, PRELOAD and EXTEST; IDCODE is optional but universal: version/part/JEP106 manufacturer/LSB = 1. The IR captures ...01. BSDL describes it all.
- **1149.6** adds pulse/train EXTEST for AC-coupled lanes; **1500** wraps cores (WSP, WIR, WBY, WBR); **1687 IJTAG** builds SIB-based instrument networks described in ICL and operated through PDL.

- **CoreSight DAP** = one DP (JTAG-DP, SW-DP, SWJ-DP) + APs (MEM-AP with CSW/TAR/DRW); AP reads are posted; debug power-up handshakes must be wired.
- **SWD**: 8-bit request (Start, APnDP, RnW, A[2:3], even parity, Stop, Park), turnaround, 3-bit ACK (OK/WAIT/FAULT), 32 data bits + parity; line reset and the 0xE79E JTAG-to-SWD switch.
- **RISC-V debug**: DTM (dtmcs/dmi) -> DMI -> Debug Module with abstract commands, program buffer and system bus access.
- **Trace**: ETM/PTM, ITM, STM sources; funnels/replicators; TPIU, ETB/ETF/ETR sinks; RISC-V E-trace/N-trace; CTI/CTM cross-triggering.
- Debug must be secure (life-cycle gated) and must survive the failures it is meant to diagnose.

23.12 Exercises

- 1 A board has three devices in a JTAG chain with IR lengths 4, 5 and 8. After reset, how many bits must be shifted through Shift-DR to read all IDCODEs, and how does software discover the IR lengths without prior knowledge? (Hint: the 01 capture rule.)
- 2 Add a USERCODE instruction and a 2-bit IJTAG-style SIB in front of an 8-bit instrument register to the TAP design. Write a testbench that opens the SIB, writes the instrument and reads it back.
- 3 Write the SWD read-transaction engine: it sends a request built by `swd_req`, performs turnaround, samples ACK, reads 32 bits plus parity, and retries on WAIT. Test it against a target model that answers WAIT twice before OK.
- 4 Using the RISC-V debug spec, write the sequence of DMI writes and reads needed to halt hart 0, read register x10 with an abstract command, and resume it.
- 5 An SoC has 8 cores traced with ETM at an average of 0.6 bits per instruction and 2 instructions per cycle at 1.5 GHz. Can a 16-bit TPIU at 250 MHz DDR carry it? How large an ETR buffer captures 10 ms?
- 6 List every signal and condition that must be true for a debugger to halt a core that is currently in a power-down state in your favourite SoC, and identify which team owns each.

PART V

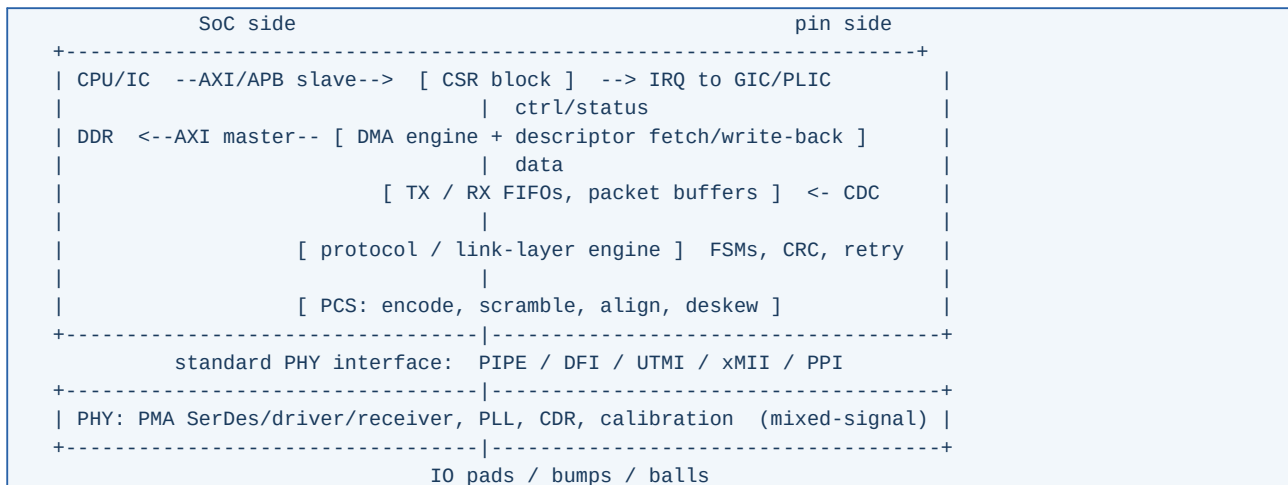
Designing With Protocols

Parts II to IV studied protocols one at a time. Real projects never meet a protocol alone: a USB, Ethernet or PCIe port is a controller with registers, DMA, buffers, several clock domains and a PHY behind a pad ring; it must be verified against a specification, certified by a standards body and made reliable, safe and secure; and it must be chosen and budgeted against every other interface on the chip. This final part is about those engineering skills - the architecture of a controller and its PHY integration, protocol verification from assertions to plugfests, reliability and security on links, and how to pick, size and budget the protocols of a whole SoC. It closes with a learning roadmap.

24. Architecture of a Protocol Controller and PHY Integration

Open the block diagram of any interface IP - a USB device controller, an Ethernet MAC, a PCIe endpoint, an eMMC host, a MIPI CSI-2 receiver - and you find the same skeleton. On one side is the SoC: an AXI or AHB port for register access and another for DMA. On the other side is the pin: a PHY that turns bits into volts. In between sit the same half-dozen layers, whatever the protocol. Learn the skeleton once and every new interface becomes a variation on a theme; you also learn where the integration bugs live, because they live at the seams between those layers.

24.1 The layered controller



The universal skeleton of a protocol controller. Clock-domain crossings usually sit at the FIFOs (bus clock vs. link clock) and at the PHY interface (link clock vs. PHY-recovered clock).

Layer	Job	Typical implementation	Owner
Host/bus interface	Register access; DMA master port	AXI4-Lite/APB slave, AXI4 master (Ch 5-8)	RTL + SoC integration
CSRs	Configuration, status, interrupts, counters	Generated from IP-XACT/SystemRDL	RTL + firmware
DMA engine	Move payload between memory and FIFOs	Descriptor rings/lists, scatter-gather	RTL + driver
Buffers / FIFOs	Rate matching, CDC, retry storage, jitter absorption	SRAM macros, async FIFOs	RTL + PD (SRAM)
Protocol / link engine	Framing, handshakes, ACK/NAK, CRC, flow control, state machines	FSMs, CRC units, timers	RTL (the protocol expert)
PCS	Line coding, scrambling, lane alignment, deskew, elastic buffers	8b/10b, 64b/66b, 128b/130b logic	RTL / PHY
PHY (PMA + analog)	Serialize, drive, equalize, recover clock	Hard macro from a PHY vendor	Analog/PHY team or vendor

The split is not arbitrary. Everything above the PCS runs at a **parallel** clock of a few hundred MHz and is ordinary synthesizable RTL. The PCS may run at the same clock with a wide datapath (for example 32 or 64 bits per lane). The PMA runs at the **line rate** - gigahertz - and is custom analog and high-speed digital design that only a PHY team can build and characterize. The standard PHY interfaces exist precisely so that the two sides can come from different teams or companies.

24.2 The host interface and the register block

Software sees a controller only through its registers, so the CSR block is the controller's user interface. A representative map for a generic packet controller:

Offset	Register	Access	Contents
0x000	ID / VERSION	RO	IP identifier and revision - lets one driver support several versions
0x004	CTRL	RW	Enable, soft reset, loopback, mode bits
0x008	STATUS	RO	Link up, speed, FIFO levels, busy
0x00C	INT_STATUS	RW1C	Event bits: RX done, TX done, error; write 1 to clear
0x010	INT_ENABLE	RW	Mask for each INT_STATUS bit
0x020	TX_DESC_BASE / RX_DESC_BASE	RW	Ring base addresses (64-bit: LO/HI pair)
0x028	TX_TAIL (doorbell)	WO	Software writes the index of the last valid descriptor
0x02C	TX_HEAD	RO	Hardware's current position
0x100+	Counters	RO/RC	Good/bad frames, CRC errors, drops, retries

A typical controller register map. The exact layout is yours to define; the kinds of registers recur in every IP.

PITFALL: STATUS AND INTERRUPT REGISTER RACES

Classic CSR bugs: (1) a W1C bit that is set by hardware in the same cycle software writes 1 to clear it - hardware must win, or the event is lost; (2) read-to-clear counters that are cleared by a debugger's memory view; (3) 64-bit registers read as two 32-bit halves while the counter carries between the reads (latch the high half when the low half is read); (4) a doorbell register that is posted through a write buffer and overtakes the descriptor write it announces. Specify the behaviour of each case explicitly and check it in the testbench.

24.3 DMA engines and descriptors

Any interface faster than a few Mb/s moves data by **DMA**: software prepares buffers in memory and describes them with **descriptors**; the controller fetches descriptors, moves the data and writes the descriptors back with status. The two common organizations:

Descriptor RING (NICs, USB xHCI, NVMe)	Linked LIST (many SoC DMAs)
base -> [d0][d1][d2][d3][d4][d5][d6][d7] ^HEAD (HW) ^TAIL (SW) HW consumes HEAD..TAIL-1, advances HEAD SW fills at TAIL, writes TAIL doorbell index wraps modulo ring size	desc_base v [ctrl len buf next]--+ v [ctrl len buf next]--+ v [ctrl len buf EOC]
ownership: HEAD/TAIL indices or an OWN bit in each descriptor	

Rings suit continuous streams with a fixed pool of buffers; linked lists suit scatter-gather transfers of arbitrary shape.

Every descriptor format answers the same questions: **where** is the buffer (address, possibly 64-bit), **how long** it is, **what to do** (end of packet, interrupt on completion, checksum offload, timestamp) and **who owns it** now. Ownership is the heart of the protocol between hardware and software: software sets OWN=1 after it has fully written the descriptor, hardware clears it when it is done, and neither side touches a descriptor the other owns.

- **Ordering.** The driver must write the descriptor, then a memory barrier (`dma_wmb()`/`wmb()` in Linux, `DSB` on Arm), then the doorbell. Hardware must write the data and status, then clear OWN (or advance HEAD), then raise the interrupt - in that order as seen by the CPU.
- **Coherency.** If the DMA port is not I/O-coherent (no ACE-Lite or CHI snooping, Chapter 9), the driver must clean the cache before a TX DMA and invalidate it before reading RX data. A missing invalidate returns stale data only sometimes - a notorious heisenbug.

- **Descriptor prefetch.** Fetching a descriptor costs one memory round-trip - often hundreds of nanoseconds behind an interconnect. High-rate engines prefetch several descriptors ahead and write back status in batches.
- **Scatter-gather.** One packet may span several buffers (a header buffer plus page-sized payload buffers); the end-of-packet flag marks where the frame ends, which on a stream interface becomes **TLAST** (Chapter 8).

24.4 Buffers, FIFOs and how big they must be

Buffers exist for four reasons, and each gives a sizing rule:

Reason	Sizing rule	Example
Round-trip latency (credits, pause, ACK)	$\text{depth} \geq \text{bandwidth} \times \text{round-trip time}$	PCIe replay buffer, Ethernet RX FIFO covering PAUSE reaction
Burst absorption (producer bursts, consumer steady)	$\text{depth} \geq \text{burst} \times (1 - r_{\text{out}} / r_{\text{in}})$	Camera line bursts into a slower bus
Latency jitter (memory busy for a while)	$\text{depth} \geq \text{rate} \times \text{worst-case stall}$	Display FIFO riding out a DDR refresh or a bank conflict
Whole-packet storage	$\text{depth} \geq \text{max packet (store and forward)}$	MAC TX: checksum insertion, cut-through not allowed

Credit / round-trip sizing (the bandwidth-delay product):

$$\text{depth_bytes} \geq \text{BW_bytes_per_s} \times \text{RTT_s}$$

Example: a link carrying 4 GB/s whose flow-control round trip is 500 ns

depth $\geq 4\text{e}9 \times 500\text{e-}9 = 2000$ bytes -> round up, e.g. 2 KB + one max packet

Jitter example: display reads 1920x1080x4 B at 60 Hz = 498 MB/s; DDR may stall the reader for 5 us (refresh + contention):

$$\text{depth} \geq 498\text{e}6 \times 5\text{e-}6 = 2490 \text{ bytes} \quad \text{-> e.g. 4 KB line buffer}$$

If the buffer is smaller than bandwidth x round-trip, the link stalls waiting for credits and never reaches full rate.

Credit-based flow control (PCIe, CXL, UCIe, AXI-Stream bridges, NoCs) makes the rule explicit: the receiver advertises how much buffer it has, the transmitter decrements a counter for each unit sent, and the receiver returns credits as it frees space. With too few credits the transmitter idles during every round trip; with too many the receiver's SRAM is wasted. The worked example at the end of this chapter uses the same idea in miniature: the DMA engine issues a memory read only when its FIFO has room for the returning data, counting reads still in flight - a local credit loop.

BUFFER SIZE IS A SYSTEM DECISION

Buffers are usually the largest area in a controller (a 10G MAC with jumbo-frame support, a PCIe controller with a replay buffer and receive queues). Size them from the worst-case round trip at the highest rate you must sustain, add one maximum packet, and state the assumption in the spec - then let the architect trade SRAM against performance explicitly.

24.5 The protocol engine and the PCS

The protocol engine is where the chapters of Parts III and IV live: the USB token/data/handshake sequencer, the Ethernet MAC's framing, CRC and inter-frame gap, the PCIe transaction and data-link layers with their sequence numbers, ACK/NAK and replay buffer, the I2C master's START/STOP/arbitration logic. It is FSM-heavy, full of timers (timeouts, inter-packet gaps, retry intervals) and counters, and it is where most protocol bugs are found. Keep it separate from the DMA and buffer logic so it can be verified with protocol VIP alone.

Below it, the **physical coding sublayer (PCS)** handles line coding (8b/10b, 64b/66b, 128b/130b), scrambling, block/symbol alignment, multi-lane deskew and the **elastic buffer** that absorbs the ppm difference between the recovered receive clock and the local clock by inserting or deleting SKP/idle symbols (Chapter 2). Depending on the PHY interface, the PCS is either in the controller (PIPE 'SerDes architecture', where the PHY delivers raw parallel symbols) or in the PHY (original PIPE, where the PHY does 8b/10b and the elastic buffer).

24.6 Standard PHY interfaces

Interface	Protocols	What crosses it	Notes
PIPE (PHY Interface for PCI Express, SATA, USB, DisplayPort and USB4)	PCIe, USB 3.x, SATA, DP, USB4	Parallel data per lane (8-64 bits), K/control indicators, power-state and rate controls, RX status	Intel-published; recent versions use a SerDes (raw data) mode and a message bus
DFI (DDR PHY Interface)	DDR3/4/5, LPDDR4/5	Command/address, write data + mask, read data valid, training/update handshakes	Frequency ratios 1:1, 1:2, 1:4 between controller and PHY clocks (Ch 20)
UTMI / UTMI+ / ULPI	USB 2.0	8/16-bit data at 60/30 MHz, line state, op-mode, termination control	ULPI is a 12-pin reduced interface to an external PHY
xMII : MII, RMII, GMII, RGMII, SGMII, XGMII, USXGMII	Ethernet	Nibble/byte/word data + enable/error, or a SerDes	MDIO/MDC for PHY management (Ch 16, 19)
PPI (PHY Protocol Interface)	MIPI CSI-2/DSI over D-PHY/C-PHY	Byte/word per lane, HS/LP mode controls, escape mode	Defined in the D-PHY/C-PHY specs
RDI / FDI	UCIe	Raw or flit-aware die-to-die data between adapter and PHY/protocol layer	Chapter 22

WHY STANDARD PHY INTERFACES MATTER

They let you buy a PHY hard macro from one vendor, a controller from another (or write it yourself), and integrate them with predictable effort. They also define the verification boundary: a controller can be verified against a PIPE/DFI/UTMI bus-functional model of the PHY long before the analog macro exists.

24.7 Clock domains and CDC points

A controller rarely lives in one clock domain. A typical high-speed controller has at least three:

bus clock (200-400 MHz)	link/core clock (from PHY PLL)	PHY clocks
+-----+	+-----+	+-----+
AXI slave, CSRs,	protocol engine, PCS,	TX serial
DMA master	CRC, retry, timers	RX (CDR)
[async FIFO] <==== =====> [packet buffers]		
CSR --[2FF / hshk]- ----> config (quasi-static)		
IRQ <-[pulse sync]- ---- -- events		
[elastic buffer] <===== == = rx_clk		
+-----+	+-----+	+-----+
aux clock (always-on, e.g. 32 kHz or ref clk): wake-up, low-power FSMs		

CDC points of a typical controller: data through async FIFOs, control through synchronizers or handshakes, receive data through the PCS elastic buffer.

- **Bus clock**: the SoC interconnect frequency - chosen by the SoC, not the protocol.
- **Link/core clock**: derived from the PHY PLL, e.g. line rate / datapath width (PCIe Gen4 at 16 GT/s with a 32-bit PIPE datapath and 128b/130b encoding runs around 500 MHz per lane).
- **Recovered receive clock**: from the CDR; same nominal frequency as the local clock but plesiochronous (up to a few hundred ppm apart), so the elastic buffer is mandatory.
- **Aux/always-on clock**: keeps wake-up detection alive while the main PLL is off (USB suspend, PCIe L1/L2, Ethernet energy-efficient idle).

PITFALL: CONFIGURATION REGISTERS CROSSING DOMAINS

A 32-bit control register written in the bus domain and used directly in the link domain is a multi-bit CDC bug: during the update the link logic can see a mixture of old and new bits. Either declare such fields 'static' (only written while the link is disabled, documented and checked by an assertion) or transfer them with a handshake/MCP synchronizer. CDC tools will flag the crossing; the waiver must say which rule makes it safe.

24.8 Reset, power-up and PHY bring-up sequencing

A PHY is not ready the moment reset is released. Its PLL must lock, its termination and drive strength must be **calibrated** against a precision resistor or a reference, its receivers may need offset calibration, and only then can the link-training state machine start. The controller and the firmware must follow the sequence the PHY data book specifies - a common source of 'link does not come up' bugs.

```

power good -> ref clock stable -> PHY reset released
|
v
PHY PLL locking ..... wait for pll_lock (typically tens of us)
|
v
calibration ..... impedance/termination, RX offset, (DDR: ZQ cal)
|                wait for cal_done / phy_ready / PhyStatus
v
controller reset released (link/core clock now valid and stable)
|
v
firmware: program CSRs (mode, speed, buffers, descriptors) -> enable
|
v
link training / initialization: e.g. PCIe LTSSM Detect->Polling->Config->L0,
USB chirp / LFPS + training, Ethernet auto-negotiation, DDR write leveling
|
v
link up -> STATUS.link_up, interrupt -> data traffic

```

A generic bring-up sequence. The real one is in the PHY data book and the controller's programming guide; the order matters.

- Hold the controller's link-domain logic in reset until the clock it runs on is stable - a PHY-generated clock that is still locking can glitch.
- Reset each clock domain with its own synchronized reset (reset synchronizer per domain), and define what a **soft reset** from the CSR resets (usually the link and DMA state, not the CSRs themselves).
- Low-power states reverse the sequence: stop traffic, drain DMA, enter the protocol low-power state (USB suspend/U3, PCIe L1, EEE LPI), gate clocks, possibly power down the PHY PLL, and keep the wake-up detector on the aux clock.

24.9 Interrupts

Interrupts connect the controller's events to software. Good practice:

- One **INT_STATUS** register with W1C event bits, an **INT_ENABLE** mask and a single level-sensitive output that is the OR of enabled pending bits. Level-sensitive lines do not lose events if software is slow; edge/pulse interrupts do.
- **Interrupt coalescing / moderation**: at millions of packets per second one interrupt per packet overwhelms the CPU. Raise the interrupt after N completions or T microseconds, whichever comes first (NICs, USB xHCI, NVMe all do this).
- **Message-signalled interrupts** (PCIe MSI/MSI-X, Arm GIC LPIs via ITS) are memory writes; they are ordered behind the data writes on the same path, which removes a whole class of races.
- Error interrupts (CRC errors, timeouts, overflows) usually go to a separate vector or a RAS/safety controller (Chapter 26).

24.10 Firmware and driver interaction

A controller is only as good as the programming model it gives software. The driver writer's view of the DMA controller built later in this chapter looks like this:

```
// transmit one packet made of two buffers (pseudo-C driver code)
d = &ring[tail];
d[0].buf = hdr_dma; d[0].len = 64; d[0].ctrl = OWN;
d[1].buf = pay_dma; d[1].len = 1400; d[1].ctrl = OWN | EOP | IRQ;
dma_wmb(); // descriptors visible before doorbell
writel(tail + 2, base + TX_TAIL); // doorbell (posted write)
...
// interrupt handler
st = readl(base + INT_STATUS);
writel(st, base + INT_STATUS); // W1C: acknowledge what we saw
while (!(ring[head].ctrl & OWN)) { reclaim(head); head = next(head); }
```

The software half of the descriptor protocol. The barrier and the ownership check are the parts that break when the hardware gets the ordering wrong.

- Provide an **ID/version register** and capability registers so one driver can handle several configurations of the IP.
- Make every error **observable** (sticky status bits, counters, the descriptor that failed) - post-silicon debug depends on it.
- Avoid registers that must be written in a magic order or within a time window; if unavoidable, document it in the programming guide and enforce it in hardware where possible.
- Deliver a **reference driver** or bare-metal test with the IP; the firmware team will use it for bring-up and the DV team can run it on an emulator.

24.11 IP selection: build or buy, hard or soft

Decision	Build it yourself when...	Buy (license) it when...
Controller	Simple protocol (UART, SPI, I2C, APB/AXI peripherals), differentiation matters, or you need unusual features	Complex, spec-heavy protocols (USB 3.x/4, PCIe, CXL, DDR, HDMI/DP, MIPI CSI-2/DSI) where compliance history is worth more than license fees
PHY	Almost never for high-speed SerDes unless you are a PHY company; simple GPIO-based PHYs (I2C, SPI, UART) are just pads	Any SerDes, DDR/LPDDR/HBM PHY, MIPI D-/C-PHY, USB 2.0 PHY: buy a silicon-proven hard macro in your exact process node
Verification IP	Rarely - writing a full compliance-grade VIP takes person-years	Always for standard protocols; VIP vendors track spec errata and compliance checklists

- **Hard PHY**: a laid-out, characterized macro for one process (e.g. a 5 nm LPDDR5X PHY). It comes with timing models (LIB), abstracts (LEF), IBIS/S-parameter models and a silicon report. You cannot change it, only configure it.
- **Soft PHY / soft PCS**: synthesizable logic, portable across processes - realistic only for low-speed interfaces or the digital parts (PCS, PHY-side controller logic).
- **Evaluate** licensed IP on: silicon proof in your node, compliance certificates (USB-IF, PCI-SIG), deliverables (RTL, VIP, drivers, test chips), configurability, PPA numbers, support and errata history, and license terms (per-design vs. royalties).

24.12 Pad ring and IO cells

Every protocol ends at a pad. The IO library and the pad ring are physical-design and package decisions that constrain the protocols you can use.

IO type	Used for	Characteristics
GPIO (1.8/3.3 V, programmable drive, pull-up/down, Schmitt input)	UART, SPI, I2C (open-drain mode), I2S, JTAG, CAN/LIN to transceivers	Tens of MHz; muxed with other functions through pin-mux registers
Open-drain / fail-safe IO	I2C/SMBus/I3C	Must not load the bus when unpowered; I3C needs push-pull and open-drain modes
LVDS / sub-LVDS / SLVS	Legacy camera/display, clock inputs	Differential, few hundred mV swing, up to ~1 Gb/s per pair
DDR IO (SSTL/POD/LVSTL)	DDR4 (POD12), LPDDR4/5 (LVSTL)	Part of the DDR PHY macro with calibrated ODT/drive
SerDes macros (bump-attached)	PCIe, USB 3.x, Ethernet, DP, SATA	Not in the pad ring at all in flip-chip designs - the macro sits at the die edge with its own bumps, ESD and supplies
Analog / special pads	Crystal (XO), reference resistor for calibration, PLL supplies	Dedicated, noise-isolated supply domains

PITFALL: PIN-MUX AND IO VOLTAGE SURPRISES

Late-found integration bugs: an I2C function muxed onto a pad that has no open-drain mode; an SD card interface that must switch from 3.3 V to 1.8 V signalling (UHS-I) on a pad that supports only one voltage; a 3.3 V-only IO bank that shares a supply with a 1.8 V sensor interface; an RGMII bus spread over pads with mismatched delays. Review the pin list with the board and package teams before the pad ring is frozen.

24.13 Worked example: a descriptor-based DMA-to-stream engine

To make the architecture concrete, here is a complete, small DMA engine of the kind that feeds a transmit protocol engine. It walks a **linked list of descriptors** in memory, checks the OWN bit, streams each buffer out on an AXI4-Stream-style **valid/ready** port with **TLAST** at the end of each packet, writes each descriptor back with OWN cleared, and raises an interrupt at the end of the chain. Its data path uses a 4-entry FIFO and a **credit rule**: a memory read is issued only if the FIFO has room for it plus every read still in flight, so back-pressure from the stream never overflows the FIFO - the bandwidth-delay rule of this chapter in one line of RTL (**can_issue**).

```

word 31 30 29 28 ..... 16 15 ..... 0
dp+0 [OWN][EOP][EOC][ reserved ][ LEN (words) ]
dp+1 [ buffer address (word) ]
dp+2 [ next descriptor address ]

FSM: IDLE -start-> F0 -> F1 -> F2 -> F3 --OWN=1--> DATA --> WB --EOC=0--> F0
                                     |
                                     +--OWN=0: err, irq
                                     -> IDLE
                                     (irq) -> IDLE
                                     +--EOC=1--> DRAIN

```

Descriptor format and control FSM of the example DMA engine. F0-F3 fetch the three descriptor words through a one-cycle-latency memory port.

```

// Descriptor-based DMA: walks a linked list of descriptors in memory and streams each
// buffer out on an AXI4-Stream-style port. Descriptor = 3 words at address dp:
// dp+0: [31] OWN (1 = HW owns) [30] EOP (TLAST at end) [29] EOC (end of chain) [15:0] LEN
// dp+1: buffer address (word) dp+2: next descriptor address
module desc_dma #(parameter int AW = 8, parameter int DEPTH = 4) (
    input logic clk, rst_n,
    input logic start, // CSR "GO" (pulse)
    input logic [AW-1:0] desc_base, // CSR: first descriptor
    output logic busy, irq, err, // irq: chain done, err: descriptor not owned
    // one-port synchronous memory (read data valid 1 cycle after mem_re)
    output logic mem_re, mem_we,

```

```

output logic [AW-1:0] mem_addr,
output logic [31:0] mem_wdata,
input logic [31:0] mem_rdata,
// stream output
output logic m_tvalid, m_tlast,
output logic [31:0] m_tdata,
input logic m_tready
);
typedef enum logic [2:0] {IDLE, F0, F1, F2, F3, DATA, WB, DRAIN} st_t;
st_t st;
logic [AW-1:0] dp, ptr, nxt;
logic [31:0] w0;
logic [15:0] rem;
// ---- data FIFO {last, data} + one read in flight (credit = free slots) ----
logic [32:0] fifo [DEPTH];
logic [$clog2(DEPTH):0] cnt;
logic [$clog2(DEPTH)-1:0] wp, rp;
logic pend, pend_last;
wire push = pend; // read data returns -> push
wire pop = m_tvalid && m_tready;
wire can_issue = (st == DATA) && (rem != 0) && (cnt + pend < DEPTH);
assign m_tvalid = (cnt != 0);
assign {m_tlast, m_tdata} = fifo[rp];
assign busy = (st != IDLE);
wire [31:0] w0_done = {1'b0, w0[30:0]}; // write-back value: OWN cleared

always_comb begin
    mem_re = 1'b0; mem_we = 1'b0; mem_addr = dp; mem_wdata = '0;
    case (st)
        F0: begin mem_re = 1'b1; mem_addr = dp; end
        F1: begin mem_re = 1'b1; mem_addr = dp + 1; end
        F2: begin mem_re = 1'b1; mem_addr = dp + 2; end
        DATA: begin mem_re = can_issue; mem_addr = ptr; end
        WB: begin mem_we = 1'b1; mem_addr = dp; mem_wdata = w0_done; end
        default: ;
    endcase
end

always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin
        st <= IDLE; irq <= 0; err <= 0; pend <= 0; pend_last <= 0;
        cnt <= 0; wp <= 0; rp <= 0; dp <= 0; ptr <= 0; nxt <= 0; rem <= 0; w0 <= 0;
    end else begin
        irq <= 1'b0;
        pend <= can_issue;
        pend_last <= can_issue && (rem == 1) && w0[30];
        if (push) begin fifo[wp] <= {pend_last, mem_rdata}; wp <= wp + 1; end
        if (pop) rp <= rp + 1;
        cnt <= cnt + push - pop;
        case (st)
            IDLE: if (start) begin dp <= desc_base; err <= 1'b0; st <= F0; end
            F0: st <= F1;
            F1: begin w0 <= mem_rdata; st <= F2; end
            F2: begin ptr <= mem_rdata[AW-1:0]; st <= F3; end
            F3: begin
                nxt <= mem_rdata[AW-1:0];
                rem <= w0[15:0];
                if (!w0[31]) begin err <= 1'b1; irq <= 1'b1; st <= IDLE; end // not ours
                else st <= DATA;
            end
            DATA: if (can_issue) begin ptr <= ptr + 1; rem <= rem - 1; end
        endcase
    end
end

```

```

        else if (rem == 0 && !pend) st <= WB;
WB:    if (w0[29]) st <= DRAIN;          // OWN cleared this cycle: buffer returned
        else begin dp <= nxt; st <= F0; end
DRAIN: if (cnt == 0) begin irq <= 1'b1; st <= IDLE; end
        default: st <= IDLE;
    endcase
end
endmodule

```

desc_dma.sv - descriptor fetch, credit-controlled data reads, stream FIFO, write-back of the OWN bit and completion interrupt.

The testbench builds a three-descriptor chain in a memory model: two buffers (3 + 2 words) form the first packet (EOP on the second descriptor) and one 4-word buffer forms the second packet (EOP and EOC). It runs the chain with a sink that is always ready, then again with 50 % random back-pressure, and finally restarts the engine on the same chain without software handing the descriptors back.

```

module tb;
    logic clk = 0, rst_n = 0, start = 0;
    always #5 clk = ~clk;
    logic busy, irq, err, mem_re, mem_we, tv, tl, tr;
    logic [7:0] maddr; logic [31:0] mwd, mrd, td;
    logic [31:0] mem [256];
    desc_dma dut (.clk, .rst_n, .start, .desc_base(8'h10), .busy, .irq, .err,
        .mem_re, .mem_we, .mem_addr(maddr), .mem_wdata(mwd), .mem_rdata(mrd),
        .m_tvalid(tv), .m_tlast(tl), .m_tdata(td), .m_tready(tr));
    always_ff @(posedge clk) begin                                // memory model, 1-cycle read
        if (mem_re) mrd <= mem[maddr];
        if (mem_we) mem[maddr] <= mwd;
    end
    int cyc = 0, beats = 0, t0, t_first, bp;
    always @(posedge clk) cyc <= cyc + 1;
    always @(posedge clk) if (tv && tr) begin
        beats++;
        if (beats == 1) t_first = cyc;
        $display("cyc %3d beat %0d tdata=%08h tlast=%0d", cyc - t0, beats, td, tl);
    end
    task automatic run(input int bp_pct);
        bp = bp_pct; beats = 0;
        // descriptor chain: 0x10 -> 0x20 -> 0x30 (OWN | EOP | EOC | LEN)
        mem[8'h10] = 32'h8000_0003; mem[8'h11] = 32'h40; mem[8'h12] = 32'h20;
        mem[8'h20] = 32'hC000_0002; mem[8'h21] = 32'h50; mem[8'h22] = 32'h30;
        mem[8'h30] = 32'hE000_0004; mem[8'h31] = 32'h60; mem[8'h32] = 32'h00;
        for (int i = 0; i < 16; i++) begin
            mem[8'h40 + i] = 32'hA000_0000 + i; mem[8'h50 + i] = 32'hB000_0000 + i;
            mem[8'h60 + i] = 32'hC000_0000 + i;
        end
        @(posedge clk); t0 = cyc + 1; start <= 1; @(posedge clk); start <= 0;
        @(posedge irq);
        $display("irq at cyc %0d err=%0d beats=%0d desc word0 after: %08h %08h %08h",
            cyc - t0, err, beats, mem[8'h10], mem[8'h20], mem[8'h30]);
    endtask
    always @(posedge clk) tr <= ($urandom_range(99) >= bp);
    initial begin
        repeat (2) @(posedge clk); rst_n = 1;
        $display("--- run 1: sink always ready ---"); run(0);
        $display("--- run 2: sink ready 50%% of cycles ---"); run(50);
        $display("--- run 3: restart on a chain software has not handed back ---");
        start <= 1; @(posedge clk); start <= 0; @(posedge irq);
        $display("irq err=%0d (descriptor 0x10 has OWN=0)", err);
        $finish;
    end
end

```

endmodule

tb_dma.sv - memory model, descriptor chain, random back-pressure and a negative test of the ownership check.

```

--- run 1: sink always ready ---
cyc  7 beat 1 tdata=a0000000 tlast=0
cyc  8 beat 2 tdata=a0000001 tlast=0
cyc  9 beat 3 tdata=a0000002 tlast=0
cyc 17 beat 4 tdata=b0000000 tlast=0
cyc 18 beat 5 tdata=b0000001 tlast=1
cyc 26 beat 6 tdata=c0000000 tlast=0
cyc 27 beat 7 tdata=c0000001 tlast=0
cyc 28 beat 8 tdata=c0000002 tlast=0
cyc 29 beat 9 tdata=c0000003 tlast=1
irq at cyc 32 err=0 beats=9 desc word0 after: 00000003 40000002 60000004
--- run 2: sink ready 50% of cycles ---
cyc  9 beat 1 tdata=a0000000 tlast=0
cyc 11 beat 2 tdata=a0000001 tlast=0
cyc 12 beat 3 tdata=a0000002 tlast=0
cyc 17 beat 4 tdata=b0000000 tlast=0
cyc 19 beat 5 tdata=b0000001 tlast=1
cyc 26 beat 6 tdata=c0000000 tlast=0
cyc 29 beat 7 tdata=c0000001 tlast=0
cyc 30 beat 8 tdata=c0000002 tlast=0
cyc 32 beat 9 tdata=c0000003 tlast=1
irq at cyc 34 err=0 beats=9 desc word0 after: 00000003 40000002 60000004
--- run 3: restart on a chain software has not handed back ---
irq err=1 (descriptor 0x10 has OWN=0)

```

Real output (Icarus Verilog 12). Cycle numbers are relative to the start pulse.

What the run shows:

- **Packetization is correct:** 9 beats, **TLAST** on beat 5 (end of the two-buffer packet) and beat 9, even though the first packet spans two descriptors - that is scatter-gather.
- **Back-pressure is lossless:** with the sink ready only half the time, the same 9 words arrive in order; the credit rule stalls memory reads instead of dropping data.
- **Write-back:** word 0 of each descriptor reads back with bit 31 (OWN) cleared (**80000003** -> **00000003**, **C0000002** -> **40000002**, **E0000004** -> **60000004**), handing the buffers back to software.
- **Ownership is enforced:** restarting on descriptors software has not re-armed ends immediately with **err=1** instead of re-sending stale buffers.
- **The cost of descriptor fetch is visible:** within a buffer the engine streams one word per cycle, but there is an 8-cycle bubble between buffers (write-back plus three dependent descriptor reads). With 3-word buffers that halves the throughput; with realistic memory latency (100+ cycles) it would be far worse. That is why production DMA engines **prefetch descriptors** into a small descriptor cache and overlap write-back with the next transfer.

INTERVIEW ANGLE: SIZING THE DMA FIFO

Q: 'Your DMA reads from DDR with 200 ns latency and must stream 8 bytes per cycle at 500 MHz. How deep is the FIFO and how many reads must be outstanding?' A: bandwidth = 4 GB/s; bandwidth x latency = 800 bytes = 100 beats must be in flight to keep the stream busy. The FIFO must accept all of them when they return (at least 100 entries, in practice 128) and the AXI master must support enough outstanding transactions: with 64-byte bursts, $800/64 = 12.5$, so at least 13 outstanding reads (IDs or a deep reorder-free queue). Too few outstanding transactions is the most common reason a DMA falls short of its bandwidth target.

24.14 Integration checklist

Integrating a protocol controller and PHY

- ☐ Clocks: every clock the IP needs is listed with frequency, source, jitter requirement and whether it may be gated; SDC constraints provided by the IP vendor are merged and reviewed.
- ☐ CDC: all crossings are identified (FIFOs, synchronizers, quasi-static configuration), and the CDC tool runs clean with justified waivers.
- ☐ Resets: reset per domain, release order matches the PHY data book, soft-reset scope documented.
- ☐ Power: power domains, isolation and retention for the controller, always-on wake-up logic, PHY power-down sequence, UPF reviewed.
- ☐ Bus interfaces: address map entry, AXI ID widths, outstanding transaction limits, burst sizes, QoS settings, coherency (ACE-Lite or cache maintenance in the driver).
- ☐ Buffers: SRAM sizes justified by bandwidth x latency, memory BIST and ECC/parity inserted where required.
- ☐ Interrupts: routed to the interrupt controller with the right type (level/edge/MSI), coalescing configured.
- ☐ PHY: interface (PIPE/DFI/UTMI/xMII/PPI) configuration matches on both sides; calibration resistors, reference clocks and supplies reach the macro; test modes (loopback, PRBS, BIST) are accessible.
- ☐ Pads: pin-mux, IO voltages, open-drain/fail-safe needs, ESD and package escape reviewed with board and package teams.
- ☐ DFT: scan wrappers or bypass for the PHY, IEEE 1500 / JTAG access to PHY test registers (Chapter 23).
- ☐ Software: register map published, reference driver running on emulation, programming guide covers bring-up and error recovery.
- ☐ Verification: controller tested with protocol VIP, SoC-level test through the real PHY model, compliance plan agreed (Chapter 25).

24.15 Summary

- Every protocol controller follows one skeleton: bus interface and CSRs, DMA, buffers, protocol engine, PCS, standard PHY interface, PHY and pads. Bugs live at the seams between the layers.
- Descriptors are a protocol between hardware and software: format, ownership, ordering (barrier before doorbell, data before status before interrupt) and coherency must all be specified.
- Size buffers from bandwidth x round-trip latency, burst and jitter requirements, and packet size; credits make the round-trip rule explicit.
- Standard PHY interfaces (PIPE, DFI, UTMI/ULPI, xMII, PPI, RDI/FDI) separate digital controllers from mixed-signal PHYs and define the verification boundary.
- Controllers span several clock domains; CDC points are the FIFOs, the configuration path, the interrupt path and the PCS elastic buffer.
- PHY bring-up follows a strict sequence: PLL lock, calibration, controller reset release, configuration, link training.
- Buy complex controllers, PHYs and VIP; build simple peripherals; review pads and pin-mux early; deliver a clean programming model.

24.16 Exercises

- 1 Draw the layered architecture of a USB 2.0 device controller with a UTMI+ PHY. Mark each clock domain and every CDC point.
- 2 A 10 Gb/s Ethernet MAC must honour PAUSE frames. The link partner may keep sending for up to 1 us after you request a pause (cable delay, frames in flight, reaction time). How much RX FIFO headroom is needed above the pause threshold? Add one maximum-size jumbo frame (9018 bytes) and give the total.
- 3 Extend the example DMA engine with a two-entry descriptor prefetch so that the next descriptor is fetched while the current buffer is streaming. Estimate the new throughput for 3-word buffers.

- 4 Write the driver sequence for receiving packets with an RX descriptor ring on a non-coherent DMA port. Where exactly do the cache clean and invalidate operations go, and what breaks if either is missing?
- 5 List the checks you would make before accepting a licensed PCIe Gen5 PHY hard macro for a 5 nm SoC.
- 6 Explain why a level-sensitive interrupt with W1C status bits cannot lose an event, while a pulse interrupt can. What must the hardware do when a set and a clear hit the same bit in the same cycle?

25. Verifying Protocols: VIP, Assertions, Compliance and Interoperability

A protocol controller is verified three times. First in simulation, against a model of the specification (verification IP, assertions, scoreboards, coverage). Then against **other people's implementations** - compliance test suites, certification labs and interoperability plugfests - because two designs that both 'meet the spec' can still fail to talk to each other. Finally in silicon, with protocol analyzers, bit-error-rate testers and eye scans. This chapter walks through all three, with a running assertion checker and an error-injection testbench that you can reuse as templates. The companion Verilog & SystemVerilog guide covers UVM mechanics; here the focus is on what is special about **protocols**.

25.1 Why protocols are hard to verify

- **The specification is the reference model** - hundreds of pages of prose, timing tables and state machines, with optional features, errata and ECNs. Every 'shall' is a check someone has to write.
- **Two sides must agree.** Bugs appear only with particular partners: a host that sends a legal but unusual sequence, a device that responds at the edge of a timing window.
- **Error paths dominate.** A link spends most of its code on retries, timeouts, recovery and low-power transitions that normal traffic never exercises.
- **Layers and clocks.** Transaction, link and physical layers interact across clock domains and power states; a bug may need a retry during a speed change during a power-state exit.
- **Timescales.** Link training, low-power entry/exit and timeouts take micro- to milliseconds - millions of cycles of simulation.

25.2 Architecture of protocol verification IP

Verification IP (VIP) packages the knowledge of a protocol into reusable testbench components. Whether written in UVM, cocotb or plain SystemVerilog, a VIP has the same shape:



The canonical VIP agent. A passive agent has only the monitor and checker and is used to observe a link between two real designs.

Component	Role for a protocol	Examples
Sequence items / sequences	Transactions at the protocol's level of abstraction, with constraints for legal and illegal values	AXI burst, USB transfer, PCIe TLP, Ethernet frame
Driver	Turns transactions into pin activity with correct timing; can be told to violate it (error injection)	Drives VALID/READY, SCL/SDA, D+/D-, PIPE lanes
Monitor	Reconstructs transactions from pins, never drives	Assembles bursts, frames, TLPs; timestamps them
Protocol checker	Checks every rule of the spec on every cycle (assertions)	Handshake stability, timing windows, legal state transitions
Scoreboard	Compares what came out with what should have, using a reference model	Data integrity, ordering, responses

Component	Role for a protocol	Examples
Coverage	Measures which features and corner cases were exercised	Transaction types x sizes x responses x errors
Configuration	Selects spec version, options, speeds, roles (host/device, master/slave)	USB 2.0 vs 3.2, AXI4 vs AXI5, lane count

COMMERCIAL VIP VERSUS HOME-GROWN

For complex standard protocols (PCIe, USB, Ethernet, DDR, MIPI, CXL, UCIe) teams almost always license VIP: it comes with thousands of spec-derived checks, compliance-style test suites and maintenance as the specification evolves. Home-grown VIP is right for simple buses, proprietary protocols and quick block-level checkers - like the ones in this chapter.

25.3 Protocol assertion libraries

Assertions are the cheapest, densest form of protocol checking: one line per rule, evaluated every cycle in simulation, in emulation (if synthesizable) and in formal verification. A good library is written once per protocol and **bound** to every instance of the interface. Some representative rules (SystemVerilog Assertions; shown for reference, not all of these constructs are run below):

```
// ---- AXI (any channel with VALID/READY/payload) ----
a_valid_hold: assert property (@(posedge aclk) disable iff (!aresetn)
    avalid && !awready | => avalid);
a_addr_stable: assert property (@(posedge aclk) disable iff (!aresetn)
    avalid && !awready | => $stable({awaddr, awlen, awsize, awburst}));
a_wrap_len:    assert property (@(posedge aclk) disable iff (!aresetn)
    avalid && awburst == 2'b10 | -> awlen inside {1, 3, 7, 15});
a_4k:         assert property (@(posedge aclk) disable iff (!aresetn)
    avalid && awburst == 2'b01 | ->
        (awaddr[11:0] + ((awlen + 1) << awsize)) <= 13'h1000);
// ---- APB: SETUP phase is exactly one cycle, then ACCESS ----
a_setup:      assert property (@(posedge pclk) disable iff (!presetn)
    psel && !penable | => psel && penable);
a_access_hold: assert property (@(posedge pclk) disable iff (!presetn)
    psel && penable && !pready | => psel && penable &&
    $stable({paddr, pwrite, pwrite}));
// ---- SPI mode 0: MOSI must not change at a rising (sampling) SCLK edge ----
a_spi_mode0:  assert property (@(posedge clk_chk) disable iff (cs_n)
    $rose(sclk) | -> $stable(mosi)); // clk_chk oversamples SCLK
// ---- liveness with a bound: every request gets a response ----
a_resp:      assert property (@(posedge aclk) disable iff (!aresetn)
    arvalid && arready | -> ##[1:256] rvalid);
```

A small AXI/APB/SPI assertion library (illustrative). The AXI rules come straight from the AMBA specification: no VALID withdrawal, stable payload during a stall, legal WRAP lengths, bursts must not cross 4 KB.

The last rule deserves a comment: a real AXI response has no fixed latency, so the 'within 256 cycles' bound is a **testbench assumption** (a watchdog), not a spec rule. Formal tools can prove 'eventually' properties without an arbitrary bound. The SPI rule shows another point: assertions on source-synchronous interfaces must be clocked by the interface clock or oversampled by a faster checker clock, and they must be disabled when chip select is inactive.

Now a runnable example. The checker below captures the two core valid/ready rules plus an X-check, and is connected to a deliberately buggy source that, when stalled, sometimes withdraws VALID and sometimes changes its payload. Verilator 5.020 supports simple concurrent assertions (implication, `$stable`, `$isunknown`) but not cycle-delay ranges in sequences, which is why the cover property is kept to a single-cycle expression.

```
// Protocol checker for a valid/ready (AXI-style) channel: bind it to any interface.
module vr_checker #(parameter int W = 8) (
    input logic clk, rst_n, valid, ready, input logic [W-1:0] data
);
    // once asserted, VALID must stay high until the handshake (AXI: no withdrawal)
    a_hold: assert property (@(posedge clk) disable iff (!rst_n)
        valid && !ready | => valid)
        else $error("VALID dropped before READY");
    // payload must be stable while waiting
    a_stable: assert property (@(posedge clk) disable iff (!rst_n)
        valid && !ready | => $stable(data))
        else $error("DATA changed during a stall");
    // no X on VALID out of reset
    a_known: assert property (@(posedge clk) disable iff (!rst_n) !$isunknown(valid));
    c_stall: cover property (@(posedge clk) disable iff (!rst_n) valid && !ready);
endmodule

// a deliberately buggy source that breaks both rules when it is stalled
module bad_src (input logic clk, rst_n, ready, output logic valid, output logic [7:0] data);
    logic [2:0] t;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin valid <= 0; data <= 0; t <= 0; end
        else begin
            t <= t + 1;
            if (!valid || ready) begin valid <= 1; data <= data + 1; end
            else if (t == 6) data <= data + 8'h10; // BUG 1: payload changes in a stall
            else if (t == 3) valid <= 1'b0; // BUG 2: VALID withdrawn
        end
end
endmodule

module tb;
    logic clk = 0, rst_n = 0, ready, valid; logic [7:0] data;
    always #5 clk = ~clk;
    bad_src src (.*);
    vr_checker chk (.*);
    initial begin
        ready = 0;
        #12 rst_n = 1;
        repeat (20) begin @(negedge clk); ready = ($urandom_range(3) == 0); end
        $finish;
    end
end
endmodule
```

sva.sv - a reusable valid/ready protocol checker, a buggy source and a small testbench.

```
[55] %Error: sva.sv:8: Assertion failed in TOP.tb.chk.a_hold: VALID dropped before READY
-Info: sva.sv:8: Verilog $stop, ignored due to +verilator+error+limit
[135] %Error: sva.sv:8: Assertion failed in TOP.tb.chk.a_hold: VALID dropped before READY
-Info: sva.sv:8: Verilog $stop, ignored due to +verilator+error+limit
[165] %Error: sva.sv:12: Assertion failed in TOP.tb.chk.a_stable: DATA changed during a stall
-Info: sva.sv:12: Verilog $stop, ignored due to +verilator+error+limit
```

Real output of Verilator 5.020 (`--binary --timing --assert`, run with `+verilator+error+limit+100` so that simulation continues after the first failure). Times are in ns.

Both bugs are caught at the exact cycle they occur, with the name of the rule that broke - far easier to debug than a scoreboard mismatch hundreds of cycles later. This is why every protocol interface in a serious SoC has its checker bound in every simulation, and why assertion-based VIP is also the property set for formal verification of simple interfaces (APB, AXI4-Lite slaves), where a formal tool can prove that no input sequence violates them.

25.4 Scoreboards for packet protocols

A scoreboard answers 'did the right data come out?'. For packet protocols that question has several parts, each needing a design decision:

Question	Scoreboard technique
Was every packet delivered, exactly once?	Expected queue; at end of test the queue must be empty; duplicates flagged
In the right order?	In-order FIFO compare when the protocol guarantees order (Ethernet port, AXI same ID); per-ID/per-stream queues when order is only per ID or per virtual channel (AXI different IDs, PCIe relaxed ordering)
With the right content?	Reference model applies the transformation (header insertion, CRC, padding, VLAN tag, encryption) before comparing
Were dropped packets supposed to be dropped?	Model predicts drops (bad CRC, filter miss, FIFO overflow policy) and counts them
Did errors get reported correctly?	Compare status fields, counters and interrupts with the injected errors

PITFALL: A SCOREBOARD THAT CANNOT FAIL

Common testbench bugs: the expected queue is filled from the DUT's own output (so it always matches); comparisons are skipped when the queue is empty; the end-of-test check for leftover packets is missing; or errors are counted but never make the test fail. Always run a **negative test** - break the DUT on purpose (the buggy source above, a flipped bit below) and confirm the testbench reports it.

25.5 Error injection

Error handling is specified as carefully as normal operation - and exercised far less. Error injection makes those paths run. Every protocol has its characteristic faults:

Injection	What it tests	Protocols
Bit flips in data or CRC	CRC checking, discard, error counters, retry	Ethernet, USB, PCIe (LCRC/ECRC), CAN, MIPI CSI-2 (ECC/CRC)
NACK / NAK / STALL / retry responses	Retry logic and limits	I2C NACK, USB NAK/STALL, PCIe NAK DLLP, AXI SLVERR/DECERR
Dropped or truncated packets	Timeouts, sequence-number gaps	PCIe replay timer, USB timeouts, TCP-like layers
Timeouts (no response)	Watchdogs and completion timeouts	PCIe completion timeout, I2C clock stretching limits, SMBus 35 ms timeout
Back-pressure / credit starvation	Buffers, flow control, deadlock freedom	AXI READY low, PCIe credits withheld, Ethernet PAUSE
Protocol violations	Robustness to a misbehaving partner	Illegal tokens, bad lengths, framing errors, arbitration loss
Physical impairments	CDR, equalization, elastic buffer	Jitter, ppm offset, lane skew, polarity inversion (in AMS or PHY models)

The following self-checking testbench exercises a small **CRC-8 protected packet link**: a transmitter appends a CRC-8 frame check sequence (polynomial 0x07, the CRC-8/SMBUS variant used by SMBus packet error checking), a receiver checks it and runs a timeout for packets that stall. Between them sits an error injector - the 'channel' - that for each packet chooses no error, a single-bit flip, a burst of up to 8 bits, three corrupted bytes, or a dropped FCS byte.

```
// CRC-8 (poly x^8+x^2+x+1 = 0x07, init 0x00, MSB first) packet link: TX appends the CRC,
// RX checks that the CRC over payload+CRC is zero and flags a timeout on a stalled packet.
```

```

function automatic logic [7:0] crc8_step(input logic [7:0] c, input logic [7:0] d);
    logic [7:0] x;
    x = c ^ d;
    for (int i = 0; i < 8; i++) x = x[7] ? ((x << 1) ^ 8'h07) : (x << 1);
    return x;
endfunction

module link_tx (
    input  logic clk, rst_n,
    input  logic s_valid, s_last, input logic [7:0] s_data, output logic s_ready,
    output logic m_valid, m_last, output logic [7:0] m_data
);
    logic [7:0] crc; logic send_crc;
    assign s_ready = !send_crc;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin crc <= 0; send_crc <= 0; m_valid <= 0; m_last <= 0; m_data <= 0; end
        else if (send_crc) begin // append FCS byte, end of packet
            m_valid <= 1; m_last <= 1; m_data <= crc; crc <= 0; send_crc <= 0;
        end else begin
            m_valid <= s_valid; m_last <= 0; m_data <= s_data;
            if (s_valid) begin crc <= crc8_step(crc, s_data); send_crc <= s_last; end
        end
    end
endmodule

module link_rx #(parameter int TMO = 16) (
    input  logic clk, rst_n,
    input  logic valid, last, input logic [7:0] data,
    output logic done, crc_ok, timeout
);
    logic [7:0] crc; logic in_pkt; logic [7:0] idle;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            crc <= 0; in_pkt <= 0; idle <= 0; done <= 0; crc_ok <= 0; timeout <= 0;
        end
        else begin
            done <= 0; timeout <= 0;
            if (valid) begin
                idle <= 0;
                if (last) begin // residue over payload+FCS must be 0
                    done <= 1; crc_ok <= (crc8_step(crc, data) == 0); crc <= 0; in_pkt <= 0;
                end
                else begin crc <= crc8_step(crc, data); in_pkt <= 1; end
            end else if (in_pkt) begin
                idle <= idle + 1;
                if (idle == TMO - 1) begin timeout <= 1; in_pkt <= 0; crc <= 0; idle <= 0; end
            end
        end
    end
endmodule

```

link.sv - CRC-8 transmitter and receiver with timeout.

```

module tb;
    logic clk = 0, rst_n = 0; always #5 clk = ~clk;
    logic sv = 0, sl = 0, sr, tv, tl, rv, rl, done, ok, tmo;
    logic [7:0] sd, td, rd;
    link_tx tx (.clk, .rst_n, .s_valid(sv), .s_last(sl), .s_data(sd), .s_ready(sr),
               .m_valid(tv), .m_last(tl), .m_data(td));
    link_rx rx (.clk, .rst_n, .valid(rv), .last(rl), .data(rd), .done, .crc_ok(ok),
               .timeout(tmo));

    // ----- error injector on the wire (the "channel") -----
    typedef enum {NONE, BIT1, BURST8, MULTI, DROP} inj_t;

```

```

inj_t kind; int pos, len, idx = 0; logic [7:0] crc_seen;
bit [7:0] flip [32]; // per-byte XOR pattern for this packet
logic [7:0] cur_flip;
always @* cur_flip = flip[idx];
assign rd = td ^ cur_flip;
assign rl = tl;
assign rv = tv && !(kind == DROP && tl); // DROP: lose the FCS byte
always @(posedge clk) if (tv) idx <= tl ? 0 : idx + 1;
// ----- monitor + scoreboard -----
bit [7:0] p [$], got_q [$];
int n_pkts, n_inj, n_det_crc, n_det_tmo, n_escape, n_false, n_good, logged;
always @(posedge clk) if (rv && !rl) got_q.push_back(rd);
function bit same(); // received payload == sent payload?
    if (got_q.size() != p.size()) return 0;
    foreach (p[i]) if (got_q[i] != p[i]) return 0;
    return 1;
endfunction
task check();
    string res;
    do @(negedge clk); while (!(done || tmo)); // RX verdict for this packet
    if (tmo) begin res = "TIMEOUT"; n_det_tmo++; end
    else if (!ok) begin res = "CRC ERROR"; n_det_crc++; end
    else if (!same()) begin res = "SILENT CORRUPTION (CRC escape)"; n_escape++; end
    else begin res = "ok"; n_good++; end
    if (kind == NONE && res != "ok") n_false++;
    if ((kind != NONE && logged < 5) || (kind != NONE && res == "ok"))
        || res.substr(0, 0) == "S" begin
        logged++; $display("pkt %4d inject=%-6s -> %s", n_pkts, kind.name(), res);
    end
    got_q.delete();
endtask
task send();
    for (int i = 0; i < p.size(); i++) begin
        sv <= 1; sd <= p[i]; sl <= (i == p.size() - 1); @(posedge clk);
    end
    sv <= 0; sl <= 0; @(posedge clk); // FCS cycle
endtask
initial begin
    int r, n;
    repeat (2) @(posedge clk); rst_n <= 1; @(posedge clk);
    // known-answer test: CRC-8 of ASCII "123456789" must be 0xF4 (CRC-8/SMBUS check value)
    p = '{"1","2","3","4","5","6","7","8","9"}'; kind = NONE; foreach (flip[i]) flip[i] = 0;
    fork send(); begin wait (tv && tl); crc_seen = td; end join
    $display("KAT: CRC-8 of \"123456789\" = %02h (expected f4)", crc_seen);
    check(); n_good = 0; logged = 0; repeat (20) @(posedge clk);
    for (n_pkts = 0; n_pkts < 4000; n_pkts++) begin
        p.delete(); n = $urandom_range(16, 4);
        repeat (n) p.push_back($urandom);
        r = $urandom_range(99);
        kind = (r < 50) ? NONE : (r < 62) ? BIT1 : (r < 75) ? BURST8 : (r < 90) ? MULTI : DROP;
        foreach (flip[i]) flip[i] = 0;
        case (kind)
            BIT1: begin
                pos = $urandom_range(8 * (n + 1) - 1); flip[pos / 8] ^= 8'h80 >> (pos % 8);
            end
            BURST8: begin // up to 8 consecutive bits, ends are flipped
                len = $urandom_range(8, 2); pos = $urandom_range(8 * (n + 1) - len);
                for (int b = 0; b < len; b++)
                    if (b == 0 || b == len - 1 || $urandom_range(1))
                        flip[(pos + b) / 8] ^= 8'h80 >> ((pos + b) % 8);
            end
        endcase
    end
end

```

```

        MULTI: repeat (3) flip[$urandom_range(n)] ^= $urandom_range(255, 1);
        default: ;
    endcase
    if (kind != NONE) n_inj++;
    fork send(); check(); join
    repeat (20) @(posedge clk); // idle gap > RX timeout
end
$display("packets=%0d injected=%0d | crc_err=%0d timeout=%0d escaped=%0d | %s",
        n_pkts, n_inj, n_det_crc, n_det_tmo, n_escape,
        $sformatf("ok=%0d false=%0d", n_good, n_false));
$finish;
end
endmodule

```

tb_link.v - error injector, monitor and scoreboard. The scoreboard compares the received payload with what was sent, independently of the CRC verdict.

```

KAT: CRC-8 of "123456789" = f4 (expected f4)
pkt   0  inject=BURST8 -> CRC ERROR
pkt   2  inject=BIT1   -> CRC ERROR
pkt   3  inject=MULTI  -> CRC ERROR
pkt   4  inject=DROP   -> TIMEOUT
pkt   6  inject=DROP   -> TIMEOUT
pkt  434 inject=MULTI  -> SILENT CORRUPTION (CRC escape)
pkt 2865 inject=MULTI  -> SILENT CORRUPTION (CRC escape)
pkt 3066 inject=MULTI  -> SILENT CORRUPTION (CRC escape)
packets=4000 injected=2022 | crc_err=1671 timeout=348 escaped=3 | ok=1978 false=0

```

Real output of Verilator 5.020 (`--binary --timing`). The first five injected packets are logged, plus every silent corruption.

Reading the result:

- The **known-answer test** passes: CRC-8 of the ASCII string '123456789' is 0xF4, the published check value of CRC-8/SMBUS, and the same value was computed by an independent Python reference.
- Every single-bit error and every burst of up to 8 bits was caught (an n-bit CRC detects all bursts of length $\leq n$), and every dropped FCS byte ended in a TIMEOUT rather than a hang.
- **Three packets with multi-byte corruption passed the CRC check.** The scoreboard caught them anyway because it compares payloads, not CRC verdicts, and reported **silent corruption**. About 600 packets got three corrupted bytes, and a random error pattern escapes an 8-bit CRC with probability about 1/256, so 2 to 3 escapes is exactly what theory predicts. Chapter 26 measures this for CRC-8 versus CRC-32.
- **No false alarms:** none of the 1978 clean packets was flagged. A checker that raises false errors is as broken as one that misses real ones.

INTERVIEW ANGLE: WHAT DOES THIS TB PROVE?

It proves the RX detects what a CRC-8 can detect, reports timeouts, and never flags good packets. It also demonstrates that CRC-8 is too weak for this channel if multi-byte corruption is plausible - a system-level finding, not an RTL bug. A good DV engineer reports both kinds of result.

25.6 Coverage models for protocols

Functional coverage is the answer to 'are we done?'. For a protocol it is built from the specification's features and their combinations:

```

covergroup axi_wr_cg with function sample(axi_txn t);
  burst: coverpoint t.burst { bins fixed = {0}; bins incr = {1}; bins wrap = {2}; }
  len:   coverpoint t.len   { bins one = {0}; bins short_ = {[1:3]};
                           bins mid = {[4:15]}; bins long_ = {[16:255]}; }
  size:  coverpoint t.size  { bins b[] = {[0:3]}; } // 1..8 bytes/beat
  resp:  coverpoint t.resp  { bins okay = {0}; bins exokay = {1};
                           bins slverr = {2}; bins decerr = {3}; }

  unal:  coverpoint t.unaligned;
  stall: coverpoint t.max_wready_stall { bins none = {0}; bins some = {[1:4]};
                                       bins long_ = {[5:$]}; }

  burst_x_len: cross burst, len { ignore_bins wrap_one = binsof(burst.wrap)
                                && binsof(len.one); }

  burst_x_resp: cross burst, resp;
endgroup

```

A protocol coverage model: transaction types x sizes x responses x timing, with illegal combinations excluded (a WRAP burst of length 1 is illegal in AXI).

- **Transaction space:** every type, size, length, alignment and response, and their meaningful crosses.
- **State space:** every state and legal transition of the protocol FSMs (link training states, power states), including transitions taken during errors.
- **Error space:** every injected error type in every relevant state (a CRC error during retry, a timeout during a speed change).
- **Timing space:** back-to-back traffic, maximum stalls, minimum gaps, simultaneous events on different channels.
- **Configuration space:** speeds, widths, lane counts, optional features on and off.

LINK COVERAGE TO THE SPECIFICATION

Mature teams keep a verification plan that maps every numbered requirement of the spec (and every compliance checklist item) to assertions, tests and coverage points. When a spec errata or ECN arrives, the plan shows exactly which checks must change.

25.7 Compliance test suites and certification

Passing your own testbench proves you implemented your reading of the specification. Compliance testing checks it against the standards body's reading, using independent test equipment and suites. For consumer and interoperability-critical protocols, passing is the price of using the logo.

Protocol family	Organization / programme	What is tested
USB 2.0 / 3.x / USB4 / Type-C / PD	USB-IF compliance workshops and authorized independent test labs; products listed on the integrators list	Electrical (eye, jitter, signal quality), link and protocol layer, Type-C/PD, interoperability with a gold suite of hosts/devices
PCI Express	PCI-SIG compliance workshops; Integrators List	Electrical (TX/RX, reference clock), link/transaction layer, configuration space checks, interoperability
Ethernet (and many others)	UNH-IOL (University of New Hampshire InterOperability Laboratory); IEEE 802.3 conformance test plans	PMA/PMD electrical, auto-negotiation, MAC/PCS conformance, interoperability
MIPI (D-PHY, C-PHY, CSI-2, DSI, I3C)	MIPI Alliance conformance test suites, member interoperability events and third-party labs	PHY electrical conformance, protocol conformance
HDMI / DisplayPort	HDMI Authorized Test Centers; VESA compliance programme	Electrical, protocol, HDCP, EDID and link training
CAN / CAN FD	ISO 16845 conformance test plans, often executed by test houses	Protocol controller conformance

Protocol family	Organization / programme	What is tested
Bluetooth, Wi-Fi (for completeness)	Bluetooth SIG qualification; Wi-Fi Alliance certification	RF and protocol

- Compliance tests are mostly **black-box tests at the pins** run on a test board, so the silicon must have the required test modes: compliance patterns, loopback, PRBS generators/checkers, test modes such as USB 2.0 Test_J/Test_K/Test_Packet or the PCIe compliance state of the LTSSM. Missing test modes are an RTL bug found far too late.
- Most standards bodies make **pre-silicon** checks possible too: commercial VIP ships with compliance-style suites, and some bodies publish test specifications to members.
- Budget time for compliance: a failed electrical test may need a board respin or PHY setting changes; a failed protocol test may need a firmware patch or - worst case - a silicon respin.

25.8 Interoperability plugfests

A **plugfest** is an event where many vendors bring pre-release products and test them against each other, typically under NDA and organized by the standards body or a lab. Interoperability failures found there are often ambiguities in the specification rather than clear bugs: two readings of a timing window, an optional feature one side assumes, an unusual but legal packet sequence. The fixes flow back into errata, ECNs and the next compliance suite. For an SoC team the practical lesson is to test early against **several** real partners (hosts, switches, memories, displays), not just the one on the evaluation board.

25.9 Emulation and FPGA prototyping

Simulation runs a large SoC at perhaps 10-1000 cycles per second, hardware emulators at around a megahertz, FPGA prototypes at tens of megahertz. Protocols are a special difficulty for both, because the outside world does not slow down: a USB host, an Ethernet switch or a PCIe root complex expects real timing.

real PC / switch / phone (PCIe Gen4 x4 at full rate) <==>	speed bridge [PHY + controller] <==>	emulator [DUT at 1 MHz]
	[large buffers,]	transactors
	[protocol-level]	or pins
	[rate adaptation]	

alternatives: virtual host models (transactors in the emulator), FPGA prototypes with the IP's FPGA PHY, or a slowed-down link where allowed

A speed bridge terminates the real-speed protocol and replays it to the slow DUT, stretching timers where the protocol allows.

- **Speed bridges / rate adapters:** boxes that terminate the protocol at full speed on one side and talk to the emulator at low speed on the other. They work well for protocols with flow control and tolerant timers (PCIe, USB, Ethernet with buffering), badly for hard real-time links.
- **Transactors / virtual models:** the peer device (a host, a memory) is modelled in software connected to the emulator through a transaction-level interface. Faster to set up, no real-world quirks.
- **FPGA prototypes:** controller RTL on an FPGA with the FPGA's SerDes standing in for the ASIC PHY; excellent for firmware and driver bring-up, and for running real operating systems and compliance-like software tests before tapeout.
- What prototypes cannot tell you: analog behaviour, the real PHY's equalization, and timing at the target clock frequency.

25.10 Post-silicon validation

When silicon returns, protocol validation moves to the lab. Its tools:

Tool	What it shows	Used for
Protocol analyzer / exerciser	Decoded packets, timing, errors, LTSSM or link-state traces; exercisers also generate traffic and inject errors	PCIe, USB, Ethernet, MIPI, DDR bus analyzers, low-speed (I2C/SPI/CAN) decoders on logic analyzers and scopes
BERT (bit error rate tester)	Pattern generator + error detector: measures BER, jitter tolerance, stressed-eye RX performance	SerDes receiver characterization and compliance
Real-time oscilloscope	Eye diagrams, jitter decomposition (RJ/DJ), rise times, compliance masks	TX electrical compliance
On-die eye scan / margining	Receiver's own view of the eye: sweep sampling phase and threshold and count errors	In-system margin checks; PCIe Lane Margining at the Receiver (Gen4+)
PRBS generators/checkers and loopbacks	Near-end/far-end loopback, PRBS7/15/23/31 patterns with error counters	Bring-up and production test of links without a partner
On-chip trace and counters	Error counters, state histories, trace buffers (Chapter 23)	Debugging rare failures in the field

DESIGN FOR VALIDATION

Everything in the table above except the external instruments must be designed into the RTL: PRBS generators/checkers per lane, loopback paths, eye-scan hooks in the PHY, sticky error counters, link-state trace buffers, test-mode registers reachable over JTAG. They cost little area and save weeks in the lab.

25.11 Protocol verification sign-off checklist

Before declaring a protocol block verified

- ☐ Verification plan maps spec requirements and compliance checklist items to tests, assertions and coverage.
- ☐ Protocol checkers (SVA) are bound on every interface instance in every test, with zero unexpected failures.
- ☐ Scoreboard checks delivery, ordering, content and error reporting; negative tests prove it can fail.
- ☐ Error injection covers each fault class in each relevant state; recovery returns the link to normal operation.
- ☐ Functional coverage closed (or holes reviewed and waived); code coverage reviewed for unreachable or untested logic.
- ☐ Low-power entry/exit, reset during traffic and speed/width changes tested.
- ☐ Interoperability tests with at least one independent model or real partner (VIP from another vendor, FPGA prototype with real devices).
- ☐ Compliance test modes (patterns, loopback, PRBS) present and tested; compliance plan and lab slots booked.
- ☐ Formal proof of the simple interfaces (APB/AXI-Lite slaves, arbiters) and CDC/RDC sign-off done.

25.12 Summary

- Protocol verification happens in three stages: simulation against a model of the spec, compliance and interoperability against other implementations, and post-silicon validation in the lab.
- VIP = sequences, driver, monitor, protocol checker, scoreboard and coverage; license it for complex standards, write it for simple ones.
- Assertions catch protocol violations at the exact cycle; bind a checker to every interface instance and reuse the same properties for formal verification.
- Packet scoreboards must check delivery, order, content and error reporting - and must be proven able to fail.
- Error injection exercises the code that matters most in the field; the example showed CRC-8 catching all short bursts but letting about 1/256 of random multi-byte corruptions through.

- Compliance (USB-IF, PCI-SIG, UNH-IOL, MIPI, HDMI/VESA) needs test modes in the silicon; plugfests find specification ambiguities.
- Emulation needs speed bridges or transactors for real-world protocols; silicon validation needs analyzers, BERTs, eye scans and designed-in PRBS/loopback/counters.

25.13 Exercises

- 1 Write SVA properties for an I2C target: (a) SDA may change only while SCL is low, except for START and STOP; (b) after the eighth data bit the transmitter releases SDA for the ACK bit. How do you clock the checker?
- 2 Design a scoreboard for an AXI interconnect with 4 managers and 8 subordinates, where ordering is only guaranteed per ID. What data structure do you use and how do you detect a lost transaction?
- 3 Modify the CRC-8 link testbench to use a 16-bit CRC (CRC-16/CCITT, polynomial 0x1021). Predict the number of silent corruptions for the same run and check your prediction.
- 4 Write a coverage model for USB 2.0 bulk transfers: token types, data PIDs (DATA0/DATA1 toggle), handshake responses, packet sizes and the error cases you would inject.
- 5 List the test modes a PCIe Gen4 controller and PHY must provide for compliance and lab bring-up. Which of them must be designed into the RTL?
- 6 Explain why a speed bridge can work for PCIe but is much harder for MIPI CSI-2 from a real camera sensor.

26. Reliability and Security on Links: CRC/ECC, Retry, Safety and Link Encryption

Every link eventually delivers a wrong bit. Every link that leaves a chip can eventually be probed, replayed or tampered with. And in cars, factories and medical devices a wrong bit can hurt someone. This chapter pulls together the techniques that make links **reliable** (detect, correct, retry, report), **safe** (ISO 26262 diagnostic coverage) and **secure** (authentication, encryption, access control on buses, secure debug). Chapter 3 introduced parity, CRC and ECC; here we quantify them and place them in real protocols.

26.1 Error models: how links fail

Error type	Cause	Typical protection
Random single-bit errors	Thermal/random noise and jitter on SerDes links; the bit error rate (BER) is set by the eye margin	CRC + retry; FEC at high rates
Burst errors	DFE error propagation in equalized receivers, crosstalk events, supply noise, a lost CDR lock, scrambler multiplication	CRC (detects bursts $\leq$ its width), interleaved or symbol-based FEC (Reed-Solomon)
Soft errors in storage	Particle strikes (alpha, neutrons) on SRAM/DRAM/flops	ECC on memories, parity on buffers, scrubbing
Hard / permanent faults	Opens, shorts, stuck-at, aging, electromigration	BIST, lane degrade/spare lanes, safety mechanisms, redundancy
Systematic / protocol errors	Design bugs, misconfiguration, a misbehaving partner	Timeouts, protocol checkers, sequence numbers, error reporting
Malicious	Probing, injection, replay, glitching	Authentication, encryption with integrity, tamper detection

$$\text{Errors per second} = \text{BER} \times \text{line rate}$$

$$\text{BER } 1\text{e-}12 \text{ at } 16 \text{ Gb/s} \rightarrow 0.016 \text{ errors/s} = \text{about one error per minute}$$

$$\text{BER } 1\text{e-}12 \text{ at } 16 \text{ lanes} \times 32 \text{ Gb/s} \rightarrow \text{about one error every } 2 \text{ s}$$

$$\text{BER } 1\text{e-}6 \text{ (PAM4 raw, before FEC) at } 64 \text{ Gb/s} \rightarrow 64,000 \text{ errors/s}$$

$$\text{Poisson: } P(k \text{ errors in an } n\text{-bit frame}) = (n p)^k e^{-(n p)} / k!, \quad p = \text{BER}$$

Even an excellent link makes errors routinely; the protocol must detect every one that matters.

Two consequences drive protocol design. First, at modern rates **detection plus retry** is not optional anywhere errors can corrupt state. Second, PAM4 signalling (PCIe 6.0, 200G/400G Ethernet) trades raw BER for bandwidth: its raw BER is orders of magnitude worse than NRZ, so **forward error correction** becomes mandatory and its latency becomes a design parameter.

26.2 What a CRC can detect

A CRC with generator polynomial $g(x)$ of degree r appends r check bits. An error pattern $e(x)$ is **undetected** exactly when $g(x)$ divides $e(x)$. From that algebra come the guarantees:

Property	Condition	Consequence
All single-bit errors	$g(x)$ has at least two terms	Always true for standard CRCs
All odd numbers of bit errors	$(x + 1)$ divides $g(x)$	CRC-8 0x07, CRC-16/CCITT; not CRC-32 (IEEE)

Property	Condition	Consequence
All double-bit errors	Codeword length $\leq$ order of $g(x)$ (the smallest d with $g \mid x^d + 1$)	The Hamming distance falls to 2 beyond that length
All bursts of length $\leq r$	$g(x)$ has a nonzero x^0 term	Any 8-bit CRC catches every 8-bit burst
Bursts of length $r + 1$	-	Escape probability $2^{-(r-1)}$
Longer bursts / random garbage	-	Escape probability about 2^{-r}

The **Hamming distance (HD)** of a CRC-protected frame is the smallest number of bit flips that can go undetected; it depends on both the polynomial and the frame length. Koopman's published tables are the standard reference: for example the IEEE 802.3 CRC-32 gives HD = 4 over every Ethernet frame length, while the simple CRC-8 0x07 gives HD = 4 only up to 119 data bits (15 bytes) and HD = 2 beyond. The choice of CRC is therefore a function of frame size:

CRC	Where	Protects
CRC-5	USB token packets	11-bit address/endpoint or frame number
CRC-16	USB data packets; many serial links	Up to 1024-byte USB 2.0 payloads
CRC-15, CRC-17, CRC-21	CAN, CAN FD (17 for ≤ 16 data bytes, 21 for longer)	Short automotive frames (plus bit stuffing checks)
CRC-32 (IEEE 802.3)	Ethernet FCS, SATA, many others	Frames up to 1518 bytes; HD = 4 still holds for 9 KB jumbo frames
32-bit LCRC / ECRC	PCIe data link layer / end-to-end	TLPs per hop / end to end
CRC-16 or CRC-32 variants	MIPI CSI-2 packet footer (16-bit); SATA, UCIe, CXL flits	Payloads and flits

CRCs you will meet. Always pair a CRC with a frame length limit; its guarantees hold only up to a length.

Experiment: CRC-8 versus CRC-32 under random and burst errors

The script below builds 64-byte frames with random payloads, appends either a CRC-8 (polynomial 0x07) or the Ethernet CRC-32, injects errors of a chosen kind and counts how many corrupted frames pass the check. It first asserts the known check values of both CRCs (0xF4 and 0xCBF43926 for '123456789'), then finishes with an exact calculation of how many two-bit error patterns escape CRC-8 at different frame lengths.

```
# Inject random and burst errors into CRC-protected frames; count what gets through.
import random, zlib

def make_crc8_table(poly=0x07):
    # CRC-8, MSB first, init 0x00 (CRC-8/SMBUS)
    t = []
    for b in range(256):
        c = b
        for _ in range(8):
            c = ((c << 1) ^ poly) & 0xFF if c & 0x80 else (c << 1) & 0xFF
        t.append(c)
    return t
T8 = make_crc8_table()

def crc8(data):
    c = 0
    for b in data:
        c = T8[c ^ b]
    return c

CRCS = {"CRC-8 ": (crc8, 1),
        "CRC-32": (lambda d: zlib.crc32(d), 4)} # IEEE 802.3 CRC-32
assert crc8(b"123456789") == 0xF4 and zlib.crc32(b"123456789") == 0xCBF43926
```

```

def frame(payload, name):
    f, n = CRCS[name]
    return payload + f(payload).to_bytes(n, "big")

def detected(rx, name):
    f, n = CRCS[name]
    return f(rx[:n]) != int.from_bytes(rx[:n], "big")

def corrupt(fr, kind, rng):
    nbits = 8 * len(fr)
    v = int.from_bytes(fr, "big")
    if kind[0] == "rand":
        # k independent random bit flips
        for pos in rng.sample(range(nbits), kind[1]):
            v ^= 1 << pos
    elif kind[0] == "burst":
        # burst of length L: both ends flipped
        L = kind[1]
        s = rng.randrange(nbits - L + 1)
        pat = (1 << (L - 1)) | 1 | (rng.getrandbits(L) if L > 2 else 0)
        v ^= (pat & ((1 << L) - 1)) << s
    else:
        # whole frame replaced by garbage
        v = rng.getrandbits(nbits)
    return v.to_bytes(len(fr), "big")

rng = random.Random(2026)
TRIALS, PAYLOAD = 100000, 64
kinds = [("rand", 1), ("rand", 2), ("rand", 3), ("rand", 4), ("burst", 8), ("burst", 9),
          ("burst", 16), ("burst", 32), ("burst", 33), ("burst", 64), ("garbage", 0)]
print("%d trials per row, %d-byte payload + FCS (undetected / trials)" % (TRIALS, PAYLOAD))
print("%-14s %-22s %-22s" % ("error pattern", "CRC-8 (0x07)", "CRC-32 (802.3)"))
for kind in kinds:
    row = []
    for name in CRCS:
        miss = 0
        for _ in range(TRIALS):
            p = rng.randbytes(PAYLOAD)
            fr = frame(p, name)
            rx = corrupt(fr, kind, rng)
            if rx != fr and not detected(rx, name):
                miss += 1
        row.append("%5d (%.2e)" % (miss, miss / TRIALS))
    label = "%d random bits" % kind[1] if kind[0] == "rand" else \
            "burst L=%d" % kind[1] if kind[0] == "burst" else "garbage frame"
    print("%-14s %-22s %-22s" % (label, row[0], row[1]))

# Exhaustive: 2-bit errors x^i + x^j escape iff g(x) divides x^(j-i) + 1
def pmod(a, g):
    dg = g.bit_length() - 1
    while a.bit_length() - 1 >= dg:
        a ^= g << (a.bit_length() - 1 - dg)
    return a
G8 = 0x107
order = next(d for d in range(1, 1000) if pmod((1 << d) | 1, G8) == 0)
print("CRC-8 0x07: smallest d with x^d = 1 mod g(x) is %d -> two flips %d bits apart escape"
      % (order, order))
for nb in (8, 16, 64):
    n = 8 * (nb + 1)
    bad = sum(n - d for d in range(1, n) if pmod((1 << d) | 1, G8) == 0)
    print(" %2d-byte payload: %6d of %7d two-bit patterns undetected"
          % (nb, bad, n * (n - 1) // 2))

```

crc_exp.py - Monte-Carlo error injection on CRC-protected frames plus an exhaustive two-bit analysis.

```

100000 trials per row, 64-byte payload + FCS (undetected / trials)
error pattern CRC-8 (0x07) CRC-32 (802.3)
1 random bits 0 (0.00e+00) 0 (0.00e+00)
2 random bits 554 (5.54e-03) 0 (0.00e+00)
3 random bits 0 (0.00e+00) 0 (0.00e+00)
4 random bits 789 (7.89e-03) 0 (0.00e+00)
burst L=8 0 (0.00e+00) 0 (0.00e+00)
burst L=9 787 (7.87e-03) 0 (0.00e+00)
burst L=16 390 (3.90e-03) 0 (0.00e+00)
burst L=32 378 (3.78e-03) 0 (0.00e+00)
burst L=33 449 (4.49e-03) 0 (0.00e+00)
burst L=64 383 (3.83e-03) 0 (0.00e+00)
garbage frame 403 (4.03e-03) 0 (0.00e+00)
CRC-8 0x07: smallest d with x^d = 1 mod g(x) is 127 -> two flips 127 bits apart escape
8-byte payload: 0 of 2556 two-bit patterns undetected
16-byte payload: 9 of 9180 two-bit patterns undetected
64-byte payload: 810 of 134940 two-bit patterns undetected

```

Real output (Python 3, about 10 s).

Every line of that table matches the theory:

- **Odd numbers of flips never escape CRC-8** (1 and 3 random bits: 0 escapes) because $0x107 = (x + 1)(x^7 + x^6 + x^5 + x^4 + x^3 + x^2 + 1)$ has the factor $x + 1$.
- **Two random flips do escape CRC-8** at this frame length: the order of $g(x)$ is 127, so two flips exactly 127 bits apart (or a multiple) are invisible. On a 520-bit codeword 810 of the 134,940 two-bit patterns (0.60 %) escape; the Monte-Carlo run measured 0.55 %. On an 8-byte payload the codeword is shorter than 127 bits and there are no such escapes - HD depends on length.
- **Bursts up to 8 bits are always caught**, bursts of 9 escape at $7.87e-3$, matching $2^{-7} = 7.81e-3$, and longer bursts or garbage escape at about $3.9e-3 = 2^{-8}$.
- **CRC-32 caught everything** in 1.1 million corrupted frames. Its escape probability for random garbage is about $2^{-32} = 2.3e-10$, so zero escapes is expected; demonstrating an escape would need billions of trials. For the 4-random-bit row, CRC-32's HD of 4 at this length means some 4-bit patterns can escape in principle, but they are a vanishingly small fraction.

RESIDUAL ERROR RATE

What a system designer actually needs is the **residual (undetected) error rate** = frame error rate x probability that an error escapes the check. With BER $1e-12$, 1500-byte frames and CRC-32 it is far below one undetected frame in the life of the universe for random errors - but only if the error model is right. Bursts from DFE error propagation or a scrambler with multiplying behaviour change the picture, which is why standards analyse CRC and scrambler together.

26.3 Forward error correction

FEC adds redundancy so the receiver can **correct** errors without a retransmission. Links use it when the raw BER is too high for detect-and-retry to be efficient, or when there is no return channel or no time to retry.

FEC	Where	Key numbers
RS(528,514) 'KR4'	100GBASE-KR4/CR4 and other 25G-per-lane NRZ Ethernet	Reed-Solomon over 10-bit symbols, 14 parity symbols, corrects up to 7 symbol errors per codeword
RS(544,514) 'KP4'	50G/100G/200G/400G PAM4 Ethernet (e.g. 400GBASE-DR4), 100GBASE-KP4	30 parity symbols, corrects up to 15 symbol errors; a 10-bit symbol absorbs a whole short burst
PCIe 6.0 FLIT FEC	PCIe 6.0 (64 GT/s PAM4), CXL 3.x	256-byte flit: 236 bytes TLP, 6 bytes DLP, 8 bytes CRC, 6 bytes FEC made of three interleaved single-symbol-correcting codes; lightweight for low latency, with CRC + retry behind it

FEC	Where	Key numbers
LDPC	10GBASE-T, Wi-Fi, 5G, some SerDes proposals	Near-Shannon performance, iterative decoding, higher latency and power
Hamming / BCH	Memories, NAND flash (BCH/LDPC), some low-latency links	Bit-level correction, simple decoders

PCIe 6.0 flit (256 bytes)

```
+-----+-----+-----+-----+
| TLP bytes (236)          | DLP 6 | CRC 8 | FEC 6 |
+-----+-----+-----+-----+
```

receiver: FEC corrects small errors -> CRC checks the result
CRC fail -> NAK -> replay the flit (link-level retry)

Ethernet 400G: 64b/66b blocks -> transcode 256b/257b -> RS(544,514)
symbols interleaved across lanes; uncorrectable -> mark frame bad

FEC in two modern links. PCIe keeps FEC deliberately light (a few ns of latency) and relies on CRC + retry for the rest; Ethernet uses a strong RS code because it has no link-level retry.

WHY SYMBOL-BASED CODES LIKE BURSTS

A Reed-Solomon code over 10-bit symbols counts errors in symbols, not bits. A 9-bit burst that lands inside one or two symbols costs only one or two of its 15 correctable symbols. That is exactly the error shape a DFE receiver produces, and why RS rather than a bit-level code protects PAM4 Ethernet.

FEC is not free: RS(544,514) decoding adds on the order of 100 ns of latency at 100G-class rates, and its gates and SRAM are a noticeable part of a high-speed MAC/PCS. PCIe 6.0 chose a lightweight FEC precisely to keep load-to-use latency low.

26.4 ECC on memories and links

Error-correcting codes protect data **at rest**: SRAM buffers, caches, DRAM, retry buffers. The workhorse is the extended Hamming **SECDED** code - single-error correction, double-error detection. For 64 data bits it needs 8 check bits, the familiar (72,64) code of ECC DIMMs and many on-chip memories. Here is a byte-wide version, SECDED(13,8), with an exhaustive test of every single, double and triple error on every data value:

```
// Extended Hamming SECDED(13,8): code bit 0 = overall parity, bits 1..12 = Hamming
// positions (parity at 1,2,4,8; data at 3,5,6,7,9,10,11,12).
module secded_enc (input logic [7:0] d, output logic [12:0] c);
    always @* begin
        c = '0;
        {c[12], c[11], c[10], c[9], c[7], c[6], c[5], c[3]} = d;
        c[1] = c[3] ^ c[5] ^ c[7] ^ c[9] ^ c[11];
        c[2] = c[3] ^ c[6] ^ c[7] ^ c[10] ^ c[11];
        c[4] = c[5] ^ c[6] ^ c[7] ^ c[12];
        c[8] = c[9] ^ c[10] ^ c[11] ^ c[12];
        c[0] = ^c[12:1]; // overall parity -> even weight
    end
endmodule

module secded_dec (input logic [12:0] c, output logic [7:0] d,
                  output logic ce, output logic ue); // corrected / uncorrectable
    logic [3:0] s; logic odd; logic [12:0] f;
    always @* begin
        s = '0;
        for (int i = 1; i <= 12; i++) if (c[i]) s ^= 4'(i); // syndrome = XOR of set positions
        odd = ^c;
        f = c;
        ce = odd; // odd weight: one bit flipped
        ue = !odd && (s != 0); // even weight, bad syndrome: two bits
    end
endmodule
```

```

    if (odd && s <= 12) f[s] = ~f[s];          // s == 0 -> the parity bit itself
    d = {f[12], f[11], f[10], f[9], f[7], f[6], f[5], f[3]};
end
endmodule

```

secded.sv - SECDDED(13,8) encoder and decoder. The syndrome is the XOR of the positions of all set bits; the overall parity bit distinguishes single from double errors.

```

module tb;
  logic [7:0] d, q; logic [12:0] c, r; logic ce, ue;
  secded_enc enc (.d, .c);
  secded_dec dec (.c(r), .d(q), .ce, .ue);
  int n [4], ok [4], det [4], silent [4];          // indexed by number of flipped bits
  task automatic apply(input int k, input logic [12:0] e);
    r = c ^ e; #1;
    n[k]++;
    if (ue)          det[k]++;                    // flagged uncorrectable
    else if (q == d) ok[k]++;                    // clean or correctly corrected
    else             silent[k]++;                // wrong data delivered as good
  endtask
  initial begin
    for (int v = 0; v < 256; v++) begin
      d = v; #1;
      apply(0, 0);
      for (int i = 0; i < 13; i++) begin
        apply(1, 13'(1) << i);
        for (int j = i + 1; j < 13; j++) begin
          apply(2, (13'(1) << i) | (13'(1) << j));
          for (int k = j + 1; k < 13; k++)
            apply(3, (13'(1) << i) | (13'(1) << j) | (13'(1) << k));
        end
      end
    end
    for (int k = 0; k < 4; k++)
      $display("%0d-bit errors: %6d cases  correct data=%6d  flagged UE=%6d  silent bad=%6d",
        k, n[k], ok[k], det[k], silent[k]);
  end
endmodule

```

tb_secded.sv - exhaustive error injection: all 256 data values x all 1-, 2- and 3-bit error patterns.

0-bit errors:	256 cases	correct data=	256	flagged UE=	0	silent bad=	0
1-bit errors:	3328 cases	correct data=	3328	flagged UE=	0	silent bad=	0
2-bit errors:	19968 cases	correct data=	0	flagged UE=	19968	silent bad=	0
3-bit errors:	73216 cases	correct data=	512	flagged UE=	0	silent bad=	72704

Real output (Icarus Verilog 12).

- All 3328 single-bit errors are **corrected** and all 19,968 double-bit errors are **flagged uncorrectable** - exactly the SECDDED contract.
- Triple errors are the trap:** 72,704 of 73,216 produce wrong data reported as a corrected single error (the parity looks odd, so the decoder 'fixes' one bit and makes things worse or leaves them wrong). SECDDED says nothing about three errors, which is why memory designs **scrub** (periodically read and correct) so single errors never accumulate into multiple ones, and why bit-interleaving places the bits of one codeword physically apart so that one particle strike upsets only one bit per word.

Where ECC appears	Scheme
On-chip SRAM (caches, buffers)	SECDDED per 32/64-bit word, or parity for data that can be re-fetched (instruction caches)
DDR4/DDR5 DIMMs (side-band ECC)	Extra DRAM devices store check bits: 72-bit channels for DDR4 (SECDDED or stronger), 40-bit subchannels on DDR5 ECC DIMMs; server controllers use symbol-based 'chipkill' codes that survive a whole device failing

Where ECC appears	Scheme
DDR5 on-die ECC	Inside every DDR5 device (single-error correction over 128 data bits); invisible to the controller and not a replacement for system ECC
LPDDR (inline ECC)	No extra pins: the controller stores check bits in a reserved part of the same memory, costing bandwidth and capacity
Link retry buffers, NoC packets	SECEDED or parity on the stored flits
HBM	ECC bits carried alongside data (device-specific options)

26.5 Link-level retry versus end-to-end protection

Detection alone only tells you something is wrong. A protocol must then decide **who retransmits**. There are two schools, and real systems use both:

Approach	How	Examples	Trade-off
Link-level retry	Each hop keeps a replay buffer; the receiver ACKs good packets and NAKs bad ones; the transmitter replays from the buffer	PCIe DLL (12-bit sequence number, LCRC, ACK/NAK DLLPs, replay timer), CXL, UCIe, USB 3.x link layer, CAN automatic retransmission	Fast recovery (sub-microsecond), invisible to software; costs a replay buffer per hop
End-to-end	Only the endpoints check; lost or corrupted data is retransmitted by a higher layer	Ethernet (MAC drops bad frames, TCP retransmits), PCIe ECRC, AUTOSAR E2E	No per-hop state; recovery is slower (milliseconds for TCP) but also catches errors inside switches and bridges
Transaction retry	Host re-issues the transaction a limited number of times	USB 2.0 (a transaction is typically retried up to three times before the host reports an error), I2C NACK handling in software	Simple; relies on the host

THE END-TO-END ARGUMENT

A link CRC protects the wire, not the switch in the middle: a bit flipped inside a switch's buffer after the CRC was checked and before it was regenerated arrives with a perfect new CRC. That is why PCIe offers ECRC in addition to LCRC, why switches protect their internal buffers with ECC/parity, and why safety-critical software adds its own end-to-end checks. Link retry makes the common case fast; end-to-end checks make the rare case safe.

26.6 Timeouts and error reporting

A link that stops responding must not hang the SoC. Every requester needs a timeout, and every error needs somewhere to go.

- **Completion timeouts:** a PCIe requester that never receives a completion reports a completion timeout (the configurable ranges span tens of microseconds to seconds; the default is typically in the 50 us to 50 ms range). On-chip, an AXI transaction to a powered-down or hung slave needs an interconnect timeout unit that returns SLVERR/DECERR instead of stalling the CPU forever.
- **Response codes:** AXI BRESP/RRESP (OKAY, EXOKAY, SLVERR, DECERR), APB PSLVERR, AHB HRESP, PCIe completion status (SC, UR, CA, CRS), USB STALL.
- **PCIe Advanced Error Reporting (AER)** classifies errors as **correctable** (e.g. receiver error, bad TLP/DLLP fixed by replay), **uncorrectable non-fatal** (e.g. completion timeout, unsupported request, poisoned TLP) and **uncorrectable fatal** (e.g. data link protocol error, receiver overflow), with status, mask and severity registers and a header log of the offending TLP.
- **Poisoning:** data known to be bad (uncorrectable ECC error) is forwarded marked as poisoned (PCIe EP bit, CHI data poison, AXI5 poison) so it is not silently consumed, and the error is reported where the data is used.

- **RAS architecture:** on SoCs, errors from memories, links and interconnect are collected in error records (Arm RAS extension or a vendor RAS controller) and raised as interrupts to firmware, which logs, corrects, isolates or resets.

PITFALL: ERRORS THAT DISAPPEAR

Frequent post-silicon complaint: 'the link retrains every few hours and nobody knows why'. Correctable errors were counted in a register that wraps, or masked by default, or cleared by the driver without logging. Make error counters saturating and sticky, expose them to software, and log them - correctable-error trends are the early warning of a marginal link.

26.7 Functional safety: ISO 26262 on links and buses

In automotive (and similarly IEC 61508 industrial) designs, reliability becomes a quantified, audited requirement. ISO 26262 assigns each safety goal an **ASIL** from A (lowest) to D (highest) and, for hardware, sets targets on how well random faults are detected:

Metric	ASIL B	ASIL C	ASIL D
Single-point fault metric (SPFM)	$\geq 90\%$	$\geq 97\%$	$\geq 99\%$
Latent fault metric (LFM)	$\geq 60\%$	$\geq 80\%$	$\geq 90\%$
Probabilistic metric for random hardware failures (PMHF)	< 100 FIT	< 100 FIT	< 10 FIT

ISO 26262-5 hardware architectural metric targets (1 FIT = 1 failure per 10^9 device-hours).

Diagnostic coverage (DC) is the fraction of faults in a block that a safety mechanism detects. Communication paths have their own catalogue of mechanisms:

Safety mechanism	Protects against	Typical DC claim
Parity on buses / registers	Single-bit faults in data/address	Low-medium (misses even-bit faults)
ECC (SECCDED) on memories and buses	Single and double-bit faults	High
CRC on messages	Corruption on links	High (depends on CRC and length)
AMBA 5 interface protection	Faults on AXI/CHI wires and in the interconnect	Parity check signals per signal group (names ending in CHK, e.g. AWADDRCHK , WDATACHK) generated at the source, checked at the destination
Timeouts / watchdogs	Lost or stuck transactions	Medium-high
Lockstep (dual-core or duplicated logic, delayed by a few cycles and compared)	Any fault in the duplicated logic, including the bus master	High
E2E protection (AUTOSAR E2E profiles)	Corruption, loss, repetition, delay, masquerade anywhere between sender and receiver software	High for communication faults
Built-in self-test at start-up (LBIST/MBIST)	Latent permanent faults	Improves LFM

AUTOSAR **E2E protection** is the software counterpart of the end-to-end argument: the sender adds a CRC computed over the data and a **data ID**, plus an **alive/sequence counter**; the receiver checks CRC, counter continuity and timing. This detects not only corruption but lost, repeated, reordered, delayed and misrouted messages - the full list of communication faults the standard asks you to consider - regardless of how many buses and gateways the message crossed.

INTERVIEW ANGLE: WHY IS PARITY NOT ENOUGH FOR ASIL D?

Parity detects only odd numbers of flipped bits, and a single fault in shared logic (a mux select, an address decoder) can corrupt data and its parity consistently. High-ASIL designs therefore combine ECC or CRC for data, protection of control signals (AMBA 5 check signals cover VALID/READY too), timeouts for liveness, and redundancy (lockstep) for logic, then prove the coverage with fault injection campaigns in simulation or emulation.

26.8 Security on links

Security threats to links are different from random errors: an attacker chooses the errors. A CRC is no protection at all - anyone can recompute it. Integrity against an attacker needs a cryptographic MAC (message authentication code), confidentiality needs encryption, and both need keys that were exchanged with an **authenticated** partner.

Mechanism	Link	How it works
PCIe IDE (Integrity and Data Encryption)	PCIe 5.0 (as an ECN) and 6.0	AES-GCM with 256-bit keys protects TLPs per stream (link IDE between neighbours, selective IDE end to end through switches); keys are programmed with the IDE_KM protocol over DMTF SPDM secure sessions after device authentication
CXL IDE	CXL 2.0 and later	AES-GCM-256 on CXL.io, CXL.cache and CXL.mem flits, with a containment or skid mode to trade latency against when data may be consumed
MACsec (IEEE 802.1AE)	Ethernet, hop by hop	GCM-AES-128 (256 optional) per frame; a SecTAG after the MAC addresses (EtherType 0x88E5) and a 16-byte ICV before the FCS; keys agreed by MKA (IEEE 802.1X); line-rate crypto in the MAC/PHY
HDCP 2.x	HDMI, DisplayPort, (wireless) display links	Receiver authentication with certificates, a locality check (round-trip time limit) and AES-128 in counter mode to encrypt content
USB Type-C Authentication	USB	Certificate-based challenge/response so a host can verify a device or charger before trusting it
SPDM (DMTF)	PCIe, CXL, USB, MCTP-based management	Device identity (certificates), measurement attestation and secure sessions: the common authentication layer under several of the above

- **Latency and area:** an AES-GCM engine at hundreds of Gb/s is a wide, pipelined datapath adding tens of nanoseconds; on-chip links are usually trusted and left unencrypted, while off-chip links that cross untrusted hardware (a PCIe bus in a cloud server, a CXL memory expander) get IDE.
- **Replay and ordering:** authenticated encryption uses a counter or packet number as the nonce, which also detects replayed or dropped packets - the security analogue of sequence numbers.
- **Keys** live in hardware key slots loaded by a security controller; the RTL must make sure they are not readable over the normal bus or visible in scan chains.

26.9 On-chip access control: TrustZone, AxPROT and firewalls

Inside the SoC the threat is a less-privileged master (a normal-world OS, a DMA-capable peripheral, a compromised accelerator) reading or writing what it should not. The bus protocols carry the attributes needed to stop it:

Signal	Meaning	Notes
AxPROT[0]	1 = privileged access	From the CPU's exception level / mode
AxPROT[1]	1 = non-secure access (the TrustZone NS bit)	Secure memory and peripherals reject or ignore NS accesses
AxPROT[2]	1 = instruction access	Hint for caches/firewalls

Signal	Meaning	Notes
HPROT , HNONSEC (AHB5), PPROT (APB4+)	Equivalent attributes on AHB and APB	Must be propagated through bridges
AxUSER , AxID , stream IDs	Master identity	Used by SMMUs and firewalls to apply per-master rules

- **TrustZone address-space controllers / memory firewalls** (e.g. Arm TZC-400-style components) sit in front of DRAM and check each access's security attribute and master ID against programmable regions.
- **Peripheral protection controllers** mark each peripheral secure or non-secure; the interconnect or bridge returns an error (or read-as-zero, write-ignored) for violations.
- **MPUs and SMMUs/IOMMUs** restrict what DMA masters can reach - essential because a DMA engine that can write anywhere bypasses all CPU-side protection.
- **Propagation** is where bugs hide: a bridge that drops **PPROT**, an AXI-to-AHB converter that ties **HNONSEC** to 0 (making every access secure!), a DMA that issues all accesses with its own secure attribute on behalf of non-secure software (the 'confused deputy').

PITFALL: TIE-OFFS THAT GRANT PRIVILEGES

When integrating IP with fewer attribute bits than the interconnect, engineers tie the missing inputs to constants. Tying **AxPROT** to 3'b000 declares every access **secure and privileged**. Tie-offs of security attributes must default to the least privilege (non-secure, unprivileged) and be reviewed by the security architect.

26.10 Secure debug and lifecycle

Debug ports (JTAG, SWD, CoreSight DAP, Chapter 23) can read and write everything - that is their job, and it makes them the easiest attack path into a product. Secure SoCs therefore gate debug:

- **Authentication signals:** Arm cores and CoreSight components take **DBGEN** (invasive debug), **NIDEN** (non-invasive: trace, profiling) and secure variants **SPIDEN/SPNIDEN**; a security controller drives them, not a tie-off.
- **Debug authentication:** a challenge-response protocol (signed certificate or token) unlocks debug for an authorized user, possibly only for the non-secure world.
- **Lifecycle states** stored in OTP/fuses: e.g. test/manufacturing, development, production (debug locked), RMA (return: re-opened after secrets are erased), end of life. Transitions are one-way or authenticated; production parts must not expose scan chains or memory BIST that can dump secrets.
- **Scan and DFT:** scan chains can shift out key registers; secure designs reset or exclude secret-holding flops when entering test mode.

26.11 Fault-injection and physical attacks

An attacker with physical access can cause errors deliberately to skip a security check - for example glitching the clock or supply exactly when the boot ROM compares a signature. Links and their error handling are part of this attack surface.

Attack	Method	Countermeasures
Clock glitching	Insert a short clock pulse so a flop captures before logic settles	Clock monitors, glitch filters, internal oscillators for security logic
Voltage glitching	Brief supply droop or spike	Voltage/brown-out detectors, sensors that reset the chip
Electromagnetic / laser fault injection	Localized upsets of individual gates or flops	Redundant computation and comparison, duplicated checks, light sensors, shields
Side-channel analysis	Power or EM traces leak key bits	Masked/constant-time crypto implementations

Attack	Method	Countermeasures
Bus probing / interposers	Read or modify off-chip traffic	Link encryption (IDE, MACsec), memory encryption, tamper detection

RELIABILITY AND SECURITY MEET

The same hardware serves both goals: parity and ECC also detect many injected faults, duplicated FSMs with comparison (lockstep for security) catch glitches, and error counters reveal an attack in progress. Design security-critical FSMs with sparse, Hamming-distant state encodings and a safe default for illegal states.

26.12 Summary

- Links fail randomly (BER), in bursts, through soft and hard faults, through design bugs and through attacks; each needs a different defence.
- An r -bit CRC detects all bursts up to r bits, lets about 2^{-r} of random garbage through, and its Hamming distance depends on the frame length - the experiment showed CRC-8 losing HD = 4 beyond 119 data bits while CRC-32 caught every corrupted frame.
- FEC (RS(528,514), RS(544,514), PCIe 6.0 flit FEC, LDPC) corrects errors at the cost of latency and area; PAM4 links need it.
- SECDED corrects one and detects two errors per word but miscorrects most triple errors; scrubbing and bit-interleaving keep errors single.
- Link-level retry (PCIe, CXL, UCIe) gives fast recovery; end-to-end checks (ECRC, TCP, AUTOSAR E2E) catch errors inside the path.
- Timeouts, response codes, AER-style classification, poisoning and a RAS architecture make errors visible and recoverable.
- ISO 26262 quantifies safety (SPFM/LFM/PMHF per ASIL) and uses parity, ECC, CRC, AMBA 5 check signals, timeouts, lockstep and E2E protection as safety mechanisms.
- Security needs cryptographic integrity (PCIe/CXL IDE, MACsec, HDCP) with authenticated key exchange (SPDM), correct propagation of AXPOT/NS attributes, firewalls and SMMUs, locked-down debug and fault-injection countermeasures.

26.13 Exercises

- 1 A 25 Gb/s link runs at BER $1e-12$ and carries 1500-byte frames protected by CRC-32. Estimate the frame error rate and the rate of undetected frames assuming random garbage escapes with probability 2^{-32} . How would a $1e-6$ raw BER change the design?
- 2 Modify `crc_exp.py` to measure CRC-16/CCITT (polynomial 0x1021) on the same frames. Which rows change, and why?
- 3 Extend the SECDED(13,8) decoder so that it also flags syndromes 13-15 (which cannot come from a single error) as uncorrectable. How do the triple-error statistics change?
- 4 Explain step by step how PCIe's data link layer recovers from a corrupted TLP and from a lost ACK DLLP. Which timers and counters are involved?
- 5 For an ASIL D brake controller receiving wheel-speed messages over CAN FD through a gateway, list the communication faults ISO 26262 asks you to consider and the mechanism that detects each.
- 6 An AXI DMA engine executes descriptors written by non-secure software. What attributes should its memory accesses carry, and what goes wrong if it always issues secure accesses?

27. Choosing and Budgeting Protocols for an SoC, and the Learning Roadmap

The previous 26 chapters described protocols one by one. An SoC architect faces the inverse problem: given a product - a camera, a phone, an automotive controller, an AI accelerator card - which protocols should the chip have, how many lanes and pins, how much memory bandwidth, and what will they cost in area, power and licences? This chapter gives a method, a comparison table to choose from, worked bandwidth and latency budgets, a reference edge-AI SoC and, to close the book, a roadmap for learning all of this.

27.1 From requirements to a protocol

```

product requirements (use cases, data rates, partners, cost, safety)
  |
  v
1. Who is on the other end? (standard device -> the standard it speaks:
  |                               DDR/LPDDR, camera -> CSI-2, SSD -> PCIe/NVMe)
  v
2. Bandwidth (sustained + peak) and latency needed? -> class of link
  |
  v
3. Distance / medium: on-die, die-to-die, board, cable, vehicle harness
  |
  v
4. Pins, power, area, licence cost, ecosystem, certification
  |
  v
5. Reliability / safety / security requirements (CRC? retry? ASIL? IDE?)
  |
  v
choice + lane/pin count + controller/PHY IP + budget entries

```

A protocol selection flow. The first question usually decides: if the partner is a standard part, you speak its protocol.

- **Partner first.** A DRAM speaks DDR/LPDDR/HBM, a camera sensor speaks MIPI CSI-2 (sometimes SLVS-EC or LVDS), an SSD speaks PCIe (NVMe), a PMIC speaks I2C/SPMI/I3C, a sensor hub speaks I2C, SPI or I3C, a car network speaks CAN FD, LIN or automotive Ethernet. The choice is often made for you; the architecture work is in the number of instances, lanes and speeds.
- **On-chip** choices are yours: APB for registers, AXI for high-bandwidth masters and memory, ACE-Lite/CHI where coherency is needed, AXI-Stream for pipelines, a NoC when the number of initiators and targets grows (Chapters 5-10).
- **Chip-to-chip between your own devices:** PCIe/CXL if you want software and ecosystem support, a lightweight SerDes protocol (Aurora, Interlaken, JESD204 for converters) if you want low overhead, UCle/BoW for dies in one package (Chapter 22).
- **Prefer the simplest protocol that meets the need:** a sensor that sends 100 bytes a second does not need I3C HDR; a status LED controller does not need SPI when I2C is already routed.

27.2 The big comparison table

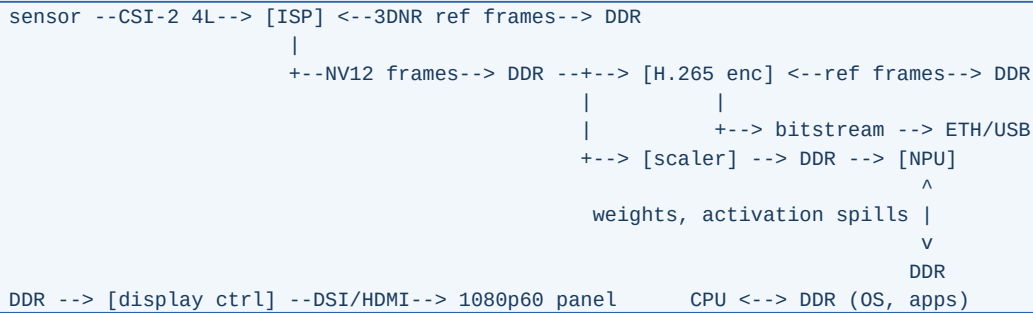
Protocol	Rate (typ./max)	Signals / pins	Reach	Topology	Cplx	Typical SoC use
UART	115.2 kb/s - few Mb/s	2 (TX, RX) + opt. RTS/CTS	Board; km with RS-485	Point-to-point	Low	Console, debug, BT/GNSS modules
SPI	10-100+ MHz	4 + 1 CS per target	Board	1 controller, N targets	Low	Sensors, displays, NOR flash, ADCs
Quad/Octal SPI, xSPI	Up to ~200 MHz DDR x8 (~400 MB/s)	6-12	Board	Point-to-point	Med	Boot flash, XIP, PSRAM

Protocol	Rate (typ./max)	Signals / pins	Reach	Topology	Cplx	Typical SoC use
I2C	100 k / 400 k / 1 M / 3.4 Mb/s	2 open-drain	Board	Multi-drop, multi-controller	Low/Med	PMIC, EEPROM, sensors, config
I3C	12.5 Mb/s SDR, more in HDR modes	2	Board	Multi-drop, dynamic addressing, in-band interrupts	Med	Sensor hubs, DDR5 SPD/sideband
CAN FD	Arbitration ≤ 1 Mb/s, data phase typically 2-8 Mb/s	2 (via transceiver)	~40 m (vehicle)	Multi-drop bus	Med	Automotive ECUs
I2S / TDM	Few Mb/s	3-4	Board	Point-to-point	Low	Audio codecs, microphones
APB / AHB / AXI	Bus clock x width (AXI: tens of GB/s)	On-chip wires	On-die	Bus / crossbar / NoC	Low to High	Registers, memory, DMA
USB 2.0 / 3.2 / USB4	480 Mb/s / 5-20 Gb/s / 40-80 Gb/s	2 / +4 / Type-C	Cable, few m	Tiered star (host-centric)	High	Peripherals, charging, docking
PCIe Gen4 / 5 / 6	16 / 32 / 64 GT/s per lane	4 per lane + refclk	Board, cable with retimers	Point-to-point, switches	Very high	SSD, NIC, accelerators, CXL
Ethernet (1G-400G)	1-400 Gb/s	RGMI 12, SGMII/SerDes 4	100 m copper, km fibre	Switched	High	Networking, automotive backbone
LPDDR5/5X	6400-8533+ MT/s x16 channel	~35-45 per x16 channel	Package/board, cm	Point-to-point	Very high	Main memory of mobile/edge SoCs
HBM3/3E	6.4-9.6 Gb/s per pin x 1024	1024 data + CA; thousands of micro-bumps	Interposer, mm	Point-to-point	Very high	AI/HPC memory
MIPI CSI-2 / DSI (D-PHY)	Up to 2.5 (v1.2) / 4.5 (v2.5) Gb/s per lane, 1-4 lanes	2 per lane + clock pair	Board / flex, cm	Point-to-point	Med/High	Cameras, displays
UCIe	4-32 GT/s per lane	Many micro-bumps per module	Package, mm	Die-to-die	High	Chiplets
JTAG / SWD	~10-50 MHz	4-5 / 2	Board	Daisy chain / point-to-point	Low/Med	Debug, boundary scan, test

Protocol comparison (typical values; exact maxima depend on the spec version and implementation). Appendix A has a longer quick-reference.

27.3 Bandwidth budgeting: a worked example

The most consequential budget in most SoCs is **DRAM bandwidth**, because almost every high-rate client passes through it. Consider an edge-AI camera: a 4K30 sensor, an ISP, a neural-network accelerator (NPU) running object detection, an H.265 encoder for recording and streaming, and a 1080p60 local display.



Data flow of the edge-AI camera. Every arrow that touches DDR costs bandwidth - often more than once per frame.

The script below turns the data flow into numbers. Each client's traffic is derived from frame sizes and rates; the less predictable clients (NPU weights and activation spills, CPU) are explicit assumptions that the architect would refine with the NPU compiler and CPU benchmarks.

```

# DDR bandwidth budget for an edge-AI camera SoC (all rates in MB/s, 1 MB = 1e6 bytes)
W, H, FPS = 3840, 2160, 30
raw_bits = 12
nv12 = W * H * 1.5                                # bytes per NV12 (YUV 4:2:0) frame

# 1) MIPI CSI-2 link from the sensor (RAW12, ~20% blanking + packet overhead assumed)
payload_gbps = W * H * FPS * raw_bits / 1e9
line_gbps = payload_gbps * 1.20
for lanes in (2, 4):
    print("CSI-2 RAW12 4K30: %.2f Gb/s payload, %.2f Gb/s on wire -> %d lanes x %.2f Gb/s"
          % (payload_gbps, line_gbps, lanes, line_gbps / lanes))

# 2) DDR clients: (name, read MB/s, write MB/s)
clients = [
    ("ISP out (NV12 4K30)",          0,          nv12 * FPS),
    ("ISP 3DNR ref (rd+wr)",         nv12 * FPS,   nv12 * FPS),
    ("Scaler -> NPU input 640x640x3", W * H * 1.5 * FPS, 640 * 640 * 3 * FPS),
    ("NPU weights 12 MB/frame",      12e6 * FPS,   0),
    ("NPU activation spill 20 MB/fr", 20e6 * FPS,   20e6 * FPS),
    ("H.265 enc (cur + 2 refs, recon)", 3 * nv12 * FPS, nv12 * FPS),
    ("Display 1080p60 ARGB8888",     1920 * 1080 * 4 * 60, 0),
    ("CPU cluster + OS (allowance)",  800e6,        400e6),
    ("Ethernet/USB/PCIe DMA",         150e6,        150e6),
]
tot_r = sum(c[1] for c in clients); tot_w = sum(c[2] for c in clients)
print("%-34s %9s %9s" % ("client", "read", "write"))
for n, r, w in clients:
    print("%-34s %9.0f %9.0f" % (n, r / 1e6, w / 1e6))
print("%-34s %9.0f %9.0f  total %.0f MB/s" % ("SUM", tot_r / 1e6, tot_w / 1e6,
                                              (tot_r + tot_w) / 1e6))

# 3) Compare with DRAM options at a realistic efficiency
for name, mts, bits in (("LPDDR4X-4266 x32", 4266, 32), ("LPDDR5-6400 x32", 6400, 32),
                        ("LPDDR5X-8533 x32", 8533, 32)):
    peak = mts * 1e6 * bits / 8
    for eff in (0.65, 0.75):
        use = (tot_r + tot_w) / (peak * eff)
        print("%-17s peak %.0f MB/s  eff %.0f%% -> usable %.0f MB/s  load %.0f%%"
              % (name, peak / 1e6, eff * 100, peak * eff / 1e6, use * 100))

```

bw.py - a first-order DRAM bandwidth budget and a CSI-2 lane check.

```

CSI-2 RAW12 4K30: 2.99 Gb/s payload, 3.58 Gb/s on wire -> 2 lanes x 1.79 Gb/s
CSI-2 RAW12 4K30: 2.99 Gb/s payload, 3.58 Gb/s on wire -> 4 lanes x 0.90 Gb/s
client          read      write
ISP out (NV12 4K30)      0      373

```


ISP 3DNR ref (rd+wr)	373	373							
Scaler -> NPU input 640x640x3	373	37							
NPU weights 12 MB/frame	360	0							
NPU activation spill 20 MB/fr	600	600							
H.265 enc (cur + 2 refs, recon)	1120	373							
Display 1080p60 ARGB8888	498	0							
CPU cluster + OS (allowance)	800	400							
Ethernet/USB/PCIe DMA	150	150							
SUM	4274	2307	total	6581 MB/s					
LPDDR4X-4266 x32 peak	17064 MB/s	eff 65% -> usable	11092 MB/s	load	59%				
LPDDR4X-4266 x32 peak	17064 MB/s	eff 75% -> usable	12798 MB/s	load	51%				
LPDDR5-6400 x32 peak	25600 MB/s	eff 65% -> usable	16640 MB/s	load	40%				
LPDDR5-6400 x32 peak	25600 MB/s	eff 75% -> usable	19200 MB/s	load	34%				
LPDDR5X-8533 x32 peak	34132 MB/s	eff 65% -> usable	22186 MB/s	load	30%				
LPDDR5X-8533 x32 peak	34132 MB/s	eff 75% -> usable	25599 MB/s	load	26%				

Real output (Python 3).

How to read the budget:

- **The camera link:** 4K30 RAW12 is 2.99 Gb/s of pixels; with an assumed 20 % for blanking and packet overhead it needs about 3.6 Gb/s. Four D-PHY lanes at about 0.9 Gb/s each leave plenty of margin (D-PHY v1.2 allows up to 2.5 Gb/s per lane); two lanes at 1.8 Gb/s also fit, saving four pins at a smaller margin.
- **Total DRAM traffic** is about 6.6 GB/s - though the sensor delivers only 0.37 GB/s: every stage that stores and re-reads a frame multiplies traffic. The encoder alone (current frame plus reference reads and reconstructed-frame writes) is almost 1.5 GB/s.
- **Efficiency** is the key assumption: real DRAM delivers perhaps 60-80 % of its peak because of refresh, bank conflicts, read/write turnarounds and page misses (Chapter 20). At 65 % efficiency an LPDDR4X-4266 x32 interface would be 59 % loaded - workable but with little headroom for peaks; LPDDR5-6400 x32 brings it to about 40 %.
- **Headroom:** architects typically keep the average load well below the usable bandwidth (often targeting 60-70 % or less) because clients are bursty and real-time clients (display, ISP) must never starve. QoS in the interconnect and memory controller gives the display and ISP priority over the CPU and NPU.

CUT TRAFFIC BEFORE BUYING BANDWIDTH

The cheapest bandwidth is the traffic you never generate: stream ISP output directly to the encoder and scaler through on-chip buffers instead of DDR, keep NPU weights in on-chip SRAM when the model fits, use frame-buffer compression (lossless compression schemes for display and video buffers typically save 30-50 %), and pick 8-bit instead of 10-bit formats where quality allows.

PITFALL: BUDGETING AVERAGES ONLY

A budget computed from averages hides peaks: a 20 ms NPU burst overlapping an encoder reference fetch and a display refresh can exceed peak bandwidth even when the average load is 50 %. Model the worst concurrent phase, check the FIFO depth of every real-time client against the worst stall (Chapter 24), and validate with a cycle-accurate performance model or emulation running real traffic.

27.4 Latency budgets

Some products care more about latency than bandwidth: a driver-assistance camera must go from photons to braking decision in tens of milliseconds; a control loop over an industrial bus must close in microseconds; a CPU load that misses the cache waits for DRAM. Latency budgets are built the same way - sum every stage and add margin:

Stage (camera-to-decision example)	Typical contribution	Levers
Sensor exposure + readout	Several ms to one frame time (33 ms at 30 fps)	Higher frame rate, rolling vs global shutter

Stage (camera-to-decision example)	Typical contribution	Levers
CSI-2 transfer	Same as readout (streaming, line by line)	More lanes do not reduce it if the sensor reads out slower
ISP	A few lines to a frame, depending on whether it stores full frames (3DNR)	Line-based processing, bypass stages
DDR write + NPU read	Up to one frame if the NPU waits for a complete frame	Tile/slice-based hand-off
NPU inference	Milliseconds to tens of ms	Model size, INT8, SRAM tiling
Decision output over CAN FD / Ethernet	Hundreds of us to ms	Priority/TSN scheduling

Transaction	Order of magnitude
AXI access to a local SRAM	A few cycles
AXI read through an interconnect to DRAM (loaded)	About 80-150 ns
PCIe read round trip (host memory from a device)	About 0.5-1 us
UCle / die-to-die hop	A few ns (adapter + PHY)
I2C register read at 400 kHz (address + reg + data, ~40 bits)	About 100 us
CAN FD frame, 64 bytes, data phase 2-5 Mb/s	Around 150-400 us

Latency reference points (orders of magnitude, not guarantees).

27.5 Pin and IO budgets

Pins are expensive: they set the package size and ball count, board layers and cost. An IO budget lists every interface, its signal count, IO type and voltage, and the pads it needs:

Interface	Signals	IO type	Notes
LPDDR5 x32 (2 x16 channels)	~80-90	DDR PHY bumps	Plus supplies; placed on the die edge nearest the DRAM
MIPI CSI-2 4-lane D-PHY	10 (4 data pairs + clock pair)	D-PHY macro	x2 for two cameras
MIPI DSI 4-lane or HDMI 2.0	10 / 19-pin connector	PHY macro	Plus HPD, DDC (I2C)
PCIe Gen4 x1 or x2	4 per lane + 2 refclk	SerDes macro	For Wi-Fi module or NVMe
USB 3.2 Gen1 + USB 2.0	4 SS + 2 HS (+ Type-C CC)	SerDes + USB 2.0 PHY	Type-C needs a PD controller or on-chip logic
Gigabit Ethernet RGMII	12 + MDIO/MDC 2	GPIO (1.8/2.5/3.3 V)	Delay (2 ns) on RGMII clocks: PHY, MAC or PCB
eMMC/SD or UFS	11 (eMMC 8-bit) / 4+refclk (UFS)	GPIO / M-PHY	Boot and storage
Octal SPI flash	11-12	GPIO	Boot ROM path
I2C x4, SPI x2, UART x3, I2S x2, CAN FD x2	~40	GPIO (muxed)	Pin-mux makes these flexible
JTAG/SWD, boot mode, reset, clocks, PMIC	~15	GPIO / analog	Fixed function, not muxed

Pin-muxing lets many low-speed peripherals share pads - the product chooses which functions appear on which balls. High-speed interfaces cannot be muxed freely: their pads are part of PHY macros with fixed placement, and their package routing is part of the signal-integrity budget.

27.6 Power, area and cost

Link class	Energy per bit (order of magnitude)	Comment
On-chip wire / NoC	~0.1 pJ/bit per mm plus switching	Cheapest; grows with distance
Die-to-die (UCIe, BoW) in a package	Roughly 0.3-1 pJ/bit	Short, unterminated or lightly terminated, dense bumps
HBM	A few pJ/bit (interface)	Wide and slow per pin; interposer
LPDDR5/5X	A few pJ/bit (interface), more with DRAM core	Low-swing, unterminated or lightly terminated
Board-level SerDes (PCIe, USB, Ethernet)	Roughly 5-15 pJ/bit	Equalization, CDR, termination; long-reach costs most
Low-speed GPIO protocols	High per bit, tiny in total	Irrelevant to power at kb/s-Mb/s rates

Indicative energy per bit. Use vendor data for real budgets; these ratios are what matters for architecture.

- **Idle power** often dominates average power: link power states (USB U1-U3, PCIe L0s/L1/L1 substates, Ethernet EEE, D-PHY LP mode, LPDDR self-refresh) and clock gating matter as much as active pJ/bit.
- **Area:** a SerDes lane is a large mixed-signal macro; a DDR PHY occupies a long stretch of die edge; controllers with deep buffers are SRAM-dominated. Low-speed peripherals are nearly free by comparison.
- **Cost** includes licence fees and royalties for controllers and PHYs, VIP licences, compliance testing and logo programmes, standards-body membership, package and board cost of pins, and the schedule risk of a complex interface.

27.7 A reference edge-AI SoC and its protocol map

LPDDR5X x32 <===== >	DDR PHY LPDDR ctrl <----- >	coherent NoC
2x cameras -CSI-2 4L-->	D-PHY RX CSI-2 RX ISP -AXI->	(CHI + AXI)
panel <----DSI 4L----->	D-PHY TX display <-AXI----->	with QoS
	CPU cluster (4x) -CHI----->	
	NPU (SRAM + DMA) -AXI----->	
	H.265 codec -AXI----->	
NVMe/Wi-Fi <-PCIe G4-->	SerDes PCIe RC -ACE-Lite----->	
USB-C <-USB 3.2 / 2.0->	SerDes+UTMI xHCI/DRD -AXI---->	
Ethernet PHY <-RGMII-->	GMAC (TSN, MACsec) -AXI----->	
boot flash <--xSPI---->	xSPI controller -AXI----->	
eMMC / SD <----->	SDHC -AXI----->	
	APB bridge <----- >	
	-> I2C x4, I3C, SPI x2, UART x3,	
	I2S x2, CAN FD x2, PWM, GPIO	
PMIC <--I2C/SPMI----->	security subsystem (keys, OTP, lifecycle)	
debugger <-JTAG/SWD-->	CoreSight DAP + trace; 1500/IJTAG to PHYS	
	+----- >	

Protocol map of a representative edge-AI vision SoC. Every protocol of this book appears somewhere on it.

Subsystem	Protocols	Chapter
Memory	LPDDR5X, DFI, AXI/CHI to the memory controller, QoS	7, 9, 20
Vision in / out	MIPI CSI-2 and DSI over D-PHY; HDMI/DP on some variants	21
Compute	CHI (coherent CPU cluster), AXI (NPU, codec), AXI-Stream inside pipelines	7-9
Expansion and storage	PCIe Gen4 (NVMe/Wi-Fi), USB 3.2 + 2.0 with Type-C, xSPI, eMMC/SD	12, 17, 18, 20
Networking	Gigabit Ethernet RGMII with TSN; optional MACsec	19, 26
Control and peripherals	APB, I2C, I3C, SPI, UART, I2S, CAN FD	5, 11-16
Debug, test, security	JTAG/SWD, CoreSight, IEEE 1500/1687, TrustZone attributes, secure debug	23, 26

27.8 The learning roadmap

Protocols are learned best by building them. The roadmap below orders them so that each step reuses what the previous one taught; every step ends with RTL, a self-checking testbench and - from step 2 on - a demonstration on an inexpensive FPGA board talking to real devices.

Step	Learn	Build (and put on an FPGA)	Skills gained
1	Digital basics, FSMs, CDC (Ch 2-4)	UART TX/RX with oversampling; talk to a PC terminal	Serial framing, oversampling, synchronizers, testbenches
2	SPI (Ch 12)	SPI controller reading a sensor or an ADC; SPI flash read	Modes (CPOL/CPHA), shift registers, chip select timing
3	I2C (Ch 13)	I2C controller reading an EEPROM or temperature sensor	Open-drain, arbitration, clock stretching, ACK/NACK
4	APB (Ch 5)	APB register block + APB wrappers for your UART/SPI/I2C	CSRs, bus handshakes, register maps
5	AXI4-Lite / AXI4 / AXI-Stream (Ch 7-8)	AXI-Lite slave, AXI DMA to stream (Ch 24), connect to a soft or hard CPU	Channels, bursts, IDs, back-pressure, performance
6	Ethernet MAC (Ch 19)	10/100/1G MAC with RGMII, CRC-32, send UDP packets to a PC	Framing, CRC, xMII, MDIO, PHY bring-up, Wireshark debugging
7	PCIe or USB (Ch 17-18)	Use the FPGA's hard PCIe/USB block; write the DMA engine and a Linux driver	Layered protocols, enumeration, config space, drivers
8	Memory, video, D2D, safety/security (Ch 20-26)	DDR controller via vendor PHY; CSI-2 receiver; verification with SVA/UVM and error injection	System-level integration and verification

- **Read the specs:** start with the short, public ones (NXP UM10204 for I2C, Arm AMBA APB/AXI specifications, Motorola/NXP SPI application notes), then move to summaries of the large ones (PCIe, USB) before tackling their full text.
- **Use real tools:** a logic analyzer with protocol decoders for UART/SPI/I2C/CAN, Wireshark for Ethernet, lspci/lssusb and kernel logs for PCIe/USB. Seeing real traffic teaches what the timing diagrams mean.
- **Verify like a professional:** every project gets a self-checking testbench, assertions on its interfaces, error injection and coverage - the habits of Chapter 25.
- **Publish:** a clean repository of these projects with testbenches and waveforms is the best evidence of skill in an interview.

27.9 Roles: who does what with protocols

Role	Protocol work
SoC architect	Selects protocols, sizes lanes, bandwidth, latency and buffers; defines the interconnect and QoS
RTL / IP design engineer	Designs controllers, bridges, DMA engines, PCS logic; integrates vendor IP
Design verification (DV)	Builds testbenches with VIP, assertions, scoreboards, coverage; runs error injection and compliance-style suites
PHY / analog / mixed-signal	Designs SerDes, DDR and MIPI PHYs, PLLs, equalizers; characterizes jitter and eyes
Physical design / package / SI-PI	Places PHY macros and pad rings, routes high-speed signals, does signal and power integrity for package and board
DFT	Test access to PHYs (IEEE 1500/1687), loopback and PRBS for production test, boundary scan
Firmware / driver	Bring-up sequences, drivers, link training firmware, error handling and power management

Role	Protocol work
Post-silicon validation	Protocol analyzers, BERT, compliance testing, interoperability, margining
Safety / security architect	Safety mechanisms and metrics, link encryption, access control, secure debug

27.10 Interview preparation

Protocol questions are staples of RTL, DV and architecture interviews. They test whether you understand **why** a protocol works the way it does, not whether you memorized a table. Prepare to:

- **Draw timing diagrams** from memory: APB write with wait states, AXI write with WVALID before AWVALID, SPI mode 0 and 3, an I2C read with repeated START, a UART frame with parity.
- **Explain design choices:** why AXI has separate address and data channels, why I2C is open-drain, why PCIe uses credits and ACK/NAK, why USB uses NRZI with bit stuffing, why CAN arbitration is non-destructive, why HBM is wide and slow.
- **Compute:** link bandwidth after encoding overhead (8b/10b, 128b/130b), DRAM bandwidth with efficiency, FIFO depth for a burst or a round trip, number of outstanding transactions for a target throughput, CRC properties.
- **Write RTL on a whiteboard:** a UART transmitter, an SPI controller, an APB slave, a valid/ready skid buffer, a CRC generator, a round-robin arbiter.
- **Debug scenarios:** 'the AXI master hangs', 'the I2C bus is stuck low', 'PCIe link trains to Gen1 only', 'Ethernet drops 1 packet in 10⁵', 'USB enumerates on one host but not another'. Walk through hypotheses and the measurements that distinguish them.
- Appendix C has 100 protocol interview questions with answers.

THREE QUESTIONS THAT SEPARATE LEVELS

(1) 'Can an AXI slave wait for WVALID before asserting AWREADY?' - yes; it is the master that must not make VALID depend on READY. (2) 'Why does PCIe need both flow-control credits and ACK/NAK?' - credits prevent receiver buffer overflow, ACK/NAK recovers from transmission errors; they solve different problems at different layers. (3) 'How many outstanding reads does a DMA need to reach 10 GB/s with 150 ns memory latency and 64-byte bursts?' - bandwidth x latency = 1500 bytes, so at least 24 outstanding 64-byte reads.

27.11 Final checklist

Protocols on an SoC: the end-to-end checklist

- ☐ Each external interface is chosen from the partner's standard, with version, speed, lane count and options documented.
- ☐ On-chip protocols and interconnect topology chosen per traffic class (registers, bulk data, coherent, streaming), with QoS.
- ☐ Bandwidth budget per client and for DRAM, with efficiency and peak phases; latency budget for the critical paths.
- ☐ Buffers and outstanding transactions sized from bandwidth x latency; FIFOs of real-time clients cover worst-case stalls.
- ☐ Pin/IO budget agreed with package and board; pin-mux, IO voltages and open-drain needs verified.
- ☐ Clock, reset, power-domain and low-power-state plans per interface; PHY bring-up sequence captured in the programming guide.
- ☐ Controller/PHY/VIP make-or-buy decided; silicon proof and compliance status of licensed IP checked.
- ☐ Verification plan with protocol checkers, scoreboards, error injection, coverage, formal where it fits; compliance and interoperability testing scheduled.
- ☐ Reliability: CRC/FEC/ECC/retry/timeouts/error reporting specified; error counters visible to software.

- ☐ Safety (ASIL targets, safety mechanisms, metrics) and security (IDE/MACsec, access control attributes, secure debug) reviewed.
- ☐ Test modes for bring-up and production (loopback, PRBS, BIST, IEEE 1500 access) designed in.
- ☐ Firmware and drivers developed on FPGA/emulation before silicon.

27.12 Summary

- Choose protocols from the partner outward: standard devices dictate their interfaces; on-chip and chip-to-chip choices trade bandwidth, latency, pins, power, cost and ecosystem.
- Budget DRAM bandwidth client by client with realistic efficiency (roughly 60-80 %) and headroom; reduce traffic before adding bandwidth.
- Budget latency stage by stage, and pins, power, area and licences per interface.
- A reference edge-AI SoC uses nearly every protocol in this book: LPDDR, CSI-2/DSI, CHI/AXI/APB, PCIe, USB, Ethernet, xSPI, eMMC, I2C/I3C/SPI/UART/I2S/CAN FD and JTAG/SWD.
- Learn by building, in order: UART, SPI, I2C, APB, AXI, Ethernet MAC, then PCIe/USB with FPGA hard blocks - each with a self-checking testbench.
- Protocol work spans architecture, RTL, DV, PHY, PD, DFT, firmware, validation, safety and security - the best engineers understand enough of each to work across the seams.

27.13 Exercises

- 1 Redo the bandwidth budget for an 8-megapixel 60 fps camera and a 4K60 display. Which LPDDR configuration do you need at 70 % efficiency and a 65 % maximum load?
- 2 The NPU's weights (12 MB per frame) could be kept in on-chip SRAM if you add 12 MB of SRAM. Estimate the bandwidth saved and discuss whether the area is worth it.
- 3 Choose protocols for a smart industrial sensor node: a 16-bit ADC at 1 MS/s, a temperature sensor, a 10 Mb/s-class network link to a PLC over 50 m, and a firmware update path. Justify each choice and count the pins.
- 4 Build the latency budget for a lane-keeping camera that must react within 50 ms from photon to CAN FD message. Which stages would you optimize first?
- 5 Plan your own learning roadmap for the next six months using the table in this chapter: pick a board, list the peripherals you will connect and the testbench you will write for each step.
- 6 Answer the three 'questions that separate levels' in your own words, then write one more question of that kind for USB and one for DDR.

APPENDICES

Appendices

Protocol comparison tables with a verified CRC catalogue, a glossary of about 180 terms, 100 interview questions with worked answers, and the specifications, open-source IP, tools and books that make up the protocol engineer's library.

Appendix A. Protocol Comparison Quick Reference

Dense look-up tables for the protocols of this book. Numbers are the headline figures of the public specifications or widely published summaries of them; where a spec leaves a value to the implementation (or where vendors routinely exceed it) the table says *typically*. Chapter numbers in parentheses point to the in-depth treatment. Speeds of the newest generations are marked (*) - they move fast, so check the current spec revision before quoting them in a design review.

On-chip buses (Chapters 5-10)

Bus	Channels / phases	Pipelining	Bursts	Outstanding / ordering	Typical use
APB (APB3/4/5)	One shared address+data transfer: SETUP then ACCESS phase; PSEL PENABLE PWRITE PADDR PWDATA PRDATA PREADY PSLVERR	None: ≥ 2 cycles per transfer	None	1; strictly in order	Peripheral control/status registers (5)
AHB / AHB-Lite (AHB5)	Address phase and data phase; HTRANS HADDR HBURST HSIZE HWRITE HREADY HRESP	Address of N+1 overlaps data of N	SINGLE, INCR, INCR4/8/16, WRAP4/8/16; 1 KB boundary	1 in data phase; in order; HREADY stalls	MCU system bus, legacy memory buses (6)
AXI3 / AXI4 (AXI5)	5 independent channels AW, W, B, AR, R; VALID/READY on each	Fully decoupled; address ahead of data	INCR 1-256 beats (AXI3: 16), WRAP 2/4/8/16, FIXED 1-16; no 4 KB crossing	Many; per-ID ordering, different IDs may complete out of order	High-performance interconnect, DMA, DDR ports (7)
AXI4-Lite	Same 5 channels, no ID/LEN/SIZE fields	Channel-decoupled	Single beat only	Usually 1 (protocol allows more, in order)	Register slaves, CSR blocks (8)
AXI4-Stream	One channel: TVALID TREADY TDATA TLAST TKEEP TSTRB TID TDEST TUSER	Every beat	Unbounded packets marked by TLAST	N/A (no addresses)	DSP, video, network datapaths (8)
ACE / ACE-Lite	AXI + snoop channels AC (address), CR (response), CD (data)	As AXI	Cache-line bursts	Many; barriers, 5-state lines UC/UD/SC/SD/I	Cluster-level coherency; ACE-Lite for IO-coherent masters (9)
CHI (B-E)	Packetised layers (protocol, network, link); channels REQ, RSP, SNP, DAT per direction	Flit pipelined, credit-based link	Cache-line transactions	Many; TxnID based, home-node ordering	Mesh coherent interconnect, many-core servers (9)
Wishbone B4	Classic CYC STB WE ACK (+STALL in pipelined mode)	Pipelined mode only	Registered-feedback bursts (CTI/BTE)	Pipelined: several; in order	Open-source SoCs, FPGA (10)
Avalon-MM / -ST	address read write waitrequest readdatavalid burstcount	Pipelined reads (readdatavalid)	burstcount	Several reads; in order	Intel FPGA Platform Designer systems (10)

Bus	Channels / phases	Pipelining	Bursts	Outstanding / ordering	Typical use
TileLink (UL/UH/C)	Channels A, B, C, D, E; TL-UL uses A and D only	Channel-decoupled, ready/valid	TL-UH/TL-C: multi-beat (power-of-2 size)	Many; per-source ID	RISC-V SoCs (Rocket/Chipyard); TL-C is coherent (10)

HOW TO READ THE TABLE IN AN INTERVIEW

The two questions that separate the buses are *how many transactions can be in flight* (1 for APB/AHB, many for AXI/CHI/TileLink) and *does the protocol carry coherency messages* (ACE, CHI, TL-C). Everything else - widths, burst types - follows from those two design choices.

Low-speed peripheral and field buses (Chapters 11-16)

Protocol	Wires	Max speed (typ.)	Duplex	Topology	Addressing	Error check
UART (RS-232/485 PHY)	TX, RX (+RTS/CTS); RS-485: 1 diff pair	115.2 kbaud common; a few Mbaud typical max	Full (RS-485 2-wire: half)	Point-to-point (RS-485: multi-drop)	None (9-bit mode in multi-drop)	Optional parity, framing error
SPI	SCLK, COPI (MOSI), CIPO (MISO), CS_n per target	Tens of MHz; ~50-100 MHz typical	Full	1 controller, star of CS lines or daisy chain	Chip select	None in protocol
Quad/Octal SPI, xSPI	CLK, CS_n, IO0-3 (or IO0-7) + DQS	~100-200 MHz, SDR or DDR (xSPI: DDR up to ~400 MB/s)	Half	Controller + 1 flash per CS	Command + address in frame	Optional (device-specific ECC/CRC)
I2C	SCL, SDA (open drain, pull-ups)	100 k / 400 k / 1 M (Fm+) / 3.4 M (Hs); 5 M UFM (one-way)	Half	Multi-controller, multi-drop	7- or 10-bit	ACK/NACK only (SMBus adds CRC-8 PEC)
I3C	SCL (push-pull), SDA	12.5 MHz SDR; HDR modes higher (HDR-DDR ~2x)	Half	Multi-drop, 1 active controller	Dynamic 7-bit (ENTDAA)	T-bit parity (SDR); CRC-5 (HDR-DDR)
CAN (Classical)	CAN_H, CAN_L differential	1 Mb/s (at ~40 m)	Half	Multi-master bus, 120 ohm ends	11- or 29-bit message ID	CRC-15, stuffing, form, ACK, bit monitor
CAN FD	Same	Arbitration ≤ 1 Mb/s; data typically 2-8 Mb/s	Half	As CAN	As CAN	CRC-17/21 + stuff count
LIN	1 wire (+GND, 12 V battery level)	20 kb/s	Half	1 commander, up to ~16 responders	6-bit frame ID + 2 parity	8-bit inverted checksum (classic/enhanced)
I2S	SCK (BCLK), WS (LRCLK), SD (+MCLK)	e.g. 3.072 MHz (48 kHz x 2 x 32 bit)	Simplex per data line	Point-to-point	None (slot by WS)	None
1-Wire	DQ + GND (parasite power)	~16 kb/s standard, ~140 kb/s overdrive	Half	Multi-drop	64-bit ROM ID	CRC-8/MAXIM
MDIO (Clause 22/45)	MDC, MDIO	2.5 MHz per 802.3 (many PHYs faster)	Half	1 station manager, up to 32 PHYs	C22: 5-bit PHY + 5-bit reg; C45: +5-bit MMD, 16-bit reg	None (turnaround bits only)

High-speed serial links (Chapters 17-22)

Link	Rate per lane	Encoding	Lanes	Reach (typ.)
USB 1.1 LS / FS	1.5 / 12 Mb/s	NRZI + bit stuffing	1 half-duplex D+/D-	3 m (LS) / 5 m cable
USB 2.0 HS	480 Mb/s	NRZI + bit stuffing	1 half-duplex pair	5 m cable
USB 3.2 Gen 1 / Gen 2	5 / 10 Gb/s	8b/10b / 128b/132b	1 TX + 1 RX pair (x2: 20 Gb/s on Type-C)	~1-3 m cable (passive)
USB4 v1 (Gen 2 / Gen 3)	10 / 20 Gb/s	64b/66b / 128b/132b (+RS-FEC at Gen 3)	2 lanes -> 20 / 40 Gb/s	~0.8 m passive at 40G
USB4 v2 (Gen 4) (*)	40 Gb/s (PAM3)	11b/7T PAM3 + FEC	2 lanes -> 80 Gb/s (120/40 asymmetric)	~1 m passive
PCIe 1.x / 2.x	2.5 / 5 GT/s	8b/10b	x1-x16 (x32 in early specs)	~0.3-0.5 m FR4 channel
PCIe 3.x / 4.0 / 5.0	8 / 16 / 32 GT/s	128b/130b	x1-x16	Loss budget ~22 / 28 / 36 dB at Nyquist; retimers beyond
PCIe 6.x / 7.0 (*)	64 / 128 GT/s (PAM4)	1b/1b FLIT mode + light FEC + CRC	x1-x16	~32-36 dB class channels
Ethernet 10/100/1000BASE-T	10 M / 100 M / 1 G	Manchester / 4B5B+MLT-3 / PAM5 (4 pairs)	1-4 twisted pairs	100 m
10GBASE-R / 25GBASE-R	10.3125 / 25.78125 GBd	64b/66b (+RS-FEC at 25G)	1 per direction	Backplane ~1 m, optics km
50/100/200 G per lane Ethernet (*)	26.6 / 53.1 / 106.25 GBd PAM4	64b/66b -> 256b/257b + RS(544,514) FEC	1-8 (200G-1.6T ports)	Host ~ tens of cm; optics
MIPI D-PHY v1.2 / v2.x	Up to 2.5 / 4.5 Gb/s	None (raw, HS/LP modes)	1 clock + 1-4 data lanes	Tens of cm (in-phone flex)
MIPI C-PHY v1.x / v2.x (*)	Up to ~2.5 / ~6 Gsym/s (x 2.28 bit/sym)	16 bits -> 7 symbols, 3-wire trios	1-3 trios, embedded clock	Tens of cm
HDMI 1.4 / 2.0 (TMDS)	3.4 / 6 Gb/s	TMDS 8b/10b variant	3 data + 1 clock	~5-10 m cable
HDMI 2.1 FRL / 2.2 (*)	12 / 24 Gb/s	16b/18b + RS FEC	4 lanes -> 48 / 96 Gb/s	~2-5 m (Ultra High Speed cable)
DisplayPort 1.4 (HBR3)	8.1 Gb/s	8b/10b	1, 2 or 4 main lanes + AUX	~2-3 m
DisplayPort 2.x (UHBR10/13.5/20)	10 / 13.5 / 20 Gb/s	128b/132b	Up to 4 lanes -> 80 Gb/s	~1-2 m at UHBR20
JESD204B	Up to 12.5 Gb/s	8b/10b	1-32+ lanes	Board level
JESD204C	Up to 32 Gb/s	64b/66b (also 8b/10b, 64b/80b)	1-32+ lanes	Board level
UCle 1.x-2.0 / 3.0 (*)	4-32 GT/s / up to 64 GT/s	None on the wire; 256 B flits with CRC + retry in the D2D adapter	x16 standard package, x64 (x32) advanced	~25 mm standard, ~2 mm advanced package

GT/S VERSUS GB/S

GT/s counts line symbols (transfers) per second before coding overhead; Gb/s is usually payload bits. PCIe 3.0: 8 GT/s x 128/130 = 7.88 Gb/s per lane per direction = 0.985 GB/s. For PAM4 links the baud rate is half the bit rate (2 bits per symbol).

Memory interfaces (Chapter 20)

Memory	Data rate per pin	Device / channel width	Voltage (core/IO)	Peak bandwidth per device (example)
DDR4 (JESD79-4)	1600-3200 MT/s	x4 / x8 / x16 device; 64-bit (+8 ECC) DIMM	VDD = VDDQ 1.2 V, VPP 2.5 V	x16 @ 3200: 6.4 GB/s; DIMM: 25.6 GB/s
DDR5 (JESD79-5)	3200-6400 MT/s at launch, extended toward ~8800 (*)	x4 / x8 / x16; DIMM = 2 x 32-bit subchannels	1.1 V (VPP 1.8 V); PMIC on DIMM	x16 @ 6400: 12.8 GB/s; DIMM: 51.2 GB/s
LPDDR4X (JESD209-4)	Up to 4266 MT/s	x16 channels (2 per die)	VDD2 1.1 V, VDDQ 0.6 V	x16 @ 4266: 8.5 GB/s
LPDDR5 (JESD209-5)	Up to 6400 MT/s	x16 channels	VDD2 ~1.05 V, VDDQ 0.5 V	x16 @ 6400: 12.8 GB/s
LPDDR5X (*)	Up to 8533 MT/s (later bins ~9600-10700)	x16 channels	As LPDDR5 (lower VDD2 options)	x16 @ 8533: 17.1 GB/s; x64 package: 68.3 GB/s
GDDR6 (JESD250)	Typically 14-20 Gb/s (NRZ)	x32 device = 2 x16 channels (x8 clamshell)	1.35 V / 1.25 V	16 Gb/s x 32: 64 GB/s
GDDR7 (JESD239) (*)	~28-32 Gb/s at launch, spec up to ~48 (PAM3)	x32 device, 4 channels	~1.2 / 1.1 V	32 Gb/s x 32: 128 GB/s
HBM2E	3.2-3.6 Gb/s	1024-bit stack, 8 x 128-bit channels	1.2 V	3.6 Gb/s: 460.8 GB/s per stack
HBM3 (JESD238)	6.4 Gb/s	1024-bit, 16 x 64-bit channels (32 pseudo-channels)	VDD 1.1 V, VDDQ 0.4 V	819.2 GB/s per stack
HBM3E (*)	~8-9.8 Gb/s (vendor bins)	1024-bit	As HBM3	9.6 Gb/s: 1228.8 GB/s per stack
HBM4 (JESD270-4) (*)	~8 Gb/s baseline	2048-bit, 32 channels	Vendor-specific options	8 Gb/s: 2048 GB/s per stack

Peak BW [GB/s] = data rate [MT/s or Gb/s per pin] x width [bits] / 8

DDR5-6400 DIMM: $6400 \times 64 / 8 = 51.2$ GB/s (sustained ~70-85%)

HBM3 stack : $6.4 \times 1024 / 8 = 819.2$ GB/s

Every peak number in the table is this one line; real efficiency is lower (refresh, turnarounds, page misses).

CRC polynomials per protocol (verified)

Parameters use the Rocksoft/Williams model that CRC catalogues use: *poly* in normal (MSB-first) form without the implicit top bit, *init* register preset, *refin/refout* whether bytes enter and the result leaves LSB-first, and *xorout* the final inversion. The *check* column is the CRC of the ASCII string "123456789" - the universal self-test for a CRC implementation. Every check value below was reproduced by the Python script that follows and matched the published catalogue value (18/18 PASS). How a protocol maps fields onto the bit stream (which bits are covered, bit order on the wire, stuff bits) is protocol-specific - see the chapter.

Protocol / use	Catalogue name	W	Poly	Init	Ref I.	XorOut	Check
USB token / SOF (17)	CRC-5/USB	5	0x05	0x1F	yes	0x1F	0x19
SD/MMC command (20)	CRC-7/MMC	7	0x09	0x00	no	0x00	0x75
SMBus PEC; DDR4/5 write-CRC polynomial (13, 20)	CRC-8/SMBUS	8	0x07	0x00	no	0x00	0xF4
1-Wire ROM / scratchpad (16)	CRC-8/MAXIM-DO W	8	0x31	0x00	yes	0x00	0xA1
SAE J1850, AUTOSAR E2E CRC-8 (15)	CRC-8/SAE-J1850	8	0x1D	0xFF	no	0xFF	0x4B
FlexRay header (15)	CRC-11/FLEXRAY	11	0x385	0x01A	no	0x000	0x5A3
Classical CAN (15)	CRC-15/CAN	15	0x4599	0x0000	no	0x0000	0x059E
USB data packets (17)	CRC-16/USB	16	0x8005	0xFFFF	yes	0xFFFF	0xB4C8
SD data lines, XMODEM (20)	CRC-16/XMODEM	16	0x1021	0x0000	no	0x0000	0x31C3
IEEE 802.15.4, Bluetooth BR (-)	CRC-16/KERMIT	16	0x1021	0x0000	yes	0x0000	0x2189
HDLC / X.25 / PPP FCS-16 (3)	CRC-16/IBM-SDLC	16	0x1021	0xFFFF	yes	0xFFFF	0x906E
MIPI CSI-2 / DSI packet footer (21)	CRC-16/MCRF4X X	16	0x1021	0xFFFF	yes	0x0000	0x6F91
CAN FD, <= 16 data bytes (15)	CRC-17/CAN-FD	17	0x1685B	0x00000 (see note)	no	0x00000	0x04F03
CAN FD, > 16 data bytes (15)	CRC-21/CAN-FD	21	0x102899	0x000000 (see note)	no	0x000000	0x0ED841
Bluetooth LE link layer (-)	CRC-24/BLE	24	0x00065B	0x555555	yes	0x000000	0xC25A56
FlexRay frame, channel A (15)	CRC-24/FLEXRAY -A	24	0x5D6DCB	0xFEDCBA	no	0x000000	0x7979BD
Ethernet FCS, PCIe LCRC/ECRC poly, SATA, HDLC-32 (18, 19)	CRC-32/ISO-HDLC	32	0x04C11DB7	0xFFFFFFFF	yes	0xFFFFFFFF	0xCBF43926
iSCSI, SCTP, NVMe-oF data digest, ext4 (-)	CRC-32/ISCSI (CRC-32C)	32	0x1EDC6F41	0xFFFFFFFF	yes	0xFFFFFFFF	0xE3069283

```
# Generic bit-serial CRC (Rocksoft/Williams model) and catalogue check.
```

```
def crc(data, w, poly, init, refin, refout, xorout):
    top, mask = 1 << (w - 1), (1 << w) - 1
    reg = init
    for byte in data:
        if refin:
            byte = int(f"{byte:08b}"[::-1], 2)
```

```

        for i in range(7, -1, -1):
            fb = ((reg & top) != 0) ^ ((byte >> i) & 1)
            reg = ((reg << 1) & mask) ^ (poly if fb else 0)
    if refout:
        reg = int(f"{reg:0{w}b}"[::-1], 2)
    return reg ^ xorout

# name, width, poly, init, refin, refout, xorout, expected check
T = [
    ("CRC-5/USB", 5, 0x05, 0x1F, 1, 1, 0x1F, 0x19),
    ("CRC-7/MMC", 7, 0x09, 0x00, 0, 0, 0x00, 0x75),
    ("CRC-8/SMBUS", 8, 0x07, 0x00, 0, 0, 0x00, 0xF4),
    ("CRC-8/MAXIM-DOW", 8, 0x31, 0x00, 1, 1, 0x00, 0xA1),
    ("CRC-8/SAE-J1850", 8, 0x1D, 0xFF, 0, 0, 0xFF, 0x4B),
    ("CRC-11/FLEXRAY", 11, 0x385, 0x01A, 0, 0, 0x000, 0x5A3),
    ("CRC-15/CAN", 15, 0x4599, 0x0000, 0, 0, 0x0000, 0x059E),
    ("CRC-16/USB", 16, 0x8005, 0xFFFF, 1, 1, 0xFFFF, 0xB4C8),
    ("CRC-16/XMODEM", 16, 0x1021, 0x0000, 0, 0, 0x0000, 0x31C3),
    ("CRC-16/KERMIT", 16, 0x1021, 0x0000, 1, 1, 0x0000, 0x2189),
    ("CRC-16/IBM-SDLC", 16, 0x1021, 0xFFFF, 1, 1, 0xFFFF, 0x906E),
    ("CRC-16/MCRF4XX", 16, 0x1021, 0xFFFF, 1, 1, 0x0000, 0x6F91),
    ("CRC-17/CAN-FD", 17, 0x1685B, 0x00000, 0, 0, 0x00000, 0x04F03),
    ("CRC-21/CAN-FD", 21, 0x102899, 0x000000, 0, 0, 0x000000, 0x0ED841),
    ("CRC-24/BLE", 24, 0x00065B, 0x555555, 1, 1, 0x000000, 0xC25A56),
    ("CRC-24/FLEXRAY-A", 24, 0x5D6DCB, 0xFEDCBA, 0, 0, 0x000000, 0x7979BD),
    ("CRC-32/ISO-HDLC", 32, 0x04C11DB7, 0xFFFFFFFF, 1, 1, 0xFFFFFFFF, 0xCBF43926),
    ("CRC-32/ISCSI", 32, 0x1EDC6F41, 0xFFFFFFFF, 1, 1, 0xFFFFFFFF, 0xE3069283),
]

msg = b"123456789"
fails = 0
for name, w, poly, init, ri, ro, xo, exp in T:
    got = crc(msg, w, poly, init, ri, ro, xo)
    ok = got == exp
    fails += not ok
    d = (w + 3) // 4
    print(f"{name:17s} w={w:2d} poly=0x{poly:0{d}X} check=0x{got:0{d}X} "
          f"expect=0x{exp:0{d}X} {'PASS' if ok else 'FAIL'}")
import binascii, zlib
print(f"cross-check zlib.crc32      = 0x{zlib.crc32(msg):08X}")
print(f"cross-check binascii.crc_hqx= 0x{binascii.crc_hqx(msg, 0):04X} (XMODEM)")
print(f"{len(T) - fails}/{len(T)} catalogue check values reproduced")

```

crc_check.py - one generic bit-serial CRC engine reproduces every catalogue check value (the same shift-register structure as the RTL CRC generators of Chapters 3 and 26).

```

$ python3 crc_check.py
CRC-5/USB      w= 5 poly=0x05 check=0x19 expect=0x19 PASS
CRC-7/MMC      w= 7 poly=0x09 check=0x75 expect=0x75 PASS
CRC-8/SMBUS    w= 8 poly=0x07 check=0xF4 expect=0xF4 PASS
CRC-8/MAXIM-DOW w= 8 poly=0x31 check=0xA1 expect=0xA1 PASS
CRC-8/SAE-J1850 w= 8 poly=0x1D check=0x4B expect=0x4B PASS
CRC-11/FLEXRAY w=11 poly=0x385 check=0x5A3 expect=0x5A3 PASS
CRC-15/CAN     w=15 poly=0x4599 check=0x059E expect=0x059E PASS
CRC-16/USB     w=16 poly=0x8005 check=0xB4C8 expect=0xB4C8 PASS
CRC-16/XMODEM w=16 poly=0x1021 check=0x31C3 expect=0x31C3 PASS
CRC-16/KERMIT w=16 poly=0x1021 check=0x2189 expect=0x2189 PASS
CRC-16/IBM-SDLC w=16 poly=0x1021 check=0x906E expect=0x906E PASS
CRC-16/MCRF4XX w=16 poly=0x1021 check=0x6F91 expect=0x6F91 PASS
CRC-17/CAN-FD w=17 poly=0x1685B check=0x04F03 expect=0x04F03 PASS
CRC-21/CAN-FD w=21 poly=0x102899 check=0x0ED841 expect=0x0ED841 PASS
CRC-24/BLE     w=24 poly=0x00065B check=0xC25A56 expect=0xC25A56 PASS
CRC-24/FLEXRAY-A w=24 poly=0x5D6DCB check=0x7979BD expect=0x7979BD PASS

```

```

CRC-32/ISO-HDLC    w=32 poly=0x04C11DB7 check=0xCBF43926 expect=0xCBF43926 PASS
CRC-32/ISCSI       w=32 poly=0x1EDC6F41 check=0xE3069283 expect=0xE3069283 PASS
cross-check zlib.crc32      = 0xCBF43926
cross-check binascii.crc_hqx= 0x31C3 (XMODEM)
18/18 catalogue check values reproduced

```

Real output (Python 3). zlib and binascii give independent cross-checks for CRC-32 and CRC-16/XMODEM.

PITFALL - SAME POLYNOMIAL, DIFFERENT CRC

The CAN FD standard presets the CRC register with a 1 in its MSB and includes stuff bits in the calculation; the catalogue entries above use init = 0 over plain bytes, so they are a self-test of the polynomial engine, not a drop-in frame CRC. Likewise PCIe LCRC uses the Ethernet polynomial but its own bit mapping, and DDR4/5 write CRC uses x^8+x^2+x+1 over a JEDEC-defined bit matrix. Always validate against a known-good frame from the spec or an analyzer capture, not just "123456789".

Line coding (Chapter 2)

Code	Bits -> symbols	Overhead	DC balance / run length	Clock recovery	Used by
NRZ	1 bit -> 1 level	0%	None / unbounded	Needs scrambling or separate clock	Parallel buses, SPI, DDR, most PAM2 SerDes after scrambling
NRZI + bit stuffing	1 -> no transition, 0 -> transition; stuff 0 after 6 ones	0-16.7% (data-dependent)	Not DC-balanced / max 7 bits	Transitions guaranteed	USB 1.x/2.0
Manchester	1 bit -> 2 half-bit levels	100% (2x baud)	Balanced / 1 bit	Every bit has a mid transition	10BASE-T, some RFID, DALI
4B/5B (+ MLT-3)	4 bits -> 5 bits; 3-level line	25%	Bounded run length	Good	100BASE-TX, FDDI
8b/10b	8 bits -> 10 bits, running disparity, K-codes	25% (80% efficiency)	Strict DC balance / max run 5	Excellent; commas for alignment	PCIe 1-2, USB 3 Gen 1, SATA, 1000BASE-X, DP 1.x, JESD204B
TMDS	8 bits -> 10 bits, transition-minimised	25%	DC balanced	Separate clock channel	HDMI 1.x-2.0, DVI
64b/66b	64 bits + 2-bit sync header, scrambled	3.1%	Statistical (scrambler) / sync header forces 1 transition per 66 bits	Good	10G/25G/100G Ethernet, JESD204C, USB4 Gen 2
128b/130b	128 bits + 2-bit sync header, scrambled	1.5%	Statistical	Good	PCIe 3.0-5.0
128b/132b	128 bits + 4-bit header, scrambled	3.0%	Statistical	Good	USB 3.2 Gen 2, USB4 Gen 3, DP 2.x
16b/18b	16 bits + 2-bit header	12.5%	Statistical	Good	HDMI 2.1 FRL
PAM4	2 bits -> 1 of 4 levels (Gray coded)	0% (needs FEC)	Scrambled	Half the baud of NRZ	PCIe 6/7, 50-200 G/lane Ethernet, GDDR6X
PAM3	3 bits -> 2 ternary symbols (GDDR7); 11 b -> 7 trits (USB4 v2)	~5% vs $\log_2(3)$ ideal	Scrambled	Good	GDDR7, USB4 v2, 100BASE-T1
PAM5 / PAM16	Multi-level with trellis/DSQ coding	-	Scrambled	DSP-based	1000BASE-T / 10GBASE-T
C-PHY 3-phase	16 bits -> 7 symbols on 3 wires	2.28 bit/symbol	Every symbol has a transition	Embedded clock	MIPI C-PHY

Appendix B. Glossary

Terms used across the book, in alphabetical order. Numbers in parentheses point to the chapter where the concept is treated in depth.

Term	Definition
128b/130b	PCIe 3.0-5.0 encoding: 128-bit block plus 2-bit sync header (18).
64b/66b	Scrambled 64-bit block with a 2-bit sync header (01 data, 10 control); 3% overhead (2, 19).
8b/10b	DC-balanced line code mapping 8 data bits to 10-bit symbols with running disparity and control (K) characters (2).
ACE	AXI Coherency Extensions: adds snoop channels AC/CR/CD and cache-state signalling to AXI (9).
ACK / NAK	Positive / negative acknowledgement; in PCIe DLLPs that retire or replay TLPs from the replay buffer (3, 18).
AER	PCIe Advanced Error Reporting capability (18).
AHB	AMBA Advanced High-performance Bus: pipelined address/data phases, single outstanding transfer (6).
Alternate mode	USB Type-C mechanism that hands SuperSpeed wires to DisplayPort or Thunderbolt (17).
AMBA	Arm's Advanced Microcontroller Bus Architecture family: APB, AHB, AXI, ACE, CHI, ATB, DTI (5-9).
APB	AMBA Advanced Peripheral Bus: two-phase, non-pipelined register bus (5).
Arbitration	Deciding which requester wins a shared resource; bitwise (CAN, I2C) or logic-based (interconnect) (3, 15).
ATB	AMBA Trace Bus carrying trace data to funnels and sinks (23).
AUX channel	DisplayPort half-duplex side channel for link training, EDID and DPCD access (21).
Avalon	Intel FPGA on-chip interface family (Avalon-MM, Avalon-ST) (10).
AXI	AMBA Advanced eXtensible Interface: five independent VALID/READY channels, IDs and bursts (7).
AXI4-Stream	Address-less streaming interface with TVALID/TREADY/TLAST (8).
Backpressure	A receiver throttling the sender (READY low, credits exhausted, PAUSE frames) (3).
Bathtub curve	BER versus sampling phase; its opening at a target BER is the timing margin (2).
Baud	Symbols per second; equals bit rate only for 2-level signalling (2).
BER	Bit error ratio; SerDes specs typically require 1e-12 or better (pre-FEC targets are looser) (2).
Bit stuffing	Inserting a complementary bit after N identical bits to guarantee edges (CAN after 5, USB after 6 ones) (11, 15, 17).
Boundary scan	IEEE 1149.1 chain of cells at the pins, used to test board interconnect (23).
BoW	Bunch of Wires: OCP (Open Compute) die-to-die parallel interface (22).
BTA	MIPI D-PHY Bus Turn-Around handing the lane to the other side (21).
Burst	Several data beats for one address phase (6, 7).
C-PHY	MIPI 3-wire, 3-phase PHY with embedded clock (21).
CAN	Controller Area Network, ISO 11898; multi-master differential bus with bitwise arbitration (15).
CAN FD	CAN with Flexible Data rate: up to 64 data bytes and a faster data phase (15).
CAN XL	Third CAN generation: up to 2048 data bytes, data phase typically 10-20 Mb/s (15).
CDC	Clock domain crossing; synchronizers and async FIFOs (4).

Term	Definition
CDR	Clock and data recovery: extracts a sampling clock from the data transitions (2).
CHI	AMBA Coherent Hub Interface: packetised, credit-based coherent protocol for meshes (9).
Chip select	Per-target enable line in SPI (12).
Clock stretching	I2C target holding SCL low to delay the controller (13).
CMN	Arm Coherent Mesh Network, a CHI interconnect (9).
Comma	8b/10b symbol (K28.5) whose bit pattern cannot appear across symbol boundaries; used for alignment (2).
Completer	PCIe device that services a request (18).
Compliance test	Standardised test suite a product must pass for certification/logo (25).
Controller / target	Current I2C/SPI/I3C terms replacing master/slave (12-14).
CoreSight	Arm debug and trace architecture (DAP, CTI, ETM, funnels) (23).
CPOL / CPHA	SPI clock polarity and phase; four modes 0-3 (12).
CRC	Cyclic redundancy check: remainder of polynomial division over GF(2) (3, 26).
Credit-based flow control	Sender may transmit only while it holds credits advertised by the receiver (PCIe, CHI, UCIe) (3).
CSI-2	MIPI Camera Serial Interface 2 protocol layer (21).
CTLE	Continuous-time linear equaliser in a SerDes receiver (2).
CXL	Compute Express Link: CXL.io, CXL.cache and CXL.mem over the PCIe PHY (18).
D2D adapter	UCIe layer that adds CRC, retry and arbitration between protocol and PHY (22).
DAP	Arm Debug Access Port reached through JTAG or SWD (23).
DBI	Data bus inversion: inverting a byte to reduce transitions or zeros (DDR4, GDDR) (20).
DDR	Double data rate: data on both clock edges; also the JEDEC DRAM family (20).
De-emphasis	Transmitter FIR that attenuates repeated bits to pre-compensate channel loss (2).
Deadlock	Circular wait between agents or channels; prevented by ordering rules and virtual channels (3, 9).
Decision feedback equaliser (DFE)	Receiver equaliser that subtracts ISI of already-decided bits (2).
DFI	DDR PHY Interface between memory controller and PHY (20).
Differential signalling	Information carried as the difference of two complementary wires; rejects common-mode noise (2).
DLLP	PCIe Data Link Layer Packet: ACK/NAK, flow control, power management (18).
DMA	Direct memory access engine moving data without the CPU (24).
DP (DisplayPort)	VESA packetised display link with AUX channel (21).
DQS	DDR data strobe, source-synchronous with DQ (20).
DSI	MIPI Display Serial Interface (21).
ECC	Error correcting code, e.g. SECDED Hamming on memory (3, 26).
ECRC	PCIe end-to-end CRC on a TLP (18).
EDID	Extended Display Identification Data read over DDC (I2C) (21).
Elastic buffer	FIFO absorbing ppm frequency offset using SKP/idle insertion and deletion (2, 18).
Embedded clock	Clock recovered from data transitions rather than sent on its own wire (2).
Endpoint	USB: addressable buffer in a device; PCIe: a function at the leaf of the hierarchy (17, 18).

Term	Definition
ENTDAA	I3C broadcast command that assigns dynamic addresses (14).
Enumeration	Host discovering and configuring devices (USB, PCIe) (17, 18).
Equalisation	TX FIR plus RX CTLE/DFE compensating channel loss (2).
Exclusive access	AXI/AHB load-exclusive/store-exclusive monitor-based atomic mechanism (7).
Eye diagram	Overlay of unit intervals showing voltage and timing margin (2).
FEC	Forward error correction, e.g. RS(544,514) in Ethernet (19, 26).
FIFO	First-in first-out buffer; async FIFOs cross clock domains (4).
FlexRay	Deterministic TDMA automotive bus at 10 Mb/s (15).
FLIT	Flow control unit: fixed-size link-layer packet (CHI, PCIe 6 FLIT mode, UCle) (9, 18, 22).
Frame	Delimited unit of transfer at the link layer (Ethernet, CAN, UART) (3).
GDDR	Graphics DDR DRAM family with high per-pin rates (20).
Gray code	Code in which consecutive values differ by one bit; used for async FIFO pointers and PAM4 levels (2, 4).
GT/s	Giga-transfers per second (symbol rate before coding) (18).
Handshake	Signal exchange that qualifies a transfer, e.g. VALID/READY, REQ/ACK (3).
HBM	High Bandwidth Memory: 3D-stacked DRAM with 1024-bit or wider interface on an interposer (20).
HDCP	High-bandwidth Digital Content Protection on HDMI/DP (26).
HDMI	High-Definition Multimedia Interface (21).
HDR (I3C)	I3C High Data Rate modes (HDR-DDR, HDR-BT, ...) (14).
Hot plug	Attaching a device while powered; detection via HPD, VBUS or presence pins (17, 21).
I2C	Inter-Integrated Circuit two-wire open-drain bus (NXP UM10204) (13).
I2S	Inter-IC Sound serial audio bus (16).
I3C	MIPI Improved Inter-Integrated Circuit: push-pull, in-band interrupts, dynamic addressing (14).
IBI	I3C In-Band Interrupt raised by a target on SDA (14).
IDE	PCIe Integrity and Data Encryption (AES-GCM on TLPs) (26).
IEEE 1500	Standard for embedded core test wrappers (23).
IEEE 1588 (PTP)	Precision Time Protocol for sub-microsecond clock sync over Ethernet (19).
IEEE 1687 (IJTAG)	Access network for embedded instruments using SIBs and ICL/PDL (23).
IFG / IPG	Ethernet inter-frame gap, minimum 96 bit times (19).
Interleaving	Spreading addresses over banks/channels to overlap accesses (20).
Interposer	Silicon or organic substrate connecting dies in a 2.5D package (20, 22).
ISI	Inter-symbol interference caused by channel loss and reflections (2).
JESD204	JEDEC serial interface for data converters (B: 8b/10b, C: 64b/66b) (22).
Jitter	Deviation of edges from ideal time; random (RJ) and deterministic (DJ) parts (2).
JTAG	IEEE 1149.1 test access port: TCK, TMS, TDI, TDO, optional TRST (23).
K-code	8b/10b control symbol (COM, SKP, STP, ...) (2).
Lane	One differential pair per direction (or pair of pairs) of a serial link (2).
Latency	Time from request to response; with bandwidth sets outstanding requirements (Little's law) (7, 27).

Term	Definition
LCRC	PCIe link CRC (32-bit) protecting a TLP hop by hop (18).
LFSR	Linear feedback shift register used for CRC, scramblers and PRBS (3).
LIN	Local Interconnect Network: single-wire 20 kb/s automotive bus (15).
Link training	Automatic negotiation of rate, width and equalisation (PCIe LTSSM, DP) (18, 21).
LPDDR	Low-power DDR DRAM for mobile (20).
LTSSM	PCIe Link Training and Status State Machine (18).
LVDS	Low-voltage differential signalling, ~350 mV swing (2).
MAC	Media Access Control layer (Ethernet framing, CRC, pause) (19).
Manchester code	Line code with a transition at mid-bit (2).
MDIO	Management Data I/O bus to Ethernet PHY registers (19).
MESI / MOESI	Cache coherence state protocols (9).
Metastability	Unresolved flop state after a setup/hold violation (4).
MIPI	MIPI Alliance: mobile interface specs (D-PHY, C-PHY, CSI-2, DSI, I3C) (14, 21).
MLT-3	Three-level line code cycling -1/0/+1 used by 100BASE-TX (19).
MPS / MRRS	PCIe Max Payload Size / Max Read Request Size (18).
MSI / MSI-X	Message-signalled interrupts as memory writes (18).
Multi-drop	Bus topology with several devices on the same wires (2).
NoC	Network-on-chip: packet-switched on-chip interconnect (10).
NRZ	Non-return-to-zero, two-level signalling (2).
NRZI	NRZ inverted: data encoded as presence/absence of a transition (17).
OCP	Open Core Protocol (OCP-IP; now under Accellera) socket standard (10).
ONFI	Open NAND Flash Interface (20).
Open drain	Output that only pulls low; high level from a pull-up resistor (13).
Ordering model	Rules on which transactions may pass others (AXI ID, PCIe relaxed ordering) (3, 7, 18).
Outstanding transaction	Request issued whose response has not yet returned (7).
PAM4	Four-level pulse-amplitude modulation, 2 bits per symbol (2).
Parity	Single-bit even/odd check detecting odd numbers of errors (3).
PCIe	PCI Express, packetised serial load/store interconnect (18).
PCS	Physical Coding Sublayer: encoding, scrambling, lane alignment (19).
PHY	Physical layer: analog front end plus PCS (2, 24).
PIPE	PHY Interface for PCI Express (and USB/SATA) between MAC and PHY (18, 24).
PLL	Phase-locked loop generating clocks from a reference (2).
PMA	Physical Medium Attachment: SerDes, CDR (19).
PMBus	Power Management Bus built on SMBus (13).
Posted write	Write needing no completion (PCIe memory write, AXI early response) (18).
PRBS	Pseudo-random bit sequence (PRBS7/15/31) for link test (2).
Pre-emphasis	TX boost of transitions to compensate loss (2).
Preamble	Known pattern before a frame for sync (Ethernet 7 x 0x55 + SFD) (19).
QSPI	Quad SPI: four bidirectional data lines (12).

Term	Definition
Refresh	Periodic DRAM row restore (tREFI ~3.9-7.8 us) (20).
Repeated START	I2C START without a preceding STOP (13).
Replay buffer	PCIe transmitter copy of unacknowledged TLPs (18).
Retimer	Active device that fully recovers and retransmits a link, resetting the jitter and loss budget (18).
Root complex	PCIe hierarchy root connecting CPU/memory (18).
RS-232 / RS-485	EIA/TIA electrical standards for UART links; single-ended / differential multi-drop (11).
Running disparity	8b/10b state tracking the ones-minus-zeros balance (2).
Sample point	Position within a bit where the receiver samples; CAN typically 75-87.5% (15).
Scrambler	LFSR XORed onto data to whiten the spectrum (2).
SDR	Single data rate (one transfer per clock) (2).
SerDes	Serializer/deserializer with CDR (2).
SGMII	Serial Gigabit MII, 1.25 GBd 8b/10b MAC-PHY link (19).
Skew	Arrival-time difference between parallel bits or lanes; removed by deskew (2).
SKP ordered set	PCIe/USB symbols inserted for clock compensation (18).
SMBus	System Management Bus, a stricter I2C derivative (13).
Snoop	Coherency query to other caches (9).
Source-synchronous	Clock or strobe sent alongside data (DDR DQS, D-PHY) (2).
SPI	Serial Peripheral Interface, 4-wire synchronous (12).
Split transaction	Request and response decoupled in time (AXI, PCIe, USB 2.0 hub split) (3).
SSC	Spread-spectrum clocking to cut EMI, e.g. -5000 ppm down-spread in PCIe (2).
SWD	Arm Serial Wire Debug, 2-pin alternative to JTAG (23).
TDM	Time-division multiplexing of several channels on one line (16).
Termination	Resistive match to the line impedance to stop reflections (2).
TileLink	Chip-scale interconnect standard from SiFive/Berkeley (10).
TLP	PCIe Transaction Layer Packet (18).
Transaction ID	Tag matching responses to requests (AXI ID, PCIe tag, CHI TxnID) (3).
UCIe	Universal Chiplet Interconnect Express die-to-die standard (22).
UI	Unit interval: one symbol time (2).
USB	Universal Serial Bus (17).
VALID / READY	AMBA handshake: transfer when both high on a clock edge (7).
VIP	Verification IP: protocol BFM, monitor, checker and coverage (25).
Virtual channel	Independent logical queues on one link preventing head-of-line blocking/deadlock (3, 18).
Wishbone	Open-source on-chip bus (OpenCores) (10).
WRAP burst	Burst whose address wraps at a size-aligned boundary, for critical-word-first cache fills (6, 7).
Write strobe	Byte enables (WSTRB , PSTRB , HWSTRB) (5, 7).
xMII	Ethernet MAC-PHY family: MII, RMII, GMII, RGMII, SGMII, XGMII, USXGMII (19).
xSPI	JEDEC JESD251 expanded SPI for octal DDR flash (12).

Appendix C. 100 Communication Protocol Interview Questions with Answers

Grouped by topic, from screening questions to on-site design puzzles. Answer out loud first, then check. Numeric answers were worked out with Python; assumptions are stated so you can redo them with an interviewer's numbers.

Fundamentals and signalling

Q1. What is the difference between synchronous, source-synchronous and embedded-clock (asynchronous serial) links?

Synchronous: a common clock goes to both ends (APB, SPI from the controller's view). Source-synchronous: the transmitter sends a clock or strobe with the data (DDR DQS, D-PHY clock lane) so the flight times track. Embedded clock: no clock wire; the receiver's CDR recovers timing from data transitions (PCIe, USB 3, Ethernet), which requires a line code or scrambler that guarantees edges.

Q2. Why are high-speed links differential?

Common-mode noise and ground shifts cancel, radiated EMI is lower because the fields of the two wires cancel, swing per wire can be small (hundreds of mV), and the receiver has a well-defined crossing point, which reduces jitter.

Q3. What is the unit interval of a 16 GT/s link, and what is the Nyquist frequency?

UI = $1/16\text{e}9 = 62.5$ ps. Nyquist (fastest 1010 pattern) is half the symbol rate: 8 GHz. Channel loss is always quoted at Nyquist.

Q4. Compare 8b/10b and 128b/130b.

8b/10b: 20% of the line rate is overhead, strict DC balance, max run length 5, commas for alignment. 128b/130b: 1.5% overhead, relies on a scrambler for DC balance/transitions and a 2-bit sync header for block alignment. PCIe 3.0 moved to 128b/130b so 8 GT/s delivers ~2x the payload of 5 GT/s 8b/10b.

Q5. Why does PAM4 need FEC when NRZ at the same bit rate often does not?

Three eyes stacked in the same swing: each eye has one third of the amplitude (about -9.5 dB SNR penalty), so raw BER is far worse ($\sim 1\text{e-}6$ to $1\text{e-}4$ pre-FEC). FEC (RS in Ethernet, a light FEC plus CRC and retry in PCIe 6) recovers the target error rate.

Q6. What is the purpose of a scrambler, and what is the difference between additive and self-synchronising scramblers?

It whitens the data spectrum (EMI, DC balance, transitions for CDR). Additive (frame-synchronous, PCIe/USB): an LFSR sequence XORed onto data, both ends reset in lock-step - no error multiplication. Self-synchronising (64b/66b, $x^{58} + x^{39} + 1$): the scrambled output feeds the LFSR, so the descrambler syncs automatically but one line error becomes up to 3 bit errors.

Q7. Explain TX de-emphasis, CTLE and DFE in one sentence each.

De-emphasis: a TX FIR reduces the amplitude of repeated bits so high frequencies are relatively boosted. CTLE: an RX analog filter with a zero that boosts high frequencies (and noise). DFE: subtracts the ISI of already-decided bits, boosting no noise but vulnerable to error propagation.

Q8. What is the difference between a redriver and a retimer?

A redriver is an analog amplifier/equaliser: it restores amplitude but also passes jitter and noise through, and is protocol-unaware. A retimer has a CDR and full PHY (and for PCIe participates in link training), so it resets the jitter and loss budget at the cost of latency (tens of ns) and power.

Handshakes, flow control and CRC

Q9. State the AXI VALID/READY rules that prevent deadlock.

A source must not wait for READY before asserting VALID, and once VALID is asserted it must stay high with stable payload until the handshake. A destination may wait for VALID before asserting READY. The transfer happens on the edge where both are high.

Q10. Credit-based versus ready/valid flow control - when is each better?

Ready/valid is a per-cycle combinational back-pressure: simple, but READY must propagate backward within a cycle, which hurts over long distances. Credits decouple: the receiver pre-advertises buffer space, so the sender needs no same-cycle feedback; loop latency is hidden by enough credits (buffer $\geq$ bandwidth $\times$ round trip). Used by PCIe, CHI, UCIe, NoCs.

Q11. How much receive buffer do you need for credit flow control on a link with a 200 ns credit round trip at 16 GB/s?

Buffer $\geq$ bandwidth $\times$ round-trip time = $16\text{e}9 \times 200\text{e-}9 = 3200$ bytes. Anything less and the sender idles waiting for credits.

Q12. What errors does a CRC-n always detect?

All single-bit errors, all burst errors of length $\leq n$, all odd numbers of bit errors if the polynomial has $(x+1)$ as a factor, and all double-bit errors within the polynomial's period. Longer random errors escape with probability $\sim 2^{-n}$.

Q13. Why are CRC registers preset to all ones and the result inverted?

Preset: with init 0, leading zero bytes do not change the CRC, so missing/extra leading zeros go undetected. XorOut: makes appended zero bytes detectable and gives a constant non-zero residue at the receiver (for Ethernet CRC-32, running the CRC over frame + FCS always leaves the register at 0xDEBB20E3 before the final XOR).

Q14. What is the CRC check value and why do engineers quote it?

The CRC of ASCII "123456789". It pins down all parameters at once (poly, init, reflection, xorout): e.g. CRC-32 = 0xCBF43926, CRC-15/CAN = 0x059E. Appendix A shows a script reproducing 18 of them.

Q15. How do you compute a CRC over a 64-bit word per clock in RTL?

Unroll the bit-serial LFSR 64 times symbolically; each next-state bit becomes an XOR of some state bits and data bits (a GF(2) matrix multiply). Generate the equations with a script, verify against a bit-serial model, and pipeline the XOR trees if timing is tight.

Q16. Detection versus correction: when do links use retry and when FEC?

Retry (CRC + ACK/NAK replay, PCIe/USB 3/UCIe) is cheap when errors are rare, but costs round-trip latency on errors. FEC corrects without a return path at fixed latency cost; needed when raw BER is high (PAM4) or no retransmission exists (video, JESD204C).

APB and AHB

Q17. Draw an APB write with one wait state.

T1 SETUP: PSEL=1, PENABLE=0, PADDR/PWRITE/PWDATA valid. T2 ACCESS: PENABLE=1, PREADY=0 (wait). T3: PREADY=1 - transfer completes at the end of this cycle. Total 3 cycles; address/data held stable throughout.

Q18. What did APB3 and APB4 add?

APB3: PREADY (wait states) and PSLVERR (error). APB4: PPROT (protection) and PSTRB (byte write strobes). APB5 adds wake-up, user signals and optional parity protection.

Q19. How does AHB pipelining work, and what stalls it?

The address phase of transfer N+1 overlaps the data phase of N. HREADY low extends the current data phase and therefore also holds the next address phase; all masters/slaves see the same HREADY (HREADYOUT from the selected slave via the mux).

Q20. Why does an AHB ERROR response take two cycles?

HRESP=ERROR is signalled first with HREADY low, then with HREADY high. The first cycle gives the master time to cancel the already pipelined next address phase (drive HTRANS=IDLE).

Q21. What is the 1 KB boundary rule in AHB?

A burst must not cross a 1 KB address boundary, so a slave's decoding (minimum slave region 1 KB) never changes mid-burst.

Q22. Why do SoCs put an AHB/AXI-to-APB bridge in front of peripherals?

APB is small and low-power: no pipelining, no bursts, simple decode. The bridge absorbs the complex protocol once, and hundreds of register blocks stay trivial and can sit on a slower clock.

Q23. What are HTRANS encodings?

IDLE (00), BUSY (01, master inserts a gap inside a burst), NONSEQ (10, first/single transfer), SEQ (11, subsequent burst beat).

AXI

Q24. Name the five AXI channels and their direction.

AW write address (M->S), W write data (M->S), B write response (S->M), AR read address (M->S), R read data + response (S->M).

Q25. Can write data arrive before the write address?

Yes. AXI allows W before AW; a slave must handle it (typically by buffering or by holding WREADY low until it sees AW, which is legal because a destination may wait). The B response must not be given before both the last W beat and AW have been accepted.

Q26. What are the AXI4 burst length limits and the 4 KB rule?

INCR: 1-256 beats (AXI3: 16); WRAP: 2, 4, 8 or 16 beats; FIXED: up to 16. No burst may cross a 4 KB boundary, because that is the smallest page/slave region.

Q27. Compute the address of each beat of a 4-beat WRAP burst, 4-byte beats, start 0x38.

Wrap boundary = $4 \times 4 = 16$ bytes, lower boundary 0x30. Beats: 0x38, 0x3C, 0x30, 0x34.

Q28. What ordering does AXI guarantee?

Transactions with the same ID in the same direction complete in issue order; different IDs may complete out of order. There is no ordering between reads and writes; a master must wait for B before issuing a dependent read. AXI4 removed write interleaving (WID).

Q29. What is the difference between OKAY, EXOKAY, SLVERR and DECERR?

OKAY normal success (or failed exclusive), EXOKAY exclusive succeeded, SLVERR the slave was reached but reports an error, DECERR no slave at that address (usually from the interconnect's default slave).

Q30. What is the purpose of AxCACHE and AxPROT?

AxCACHE: bufferable, modifiable, read/write-allocate hints - tell interconnect and caches whether a transfer may be merged, split, buffered or cached. AxPROT: privileged, non-secure, instruction - used by firewalls/TrustZone filters.

Q31. How do you register-slice an AXI channel without losing throughput?

Use a skid buffer: register the payload forward and READY backward, with a second holding register to catch the beat that arrives in the cycle READY drops. It sustains 1 beat/clock and cuts both timing paths.

Q32. What does an AXI interconnect do with IDs?

It extends master IDs with port bits so responses route back, and tracks outstanding IDs per slave to maintain per-ID ordering (e.g. stalls a same-ID request to a different slave until earlier ones finish).

Q33. Why can a master deadlock if it waits for AWREADY before driving WVALID?

Because a slave is allowed to wait for WVALID before asserting AWREADY. Neither side moves. The spec forbids dependencies of VALID on READY exactly to avoid this.

Cache coherency

Q34. Explain the MESI states.

Modified: only copy, dirty. Exclusive: only copy, clean - can be written silently. Shared: possibly several clean copies. Invalid. MOESI adds Owned: dirty but shared, owner supplies data and is responsible for write-back.

Q35. Snooping versus directory coherence?

Snooping broadcasts requests to all caches - simple, low latency, but bandwidth grows with cores. Directory/snoop-filter keeps track of which caches hold each line and sends targeted snoops; scales to meshes (CHI home nodes).

Q36. What is ACE-Lite used for?

IO-coherent masters (DMA, GPU, accelerators) that have no cache (or do not need to be snooped) but whose reads must see dirty CPU data and whose writes must invalidate CPU copies.

Q37. What is false sharing and why is it a protocol issue?

Two cores write different variables in the same cache line; the line ping-pongs between Modified states with snoops and data transfers, though no data is truly shared. Fix by padding to the line size.

Q38. Name the CHI node types.

RN (request node: RN-F fully coherent, RN-I IO, RN-D with DVM), HN (home node: HN-F with snoop filter/SLC, HN-I for IO), SN (subordinate node, e.g. memory controller SN-F), and MN (miscellaneous node for DVM).

UART, SPI, I2C and I3C

Q39. How many bit times does a UART 8N1 byte take, and what is the throughput at 115200 baud?

10 bits (start + 8 + stop). $115200 / 10 = 11520$ bytes/s.

Q40. Why do UART receivers oversample 16x?

To find the start-bit falling edge within 1/16 bit and then sample each bit near its centre (often majority of 3 samples), leaving margin for clock mismatch and noise.

Q41. List the four SPI modes.

Mode 0: CPOL=0, CPHA=0 - idle low, sample on rising edge. Mode 1: CPOL=0, CPHA=1 - sample on falling. Mode 2: CPOL=1, CPHA=0 - idle high, sample on falling. Mode 3: CPOL=1, CPHA=1 - sample on rising. Modes 0 and 3 are the most common.

Q42. Why can SPI run much faster than I2C?

Push-pull drivers (no RC pull-up rise time), separate data directions, no ACK or arbitration and no address phase. I2C's open-drain rise time with pull-ups and bus capacitance limits it.

Q43. What happens during I2C arbitration?

Each controller drives SDA and reads it back; open drain makes the bus a wired-AND. A controller that drives 1 but reads 0 has lost and backs off. The winner's message is undamaged, so arbitration is non-destructive.

Q44. What are START and STOP conditions?

START: SDA falls while SCL is high. STOP: SDA rises while SCL is high. All data changes happen while SCL is low.

Q45. Name three things I3C improves over I2C.

Push-pull SDR at 12.5 MHz (and HDR modes), in-band interrupts without extra pins, dynamic address assignment, common command codes, hot-join, and lower power. I2C legacy targets can share the bus if they have a spike filter.

Q46. How does an I2C target signal "not ready"?

By clock stretching (holding SCL low) or by NACKing its address until ready. SMBus bounds stretching by timeouts (25-35 ms).

Q47. What is QSPI XIP?

Execute-in-place: the QSPI controller maps flash into the CPU address space and translates AHB/AXI reads into read commands, often with a prefetch cache, so code runs directly from flash.

Q48. How do you connect several SPI targets?

Shared SCLK/COPI/CIPO with one CS_n per target (CIPO tri-stated when deselected), or daisy chain where CIPO of one feeds COPI of the next and one long shift frame covers all.

CAN

Q49. How does CAN arbitration work?

Dominant (0) overwrites recessive (1) on the bus. Nodes send the identifier MSB first and monitor the bus; a node reading dominant while sending recessive loses and becomes a receiver. Lowest ID wins, with no lost bus time.

Q50. Why must the CAN bit time exceed about twice the propagation delay?

During arbitration every node must see the other nodes' bits within the same bit before the sample point: signal travels to the far node and its dominant bit travels back. At ~5 ns/m plus transceiver delays (~100-250 ns loop each), 1 Mb/s limits the bus to about 40 m.

Q51. How long is a classical CAN frame with 8 data bytes, worst case?

Standard (11-bit) data frame: 44 overhead bits + 64 data = 108 bits. Stuffing applies to the first 98 bits (SOF through CRC): worst case $(34+64-1)/4 = 24$ stuff bits. Plus 3 bits intermission -> 135 bit times, i.e. 135 us at 1 Mb/s.

Q52. Describe CAN fault confinement.

Each node keeps TEC/REC counters: error-active (< 128) sends active error flags, error-passive (>= 128) sends passive flags and waits extra, bus-off (TEC > 255) disconnects until 128 x 11 recessive bits are seen.

Q53. What changes in CAN FD?

Up to 64 data bytes, a faster data phase switched by the BRS bit, no remote frames, CRC-17/CRC-21 with a stuff-bit count and fixed stuff bits in the CRC field, and an ESI bit.

USB

Q54. What are the four USB transfer types?

Control (enumeration, endpoint 0), bulk (guaranteed delivery, no latency guarantee), interrupt (bounded latency polling), isochronous (reserved bandwidth, no retry).

Q55. What is the maximum USB 2.0 high-speed bulk throughput?

13 packets of 512 bytes per 125 us microframe: $6656 / 125e-6 = 53.2$ MB/s theoretical, typically 35-45 MB/s real.

Q56. How does USB 2.0 encode bits, and why bit stuffing?

NRZI: a 0 is a transition, a 1 is no transition. Long runs of 1s would give no edges, so a 0 is stuffed after six consecutive 1s.

Q57. What is the role of the UTMI/ULPI interface?

It separates the digital USB controller (link) from the analog PHY: UTMI is a parallel 8/16-bit on-chip interface at 60/30 MHz; ULPI is a reduced 12-pin version for an external PHY.

Q58. How does USB detect full-speed versus low-speed and high-speed?

A 1.5 k pull-up on D+ (FS) or D- (LS) at the device; HS capable devices then perform the chirp K/J handshake during reset to switch to HS terminations.

Q59. What is USB Power Delivery?

A protocol on the Type-C CC wire (BMC coded, 300 kb/s) negotiating voltage/current contracts up to 48 V / 5 A (240 W in EPR) and alternate modes.

PCI Express

Q60. What are the three PCIe layers?

Transaction layer (TLPs: memory, IO, config, message, completions; flow-control credits), data link layer (sequence numbers, LCRC, ACK/NAK replay, DLLPs), physical layer (encoding, scrambling, LTSSM link training, lane deskew).

Q61. Compute PCIe 4.0 x16 bandwidth per direction.

$16 \text{ GT/s} \times 128/130 \times 16 \text{ lanes} / 8 = 31.5 \text{ GB/s}$ per direction (~63 GB/s bidirectional) before TLP/DLLP overhead.

Q62. What is the efficiency of 256-byte TLPs on Gen3+?

Overhead per TLP roughly 4 framing + 2 sequence + 16 header (4DW) + 4 LCRC = 26 bytes: $256 / 282 = 91\%$. With MPS = 128 it drops to ~83%. ACK and flow-control DLLPs take a few percent more.

Q63. Posted versus non-posted requests?

Posted (memory writes, messages) need no completion - they consume only posted credits. Non-posted (memory/IO reads, config, IO writes) require a completion. Ordering rules let posted writes pass reads to avoid deadlock.

Q64. What is the LTSSM and what are its main states?

Link Training and Status State Machine: Detect -> Polling -> Configuration -> L0, with Recovery (speed change, equalisation, error recovery), L0s/L1/L2 power states, Loopback, Hot Reset, Disabled.

Q65. How do BARs work?

Software writes all ones to a BAR and reads back: the zeroed low bits give the size (e.g. 0xFFFF0000 -> 1 MB). It then writes the assigned base address; the endpoint claims TLPs in that window.

Q66. What changed in PCIe 6.0?

PAM4 signalling at 64 GT/s, FLIT mode (fixed 256-byte flits with light FEC and CRC, 1b/1b encoding), L0p low-power width scaling; per-direction x16 ~ 121 GB/s after flit overhead.

Q67. How does CXL relate to PCIe?

CXL runs on the PCIe PHY (negotiated via alternate protocol in training) and multiplexes CXL.io (PCIe semantics), CXL.cache (device caches host memory coherently) and CXL.mem (host accesses device-attached memory). Type 1/2/3 devices use different subsets.

Ethernet

Q68. What is the minimum Ethernet frame and why?

64 bytes (dest 6 + src 6 + type 2 + payload 46 + FCS 4); for half-duplex CSMA/CD the frame had to last longer than a collision round trip (512 bit times). Kept for compatibility.

Q69. What is the maximum packet rate of 10 GbE?

Minimum frame 64 + preamble/SFD 8 + IFG 12 = 84 bytes = 672 bits: $10\text{e}9 / 672 = 14.88 \text{ Mpps}$ (1.488 Mpps at 1 GbE). A 64-bit datapath at 156.25 MHz has ~10.5 cycles per packet.

Q70. MAC, PCS, PMA, PMD - who does what?

MAC: framing, FCS, pause, statistics. PCS: encoding (64b/66b), scrambling, lane alignment. PMA: serialisation, CDR. PMD: the medium-dependent driver/optics. The MII family (GMII, XGMII) sits between MAC and PCS.

Q71. RGMII versus SGMII?

RGMII: 4-bit DDR data + control at 125 MHz (12 pins), needs 2 ns clock-to-data skew (PHY or MAC internal delay or PCB). SGMII: one 1.25 GBd 8b/10b SerDes pair each way, carries 10/100/1000 by symbol replication.

Q72. What is IEEE 1588 PTP and what does hardware need?

Precision Time Protocol: timestamp sync/delay messages at the MAC /PHY boundary so software can compute offset and path delay; accuracy of tens of ns requires hardware timestamping and an adjustable hardware clock.

Q73. What is the Ethernet payload efficiency at 1500-byte payload?

$1500 / (1500 + 14 \text{ header} + 4 \text{ FCS} + 8 \text{ preamble} + 12 \text{ IFG}) = 1500/1538 = 97.5\%$ (before IP/TCP headers).

DDR and memory

Q74. Explain tRCD, CL and tRP.

tRCD: ACTIVATE to READ/WRITE (open row). CL (tAA): READ command to first data. tRP: PRECHARGE to next ACTIVATE on that bank. A page miss costs tRP + tRCD + CL.

Q75. Convert DDR4-3200 CL22 to nanoseconds.

Clock = 1600 MHz, tCK = 0.625 ns; $22 \times 0.625 = 13.75 \text{ ns}$. Absolute latency barely changes across generations; the clock count grows.

Q76. Why does DDR use a data strobe and read leveling/write leveling?

DQS is source-synchronous so data and strobe see the same flight time. With fly-by clock routing on DIMMs each DRAM sees CK at a different time; write leveling aligns DQS to CK per byte lane, read training finds the DQS/DQ eye centre.

Q77. Why did DDR5 split the DIMM into two subchannels?

Two independent 32-bit (40 with ECC) channels with BL16 give a 64-byte access per subchannel, doubling concurrency and improving efficiency; it also added on-die ECC and an on-DIMM PMIC.

Q78. What is the role of the DFI interface?

The DDR PHY Interface standardises the boundary between memory controller and PHY: command, write data, read data, update and training handshakes, with a frequency ratio (1:2, 1:4).

Q79. How does refresh affect bandwidth?

Every tREFI (~3.9 us at high temperature, 7.8 us normal for DDR4) a REF blocks the rank for tRFC (~350 ns for 8 Gb dies): $\sim 350/7800 = 4.5\%$ loss at normal temperature, twice that when hot. Per-bank/same-bank refresh in LPDDR/DDR5 reduces the impact.

Q80. HBM versus GDDR: when to use which?

HBM: very wide (1024-2048 bits) at modest pin rates, lowest pJ/bit and highest bandwidth per device, but needs 2.5D packaging and costs more. GDDR: narrow (x32) at very high pin rates on a standard PCB. AI accelerators favour HBM; consumer GPUs use GDDR.

MIPI, display and video

Q81. What are D-PHY LP and HS modes?

LP: 1.2 V single-ended, ~10 Mb/s, used for control, escape mode and turnaround. HS: ~200 mV differential, terminated, for bulk data. A lane enters HS via the LP-11 -> LP-01 -> LP-00 -> HS-0 sequence.

Q82. How many D-PHY lanes for 4K60 RAW10 from a camera?

$3840 \times 2160 \times 60 \times 10 = 4.98$ Gb/s of pixels; with ~20% blanking and packet overhead ~6 Gb/s. Four lanes at ~1.5-2 Gb/s each fits (D-PHY v1.2 allows 2.5 Gb/s per lane).

Q83. What is a CSI-2 long packet?

Header: Data ID (VC + data type), 16-bit word count, ECC (6-bit, v2.0+ virtual channel extension), then payload and a 16-bit CRC footer. Short packets carry frame/line start/end.

Q84. What is the TMDS clock for 1080p60 HDMI?

Pixel clock 148.5 MHz; each TMDS channel carries 10 bits per pixel clock: 1.485 Gb/s per lane for 8-bit colour.

Q85. DisplayPort versus HDMI architecture?

DP is packetised with a fixed set of link rates and an AUX channel, and the source clock is regenerated from M/N values; HDMI 1.x/2.0 carries a pixel-locked TMDS clock (HDMI 2.1 FRL moved to packetised fixed rates too).

Debug and test

Q86. What are the JTAG pins and the TAP controller?

TCK, TMS, TDI, TDO, optional TRST. The TAP is a 16-state FSM driven by TMS; five TCKs with TMS=1 always reach Test-Logic-Reset. IR selects which data register (BYPASS, IDCODE, boundary scan, user) sits between TDI and TDO.

Q87. What does SWD replace and how?

It replaces 4/5-pin JTAG with SWDIO (bidirectional) and SWCLK, using packet requests with parity and ACK to access DP/AP registers of an Arm DAP.

Q88. IEEE 1500 versus IEEE 1687?

1500 standardises a test wrapper around cores (WIR, WBR, serial and parallel ports) for core-based test. 1687 (IJTAG) standardises a reconfigurable network (SIBs) to reach embedded instruments, described in ICL/PDL.

Q89. How would you debug a link that trains but has CRC errors?

Check eye margins (on-die eye monitor, BER vs equaliser settings), reference clock quality and SSC settings, crosstalk and power noise, lane polarity/ordering and CDC between PCS and controller; reproduce with PRBS loopback to separate PHY from protocol.

Design puzzles

Q90. SPI controller: pad + board delay 2 ns each way, target clock-to-out 8 ns, controller input setup 2 ns. Maximum SCLK?

In mode 0 the target launches CIP0 on the falling edge and the controller samples on the next rising edge: half a period is available. Round trip = 2 (SCLK out) + 8 (tco) + 2 (back) + 2 (setup) = 14 ns $\leq T/2 \rightarrow T \geq 28$ ns $\rightarrow$ fmax = 35.7 MHz.

Fix: sample on a later edge (delayed/read-capture sampling): a full period is available, $T \geq 14 \text{ ns} \rightarrow 71.4 \text{ MHz}$. This is why QSPI controllers have programmable read-capture delay or use DQS.

Q91. UART 8N1: how much total baud-rate mismatch can be tolerated?

The receiver samples the stop bit at 9.5 bit times after the start edge; the accumulated error must stay under half a bit: $0.5 / 9.5 = 5.26\%$ total. Subtract the start-detection uncertainty of $1/16$ bit with 16x oversampling: $(0.5 - 0.0625) / 9.5 = 4.6\%$.

Split between both ends and leave margin for edge distortion: the usual design rule is $\pm 2\%$ per side (1-1.5% for long frames or noisy lines).

Q92. Choose an I2C pull-up for a 3.3 V Fast-mode bus with 200 pF.

Minimum (sink current): $R_{p_{\min}} = (V_{DD} - V_{OL}) / I_{OL} = (3.3 - 0.4) / 3 \text{ mA} = 967 \text{ ohm}$.

Maximum (rise time, 30% to 70% = 0.8473 RC): $R_{p_{\max}} = t_r / (0.8473 \times C_b) = 300 \text{ ns} / (0.8473 \times 200 \text{ pF}) = 1.77 \text{ k}$.

Pick $\sim 1.2\text{-}1.5 \text{ k}$. (Standard mode, 1000 ns, 400 pF: up to 2.95 k.)

Q93. An AXI master must sustain 16 GB/s on a 128-bit bus at 1 GHz with 200 ns read latency and 64-byte bursts. How many outstanding reads?

Little's law: bytes in flight = bandwidth x latency = $16 \text{ B/cycle} \times 200 \text{ cycles} = 3200 \text{ B}$. $3200 / 64 = 50$ outstanding transactions (so at least 6-bit ID/tag space or equivalent tracking, and 3.2 KB of read reorder buffering).

Q94. What is the peak and realistic bandwidth of a DDR5-6400 DIMM?

$6400 \text{ MT/s} \times 64 \text{ bits} / 8 = 51.2 \text{ GB/s}$ peak. With refresh, bank conflicts, read/write turnarounds and page misses, typically 70-85% efficiency: $\sim 36\text{-}43 \text{ GB/s}$.

Q95. Writer 100 MHz sends 120-word bursts back-to-back; reader drains one word per cycle at 80 MHz. Minimum FIFO depth?

Burst lasts $120 / 100 \text{ MHz} = 1.2 \text{ us}$. Reader removes $1.2 \text{ us} \times 80 \text{ MHz} = 96$ words meanwhile. Depth $\geq 120 - 96 = 24$ words, plus a few entries for synchronizer latency on the async pointers (round up to 32). Also verify the average rate: the idle gap between bursts must be $\geq 24 / 80 \text{ MHz} = 0.3 \text{ us}$.

Q96. How many lanes of PCIe 5.0 does a 400 Gb/s NIC need?

$400 \text{ Gb/s} = 50 \text{ GB/s}$ per direction. Gen5 lane: $32 \times 128/130 / 8 = 3.94 \text{ GB/s}$; with $\sim 88\%$ TLP efficiency $\sim 3.5 \text{ GB/s} \rightarrow 50 / 3.5 = 14.3$, so x16 (with little headroom).

Q97. I2C at 400 kHz: how many bytes per second can you read from a sensor register block?

Each byte takes 9 SCL cycles (8 + ACK): max 44.4 kB/s raw. A register read of N bytes costs START + addr(W) + reg + Sr + addr(R) + N data + STOP, so $\sim (3 + N) \times 9 + \sim 3$ clocks; for N = 6: ~ 84 clocks = 210 us , i.e. $\sim 28.6 \text{ kB/s}$ effective.

Q98. HBM3 stack bandwidth, and how many stacks for 3 TB/s?

$6.4 \text{ Gb/s} \times 1024 / 8 = 819.2 \text{ GB/s}$ per stack; $3000 / 819.2 = 3.7 \rightarrow 4$ stacks (or 3 stacks of HBM3E at $\sim 1.2 \text{ TB/s}$ each).

Q99. A camera outputs 1080p60 YUV422 8-bit. Minimum AXI write bandwidth into DDR, and the burst rate on a 64-bit, 250 MHz port?

$1920 \times 1080 \times 60 \times 2 \text{ bytes} = 249 \text{ MB/s}$. The port provides $8 \text{ B} \times 250 \text{ MHz} = 2 \text{ GB/s}$, so $\sim 12.4\%$ utilisation; with 16-beat (128 B) bursts that is $\sim 1.95 \text{ M bursts/s}$ - about one burst every 512 ns.

Q100. Design a 4-to-1 SPI-flash read path that hides 100 ns flash latency at 50 MB/s. What prefetch buffer?

Latency x bandwidth = $100\text{e-}9 \times 50\text{e}6 = 5 \text{ bytes}$ in flight; round up to the command granularity: a 2-line (e.g. 2 x 32-byte) prefetch buffer so the next line is requested while the current one drains. Also check command/address overhead: a 1-4-4 read with 8 dummy clocks costs ~ 16 clocks before data.

Appendix D. Specifications, Standards Bodies and Resources

Where the authoritative documents live, what open-source RTL to study, and which tools to install. Access models vary: some specifications are free after registration (Arm AMBA, NXP I2C, several MIPI and JEDEC documents), others need membership or purchase (PCI-SIG, IEEE, ISO, full MIPI PHY specs). Items marked (*) are vendor, product or access claims that may change - verify on the owner's website.

Standards bodies and key documents

Body	Protocols	Key documents (as commonly referenced)
Arm (AMBA)	APB, AHB, AXI, ACE, CHI, ATB, AXI-Stream, DTI	AMBA APB Protocol Spec (IHI 0024), AMBA AHB/AHB5 (IHI 0033), AMBA AXI and ACE / AXI Protocol Spec (IHI 0022), AMBA CHI Architecture Spec (IHI 0050), AXI4-Stream (IHI 0051); CoreSight Architecture Spec, ADIV5/ADIV6 (IHI 0031 / IHI 0074). Free download (*)
USB-IF	USB 2.0, 3.2, USB4, Type-C, PD, device classes	USB 2.0 Spec, USB 3.2 Spec, USB4 Spec (v1, v2), USB Type-C Cable and Connector Spec, USB Power Delivery Spec; UTMI+ and PIPE (Intel-published) for PHY interfaces
PCI-SIG	PCI Express	PCI Express Base Specification (Rev. 1.0-7.0), CEM (card electromechanical) spec, PIPE; member access (*)
CXL Consortium	Compute Express Link	CXL Specification 1.1, 2.0, 3.x
IEEE	Ethernet, test, time	802.3 (Ethernet; clauses per PHY type, e.g. 22 MII/MDIO, 45 MDIO, 49 10GBASE-R, 91 RS-FEC), 802.1Q/TSN, 1149.1 (JTAG), 1149.6 (AC-coupled boundary scan), 1500 (core test wrapper), 1687 (IJTAG), 1588 (PTP), 1800 (SystemVerilog), 1800.2 (UVM)
JEDEC	DRAM, flash, converters	JESD79-4 (DDR4), JESD79-5 (DDR5), JESD209-4/-5 (LPDDR4/5, 5X), JESD250 (GDDR6), JESD239 (GDDR7), JESD235 (HBM/HBM2), JESD238 (HBM3), JESD270-4 (HBM4), JESD84 (eMMC), JESD220 (UFS), JESD251 (xSPI), JESD216 (SFDP), JESD204B/C (serial converter interface). Many free after registration (*)
MIPI Alliance	Camera, display, sensors	D-PHY, C-PHY, M-PHY, A-PHY, CSI-2, DSI/DSI-2, I3C and I3C Basic (I3C Basic is publicly downloadable (*)), SoundWire, RFEE
VESA	DisplayPort, eDP, DSC	DisplayPort Standard 1.4 / 2.1, eDP, Display Stream Compression (DSC), DisplayID
HDMI Forum / HDMI LA	HDMI	HDMI Specification 1.4b, 2.0, 2.1, 2.2 (adopter access (*))
ISO	Automotive	ISO 11898-1 (CAN data link, incl. CAN FD), ISO 11898-2 (high-speed CAN PHY), ISO 17987 (LIN), ISO 17458 (FlexRay), ISO 26262 (functional safety); CiA (CAN in Automation) publishes CAN XL (CiA 610) and application layers
NXP	I2C	UM10204 I2C-bus specification and user manual (free)
SMBus / PMBus (SBS-IF, SMIF)	SMBus, PMBus	System Management Bus Specification 3.x, PMBus Parts I-III
UCIe Consortium	Die-to-die	UCIe Specification 1.0, 1.1, 2.0, 3.0
Open Compute Project	Die-to-die	Bunch of Wires (BoW) specification
RISC-V International / CHIPS Alliance	TileLink, debug	TileLink Specification (SiFive, maintained in the CHIPS Alliance), RISC-V External Debug Support spec
Accellera	OCP, UVM, IP-XACT	Open Core Protocol specification, UVM library, IEEE 1685 IP-XACT
OpenCores	Wishbone	Wishbone B4 specification (free)

Body	Protocols	Key documents (as commonly referenced)
ONFI / SD Association	NAND, SD	ONFI 4.x/5.x, SD Physical Layer Simplified Specification (free)
TIA / EIA	Serial lines	TIA-232-F (RS-232), TIA-485-A (RS-485), TIA-644 (LVDS)

READING A BIG SPEC EFFICIENTLY

Start with the overview/architecture chapter and the list of state machines, then read the signal or packet definitions, then the ordering and error rules. Keep the errata and ECN list next to the base spec: many interview and integration bugs live in ECNs.

Open-source IP to study

Project	What to learn from it
Alex Forencich: verilog-axi, verilog-axis, verilog-ethernet, verilog-pcie, verilog-i2c, verilog-uart; Corundum NIC	Production-quality AXI crossbars, register slices, DMA, 10/25G Ethernet MAC/PCS, PCIe DMA; paired cocotb/axi / -eth / -pcie Python models for verification. Some repos have been reorganised or archived over time (*)
lowRISC OpenTitan	Security-grade UART, SPI host/device, I2C, USB device, TileLink-UL crossbar, with DV (UVM) environments and documentation of every block
PULP Platform axi (ETH Zurich)	Parameterised AXI4+ATOPs crossbars, bursts splitters, ID remappers, CDC, with randomised verification
LiteX: LiteDRAM, LitePCle, LiteEth, LiteSPI, LiteSATA	Complete Python-generated SoC ecosystem (Migen/Amaranth-style HDL): DRAM controllers and PHYs, PCIe DMA, Ethernet MAC
OpenCores I2C master, SPI, UART16550	Classic small cores (Wishbone) - good to read, but check for known bugs before reusing
Chippyard / Rocket Chip (Berkeley)	TileLink (Diplomacy) interconnect generators, AXI/TileLink bridges, SiFive blocks
ZipCPU: wb2axip and blog	Formally verified AXI/AXI-Lite/Wishbone slaves, bridges and skid buffers; articles on common AXI bugs found by formal
Other	SpinalHDL/VexRiscv bus libraries, Verilog USB (e.g. usb_cdc cores), Enjoy-Digital/LiteX boards, open-source MIPI CSI-2 receivers - quality varies (*)

Tools

Category	Examples	Notes
Simulation / formal	Icarus Verilog, Verilator, cocotb, SymbiYosys; commercial Xcelium, VCS, Questa, JasperGold, VC Formal	Use protocol assertions (Arm-published AXI SVA, ZipCPU formal properties) early
Verification IP	Commercial VIP from Cadence, Synopsys, Siemens, Avery, SmartDV (*); open cocotb/* models	Compliance-level VIP for PCIe/USB/DDR is almost always bought
Logic analyzers / decoders	sigrok + PulseView (open-source, 100+ decoders: I2C, SPI, UART, CAN, 1-Wire, JTAG, USB LS/FS), Saleae Logic (*)	Cheap FX2-based analyzers work with sigrok for low-speed buses
Protocol analyzers	Total Phase Beagle/Aardvark (I2C/SPI/USB), Teledyne LeCroy and Ellisys (USB, PCIe), Keysight (PCIe, DDR), Vector CANalyzer/CANoe and PEAK PCAN (CAN) (*)	Necessary for high-speed protocol bring-up; often rented
Oscilloscopes / BERT	Real-time scopes with serial decode and compliance apps, BERTs for jitter tolerance (Keysight, Tektronix, R&S, Anritsu) (*)	PHY/SI team owns eye, jitter and compliance measurements
Packet capture	Wireshark (Ethernet, USB via usbmon/USBPcap, CAN via SocketCAN), tcpdump	Great for decoding real traffic against your mental model

Category	Examples	Notes
Debug	OpenOCD, pyOCD, Segger J-Link, Lauterbach TRACE32 (*)	JTAG/SWD access; CoreSight trace capture
CRC / coding helpers	Greg Cook's CRC RevEng catalogue, Python crcmod/crccheck, OutputLogic-style parallel CRC generators (*)	Verify every CRC against a catalogue check value and a real frame

Books and courses

Title / resource	Covers
Budruk, Anderson, Shanley / Jackson, Budruk: <i>PCI Express System Architecture and PCI Express Technology 3.0</i> (MindShare)	PCIe layers, TLPs, LTSSM in great depth
Jan Axelson: <i>USB Complete, Serial Port Complete</i>	USB from the device/firmware side; UART/RS-232/RS-485
Howard Johnson, Martin Graham: <i>High-Speed Digital Design: A Handbook of Black Magic</i>	Transmission lines, termination, crosstalk
Eric Bogatin: <i>Signal and Power Integrity - Simplified</i>	Channel loss, reflections, SI intuition for SerDes
Bruce Jacob, Spencer Ng, David Wang: <i>Memory Systems: Cache, DRAM, Disk</i>	DRAM timing, controllers, scheduling
Nagarajan, Sorin, Hill, Wood: <i>A Primer on Memory Consistency and Cache Coherence</i> (2nd ed.)	MESI/MOESI, directories, consistency models - free PDF (*)
Charles Spurgeon, Joann Zimmerman: <i>Ethernet: The Definitive Guide</i>	Ethernet media, frames, auto-negotiation
Kenneth Parker: <i>The Boundary-Scan Handbook</i>	IEEE 1149.x, board test
Nicola Da Dalt, Ali Sheikholeslami: <i>Understanding Jitter and Phase Noise</i>	Jitter decomposition, CDR, PLL noise
Behzad Razavi: <i>Design of Integrated Circuits for Optical Communications</i>	CDR, SerDes front ends
Arm Developer documentation and AMBA training; Verification Academy (Siemens); ChipVerify; ZipCPU blog (*)	Protocol tutorials, UVM and formal verification of buses
Vendor application notes (TI, Analog Devices, NXP, Microchip, Intel/AMD FPGA IP user guides) (*)	Practical I2C pull-up, CAN bit timing, JESD204 and SerDes configuration guides

A SUGGESTED READING ORDER

1) The free specs first: NXP UM10204, AMBA APB/AXI, Wishbone B4, SD simplified spec. 2) Study one open-source controller per spec (verilog-i2c, verilog-axi, OpenTitan SPI) and simulate it. 3) Move to SerDes-based protocols with the MindShare PCIe book and verilog-ethernet. 4) Finish with coherency (Primer, CHI spec) and memory (JEDEC DDR5 plus the Jacob book).